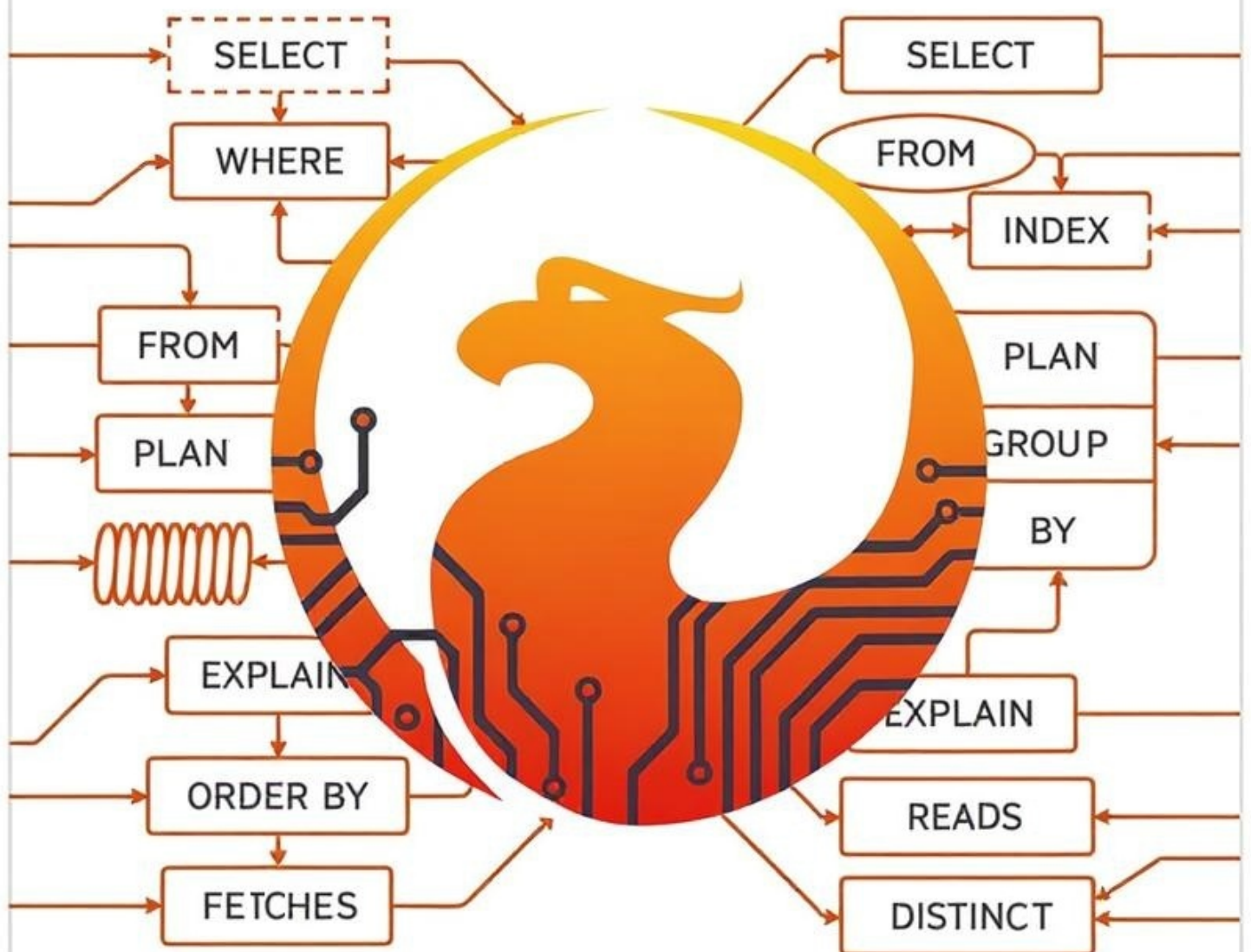


Segredos do Otimizador SQL Firebird



D.Simonov
2025

Segredos do Otimizador do Firebird

Simonov Denis

Versão 1.0 by 2026-01-04

Sumário

Introdução: O Caminho para Entender o Otimizador do Firebird	2
Por que o desempenho é importante	2
A evolução das abordagens de otimização: do hardware ao entendimento	2
A armadilha do aumento infinito de recursos	3
Erros típicos: quando um bom código se torna lento	4
Sobre este livro: um roteiro por capítulos	6
Como ler este livro: recomendações práticas	8
O que você obterá após a leitura deste livro	10
1. Como medir o desempenho de consultas?	12
1.1. Ativando a saída de estatísticas de execução no isql	12
1.2. O que esses números significam?	13
1.3. Como obter o tempo de extração de todas as linhas	15
1.4. Transformação de Consultas	16
1.5. Medição do Desempenho de Consultas Sob Carga	18
1.6. Usando Rastreamento para Medir o Desempenho de Consultas	20
1.7. Conclusões	23
2. O que é um plano de execução de consulta?	24
2.1. Plano Legacy	24
2.2. Plano Explain	25
2.3. Como obter um plano de execução de consulta?	26
2.3.1. Obtendo o plano de execução da consulta na Firebird API	26
Obtendo o plano de execução da consulta no isql	27
2.3.2. Obtendo o plano de execução da consulta no rastreamento	28
2.4. Obtendo o plano de execução de consulta nas tabelas MON\$	32
2.4.1. Tabela MON\$STATEMENTS	32
2.4.2. Tabela MON\$COMPILED_STATEMENTS	33
2.5. Um Olhar para o Futuro	36
2.5.1. Obtendo o Plano com RDB\$SQL.EXPLAIN	36
2.6. Conclusão	38
3. Métodos de Acesso Utilizados no Firebird	39
3.1. Terminologia	39
3.2. Métodos de Acesso Primários	40
3.2.1. Leitura de Tabela	40
Varredura Completa (Full table scan)	40
Acesso via identificador de registro	42
Acesso posicionado	42
3.2.2. Acesso por índice	42
Seletividade do índice	44

Índices parciais	46
Mapas de bits	47
Varredura de intervalo	48
Interseção e união de mapas de bits	56
Navegação por índice	57
3.2.3. Acesso via RDB\$DB_KEY	60
3.2.4. Tabela externa (External table scan)	61
3.2.5. Tabela virtual (Virtual table scan)	61
3.2.6. Tabela temporária local (Local table)	62
3.2.7. Acesso procedural	63
3.3. Filtros	64
3.3.1. Verificação de predicados	64
Verificação de predicados invariantes	66
3.3.2. Ordenação	67
Refetch	69
3.3.3. Agregação	71
Filtragem na cláusula HAVING	74
3.3.4. Contadores	75
3.3.5. Verificação de singularidade	77
3.3.6. Bloqueio de registro	78
3.3.7. Ramificação Condicional de Fluxos (Conditional Stream)	79
3.3.8. Buffer de Registros	80
3.3.9. Janela Deslizante (Window)	80
3.4. Métodos de Mesclagem	85
3.4.1. Junções (Joins)	85
Junção por Laços Aninhados (Nested Loop)	86
Junção por Hash (Hash join)	92
Fusão de Passagem Única (Merge)	97
Full Outer Join	99
Junção com procedimento armazenado	100
Junção com expressões de tabela	103
Conexão com visões	110
3.4.2. Uniões (Union)	110
Materialização de expressões não determinísticas	111
Materialização de subconsultas	113
3.4.3. Recursão	113
3.5. Estratégias de Otimização	117
3.6. Conclusão	119
4. Transformação de Consultas	120
4.1. Transformação de Predicados de Filtragem	120
4.1.1. Transformação do Predicado LIKE	120

4.1.2. Transformação do Predicado SIMILAR TO	124
4.1.3. Inversão de predicados	125
4.1.4. Transformação do predicado IN com lista de valores	127
4.2. Transformação de subconsultas	128
4.2.1. Transformação de IN em SOME/ANY	128
4.2.2. Transformação de subconsultas ANY/ALL em forma correlacionada	129
4.2.3. Transformação de NOT IN, NOT ANY, ALL com o operador <>	130
4.2.4. Transformação em semi-join	132
4.2.5. Transformação em anti-join	133
4.3. Transformação de junções (JOINS)	134
4.3.1. Transformação de RIGHT JOIN em LEFT JOIN	134
4.3.2. Transformação de OUTER JOIN em INNER JOIN	135
4.3.3. Transformação de OUTER JOIN em anti-join	141
4.4. Conclusão	142
5. Estatísticas por tabela	143
5.1. Obtendo estatísticas por tabela no isql	143
5.2. Obtenção de estatísticas por tabela usando trace	147
5.3. Obtenção de estatísticas por tabela usando tabelas de monitoramento	148
5.3.1. MON\$TABLE_STATS	149
5.3.2. MON\$RECORD_STATS	150
5.3.3. Exemplos de obtenção de estatísticas por tabela usando tabelas MON\$	151
5.4. Conclusão	153
6. Obtendo estatísticas com gstat	154
6.1. Descrição da utilidade gstat	154
6.2. Obtendo estatísticas com fbsvcmgr	157
6.2.1. Sintaxe dos parâmetros do fbsvcmgr	158
Sintaxe SPB	158
6.2.2. Obtendo ajuda	159
6.2.3. Parâmetros de conexão do serviço	160
6.2.4. Usando serviços com um banco de dados de segurança não padrão	160
6.2.5. Parâmetros para obter estatísticas com fbsvcmgr	161
6.2.6. Exemplo de análise de estatísticas para tabelas e índices de uma consulta	161
6.3. Análise das estatísticas obtidas	162
6.3.1. Estatísticas da página de cabeçalho	162
6.3.2. Estatísticas da tabela	168
Páginas primárias e secundárias	172
Versões e fragmentos de registros	174
6.3.3. Estatísticas de índices da tabela	182
Profundidade do índice	185
Número de páginas folha	185
Número de nós e duplicatas de chaves	185

Índices parciais	186
Tamanho da chave e taxa de compressão	187
Fator de clusterização	188
7. Índices	190
7.1. Tipos de índices	190
7.2. Quando o otimizador pode usar um índice?	190
7.3. Uso de índices para filtragem	191
7.3.1. Operador BETWEEN	192
7.3.2. Predicado IN	192
7.3.3. Índice para colunas ou expressões booleanas	193
7.3.4. Índice por expressão	193
7.3.5. Desabilitando o uso de índices	197
7.3.6. Estatísticas de índices	198
Seletividade do índice	199
Seletividade dos segmentos de um índice composto	199
Seletividade de predicados indexados	200
7.3.7. Índices compostos	200
Exemplo de ordem incorreta de campos em um índice composto	203
Tamanho do índice	205
7.3.8. Uso de múltiplos índices	206
7.3.9. Ramificação condicional de fluxos	210
7.3.10. Índice composto ou vários índices simples?	213
A filtragem por índice composto é mais barata	214
Seletividade do índice composto é mais precisa	219
Índices compostos têm custo de manutenção mais alto	219
Índices compostos não podem ser usados para unir predicados com OR	219
É impossível definir uma expressão para uma das colunas que compõem um índice composto	220
Quando é necessário criar um índice composto?	221
7.3.11. Índices parciais	221
Índices parciais únicos	222
Quando índices parciais podem ser usados pelo otimizador?	223
Seletividade do índice parcial	223
Redução do tamanho do índice	224
Uso de índices parciais com predicados não seletivos	227
Índices parciais com condição que não inclui a coluna chave	230
Índices parciais com condições unidas por OR	235
Índices parciais com predicado IN na condição de filtro	236
Quando índices parciais não podem ser usados	238
7.4. Uso de índices em condições de junção de tabelas	239
7.4.1. Como os índices são usados em condições de junção?	240

7.4.2. Uso de índices pelo algoritmo de junção Nested loop	241
Uso de Múltiplos Índices	244
Uso de Índices Compostos	247
7.4.3. Uso de estatísticas de índice pelo algoritmo de junção Hash	251
7.5. Usando um índice para ordenação	256
7.5.1. Quando um índice pode ser usado para ordenação?	256
7.5.2. Navegação (ordenação) por um índice simples	256
7.5.3. Navegação (ordenação) usando um índice por expressão	258
7.5.4. Desativando a navegação por índice	263
7.5.5. Navegação (ordenação) por índice composto	264
7.5.6. Filtragem durante navegação (ordenação) por índice	268
Predicado de filtragem não indexado	268
Filtragem por outro índice	272
Filtragem pelo mesmo índice que é usado para navegação	274
Filtragem por segmento do mesmo índice composto que é usado para navegação	276
7.5.7. Limitação do número de linhas	279
Pular N registros	281
Contadores e Filtros	283
7.5.8. Navegação (ordenação) por índice em consultas com múltiplas tabelas	288
Limitação no número de registros	291
7.5.9. Custo de navegação (ordenação) por índice	294
7.6. Uso de índice para agrupamento	298
7.6.1. Filtragem no agrupamento	301
Predicado de filtragem não indexado	302
Filtragem por outro índice	304
Filtragem pelo mesmo índice usado para agrupamento	304
Filtragem por segmento do mesmo índice composto usado para agrupamento	305
7.6.2. Agrupamento e limitadores FIRST/ROWS/OFFSET	306
7.6.3. Agrupamento e ordenação	307
7.6.4. Agrupamento em consultas com múltiplas tabelas	310
7.7. Usando um índice para calcular funções agregadas MIN/MAX	314
7.7.1. MIN/MAX e NULL	317
7.7.2. Desativando o índice ao calcular MIN/MAX	319
7.7.3. Cálculo de MIN/MAX e filtragem	319
7.7.4. Predicado de filtragem não indexado	319
7.7.5. Predicado de filtro indexado por outro índice	324
7.7.6. Predicado de filtro indexado pelo mesmo índice	325
7.7.7. Predicado de filtro indexado por um segmento do mesmo índice	326
7.7.8. Cálculo de MIN/MAX com agrupamento	328
7.7.9. Cálculo de MIN/MAX em consultas com junções de tabelas	330
7.8. Conclusão	332

8. Otimização de junções (JOIN)	334
8.1. Junção de duas tabelas	334
8.1.1. INNER JOIN de duas tabelas	334
Índices para campos e expressões	336
Recalcular estatísticas dos índices	336
Dicas +0 ou ' ' para o otimizador	336
Usando LEFT JOIN como uma dica	339
Usando um plano explicitamente especificado	340
Condições adicionais de filtragem	342
8.1.2. LEFT/RIGHT JOIN de duas tabelas	347
Condições Adicionais de Filtragem	348
E se for necessário evitar a transformação de junções externas?	351
8.1.3. FULL JOIN de duas tabelas	352
Condições adicionais de filtragem	354
8.2. Junção de tabela com procedimento armazenado	357
8.2.1. Junção interna de procedimento armazenado e tabela (sem dependências através de parâmetros de entrada)	357
8.2.2. Junções externas unidirecionais de procedimento armazenado e tabela	362
8.2.3. Junção com procedimento dependente de fluxo de dados via parâmetros de entrada	363
8.3. Junção de tabela com expressão de tabela	365
8.3.1. Junções com expressões de tabela simples	366
8.3.2. Junções internas com expressões de tabela complexas	367
8.3.3. Junções externas unidirecionais com expressões de tabela complexas	373
8.3.4. Junções com tabelas derivadas LATERAL	377
Substituição de subconsultas semelhantes	381
Eliminação de duplicação de parte de consultas em UNION ALL	383
Usando LATERAL para definir a ordem de junção com tabelas derivadas	391
8.4. Junções de três ou mais tabelas	393
8.4.1. Definindo a ordem e/ou algoritmo para junções internas de três tabelas	393
8.4.2. Junções internas e externas em uma única consulta	397
8.5. Conclusão	402
9. Otimização de ordenações	403
9.1. Estimativa do custo da ordenação externa	403
9.2. Otimização de ordenações externas amplas (Refetch)	408
9.2.1. Quais valores de configuração escolher?	413
9.3. Transformação da consulta para reduzir a largura da ordenação	413
9.4. Conclusões	418
10. Otimização de agrupamentos	419
10.1. Eficiência do agrupamento usando ordenação externa	419
10.2. Uso de funções agregadas adicionais para reduzir a largura da ordenação na agregação	421
10.3. Conexão com outras tabelas após agrupamento	423

10.4. GROUP BY ou DISTINCT	429
10.5. Conclusões	432
11. Conclusão	433
11.1. Feedback	433

Este material foi criado com o apoio e patrocínio da empresa [iBase.ru](http://ibase.ru), que desenvolve ferramentas Firebird SQL para empresas e fornece serviço de suporte técnico para Firebird SQL.

Introdução: O Caminho para Entender o Otimizador do Firebird

Por que o desempenho é importante

No mundo moderno dos sistemas de informação, o desempenho do banco de dados não é apenas uma métrica técnica, mas um fator crítico para o sucesso dos negócios. Uma resposta lenta do sistema durante a entrada de dados irrita os usuários, reduz sua produtividade e pode levar à perda de clientes. Relatórios analíticos que levam horas em vez de minutos retardam a tomada de decisões gerenciais. Em uma era onde cada segundo conta, o desempenho das consultas SQL se torna um verdadeiro campo de batalha por vantagem competitiva.

Imagine uma situação típica: você desenvolveu um ótimo aplicativo com uma interface bem pensada, mas ao tentar abrir um catálogo, o usuário espera 5-10 segundos. Ou um relatório de vendas do mês é gerado por horas. Cenário familiar? Infelizmente, tais problemas são comuns, e muitas vezes a causa não está na falta de recursos do servidor, mas na falta de compreensão de como o banco de dados processa as consultas.

É especialmente desagradável quando os problemas de desempenho se manifestam com o crescimento do tamanho do banco de dados: os desenvolvedores usam um banco de dados pequeno para criar consultas SQL, com esse volume o Firebird retorna os dados muito rapidamente mesmo com um plano de consulta não ideal, mas após alguns anos o trabalho com o banco de dados se torna dolorosamente lento e desacelera cada vez mais, e em algum momento os usuários abandonam o sistema completamente.

Firebird — um poderoso, confiável e comprovado SGBD que funciona com sucesso em milhões de servidores em todo o mundo. Ele pode processar milhões de registros de forma eficiente e rápida, mas sob uma condição: se o desenvolvedor entender como funciona o mecanismo de execução de consultas. É por isso que este livro foi escrito.

A evolução das abordagens de otimização: do hardware ao entendimento

A história da otimização de bancos de dados está cheia de equívocos e mitos. No alvorecer dos sistemas de informação, quando o desempenho deixava a desejar, a solução parecia óbvia: adicionar mais memória RAM, instalar um processador mais rápido, migrar para discos SSD. Essa abordagem pode ser chamada de "otimização por hardware", e ela realmente funcionou... até certo ponto, até esbarrar na lei dos rendimentos decrescentes.

Nos anos 2000, quando o custo do hardware começou a cair rapidamente, muitas organizações seguiram o caminho do aumento de capacidade. Aumentar o volume de RAM de 4 para 32 gigabytes dava um ganho de desempenho perceptível. A migração para arrays RAID acelerava as operações de disco. Mas mais cedo ou mais tarde chegava o momento da verdade: dobrar a memória novamente não trazia o resultado esperado. O sistema com 128 GB de memória RAM continuava executando algumas consultas de forma dolorosamente lenta, e os administradores, olhando para os monitores do sistema com gigabytes de memória não utilizada, ficavam perplexos. O hardware

ficava ocioso, mas as consultas arrastavam-se.

O próximo estágio é a otimização da configuração. Desenvolvedores e administradores estudam os parâmetros do sistema operacional e as configurações do Firebird: tamanho do cache de páginas, parâmetros de ordenação, configurações de buffers de rede, usam a calculadora de configurações do Firebird. Este é um passo na direção certa, mas aqui também há decepções. Aumentar o tamanho do cache de 256 para 131072 páginas pode tanto acelerar o trabalho quanto, ao contrário, desacelerá-lo (por exemplo, na arquitetura Classic). O ajuste fino dos parâmetros requer a compreensão de como esses parâmetros afetam o processamento de consultas específicas.

Mas a comunidade profissional sempre soube: a verdadeira otimização requer não apenas hardware poderoso e arquivos de configuração otimizados. É preciso saber **como o otimizador toma decisões sobre a execução de consultas**.

Claro, não negamos a importância dos recursos de hardware e da configuração correta. Tentar executar um sistema de produção sério em uma máquina virtual com 4 GB de memória RAM e um disco HDD lento está fadado ao fracasso. Ajustar os parâmetros do Firebird para uma tarefa específica — por exemplo, aumentar o TempCacheLimit para sistemas com muitas ordenações — pode dar um ganho substancial. Mas tudo isso são condições necessárias, mas não suficientes para um alto desempenho.

Resultados revolucionários só podem ser alcançados de uma maneira: entendendo como funciona o otimizador de consultas e sendo capaz de influenciar suas decisões.

A armadilha do aumento infinito de recursos

Vamos considerar um exemplo típico, ilustrando a limitação da abordagem "mais hardware — maior desempenho". Uma empresa enfrentou o problema da geração lenta de um relatório complexo de vendas. A primeira solução parecia óbvia: aumentar a memória RAM do servidor de 16 para 64 GB. Investiram dinheiro, aguardaram a reinicialização, executaram o relatório... e viram 8 minutos em vez dos anteriores 10. Há progresso, mas também decepção.

O próximo passo foi a compra de caros discos SSD NVMe da última geração. Mais um minuto ganho — agora 7 minutos. Depois migraram para um processador mais potente com o dobro de núcleos — mais meio minuto economizado. O orçamento para hardware está esgotado, no total foram gastos milhares de dólares, e o relatório ainda é gerado em 6-7 minutos, o que é inaceitável para os processos de negócios da empresa.

Agora imagine outro cenário: um desenvolvedor que entende o funcionamento do otimizador gastou 15 minutos analisando o plano de execução da consulta e descobriu que, ao unir três tabelas, o otimizador escolheu uma ordem de JOIN catastróficamente não ideal devido à falta de estatísticas atualizadas. Recalcular as estatísticas com o comando SET STATISTICS, uma pequena reestruturação da consulta usando CTE — e o relatório é gerado em 4 segundos. No hardware antigo. Sem um único centavo de investimento adicional. Um ganho de desempenho de 100 vezes.

Esta não é uma história inventada. Tais situações acontecem constantemente. Consultas que levam minutos para executar, após a otimização correta, começam a funcionar em frações de segundo. Ganhos de desempenho de 10, 50, 100 e até mais vezes — esta é a realidade disponível para aqueles que entendem os mecanismos do otimizador.

Erros típicos: quando um bom código se torna lento

Por que então os problemas de desempenho surgem com tanta frequência? Afinal, a maioria dos desenvolvedores conhece SQL, sabe escrever consultas e até usa índices. O problema é que conhecer a sintaxe e entender os mecanismos de execução são coisas completamente diferentes. É como conhecer as regras de trânsito, mas não entender como funciona o motor de um carro: o carro vai andar, mas no primeiro problema sério você ficará impotente.

Os erros mais comuns, que ocorrem até mesmo em desenvolvedores experientes:

Indexação incorreta — criação de índices redundantes ou, ao contrário, falta de índices necessários onde são críticos. Exemplo clássico: o desenvolvedor criou um índice em um campo usado em WHERE, antecipa a aceleração com alegria, executa a consulta... e vê no plano um Full Table Scan. O otimizador ignora o índice devido à baixa seletividade ou devido a uma função na condição de filtro. Ou a situação inversa: criaram dez índices em uma tabela "por precaução", o que desacelera cada operação de inserção e atualização, e nenhum desses índices traz benefício real.

Ordem de JOIN incorreta — possivelmente o problema mais traiçoeiro e difícil de detectar. O desenvolvedor escreve uma consulta, unindo tabelas em uma ordem lógica e intuitivamente compreensível: primeiro pedidos, depois clientes, depois produtos. A consulta funciona... lentamente. E a questão é que o otimizador pode escolher uma ordem de execução das junções completamente diferente, levando à varredura múltipla de tabelas grandes. Por exemplo, começar com a tabela de produtos, contendo um milhão de registros, em vez de primeiro filtrar os pedidos necessários. Entender como o otimizador determina a ordem das junções e como isso pode ser influenciado pode literalmente transformar uma consulta lenta em rápida com uma linha de código.

Ignorar as estatísticas — o otimizador toma decisões com base em dados estatísticos sobre a distribuição de valores nas tabelas. Estatísticas desatualizadas ou ausentes levam a planos de execução não ideais. Muitos desenvolvedores nem sabem da existência de comandos para coleta e atualização de estatísticas.

Não compreender os métodos de acesso — usar a varredura completa da tabela onde o acesso por índice é possível, ou vice-versa, tentar aplicar um índice a uma tabela com três registros. Cada método de acesso tem sua área de aplicação, e a escolha do método errado pode anular todos os esforços de otimização.

Ordenações e agrupamentos ineficientes — as operações ORDER BY e GROUP BY podem ser extremamente custosas se o otimizador for forçado a ordenar grandes volumes de dados. No entanto, muitas vezes existem maneiras de evitar a ordenação ou reduzir significativamente o volume de dados ordenados.

Todos esses erros têm uma coisa em comum: eles surgem não da falta de conhecimento de SQL como linguagem, mas da falta de compreensão do que acontece "debaixo do capô" durante a execução de uma consulta. == Evolução do Firebird e do Otimizador: do Firebird 1 ao 5

Para realmente entender as capacidades do Firebird moderno, é útil saber como o otimizador evoluiu ao longo das últimas duas décadas. Isso não é apenas um registro histórico — muitas características do comportamento do otimizador podem ser explicadas conhecendo o seu caminho

evolutivo.

Firebird 1.0-1.5: O Legado do InterBase

No início dos anos 2000, o Firebird acabara de se separar do InterBase comercial e começou sua jornada como um projeto de código aberto. O otimizador daquela época era relativamente simples, mas confiável. Ele sabia usar índices, executar junções básicas via Nested Loop Join e aplicar regras simples de otimização. Muitos conceitos estabelecidos naqueles anos continuam funcionando hoje, garantindo continuidade e previsibilidade de comportamento.

Firebird 2.0-2.5: Expansão de Capacidades

Esta era trouxe melhorias significativas. Surgiram as tabelas derivadas, permitindo escrever consultas mais complexas e expressivas. O otimizador aprendeu a trabalhar melhor com subconsultas, aplicando várias técnicas de transformação. Os algoritmos de avaliação da seletividade de índices foram aprimorados e novas capacidades para ordenação apareceram. Foi na versão 2.5 que muitas empresas começaram a usar o Firebird seriamente em grandes projetos, e o otimizador lidou com essa carga.

A introdução das Common Table Expressions (CTE) mudou radicalmente a abordagem para escrever consultas complexas. Agora era possível dividir consultas grandes em blocos lógicos, tornando-as legíveis, mantíveis e gerenciáveis. O otimizador aprendeu a trabalhar com CTEs recursivas, abrindo novas possibilidades para processar dados hierárquicos — árvores de categorias, estruturas organizacionais, rotas tecnológicas.

Além disso, no nível do Firebird 2.0-2.5, surgiu a estatística para segmentos de índices compostos, permitindo que o otimizador processasse melhor condições complexas.

Firebird 3.0: Um Passo Revolucionário

A versão 3.0 se tornou um verdadeiro avanço e um ponto de virada na história do Firebird.

As funções de janela (Window Functions) adicionaram um conjunto de ferramentas poderoso para consultas analíticas. Tarefas como "calcular a soma acumulada" ou "encontrar os 3 principais em cada categoria", que antes exigiam complexas autojunções ou código procedural com cursores, agora eram resolvidas com consultas declarativas elegantes de poucas linhas. O otimizador recebeu algoritmos especializados para processar funções de janela de forma eficiente, tornando a análise diretamente em SQL uma realidade.

A versão 3.0 trouxe suporte ao Hash Join — um algoritmo alternativo de junção de tabelas que, em certos cenários, funciona ordens de magnitude mais eficiente do que o tradicional Nested Loop Join. Para consultas que unem grandes tabelas com boa seletividade de filtros, o Hash Join pode proporcionar um ganho de desempenho múltiplo. Imagine: uma consulta que levava 2 minutos com Nested Loop, e depois que o otimizador escolheu o Hash Join — 5 segundos. Tais maravilhas se tornaram realidade.

Além disso, na versão 3.0, a arquitetura do mecanismo foi significativamente reformulada, criando a base para futuras melhorias no otimizador.

Firebird 4.0: LATERAL e Mudanças Evolutivas

O surgimento das junções LATERAL expandiu os horizontes do possível, permitindo a criação de subconsultas correlacionadas com acesso a colunas de tabelas externas. Tarefas como "para cada cliente encontrar seus últimos 3 pedidos" ou "obter os principais produtos por categoria", que antes eram extremamente difíceis ou mesmo impossíveis no âmbito de uma única consulta SQL eficiente, agora eram resolvidas de forma elegante e rápida.

Além disso, no Firebird 4.0, surgiu a otimização de ordenações amplas com Refetch, expandindo as possibilidades de otimização de consultas "amplas" (relatórios, exportações, etc.).

Firebird 5.0: Capacidades Modernas

A última versão principal continuou a evolução do otimizador. Foram feitas melhorias adicionais nos algoritmos de estimativa de custo de consultas, expandidas as capacidades para transformação de subconsultas e adicionadas novas otimizações para padrões específicos de consultas. O otimizador se tornou mais "inteligente" na escolha entre Hash Join e Nested Loop Join, aprendeu a trabalhar melhor com predicados complexos e melhorou o suporte a IN.

É importante notar que, com cada versão, o otimizador não apenas ganhou novas capacidades — ele se tornou mais previsível e gerenciável. Novas ferramentas de monitoramento surgiram, como a tabela MON\$COMPILED_STATEMENTS no Firebird 5.0, que permite espiar o cache de consultas compiladas e ver os planos de execução sem a necessidade de reexecutar a consulta.

Sobre este livro: um roteiro por capítulos

À sua frente não está apenas um manual de SQL ou uma descrição das capacidades do Firebird. Este é um guia prático que revela sistematicamente os segredos do otimizador e ensina a pensar como ele. Cada capítulo é construído com base no princípio "da teoria à prática", onde primeiro os conceitos são explicados e, em seguida, é mostrado como aplicar esse conhecimento para escrever consultas eficientes.

Como medir o desempenho das consultas

O caminho para a otimização começa com a capacidade de medir corretamente. No primeiro capítulo, você aprenderá a usar as ferramentas integradas do Firebird para medir o desempenho. Você descobrirá o que significam os contadores Reads, Writes, Fetches e como eles se relacionam com o tempo real de execução da consulta. Você aprenderá a usar o isql para obter estatísticas e entenderá a diferença entre uma execução "fria" da consulta (quando os dados são lidos do disco) e uma "quente" (quando tudo já está no cache). Este capítulo estabelece a base para todo o resto, porque é impossível otimizar o que você não pode medir corretamente. Como disse Lord Kelvin: "Se você não pode medir, você não pode melhorar".

O que é um plano de execução de consulta

Aqui começa a verdadeira imersão no trabalho do otimizador, o momento em que o véu do mistério é levantado. Você aprenderá o que são os planos Legacy e Explain e, o mais importante — aprenderá a lê-los e interpretá-los, como um detetive experiente lê pistas. Você verá como obter o plano de uma consulta de várias maneiras, incluindo o uso de SET EXPLAIN no isql e das tabelas de monitoramento MON\$STATEMENTS e MON\$COMPILED_STATEMENTS. Compreender o plano de execução é uma habilidade fundamental, absolutamente crítica para o sucesso. Sem ela, todo o resto do

conhecimento sobre otimização será incompleto, como tentar dirigir um carro com os olhos vendados.

Métodos de acesso usados no Firebird

Este é um dos capítulos centrais e mais importantes do livro, o coração de todo o sistema de conhecimento sobre o otimizador. Aqui são detalhados, com exemplos e explicações, todos os métodos que o otimizador pode usar para extrair dados: varredura completa da tabela (Full Table Scan), vários tipos de acesso por índice, mapas de bits (Bitmap), varredura de intervalo (Range Scan). Para cada método, são explicadas detalhadamente as condições de sua aplicação, vantagens, desvantagens e custo. Você entenderá por que, às vezes, uma varredura completa de uma tabela pequena é mais rápida do que o acesso por índice, e como o otimizador toma essa decisão com base em estatísticas. Atenção especial é dada ao conceito de seletividade de índices e como ele influencia criticamente a escolha do método de acesso.

Este capítulo foi publicado anteriormente como um white paper para o público em geral e contém os pontos-chave do funcionamento do otimizador.

Transformações de consultas

O otimizador não apenas executa a consulta como você a escreveu — ele transforma a consulta em uma forma equivalente, mas mais eficiente. Neste capítulo, são revelados os segredos de tais transformações: como o otimizador trabalha com o predicado LIKE, transforma subconsultas, aplica inversão de predicados. Compreender esses mecanismos permite escrever consultas que são mais facilmente otimizáveis.

Estatísticas de tabelas e gstat

Aqui você aprenderá sobre a importância crítica de estatísticas atualizadas para o correto funcionamento do otimizador. Você verá como as estatísticas são coletadas, o que é seletividade e como usar a ferramenta gstat para analisar a distribuição dos dados. Você aprenderá a determinar quando as estatísticas estão desatualizadas e exigem atualização e entenderá a conexão entre estatísticas e planos de execução de consultas.

Uso de índices

Índices são uma ferramenta poderosa, mas apenas quando aplicados corretamente. Neste capítulo, todo o conhecimento sobre índices é sistematizado: quando eles são usados, quando são ignorados, como criar índices compostos eficientes, o que são índices parciais e como funcionam. Atenção especial é dada às condições sob as quais os índices são aplicados para ORDER BY e GROUP BY sem ordenação adicional.

Otimização de junções (JOIN)

As junções de tabelas são frequentemente o ponto mais estreito e problemático no desempenho das consultas, a fonte de uma parte significativa dos problemas. Aqui, todos os algoritmos de junção disponíveis são examinados em detalhes: o clássico Nested Loop Join, o moderno Hash Join e o Merge Join. Você aprenderá como o otimizador determina a ordem de junção das tabelas, o que é "custo" de junção e como ele é calculado e, o mais importante — como você pode influenciar as decisões do otimizador quando ele erra. Atenção especial é dada às diferenças fundamentais entre

INNER JOIN e LEFT JOIN do ponto de vista da otimização — a diferença aqui pode ser dramática. Você entenderá quando o Hash Join lhe dará um ganho de 10 vezes e quando permanecerá não utilizado.

Otimização de ordenações

A ordenação pode ser uma operação muito custosa, especialmente para grandes volumes de dados. Neste capítulo, você aprenderá quando a ordenação é inevitável e quando pode ser evitada usando a navegação por índice. Neste capítulo, o método Refetch é analisado, você verá como ele ajuda a reduzir o volume de dados a serem ordenados. Você entenderá a conexão entre a memória para ordenação (TempCacheLimit) e o desempenho e aprenderá a otimizar consultas com ORDER BY.

Otimização de agrupamentos

O agrupamento com GROUP BY é outra operação potencialmente cara. Aqui são consideradas as condições de uso de índices para agrupamento, maneiras de reduzir o custo do agrupamento movendo-o para uma CTE, otimização de funções agregadas. Você verá como a estrutura da consulta influencia a eficiência do agrupamento e aprenderá a escolher a abordagem ideal para cada tarefa específica.

Conclusão

Na parte final, são apresentadas as conclusões e discutidos tópicos que ficaram de fora desta edição, mas serão abordados em versões futuras do livro: otimização de código procedural, uso do profiler, trabalho com funções de janela, desnormalização e agregados armazenados.

Como ler este livro: recomendações práticas

Este livro foi escrito para ser útil a leitores com diferentes níveis de preparação e diversos objetivos. Aqui estão alguns cenários de uso:

Para desenvolvedores iniciantes:

Se você está apenas começando a trabalhar com Firebird ou tem conhecimentos básicos de SQL, recomendamos fortemente ler o livro de forma sequencial, do primeiro ao último capítulo, sem pular nada. O material é cuidadosamente estruturado com uma progressão de complexidade: primeiro são estabelecidos os conceitos e a terminologia básicos, depois, sobre essa base sólida, são explicados tópicos mais complexos e avançados. Não tenha pressa e não tente "passar os olhos" pelo livro em um fim de semana — reserve tempo para um entendimento profundo de cada conceito, reflita sobre os exemplos, experimente em seu próprio banco de dados de teste.

É especialmente importante dominar minuciosamente os três primeiros capítulos: medição de desempenho, leitura de planos de execução e métodos de acesso. Este é o alicerce, as pedras angulares sobre as quais todo o edifício do seu conhecimento sobre otimização é construído. Se algo não ficou claro — não hesite em voltar e reler. Crie seu próprio banco de dados de teste com os exemplos do livro, experimente, alterando as consultas e observando atentamente as mudanças nos planos de execução e nas estatísticas. A prática através de experimentos é o melhor professor.

Para desenvolvedores experientes:

Se você já tem experiência com SQL e Firebird, pode usar o livro como uma referência, consultando

capítulos específicos conforme a necessidade. Enfrentou uma junção de tabelas lenta — leia o capítulo sobre JOIN. Problemas com ordenação — vá para a seção correspondente.

No entanto, recomendamos pelo menos dar uma olhada rápida em todos os capítulos — você certamente encontrará técnicas e abordagens que não conhecia ou sobre as quais não havia pensado. Preste atenção especial aos capítulos sobre transformações de consultas e uso de índices — há muitos detalhes não óbvios que podem explicar o comportamento misterioso do otimizador em suas consultas.

Para administradores de banco de dados:

Se sua principal tarefa é garantir o desempenho de sistemas existentes, preste atenção especial aos capítulos sobre estatísticas, gstat e métodos de acesso. Compreender como o otimizador usa estatísticas para tomar decisões ajudará você a configurar bancos de dados corretamente e criar índices eficazes.

Também serão úteis os capítulos sobre junções e ordenações — eles ajudarão a identificar gargalos em consultas escritas por desenvolvedores e a sugerir formas de otimizá-las.

Para resolver problemas específicos:

Se você veio a este livro com um problema de desempenho específico, aqui está um caminho rápido:

1. Comece com o capítulo "Como medir o desempenho" — certifique-se de que está medindo o problema corretamente.
2. Estude o capítulo "O que é um plano de execução" — obtenha o plano da sua consulta lenta.
3. Encontre no plano o gargalo (geralmente é uma operação com alta cardinalidade ou um alto número de Reads).
4. Vá para o capítulo correspondente: se o problema está na junção — leia sobre JOIN, se na ordenação — sobre ORDER BY, se na varredura completa de uma tabela grande — sobre métodos de acesso e índices.

Exercícios práticos:

Embora este livro apresente inúmeros exemplos, você obterá o maior benefício se experimentar por conta própria. Se você não tem um banco de dados de tamanho adequado à mão, crie um banco de dados de teste, preencha-o com dados (quanto mais, melhor — pelo menos dezenas ou centenas de milhares de registros nas tabelas principais) e tente reproduzir as situações descritas no livro.

O melhor é pegar suas consultas reais de projetos de trabalho e analisá-las usando o conhecimento adquirido. Obtenha os planos de execução, observe as estatísticas, tente aplicar as técnicas de otimização.

Sobre os exemplos e versões:

Os exemplos neste livro são aplicáveis ao Firebird versões 3.0 e superiores, com observações separadas para funcionalidades introduzidas nas versões 4.0 e 5.0. Se você trabalha com versões mais antigas, a maior parte do material ainda será relevante, mas algumas funcionalidades (por

exemplo, Hash Join) não estarão disponíveis.

Ferramentas para o trabalho:

O livro utiliza as ferramentas padrão do Firebird: o utilitário de linha de comando isql, rastreamento (tracemgr), o utilitário gstat. Todos eles são fornecidos junto com o Firebird e não requerem instalação adicional. Embora você possa usar qualquer ferramenta gráfica de administração para executar consultas e obter estatísticas, os exemplos no livro são dados especificamente para isql, para eliminar a dependência de ferramentas de terceiros e mostrar o funcionamento "de forma pura".

O que você obterá após a leitura deste livro

Ao concluir o estudo do material, você possuirá conhecimento sistêmico sobre como o Firebird executa consultas SQL. Não é apenas um conjunto de técnicas e truques isolados — é uma compreensão holística e profunda do funcionamento do otimizador, que mudará fundamentalmente sua abordagem para escrever consultas e projetar bancos de dados.

Você aprenderá a "pensar como o otimizador" — prever qual plano de execução será escolhido para sua consulta antes mesmo de executá-la, e será capaz de escrever antecipadamente código que será executado com eficiência na primeira tentativa. Você deixará de depender de suposições, intuição e do método de tentativa e erro, e começará a tomar decisões fundamentadas e confiantes com base na compreensão dos mecanismos internos de funcionamento do SGBD.

Habilidades práticas que você obterá e poderá aplicar todos os dias:

- Capacidade de identificar rapidamente gargalos de desempenho em consultas em minutos, não em horas
- Capacidade de ler e analisar planos de execução de consultas como um livro aberto
- Compreensão de quando, onde e quais índices criar para máxima eficiência
- Habilidade de otimizar junções complexas de múltiplas tabelas usando diferentes algoritmos JOIN
- Capacidade de usar ordenações e agrupamentos de forma eficaz, evitando operações custosas
- Conhecimento das ferramentas internas para monitoramento, diagnóstico e perfilamento de consultas
- Capacidade de prever o comportamento do otimizador antes mesmo de executar a consulta

Mas o principal — você obterá confiança e liberdade profissional. Confiança de que pode resolver qualquer problema de desempenho, munido de conhecimento e ferramentas, e não apenas experimentar cegamente diferentes variações de consulta na esperança desesperada de uma melhoria aleatória. Confiança de que o código que você escreve funcionará rapidamente não apenas com dados de teste de dez registros, mas também na operação real em produção com milhões de registros, quando a satisfação do usuário e o sucesso do negócio estão em jogo.

Este livro não o transformará em um guru de otimização da noite para o dia — a verdadeira maestria vem apenas com a prática e a experiência. Mas ele lhe dará um mapa preciso do terreno, uma bússola confiável para navegação e um conjunto comprovado de ferramentas profissionais

para uma jornada bem-sucedida no mundo das consultas SQL de alto desempenho. O resto está em suas mãos.

Bem-vindo ao fascinante mundo do otimizador do Firebird. Aqui não há espaço para magia e acaso — apenas lógica, compreensão e resultados mensuráveis. Vamos começar esta emocionante jornada rumo à maestria!

Alexey Kovyazin, Vice-Presidente do Firebird Foundation (2025-2026)

Capítulo 1. Como medir o desempenho de consultas?

A principal medida de desempenho de uma consulta é o seu tempo de execução. Para consultas SQL que retornam um cursor, o seu tempo de execução é o tempo de extração de **todas** as linhas da consulta.



Em alguns casos, a responsividade da aplicação é importante, quando o primeiro lote de dados é mostrado ao usuário o mais rápido possível, e todas as linhas do cursor podem nem ser selecionadas. Geralmente, são aplicações com grades de dados. Neste caso, o tempo de execução pode ser medido como o tempo para obter o primeiro lote de dados (a quantidade de linhas pode variar). Além disso, tais consultas exigem uma mudança na estratégia de otimização. As estratégias de otimização serão descritas em detalhes posteriormente na seção correspondente.

Como então medir o tempo de execução de uma consulta? Existem muitas ferramentas para isso. A mais simples delas é um cronômetro em sua aplicação, que obtém o tempo antes do início da execução da consulta e após sua conclusão, e então calcula a diferença entre esses pontos. Por exemplo, é assim que a ferramenta interativa `isql` calcula o tempo de execução de consultas SQL. Uma ferramenta mais precisa para medir o desempenho de consultas é o rastreamento (`trace`), pois realiza as medições no lado do servidor.

Além disso, existe o profiler PSQL, que também mede o tempo de execução de consultas e suas partes. Mas ele introduz uma sobrecarga significativa, pois pode realizar medições dezenas de milhares de vezes por segundo, e destina-se principalmente a identificar gargalos em lógica procedural complexa. Neste caso, a precisão da medição do tempo total de execução da consulta não é tão importante quanto a proporção dos tempos de execução de suas partes individuais (métodos de acesso ou instruções PSQL).

Falaremos sobre rastreamento e profiler mais tarde, mas por enquanto vamos aprender a medir o desempenho de consultas usando `isql`.

1.1. Ativando a saída de estatísticas de execução no `isql`

Por padrão, o `isql` exibe apenas os resultados da execução da consulta, mas não as estatísticas. Para ativar a saída das estatísticas de execução da consulta, é necessário usar o comando `SET STAT`.

Sintaxe do comando SET STAT

```
SET STAT[s] [ON|OFF]
```

Vamos ver o que é exibido quando as estatísticas estão ativadas.

```
SQL> connect inet://localhost/test user SYSDBA password 'masterkey';
Database: inet://localhost/test, User: SYSDBA
SQL> SET STAT ON;
```

```
SQL> SELECT COUNT(*) FROM HORSE;
```

```

          COUNT
=====
          538832

Current memory = 551606960
Delta memory = 334112
Max memory = 551691824
Elapsed time = 3.491 sec
Buffers = 32768
Reads = 9468
Writes = 0
Fetches = 577191

```

Como você pode ver, primeiro é exibido o resultado da execução da consulta e depois as estatísticas da sua execução.

1.2. O que esses números significam?

- **Current memory** — é o consumo atual de memória em bytes pelo processo Firebird.
- **Delta memory** — é a mudança no consumo de memória pelo processo Firebird após a execução do comando atual. Pode ser negativo se a memória foi devolvida ao SO.
- **Max memory** — é o consumo máximo de memória pelo processo Firebird.
- **Elapsed time** — tempo de execução da última instrução. Inclui também o tempo de preparação da instrução SQL, o tempo de execução no lado do servidor e o tempo de retorno de todas as linhas do cursor.
- **Buffers** — tamanho do cache de páginas em páginas. Este valor é constante pelo menos dentro da sessão. Geralmente é igual ao valor do parâmetro de configuração `DefaultDbCachePages` ou ao valor especificado no cabeçalho do banco de dados.
- **Reads** — número de páginas lidas do disco.
- **Writes** — número de páginas gravadas no disco.
- **Fetches** — número de leituras lógicas de páginas. Número de leituras do cache de páginas.

Vamos tentar executar a mesma consulta novamente.

```
SQL> SELECT COUNT(*) FROM HORSE;
```

```

          COUNT
=====
          538832

Current memory = 551607056
Delta memory = 96
Max memory = 551691824
Elapsed time = 0.322 sec
Buffers = 32768
Reads = 0

```

```
Writes = 0
Fetches = 576500
```

Como você pode ver, as estatísticas de execução mudaram. A primeira coisa que chama a atenção é o tempo de execução diferente. A primeira execução foi em um banco de dados "frio" com cache não aquecido, por isso foi lenta. A segunda execução foi rápida. O tamanho do cache de páginas foi suficiente para que todas as páginas coubessem nele, portanto a segunda execução da mesma consulta não lê páginas do disco. Tudo está no cache de páginas.

Agora vamos encerrar a conexão e executar a mesma consulta novamente em uma nova conexão.

```
SQL> connect inet://localhost/test user SYSDBA password 'masterkey';
Commit current transaction (y/n)?y
Committing.
Database: inet://localhost/test, User: SYSDBA
SQL> SELECT COUNT(*) FROM HORSE;

              COUNT
=====
              538832

Current memory = 551590528
Delta memory = 334112
Max memory = 551675392
Elapsed time = 0.545 sec
Buffers = 32768
Reads = 9469
Writes = 0
Fetches = 577191
```

As estatísticas mudaram novamente. A execução é claramente mais rápida que a primeira vez, mas mais lenta que a segunda. Qual é o motivo?

Como se sabe, além do próprio cache de páginas, o Firebird também pode usar o cache de arquivos do sistema operacional. Isso depende dos parâmetros de configuração `UseFileSystemCache` ou `FileSystemCacheThreshold`. Neste caso, estamos usando a arquitetura `SuperServer`, a conexão com o banco de dados é única. Quando a última conexão com o banco de dados é encerrada, o cache de páginas é liberado. No entanto, o cache de arquivos do SO vive sob outras leis, quando será liberado, o SO decide. Neste caso, parte do banco de dados ainda permaneceu no cache de arquivos do SO, portanto, apesar da presença de `Reads`, a consulta é executada rapidamente, embora mais lenta do que quando as páginas necessárias estão no cache de páginas.

Então, qual resultado usar? Normalmente, mais de uma conexão trabalha com seu sistema, e essas conexões são mais ou menos permanentes, portanto, a forma mais correta será comparar o desempenho das consultas pela segunda execução. No futuro, ao apresentar os materiais deste livro, sempre operaremos com os resultados da segunda execução da consulta.

1.3. Como obter o tempo de extração de todas as linhas

Nos exemplos anteriores, medimos o desempenho de consultas que retornavam apenas uma linha. Nesse caso, os custos de exibição dos dados no console e sua transmissão pela rede eram mínimos, portanto o tempo de execução da consulta não difere muito do resultado obtido. Mas e se for necessário medir o tempo de execução de uma consulta que retorna centenas de milhares de linhas?

Primeiro, vamos tentar medir e depois minimizar os custos de exibição dos dados.

Vamos executar a seguinte consulta, que preencherá o cache de páginas e exibirá a quantidade total de dados na tabela TRIAL.

```
SQL> set stat on;
SQL> select count(*) from trial;
```

```

          COUNT
=====
          160828

```

```

Current memory = 551532192
Delta memory = 254208
Max memory = 551616288
Elapsed time = 0.130 sec
Buffers = 32768
Reads = 1855
Writes = 0
Fetches = 168550

```

E agora vamos executar uma consulta que exibe uma das colunas desta tabela

```
SQL> select code_trial from trial;
```

```

...
      CODE_TRIAL
=====
      486580
      486581
      486582
      486583
      486584
      486585
      486586
      486587

```

```

Current memory = 551555472
Delta memory = 23280
Max memory = 551628576
Elapsed time = 68.636 sec
Buffers = 32768
Reads = 1
Writes = 0

```



```
Fetches = 168098
```

Tenha paciência, a consulta levará muito tempo. No lugar das reticências, muitos dados serão exibidos. Isso não apenas distorce o tempo de execução da consulta, mas também torna as medições com `isql` inconvenientes. Mas há uma saída simples para esta situação: é necessário redirecionar a saída dos resultados da consulta para um dispositivo nulo. Para isso, execute o comando

```
OUTPUT nul;
```

Vamos tentar novamente

```
SQL> OUTPUT nul;
SQL> select code_trial from trial;
Current memory = 551555568
Delta memory = 96
Max memory = 551628576
Elapsed time = 0.903 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 168085
```

Assim, eliminamos completamente os atrasos na exibição dos dados.

Para retornar ao modo de exibição de dados, digite o comando

```
OUTPUT;
```

1.4. Transformação de Consultas

Como eliminar atrasos na transferência de dados pela rede? Atualmente, não há como.

No entanto, ao otimizar consultas SQL, é possível otimizar não a consulta original, mas outra consulta transformada, e medir o tempo de execução dessa consulta transformada. Após a otimização estar concluída, retorna-se à consulta original.

Normalmente, essa transformação se resume a

```
SELECT
  COUNT(*)
FROM (<source_query>)
```

onde `<source_query>` é a consulta original a ser otimizada.



Por que essa forma específica de transformação é usada? Porque essa consulta

força a extração de todos os registros da consulta original e, ao mesmo tempo, retorna exatamente um registro.

Suponha que você esteja tentando otimizar a seguinte consulta

```
SELECT
  H.NAME
FROM
  HORSE H
  JOIN FARM ON FARM.CODE_FARM = H.CODE_FARM
WHERE H.CODE_DEPARTURE = 1
      AND FARM.CODE_COUNTRY = 1
```

Vamos aplicar a transformação mencionada acima

```
SELECT
  COUNT(*)
FROM (
  SELECT
    H.NAME
  FROM
    HORSE H
    JOIN FARM ON FARM.CODE_FARM = H.CODE_FARM
  WHERE H.CODE_DEPARTURE = 1
        AND FARM.CODE_COUNTRY = 1
)
```

A consulta original não possui partes que são calculadas após as cláusulas FROM e WHERE, portanto, a consulta pode ser simplificada.

```
SELECT
  COUNT(*)
FROM
  HORSE H
  JOIN FARM ON FARM.CODE_FARM = H.CODE_FARM
WHERE H.CODE_DEPARTURE = 1
      AND FARM.CODE_COUNTRY = 1
```

Agora é possível otimizar a consulta que retorna exatamente um registro. Você pode criar ou remover índices, usar várias dicas e comparar o tempo de execução da nova consulta. Quando ele melhorar, você pode voltar à consulta original, substituindo COUNT(*) pela lista de campos originais.

Se a consulta for mais complexa, tais simplificações não podem ser aplicadas. Por exemplo, se estivermos otimizando a seguinte consulta

```
SELECT
  MEASURE.CODE_HORSE,
  AVG(MEASURE.HEIGHT_HORSE) AS AVG_HEIGHT_HORSE
FROM
```

```

MEASURE
WHERE EXTRACT(YEAR FROM MEASURE.BYDATE) = 2022
GROUP BY MEASURE.CODE_HORSE

```

não é possível substituir o conteúdo da cláusula SELECT, portanto, a consulta transformada ficará assim

```

SELECT COUNT(*)
FROM (
  SELECT
    MEASURE.CODE_HORSE,
    AVG(MEASURE.HEIGHT_HORSE) AS AVG_HEIGHT_HORSE
  FROM
    MEASURE
  WHERE EXTRACT(YEAR FROM MEASURE.BYDATE) = 2022
  GROUP BY MEASURE.CODE_HORSE
)

```

Ao otimizar consultas, tais transformações serão usadas com bastante frequência, portanto, recomendo memorizá-las.

1.5. Medição do Desempenho de Consultas Sob Carga

E se for necessário medir o desempenho de uma consulta sob carga de trabalho, ou seja, quando há outras conexões trabalhando com o banco de dados? Isso pode ser feito da mesma forma que ao medir uma consulta sem carga, mas há nuances.

Vamos ver como a carga afeta os resultados das medições. Primeiro, vamos medir o desempenho da consulta sem carga adicional.

```

SELECT COUNT(*) FROM HORSE;

```

Obtemos o seguinte resultado:

```

              COUNT
=====
              525875

Current memory = 551664112
Delta memory = 96
Max memory = 551748880
Elapsed time = 0.117 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 562542

```

Agora, vamos emular uma carga de trabalho. Para isso, iniciamos outra sessão isql e executamos

uma consulta SQL que consome muitos recursos:

```
OUTPUT NUL;
SELECT * FROM COVER, TRIAL_LINE;
```



Aqui, uma consulta sem sentido é escrita intencionalmente, executando um CROSS JOIN de tabelas grandes, o que garante que a consulta será executada por tempo suficiente.

Executamos a consulta original novamente:

```
SELECT COUNT(*) FROM HORSE;
```

E obtemos:

```

              COUNT
=====
              525875

Current memory = 553062784
Delta memory = 0
Max memory = 553136800
Elapsed time = 0.126 sec
Buffers = 32768
Reads = 42
Writes = 0
Fetches = 566702
```

O tempo de execução aumentou um pouco, o que é normal, mas o contador Fetches também aumentou. Mas como isso é possível, se a consulta é a mesma? Vamos tentar executar mais uma vez:

```

              COUNT
=====
              525875

Current memory = 553062784
Delta memory = 0
Max memory = 553136800
Elapsed time = 0.128 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 566775
```

Novamente, um valor diferente para Fetches. Qual é o motivo? O motivo é que o Firebird está funcionando no modo Super Server. Nesse modo, todos os contadores relacionados a entrada/saída são compartilhados. Como os Reads, Writes e Fetches são calculados para a consulta executada? No

cache de páginas, há contadores para essas operações, que são sempre incrementados. A estatística de execução da consulta exibe a diferença entre os valores desses contadores após a execução e antes da execução da consulta. Portanto, se o servidor estiver funcionando no modo Super Server, os valores de Reads, Writes e Fetches sob carga serão variáveis aleatórias e, conseqüentemente, não se pode tirar conclusões com base neles. No entanto, o tempo de execução da consulta sob carga ainda pode ser medido.



No Firebird 5.0, essa influência é muito menor do que nas versões anteriores. A razão é a melhoria [CORE-7814](#).

Nos modos Classic e Super Classic, o cache de páginas é separado e, portanto, seus contadores são individuais para cada conexão. Assim, os valores de Reads, Writes e Fetches não diferirão entre o caso de uma única conexão e o caso sob carga de trabalho.



No Firebird 6.0, a API para obtenção de contadores de desempenho foi ampliada. Agora é possível obter os contadores Reads, Writes e Fetches tanto no nível do banco de dados (todas as conexões) quanto no nível da conexão (apenas a própria conexão).

No Firebird 6.0, a ferramenta isql exibe especificamente os contadores de página privados, o que permite obter os valores corretos desses contadores para medir o desempenho de consultas sob carga (se houver outras conexões).

1.6. Usando Rastreamento para Medir o Desempenho de Consultas

E se ainda precisarmos avaliar Reads, Writes e Fetches sob carga no modo Super Server? Para isso, podemos usar o rastreamento.

Diferentemente do isql e de outras ferramentas para obter estatísticas de execução de consultas, no rastreamento os contadores Reads, Writes, Fetches e Marks são medidos diretamente para a consulta executada, e não são obtidos da memória compartilhada. Portanto, os valores desses contadores são mais precisos e não são afetados por outras conexões com o banco de dados.

Vamos ver como medir o desempenho de uma consulta usando rastreamento.

Antes de iniciar uma sessão de rastreamento, é necessário preparar um arquivo de configuração que permita exibir as estatísticas de execução de consultas. A configuração deve ser ajustada de forma que apenas os eventos do nosso banco de dados e apenas da conexão monitorada sejam incluídos no rastreamento. Para isso, usamos o seguinte modelo de arquivo de configuração:

```
database = <db_name>
{
    enabled = true
    log_initfini = false

    connection_id = <connection_id>
```

```

log_statement_prepare = true
log_statement_finish = true

print_perf = true

time_threshold = 0

max_sql_length = 8000
max_arg_length = 400
}

```

Onde <db_name> é o caminho para o banco de dados ou seu alias. Aqui, em vez do nome completo do banco de dados, pode-se escrever uma expressão regular. Nesse caso, os eventos de todos os bancos de dados cujo nome de arquivo ou alias corresponder à expressão regular serão incluídos no rastreamento.

<connection_id> é o identificador da conexão a ser monitorada. Ele pode ser obtido com a seguinte consulta:

```
SELECT CURRENT_CONNECTION FROM RDB$DATABASE;
```

```

CURRENT_CONNECTION
=====
                  136

```

Agora podemos criar o arquivo de configuração para o rastreamento:

```

database = test
{
    enabled = true
    log_initfini = false
    connection_id = 136
    log_statement_prepare = true
    log_statement_finish = true
    print_perf = true
    time_threshold = 0
    max_sql_length = 8000
    max_arg_length = 400
}

```

Vamos salvá-lo com o nome test_trace.conf.

Em seguida, iniciamos uma sessão interativa de rastreamento com o comando:

```
fbtracemgr -SE inet://localhost/service_mgr -START -NAME test_trace -USER SYSDBA -PASS masterkey -CONFIG test_trace.conf
```

Agora, vamos repetir nosso experimento de medição do desempenho da consulta:

```
SELECT COUNT(*) FROM HORSE;
```

A sessão de rastreamento interativo exibirá o seguinte:

```
2024-09-18T20:05:13.1940 (3996:00000000032F17C0) PREPARE_STATEMENT
  test (ATT_136, SYSDBA:NONE, UTF8, TCPv6:::1/58645)
  c:\Firebird\5.0\isql.exe:6588
    (TRA_3300, READ_COMMITTED | READ_CONSISTENCY | WAIT | READ_WRITE)

Statement 160:
-----
SELECT COUNT(*) FROM HORSE
      0 ms

2024-09-18T20:05:13.3030 (3996:00000000032F17C0) EXECUTE_STATEMENT_FINISH
  test (ATT_136, SYSDBA:NONE, UTF8, TCPv6:::1/58645)
  c:\Firebird\5.0\isql.exe:6588
    (TRA_3299, CONCURRENCY | WAIT | READ_WRITE)

Statement 160:
-----
SELECT COUNT(*) FROM HORSE
1 records fetched
      108 ms, 562542 fetch(es)

Table                                     Natural      Index      Update      Insert      Delete      Backout      Purge      Expunge
*****
HORSE                                     525875
```

Vamos iniciar, em uma nova sessão, a consulta que emula a carga de trabalho:

```
OUTPUT NUL;
SELECT * FROM COVER, TRIAL_LINE;
```

Na sessão em que estamos realizando as medições, executamos a consulta novamente:

```
SELECT COUNT(*) FROM HORSE;
```

A sessão de rastreamento interativo exibirá o seguinte:

```
2024-09-18T20:08:30.5300 (3996:00000000032F17C0) PREPARE_STATEMENT
  test (ATT_136, SYSDBA:NONE, UTF8, TCPv6:::1/58645)
  c:\Firebird\5.0\isql.exe:6588
    (TRA_3300, READ_COMMITTED | READ_CONSISTENCY | WAIT | READ_WRITE)

Statement 160:
-----
SELECT COUNT(*) FROM HORSE
      0 ms

2024-09-18T20:08:30.6450 (3996:00000000032F17C0) EXECUTE_STATEMENT_FINISH
  test (ATT_136, SYSDBA:NONE, UTF8, TCPv6:::1/58645)
  c:\Firebird\5.0\isql.exe:6588
```

```
(TRA_3299, CONCURRENCY | WAIT | READ_WRITE)
```

```
Statement 160:
```

```
-----  
SELECT COUNT(*) FROM HORSE
```

```
1 records fetched
```

```
114 ms, 562542 fetch(es)
```

Table	Natural	Index	Update	Insert	Delete	Backout	Purge	Expunge

HORSE	525875							

Como você pode ver, a quantidade de Fetches não mudou. Além disso, recebemos estatísticas adicionais de execução da consulta por tabela. Falaremos sobre o que isso é e como analisá-lo mais tarde.

1.7. Conclusões

- Ao medir o tempo de execução de consultas, para consultas que retornam um cursor, sempre busque todos os registros.
- Para reduzir o impacto da impressão de dados no console do `isql`, redirecione a saída para um dispositivo nulo usando o comando `OUTPUT nul`.
- Ao otimizar consultas, você pode aplicar várias transformações e medir o tempo das consultas transformadas para verificar suas hipóteses.
- Se o Firebird estiver em execução no modo SuperServer, ao medir o desempenho no `isql` ou em ferramentas GUI, os valores dos contadores Reads, Writes, Fetches e Marks são confiáveis apenas se sua conexão for a única.
- Se você precisar de valores precisos dos contadores Reads, Writes, Fetches e Marks sob carga, use o rastreamento personalizado com filtragem pelo seu banco de dados e identificador de conexão.

Capítulo 2. O que é um plano de execução de consulta?

Um plano de execução de consulta é uma sequência de operações necessárias para obter o resultado da execução de uma consulta SQL. A tarefa do otimizador é selecionar um plano de consulta no qual o resultado da execução da consulta seja obtido o mais rápido possível. O plano de execução da consulta é construído na fase de preparação ou compilação da consulta (prepare).

A habilidade de ler e analisar um plano de consulta é a principal competência para uma otimização bem-sucedida de consultas. No Firebird, existem duas formas de exibir o plano: Legacy e Explain.

2.1. Plano Legacy

A forma Legacy do plano está disponível desde as versões mais antigas do Firebird (herdada do Interbase). Ela é uma expressão que começa com a palavra-chave `PLAN`, seguida opcionalmente por um tipo de junção, após a qual um ou mais elementos do plano são especificados entre parênteses. Um elemento do plano pode ser outra expressão de plano (sem a palavra-chave `PLAN`). O plano na forma Legacy pode ser especificado diretamente na própria consulta SQL.



A expressão `PLAN` não pode descrever todos os métodos de acesso disponíveis nas versões modernas do Firebird, portanto, sua especificação manual em consultas atualmente é categoricamente desencorajada.

A sintaxe geral do plano Legacy é a seguinte.

Sintaxe da cláusula `PLAN`

```
PLAN <plan-expr>

<plan-expr> ::=
    (<plan-item> [, <plan-item> ...])
  | <sorted-item>
  | <joined-item>
  | <merged-item>
  | <hash-item>

<sorted-item> ::= SORT (<plan-item>)

<joined-item> ::= JOIN (<plan-item>, <plan-item> [, <plan-item> ...])

<merged-item> ::=
    [SORT] MERGE (<sorted-item>, <sorted-item> [, <sorted-item> ...])

<hash-item> ::= HASH (<plan-item>, <plan-item> [, <plan-item> ...])

<plan-item> ::= <basic-item> | <plan-expr>

<basic-item> ::= <relation> {
    NATURAL
  | INDEX (<indexlist>)
```

```
| ORDER index [INDEX (<indexlist>)]
}

<relation> ::= table | view [table]

<indexlist> ::= index [, index ...]
```

Seus elementos serão examinados posteriormente, durante a análise detalhada dos métodos de acesso.

Exemplos de consultas SQL e seus planos Legacy:

```
SELECT * FROM students;

PLAN (students NATURAL)
```

```
SELECT *
FROM students
WHERE class = '3C';

PLAN (students INDEX (ix_stud_class))
```

```
SELECT *
FROM students
ORDER BY id;

PLAN (students ORDER pk_students)
```

```
SELECT s.id, s.name, s.class, c.mentor
FROM students s
JOIN classes c ON c.name = s.class;

PLAN JOIN (s NATURAL, c INDEX (pk_classes))
```

2.2. Plano Explain

A forma Explain do plano apareceu no Firebird 3.0 e superior. Ela descreve o caminho de acesso com o maior nível de detalhe. **Caminho de acesso** é um conjunto de operações sobre dados executadas pelo servidor para obter o resultado de uma seleção especificada. O plano Explain é uma árvore com uma raiz que representa o resultado final da consulta. Cada nó dessa árvore é chamado de **método de acesso** ou **fonte de dados**. Os objetos das operações nos métodos de acesso são **fluxos de dados**. Cada método de acesso ou forma um fluxo de dados ou o transforma de acordo com regras específicas. Os nós folha da árvore são chamados de **métodos de acesso primários**. Sua única tarefa é a formação de fluxos de dados.

Ao contrário do plano Legacy, o plano Explain não pode ser especificado diretamente no corpo de

uma consulta SQL.

Exemplos de consultas SQL e seus planos Explain:

```
SELECT * FROM students;
```

Select Expression

-> Table "STUDENTS" Full Scan

```
SELECT *
FROM students
WHERE class = '3C';
```

Select Expression

-> Filter

-> Table "STUDENTS" Access By ID

-> Bitmap

-> Index "IX_STUD_CLASS" Range Scan (full match)

Os elementos do plano Explain serão descritos posteriormente, durante a análise detalhada dos métodos de acesso.

2.3. Como obter um plano de execução de consulta?

Antes de mergulhar na descrição dos métodos de acesso e na análise dos planos de execução de consultas, vamos examinar como você pode obter um plano de execução de consulta usando as ferramentas internas do Firebird.

2.3.1. Obtendo o plano de execução da consulta na Firebird API

Qualquer aplicativo que trabalhe com o SGBD Firebird pode obter o plano de execução de uma consulta usando a Firebird API.

O plano de execução da consulta pode ser obtido imediatamente após a preparação da consulta. Na OO API, para isso é usado o método `getPlan` da interface `IStatement`. Sua assinatura é a seguinte:

```
const char* getPlan(StatusType* status, FB_BOOLEAN detailed)
```

Aqui, o primeiro parâmetro é um ponteiro para o objeto do vetor de status, para tratamento de erros, e o segundo é um sinalizador que determina se o plano será retornado na forma Explain (valor `true`) ou na forma Legacy (valor `false`).

Na Legacy API, para isso é usada a função `isc_dsql_sql_info()`. Para obter o plano Legacy, passe a tag `isc_info_sql_get_plan` no buffer de parâmetros, e para obter o plano Explain, passe `isc_info_sql_explain_plan`.

Para informações mais detalhadas, consulte o Firebird API Guide.

Obtendo o plano de execução da consulta no isql

O plano da consulta pode ser obtido usando a utilidade de console isql. Por padrão, o plano da consulta não é exibido; para ativar ou desativar sua exibição, use o comando SET PLAN.

Sintaxe do comando SET PLAN

```
SET PLAN [ON|OFF]
```

```
SQL> set plan on;
SQL> select count(*) from rdb$relations;
```

```
PLAN (RDB$RELATIONS NATURAL)
```

```

              COUNT
=====
              270

```

Por padrão, o plano é exibido na forma Legacy; se você quiser exibir o plano na forma Explain, deve usar o comando SET EXPLAIN.

Sintaxe do comando SET EXPLAIN

```
SET EXPLAIN [ON|OFF]
```

```
SQL> set explain on;
SQL> select count(*) from rdb$relations;
```

```

Select Expression
  -> Aggregate
    -> Table "RDB$RELATIONS" Full Scan

```

```

              COUNT
=====
              270

```



O comando SET EXPLAIN ON ativa implicitamente o sinalizador de exibição do plano, portanto, executar SET PLAN ON antes dele não faz sentido. O comando SET EXPLAIN OFF não desativa a exibição do plano, apenas muda a forma de exibição para Legacy. Para desativar a exibição do plano, será necessário executar SET PLAN OFF.

O comando SET PLAN [ON|OFF] não redefine a forma de exibição do plano.

Como você pode ter visto nos exemplos anteriores, não apenas os próprios planos das consultas são exibidos, mas as consultas também são executadas e seus resultados são mostrados. Se você quiser apenas ver o plano da consulta sem executá-la, deve usar o comando SET PLANONLY.

Sintaxe do comando SET PLANONLY

```
SET PLANONLY [ON|OFF]
```

```
SQL> set planonly;
SQL> select count(*) from rdb$relations;

PLAN (RDB$RELATIONS NATURAL)
SQL> set explain on;
SQL> select count(*) from rdb$relations;

Select Expression
  -> Aggregate
    -> Table "RDB$RELATIONS" Full Scan
SQL>
```

Claro, você pode combinar a exibição de estatísticas de execução e do plano da consulta usando o seguinte conjunto de comandos.

```
set explain on;
set stat on;
output nul;
select * from rdb$relations;
```

```
Select Expression
  -> Table "RDB$RELATIONS" Full Scan
Current memory = 551378048
Delta memory = 26336
Max memory = 551462624
Elapsed time = 0.041 sec
Buffers = 32768
Reads = 12
Writes = 0
Fetches = 790
```

2.3.2. Obtendo o plano de execução da consulta no rastreamento

Os planos de consulta também podem ser obtidos usando o rastreamento do usuário. Para isso, é necessário adicionar a seguinte linha ao arquivo de configuração

```
print_plan = true
```

Este parâmetro adicionará a exibição do plano de consultas na forma Legacy. Se for necessário que o plano seja exibido na forma Explain, então é necessário adicionar mais um parâmetro.

```
explain_plan = true
```



```

-> Aggregate
  -> Table "HORSE" Full Scan
1 records fetched
595 ms, 9415 read(s), 576500 fetch(es)

Table                                     Natural   Index   Update   Insert   Delete   Backout   Purge   Expunge
*****
HORSE                                     538832

```

Como você pode ver, a exibição do plano ocorre tanto para o evento `PREPARE_STATEMENT` quanto para o evento `EXECUTE_STATEMENT_FINISH`.

O rastreamento é uma ferramenta poderosa. Você pode obter não apenas os planos das próprias consultas, mas também os planos para cursores localizados dentro de procedimentos armazenados, funções e gatilhos. Os planos de cursores de objetos armazenados podem ser obtidos durante a fase de compilação, após a qual eles entram no cache de metadados. Para obter os planos de cursores de objetos armazenados, você precisa complementar a configuração com os seguintes parâmetros (você pode ativar apenas os necessários).

```

log_procedure_compile = true
log_function_compile = true
log_trigger_compile = true

```

Vamos tentar exibir os planos de cursores de um procedimento armazenado no rastreamento. Para isso, usamos o seguinte arquivo de configuração.

```

database = employee
{
  enabled = true
  log_initfini = false
  log_statement_prepare = true
  log_statement_finish = true
  log_procedure_compile = true
  print_plan = true
  explain_plan = true
  print_perf = true
  time_threshold = 0
  max_sql_length = 8000
  max_arg_length = 400
}

```

Vamos salvá-lo com o nome `employee_trace.conf`.

Em seguida, iniciamos uma sessão interativa de rastreamento com o comando:

```

fbtracemgr -SE inet://localhost/service_mgr -START -NAME employee_trace -USER SYSDBA -PASS masterkey -CONFIG
employee_trace.conf

```

Agora, execute a seguinte consulta no `isql`:

Os planos para cursores dentro do procedimento armazenado são exibidos no evento `COMPILE_PROCEDURE`. Observe que este evento ocorreu duas vezes: para o procedimento `SHOW_LANGS` e para o procedimento `ALL_LANGS`. O fato é que o procedimento `SHOW_LANGS` é chamado dentro do procedimento `ALL_LANGS`. Nos nós raiz do plano (Select Expression) são exibidos os números da linha e da coluna onde esses cursores são definidos.

2.4. Obtendo o plano de execução de consulta nas tabelas MON\$

Além do rastreamento e auditoria, o Firebird possui tabelas de monitoramento que fornecem um instantâneo do estado atual do banco de dados. Este instantâneo captura os valores de vários contadores de desempenho, memória consumida, lista de conexões ativas, transações e consultas. Usando as tabelas de monitoramento, também podemos visualizar os planos de execução de consultas e cursores. Os nomes das tabelas de monitoramento têm o prefixo `MON$`.

2.4.1. Tabela MON\$STATEMENTS

A tabela `MON$STATEMENTS` destina-se a exibir consultas preparadas e consultas que estão sendo executadas no momento atual. O plano na forma `Explain` é exibido na coluna `MON$EXPLAINED_PLAN`, e o texto da consulta na coluna `MON$SQL_TEXT`.



Observe que na tabela `MON$STATEMENTS` você não verá consultas que foram executadas e depois liberadas. Em particular, a ferramenta `isql` após executar uma consulta e extrair todos os registros dela, destrói imediatamente a consulta, portanto, se a consulta for executada rapidamente, você não a encontrará na tabela `MON$STATEMENTS`.

Tabela 1. Descrição das colunas da tabela `MON$STATEMENTS`

Nome da coluna	Tipo de dados	Descrição
<code>MON\$STATEMENT_ID</code>	<code>BIGINT</code>	Identificador da instrução.
<code>MON\$ATTACHMENT_ID</code>	<code>BIGINT</code>	Identificador da conexão.
<code>MON\$TRANSACTION_ID</code>	<code>BIGINT</code>	Identificador da transação.
<code>MON\$STATE</code>	<code>SMALLINT</code>	Estado da instrução: 0 — inativo (idle); 1 — em execução (active); 2 — suspenso (stalled).
<code>MON\$TIMESTAMP</code>	<code>TIMESTAMP</code>	Data e hora de início da instrução.
<code>MON\$SQL_TEXT</code>	<code>BLOB TEXT</code>	Texto da instrução em SQL.
<code>MON\$STAT_ID</code>	<code>INTEGER</code>	Identificador da estatística.
<code>MON\$EXPLAINED_PLAN</code>	<code>BLOB TEXT</code>	Plano da instrução na forma <code>explain</code> .

Nome da coluna	Tipo de dados	Descrição
MON\$STATEMENT_TIMEOUT	INTEGER	Tempo limite da instrução SQL no nível da instrução SQL. Contém o valor do tempo limite, definido no nível da conexão/instrução, em milissegundos. Se o tempo limite não estiver definido — 0.
MON\$STATEMENT_TIMER	TIMESTAMP	Hora de expiração do temporizador da instrução SQL. Contém NULL se o tempo limite da instrução SQL não estiver definido, ou se o temporizador não estiver em execução.
MON\$COMPILED_STATEMENT_ID	BIGINT	Identificador da consulta compilada (referência a MON\$COMPILED_STATEMENTS).

O estado da instrução **STALLED** — é o estado “suspensão”. É possível para uma consulta que iniciou sua execução, ainda não a concluiu, mas no momento não está sendo executada. Por exemplo, aguardando parâmetros de entrada ou a próxima busca (fetch) do cliente.

2.4.2. Tabela MON\$COMPILED_STATEMENTS

A partir do Firebird 5.0, está disponível a tabela de monitoramento MON\$COMPILED_STATEMENTS. Ela destina-se a exibir consultas compiladas e procedimentos armazenados, funções e triggers. Essencialmente, ela exibe o conteúdo do cache de consultas compiladas. Diferente da tabela MON\$STATEMENTS, em MON\$COMPILED_STATEMENTS as consultas são preservadas mesmo após sua destruição, até que o cache de consultas compiladas seja limpo.



O cache de consultas compiladas é limpo apenas durante a execução de instruções DDL. Como atualmente o cache de consultas compiladas em todas as arquiteturas é próprio para cada conexão, ele também é limpo quando a conexão é finalizada. Também vale notar que os registros na tabela MON\$COMPILED_STATEMENTS podem estar duplicados, pois consultas com o mesmo texto são diferentes para cada conexão. Isso pode mudar em versões futuras do Firebird.

Tabela 2. Descrição das colunas da tabela MON\$COMPILED_STATEMENTS

Nome da coluna	Tipo de dados	Descrição
MON\$COMPILED_STATEMENT_ID	BIGINT	Identificador da consulta compilada.
MON\$SQL_TEXT	BLOB TEXT	Texto da instrução em linguagem SQL. Dentro de objetos PSQL, o texto das instruções SQL não é exibido.
MON\$EXPLAINED_PLAN	BLOB TEXT	Plano da instrução na forma explain.
MON\$OBJECT_NAME	CHAR(63)	Nome do objeto PSQL no qual a instrução SQL foi compilada.

Nome da coluna	Tipo de dados	Descrição
MON\$OBJECT_TYPE	SMALLINT	Tipo do objeto. 2 — trigger; 5 — procedimento armazenado; 15 — função armazenada.
MON\$PACKAGE_NAME	CHAR(63)	Nome do pacote PSQL.
MON\$STAT_ID	INTEGER	Identificador da estatística.

Vamos ver como é possível visualizar o plano de execução usando a tabela MON\$COMPILED_STATEMENTS. A maneira mais simples é usar a consulta:

```
SELECT
  CS.MON$SQL_TEXT,
  CS.MON$EXPLAINED_PLAN
FROM MON$COMPILED_STATEMENTS CS;
```

Mas essa consulta retornará muitos registros, pois incluirá consultas internas feitas pelo isql e consultas de outras sessões ativas. Portanto, é necessário adicionar filtragem pelo texto completo da consulta ou por parte dele.

Por exemplo, se executamos no isql a consulta:

```
SELECT COUNT(*) FROM HORSE;
```

então reescreveremos nossa consulta para MON\$COMPILED_STATEMENTS da seguinte forma:

```
SELECT
  CS.MON$SQL_TEXT,
  CS.MON$EXPLAINED_PLAN
FROM MON$COMPILED_STATEMENTS CS
WHERE CS.MON$SQL_TEXT = 'SELECT COUNT(*) FROM HORSE';
```

O resultado será o seguinte:

```
MON$SQL_TEXT          0:1
SELECT COUNT(*) FROM HORSE
MON$EXPLAINED_PLAN    0:2

Select Expression
  -> Aggregate
    -> Table "HORSE" Full Scan

MON$SQL_TEXT          0:7
SELECT COUNT(*) FROM HORSE
MON$EXPLAINED_PLAN    0:8

Select Expression
  -> Aggregate
```

-> Table "HORSE" Full Scan

Observe que a consulta foi exibida duas vezes, pois na segunda sessão eu também executei a mesma consulta. Como uma consulta com o mesmo texto na maioria dos casos produzirá o mesmo plano, podemos simplesmente exibir a última.

```
SELECT
  CS.MON$SQL_TEXT,
  CS.MON$EXPLAINED_PLAN
FROM MON$COMPILED_STATEMENTS CS
WHERE CS.MON$SQL_TEXT = 'SELECT COUNT(*) FROM HORSE'
ORDER BY CS.MON$COMPILED_STATEMENT_ID DESC
FETCH FIRST ROW ONLY;
```

O texto da consulta pode ser muito longo e escrevê-lo na condição de filtragem nem sempre é possível, nem conveniente, mas você pode filtrar a consulta por alguma parte, embora neste caso eu não recomende limitar a saída ao primeiro registro.

```
SELECT
  CS.MON$SQL_TEXT,
  CS.MON$EXPLAINED_PLAN
FROM MON$COMPILED_STATEMENTS CS
WHERE CS.MON$SQL_TEXT CONTAINING 'FROM HORSE';
```

Usar MON\$COMPILED_STATEMENTS para visualizar planos de consultas executadas no isql não é uma ferramenta conveniente, é muito mais fácil obter o plano através de SET EXPLAIN ON. Mas esta tabela tem outra característica importante—ela exibe os planos de cursores em procedimentos armazenados, funções e triggers. Isso é muito mais conveniente do que executar constantemente o rastreamento para os mesmos fins.

Voltemos ao nosso exemplo com o banco de dados employee e a consulta ao procedimento armazenado ALL_LANGS.

```
SELECT COUNT(*) FROM ALL_LANGS;
```

Para obter os planos dos cursores dentro do procedimento armazenado ALL_LANGS, basta executar a seguinte consulta:

```
SELECT CS.MON$EXPLAINED_PLAN
FROM MON$COMPILED_STATEMENTS CS
WHERE CS.MON$OBJECT_NAME = 'ALL_LANGS'
AND CS.MON$OBJECT_TYPE = 5
ORDER BY CS.MON$COMPILED_STATEMENT_ID DESC
FETCH FIRST ROW ONLY;
```

MON\$EXPLAINED_PLAN

```

=====
0:38
=====
MON$EXPLAINED_PLAN:

Select Expression (line 10, column 6)
  -> Procedure "SHOW_LANGS" Scan
Select Expression (line 5, column 2)
  -> Table "JOB" Full Scan
=====

```

Diferente do texto da consulta, o nome do procedimento armazenado sempre pode ser especificado por completo, portanto, não haverá ambiguidades.

2.5. Um Olhar para o Futuro

Em outros SGBDs (MySQL e Postgres) estamos acostumados a obter o plano de consulta usando um operador especial EXPLAIN. Ele tem a seguinte forma

```
EXPLAIN <select-expr>
```

A partir do Firebird 6.0, a ferramenta isql também suporta o comando EXPLAIN, que não é um operador DSQL, ou seja, é implementado apenas na ferramenta isql. Vamos ver como ele funciona:

```
EXPLAIN SELECT COUNT(*) FROM HORSE;
```

Será exibido:

```

Select Expression
  -> Aggregate
    -> Table "HORSE" Full Scan

```

Na verdade, este comando simplesmente executa o procedimento de sistema RDB\$SQL.EXPLAIN, passando o texto da consulta para ele, e o resultado de saída deste procedimento é formatado dentro do isql.

2.5.1. Obtendo o Plano com RDB\$SQL.EXPLAIN

A equipe de desenvolvimento do Firebird decidiu que não deveria introduzir um novo operador DSQL EXPLAIN com comportamento diferente dos tipos atuais de operadores. Na classificação, EXPLAIN não é um cursor e não é a execução de um procedimento armazenado, mas ainda assim deve retornar um resultado. Em vez disso, a equipe de desenvolvimento do Firebird escolheu o caminho do Oracle — para obter o plano de uma consulta SQL, usa-se o procedimento EXPLAIN do pacote de sistema RDB\$SQL. Este procedimento recebe como entrada o texto da consulta SQL e retorna o plano Explain detalhado por fontes de dados (métodos de acesso), ou seja, cada registro na saída deste procedimento é um método de acesso.

Tabela 3. Parâmetros de entrada do procedimento `RDB$SQL.EXPLAIN`

Parâmetro	Tipo	Descrição
SQL	BLOB SUB_TYPE TEXT	Texto da consulta SQL.

Tabela 4. Parâmetros de saída do procedimento `RDB$SQL.EXPLAIN`

Parâmetro	Tipo	Descrição
PLAN_LINE	INTEGER	Número da linha do plano.
RECORD_SOURCE_ID	BIGINT	Identificador da fonte de dados.
PARENT_RECORD_SOURCE_ID	BIGINT	Identificador da fonte de dados pai.
LEVEL	INTEGER	Nível de recuo para a fonte de dados. Necessário ao construir um plano detalhado como uma única string.
OBJECT_TYPE	SMALLINT	Tipo do objeto de metadados: 0 — tabela; 5 — procedimento armazenado.
PACKAGE_NAME	CHAR(63)	Nome do pacote. Exibido se a fonte de dados for um procedimento armazenado de pacote.
OBJECT_NAME	CHAR(63)	Nome do objeto de metadados.
ALIAS	CHAR(63)	Apelido do objeto de metadados.
CARDINALITY	DOUBLE PRECISION	Cardinalidade (potência) da fonte de dados.
RECORD_LENGTH	INTEGER	Comprimento do registro em bytes.
KEY_LENGTH	INTEGER	Comprimento da chave em bytes.
ACCESS_PATH	BLOB SUB_TYPE TEXT	Descrição do método de acesso usado pela fonte de dados.

Esta implementação tem as seguintes vantagens sobre um operador `EXPLAIN` separado:

- não requer suporte especial do lado do cliente, ou seja, aplicativos antigos e o `fbclient` podem usar esta funcionalidade imediatamente, sem modificações.
- o plano pode ser exibido em qualquer formato conveniente, por exemplo, pode ser visualizado graficamente, como em algumas ferramentas populares do Oracle e MS SQL.

Além disso, o procedimento retorna uma nova coluna — a cardinalidade da fonte de dados. Esta informação atualmente não está disponível ao usar outras formas de obter o plano.

Exemplo de uso do procedimento `RDB$SQL.EXPLAIN` para obter um plano:

Exemplo 1. Usando o procedimento RDB\$SQL.EXPLAIN

```
SELECT *  
FROM RDB$SQL.EXPLAIN(Q'{  
  SELECT *  
  FROM  
  HORSE  
  JOIN COLOR ON COLOR.CODE_COLOR = HORSE.CODE_COLOR  
  JOIN BREED ON BREED.CODE_BREED = HORSE.CODE_BREED  
  WHERE HORSE.CODE_DEPARTURE = ?  
}')
```



Atualmente, o Firebird 6.0 está em desenvolvimento ativo. Todas as suas funcionalidades ainda não assumiram a forma final e podem mudar até o lançamento. Você pode "brincar" com a saída do plano usando o procedimento RDB\$SQL.EXPLAIN. Eu, por sua vez, usarei alguns de seus recursos, em particular a informação sobre a cardinalidade das fontes de dados ao explicar os métodos de acesso.

2.6. Conclusão

Neste capítulo, você aprendeu sobre todas as formas conhecidas de obter planos de execução de consultas, bem como planos de cursores dentro de procedimentos armazenados. Também demos uma olhada no futuro próximo e vimos alguns desenvolvimentos no Firebird 6.0 que simplificam a obtenção do plano de consulta e expandem a informação sobre os métodos de acesso.

Capítulo 3. Métodos de Acesso Utilizados no Firebird

Agora que você sabe como obter o plano de uma consulta SQL, vamos aprender a lê-lo. Para isso, é necessário familiarizar-se com os métodos de acesso utilizados pelo núcleo do Firebird.

Este capítulo descreve os métodos de acesso existentes no **Firebird 5.0**. Ele é baseado no artigo original [Firebird: métodos de acesso a dados](#) de Dmitry Emanov.

Em comparação com o artigo original, as seguintes alterações foram feitas:

- Foram adicionadas descrições de métodos de acesso que surgiram após o Firebird 2.0 — nas versões 2.1, 2.5, 3.0, 4.0 e Firebird 5.0.
- A árvore esquemática de execução de consultas foi substituída por planos Explain reais (introduzidos no Firebird 3.0).
- Como o artigo original apresentava exemplos de cálculo de cardinalidade e custo, eles foram adicionados aos planos Explain. Os valores de cardinalidade foram obtidos usando o procedimento `RDB$SQL.EXPLAIN` do Firebird 6.0. Os custos foram calculados usando fórmulas que estavam no artigo original ou que foram obtidas por mim através da análise do código-fonte do Firebird.
- Para o Hash Join, foi adicionada a fórmula de cálculo do seu custo.
- Foi adicionada uma descrição de como os filtros reduzem a cardinalidade (Filter, group by, distinct).

3.1. Terminologia

Caminho de acesso — é um conjunto de operações sobre dados, executadas pelo servidor para obter o resultado de uma seleção especificada. Um plano Explain representa uma árvore com uma raiz, que é o resultado final. Cada nó desta árvore é chamado de **método de acesso** ou **fonte de dados**. Os objetos das operações nos métodos de acesso são **fluxos de dados**. Cada método de acesso ou forma um fluxo de dados ou o transforma de acordo com regras específicas. Os nós folha da árvore são chamados de **métodos de acesso primários**. Sua única tarefa é a formação de fluxos de dados.

Do ponto de vista dos tipos de operações executadas, existem três classes de fontes de dados:

- método de acesso primário — realiza a leitura de uma tabela ou procedimento armazenado, formando o fluxo de dados original;
- filtro — transforma um fluxo de dados de entrada em um fluxo de dados de saída;
- junção — transforma dois ou mais fluxos de dados de entrada em um fluxo de dados de saída.

As fontes de dados podem ser em pipeline ou bufferizadas. Uma fonte de dados em pipeline emite registros durante a leitura de seus fluxos de entrada, enquanto uma fonte bufferizada primeiro precisa ler todos os registros de seus fluxos de entrada e só então poderá emitir o primeiro registro em sua saída.

Do ponto de vista da avaliação de desempenho, cada método de acesso possui dois atributos obrigatórios — cardinalidade (cardinality) e custo (cost). O primeiro reflete quantos registros serão selecionados da fonte de dados. O segundo estima o custo de execução do método de acesso. A magnitude do custo depende diretamente da cardinalidade e do mecanismo de seleção ou transformação do fluxo de dados. Nas versões atuais do servidor, o custo é determinado pela quantidade de leituras lógicas (page fetches) necessárias para retornar todos os registros pelo método de acesso. Assim, métodos mais "altos" sempre têm um custo maior do que os de baixo nível. A carga de CPU nos métodos de acesso computacionais é convertida para uma quantidade aproximadamente equivalente (de acordo com coeficientes dados) de leituras lógicas.

3.2. Métodos de Acesso Primários

Este grupo de métodos de acesso realiza a criação de um fluxo de dados com base em fontes de baixo nível, como tabelas (internas e externas) e procedimentos. A seguir, examinaremos cada uma das fontes de dados primárias separadamente.

3.2.1. Leitura de Tabela

É o método de acesso primário mais comum. Vale ressaltar que aqui estamos falando apenas de tabelas “internas”, ou seja, aquelas cujos dados estão localizados nos arquivos do banco de dados. O acesso a tabelas externas (external tables), bem como a seleção de procedimentos armazenados (stored procedures), é realizado por outros métodos e será descrito separadamente.

Varredura Completa (Full table scan)

Este método também é conhecido como varredura natural ou sequencial (Full Scan, Natural Scan, Sequential Scan).

Neste método de acesso, o servidor realiza a leitura sequencial de todas as páginas alocadas para a tabela, na ordem de sua localização física no disco. Obviamente, este método oferece o maior desempenho em termos de taxa de transferência, ou seja, a quantidade de registros buscados por unidade de tempo. Também é bastante óbvio que todos os registros desta tabela serão lidos, independentemente de serem necessários ou não.

Como o volume esperado de dados é bastante grande, durante a leitura das páginas da tabela do disco existe o problema das páginas lidas deslocarem outras, potencialmente necessárias para sessões concorrentes. Para isso, a lógica do cache de páginas é alterada — a página atual da varredura é posicionada na posição MRU (most recently used) durante a leitura de todos os registros dessa página. Assim que não há mais dados na página e a próxima precisa ser buscada, a página atual é liberada com o sinalizador LRU (least recently used), indo para o “final” da fila e sendo assim a primeira candidata à remoção do cache.

Os registros são lidos da tabela um por um, sendo imediatamente enviados para a saída. Deve-se notar que não há pré-busca de registros (pelo menos dentro de uma página) com seu buffer no servidor. Ou seja, uma seleção completa de uma tabela com 100K registros, ocupando 1000 páginas, resultará em 100K buscas de página (page fetch), e não em 1000, como se poderia supor. Também não há leitura em bloco (multi-block reads), na qual páginas adjacentes poderiam ser agrupadas e lidas do disco em “lotes” de várias de uma vez, reduzindo assim o número de operações de I/O

físico.

Ambas essas funcionalidades estão planejadas para implementação nas próximas versões do servidor.



A partir do Firebird 3.0, as páginas de dados (Data Pages ou DP) para tabelas (contendo mais de 8 DPs) são alocadas em grupos chamados **extents**. Um extent consiste em 8 páginas. Isso permite que o mecanismo use o pré-busca do SO de forma mais eficiente, pois as páginas pertencentes a uma mesma tabela estão localizadas próximas umas das outras e, portanto, a varredura completa de tabelas terá maior probabilidade de ler as páginas necessárias do cache do SO.

O otimizador, ao escolher este método de acesso, usa uma estratégia baseada em regras (rule based) — a varredura completa é realizada apenas na ausência de índices aplicáveis ao predicado da consulta. O custo deste método de acesso é igual ao número de registros na tabela (estimado aproximadamente pelo número de páginas da tabela, tamanho do registro e taxa média de compactação de registros nas páginas de dados). Teoricamente, isso está incorreto e a escolha do método de acesso deve sempre ser baseada no custo, mas na prática isso se mostra desnecessário na grande maioria dos casos. As razões para isso serão explicadas abaixo na descrição do acesso por índice.

Aqui e no futuro, ao mencionar o comportamento do otimizador, serão fornecidos: um exemplo de consulta SELECT, o plano de sua execução na forma Legacy e na forma Explain. No plano de execução Legacy, a varredura completa da tabela é indicada pela palavra “NATURAL”. Na forma Explain — Table "<nome da tabela>" Full Scan. Atualmente, os valores indicados entre colchetes (cardinalidade e custo) não são exibidos no plano explain, eles são fornecidos como um exemplo de cálculo.

Exemplo 2. Varredura completa da tabela RDB\$RELATIONS

```
SELECT *
FROM RDB$RELATIONS
```

```
PLAN (RDB$RELATIONS NATURAL)
```

```
[cardinality=120.0, cost=120.0]
Select Expression
  [cardinality=120.0, cost=120.0]
  -> Table "RDB$RELATIONS" Full Scan
```

Nas estatísticas de execução da consulta, os registros lidos por varredura completa são refletidos como leituras não indexadas (non indexed reads).

Ao usar apelidos (alias) para tabelas, o plano também exibirá os apelidos.

Exemplo 3. Varredura completa da tabela RDB\$RELATIONS com apelido

```
SELECT *
FROM RDB$RELATIONS R
```

```
PLAN (R NATURAL)
```

```
[cardinality=120.0, cost=120.0]
Select Expression
  [cardinality=120.0, cost=120.0]
    -> Table "RDB$RELATIONS" as "R" Full Scan
```

Acesso via identificador de registro

Neste caso, o subsistema de execução obtém o identificador (número físico) do registro que precisa ser lido. Este número de registro pode ser obtido de diferentes fontes, cada uma das quais será descrita a seguir.

De forma um tanto simplificada, o número físico do registro contém informações sobre a página na qual o registro está localizado e sobre o deslocamento dentro dessa página. Portanto, essa informação é suficiente para realizar o fetch da página necessária e encontrar o registro desejado nela.

Este método de acesso é de baixo nível e é usado apenas como implementação de recuperação baseada em mapa de bits (tanto para varredura de índice quanto para acesso via RDB\$DB_KEY) e navegação de índice. Esses métodos de acesso serão descritos com mais detalhes posteriormente.

O custo deste tipo de acesso é sempre igual a um. Leituras de registros são refletidas nas estatísticas como indexadas.

Acesso posicionado

Aqui estamos falando dos comandos posicionados UPDATE e DELETE (sintaxe WHERE CURRENT OF). Existe uma opinião equivocada de que este método de acesso é um análogo sintático da recuperação usando RDB\$DB_KEY, mas não é o caso. O acesso posicionado funciona apenas para um cursor ativo, ou seja, para um registro já buscado (usando comandos FOR SELECT ou FETCH). Caso contrário, o erro `isc_no_cur_rec` (“no current record for fetch operation”) será gerado. Portanto, esta é simplesmente uma forma de referenciar o registro ativo do cursor, não exigindo nenhuma operação de leitura. Enquanto a recuperação via RDB\$DB_KEY utiliza o acesso via identificador de registro e, consequentemente, sempre resulta no fetch de uma página.

3.2.2. Acesso por índice

A ideia do acesso por índice é simples – além da tabela de dados, temos uma estrutura que contém pares “chave - número do registro” de uma forma que permite uma busca rápida pelo valor da chave. No Firebird, um índice é uma árvore B+ paginada com compressão de prefixo de chaves.

Os índices podem ser simples (de segmento único) ou compostos (multisegmentados ou compostos). Deve-se notar que o conjunto de campos de um índice composto representa uma única chave. A busca no índice pode ser realizada tanto pela chave inteira quanto por sua substring (subchave). Obviamente, a busca por subchave é permitida apenas para a parte inicial da chave (por exemplo, STARTING WITH ou usando não todos os segmentos do composto). Se a busca é realizada por todos os segmentos do índice, isso é chamado de correspondência completa (full match) da chave, caso contrário, é uma correspondência parcial (partial match) da chave. Portanto, para um índice composto nos campos (A, B, C), segue-se que:

- ele pode ser usado para predicados ($A = 0$) ou ($A = 0$ and $B = 0$) ou ($A = 0$ and $B = 0$ and $C = 0$), mas não pode ser usado para predicados ($B = 0$) ou ($C = 0$) ou ($B = 0$ and $C = 0$);
- o predicado ($A = 0$ and $B > 0$ and $C = 0$) levará a uma correspondência parcial em dois segmentos, e o predicado ($A > 0$ and $B = 0$) – a uma correspondência parcial em apenas um segmento.

Obviamente, o acesso por índice exige que leiamos tanto as páginas do índice para busca quanto as páginas de dados para leitura dos registros. Outros SGBDs, em alguns casos, podem se limitar apenas às páginas de índice – por exemplo, se todos os campos da seleção estiverem incluídos no índice. Mas esse esquema é impossível no Firebird devido à sua arquitetura – a chave do índice contém apenas o número do registro sem informações sobre sua versão – portanto, o servidor deve sempre ler o próprio registro para determinar se alguma das versões com essa chave é visível para a transação atual. Muitas vezes surge a pergunta: e se incluirmos informações sobre versões (ou seja, números de transação) na chave do índice? Afinal, então seria possível implementar uma varredura puramente por índice (index only scan). Mas há dois pontos problemáticos aqui. Primeiro, o comprimento da chave aumentaria, portanto o índice ocuparia mais páginas, levando a mais I/O para a mesma operação de varredura. Segundo, cada alteração em um registro levaria à modificação da chave do índice, mesmo que os campos não chave fossem alterados. Enquanto atualmente o índice é modificado apenas quando os campos chave do registro são alterados. O primeiro problema levaria a uma degradação significativa no desempenho da varredura de índices com chaves curtas (com tipos INT, BIGINT e outros). O segundo – pelo menos triplica o número de páginas modificadas para cada comando de alteração de dados em comparação com a situação atual. Até agora, os desenvolvedores do servidor consideram isso um preço muito alto para a implementação da varredura puramente por índice.



Existe uma alternativa para a varredura puramente por índice implementada no Postgres. Ela funciona de forma eficaz para tabelas nas quais apenas uma pequena parte dos registros é alterada. Para isso, além do próprio índice, existe uma estrutura de disco adicional chamada mapa de visibilidade. O procedimento de varredura do índice, ao encontrar um registro potencialmente adequado no índice, verifica um bit no mapa de visibilidade para a página de dados correspondente. Se estiver definido, significa que esta linha está visível e os dados podem ser retornados imediatamente. Caso contrário, será necessário visitar o registro da linha e verificar se ele está visível, portanto não haverá ganho em comparação com a varredura de índice comum.

Esta variante pode ser implementada no Firebird. A partir do Firebird 3.0, para páginas DP (Data Page) existe a chamada flag swept. Ela é definida como 1 se o sweep ou o coletor de lixo visitou a página e não encontrou ou limpou todas as

versões não primárias dos registros. Esta mesma flag também está presente nas páginas de ponteiros PP (Pointer Page), que podem ser usadas como um análogo do mapa de visibilidade descrito acima.

Em comparação com outros SGBDs, os índices no Firebird têm mais uma característica – a varredura do índice é sempre unidirecional, das chaves menores para as maiores. Muitas vezes, por causa disso, o índice é chamado de unidirecional e diz-se que em seu nó há ponteiros apenas para o próximo nó e não há ponteiro para o nó anterior. Na verdade, o problema não é esse. Todas as estruturas de disco do servidor Firebird são projetadas para serem livres de deadlocks (deadlock free), sendo garantida a granularidade mínima possível de bloqueios. Além disso, também atua a regra de escrita "cuidadosa" (careful write) das páginas, que serve para recuperação instantânea após uma falha. O problema com índices bidirecionais é que eles violam essa regra ao dividir uma página. Até o momento, não se conhece uma forma de trabalhar sem bloqueios com ponteiros bidirecionais no caso em que uma das páginas deve ser escrita estritamente antes da outra.



Na verdade, existem alternativas para contornar este problema e elas estão atualmente sendo discutidas pelos desenvolvedores.

Esta característica leva à impossibilidade de usar um índice ASC para ordenação DESC ou cálculo de MAX e, inversamente, à impossibilidade de usar um índice DESC para ordenação ASC ou cálculo de MIN. Naturalmente, a unidirecionalidade não interfere de forma alguma na varredura do índice para busca.

Seletividade do índice

O principal parâmetro que afeta a otimização do acesso por índice é a **seletividade** do índice. É um valor inverso ao número de valores únicos da chave. Nos cálculos de cardinalidade e custo, assume-se uma distribuição uniforme dos valores da chave no índice.

A seletividade do índice pode ser conhecida usando a seguinte consulta

```
SELECT RDB$STATISTICS
FROM RDB$INDICES
WHERE RDB$INDEX_NAME = '<index_name>'
```

Exemplo 4. Obtendo estatísticas armazenadas para o índice CUSTNAMEX

```
SELECT RDB$STATISTICS
FROM RDB$INDICES
WHERE RDB$INDEX_NAME = 'CUSTNAMEX'
```

```
RDB$STATISTICS
=====
0.06666667014360428
```

Para índices compostos, além da seletividade do índice, o Firebird também armazena a seletividade

do conjunto de seus segmentos, desde o primeiro até o atual. A seletividade do conjunto de segmentos pode ser conhecida usando uma consulta à tabela RDB\$INDEX_SEGMENTS.

Exemplo 5. Obtendo estatísticas armazenadas para cada segmento do índice NAMEX

```
SELECT RDB$FIELD_NAME, RDB$FIELD_POSITION, RDB$STATISTICS
FROM RDB$INDEX_SEGMENTS
WHERE RDB$INDEX_NAME = 'NAMEX';
```

RDB\$FIELD_NAME	RDB\$FIELD_POSITION	RDB\$STATISTICS
=====	=====	=====
LAST_NAME	0	0.02500000037252903
FIRST_NAME	1	0.02380952425301075

Exemplo 6. Obtenção de estatísticas armazenadas para o índice NAMEX

```
SELECT RDB$STATISTICS
FROM RDB$INDICES
WHERE RDB$INDEX_NAME = 'NAMEX'
```

RDB\$STATISTICS
=====
0.02380952425301075

A seletividade do índice é calculada durante sua construção (CREATE INDEX, ALTER INDEX ... ACTIVE) e posteriormente pode se tornar desatualizada. Para atualizar as estatísticas do índice, utiliza-se a seguinte consulta SQL

```
SET STATISTICS INDEX <index_name>;
```

Exemplo 7. Coleta de estatísticas para o índice NAMEX

```
SET STATISTICS INDEX NAMEX;
```

Atualmente, o Firebird não atualiza as estatísticas dos índices automaticamente. Além disso, as estatísticas armazenadas contêm muito poucas informações para otimizar o acesso usando o índice. Na verdade, apenas a seletividade do índice e de seus segmentos são armazenadas. Mas características importantes do índice não são armazenadas, como a seletividade do valor NULL, a profundidade do índice, o comprimento médio da chave do índice, histogramas de distribuição de valores, o grau de clusterização das chaves do índice.



As estatísticas armazenadas serão expandidas nas próximas versões do Firebird. Além disso, estão planejadas algumas capacidades para sua atualização automática.

Além dos índices comuns e compostos, no Firebird é possível criar índices por expressão. Neste caso, em vez dos valores dos campos da tabela, são usados como chaves do índice os valores de uma expressão que retorna um valor escalar. Para que o otimizador use tais índices, a condição de filtragem da consulta deve usar exatamente a mesma expressão especificada na criação do índice. Índices por expressão não podem ser compostos, mas a expressão pode conter vários campos da tabela. A seletividade de tal índice é calculada da mesma forma que para índices comuns.

Índices parciais

A partir do Firebird 5.0, ao criar um índice, tornou-se possível especificar uma cláusula WHERE opcional, que define uma condição de busca que limita o subconjunto de registros da tabela a serem indexados. Tais índices são chamados de índices parciais. A condição de busca deve conter uma ou mais colunas da tabela.

O otimizador pode usar um índice parcial apenas nos seguintes casos:

- a condição WHERE inclui exatamente a mesma expressão lógica definida para o índice;
- a condição de busca definida para o índice contém expressões lógicas unidas por OR, e uma delas está explicitamente incluída na condição WHERE;
- a condição de busca definida para o índice especifica IS NOT NULL, e a condição WHERE inclui uma expressão para o mesmo campo que, sabe-se, ignora NULL.

O último ponto será demonstrado com um exemplo. Suponha que você criou o seguinte índice parcial:

```
CREATE INDEX IDX_HORSE_DEATHDATE
ON HORSE(DEATHDATE) WHERE DEATHDATE IS NOT NULL;
```

Agora o otimizador pode usar este índice não apenas para o predicado IS NOT NULL, mas também para predicados =, <, >, BETWEEN e outros, se eles não retornarem TRUE ao comparar com NULL. Para os predicados IS NULL e IS NOT DISTINCT FROM, o índice não pode ser utilizado.

```
-- o índice parcial será usado
SELECT *
FROM HORSE
WHERE DEATHDATE IS NOT NULL

-- o índice parcial será usado
SELECT *
FROM HORSE
WHERE DEATHDATE = ?

-- o índice parcial será usado
SELECT *
FROM HORSE
```



```
WHERE DEATHDATE BETWEEN ? AND ?
```

```
-- o índice parcial não pode ser usado
```

```
SELECT *
```

```
FROM HORSE
```

```
WHERE DEATHDATE IS NOT DISTINCT FROM ?
```

Se existir um índice comum e um índice parcial para o mesmo conjunto de campos, o otimizador na maioria dos casos escolherá o índice comum, mesmo que a condição WHERE inclua a mesma expressão definida no índice parcial. A razão para este comportamento é que a seletividade do índice comum é conhecida “exatamente” (calculada como o inverso do número de chaves únicas). Mas o índice parcial contém menos chaves devido à presença de uma condição de filtragem adicional, portanto a seletividade armazenada do índice será pior. A seletividade estimada do índice parcial é avaliada como a seletividade armazenada multiplicada pela proporção de registros incluídos no índice. Esta proporção atualmente não é armazenada nas estatísticas (“exatamente” não é conhecida), mas é estimada com base na seletividade da expressão de filtragem especificada na criação do índice (veja [Verificação de predicados](#)). Ao mesmo tempo, a seletividade real pode ser menor e, portanto, o índice comum (com um valor de seletividade calculado exatamente) pode vencer.



Isto pode ser alterado nas próximas versões do servidor.

Mapas de bits

O principal problema padrão do acesso por índice é a entrada/saída aleatória em relação às páginas de dados. De fato, a ordem das chaves no índice raramente coincide com a ordem dos registros correspondentes na tabela. Assim, a seleção de um número significativo de registros através de um índice provavelmente levará à busca múltipla de cada página de dados correspondente. Este problema é resolvido em outros SGBDs com índices clusterizados (termo do MSSQL) ou tabelas ordenadas por índice (termo do Oracle), nas quais os dados estão diretamente no índice em ordem crescente da chave de cluster.

No Firebird, este problema é resolvido de outra maneira. A varredura do índice não é uma operação em pipeline, mas é executada para toda a faixa de busca, incluindo a obtenção dos números de registro em um mapa de bits especial. Este mapa é uma matriz esparsa de bits, onde cada bit corresponde a um registro específico e a presença de um 1 nele é uma indicação para selecionar esse registro. A peculiaridade desta solução é que o mapa de bits, por definição, é ordenado pelos números de registro. Após o término da varredura, esta matriz serve como base para o acesso sequencial através do identificador do registro. A vantagem é óbvia — a leitura da tabela ocorre na ordem física das páginas, como na varredura completa, ou seja, cada página será lida no máximo uma vez. Assim, a simplicidade de implementação do índice aqui se combina com o acesso mais eficiente aos registros. O custo — um certo aumento no tempo de resposta para consultas com a estratégia FIRST ROWS, quando é necessário obter os primeiros registros muito rapidamente.

Operações de interseção (bit and) e união (bit or) são permitidas em mapas de bits, portanto o servidor pode usar vários índices para uma tabela.

Varredura de intervalo

A busca por índice é realizada usando um limite superior e inferior. Ou seja, se um limite inferior de varredura for especificado, primeiro a chave correspondente a ele é encontrada e só então começa a iteração sequencial das chaves, inserindo os números de registro no mapa de bits. Se um limite superior for especificado, cada chave é comparada com ele e, ao alcançá-lo, a varredura é interrompida. Este mecanismo é chamado de varredura de intervalo (range scan). No caso em que a chave do limite inferior é igual à chave do limite superior, fala-se em varredura por igualdade (equality scan). Se a varredura por igualdade for executada para um índice único, isso é chamado de varredura única (unique scan). Este tipo de varredura tem significado especial, pois pode retornar no máximo um registro e, portanto, por definição, é o mais barato. Se nenhum dos limites for especificado, temos uma varredura completa (full scan). Este método é usado exclusivamente para a navegação por índice, descrita abaixo.

Quando possível, o servidor ignora valores NULL durante a varredura de intervalo, se permitido. Na maioria dos casos, isso leva a um aumento de desempenho devido ao menor tamanho do mapa de bits e, conseqüentemente, a um menor número de leituras indexadas. Esta opção não é usada apenas para predicados indexados do tipo IS NULL e IS NOT DISTINCT FROM, que consideram valores NULL.

Ao selecionar índices para varredura, o otimizador usa uma estratégia baseada em custo (cost based). O custo da varredura de intervalo é estimado com base na seletividade do índice, no número de registros na tabela, no número médio de chaves por página de índice e na altura da árvore B+.



A altura da árvore B+ atualmente não é armazenada nas estatísticas, sendo definida no código como uma constante igual a 3.

No plano de execução Legacy, a varredura de intervalo de índice é denotada pela palavra “INDEX”, seguida entre parênteses pelos nomes de todos os índices que formam o mapa de bits final.

No plano Explain, a exibição da varredura de intervalo é um pouco mais complexa. Na forma geral, ela se parece com o seguinte

```
Table "<table_name>" Access By ID
-> Bitmap
   -> Index "<index_name>" <scan_type>
```

Onde index_name — nome do índice usado, scan_type — tipo de varredura, table_name — nome da tabela.

Unique scan

A varredura única pode ser usada apenas para índices únicos ao varrer todos os segmentos (correspondência completa) e usando apenas predicados de igualdade "=". Índices únicos são criados automaticamente para restrições de chave primária ou restrições de unicidade, mas também podem ser criados independentemente. Este tipo de varredura tem um significado especial, pois pode retornar no máximo um registro e, portanto, sua cardinalidade é sempre igual a 1. O custo deste é calculado pela fórmula

```
cost = indexDepth + 1
```

Exemplo 8. Index Unique Scan

```
SELECT *
FROM RDB$RELATIONS
WHERE RDB$RELATION_NAME = ?
```

```
PLAN (RDB$RELATIONS INDEX (RDB$INDEX_0))
```

No plano Explain, a varredura única é exibida da seguinte forma:

```
[cardinality=1.0, cost=4.000]
Select Expression
  [cardinality=1.0, cost=4.000]
  -> Filter
    [cardinality=1.0, cost=4.000]
    -> Table "RDB$RELATIONS" Access By ID
      -> Bitmap
        -> Index "RDB$INDEX_0" Unique Scan
```

Varredura por igualdade

A varredura por igualdade é usada para predicados IS NULL, IS NOT DISTINCT FROM e igualdade (=). Se todos os segmentos do índice forem usados na varredura por igualdade, o plano Explain indicará (full match) para essa varredura; caso contrário, indicará (partial match) e o número de segmentos usados.

Neste caso, a cardinalidade é calculada pela fórmula:

```
cardinality = table_cardinality * index_selectivity
```

Aqui, index_selectivity é a seletividade do conjunto de segmentos do índice usados na varredura por igualdade.

O custo da varredura de índice por igualdade é calculado de forma um pouco mais complexa:

```
cost = indexDepth + MAX(avgKeyLength * table_cardinality * index_selectivity / (page_size - BTR_SIZE), 1)
```

Onde avgKeyLength é o comprimento médio da chave, page_size é o tamanho da página, BTR_SIZE é o tamanho do cabeçalho da página de índice, atualmente 39 bytes.

Atualmente, o comprimento médio da chave não é armazenado nas estatísticas do índice, mas é calculado com base no grau médio de compactação e no comprimento da chave usando a fórmula:

```
avgKeyLength = 2 + length * factor
```

Aqui, length é o comprimento da chave do índice, factor é o grau de compactação.

O grau de compactação é considerado 0.5 para índices simples e 0.7 para índices compostos.

Exemplo 9. Index Range Scan (full match)

```
SELECT *
FROM RDB$RELATIONS
WHERE RDB$RELATION_ID = ?
```

```
PLAN (RDB$RELATIONS INDEX (RDB$INDEX_1))
```

```
[cardinality=1.02, cost=4.020]
Select Expression
  [cardinality=1.02, cost=4.020]
  -> Filter
    [cardinality=1.02, cost=4.020]
    -> Table "RDB$RELATIONS" Access By ID
      -> Bitmap
        -> Index "RDB$INDEX_1" Range Scan (full match)
```

Exemplo de correspondência completa para um índice composto:

Exemplo 10. Index Range Scan (full match) para índice composto

```
SELECT *
FROM EMPLOYEE
WHERE FIRST_NAME = ? AND LAST_NAME = ?
```

```
PLAN (EMPLOYEE INDEX (NAMEX))
```

```
[cardinality=1.0, cost=4.000]
Select Expression
  [cardinality=1.0, cost=4.000]
  -> Filter
    [cardinality=1.0, cost=4.000]
    -> Table "EMPLOYEE" Access By ID
      -> Bitmap
        -> Index "NAMEX" Range Scan (full match)
```

Agora, vejamos um exemplo em que apenas o primeiro dos dois segmentos é usado para um índice

composto.

Exemplo 11. Index Range scan (partial match)

```
SELECT *
FROM EMPLOYEE
WHERE LAST_NAME = ?
```

```
PLAN (EMPLOYEE INDEX (NAMEX))
```

```
[cardinality=1.05, cost=4.050]
Select Expression
  [cardinality=1.05, cost=4.050]
    -> Filter
      [cardinality=1.05, cost=4.050]
        -> Table "EMPLOYEE" Access By ID
          -> Bitmap
            -> Index "NAMEX" Range Scan (partial match: 1/2)
```

Range Scan com limites

Se os limites superior e inferior da varredura não coincidirem ou apenas um deles for usado, o plano Explain mostrará quais limites de varredura são usados: "lower bound" – limite inferior, "upper bound" – limite superior. Para cada limite, é indicado quantos segmentos do índice são usados. Por exemplo, (lower bound: 1/2) significa que um segmento de dois será usado para o limite inferior.

O custo e a cardinalidade da varredura de intervalo são calculados exatamente da mesma forma que para a varredura por igualdade, mas em vez da seletividade do índice, é usada a seletividade do predicado, que é calculada da seguinte maneira.

Tabela 5. Seletividade dos predicados de varredura de índice

Predicado	factor	Seletividade
<, <=, >, >=	0.05	$\text{indexSelectivity} + (1 - \text{indexSelectivity}) * \text{factor}$
BETWEEN	0.0025	$\text{indexSelectivity} + (1 - \text{indexSelectivity}) * \text{factor}$
STARTING WITH	0.01	$\text{indexSelectivity} + (1 - \text{indexSelectivity}) * \text{factor}$
IN		$\text{indexSelectivity} * N$
Outros	0.01	$\text{indexSelectivity} + (1 - \text{indexSelectivity}) * \text{factor}$

Exemplo 12. Index Range Scan com limite inferior

```
SELECT *
FROM EMPLOYEE
WHERE EMP_NO > 0
```

```
PLAN (EMPLOYEE INDEX (RDB$PRIMARY7))
```

```
[cardinality=6.09, cost=9.090]
Select Expression
  [cardinality=6.09, cost=9.090]
  -> Filter
    [cardinality=6.09, cost=9.090]
    -> Table "EMPLOYEE" Access By ID
      -> Bitmap
        -> Index "RDB$PRIMARY7" Range Scan (lower bound: 1/1)
```

Exemplo 13. Index Range Scan com limite superior

```
SELECT *
FROM EMPLOYEE
WHERE EMP_NO < 0
```

```
PLAN (EMPLOYEE INDEX (RDB$PRIMARY7))
```

```
[cardinality=6.09, cost=9.090]
Select Expression
  [cardinality=6.09, cost=9.090]
  -> Filter
    [cardinality=6.09, cost=9.090]
    -> Table "EMPLOYEE" Access By ID
      -> Bitmap
        -> Index "RDB$PRIMARY7" Range Scan (upper bound: 1/1)
```

Exemplo 14. Index Range Scan com limites inferior e superior

```
SELECT *
FROM EMPLOYEE
WHERE EMP_NO BETWEEN ? AND ?
```

```
PLAN (EMPLOYEE INDEX (RDB$PRIMARY7))
```

```
[cardinality=4.295, cost=7.295]
Select Expression
  [cardinality=4.295, cost=7.295]
  -> Filter
    [cardinality=4.295, cost=7.295]
    -> Table "EMPLOYEE" Access By ID
```

```
-> Bitmap
    -> Index "RDB$PRIMARY7" Range Scan (lower bound: 1/1, upper bound: 1/1)
```

Agora, vejamos exemplos para índices compostos, nos quais apenas parte dos segmentos é usada para os limites.

Exemplo 15. Range Scan de índice composto (limite inferior usa o primeiro de dois segmentos)

```
SELECT *
FROM EMPLOYEE
WHERE LAST_NAME > ?
```

```
PLAN (EMPLOYEE INDEX (NAMEX))
```

```
[cardinality=3.09, cost=6.090]
Select Expression
  [cardinality=3.09, cost=6.090]
  -> Filter
    [cardinality=3.09, cost=6.090]
    -> Table "EMPLOYEE" Access By ID
      -> Bitmap
        -> Index "NAMEX" Range Scan (lower bound: 1/2)
```

Exemplo 16. Range Scan de índice composto. O limite inferior usa ambos os segmentos, e o superior usa apenas o primeiro

```
SELECT *
FROM EMPLOYEE
WHERE LAST_NAME = ? AND FIRST_NAME > ?
```

```
PLAN (EMPLOYEE INDEX (NAMEX))
```

```
[cardinality=1.03, cost=4.030]
Select Expression
  [cardinality=1.03, cost=4.030]
  -> Filter
    [cardinality=1.03, cost=4.030]
    -> Table "EMPLOYEE" Access By ID
      -> Bitmap
        -> Index "NAMEX" Range Scan (lower bound: 2/2, upper bound: 1/2)
```

Vamos tentar o contrário:

```
SELECT *
FROM EMPLOYEE
WHERE LAST_NAME > ? AND FIRST_NAME = ?
```

```
PLAN (EMPLOYEE INDEX (NAMEX))
```

```
[cardinality=1.0, cost=6.097]
Select Expression
  [cardinality=1.0, cost=6.097]
    -> Filter
      [cardinality=3.097, cost=6.097]
        -> Table "EMPLOYEE" Access By ID
          -> Bitmap
            -> Index "NAMEX" Range Scan (lower bound: 1/2)
```

Como você pode ver, apenas o limite inferior do primeiro segmento é usado, e o segundo segmento não é usado de forma alguma.

Agora, vamos tentar usar apenas o segundo segmento.

```
SELECT *
FROM EMPLOYEE
WHERE FIRST_NAME = ?
```

```
PLAN (EMPLOYEE NATURAL)
```

```
[cardinality=4.2, cost=42.000]
Select Expression
  [cardinality=4.2, cost=42.000]
    -> Filter
      [cardinality=42.0, cost=42.000]
        -> Table "EMPLOYEE" Full Scan
```

Neste caso, não foi possível usar a varredura de índice.



Em alguns SGBDs (em particular, no Oracle), existe a chamada Index Skip Scan, na qual toda a chave do índice composto não é comparada, mas apenas a parte usada da chave é comparada; neste caso, o último exemplo poderia usar a varredura de índice. Atualmente, este método de acesso não existe no Firebird.

List scan

A varredura por lista (List scan) está disponível a partir do Firebird 5.0. É aplicável apenas para o predicado IN com uma lista de valores. Neste caso, um mapa de bits comum é formado para toda a lista de valores. A busca pode ser realizada por varredura vertical (a partir da raiz para cada chave) ou por varredura horizontal do intervalo entre as chaves min/max. O otimizador decide exatamente como a varredura será realizada, dependendo do que é mais barato em termos de custo.

Antes do Firebird 5.0, tal predicado era transformado em um conjunto de condições de igualdade unidas pelo predicado OR.

Ou seja,

$$F \text{ IN } (V1, V2, \dots VN)$$

era transformado em

$$(F = V1) \text{ OR } (F = V2) \text{ OR } \dots (F = VN)$$

No Firebird 5.0, isso funciona de forma diferente. Listas de constantes em IN são pré-avaliadas como invariantes e armazenadas em cache como uma árvore de busca binária, o que acelera a comparação se a condição precisar ser verificada para muitos registros ou se a lista de valores for longa. Se um índice for aplicável ao predicado, então a varredura de índice por lista (List scan) é usada.

A cardinalidade deste método de acesso é calculada pela fórmula:

$$\text{cardinality} = \text{table_cardinality} * \text{MAX}(\text{index_selectivity} * N, 1)$$

Onde N é o número de elementos na lista IN.

O custo da varredura de índice por lista é calculado assim:

$$\text{cost} = \text{indexDepth} + \text{MAX}(\text{avgKeyLength} * \text{table_cardinality} * \text{index_selectivity} * N, 1)$$
Exemplo 17. List scan

```
SELECT *
FROM RDB$RELATIONS
WHERE RDB$RELATION_NAME IN ('RDB$RELATIONS', 'RDB$PROCEDURES', 'RDB$FUNCTIONS')
```

```
PLAN (RDB$RELATIONS INDEX (RDB$INDEX_0))
```



```
[cardinality=3.2, cost=6.200]
Select Expression
  [cardinality=3.2, cost=6.200]
    -> Filter
      [cardinality=3.2, cost=6.200]
        -> Table "RDB$RELATIONS" Access By ID
          -> Bitmap
            -> Index "RDB$INDEX_0" List Scan (full match)
```

Interseção e união de mapas de bits

Como mencionado acima, as operações de interseção (bit and) e união (bit or) são permitidas em mapas de bits, permitindo assim que o servidor use vários índices para uma única tabela. Na interseção de mapas de bits, a cardinalidade resultante não aumenta. Por exemplo, para a expressão $F = 0 \text{ and } ? = 0$ sua segunda parte não é indexável e, portanto, é verificada após a seleção por índice, não influenciando o resultado final. Mas a união de mapas de bits leva a um aumento na cardinalidade resultante, portanto, apenas partes de um predicado totalmente indexado podem ser unidas. Ou seja, ao mover a segunda parte da expressão $F = 0 \text{ or } ? = 0$ para um nível superior, pode-se descobrir que era necessário varrer todos os registros. Portanto, o índice para o campo F não será usado em tal expressão.

A seletividade da interseção de dois mapas de bits é calculada pela fórmula:

$$(\text{bestSel} + (\text{worstSel} - \text{bestSel}) / (1 - \text{bestSel}) * \text{bestSel}) / 2$$

A seletividade da união de dois mapas de bits é calculada pela fórmula:

$$\text{bestSel} + \text{worstSel}$$

Como o custo de acesso via identificador de registro é igual a um, o custo final de acesso via mapa de bits será igual ao custo total da busca por índice (para todos os índices que formam o mapa de bits) mais a cardinalidade resultante do mapa de bits.

Vejamos exemplos de uso de vários índices.

Exemplo 18. Interseção de mapas de bits

```
SELECT *
FROM RDB$INDICES
WHERE RDB$RELATION_NAME = ? AND RDB$FOREIGN_KEY = ?
```

```
PLAN (RDB$INDICES INDEX (RDB$INDEX_31, RDB$INDEX_41))
```

```
[cardinality=1.0, cost=7.000]
```

```

Select Expression
  [cardinality=1.0, cost=7.000]
  -> Filter
    [cardinality=1.0, cost=7.000]
    -> Table "RDB$INDICES" Access By ID
      -> Bitmap And
        -> Bitmap
          -> Index "RDB$INDEX_31" Range Scan (full match)
        -> Bitmap
          -> Index "RDB$INDEX_41" Range Scan (full match)

```

Exemplo 19. União de mapas de bits

```

SELECT *
FROM RDB$INDICES
WHERE RDB$RELATION_NAME = ? OR RDB$FOREIGN_KEY = ?

```

```

PLAN (RDB$INDICES INDEX (RDB$INDEX_31, RDB$INDEX_41))

```

```

[cardinality=10.85, cost=16.850]
Select Expression
  [cardinality=10.85, cost=16.850]
  -> Filter
    [cardinality=10.85, cost=16.850]
    -> Table "RDB$INDICES" Access By ID
      -> Bitmap Or
        -> Bitmap
          -> Index "RDB$INDEX_31" Range Scan (full match)
        -> Bitmap
          -> Index "RDB$INDEX_41" Range Scan (full match)

```

Navegação por índice

A navegação por índice nada mais é do que a varredura sequencial das chaves do índice. E como para cada registro o servidor precisa realizar o fetch do registro e verificar sua visibilidade para nossa transação, esta operação acaba sendo bastante cara. É por isso que este método de acesso não é usado para seleções comuns (diferente do Oracle, por exemplo), sendo utilizado apenas em casos onde é justificado.



A ordenação usando vários índices não é suportada.

Atualmente, existem dois desses casos. O primeiro é o cálculo das funções agregadas MIN/MAX. Obviamente, para calcular MIN basta pegar a primeira chave em um índice ASC, e para calcular MAX - a primeira chave em um índice DESC. Se após o fetch do registro descobrir-se que ele não é visível para nós, então pegamos a próxima chave, e assim por diante. O segundo é a ordenação ou agrupamento de registros. Isto é indicado pelas cláusulas ORDER BY ou GROUP BY na consulta do

usuário. Nesta situação, simplesmente percorremos o índice, selecionando os registros à medida que varremos. Em ambos os casos, o fetch dos registros é realizado com base no acesso via seu identificador.



Sobre a unidirecionalidade da navegação por índice está escrito em [Acesso por índice](#).

Existem várias características que otimizam este processo. Primeiro, na presença de predicados restritivos no campo de ordenação, eles criam limites superior e/ou inferior para a varredura. Ou seja, no caso da consulta (WHERE A > 0 ORDER BY A) será realizada uma varredura parcial do índice em vez de uma completa. Com isso, reduzimos os custos da própria varredura. Segundo, na presença de outros predicados restritivos (não no campo de ordenação), porém otimizados via índice, um modo complexo de operação é ativado, onde a varredura do índice é combinada com o uso de um mapa de bits. Vejamos como isso funciona. Suponha que temos uma consulta da forma (WHERE A > 0 AND B > 0 ORDER BY A). Neste caso, primeiro é realizada a varredura do intervalo para o índice no campo B e um mapa de bits é composto. Em seguida, o valor 0 é estabelecido como limite inferior da varredura do índice no campo A. Então varremos este índice a partir do limite inferior e, para cada número de registro extraído do índice, verificamos sua pertinência ao mapa de bits. E somente em caso de pertinência realizamos o fetch do registro. Com isso, reduzimos os custos de fetch das páginas de dados.

A principal diferença entre a navegação por índice e a varredura (descrita em [Varredura de intervalo](#)) é a ausência de um mapa de bits entre a varredura do índice e o acesso ao registro via seu identificador. A razão é clara - ordenar os registros por números físicos neste caso é contraindicado. Disto pode-se concluir que o índice é varrido conforme o fetch do cliente, e não “de uma vez” (como no caso do uso de mapa de bits).

É bastante difícil estimar o custo da navegação. Para calcular MIN/MAX na grande maioria dos casos, será igual à altura da árvore B+ (busca da primeira chave) mais um (fetch da página). O custo de tal acesso é considerado uma quantidade insignificante, pois na prática o cálculo descrito de MIN/MAX sempre será mais rápido que alternativas. Para estimar o custo da navegação por índice, é necessário considerar tanto a quantidade e a largura média das chaves do índice, quanto a cardinalidade do mapa de bits (se houver), e também ter uma ideia do fator de clusterização (clustering factor) do índice - o coeficiente de correspondência entre a disposição das chaves e os números físicos dos registros. Atualmente, o servidor não calcula o custo da navegação, ou seja, novamente é usada uma estratégia baseada em regras (rule based). No futuro, planeja-se usar esta abordagem apenas para MIN/MAX e FIRST, e em outros casos basear-se no custo.



O fator de clusterização (clustering factor) do índice não é armazenado nas estatísticas do índice, mas você pode visualizá-lo usando a ferramenta gstat.

No Firebird 6.0, foi adicionada a estimativa de custo para escolha entre navegação por índice e ordenação externa.

No plano Legacy, a navegação por índice é denotada pela palavra “ORDER”, seguida pelo nome do índice (sem parênteses, pois tal índice pode ser apenas um). O servidor também informa os índices que formam o mapa de bits usado para filtragem. Neste caso, o plano de execução contém ambas as palavras: primeiro “ORDER”, depois “INDEX”.

Exemplo 20. Navegação por índice com varredura completa do índice

```
SELECT RDB$RELATION_NAME
FROM RDB$RELATIONS
ORDER BY RDB$RELATION_NAME
```

```
PLAN (RDB$RELATIONS ORDER RDB$INDEX_0)
```

```
[cardinality=265.0, cost=302.000]
Select Expression
  [cardinality=265.0, cost=302.000]
  -> Table "RDB$RELATIONS" Access By ID
    -> Index "RDB$INDEX_0" Full Scan
```

Exemplo 21. Navegação por índice com predicado criando limite inferior de varredura

```
SELECT MIN(RDB$RELATION_NAME)
FROM RDB$RELATIONS
WHERE RDB$RELATION_NAME > ?
```

```
PLAN (RDB$RELATIONS ORDER RDB$INDEX_0)
```

```
[cardinality=1.0, cost=26.785]
Select Expression
  [cardinality=1.0, cost=26.785]
  -> Aggregate
    [cardinality=18.785, cost=26.785]
    -> Filter
      [cardinality=18.785, cost=26.785]
      -> Table "RDB$RELATIONS" Access By ID
        -> Index "RDB$INDEX_0" Range Scan (lower bound: 1/1)
```

Exemplo 22. Navegação por índice junto com mapa de bits

```
SELECT MIN(RDB$RELATION_NAME)
FROM RDB$RELATIONS
WHERE RDB$RELATION_ID > ?
```

```
PLAN (RDB$RELATIONS ORDER RDB$INDEX_0 INDEX (RDB$INDEX_1))
```

```

[cardinality=1.0, cost=159.000]
Select Expression
  [cardinality=1.0, cost=159.000]
    -> Aggregate
      [cardinality=125.0, cost=159.000]
        -> Filter
          [cardinality=125.0, cost=159.000]
            -> Table "RDB$RELATIONS" Access By ID
              -> Index "RDB$INDEX_0" Full Scan
                -> Bitmap
                  -> Index "RDB$INDEX_1" Range Scan (lower bound: 1/1)

```

3.2.3. Acesso via RDB\$DB_KEY

O mecanismo de acesso via RDB\$DB_KEY é usado principalmente pelo servidor para fins internos. No entanto, ele também pode ser utilizado em consultas do usuário.

Do ponto de vista dos métodos de acesso, é tudo muito simples — um mapa de bits é criado e um único número de registro é colocado nele — o valor de RDB\$DB_KEY. Em seguida, a leitura do registro é realizada através do acesso via identificador de registro descrito acima. Obviamente, o custo desse acesso é igual a um. Ou seja, obtemos algo como um acesso pseudo-indexado, o que é refletido no plano de execução da consulta.

```

SELECT *
FROM RDB$RELATIONS
WHERE RDB$DB_KEY = ?

```

```

PLAN (RDB$RELATIONS INDEX ())

```

```

[cardinality=1.0, cost=1.0]
Select Expression
  [cardinality=1.0, cost=1.0]
    -> Filter
      [cardinality=1.0, cost=1.0]
        -> Table "RDB$RELATIONS" Access By ID
          -> DBKEY

```

As leituras via RDB\$DB_KEY são refletidas nas estatísticas como indexadas. A razão para isso é clara a partir da descrição acima — tudo o que é lido por meio do acesso via identificador de registro é considerado pelo servidor como acesso indexado. Uma suposição não totalmente correta, mas bastante inofensiva.

3.2.4. Tabela externa (External table scan)

Este tipo de acesso é usado apenas ao trabalhar com tabelas externas. Em essência, ele representa um análogo da varredura completa de uma tabela. Os registros são lidos do arquivo externo um por um, por meio do ponteiro (deslocamento) atual. O cache de páginas não é usado. A indexação de tabelas externas não é suportada.

Devido à ausência de métodos alternativos de acesso aos dados externos, o custo não é calculado nem utilizado. A cardinalidade da seleção de uma tabela externa é estimada como $\text{File_size} / \text{Record_size}$.

No plano Legacy, a leitura de uma tabela externa é exibida pela palavra “NATURAL”.

No plano Explain, a leitura de uma tabela externa é exibida como “Table Full Scan”.

Exemplo 23. Leitura de tabela externa

```
SELECT *
FROM EXT_TABLE
```

```
PLAN (EXT_TABLE NATURAL)
```

```
[cardinality=1000.0, cost=???]
Select Expression
  [cardinality=1000.0, cost=???]
  -> Table "EXT_TABLE" Full Scan
```

3.2.5. Tabela virtual (Virtual table scan)

Este método de acesso é usado para ler dados de tabelas virtuais (tabelas de monitoramento MON\$, tabelas virtuais de segurança SEC\$, bem como a tabela virtual RDB\$CONFIG). Os dados dessas tabelas são gerados dinamicamente e armazenados em cache na memória. Por exemplo, todas as tabelas de monitoramento são preenchidas na primeira consulta a qualquer uma delas, e seus dados são preservados até o final da transação. A tabela virtual RDB\$CONFIG também é preenchida na primeira consulta, e seus dados são preservados até o final da sessão.

Devido à ausência de métodos alternativos de acesso às tabelas virtuais, o custo não é calculado nem utilizado. A cardinalidade das tabelas virtuais é assumida como 1000.

No plano Legacy, a leitura de uma tabela virtual é exibida pela palavra “NATURAL”.

No plano Explain, a leitura de uma tabela virtual é exibida como “Table Full Scan”.

Exemplo 24. Leitura da tabela virtual MON\$ATTACHMENT

```
SELECT *
```

```
FROM MON$ATTACHMENTS
```

```
PLAN (MON$ATTACHMENTS NATURAL)
```

```
[cardinality=1000.0, cost=???]
Select Expression
  [cardinality=1000.0, cost=???]
  -> Table "MON$ATTACHMENTS" Full Scan
```

3.2.6. Tabela temporária local (Local table)

Este método de acesso está disponível a partir do Firebird 5.0. Ele é usado para retornar registros inseridos, modificados ou excluídos por instruções DML que contenham a cláusula RETURNING e retornem um cursor. Tais instruções incluem INSERT ... SELECT ... RETURNING, UPDATE ... RETURNING, DELETE ... RETURNING, MERGE ... RETURNING. A instrução INSERT ... VALUES .. RETURNING, que pode inserir e retornar apenas um registro, não se enquadra nessa categoria.

Tabelas temporárias locais são criadas "dinamicamente" com as colunas listadas na cláusula RETURNING e armazenam seus dados e estrutura até que a seleção de linhas da instrução DML seja concluída.



Em versões futuras do Firebird, está planejado adicionar a capacidade de declarar e preencher tabelas temporárias locais na seção de declaração de variáveis locais de módulos PSQL, como DECLARE LOCAL TABLE.

Devido à ausência de métodos alternativos de acesso às tabelas locais, o custo não é calculado nem utilizado. A cardinalidade das tabelas temporárias locais é assumida como 1000.

Nos planos Legacy e Explain, a leitura de uma tabela temporária local é exibida separadamente do plano principal (como para subconsultas), imediatamente após ele. No plano Legacy, a leitura da tabela local é exibida pela palavra "NATURAL", e a tabela tem o nome Local_Table. No plano Explain, a leitura da tabela temporária local é exibida como "Local Table Full Scan".

Exemplo 25. Usando uma tabela temporária local para uma instrução DML com a cláusula RETURNING.

```
UPDATE COLOR
SET NAME = ?
WHERE NAME = ?
RETURNING CODE_COLOR, NAME, OLD.NAME AS OLD_NAME
```

```
PLAN (COLOR INDEX (UNQ_COLOR_NAME))
PLAN (Local_Table NATURAL)
```

```
[cardinality=1.0, cost=4.0]
```

```

Select Expression
  [cardinality=1.0, cost=4.0]
  -> Filter
    [cardinality=1.0, cost=4.0]
    -> Table "COLOR" Access By ID
      -> Bitmap
        -> Index "UNQ_COLOR_NAME" Unique Scan
[cardinality=1000.0, cost=???]
Select Expression
  [cardinality=1000.0, cost=???]
  -> Local Table Full Scan

```

3.2.7. Acesso procedural

Este método de acesso é usado ao selecionar dados de stored procedures que utilizam a cláusula `SUSPEND` para retornar o resultado. No Firebird, uma stored procedure sempre representa uma caixa preta, sobre cujos detalhes internos e implementação o servidor não faz nenhuma suposição. A procedure é sempre considerada uma fonte de dados não determinística, ou seja, pode retornar dados diferentes em duas chamadas subsequentes executadas em condições iguais. Diante disso, é óbvio que qualquer tipo de indexação dos resultados da procedure é impossível. O método de acesso procedural também representa um análogo da varredura completa de uma tabela. A cada fetch da procedure, ela é executada a partir do ponto da última suspensão (início da procedure para o primeiro fetch) até a próxima cláusula `SUSPEND`, após o que seus parâmetros de saída formam uma linha de dados, que é retornada por este método de acesso.

Semelhante ao acesso para tabelas externas, o custo do acesso procedural também não é calculado nem considerado, pois não há alternativas. Para stored procedures, a cardinalidade é assumida como 1000.

No plano Legacy, o acesso procedural é exibido como “NATURAL”.



Em versões antigas do Firebird (antes da 3.0), pode ser exibida a detalhação (planos) das seleções dentro da procedure. Isso permite avaliar o plano de execução da consulta como um todo, considerando os “detalhes internos” de todas as procedures envolvidas, mas formalmente isso é incorreto.

Exemplo 26. Acesso procedural

```

SELECT *
FROM PROC_TABLE

```

```

PLAN (PROC_TABLE NATURAL)

```

```

[cardinality=1000.0, cost=???]
Select Expression
  [cardinality=1000.0, cost=???]

```


-> Procedure "PROC_TABLE" Scan

3.3. Filtros

Este grupo de métodos de acesso atua como um transformador dos dados recebidos. A principal diferença deste grupo em relação aos outros é que todos os filtros possuem apenas uma entrada. Eles não alteram a largura do fluxo de entrada, apenas reduzem sua cardinalidade de acordo com as regras definidas por sua função. A entrada de um filtro pode receber um fluxo de dados de uma fonte primária ou de qualquer outra fonte de dados.

Do ponto de vista da implementação, os filtros têm mais uma característica em comum — o custo não é estimado para eles. Ou seja, considera-se que todos os filtros são executados em tempo zero.

A seguir, serão descritas as implementações existentes no Firebird dos métodos-filtro e as tarefas que eles resolvem.

3.3.1. Verificação de predicados

Provavelmente, este é o caso de uso mais frequente e, ao mesmo tempo, o mais simples de entender. Este filtro verifica uma condição lógica para cada registro passado para sua entrada. Se a condição for verdadeira, o registro é passado inalterado para a saída; caso contrário, é ignorado. Dependendo dos dados de entrada e da condição lógica, um fetch de um registro deste filtro pode resultar em um ou mais fetches do fluxo de entrada. Casos degenerados de filtros de verificação são condições predefinidas como $(1 = 1)$ ou $(1 <> 1)$, que simplesmente passam os dados de entrada através de si ou os removem.

Como se pode deduzir pelo nome, esses filtros são usados para executar predicados nas cláusulas WHERE, HAVING, ON e outras. Com o objetivo de reduzir a cardinalidade resultante do conjunto de resultados, a verificação de predicados é sempre posicionada o mais “baixo” (“profundo”) possível na árvore de execução da consulta. No caso de verificação em um campo da tabela, a verificação do predicado será executada imediatamente após o fetch do registro dessa tabela.

Cada predicado tem sua própria seletividade e, assim, leva a uma redução na cardinalidade do fluxo de saída. Para acesso não indexado, os predicados têm a seguinte seletividade:

Tabela 6. Seletividade dos predicados

Predicados	Seletividade
=, IS NULL, IS NOT DISTINCT FROM	0.1
<, <=, >, >=, <>, IS NOT NULL, IS DISTINCT FROM	0.5
BETWEEN	0.25
STARTING WITH, CONTAINING	0.5
IN (<list>)	$0.1 * N$
IN (<select>), EXISTS(<select>)	0.5

A seletividade dos predicados combinados pelo operador AND é multiplicada, e pelo operador OR é

somada.

Assim, a cardinalidade do fluxo de saída é calculada pela fórmula

$$\text{cardinality} = \text{selectivity} * \text{inputCardinality}$$

No plano Legacy, a verificação de predicados não é exibida.

No plano Explain, a verificação de predicados é exibida com a palavra “Filter”, mas o próprio predicado não é mostrado.

Exemplo 27. Verificação de predicado de filtragem

```
SELECT *
FROM RDB$RELATIONS
WHERE RDB$SYSTEM_FLAG = 0
```

```
PLAN (RDB$RELATIONS NATURAL)
```

```
[cardinality=35.0, cost=350.0]
Select Expression
  [cardinality=35.0, cost=350.0]
    -> Filter
      [cardinality=350.0, cost=350.0]
        -> Table "RDB$RELATIONS" Full Scan
```

A verificação de predicado também é exibida ao usar acesso indexado.

Exemplo 28. Verificação de predicado de filtragem ao usar acesso indexado

```
SELECT *
FROM RDB$RELATIONS
WHERE RDB$RELATION_NAME = ?
```

```
PLAN (RDB$RELATIONS INDEX (RDB$INDEX_0))
```

```
[cardinality=1.0, cost=4.0]
Select Expression
  [cardinality=1.0, cost=4.0]
    -> Filter
      [cardinality=1.0, cost=4.0]
        -> Table "RDB$RELATIONS" Access By ID
          -> Bitmap
```

-> Index "RDB\$INDEX_0" Unique Scan

Com base no último exemplo, pode surgir uma pergunta: por que o Filter é usado se o predicado é implementado via INDEX UNIQUE SCAN? O fato é que o Firebird implementa uma busca difusa no índice. Ou seja, a varredura do índice é apenas uma otimização para o cálculo do predicado e, em alguns casos, pode retornar mais registros do que o necessário. É por isso que o servidor não confia apenas no resultado da varredura do índice e verifica o predicado novamente após o fetch do registro. Deve-se notar que na grande maioria dos casos isso não acarreta sobrecarga perceptível em termos de desempenho. Nesse caso, a verificação do predicado não altera a estimativa de cardinalidade, pois ela já foi obtida no acesso indexado (filtrado pelo índice).

Verificação de predicados invariantes

Um predicado é invariante se seu valor não depende dos campos dos fluxos que estão sendo filtrados. O exemplo mais simples de predicados invariantes são condições como $1=0$ e $1=1$. Predicados invariantes também podem conter marcadores de parâmetros, por exemplo $? = 1$.

A partir do Firebird 5.0, predicados invariantes e ao mesmo tempo determinísticos são reconhecidos pelo otimizador e calculados uma única vez. Diferente dos predicados de filtragem comuns, os predicados invariantes são calculados o mais cedo possível; sua verificação é sempre posicionada o mais “alto” possível na árvore de execução da consulta. Se um predicado invariante de filtragem tiver o valor FALSE, a extração de registros das fontes de dados subjacentes é imediatamente interrompida.

Diferente dos predicados de filtragem comuns, a filtragem por um predicado invariante não altera a cardinalidade do fluxo de saída.

No plano Legacy, a verificação de predicados invariantes não é exibida. No plano Explain, a verificação de predicados invariantes é exibida com a palavra “Filter (preliminary)”, mas o próprio predicado não é mostrado.

Exemplo 29. Verificação de predicados invariantes

```
SELECT *
FROM RDB$RELATIONS
WHERE RDB$SYSTEM_FLAG = 0
AND 1 = ?
```

PLAN (RDB\$RELATIONS NATURAL)

```
[cardinality=35.0, cost=350.0]
Select Expression
  [cardinality=35.0, cost=350.0]
    -> Filter (preliminary)
      [cardinality=35.0, cost=350.0]
        -> Filter
          [cardinality=350.0, cost=350.0]
```

-> Table "RDB\$RELATIONS" Full Scan

3.3.2. Ordenação

Também conhecida como ordenação externa (external sort). Este filtro é aplicado pelo otimizador quando é necessário ordenar o fluxo de entrada e não é possível usar a navegação por índice (veja [Navegação por índice](#)). Exemplos de seu uso: ordenação ou agrupamento (quando não há índices adequados ou os índices não são aplicáveis ao fluxo de entrada), construção de árvore B+ de índice, execução da operação DISTINCT, preparação de dados para mesclagem de uma única passagem (veja abaixo) e outros.

Como os dados de entrada são, por definição, não ordenados, é óbvio que o filtro de ordenação deve realizar o fetch de todos os registros de seu fluxo de entrada antes de poder emitir qualquer um deles na saída. Assim, este filtro pode ser considerado uma fonte de dados com buffer.

A ordenação externa é executada da seguinte forma. O conjunto de registros de entrada é colocado em um buffer interno, depois é ordenado pelo algoritmo de ordenação rápida (quick sort) e o bloco é movido para a memória externa. Em seguida, o próximo bloco é preenchido da mesma maneira e o processo continua até que os registros no fluxo de entrada se esgotem. Depois disso, os blocos preenchidos são lidos e uma árvore binária de mesclagem é construída a partir deles. Ao ler do filtro de ordenação, a árvore é analisada e os registros são mesclados em uma única passagem. A memória externa pode ser tanto memória virtual quanto espaço em disco, dependendo das configurações do arquivo de configuração do servidor.

Ao executar a ordenação externa, o Firebird grava tanto os campos-chave (ou seja, aqueles especificados nas cláusulas ORDER BY ou GROUP BY) quanto os campos não-chave (todos os outros campos referenciados na consulta) nos blocos de ordenação, que são mantidos na memória ou em arquivos temporários. Após a conclusão da ordenação, esses campos são lidos de volta dos blocos de ordenação.

A ordenação possui dois modos de operação: “normal” e “truncante”. O primeiro preserva registros com chaves de ordenação duplicadas, enquanto o segundo faz com que os duplicados sejam removidos. É o modo “truncante” que implementa a operação DISTINCT, por exemplo.

O custo para a ordenação externa não é calculado. No modo normal de ordenação, a estimativa de cardinalidade do fluxo de saída não é alterada; no modo truncante, a cardinalidade é dividida por 10.

No plano Legacy, a ordenação é denotada pela palavra “SORT”, seguida entre parênteses pela descrição do fluxo de entrada.

No plano Explain, a ordenação é denotada pela palavra “Sort” para o modo normal e “Unique Sort” para o modo truncante. Em seguida, entre parênteses, são indicados o comprimento dos registros sendo ordenados (record length) e o comprimento da chave de ordenação (key length). Os registros sendo ordenados sempre incluem a chave de ordenação.

É possível estimar o consumo de memória para a ordenação multiplicando o comprimento dos registros sendo ordenados (record length) pela cardinalidade do fluxo sendo ordenado. Por padrão,

o Firebird tentará realizar a ordenação na memória operacional; assim que essa memória se esgotar, o mecanismo começará a usar arquivos temporários. O volume de memória operacional disponível para ordenações é definido pelo parâmetro TempCacheLimit. Na arquitetura Classic, esse limite é definido para cada conexão, enquanto nas arquiteturas SuperServer e SuperClassic, é definido para todas as conexões do banco de dados.

Exemplo 30. Usando ordenação externa no modo normal

```
SELECT RDB$RELATION_NAME, RDB$SYSTEM_FLAG
FROM RDB$RELATIONS
ORDER BY RDB$SYSTEM_FLAG
```

```
PLAN SORT (RDB$RELATIONS NATURAL)
```

```
[cardinality=350.0, cost=350.0]
Select Expression
  [cardinality=350.0, cost=350.0]
    -> Sort (record length: 284, key length: 8)
      [cardinality=350.0, cost=350.0]
        -> Table "RDB$RELATIONS" Full Scan
```

Exemplo 31. Usando ordenação externa no modo truncante

```
SELECT DISTINCT
  RDB$RELATION_NAME, RDB$SYSTEM_FLAG
FROM RDB$RELATIONS
```

```
PLAN SORT (RDB$RELATIONS NATURAL)
```

```
[cardinality=35.0, cost=350.0]
Select Expression
  [cardinality=35.0, cost=350.0]
    -> Unique Sort (record length: 284, key length: 264)
      [cardinality=350.0, cost=350.0]
        -> Table "RDB$RELATIONS" Full Scan
```

Exemplo 32. Usando ordenação externa em conjunto com filtragem

```
SELECT RDB$RELATION_NAME, RDB$SYSTEM_FLAG
FROM RDB$RELATIONS
WHERE RDB$RELATION_NAME > ?
ORDER BY RDB$SYSTEM_FLAG
```

```
PLAN SORT (RDB$RELATIONS INDEX (RDB$INDEX_0))
```

```
[cardinality=125.0, cost=159.000]
Select Expression
  [cardinality=125.0, cost=159.000]
  -> Sort (record length: 284, key length: 8)
    [cardinality=125.0, cost=159.000]
    -> Filter
      [cardinality=125.0, cost=159.000]
      -> Table "RDB$RELATIONS" Access By ID
        -> Bitmap
          -> Index "RDB$INDEX_0" Range Scan (lower bound: 1/1)
```

Possíveis casos de ordenação redundante (por exemplo, DISTINCT e ORDER BY no mesmo campo) são eliminados pelo otimizador e resultam apenas no número mínimo necessário de ordenações.

Refetch

A partir do Firebird 4.0, foi introduzida uma otimização de ordenação para conjuntos de dados amplos, que aparece no plano Explain como Refetch. Por "conjunto de dados amplo" entende-se um conjunto de dados no qual o comprimento total dos campos do registro é grande.

Ao executar uma ordenação externa, o Firebird grava tanto os campos-chave (ou seja, aqueles especificados na cláusula ORDER BY ou GROUP BY) quanto os campos não-chave (todos os outros campos referenciados na consulta) em blocos de ordenação, que são mantidos na memória ou em arquivos temporários. Após a conclusão da ordenação, esses campos são lidos de volta dos blocos de ordenação. Normalmente, essa abordagem é considerada mais rápida, pois os registros são lidos dos arquivos temporários na ordem correspondente aos registros ordenados, em vez de serem selecionados aleatoriamente da página de dados. No entanto, se os campos não-chave forem grandes (por exemplo, usando VARCHAR longos), isso aumenta o tamanho dos blocos de ordenação e, portanto, leva a mais operações de E/S nos arquivos temporários.

A abordagem alternativa (Refetch) consiste em armazenar nos blocos de ordenação apenas os campos-chave e o DBKEY dos registros de todas as tabelas envolvidas, enquanto os campos não-chave são recuperados das páginas de dados após a ordenação. Isso melhora o desempenho da ordenação no caso de campos não-chave longos. Pela descrição do Refetch, fica claro que ele só pode ser aplicado no modo de ordenação normal; para o modo de truncamento, o Refetch não é aplicável, pois nesse modo o DBKEY não entra nos blocos de ordenação.

Para ordenação externa usando Refetch, o custo não é calculado. Para escolher o método pelo qual os dados serão ordenados, o otimizador usa o parâmetro InlineSortThreshold. O valor especificado para InlineSortThreshold determina o tamanho máximo do registro de ordenação (em bytes) que pode ser armazenado inline, ou seja, dentro do bloco de ordenação. Zero significa que os registros são sempre relidos (Refetch). O valor ideal para este parâmetro deve ser determinado experimentalmente. O valor padrão é 1000 bytes.

No plano de execução Legacy, Refetch e ordenação externa são exibidos da mesma forma, como

SORT.

No plano de execução Explain, a ordenação usando Refetch é exibida como uma árvore, onde a palavra “Refetch” é especificada como raiz, e a palavra “Sort” é especificada no nível inferior. Em seguida, entre parênteses, são fornecidos o comprimento das chaves de ordenação + DBKEY das tabelas usadas (record length) e o comprimento da chave de ordenação (key length).

Exemplo 33. Otimização de ordenação externa usando Refetch

```
SELECT *
FROM RDB$RELATIONS
ORDER BY RDB$SYSTEM_FLAG
```

```
PLAN SORT (RDB$RELATIONS NATURAL)
```

```
[cardinality=350.0, cost=350.0]
Select Expression
  [cardinality=350.0, cost=350.0]
    -> Refetch
      -> Sort (record length: 28, key length: 8)
        [cardinality=350.0, cost=350.0]
          -> Table "RDB$RELATIONS" Full Scan
```

Pelo plano Explain, pode-se ver que primeiro ocorre a ordenação pelas chaves de ordenação; nos blocos de ordenação entram as próprias chaves + DBKEY da tabela, e então os registros completos são extraídos da tabela na ordem ordenada por DBKEY usando o método Refetch.

Agora, vamos tentar usar o Refetch para o modo de ordenação de truncamento.

Exemplo 34. No modo de ordenação de truncamento, Refetch não é usado

```
SELECT DISTINCT *
FROM RDB$RELATIONS
```

```
PLAN SORT (RDB$RELATIONS NATURAL)
```

```
[cardinality=350.0, cost=350.0]
Select Expression
  [cardinality=350.0, cost=350.0]
    -> Unique Sort (record length: 1438, key length: 1412)
      [cardinality=350.0, cost=350.0]
        -> Table "RDB$RELATIONS" Full Scan
```

Aqui, apesar de ser necessário ordenar registros amplos, o Refetch não pode ser usado e, portanto, é aplicada a ordenação externa comum.

3.3.3. Agregação

Este filtro é usado exclusivamente para calcular funções agregadas (MIN, MAX, AVG, SUM, COUNT ...), incluindo casos de seu uso em agrupamentos.

A implementação é bastante trivial. Todas as funções listadas requerem um único registro temporário para armazenar o valor limite atual (MIN/MAX) ou acumulado (outras funções). Em seguida, para cada registro do fluxo de entrada, verificamos ou somamos esse registro. No caso de agrupamento, o algoritmo é um pouco mais complexo. Para o cálculo correto da função para cada grupo, os limites entre esses grupos devem ser claramente definidos. A solução mais simples é garantir a ordenação do fluxo de entrada. Nesse caso, realizamos a agregação para a mesma chave de agrupamento e, após sua alteração, emitimos o resultado e continuamos com a próxima chave. Essa abordagem é exatamente a usada no Firebird – a agregação por grupos depende estritamente da presença de ordenação na entrada. O fluxo de entrada pode ser ordenado usando ordenação externa ou navegação por índice (se possível). O método Refetch não pode ser usado para ordenar o fluxo de entrada para agrupamento.

Existe uma maneira alternativa de calcular agregados – hash. Nesse caso, a ordenação do fluxo de entrada não é necessária; no entanto, uma função hash é aplicada a cada chave de agrupamento e o registro é armazenado para cada chave (ou melhor, para cada colisão de chave) em uma tabela hash. A vantagem desse algoritmo é que não há necessidade de ordenação adicional. A desvantagem é o maior consumo de memória para armazenar a tabela hash. Esse método geralmente supera a ordenação quando há baixa seletividade das chaves de agrupamento (poucos valores únicos, portanto uma tabela hash pequena). A agregação por hash não está presente no Firebird.



A agregação por hash está planejada para implementação nas próximas versões do servidor.

Se funções agregadas forem usadas sem agrupamento, a cardinalidade do fluxo de saída é sempre igual a um. Ao usar agrupamento, a cardinalidade de entrada é dividida por 1000. O custo da agregação não é calculado.

O agrupamento também pode ser usado sem funções agregadas; nesse caso, ele é análogo a SELECT DISTINCT para eliminar registros duplicados, mas, diferentemente de DISTINCT, pode usar tanto ordenação externa quanto navegação por índice.

No plano de execução Legacy, a agregação não é mostrada.

No plano Explain, a agregação é exibida usando a palavra “Aggregate”.

Exemplo 35. Cálculo da função agregada MAX

```
SELECT MAX(RDB$RELATION_ID)
FROM RDB$RELATIONS
```



```
PLAN (RDB$RELATIONS NATURAL)
```

```
[cardinality=1.0, cost=350.0]
Select Expression
  [cardinality=1.0, cost=350.0]
  -> Aggregate
    [cardinality=350.0, cost=350.0]
    -> Table "RDB$RELATIONS" Full Scan
```

Como existe um índice ASCENDING para o campo RDB\$RELATION_ID, o cálculo da função agregada MIN pode ser realizado usando navegação por índice (veja [Navegação por índice](#)).

Exemplo 36. Cálculo da função agregada MIN usando navegação por índice

```
SELECT MIN(RDB$RELATION_ID)
FROM RDB$RELATIONS
```

```
PLAN (RDB$RELATIONS ORDER RDB$INDEX_1)
```

```
[cardinality=1.0, cost=4.0]
Select Expression
  [cardinality=1.0, cost=4.0]
  -> Aggregate
    [cardinality=1.0, cost=4.0]
    -> Table "RDB$RELATIONS" Access By ID
      -> Index "RDB$INDEX_1" Full Scan
```

Exemplo 37. Cálculo de função agregada com agrupamento. Para separar os grupos, é usada ordenação externa.

```
SELECT RDB$SYSTEM_FLAG, COUNT(*)
FROM RDB$FIELDS
GROUP BY RDB$SYSTEM_FLAG
```

```
PLAN SORT (RDB$FIELDS NATURAL)
```

```
[cardinality=7.756, cost=7756.0]
Select Expression
  [cardinality=7.756, cost=7756.0]
  -> Aggregate
    [cardinality=7756.0, cost=7756.0]
    -> Sort (record length: 28, key length: 8)
```

```
[cardinality=7756.0, cost=7756.0]
-> Table "RDB$RELATIONS" Full Scan
```

Exemplo 38. Cálculo de função agregada com agrupamento. Para separar os grupos, é usada navegação por índice.

```
SELECT RDB$RELATION_NAME, COUNT(*)
FROM RDB$RELATION_FIELDS
GROUP BY RDB$RELATION_NAME
```

```
PLAN (RDB$RELATION_FIELDS ORDER RDB$INDEX_4)
```

```
[cardinality=2.4, cost=4381.0]
Select Expression
  [cardinality=2.4, cost=4381.0]
    -> Aggregate
      [cardinality=2400.0, cost=4381.0]
        -> Table "RDB$RELATION_FIELDS" Access By ID
          -> Index "RDB$INDEX_4" Full Scan
```

Exemplo 39. Cálculo de função agregada com agrupamento e filtragem

```
SELECT RDB$RELATION_NAME, COUNT(*)
FROM RDB$RELATION_FIELDS
WHERE RDB$RELATION_NAME > ?
GROUP BY RDB$RELATION_NAME
```

```
PLAN (RDB$RELATION_FIELDS ORDER RDB$INDEX_4)
```

```
[cardinality=0.13, cost=237.3]
Select Expression
  [cardinality=0.13, cost=237.3]
    -> Aggregate
      [cardinality=130.1, cost=237.3]
        -> Filter
          [cardinality=130.1, cost=237.3]
            -> Table "RDB$RELATION_FIELDS" Access By ID
              -> Index "RDB$INDEX_4" Range Scan (lower bound: 1/1)
```

No caso de cálculo das funções SUM/AVG/COUNT no modo DISTINCT, para cada grupo de valores, antes da acumulação, eles são ordenados no modo de eliminação de duplicatas. Nesse caso, a palavra SORT não aparece no plano.

Exemplo 40. Cálculo de função agregada com agrupamento e filtragem

```
SELECT RDB$RELATION_NAME, COUNT(DISTINCT RDB$NULL_FLAG)
FROM RDB$RELATION_FIELDS
GROUP BY RDB$RELATION_NAME
```

```
PLAN (RDB$RELATION_FIELDS ORDER RDB$INDEX_4)
```

```
[cardinality=2.4, cost=4381.0]
Select Expression
  [cardinality=2.4, cost=4381.0]
  -> Aggregate
    [cardinality=2400.0, cost=4381.0]
    -> Table "RDB$RELATION_FIELDS" Access By ID
      -> Index "RDB$INDEX_4" Full Scan
```

Filtragem na cláusula HAVING

Os predicados na cláusula HAVING podem conter funções agregadas ou campos e expressões de agrupamento. Se o predicado for aplicado a uma função agregada, a filtragem ocorre após o agrupamento e o cálculo dos agregados. Se o predicado for aplicado a campos e expressões de agrupamento, ocorrerá um "push down" do predicado, de modo que ele será calculado antes do agrupamento e da agregação.

Exemplo 41. Filtragem por função agregada na cláusula HAVING

```
SELECT RDB$RELATION_NAME, COUNT(*)
FROM RDB$RELATION_FIELDS
GROUP BY RDB$RELATION_NAME
HAVING COUNT(*) > ?
```

```
PLAN (RDB$RELATION_FIELDS ORDER RDB$INDEX_4)
```

```
[cardinality=1.2, cost=4381.0]
Select Expression
  [cardinality=1.2, cost=4381.0]
  -> Filter
    [cardinality=2.4, cost=4381.0]
    -> Aggregate
      [cardinality=2400.0, cost=4381.0]
      -> Table "RDB$RELATION_FIELDS" Access By ID
        -> Index "RDB$INDEX_4" Full Scan
```

Exemplo 42. Filtragem por coluna de agrupamento

```
SELECT RDB$RELATION_NAME, COUNT(*)
FROM RDB$RELATION_FIELDS
GROUP BY RDB$RELATION_NAME
HAVING RDB$RELATION_NAME > ?
```

```
PLAN (RDB$RELATION_FIELDS ORDER RDB$INDEX_4)
```

```
[cardinality=1.0, cost=237.3]
Select Expression
  [cardinality=1.0, cost=237.3]
  -> Filter
    [cardinality=0.13, cost=237.3]
    -> Aggregate
      [cardinality=130.1, cost=237.3]
      -> Filter
        [cardinality=130.1, cost=237.3]
        -> Table "RDB$RELATION_FIELDS" Access By ID
          -> Index "RDB$INDEX_4" Range Scan (lower bound: 1/1)
```

No último exemplo, o predicado de filtragem foi “empurrado para dentro”, porém após o cálculo dos agregados a filtragem foi aplicada novamente. É importante notar que a filtragem repetida realizou uma normalização da cardinalidade (em teoria, funções agregadas com e sem agrupamento nunca podem retornar menos de um registro).

3.3.4. Contadores

Este tipo de filtro também é muito simples. Seu objetivo é retornar apenas parte dos registros do fluxo de entrada, com base em um determinado valor N de um contador interno. Existem duas variantes deste filtro, usadas para implementar as cláusulas FIRST/SKIP/ROWS/FETCH FIRST/OFFSET da consulta. A primeira variante (contador FIRST) emite na saída apenas os primeiros N registros do seu fluxo de entrada, após o que emite o sinal EOF. A segunda variante (contador SKIP) ignora os primeiros N registros do seu fluxo de entrada e começa a emití-los na saída a partir do registro N+1. Obviamente, se houver uma limitação de seleção na consulta do tipo (FIRST 100 SKIP 100), primeiro deve ser aplicado o contador SKIP, e somente depois dele o contador FIRST. O otimizador garante a aplicação correta dos contadores durante a execução da consulta.

O contador SKIP não altera a cardinalidade do fluxo de saída. A cardinalidade do fluxo de saída após o contador FIRST é igual ao valor especificado para esse contador; se o valor não for um literal, a cardinalidade é considerada como 1000.

No plano de execução Legacy, os contadores não são exibidos.

No plano Explain, o contador FIRST é exibido como “First N Records”, e o contador SKIP como “Skip N Records”.

Exemplo 43. Usando o contador FIRST

```
SELECT *
FROM RDB$RELATIONS
FETCH FIRST 10 ROWS ONLY
```

```
PLAN (RDB$RELATIONS NATURAL)
```

```
[cardinality=10.0, cost=350.5]
Select Expression
  [cardinality=10.0, cost=350.5]
  -> First N Records
    [cardinality=350.5, cost=350.5]
    -> Table "RDB$RELATIONS" Full Scan
```

Exemplo 44. Usando FIRST(?)

```
SELECT FIRST(?) *
FROM RDB$RELATIONS
```

```
PLAN (RDB$RELATIONS NATURAL)
```

```
[cardinality=1000.0, cost=350.5]
Select Expression
  [cardinality=1000.0, cost=350.5]
  -> First N Records
    [cardinality=350.5, cost=350.5]
    -> Table "RDB$RELATIONS" Full Scan
```

Exemplo 45. Usando o contador SKIP

```
SELECT *
FROM RDB$RELATIONS
OFFSET 10 ROWS
```

```
PLAN (RDB$RELATIONS NATURAL)
```

```
[cardinality=350.5, cost=350.5]
Select Expression
  [cardinality=350.5, cost=350.5]
```

```
-> Skip N Records
    [cardinality=350.5, cost=350.5]
-> Table "RDB$RELATIONS" Full Scan
```

Exemplo 46. Usando contadores FIRST e SKIP simultaneamente

```
SELECT FIRST(10) SKIP(20) *
FROM RDB$RELATIONS
```

```
PLAN (RDB$RELATIONS NATURAL)
```

```
[cardinality=10.0, cost=350.5]
Select Expression
  [cardinality=10.0, cost=350.5]
  -> First N Records
    [cardinality=350.5, cost=350.5]
    -> Skip N Records
      [cardinality=350.5, cost=350.5]
      -> Table "RDB$RELATIONS" Full Scan
```

3.3.5. Verificação de singularidade

Este filtro é usado para garantir que apenas um registro seja retornado da fonte de dados. Ele é aplicado quando subconsultas singulares (singleton subqueries) são usadas no texto da consulta.

Para cumprir sua função, o filtro de verificação de singularidade realiza duas leituras do fluxo de entrada. Se a segunda leitura retornar EOF, então emitimos o primeiro registro na saída. Caso contrário, iniciamos o erro `isc_sing_select_err` (“multiple rows in singleton select”).

A cardinalidade do fluxo de saída após a verificação de singularidade é igual a 1.

No plano de execução Legacy, a verificação de singularidade não é exibida.

No plano Explain, a verificação de singularidade é exibida como “Singularity Check”.

Exemplo 47. Verificação de singularidade de uma subconsulta correlacionada

```
SELECT
  (SELECT RDB$RELATION_NAME FROM RDB$RELATIONS R
   WHERE R.RDB$RELATION_ID = D.RDB$RELATION_ID - 1) AS LAST_RELATION,
  D.RDB$SQL_SECURITY
FROM RDB$DATABASE D
```

```
PLAN (R INDEX (RDB$INDEX_1))
```

```
PLAN (D NATURAL)
```

```
[cardinality=1.0, cost=4.62]
```

```
Sub-query
```

```
[cardinality=1.0, cost=4.62]
```

```
-> Singularity Check
```

```
[cardinality=1.62, cost=4.62]
```

```
-> Filter
```

```
[cardinality=1.62, cost=4.62]
```

```
-> Table "RDB$RELATIONS" as "R" Access By ID
```

```
-> Bitmap
```

```
-> Index "RDB$INDEX_1" Range Scan (full match)
```

```
[cardinality=1.0, cost=5.62]
```

```
Select Expression
```

```
[cardinality=1.0, cost=1.0]
```

```
-> Table "RDB$DATABASE" as "D" Full Scan
```

Na forma Legacy, o primeiro plano refere-se à subconsulta, e o segundo à consulta principal.

Exemplo 48. Verificação de singularidade de uma subconsulta não correlacionada

```
SELECT *
FROM RDB$RELATIONS
WHERE RDB$RELATION_ID = ( SELECT RDB$RELATION_ID - 1 FROM RDB$DATABASE )
```

```
PLAN (RDB$DATABASE NATURAL)
PLAN (RDB$RELATIONS INDEX (RDB$INDEX_1))
```

```
[cardinality=1.0, cost=1.0]
```

```
Sub-query (invariant)
```

```
[cardinality=1.0, cost=1.0]
```

```
-> Singularity Check
```

```
[cardinality=1.0, cost=1.0]
```

```
-> Table "RDB$DATABASE" Full Scan
```

```
[cardinality=1.62, cost=5.62]
```

```
Select Expression
```

```
[cardinality=1.62, cost=4.62]
```

```
-> Filter
```

```
[cardinality=1.62, cost=4.62]
```

```
-> Table "RDB$RELATIONS" Access By ID
```

```
-> Bitmap
```

```
-> Index "RDB$INDEX_1" Range Scan (full match)
```

3.3.6. Bloqueio de registro

Este filtro implementa o bloqueio pessimista de registro. Ele é aplicado apenas quando a cláusula

WITH LOCK está presente no texto da consulta. Cada registro lido do fluxo de entrada do filtro é retornado na saída já bloqueado pela transação atual. Para o bloqueio, é usada a criação de uma nova versão do registro, marcada com o identificador da transação atual. Caso a transação atual já tenha alterado esse registro, o bloqueio não é realizado.

Nas versões atuais, o filtro de bloqueio funciona apenas para métodos de acesso primários. Ele não altera a cardinalidade do fluxo de dados de saída. No plano de execução Legacy, o bloqueio não é exibido.

Exemplo 49. Bloqueio de registro

```
SELECT *
FROM COLOR
WITH LOCK
```

```
PLAN (COLOR NATURAL)
```

```
[cardinality=233.0, cost=233.0]
Select Expression
  [cardinality=233.0, cost=233.0]
  -> Write Lock
    [cardinality=233.0, cost=233.0]
    -> Table "COLOR" Full Scan
```

3.3.7. Ramificação Condicional de Fluxos (Conditional Stream)

A ramificação condicional de fluxos está disponível a partir do Firebird 3.0. Este método de acesso é aplicado nos casos em que, dependendo de um parâmetro de entrada, uma tabela pode ser lida usando uma varredura completa (full scan) ou usando acesso por índice. Normalmente, isso é possível para uma condição de filtro do tipo INDEXED_FIELD = ? OR 1=?.

A cardinalidade do fluxo de saída é considerada como a média entre as cardinalidades das opções de recuperação de dados. O custo também é calculado como o custo médio entre a primeira e a segunda opção. Assume-se que a probabilidade de execução das diferentes opções é a mesma.

$$\text{cardinality} = (\text{cardinality_option_1} + \text{cardinality_option_2}) / 2$$

$$\text{cost} = (\text{cost_option_1} + \text{cost_option_2}) / 2$$

No plano Legacy, a ramificação condicional não é exibida, mas ambas as opções de recuperação de dados da tabela são exibidas, separadas por vírgula.

No plano Explain, a ramificação condicional é exibida como uma árvore, cuja raiz é denotada pela palavra Condition, e as folhas da árvore são as opções de recuperação de dados da tabela.

Exemplo 50. Ramificação condicional de fluxos

```
SELECT *
FROM RDB$RELATIONS R
WHERE (R.RDB$RELATION_NAME = ? OR 1=?)
```

```
PLAN (R NATURAL, R INDEX (RDB$INDEX_0))
```

```
[cardinality=175.65, cost=177.15]
Select Expression
  [cardinality=175.65, cost=177.15]
  -> Filter
    [cardinality=175.65, cost=177.15]
    -> Condition
      [cardinality=350.0, cost=350.0]
      -> Table "RDB$RELATIONS" as "R" Full Scan
      [cardinality=1.3, cost=4.3]
      -> Table "RDB$RELATIONS" as "R" Access By ID
        -> Bitmap
          -> Index "RDB$INDEX_0" Unique Scan
```

3.3.8. Buffer de Registros

Este método de acesso é usado como auxiliar na implementação de outros métodos de acesso de nível superior, como “Janela Deslizante” e “Junção por Hash”.

O buffer de registros destina-se a salvar os resultados dos métodos de acesso subjacentes na memória operacional; assim que essa memória for esgotada, o mecanismo começará a usar arquivos temporários. O volume de memória interna disponível para o buffer de registros é definido pelo parâmetro TempCacheLimit. Na arquitetura Classic, esse limite é definido para cada conexão, e nas arquiteturas Classic e SuperClassic para todas as conexões do banco de dados.

O consumo de memória para o buffer pode ser estimado multiplicando o comprimento dos registros (record length) pela cardinalidade do fluxo de entrada.

No plano Legacy, o buffer de registros não é exibido.

No plano Explain, o buffer de registros é exibido como “Record Buffer”, após o qual, entre parênteses, é indicado o comprimento do registro como (record length: <length>).

Exemplos que usam o buffer de registros serão mostrados na descrição dos métodos de acesso “Janela Deslizante” e “Junção por Hash”.

3.3.9. Janela Deslizante (Window)

Este grupo de métodos de acesso é usado no cálculo das chamadas funções de janela ou funções analíticas.

Uma função de janela realiza cálculos para um conjunto de linhas (rows) de alguma forma relacionadas à linha atual. Sua ação pode ser comparada ao cálculo realizado por uma função agregada. No entanto, com funções de janela, as linhas não são agrupadas em uma única linha de saída, como acontece com funções agregadas comuns, não de janela. Em vez disso, essas linhas permanecem entidades separadas. Internamente, a função de janela, assim como a função agregada, pode acessar não apenas a linha atual do resultado da consulta. O conjunto de linhas sobre o qual a função de janela realiza cálculos é chamado de **janela**.

O conjunto de linhas pode ser dividido em partições (seções). Por padrão, todas as linhas da janela são incluídas em uma partição; isso pode ser alterado especificando em `PARTITION BY` como as linhas são divididas em partições. Para cada linha, a função de janela processa apenas as linhas que estão na mesma partição que a linha atual.

Dentro de cada partição, a ordem de processamento das linhas pela função de janela pode ser definida. Isso pode ser feito na expressão da janela usando `ORDER BY`.

Além disso, na expressão da janela, pode-se especificar o quadro da janela (frame), que determina quantas linhas na partição serão processadas. Por padrão, o quadro da janela depende do tipo de função de janela. Na maioria das vezes, se a ordem de processamento das linhas for especificada (`ORDER BY` na cláusula da janela) – é do início da partição até a linha ou valor atual.

A sintaxe das funções de janela e expressões de janela é descrita no capítulo “Funções de Janela (Analíticas)” do guia de linguagem SQL do Firebird.

Do exposto acima, segue-se que os métodos de acesso usados no cálculo de funções de janela não alteram a cardinalidade do conjunto de resultados. O custo dos métodos de acesso para cálculo de funções de janela não é calculado, pois não há alternativas.

O cálculo das funções de janela começa com o buffering do resultado da seleção, que é executado após todas as outras partes da consulta `SELECT`, mas antes da ordenação (`ORDER BY`), da limitação dos resultados da seleção usando contadores `first/skip` e do cálculo das expressões para as colunas na cláusula `SELECT`. Este buffer é chamado de buffer da janela. No plano `Explain`, ele é denotado como `Window Buffer` (veja também [Buffer de Registros](#)).

Em seguida, no buffer da janela, os limites das partições são definidos, o que no plano `Explain` é denotado como `Window Partition`. Isso é feito mesmo se a cláusula `PARTITION BY` estiver ausente (haverá uma partição).

Se a expressão da janela descrever a divisão em partições `PARTITION BY`, a partição original (única) será dividida em partições novamente. Para a divisão em partições, é usada a [ordenação externa](#) pelas chaves de particionamento. Para definir a ordem de processamento das linhas (`ORDER BY` dentro da janela), também é usada a ordenação externa. O Firebird combina as chaves de particionamento e as chaves para definir a ordem das linhas em uma única ordenação externa. Se o conjunto de linhas para a ordenação externa for muito amplo, o método [Refetch](#) pode ser usado para otimização.

Após a divisão em partições e a ordenação das linhas da partição, o conjunto resultante é novamente armazenado em buffer usando o método de acesso [Buffer de Registros](#) sobre o qual será usado o método de divisão em partições `Window Partition`.

Haverá tantos Window Partition + Record Buffer e possíveis ordenações externas quantas forem as janelas diferentes usadas na consulta.

Assim que todas as partições forem definidas e as linhas dentro das partições forem ordenadas, começa o cálculo das próprias funções de janela, que é exibido no plano Explain como Window. Durante o cálculo das funções de janela dentro de uma partição, o conjunto de linhas para o cálculo da função pode crescer desde o início da partição (ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW), ou diminuir ao se mover para o final da partição (ROWS BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING), ou ter um tamanho fixo movendo-se pela partição em relação à linha atual (ROWS BETWEEN BETWEEN 2 PRECEDING AND 2 FOLLOWING), ou permanecer inalterado se o quadro da janela for definido como ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING). É por isso que este método de acesso é chamado de “Janela Deslizante” (“Sliding window”).

No plano Legacy, o cálculo das funções de janela não é exibido de forma alguma, mas se a ordenação externa (SORT) for usada para dividir as partições da janela ou ordenar as linhas dentro da partição, ela será exibida.

Exemplo 51. Cálculo da função de janela mais simples

```
SELECT
  RDB$RELATION_NAME,
  COUNT(*) OVER() AS CNT
FROM RDB$RELATIONS
```

Plan:

```
PLAN (RDB$RELATIONS NATURAL)
```

Explain plan:

```
[cardinality=350.0, cost=350.0]
Select Expression
  [cardinality=350.0, cost=350.0]
  -> Window
    [cardinality=350.0, cost=350.0]
    -> Window Partition
      [cardinality=350.0, cost=350.0]
      -> Window Buffer
        [cardinality=350.0, cost=350.0]
        -> Record Buffer (record length: 273)
          [cardinality=350.0, cost=350.0]
          -> Table "RDB$RELATIONS" Full Scan
```

Aqui há apenas uma partição, que contém todos os registros da consulta.

Exemplo 52. Cálculo da função de janela particionada

```
SELECT
  RDB$RELATION_NAME,
  COUNT(*) OVER(PARTITION BY RDB$SYSTEM_FLAG) AS CNT
FROM RDB$RELATIONS
```

Plan:

```
PLAN SORT (RDB$RELATIONS NATURAL)
```

Explain plan:

```
[cardinality=350.0, cost=350.0]
Select Expression
  [cardinality=350.0, cost=350.0]
    -> Window
      [cardinality=350.0, cost=350.0]
        -> Window Partition
          [cardinality=350.0, cost=350.0]
            -> Record Buffer (record length: 546)
              [cardinality=350.0, cost=350.0]
                -> Sort (record length: 556, key length: 8)
                  [cardinality=350.0, cost=350.0]
                    -> Window Partition
                      [cardinality=350.0, cost=350.0]
                        -> Window Buffer
                          [cardinality=350.0, cost=350.0]
                            -> Record Buffer (record length: 281)
                              [cardinality=350.0, cost=350.0]
                                -> Table "RDB$RELATIONS" Full Scan
```

Adicionamos a ordem de processamento das linhas na partição, o que também definirá implicitamente o quadro da janela (ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW).

Exemplo 53. Cálculo da função de janela particionada com definição da ordem de processamento das linhas

```
SELECT
  RDB$RELATION_NAME,
  COUNT(*) OVER(PARTITION BY RDB$SYSTEM_FLAG ORDER BY RDB$RELATION_ID) AS CNT
FROM RDB$RELATIONS
```

Plan:

```
PLAN SORT (RDB$RELATIONS NATURAL)
```

Explain plan:

```
[cardinality=350.0, cost=350.0]
Select Expression
  [cardinality=350.0, cost=350.0]
  -> Window
    [cardinality=350.0, cost=350.0]
    -> Window Partition
      [cardinality=350.0, cost=350.0]
      -> Record Buffer (record length: 546)
        [cardinality=350.0, cost=350.0]
        -> Sort (record length: 564, key length: 16)
          [cardinality=350.0, cost=350.0]
          -> Window Partition
            [cardinality=350.0, cost=350.0]
            -> Window Buffer
              [cardinality=350.0, cost=350.0]
              -> Record Buffer (record length: 281)
                [cardinality=350.0, cost=350.0]
                -> Table "RDB$RELATIONS" Full Scan
```

Agora vejamos como o plano muda se várias funções de janela com diferentes janelas forem usadas.

Exemplo 54. Cálculo de várias funções de janela com janelas diferentes

```
SELECT
  RDB$RELATION_NAME,
  COUNT(*) OVER W1 AS CNT,
  LAG(RDB$RELATION_NAME) OVER W2 AS PREV
FROM RDB$RELATIONS
WINDOW
  W1 AS (PARTITION BY RDB$SYSTEM_FLAG ORDER BY RDB$RELATION_ID),
  W2 AS (ORDER BY RDB$RELATION_ID)
```

Plan:

```
PLAN SORT (SORT (RDB$RELATIONS NATURAL))
```

Explain plan:

```
[cardinality=350.0, cost=350.0]
Select Expression
  [cardinality=350.0, cost=350.0]
  -> Window
    [cardinality=350.0, cost=350.0]
    -> Window Partition
      [cardinality=350.0, cost=350.0]
      -> Record Buffer (record length: 579)
```

```

[cardinality=350.0, cost=350.0]
-> Sort (record length: 588, key length: 8)
    [cardinality=350.0, cost=350.0]
    -> Window Partition
        [cardinality=350.0, cost=350.0]
        -> Record Buffer (record length: 546)
            [cardinality=350.0, cost=350.0]
            -> Sort (record length: 564, key length: 16)
                [cardinality=350.0, cost=350.0]
                -> Window Partition
                    [cardinality=350.0, cost=350.0]
                    -> Window Buffer
                        [cardinality=350.0, cost=350.0]
                        -> Record Buffer (record length: 281)
                            [cardinality=350.0, cost=350.0]
                            -> Table "RDB$RELATIONS" Full Scan

```

3.4. Métodos de Mesclagem

Esta categoria de métodos de acesso lembra muito os filtros. Os métodos de mesclagem também transformam os dados de entrada de acordo com um algoritmo específico. A diferença formal é apenas que este grupo de métodos sempre opera com vários fluxos de entrada. Além disso, o resultado usual de seu trabalho é a expansão da seleção por campos ou o aumento de sua cardinalidade.

Existem duas classes de métodos de mesclagem — junção (join) e união (union), que executam funções diferentes. Além disso, esta categoria também inclui a execução de consultas recursivas, que é um caso especial de uniões (union). Abaixo, conheceremos sua descrição e possíveis algoritmos de implementação.

3.4.1. Junções (Joins)

Como se pode adivinhar pelo nome, este grupo de métodos implementa junções SQL (joins). O padrão SQL define dois tipos de junções: internas (inner) e externas (outer). Além disso, as junções externas são divididas em unilaterais (left ou right) e completas (full). Qualquer tipo de junção tem dois fluxos de entrada — o esquerdo e o direito. Para junções internas e externas completas, esses fluxos são semanticamente equivalentes. No caso de uma junção externa unilateral, um dos fluxos é o principal (obrigatório) e o segundo é o secundário (opcional). Muitas vezes, o fluxo principal também é chamado de externo, e o secundário de interno. Para uma junção externa esquerda, o fluxo principal (externo) é o esquerdo, e o secundário (interno) é o direito. Para uma junção externa direita — o oposto.

Imediatamente observo que, formalmente, uma junção externa esquerda é equivalente a uma junção externa direita invertida. Por inversão, neste contexto, entende-se a substituição do fluxo externo pelo interno ou vice-versa. Portanto, é suficiente implementar apenas um tipo de junção externa unilateral (geralmente a esquerda). É assim que é feito no Firebird. No entanto, a decisão de transformar uma junção direita em uma esquerda (básica) e sua subsequente otimização era tomada pelo otimizador, o que frequentemente levava a uma otimização incorreta de junções direitas. A partir da versão 1.5, o servidor converte junções direitas em esquerdas no nível de

análise do BLR, o que elimina possíveis conflitos no otimizador.

Além dos fluxos de entrada, cada junção tem outro atributo — a condição de junção. É esta condição que determina o resultado, ou seja, como exatamente os dados dos fluxos de entrada serão correspondidos. Na ausência dessa condição, obtemos um caso degenerado — o produto cartesiano (cross join) dos fluxos de entrada.

Voltemos às junções externas unilaterais. Seus fluxos não são chamados de principal e secundário à toa. Neste caso, o fluxo externo (principal) sempre deve ser lido antes do interno (secundário), caso contrário, será impossível realizar a substituição de valores NULL, exigida pelo padrão, na ausência de correspondências do fluxo interno para o externo. A partir disso, pode-se concluir que o otimizador não tem a capacidade de escolher a ordem de execução de uma junção externa unilateral, e ela sempre será determinada pelo texto da consulta. Enquanto isso, para junções internas e externas completas, os fluxos de entrada são independentes e podem ser lidos em qualquer ordem, portanto, o algoritmo de execução de tais junções é determinado exclusivamente pelo otimizador e não depende de forma alguma do texto da consulta.

Além das junções apresentadas no padrão SQL, muitos SGBDs implementam também métodos adicionais de junção: semi-junção (semi-join) e anti-junção (anti-join). Esses tipos de junção não são representados diretamente no SQL, mas podem ser usados pelo otimizador após a transformação da árvore de execução da consulta SQL. Por exemplo, eles podem ser usados para executar predicados de filtragem EXISTS e NOT EXISTS. Esses tipos de junção são por natureza assimétricos, ou seja, os fluxos unidos para eles não são equivalentes — distinguem-se o principal (obrigatório) e o secundário (aquele que é verificado no predicado de filtragem).

Junção por Laços Aninhados (Nested Loop)

Este método é o mais comum no Firebird. Em outros SGBDs, este algoritmo também é chamado de junção por laços aninhados (nested loops join).

Sua essência é simples. Um dos fluxos de entrada (externo) é aberto e um registro é lido dele. Em seguida, o segundo fluxo (interno) é aberto e um registro também é lido dele. Então, a condição de junção é verificada. Se a condição for atendida, esses dois registros são mesclados em um e enviados para a saída. Em seguida, o segundo registro é lido do fluxo interno e o processo se repete até que o EOF seja emitido dele. Se o fluxo interno não contiver nenhum registro correspondente ao fluxo externo, há duas opções possíveis. No caso de uma junção interna, o registro não é retornado para a saída. No caso de uma junção externa, mesclamos o registro do fluxo externo com a quantidade necessária de valores NULL e o enviamos para a saída. A seguir, independentemente do tipo de junção, lemos o segundo registro do fluxo externo e começamos o processo de iteração pelo fluxo interno novamente. É óbvio que este algoritmo funciona no princípio de pipeline e a junção dos fluxos é realizada diretamente durante o fetch do cliente.

A característica chave deste método de junção é a seleção “aninhada” do fluxo interno. Obviamente, ler todo o fluxo interno para cada registro do fluxo externo é muito custoso. Portanto, a junção por laços aninhados funciona de forma eficiente apenas na presença de um índice aplicável à condição de junção. Neste caso, em cada iteração, um subconjunto de registros que satisfazem o registro atual do fluxo externo será selecionado do fluxo interno. Vale a pena notar que nem todo índice é adequado para a execução eficiente deste algoritmo, mas apenas aquele que corresponde à condição de junção. Por exemplo, com uma junção do tipo (ON SLAVE.F = 0), mesmo

usando um índice no campo F da tabela interna, os mesmos registros ainda serão lidos dela em cada iteração, o que é um desperdício de recursos. Nesta situação, uma junção por mesclagem seria mais eficiente (veja abaixo).

Pode-se introduzir a definição de “dependência dos fluxos” como a presença de campos de ambos os fluxos na condição de junção e concluir que a junção por laços aninhados é eficiente apenas quando os fluxos dependem uns dos outros.

Se lembrarmos da descrição "[Verificação de predicados](#)", onde é descrito o princípio de colocação o mais “profunda” possível de filtros predicativos pelo otimizador, então um ponto se torna claro: filtros individuais de tabela irão “abaixo” dos métodos de junção. Consequentemente, no caso de um predicado do tipo (WHERE MASTER.F = 0) e ausência de registros com o campo F igual a zero na tabela MASTER, não haverá nenhum acesso ao fluxo interno da junção, pois neste caso não há iteração pelo fluxo externo (ele não tem registros).

Como as conexões lógicas AND e OR são otimizadas por meio de mapas de bits, a junção por laços aninhados pode ser usada para todos os tipos de condições de junção, e geralmente de forma bastante eficiente.

Para junções externas, a junção por laços aninhados é sempre escolhida, pois atualmente não há alternativas. Para junções internas, o otimizador escolhe a junção por laços aninhados se seu custo for menor do que as alternativas (Hash Join). Se não houver índices adequados para os campos de junção das junções internas, então quase sempre são escolhidos algoritmos alternativos de junção, se forem possíveis. Além disso, para junções internas, o otimizador escolhe a ordem de junção mais eficiente dos fluxos. Os principais critérios de escolha são: as cardinalidades de ambos os fluxos e a seletividade da condição de junção. As seguintes fórmulas são usadas para estimar o custo de uma determinada ordem de junção:

$$\text{cost} = \text{cardinality}(\text{outer}) + \text{cardinality}(\text{outer}) * (\text{indexScanCost} + \text{cardinality}(\text{inner}) * \text{selectivity}(\text{link}))$$

$$\text{cardinality} = \text{cardinality}(\text{outer}) * (\text{cardinality}(\text{inner}) * \text{selectivity}(\text{link}))$$

A última parte da fórmula determina o custo da seleção do fluxo interno em cada iteração. Multiplicando-o pelo número de iterações, obtemos o custo total da seleção do fluxo interno. O custo total é obtido adicionando o custo da seleção do fluxo externo. De todas as permutações possíveis, a opção com o menor custo é escolhida. Durante a enumeração de opções, as claramente piores são descartadas (com base nas informações de custo já disponíveis). As permutações envolvem não apenas fluxos unidos por laços aninhados (Nested Loop), mas também por outros algoritmos de junção (Hash Join), cujo custo é calculado por outras regras.

No plano de execução Legacy, as junções por laços aninhados são exibidas pela palavra "JOIN", seguida entre parênteses por uma descrição dos fluxos de entrada separados por vírgula.

No plano de execução Explain, as junções por laços aninhados são exibidas como uma árvore, cuja raiz é rotulada como “Nested Loop Join”, após o que o tipo de junção é indicado entre parênteses. Um nível abaixo, os fluxos unidos são descritos.

Nested Loop Join (inner)*Exemplo 55. Nested Loop Join (inner)*

```
SELECT *
FROM RDB$RELATIONS R
JOIN RDB$RELATION_FIELDS RF ON R.RDB$RELATION_NAME = RF.RDB$RELATION_NAME
```

```
PLAN JOIN (R NATURAL, RF INDEX (RDB$INDEX_4))
```

```
[cardinality=3212.6, cost=4620.0]
Select Expression
  [cardinality=3212.6, cost=4620.0]
  -> Nested Loop Join (inner)
    [cardinality=350.0, cost=350.0]
    -> Table "RDB$RELATIONS" as "R" Full Scan
    [cardinality=9.2, cost=12.2]
    -> Filter
      [cardinality=9.2, cost=12.2]
      -> Table "RDB$RELATION_FIELDS" as "RF" Access By ID
        -> Bitmap
          -> Index "RDB$INDEX_4" Range Scan (full match)
```

Vamos observar um ponto. Como para a junção interna todos os fluxos são equivalentes, o otimizador, no caso de mais de dois fluxos, transforma a árvore binária em uma forma "plana". Isso significa que, de fato, a junção interna opera com mais de dois fluxos de entrada. Isso não afeta o algoritmo de forma alguma, mas explica a diferença na sintaxe dos planos para junções internas e externas.

Exemplo 56. Junção interna de múltiplos fluxos com loops aninhados

```
SELECT *
FROM RDB$RELATIONS R
JOIN RDB$RELATION_FIELDS RF ON RF.RDB$RELATION_NAME = R.RDB$RELATION_NAME
JOIN RDB$FIELDS F ON F.RDB$FIELD_NAME = RF.RDB$FIELD_SOURCE
```

```
PLAN JOIN (RF NATURAL, F INDEX (RDB$INDEX_2), R INDEX (RDB$INDEX_0))
```

```
[cardinality=3212.6, cost=20152.4]
Select Expression
  [cardinality=3212.6, cost=20152.4]
  -> Nested Loop Join (inner)
    [cardinality=2428.0, cost=2428.0]
    -> Table "RDB$RELATION_FIELDS" as "RF" Full Scan
    [cardinality=1.0, cost=4.0]
    -> Filter
```

```

[cardinality=1.0, cost=4.0]
-> Table "RDB$FIELDS" as "F" Access By ID
    -> Bitmap
        -> Index "RDB$INDEX_2" Unique Scan
[cardinality=1.3, cost=4.3]
-> Filter
    [cardinality=1.3, cost=4.3]
    -> Table "RDB$RELATIONS" as "R" Access By ID
        -> Bitmap
            -> Index "RDB$INDEX_0" Unique Scan

```

A junção cruzada (cross-join) também pertence às junções internas e é sempre executada com loops aninhados.

Exemplo 57. Nested Loop Join para CROSS JOIN

```

SELECT *
FROM RDB$PAGES
CROSS JOIN RDB$PAGES

```

```

PLAN JOIN (RDB$PAGES NATURAL, RDB$PAGES NATURAL)

```

```

[cardinality=261376.5625, cost=261376.5625]
Select Expression
    [cardinality=261376.5625, cost=261376.5625]
    -> Nested Loop Join (inner)
        [cardinality=511.25, cost=511.25]
        -> Table "RDB$PAGES" Full Scan
        [cardinality=511.25, cost=511.25]
        -> Table "RDB$PAGES" Full Scan

```

Nested Loop Join (outer)

Exemplo 58. Nested Loop Join para junção externa

```

SELECT *
FROM RDB$RELATIONS R
LEFT JOIN RDB$RELATION_FIELDS RF ON R.RDB$RELATION_NAME = RF.RDB$RELATION_NAME

```

```

PLAN JOIN (R NATURAL, RF INDEX (RDB$INDEX_4))

```

```

[cardinality=3200.6, cost=4620.0]
Select Expression
    [cardinality=3200.6, cost=4620.0]

```

```

-> Nested Loop Join (outer)
  [cardinality=350.0, cost=350.0]
  -> Table "RDB$RELATIONS" as "R" Full Scan
  [cardinality=9.2, cost=12.2]
  -> Filter
    [cardinality=9.2, cost=12.2]
    -> Table "RDB$RELATION_FIELDS" as "RF" Access By ID
      -> Bitmap
        -> Index "RDB$INDEX_4" Range Scan (full match)

```

Exemplo 59. Nested Loop Join para junção externa de múltiplos fluxos

```

SELECT *
FROM RDB$RELATIONS R
LEFT JOIN RDB$RELATION_FIELDS RF ON RF.RDB$RELATION_NAME = R.RDB$RELATION_NAME
LEFT JOIN RDB$FIELDS F ON F.RDB$FIELD_NAME = RF.RDB$FIELD_SOURCE

```

```

PLAN JOIN (JOIN (R NATURAL, RF INDEX (RDB$INDEX_4)), F INDEX (RDB$INDEX_2))

```

```

[cardinality=3200.6, cost=16003.0]
Select Expression
  [cardinality=3200.6, cost=16003.0]
  -> Nested Loop Join (outer)
    [cardinality=3200.6, cost=4620.0]
    -> Nested Loop Join (outer)
      [cardinality=350.0, cost=350.0]
      -> Table "RDB$RELATIONS" as "R" Full Scan
      [cardinality=9.2, cost=12.2]
      -> Filter
        [cardinality=9.2, cost=12.2]
        -> Table "RDB$RELATION_FIELDS" as "RF" Access By ID
          -> Bitmap
            -> Index "RDB$INDEX_4" Range Scan (full match)
      [cardinality=1.0, cost=4.0]
      -> Filter
        [cardinality=1.0, cost=4.0]
        -> Table "RDB$FIELDS" as "F" Access By ID
          -> Bitmap
            -> Index "RDB$INDEX_2" Unique Scan

```

Observe que, diferentemente da junção interna, aqui o plano Explain contém “Nested Loop Join” aninhados.

Nested Loop Join (semi)

Atualmente, este tipo de junção não é utilizado pelo otimizador do Firebird.

Sua essência é a seguinte. Um dos fluxos de entrada (externo) é aberto e um registro é lido a partir

dele. Em seguida, o segundo fluxo (interno) é aberto e um registro também é lido a partir dele. Depois, a condição de junção é verificada. Se a condição for atendida, o registro do fluxo externo é emitido na saída e o fluxo interno é imediatamente fechado. Se a condição de junção não for atendida, o segundo registro é lido do fluxo interno e o processo se repete até que o EOF seja emitido a partir dele. Em seguida, o fluxo externo lê o próximo registro e tudo se repete até que o EOF seja emitido a partir dele.

Este método de acesso pode ser usado para executar subconsultas nos predicados EXISTS e IN (<select>), bem como outros predicados que são convertidos em EXISTS (ANY, SOME).

A cardinalidade do fluxo de saída é muito difícil de estimar, mas o que se pode dizer com certeza é que ela nunca excederá a cardinalidade do fluxo externo. Assume-se que metade dos registros do fluxo externo corresponde à condição de junção.

```
cardinality = cardinality(outer) * 0.5
```

```
cost = cardinality(outer) + cardinality(outer) * cost(inner first row)
```

Aqui, `cost(inner first row)` é o custo de recuperar o primeiro registro que corresponde à condição de junção. Como o predicado EXISTS pode conter uma subconsulta complexa, e não apenas a leitura de uma única tabela, calculá-lo não é tão simples. Também é bastante óbvio que, para que este algoritmo funcione de forma eficiente, o otimizador deve mudar para a estratégia de otimização FIRST ROWS ao executar a subconsulta interna.

Nested Loop Join (anti)

Atualmente, este tipo de junção não é utilizado pelo otimizador do Firebird.

Essencialmente, este tipo de junção é o oposto do “Nested Loop Join (semi)”. Um dos fluxos de entrada (externo) é aberto e um registro é lido a partir dele. Em seguida, o segundo fluxo (interno) é aberto e um registro também é lido a partir dele. Depois, a condição de junção é verificada. Se a condição não for atendida, o registro do fluxo externo é emitido na saída e o fluxo interno é imediatamente fechado. Se a condição de junção for atendida, o segundo registro é lido do fluxo interno e o processo se repete até que o EOF seja emitido a partir dele. Em seguida, o fluxo externo lê o próximo registro e tudo se repete até que o EOF seja emitido a partir dele.

Este método de acesso pode ser usado para executar subconsultas no predicado NOT EXISTS, bem como outros predicados que são convertidos em NOT EXISTS (ALL). Além disso, sob certas condições, algumas consultas externas unilaterais também podem ser convertidas nele.

A cardinalidade do fluxo de saída é muito difícil de estimar, mas o que se pode dizer com certeza é que ela nunca excederá a cardinalidade do fluxo externo. Assume-se que metade dos registros do fluxo externo não corresponde à condição de junção.

```
cardinality = cardinality(outer) * 0.5
```

```
cost = cardinality(outer) + cardinality(outer) * cost(inner first row)
```

Aqui, `cost(inner first row)` é o custo de recuperar o primeiro registro que corresponde à condição de junção. Como o predicado `NOT EXISTS` pode conter uma subconsulta complexa, e não apenas a leitura de uma única tabela, calculá-lo não é tão simples. Também é bastante óbvio que, para que este algoritmo funcione de forma eficiente, o otimizador deve mudar para a estratégia de otimização `FIRST ROWS` ao executar a subconsulta interna.

Junção por Hash (Hash join)

A junção por hash (Hash join) é um algoritmo alternativo para execução de junções. A junção de fluxos usando o algoritmo `HASH JOIN` está disponível a partir do Firebird 3.0.

Na junção por hash, os fluxos de entrada são sempre divididos em líder e seguidor, sendo que o fluxo com menor cardinalidade é geralmente escolhido como seguidor. Primeiro, o fluxo menor (seguidor) é lido completamente em um buffer interno. Durante a leitura, uma função hash é aplicada a cada chave de junção e o par {hash, ponteiro no buffer} é gravado na tabela hash. Em seguida, o fluxo líder é lido e sua chave de junção é testada na tabela hash. Se uma correspondência for encontrada, os registros de ambos os fluxos são unidos e enviados para a saída. No caso de várias duplicatas dessa chave na tabela seguidora, vários registros serão enviados para a saída. Se a chave não for encontrada na tabela hash, passamos para o próximo registro do fluxo líder e assim por diante.

É óbvio que substituir a comparação de chaves pela comparação de hashes só é possível ao comparar para igualdade estrita das chaves. Portanto, uma junção com uma condição de ligação do tipo `(ON MASTER.F > SLAVE.F)` não pode ser executada pelo algoritmo Hash Join.

No entanto, este método tem uma vantagem em comparação com a junção por loops aninhados — é permitida a junção por hash com base em expressões. Por exemplo, uma junção com uma condição de ligação do tipo `(ON MASTER.F + 1 = SLAVE.F + 2)` é facilmente executada pelo método de hash. A razão é clara: não há dependência entre os fluxos e, portanto, não há exigência de uso de índices.

Durante a construção da tabela hash, podem ocorrer colisões. Uma colisão é uma situação em que valores diferentes de chaves produzem o mesmo valor da função hash. Portanto, após a correspondência do valor da função hash, o predicado de junção é sempre recalculado. As colisões são armazenadas como uma lista, ordenada pelas chaves, o que permite realizar uma busca binária na cadeia de colisões.

Na implementação atual, a tabela hash tem um tamanho fixo de 1009 slots, que não muda com base na estimativa de cardinalidade do fluxo a ser hashado. Também não há aumento do tamanho da tabela hash e rehashing quando o comprimento das cadeias de colisão é excedido. Isso significa que a junção por hash é eficaz apenas para uma cardinalidade relativamente pequena do fluxo seguidor. Se a cardinalidade do fluxo a ser hashado exceder 1.009.000 registros, outros algoritmos de junção são escolhidos (se houver índices, `NESTED LOOP JOIN`; se não houver, `MERGE JOIN`).



Em versões futuras do Firebird, isso pode ser alterado.

Por que 1.009.000 registros? $1.009.000 / 1009 \text{ slots} = 1000$ possíveis colisões, a busca em colisões ordenadas leva $\log_2(1000) = 10$ passos, o que já é considerado ineficiente.

O algoritmo de junção por hash é capaz de processar várias condições de ligação combinadas com AND. Nesse caso, a função hash é calculada para vários campos que fazem parte da condição de ligação (ou várias ligações, se mais de um fluxo estiver sendo unido). No entanto, no caso de combinar condições de ligação com OR, o método de junção por hash não pode ser aplicado. Vale notar aqui que a função hash nem sempre é calculada para todos os campos da condição de ligação; ela pode ser calculada apenas para parte deles. Isso acontece se o número de chaves de ligação exceder 8. Nesse caso, quando o valor da função hash corresponder, mais registros do que o necessário serão retornados da tabela hash. Os registros extras serão filtrados após o cálculo de todos os predicados na condição de ligação. Nesse caso, a eficiência da junção por hash diminui. No plano Explain, a quantidade de chaves usadas na função hash não é exibida (essa informação foi adicionada no Firebird 6.0).

Outra característica da junção por hash é que ela pode ser executada ao comparar chaves para igualdade estrita, considerando valores NULL. Ou seja, é permitido usar tanto = quanto IS NOT DISTINCT FROM. No entanto, ao construir a tabela hash, não há diferença entre esses predicados, embora seja óbvio que, se o predicado = for usado, os registros com chave NULL podem não ser adicionados à tabela hash. Isso leva a um maior consumo de memória e a uma redução no desempenho da junção por hash se houver muitas chaves com valor NULL. Esta deficiência está planejada para ser corrigida em versões futuras do Firebird (veja [CORE-7769](#)).

Antes do Firebird 5.0, o método de junção HASH JOIN era aplicado apenas na ausência de índices para a condição de ligação ou na sua inaplicabilidade; caso contrário, o otimizador escolhia o algoritmo de junção por loops aninhados (NESTED LOOP) usando índices. Na verdade, isso nem sempre é ideal. Se um fluxo grande for unido a uma tabela pequena pela chave primária, cada registro dessa tabela será lido várias vezes; além disso, as páginas de índices, se usadas, também serão lidas várias vezes. Ao usar a junção HASH JOIN, a tabela menor será lida exatamente uma vez. Naturalmente, o custo do hashing e do probing não é gratuito, portanto a escolha de qual algoritmo usar é baseada no custo.

O custo da junção pelo método HASH JOIN consiste nas seguintes partes:

- custo de extração de registros do fluxo de dados para hashing;
- custo de cópia de registros para a tabela hash (incluindo o custo de cálculo da função hash);
- custo de probing da tabela hash e custo de cópia para os registros cujos hashes corresponderam.

As seguintes fórmulas são usadas para estimar a cardinalidade e o custo (veja [InnerJoin.cpp](#)):

```
// cardinalidade do fluxo de saída
cardinality = outerCardinality * innerCardinality * linkSelectivity

// cardinalidade da tabela hash, se nem todos os campos da condição de ligação são chaves da tabela hash
hashCardinality = innerCardinality * outerCardinality * hashSelectivity

// se todos os campos da condição de ligação são chaves da tabela hash
hashCardinality = innerCardinality

cost =
    // hashed stream retrieval
    innerCost +
    // hashing cost
```

```

hashCardinality * (COST_FACTOR_MEMCOPY + COST_FACTOR_HASHING) +
// probing + copying cost
outerCardinality * (COST_FACTOR_HASHING + innerCardinality * linkSelectivity * COST_FACTOR_MEMCOPY);

COST_FACTOR_MEMCOPY = 0.5

COST_FACTOR_HASHING = 0.5

```

Onde

- hashSelectivity — seletividade das chaves de ligação pelas quais a função hash foi calculada;
- linkSelectivity — seletividade da condição de ligação;
- innerCost — custo de extração de registros para o fluxo a ser hashado;
- innerCardinality — cardinalidade do fluxo seguidor (a ser hashado);
- outerCardinality — cardinalidade do fluxo líder;
- COST_FACTOR_MEMCOPY - custo de copiar um registro de/para a memória;
- COST_FACTOR_HASHING - custo de calcular a função hash.

No plano de execução Legacy, a junção por hash é exibida pela palavra "HASH", seguida entre parênteses por uma descrição dos fluxos de entrada separados por vírgula.

No plano de execução Explain, a junção por hash é exibida como uma árvore, cuja raiz é rotulada como “Hash Join” seguido entre parênteses pelo tipo de junção. No nível abaixo, os fluxos sendo unidos são descritos. O hashing da tabela seguidora é exibido como “Record Buffer”, seguido entre parênteses pelo comprimento do registro como (record length: <length>) (veja [Buffer de Registros](#)).

Hash Join (inner)

Exemplo 60. Junção pelo método Hash Join (inner)

```

SELECT *
FROM
  RDB$RELATIONS R
  JOIN RDB$PAGES P ON P.RDB$RELATION_ID = R.RDB$RELATION_ID

```

```

PLAN HASH (P NATURAL, R NATURAL)

```

```

[cardinality=829.7, cost=1369.0]
Select Expression
  [cardinality=829.7, cost=1369.0]
  -> Filter
    [cardinality=829.7, cost=1369.0]
    -> Hash Join (inner)
      [cardinality=511.25, cost=511.25]
      -> Table "RDB$PAGES" as "P" Full Scan
        [cardinality=350.6, cost=701.2]

```

```
-> Record Buffer (record length: 1345)
    [cardinality=350.6, cost=350.6]
-> Table "RDB$RELATIONS" as "R" Full Scan
```

Hash Join (outer)

Atualmente, a junção externa unidirecional por hash não está implementada no Firebird. O algoritmo de sua execução é semelhante à execução de uma junção interna, com uma exceção - sempre é feito o hash do fluxo interno (seguidor ou opcional). Para cada registro do fluxo externo (obrigatório), a tabela hash é sondada; se uma correspondência for encontrada, os dois registros são mesclados em um e enviados para a saída. Se nenhuma correspondência for encontrada, o registro do fluxo externo é complementado com a quantidade necessária de valores NULL e enviado para a saída.

Hash Join (semi)

Este método de acesso pode ser usado para executar subconsultas nos predicados EXIST e IN (<select>), bem como outros predicados que são convertidos em EXIST (ANY, SOME). Ele está disponível a partir do Firebird 5.0.1. Por padrão, este recurso está desativado; para ativá-lo, é necessário definir o parâmetro de configuração SubQueryConversion como true no arquivo firebird.conf ou database.conf.

Nem toda subconsulta em EXIST pode ser convertida em uma semi-junção. Se a subconsulta contiver limitadores FETCH/FIRST/SKIP/ROWS, a subconsulta não pode ser convertida em uma semi-junção e será executada como uma subconsulta correlacionada comum. Assim como na junção interna, para executar uma semi-junção por hash, é necessário que as chaves sejam comparadas por igualdade estrita; é permitido o uso de expressões nas condições de vinculação, e as condições de vinculação podem ser combinadas com AND.

O algoritmo da semi-junção por hash é o seguinte. O fluxo seguidor é escolhido como o fluxo da subconsulta, que é lido completamente em um buffer interno. Durante a leitura, uma função hash é aplicada a cada chave de vinculação e o par {hash, ponteiro no buffer} é gravado na tabela hash. Em seguida, o fluxo líder é lido e sua chave de vinculação é sondada na tabela hash. Se uma correspondência for encontrada, o registro do fluxo externo é enviado para a saída. Se a chave não estiver presente na tabela hash, passamos para o próximo registro do fluxo líder e assim por diante.

A cardinalidade do fluxo de saída é muito difícil de estimar, mas o que pode ser dito com certeza é que ela nunca excederá a cardinalidade do fluxo externo. Presume-se que metade dos registros do fluxo externo correspondam à condição de vinculação.

Atualmente, o custo da semi-junção por hash não é calculado.

```
cardinality = cardinality(outer) * 0.5
```

Exemplo 61. Conexão pelo método Hash Join (semi)

```
SELECT *
```



```

FROM RDB$RELATIONS R
WHERE EXISTS (
  SELECT *
  FROM RDB$PAGES P
  WHERE P.RDB$RELATION_ID = R.RDB$RELATION_ID
  AND P.RDB$PAGE_SEQUENCE > 5
)

```

```
PLAN HASH (R NATURAL, P NATURAL)
```

```

[cardinality=175.3, cost=1548.4]
Select Expression
  [cardinality=175.3, cost=1548.4]
  -> Filter
    [cardinality=175.3, cost=1548.4]
    -> Hash Join (semi)
      [cardinality=350.6, cost=350.6]
      -> Table "RDB$RELATIONS" as "R" Full Scan
      [cardinality=255.625, cost=1022.5]
      -> Record Buffer (record length: 33)
        [cardinality=255.625, cost=511.25]
        -> Filter
          [cardinality=511.25, cost=511.25]
          -> Table "RDB$PAGES" as "P" Full Scan

```

Neste caso, a condição de vinculação é `P.RDB$RELATION_ID = R.RDB$RELATION_ID`. Esta consulta é **como se** fosse convertida na seguinte forma:

```

SELECT *
FROM
  RDB$RELATIONS R
  SEMI JOIN (
    SELECT *
    FROM RDB$PAGES
    WHERE RDB$PAGES.RDB$PAGE_SEQUENCE > 5
  ) P ON P.RDB$RELATION_ID = R.RDB$RELATION_ID

```

Claro, tal sintaxe não existe em SQL. Ela é demonstrada para uma melhor compreensão do que está acontecendo em termos de junções.

Hash Join (anti)

Este método de acesso pode ser usado para executar subconsultas no predicado `NOT EXIST`, bem como outros predicados que são convertidos em `NOT EXIST (ALL)`. Além disso, sob certas condições, algumas consultas externas unidirecionais também podem ser convertidas nele. Nas versões atuais do Firebird, ele não está implementado.

O algoritmo da anti-junção por hash é o seguinte. O fluxo seguidor é escolhido como o fluxo da

subconsulta, que é lido completamente em um buffer interno. Durante a leitura, uma função hash é aplicada a cada chave de vinculação e o par {hash, ponteiro no buffer} é gravado na tabela hash. Em seguida, o fluxo líder é lido e sua chave de vinculação é sondada na tabela hash. Se nenhuma correspondência for encontrada, o registro do fluxo externo é enviado para a saída. Se uma correspondência for encontrada, passamos para o próximo registro do fluxo líder e assim por diante.

Fusão de Passagem Única (Merge)

A fusão é um algoritmo alternativo para implementar uma junção. Neste caso, os fluxos de entrada são completamente independentes. Para uma execução correta da operação, os fluxos devem ser pré-ordenados pela chave de junção, após o qual uma árvore binária de fusão é construída. Em seguida, durante o fetch, os registros são lidos de ambos os fluxos e suas chaves de vinculação são comparadas quanto à igualdade. Se as chaves coincidirem, o registro é retornado na saída. Em seguida, novos registros são lidos dos fluxos de entrada e o processo se repete. A fusão sempre é executada em uma única passagem, ou seja, cada fluxo de entrada é lido apenas uma vez. Isso é possível graças à disposição ordenada das chaves de vinculação nos fluxos de entrada.

No entanto, este algoritmo não pode ser usado para todos os tipos de junção. Como observado acima, é necessária uma comparação de chaves por igualdade estrita. Assim, uma junção com uma condição de vinculação do tipo (ON MASTER.F > SLAVE.F) não pode ser executada por meio de fusão de passagem única. Para ser justo, vale notar que junções desse tipo não podem ser executadas eficientemente por nenhum método, pois mesmo no caso de usar o algoritmo de junção por loops aninhados, em cada iteração haverá leituras repetidas do fluxo interno.



Em alguns SGBDs, como o Oracle, é possível realizar junção por fusão não apenas para igualdade estrita, ou seja, são possíveis junções com condições de vinculação do tipo (ON MASTER.F > SLAVE.F).

No entanto, este método tem uma vantagem em comparação com o algoritmo de junção por loops aninhados - é permitida a fusão com base em expressões. Por exemplo, uma junção com uma condição de vinculação do tipo (ON MASTER.F + 1 = SLAVE.F + 2) é facilmente executada pelo método de fusão. A razão é clara: não há dependência entre os fluxos e, portanto, não há requisito para usar índices.

O leitor atento pode ter notado que na descrição do algoritmo de fusão não é especificado o modo de operação do método no caso de dependência entre fluxos (junção externa unidirecional). A resposta é simples: para tais junções, este algoritmo não é suportado. Além disso, ele também não é suportado para junções externas completas, semi-junções e anti-junções.



O suporte a junções externas pelo algoritmo está planejado para as próximas versões do servidor.

Este algoritmo é capaz de processar várias condições de vinculação combinadas por AND. Neste caso, os fluxos de entrada são simplesmente ordenados por vários campos. No entanto, no caso de combinar condições de vinculação por OR, o método de fusão não pode ser aplicado.

A principal desvantagem deste algoritmo é a exigência de ordenar ambos os fluxos pelas chaves de junção. Se os fluxos já estiverem ordenados, este algoritmo de junção é o mais barato entre os

outros (NESTED LOOP, HASH JOIN). No entanto, normalmente os fluxos a serem unidos não estão ordenados pelas chaves de junção e, portanto, precisam ser ordenados, o que aumenta drasticamente o custo de execução da junção por este método. Infelizmente, atualmente o otimizador não consegue determinar se os fluxos já estão ordenados pelas chaves de junção e, portanto, este algoritmo pode não ser utilizado nos casos em que seria realmente mais barato.



Esta deficiência está planejada para ser corrigida nas próximas versões do servidor.

Antes do Firebird 3.0, as junções por fusão eram usadas apenas quando era impossível ou não ideal usar o algoritmo de junção por loops aninhados, ou seja, principalmente na ausência de índices para a condição de vinculação ou sua inaplicabilidade, bem como na ausência de dependência entre os fluxos de entrada. A partir do Firebird 3.0, o algoritmo de junção por fusão foi desativado e, em seu lugar, sempre foi usada a junção por hash. A partir do Firebird 5.0, a junção por fusão está disponível novamente. Ela é usada apenas nos casos em que os fluxos a serem unidos são muito grandes e a junção por hash se torna ineficiente (a cardinalidade de todos os fluxos a serem unidos é maior que 1009000 registros, consulte [Junção por Hash \(Hash join\)](#)), bem como na ausência de índices para a condição de vinculação, devido à qual o algoritmo de junção por loops aninhados também seria não ideal.

As seguintes fórmulas são usadas para estimar o custo e a cardinalidade da junção:

$$\text{cost} = \text{cost_1} + \text{cost_2}$$

$$\text{cardinality} = \text{cardinality_1} * \text{cardinality_2} * \text{selectivity}(\text{link})$$

Onde cost_1, cost_2 são o custo de recuperação de dados dos fluxos de entrada ordenados. E como o custo da ordenação externa não é calculado, o custo da junção por fusão também não pode ser estimado corretamente.

No plano de execução Legacy, a junção por fusão é exibida pela palavra "MERGE", seguida entre parênteses por vírgulas descrevendo os fluxos de entrada.

No plano de execução Explain, a junção por fusão é exibida como uma árvore, cuja raiz é rotulada como "Merge Join" e, em seguida, entre parênteses, é especificado o tipo de junção. No nível abaixo, são descritos os fluxos a serem unidos, que são ordenados por uma ordenação externa.

Exemplo 62. Conexão pelo método Merge Join (inner)

```
SELECT *
FROM
  WORD_DICTIONARY WD1
  JOIN WORD_DICTIONARY WD2 ON WD1.PARAMS = WD2.PARAMS
```

```
PLAN MERGE (SORT (WD1 NATURAL), SORT (WD2 NATURAL))
```

```

[cardinality=22307569204, cost=???]
Select Expression
  [cardinality=22307569204, cost=???]
  -> Filter
    [cardinality=22307569204, cost=???]
    -> Merge Join (inner)
      [cardinality=4723000, cost=???]
      -> Sort (record length: 710, key length: 328)
        [cardinality=4723000, cost=???]
        -> Table "WORD_DICTIONARY" as "WD1" Full Scan
      [cardinality=4723000, cost=???]
      -> Sort (record length: 710, key length: 328)
        [cardinality=4723000, cost=4723000]
        -> Table "WORD_DICTIONARY" as "WD2" Full Scan

```



O último exemplo é artificial, porque forçar o otimizador a realizar uma junção pelo método de fusão é bastante difícil.

Full Outer Join

Este tipo de junção não é executado diretamente; em vez disso, ele é decomposto em uma forma equivalente – primeiro, um fluxo é unido a outro usando uma junção externa esquerda (left outer join), depois os fluxos são trocados e uma anti-junção (anti-join) é executada, e os resultados de ambas as junções são combinados.

Como os fluxos em uma junção externa completa são semanticamente equivalentes, o otimizador pode trocá-los dependendo de qual opção é mais barata.

Exemplo 63. Nested Loop para junção externa completa

```

SELECT *
FROM RDB$RELATIONS R
FULL JOIN RDB$RELATION_FIELDS RF ON R.RDB$RELATION_NAME = RF.RDB$RELATION_NAME

```

```

PLAN JOIN (JOIN (RF NATURAL, R INDEX (RDB$INDEX_0)), JOIN (R NATURAL, RF INDEX
(RDB$INDEX_4)))

```

```

[cardinality=3375.0, cost=23980.4]
Select Expression
  [cardinality=3375.0, cost=23980.4]
  -> Full Outer Join
    [cardinality=3200.0, cost=22580.4]
    -> Nested Loop Join (outer)
      [cardinality=2428.0, cost=2428.0]
      -> Table "RDB$RELATION_FIELDS" as "RF" Full Scan
      [cardinality=1.3, cost=4.3]
      -> Filter
        [cardinality=1.3, cost=4.3]

```

```

-> Table "RDB$RELATIONS" as "R" Access By ID
  -> Bitmap
    -> Index "RDB$INDEX_0" Unique Scan
[cardinality=175.0, cost=1400.0]
-> Nested Loop Join (outer)
  [cardinality=350.0, cost=350.0]
  -> Table "RDB$RELATIONS" as "R" Full Scan
  [cardinality=1.0, cost=4.0]
  -> Filter
    [cardinality=9.1, cost=12.1]
    -> Table "RDB$RELATION_FIELDS" as "RF" Access By ID
      -> Bitmap
        -> Index "RDB$INDEX_4" Range Scan (full match)

```

Antes da versão 3.0, o Firebird não conseguia usar índices para junção externa completa, o que tornava esse tipo de junção extremamente ineficiente. Era possível contornar a situação reescrevendo a consulta da seguinte forma.

```

SELECT *
FROM RDB$RELATION_FIELDS RF
LEFT JOIN RDB$RELATIONS R ON R.RDB$RELATION_NAME = RF.RDB$RELATION_NAME
UNION ALL
SELECT *
FROM RDB$RELATIONS R
LEFT JOIN RDB$RELATION_FIELDS RF ON R.RDB$RELATION_NAME = RF.RDB$RELATION_NAME
WHERE RF.RDB$RELATION_NAME IS NULL

```

A segunda parte da consulta é essencialmente uma anti-junção (anti-join). Atualmente, o otimizador faz essa transformação automaticamente para você.

Junção com procedimento armazenado

Ao usar junções internas com um procedimento armazenado, há duas opções:

- os parâmetros de entrada do procedimento armazenado não dependem de outros fluxos;
- os parâmetros de entrada do procedimento armazenado dependem dos fluxos de entrada.

Esses casos são tratados de forma diferente.

- Se o procedimento armazenado não depende de outros fluxos, a ordem e o algoritmo de junção para junções internas (INNER JOIN) dependem do predicado de junção usado e da possibilidade de usar um índice no campo (ou expressão para o campo) da tabela.
 - É usado um predicado de igualdade (ou vários predicados de igualdade combinados com AND) e não há índice no campo (expressão) da tabela – o algoritmo de junção HASH JOIN é usado. O fluxo líder é aquele com menor cardinalidade. Para procedimentos armazenados, a cardinalidade é assumida como 1000;
 - É usado um predicado diferente de = ou IS NOT DISTINCT FROM, ou existe um índice no campo (expressão) da tabela e ele pode ser aplicado – o algoritmo Nested Loop Join é usado, e o procedimento armazenado se torna o fluxo líder. Isso permite executá-lo uma vez, em vez

de executá-lo novamente a cada iteração (afinal, não podemos aplicar um índice a ele).

- Se o procedimento armazenado é unido usando junções externas unilaterais (LEFT/RIGHT JOIN), a ordem da junção é determinada pela ordem da tabela e do procedimento.
- Se o procedimento armazenado depende de outros fluxos (através de parâmetros de entrada), ele se torna o fluxo interno em qualquer tipo de junção. Se a ordem da junção não puder ser reorganizada (table RIGHT JOIN proc(table.field)), ocorre um erro.

Exemplo 64. Junção com procedimento armazenado não correlacionado

```
SELECT *
FROM SP_PEDIGREE(?) P
JOIN HORSE ON HORSE.CODE_HORSE = P.CODE_HORSE
```

```
PLAN JOIN (P NATURAL, HORSE INDEX (PK_HORSE))
```

```
[cardinality=1000.0, cost=4000.0]
Select Expression
  [cardinality=1000.0, cost=4000.0]
  -> Nested Loop Join (inner)
    [cardinality=1000.0, cost=????]
    -> Procedure "SP_PEDIGREE" as "P" Scan
    [cardinality=1.0, cost=4.0]
    -> Filter
      [cardinality=1.0, cost=4.0]
      -> Table "HORSE" Access By ID
        -> Bitmap
          -> Index "PK_HORSE" Unique Scan
```

Agora, vamos alterar o texto da consulta para que não seja possível usar o índice PK_HORSE.

```
SELECT *
FROM SP_PEDIGREE(?) P
JOIN HORSE ON HORSE.CODE_HORSE+0 = P.CODE_HORSE
```

```
PLAN HASH (HORSE NATURAL, P NATURAL)
```

```
[cardinality=585403.4, cost=???)
Select Expression
  [cardinality=585403.4, cost=???)
  -> Filter
    [cardinality=585403.4, cost=???)
    -> Hash Join (inner)
      [cardinality=585403.4, cost=585403.4]
      -> Table "HORSE" Full Scan
```

```
[cardinality=1000.0, cost=????]
-> Record Buffer (record length: 137)
   [cardinality=1000.0, cost=????]
     -> Procedure "SP_PEDIGREE" as "P" Scan
```

Neste caso, a estimativa de cardinalidade da tabela HORSE acabou sendo maior que a estimativa de cardinalidade do procedimento armazenado, portanto a tabela foi escolhida como fluxo líder, e os resultados da execução do procedimento armazenado são hasheados.

Se os parâmetros de entrada do procedimento dependem de outros fluxos, o otimizador detecta essas dependências e ordena os fluxos de forma que o procedimento se torne interno aos fluxos de entrada.

Exemplo 65. Junção com procedimento armazenado correlacionado

```
SELECT *
FROM
  HORSE
  CROSS JOIN SP_PEDIGREE(HORSE.CODE_HORSE) P
WHERE HORSE.CODE_COLOR = ?
```

```
PLAN JOIN (HORSE INDEX (FK_HORSE_COLOR), P NATURAL)
```

```
[cardinality=2377700.0, cost=2618.7]
Select Expression
  [cardinality=2377700.0, cost=2618.7]
    -> Nested Loop Join (inner)
      [cardinality=2377.7, cost=2618.7]
        -> Filter
          [cardinality=2377.7, cost=2618.7]
            -> Table "HORSE" Access By ID
              -> Bitmap
                -> Index "FK_HORSE_COLOR" Range Scan (full match)
          [cardinality=1000.0, cost=???]
            -> Procedure "SP_PEDIGREE" as "P" Scan
```



O Firebird, antes da versão 3.0, não conseguia determinar a dependência dos parâmetros de entrada do procedimento em relação a outros fluxos. Nesse caso, o procedimento era colocado à frente, quando ainda nenhum registro havia sido selecionado da tabela. Consequentemente, no momento da execução, ocorria o erro `isc_no_cur_rec` (“no current record for fetch operation”). Para contornar esse problema, era usada uma indicação explícita da ordem de junção com a sintaxe:

```
SELECT *
FROM
  HORSE
  LEFT JOIN SP_PEDIGREE(HORSE.CODE_HORSE) P ON 1=1
```

```
WHERE HORSE.CODE_COLOR = ?
```

Neste caso, a tabela sempre será lida antes do procedimento e tudo funcionará corretamente.

Junção com expressões de tabela

Expressões de tabela são subconsultas usadas como fontes de dados na consulta principal. Existem dois tipos de expressões de tabela:

- tabelas derivadas (derived table);
- expressões de tabela comuns (common table expression) ou CTE, abreviadamente.

Uma tabela derivada (derived table) é uma expressão de tabela incluída na cláusula FROM da consulta. Tabelas derivadas são consultas entre parênteses. Essa consulta pode receber um alias, após o qual a expressão de tabela pode ser referenciada como uma tabela comum (usada como fonte de dados).

Exemplo 66. Consulta usando uma tabela derivada

```
SELECT
  F.RDB$RELATION_NAME
FROM (
  SELECT
    DISTINCT
    RF.RDB$RELATION_NAME,
    RF.RDB$SYSTEM_FLAG
  FROM
    RDB$RELATION_FIELDS RF
  WHERE RF.RDB$FIELD_NAME STARTING WITH 'RDB$'
) F
WHERE F.RDB$SYSTEM_FLAG = 1
```

Uma expressão de tabela comum (common table expression) ou CTE, abreviadamente – é uma expressão de tabela nomeada, declarada na cláusula WITH. Com uma expressão de tabela comum, assim como com uma tabela derivada, pode-se trabalhar como com uma tabela comum (usá-la como fonte de dados). Expressões de tabela comuns declaradas abaixo podem usar expressões de tabela declaradas acima em seu corpo. As expressões de tabela comuns são divididas em recursivas e não recursivas. Falaremos sobre CTEs recursivas mais tarde, ao considerar o método de acesso correspondente. Expressões de tabela não recursivas têm a seguinte forma:

```
WITH
  <cte_1>, <cte_2>, ... <cte_N>
<main_select_expr>

<cte> ::= _cte_name_ [( <column_aliases> )] AS <select_expr>
```


O exemplo acima pode ser reescrito usando CTE da seguinte forma:

Exemplo 67. Consulta usando CTE

```
WITH
  F AS (
    SELECT
      DISTINCT
        RF.RDB$RELATION_NAME,
        RF.RDB$SYSTEM_FLAG
    FROM
      RDB$RELATION_FIELDS RF
    WHERE RF.RDB$FIELD_NAME STARTING WITH 'RDB$'
  )
SELECT
  F.RDB$RELATION_NAME
FROM F
WHERE F.RDB$SYSTEM_FLAG = 1
```

Como as junções se comportarão ao usar expressões de tabela? Isso dependerá do conteúdo da expressão de tabela. Nos casos mais simples, uma consulta com expressão de tabela pode ser transformada em uma consulta equivalente sem o uso da expressão de tabela. Em outras palavras, consultas simples são expandidas para tabelas base. Em casos mais complexos, a expressão de tabela não pode ser expandida para tabelas base. Expressões de tabela simples contêm apenas operações de junção interna de fluxos e condições de filtragem. A adição de ordenação, DISTINCT, cálculo de agregados, agrupamento, cálculo de funções de janela, UNION e contadores FIRST/SKIP, junções externas transformam a expressão de tabela em uma complexa.

O otimizador se comporta de forma diferente para junções internas e externas com expressões de tabela complexas. No caso de uma junção interna de uma tabela com uma expressão de tabela, o otimizador tenta escolher um plano de execução que não leve à reexecução da consulta dentro da expressão de tabela. Assim, a junção da expressão de tabela com a tabela é executada por um dos dois algoritmos:

- Nested Loop Join. Neste caso, a expressão de tabela é escolhida como fluxo líder, e a tabela como fluxo interno. Este algoritmo é escolhido apenas se for impossível usar um índice no campo da tabela para o predicado de junção ou se o predicado de junção não puder ser usado para o algoritmo Hash Join.
- Hash Join. Usado se o predicado de junção for uma igualdade (=) ou equivalência (IS NOT DISTINCT FROM), e se não houver possibilidade de usar um índice no(s) campo(s) da tabela. Neste caso, a cardinalidade da expressão de tabela e da tabela é estimada. O fluxo de dados com menor cardinalidade é hashado e se torna o fluxo interno.

Em ambos os casos, a consulta dentro da expressão de tabela é otimizada separadamente, ou seja, seu plano não depende do que está sendo unido a ela. Predicados de filtragem independentes de outros fluxos, que são aplicados à expressão de tabela, serão empurrados (pushed) para dentro da expressão de tabela sempre que possível.

Para junções externas unidirecionais, a ordem da junção é sempre determinada pelo texto da

consulta, e a própria junção é executada pelo algoritmo Nested Loop Join. Se uma expressão de tabela complexa for escolhida como fluxo conduzido, ela será reexecutada para cada registro do fluxo condutor. Nesse caso, os predicados aplicáveis à expressão de tabela, incluindo a condição de junção, serão empurrados (pushed) para dentro da expressão de tabela sempre que possível.

Vejamos um exemplo de junção com uma expressão de tabela simples.

Exemplo 68. Expansão de CTE para tabelas base

```
WITH
  FIELDS AS (
    SELECT
      RF.RDB$RELATION_NAME,
      RF.RDB$FIELD_NAME,
      F.RDB$FIELD_TYPE
    FROM
      RDB$RELATION_FIELDS RF
    JOIN RDB$FIELDS F ON F.RDB$FIELD_NAME = RF.RDB$FIELD_SOURCE
  )
SELECT
  R.RDB$RELATION_NAME,
  FIELDS.RDB$FIELD_NAME,
  FIELDS.RDB$FIELD_TYPE
FROM
  RDB$RELATIONS R
JOIN FIELDS ON FIELDS.RDB$RELATION_NAME = R.RDB$RELATION_NAME
```

```
PLAN JOIN (R NATURAL, FIELDS RF INDEX (RDB$INDEX_4), FIELDS F INDEX (RDB$INDEX_2))
```

```
[cardinality=2182.9, cost=4858.3]
Select Expression
  [cardinality=2182.9, cost=4858.3]
  -> Nested Loop Join (inner)
    [cardinality=233.7, cost=233.7]
    -> Table "RDB$RELATIONS" as "R" Full Scan
    [cardinality=9.34, cost=12.34]
    -> Filter
      [cardinality=9.34, cost=12.34]
      -> Table "RDB$RELATION_FIELDS" as "FIELDS RF" Access By ID
        -> Bitmap
          -> Index "RDB$INDEX_4" Range Scan (full match)
    [cardinality=1.0, cost=4.0]
    -> Filter
      [cardinality=1.0, cost=4.0]
      -> Table "RDB$FIELDS" as "FIELDS F" Access By ID
        -> Bitmap
          -> Index "RDB$INDEX_2" Unique Scan
```

Neste caso, a CTE FIELDS foi expandida para as tabelas base e a otimização ocorreu como se

simplesmente tivéssemos unido as tabelas dentro da CTE a RDB\$RELATIONS R. Agora, vamos adicionar uma ordenação dentro da CTE (ela não altera a cardinalidade).

Exemplo 69. CTEs complexas que não podem ser expandidas para tabelas base

```
WITH
  FIELDS AS (
    SELECT
      RF.RDB$RELATION_NAME,
      RF.RDB$FIELD_NAME,
      F.RDB$FIELD_TYPE
    FROM
      RDB$RELATION_FIELDS RF
    JOIN RDB$FIELDS F ON F.RDB$FIELD_NAME = RF.RDB$FIELD_SOURCE
    ORDER BY F.RDB$FIELD_TYPE
  )
SELECT
  R.RDB$RELATION_NAME,
  FIELDS.RDB$FIELD_NAME,
  FIELDS.RDB$FIELD_TYPE
FROM
  RDB$RELATIONS R
JOIN FIELDS ON FIELDS.RDB$RELATION_NAME = R.RDB$RELATION_NAME
```

```
PLAN JOIN (SORT (JOIN (FIELDS RF NATURAL, FIELDS F INDEX (RDB$INDEX_2))), R INDEX
(RDB$INDEX_0))
```

```
Select Expression
[cardinality=2428.4, cost=21855.6]
-> Nested Loop Join (inner)
  [cardinality=2428.4, cost=12142.0]
  -> Refetch
    [cardinality=2428.4, cost=12142.0]
    -> Sort (record length: 44, key length: 8)
      [cardinality=2428.4, cost=12142.0]
      -> Nested Loop Join (inner)
        [cardinality=2428.4, cost=2428.4]
        -> Table "RDB$RELATION_FIELDS" as "FIELDS RF" Full Scan
        [cardinality=1.0, cost=4.0]
        -> Filter
          [cardinality=1.0, cost=4.0]
          -> Table "RDB$FIELDS" as "FIELDS F" Access By ID
            -> Bitmap
              -> Index "RDB$INDEX_2" Unique Scan
        [cardinality=1.0, cost=4.0]
        -> Filter
          [cardinality=1.0, cost=4.0]
          -> Table "RDB$RELATIONS" as "R" Access By ID
            -> Bitmap
              -> Index "RDB$INDEX_0" Unique Scan
```

Aqui, o plano mudou drasticamente. Primeiro, a consulta dentro da CTE foi executada, e somente depois os outros fluxos foram unidos a ela. Este plano foi escolhido porque existe um índice para o campo `RDB$RELATION_NAME` da tabela `RDB$RELATIONS`. A ordem da junção não mudará, independentemente da proporção do número de registros na tabela e na expressão de tabela.

No entanto, se não houver um índice adequado para executar a junção eficientemente pelo método `Nested Loop Join` e for possível executar essa junção com o algoritmo `Hash Join`, então este último será escolhido. Se o algoritmo de junção `Hash Join` for escolhido, a cardinalidade da tabela e da expressão de tabela é avaliada, e o fluxo menor será escolhido para o hashing e se tornará o fluxo conduzido.

```
WITH
  FIELDS AS (
    SELECT
      RF.RDB$RELATION_NAME,
      RF.RDB$FIELD_NAME,
      F.RDB$FIELD_TYPE
    FROM
      RDB$RELATION_FIELDS RF
    JOIN RDB$FIELDS F ON F.RDB$FIELD_NAME = RF.RDB$FIELD_SOURCE
    ORDER BY F.RDB$FIELD_TYPE
  )
SELECT
  R.RDB$RELATION_NAME,
  FIELDS.RDB$FIELD_NAME,
  FIELDS.RDB$FIELD_TYPE
FROM
  RDB$RELATIONS R
JOIN FIELDS ON FIELDS.RDB$RELATION_NAME = R.RDB$RELATION_NAME || ' '
```

```
PLAN HASH (SORT (JOIN (FIELDS RF NATURAL, FIELDS F INDEX (RDB$INDEX_2))), R NATURAL)
```

Select Expression

-> Filter

[cardinality=799.0, cost=18386.2]

-> Hash Join (inner) (keys: 1, total key length: 252)

[cardinality=2428.4, cost=12142.0]

-> Refetch

[cardinality=2428.4, cost=12142.0]

-> Sort (record length: 44, key length: 8)

[cardinality=2428.4, cost=12142.0]

-> Nested Loop Join (inner)

[cardinality=2428.4, cost=2428.4]

-> Table "RDB\$RELATION_FIELDS" as "FIELDS" "RF" Full Scan

[cardinality=1.0, cost=4.0]

-> Filter

[cardinality=1.0, cost=4.0]

-> Table "RDB\$FIELDS" as "FIELDS" "F" Access By ID

-> Bitmap

-> Index "RDB\$INDEX_2" Range Scan (full match)

[cardinality=322.9, cost=645.8]

-> Record Buffer (record length: 273)

```
[cardinality=322.9, cost=322.9]
-> Table "RDB$RELATIONS" as "R" Full Scan
```

Se você não estiver satisfeito com esta ordem de junção, você sempre pode usar a dica de ordem de junção via LEFT JOIN com filtragem subsequente IS NOT NULL.

Exemplo 70. CTEs complexas, unidas por LEFT JOIN

```
WITH
  FIELDS AS (
    SELECT
      RF.RDB$RELATION_NAME,
      RF.RDB$FIELD_NAME,
      F.RDB$FIELD_TYPE
    FROM
      RDB$RELATION_FIELDS RF
    JOIN RDB$FIELDS F ON F.RDB$FIELD_NAME = RF.RDB$FIELD_SOURCE
    ORDER BY F.RDB$FIELD_TYPE
  )
SELECT
  R.RDB$RELATION_NAME,
  FIELDS.RDB$FIELD_NAME,
  FIELDS.RDB$FIELD_TYPE
FROM
  RDB$RELATIONS R
LEFT JOIN FIELDS ON FIELDS.RDB$RELATION_NAME = R.RDB$RELATION_NAME
WHERE FIELDS.RDB$RELATION_NAME IS NOT NULL
```

```
PLAN JOIN (R NATURAL, SORT (JOIN (FIELDS RF INDEX (RDB$INDEX_4), FIELDS F INDEX
(RDB$INDEX_2))))
```

```
[cardinality=1091.45, cost=12082.3]
Select Expression
  [cardinality=1091.45, cost=12082.3]
  -> Filter
    [cardinality=2182.9, cost=12082.3]
    -> Nested Loop Join (outer)
      [cardinality=233.7, cost=233.7]
      -> Table "RDB$RELATIONS" as "R" Full Scan
      [cardinality=9.34, cost=50.7]
      -> Refetch
        [cardinality=9.34, cost=50.7]
        -> Sort (record length: 44, key length: 8)
          [cardinality=9.34, cost=50.7]
          -> Nested Loop Join (inner)
            [cardinality=9.34, cost=12.34]
            -> Filter
              [cardinality=9.34, cost=12.34]
              -> Table "RDB$RELATION_FIELDS" as "FIELDS RF" Access By ID
                -> Bitmap
                  -> Index "RDB$INDEX_4" Range Scan (full match)
```

```

[cardinality=1.0, cost=4.0]
-> Filter
    [cardinality=1.0, cost=4.0]
    -> Table "RDB$FIELDS" as "FIELDS F" Access By ID
        -> Bitmap
            -> Index "RDB$INDEX_2" Unique Scan

```

Além de tabelas derivadas (derived table) independentes de fluxos externos, o padrão prevê tabelas derivadas dependentes de fluxos externos. Para usar tais tabelas derivadas, é necessário usar a palavra-chave **LATERAL** antes da expressão de tabela. Expressões de tabela comuns dependentes de fluxos externos não são possíveis.

Para tabelas derivadas **LATERAL**, fazem sentido os seguintes tipos de junção: **CROSS JOIN** e **LEFT JOIN**. A sintaxe permite outros tipos de junção. Uma característica da junção com lateral derived table é que elas só podem ser executadas pelo algoritmo de busca recursiva. Outra característica diz respeito à execução do **CROSS JOIN**. O fato é que, devido à dependência da tabela derivada **LATERAL** em relação aos fluxos externos, ela não pode ser definida como fluxo condutor nas junções. O otimizador reconhece a presença de dependências e escolhe a ordem correta de junção.

Exemplo 71. CROSS JOIN LATERAL

```

SELECT
  R.RDB$RELATION_NAME,
  FIELDS.RDB$FIELD_NAME
FROM
  RDB$RELATIONS R
  CROSS JOIN LATERAL (
    SELECT RF.RDB$FIELD_NAME
    FROM RDB$RELATION_FIELDS RF
    WHERE RF.RDB$RELATION_NAME = R.RDB$RELATION_NAME
    ORDER BY RF.RDB$FIELD_POSITION
    FETCH FIRST ROW ONLY
  ) FIELDS

```

```

PLAN JOIN (R NATURAL, SORT (FIELDS RF INDEX (RDB$INDEX_4)))

```

```

[cardinality=233.7, cost=3351.26]
Select Expression
  [cardinality=233.7, cost=3351.26]
  -> Nested Loop Join (inner)
    [cardinality=233.7, cost=233.7]
    -> Table "RDB$RELATIONS" as "R" Full Scan
    [cardinality=1.0, cost=12.34]
    -> First N Records
      [cardinality=9.34, cost=12.34]
      -> Refetch
        [cardinality=9.34, cost=12.34]
        -> Sort (record length: 28, key length: 8)
          [cardinality=9.34, cost=12.34]

```

```

-> Filter
   [cardinality=9.34, cost=12.34]
   -> Table "RDB$RELATION_FIELDS" as "FIELDS RF" Access By ID
       -> Bitmap
           -> Index "RDB$INDEX_4" Range Scan (full match)

```

Conexão com visões

Uma visão (view) é uma tabela virtual (lógica) que representa uma consulta nomeada (sinônimo para uma consulta), que será substituída como uma expressão de tabela quando a visão for utilizada.

Do ponto de vista do otimizador, as conexões com visões não são diferentes das conexões com expressões de tabela descritas acima. A única diferença significativa é que, ao contrário das tabelas derivadas, as visões não podem ser correlacionadas.

3.4.2. Uniões (Union)

O nome do método de acesso fala por si. Este método de acesso executa a operação SQL de união (union). Existem dois modos de execução desta operação: ALL e DISTINCT. No primeiro caso, a implementação é trivial: este método simplesmente lê o primeiro fluxo de entrada e o emite na saída; ao receber EOF dele, começa a ler o segundo fluxo de entrada e assim por diante. No caso de DISTINCT, é necessário eliminar duplicatas completas de registros presentes no resultado da união. Para isso, um filtro de ordenação é colocado na saída do método de união, operando no modo “truncante” em todos os campos.

O custo de execução da união é igual ao custo total de todos os fluxos de entrada; a cardinalidade também é obtida por soma. No modo DISTINCT, a cardinalidade resultante é dividida por 10.

No plano de execução Legacy, a união é representada por planos separados para cada fluxo de entrada.

No plano de execução Explain, a união é representada como uma árvore, cuja raiz é rotulada como “Union”. Um nível abaixo, os fluxos que estão sendo unidos são descritos.

Exemplo 72. União de fluxos no modo ALL

```

SELECT RDB$RELATION_ID
FROM RDB$RELATIONS
WHERE RDB$SYSTEM_FLAG = 1
UNION ALL
SELECT RDB$PROCEDURE_ID
FROM RDB$PROCEDURES
WHERE RDB$SYSTEM_FLAG = 1

```

```

PLAN (RDB$RELATIONS NATURAL, RDB$PROCEDURES NATURAL)

```

```

[cardinality=90.33, cost=873.3]
Select Expression
  [cardinality=90.33, cost=873.3]
  -> Union
    [cardinality=35.06, cost=350.6]
    -> Filter
      [cardinality=350.6, cost=350.6]
      -> Table "RDB$RELATIONS" Full Scan
    [cardinality=55.27, cost=552.7]
    -> Filter
      [cardinality=552.7, cost=552.7]
      -> Table "RDB$PROCEDURES" Full Scan

```

Exemplo 73. União de fluxos no modo DISTINCT

```

SELECT RDB$RELATION_ID
FROM RDB$RELATIONS
WHERE RDB$SYSTEM_FLAG = 1
UNION
SELECT RDB$PROCEDURE_ID
FROM RDB$PROCEDURES
WHERE RDB$SYSTEM_FLAG = 1

```

```
PLAN SORT (RDB$RELATIONS NATURAL, RDB$PROCEDURES NATURAL)
```

```

[cardinality=9.033, cost=873.3]
Select Expression
  [cardinality=9.033, cost=873.3]
  -> Unique Sort (record length: 44, key length: 8)
    [cardinality=90.33, cost=873.3]
    -> Union
      [cardinality=35.06, cost=350.6]
      -> Filter
        [cardinality=350.6, cost=350.6]
        -> Table "RDB$RELATIONS" Full Scan
      [cardinality=55.27, cost=552.7]
      -> Filter
        [cardinality=552.7, cost=552.7]
        -> Table "RDB$PROCEDURES" Full Scan

```

Materialização de expressões não determinísticas

As expressões de tabela no Firebird têm uma característica desagradável, a saber: ao acessar colunas de uma expressão de tabela que se referem a expressões, essas expressões são calculadas cada vez que a coluna da expressão de tabela é mencionada. Isso é facilmente demonstrado usando funções não determinísticas.

Tente executar a seguinte consulta:

```
WITH
  T AS (
    SELECT GEN_UUID() AS UUID
    FROM RDB$DATABASE
  )
SELECT
  UUID_TO_CHAR(UUID) AS ID1,
  UUID_TO_CHAR(UUID) AS ID2
FROM T
```

O resultado será algo como

ID1	ID2
=====	=====
3C8CA94D-9D05-4A49-8788-1D9024193C4E	D5E86F5C-BF48-4AA3-AB6D-3EF9C9B58B52

Embora você esperasse que os valores nas colunas fossem iguais.

Isso também se manifesta ao ordenar por referência a uma coluna. Por exemplo:

```
SELECT GEN_ID(SEQ_SOME, 1) AS ID
FROM RDB$DATABASE
ORDER BY 1
```

A sequência aumentará não em 1, como você esperava, mas em 2.

Existe um truque que permite “materializar” os resultados dos cálculos de expressões em expressões de tabela. Para isso, basta fazer um UNION com uma consulta que retorna 0 registros, naturalmente o número total de colunas de ambas as consultas deve ser o mesmo. Vamos tentar reescrever a primeira consulta da seguinte forma:

```
WITH
  T AS (
    SELECT GEN_UUID() AS UUID
    FROM RDB$DATABASE
    UNION ALL
    SELECT NULL FROM RDB$DATABASE WHERE FALSE
  )
SELECT
  UUID_TO_CHAR(UUID) AS ID1,
  UUID_TO_CHAR(UUID) AS ID2
FROM T
```

Agora os valores ID1 e ID2 serão iguais.

ID1	ID2
-----	-----

```
=====
9599C281-DD96-4CC7-9C05-6259CCAB467F  9599C281-DD96-4CC7-9C05-6259CCAB467F
=====
```

Materialização de subconsultas

E se uma subconsulta for usada como coluna de uma expressão de tabela? A subconsulta pode ser pesada o suficiente para não querer executá-la repetidamente. Felizmente, nesse caso, o otimizador faz todo o trabalho para nós. Tente executar a seguinte consulta:

```
WITH T
  AS (
    SELECT
      (SELECT COUNT(*)
       FROM RDB$RELATION_FIELDS RF
       WHERE RF.RDB$RELATION_NAME = R.RDB$RELATION_NAME) AS CNT
    FROM RDB$RELATIONS R
  )
SELECT
  SUM(T.CNT) AS CNT,
  SUM(T.CNT) * 1e0 / COUNT(*) AS AVG_CNT
FROM T
```

Apesar da menção repetida de T.CNT, a subconsulta será executada uma única vez para cada registro de RDB\$RELATIONS.

No plano Explain, você verá o seguinte:

```
Sub-query
-> Singularity Check
   -> Aggregate
       -> Filter
           -> Table "RDB$RELATION_FIELDS" as "T RF" Access By ID
               -> Bitmap
                   -> Index "RDB$INDEX_4" Range Scan (full match)
Select Expression
-> Aggregate
   -> Materialize
       -> Table "RDB$RELATIONS" as "T R" Full Scan
```

Aqui, “Materialize” denota a materialização dos resultados da subconsulta para cada registro de RDB\$RELATIONS. Isso não é um método de acesso separado, mas uma união interna (UNION) dentro da expressão de tabela com um conjunto vazio. Ou seja, aconteceu exatamente o que foi descrito acima sobre a materialização de expressões não determinísticas.

3.4.3. Recursão

Este método de acesso é usado para executar expressões de tabela recursivas (CTEs recursivos). O corpo de um CTE recursivo é uma consulta com UNION ALL, que combina uma ou mais subconsultas chamadas elementos âncora. Além dos elementos âncora, há uma ou mais subconsultas recursivas, chamadas elementos recursivos. Essas subconsultas recursivas referem-se ao próprio CTE

recursivo. Portanto, temos uma ou mais subconsultas âncora e uma ou mais subconsultas recursivas, unidas por UNION ALL.

A essência da recursão é simples – primeiro, as subconsultas não recursivas são executadas e unidas, e para cada registro da parte não recursiva, o conjunto de dados é complementado com registros da parte recursiva, que pode usar o resultado obtido na etapa anterior. A recursão termina quando todas as partes recursivas não retornam nenhum registro.

Ao executar CTEs recursivos, há outra característica – nenhum predicado da consulta externa pode ser empurrado para dentro do CTE.

A cardinalidade e o custo da recursão são muito difíceis de estimar. O otimizador considera que a cardinalidade é igual à soma das cardinalidades das partes não recursivas, multiplicada pela cardinalidade adicionada pelas junções (joins) na parte recursiva. O custo é a soma do custo de selecionar todos os registros das partes recursivas e não recursivas. Vale notar que essa estimativa às vezes está longe da realidade.

No plano de execução Legacy, a recursão não é exibida.

No plano de execução Explain, a recursão é exibida como uma árvore, cuja raiz é rotulada como “Recursion”. Um nível abaixo, os fluxos que estão sendo unidos são descritos. A referência ao próprio CTE recursivo dentro do CTE não é descrita como um fluxo de dados separado.

Exemplo 74. Consulta recursiva mais simples

```
WITH RECURSIVE
  R(N) AS (
    SELECT 1 FROM RDB$DATABASE
    UNION ALL
    SELECT R.N + 1 FROM R
  )
SELECT N FROM R
```

```
PLAN (R RDB$DATABASE NATURAL, )
```

```
[cardinality=1.0, cost=1.0]
Select Expression
  [cardinality=1.0, cost=1.0]
  -> Recursion
    [cardinality=1.0, cost=1.0]
    -> Table "RDB$DATABASE" as "R RDB$DATABASE" Full Scan
```

A consulta descrita acima tem uma desvantagem significativa. Ela realiza uma recursão “infinita”. Mas isso é na teoria, pois na prática o Firebird limita a profundidade da recursão ao valor 1024. Vamos corrigir isso.

Exemplo 75. Consulta recursiva com limitação de profundidade

```

WITH RECURSIVE
  R(N) AS (
    SELECT 1 FROM RDB$DATABASE
    UNION ALL
    SELECT R.N + 1 FROM R WHERE R.N < 10
  )
SELECT N FROM R

```

```
PLAN (R RDB$DATABASE NATURAL, )
```

```

[cardinality=1.0, cost=1.0]
Select Expression
  [cardinality=1.0, cost=1.0]
  -> Recursion
    [cardinality=1.0, cost=1.0]
    -> Table "RDB$DATABASE" as "R RDB$DATABASE" Full Scan
    [cardinality=1.0, cost=1.0]
    -> Filter (preliminary)

```

Aqui, no plano explain, a parte recursiva se torna visível, que é filtrada, mas o fluxo em si não é mostrado. Como o membro recursivo não se junta a outras fontes de dados, a cardinalidade e o custo são retirados da parte não recursiva. Aqui, a estimativa de cardinalidade é 1, embora na realidade 10 registros serão retornados.

A parte recursiva pode ser mais complexa e conter junções (apenas internas) com outras fontes de dados.

Exemplo 76. Consulta recursiva com junção por índice não único

```

WITH RECURSIVE
  R AS (
    SELECT
      DEPT_NO,
      DEPARTMENT,
      HEAD_DEPT
    FROM DEPARTMENT
    WHERE HEAD_DEPT IS NULL
    UNION ALL
    SELECT
      DEPARTMENT.DEPT_NO,
      DEPARTMENT.DEPARTMENT,
      DEPARTMENT.HEAD_DEPT
    FROM R JOIN DEPARTMENT ON DEPARTMENT.HEAD_DEPT = R.DEPT_NO
  )
SELECT * FROM R

```

```
PLAN (R DEPARTMENT INDEX (RDB$FOREIGN6), R DEPARTMENT INDEX (RDB$FOREIGN6))
```

```
[cardinality=6.890625, cost=20.390625]
Select Expression
  [cardinality=6.890625, cost=20.390625]
  -> Recursion
    [cardinality=2.625, cost=5.625]
    -> Filter
      [cardinality=2.625, cost=5.625]
      -> Table "DEPARTMENT" as "R DEPARTMENT" Access By ID
        -> Bitmap
          -> Index "RDB$FOREIGN6" Range Scan (full match)
    [cardinality=2.625, cost=5.625]
    -> Filter
      [cardinality=2.625, cost=5.625]
      -> Table "DEPARTMENT" as "R DEPARTMENT" Access By ID
        -> Bitmap
          -> Index "RDB$FOREIGN6" Range Scan (full match)
```

Aqui, na parte recursiva da consulta, é realizada uma junção com a tabela DEPARTMENT por um índice não único, e é por isso que as cardinalidades são multiplicadas (para cada registro do membro não recursivo, são unidos 2.625 registros do membro recursivo). Na realidade, serão selecionados 21 registros.

Vamos tentar desdobrar a recursão.

Exemplo 77. Consulta recursiva com junção por índice único

```
WITH RECURSIVE
  R AS (
    SELECT
      DEPT_NO,
      DEPARTMENT,
      HEAD_DEPT
    FROM DEPARTMENT
    WHERE DEPT_NO = '672'
    UNION ALL
    SELECT
      DEPARTMENT.DEPT_NO,
      DEPARTMENT.DEPARTMENT,
      DEPARTMENT.HEAD_DEPT
    FROM R JOIN DEPARTMENT ON DEPARTMENT.DEPT_NO = R.HEAD_DEPT
  )
SELECT * FROM R
```

```
PLAN (R DEPARTMENT INDEX (RDB$PRIMARY5), R DEPARTMENT INDEX (RDB$PRIMARY5))
```

```

[cardinality=1.0, cost=8.0]
Select Expression
  [cardinality=1.0, cost=8.0]
  -> Recursion
    [cardinality=1.0, cost=4.0]
    -> Filter
      [cardinality=1.0, cost=4.0]
      -> Table "DEPARTMENT" as "R DEPARTMENT" Access By ID
        -> Bitmap
          -> Index "RDB$PRIMARY5" Unique Scan
    [cardinality=1.0, cost=4.0]
    -> Filter
      [cardinality=1.0, cost=4.0]
      -> Table "DEPARTMENT" as "R DEPARTMENT" Access By ID
        -> Bitmap
          -> Index "RDB$PRIMARY5" Unique Scan

```

Aqui, na parte recursiva da consulta, é realizada uma junção com a tabela DEPARTMENT por um índice único, portanto a cardinalidade total não aumenta (para 1 registro do membro não recursivo, é unido 1 registro do membro recursivo). Na prática, serão retornados 4 registros.

3.5. Estratégias de Otimização

No processo de análise dos métodos de acesso do Firebird, descobrimos que as fontes de dados podem ser em pipeline (streaming) ou bufferizadas. Uma fonte de dados em pipeline emite registros durante a leitura de seus fluxos de entrada, enquanto uma fonte bufferizada primeiro precisa ler todos os registros de seus fluxos de entrada e só então poderá emitir o primeiro registro em sua saída.

A maioria dos métodos de acesso são em pipeline. Os métodos de acesso bufferizados incluem a ordenação externa (SORT) e o buffer de registros (Record Buffer). Além disso, alguns métodos de acesso exigem o uso de fontes de dados bufferizadas. Para a junção por hash (Hash Join) é necessária bufferização na construção da tabela hash. A junção por merge (Merge Join) requer que os fluxos de entrada estejam ordenados pelas chaves de junção, para o que é usada a ordenação externa. Para o cálculo de funções de janela (Window) também é necessária bufferização.

No Firebird, o otimizador pode operar em dois modos:

- **FIRST ROWS** — o otimizador constrói o plano de consulta para recuperar mais rapidamente apenas as primeiras linhas do resultado da consulta;
- **ALL ROWS** — o otimizador constrói o plano de consulta para recuperar mais rapidamente todas as linhas do resultado da consulta.

Na maioria dos casos, a estratégia de otimização ALL ROWS é necessária. No entanto, se você tiver aplicativos com grades de dados que exibem apenas as primeiras linhas do resultado, e as demais são recuperadas conforme necessário, então a estratégia FIRST ROWS pode ser mais preferível, pois reduz o tempo de resposta.

Ao usar a estratégia FIRST ROWS, o otimizador tenta substituir métodos de acesso bufferizados por

alternativas em pipeline, se possível e mais barato em termos de custo de recuperação da primeira linha. Ou seja, Hash/Merge join são substituídos por Nested Loop Join, e a ordenação externa (Sort) é substituída por navegação via índice.

Por padrão, é usada a estratégia de otimização especificada no parâmetro `OptimizeForFirstRows` do arquivo de configuração `firebird.conf` ou `database.conf`. `OptimizeForFirstRows = false` corresponde à estratégia `ALL ROWS`, `OptimizeForFirstRows = true` corresponde à estratégia `FIRST ROWS`.

A estratégia de otimização pode ser sobrescrita diretamente no texto da consulta SQL usando a cláusula `OPTIMIZE FOR`. Além disso, o uso de contadores `first/skip` em uma consulta muda implicitamente a estratégia de otimização para `FIRST ROWS`. Vamos comparar os planos de consulta que usam diferentes estratégias de otimização.

Exemplo 78. Consulta com estratégia de otimização ALL ROWS

```
SELECT
  R.RDB$RELATION_NAME,
  P.RDB$PAGE_SEQUENCE
FROM
  RDB$RELATIONS R
  JOIN RDB$PAGES P ON P.RDB$RELATION_ID = R.RDB$RELATION_ID
OPTIMIZE FOR ALL ROWS
```

```
PLAN HASH (P NATURAL, R NATURAL)
```

```
Select Expression
-> Filter
    -> Hash Join (inner)
        -> Table "RDB$PAGES" as "P" Full Scan
        -> Record Buffer (record length: 281)
            -> Table "RDB$RELATIONS" as "R" Full Scan
```

Exemplo 79. Consulta com estratégia de otimização FIRST ROWS

```
SELECT
  R.RDB$RELATION_NAME,
  P.RDB$PAGE_SEQUENCE
FROM
  RDB$RELATIONS R
  JOIN RDB$PAGES P ON P.RDB$RELATION_ID = R.RDB$RELATION_ID
OPTIMIZE FOR FIRST ROWS
```

```
PLAN JOIN (P NATURAL, R INDEX (RDB$INDEX_1))
```

```
Select Expression
```

```
-> Nested Loop Join (inner)
  -> Table "RDB$PAGES" as "P" Full Scan
  -> Filter
    -> Table "RDB$RELATIONS" as "R" Access By ID
    -> Bitmap
      -> Index "RDB$INDEX_1" Range Scan (full match)
```

3.6. Conclusão

Acima, examinamos todos os tipos de operações que participam da execução de consultas SQL. Para cada algoritmo, é fornecida uma descrição detalhada e exemplos, incluindo a detalhamento da árvore de execução da consulta (plano Explain). Vale a pena notar que o Firebird implementa não tantos métodos de acesso em comparação com os concorrentes, no entanto, muitas vezes isso é explicado pelas peculiaridades da arquitetura ou pela eficiência comparativamente maior dos métodos existentes.

Capítulo 4. Transformação de Consultas

No capítulo anterior, examinamos detalhadamente os métodos de acesso utilizados pelo otimizador do Firebird. Além disso, aprendemos a ler o plano de execução (Explain) de uma consulta e analisá-lo para entender quais métodos de acesso foram usados para uma determinada consulta. Agora é hora de falar sobre algumas “transformações de consultas” que desempenham um papel importante no processo de otimização. As próprias transformações não são exibidas no plano de execução, mas o resultado de sua aplicação será visível.

O que se entende por transformação de consulta? O texto da consulta em si nunca é convertido em outro texto. Durante a preparação da consulta, ela é analisada sintaticamente e convertida em uma representação binária chamada BLR (Binary Language Representation). Em seguida, a árvore de execução da consulta é construída com base nessa representação. Parte das transformações ocorre no nível da análise do SQL e de sua conversão para BLR, outra parte — no nível da árvore de execução.

Por que as transformações são necessárias? A linguagem SQL é bastante diversificada e muitas vezes permite resolver as mesmas tarefas de várias maneiras diferentes; as transformações permitem unificar a representação interna das consultas. Outro objetivo importante das transformações de consultas é levar a consulta a uma forma em que o otimizador possa considerar o maior número possível de opções de execução e avaliá-las em termos de custo de execução.

Existe um número considerável de transformações que modificam a consulta original. Eu destaco os seguintes tipos de transformações:

- transformações de predicados de filtragem;
- transformação de subconsultas;
- transformação de junções (JOINS);
- fusão de expressões de tabela com a consulta principal;
- empurramento de predicados de filtragem para dentro de expressões de tabela.

A fusão de expressões de tabela com a consulta principal, bem como o empurramento de predicados de filtragem para dentro de expressões de tabela, são descritos em [Junção com expressões de tabela](#) e [Conexão com visões](#). Os outros tipos de transformações serão abordados neste capítulo.

4.1. Transformação de Predicados de Filtragem

No Firebird, existe uma grande variedade de predicados de filtragem. Não vamos examinar todas as transformações de predicados, mas nos limitaremos àquelas que afetam a otimização.

4.1.1. Transformação do Predicado LIKE

Em geral, o predicado LIKE não pode ser usado para varredura indexada. Ou seja, a consulta

```
SELECT *
```

```
FROM T
WHERE field LIKE ?
```

não usará varredura indexada, mesmo que exista um índice para o campo field. A razão é que o plano da consulta é formado na fase de sua preparação, quando o valor do parâmetro de substituição ainda não é conhecido. Como você já sabe, a busca em um índice pode ser realizada pela chave inteira ou por sua substring (subchave). A busca por subchave é permitida apenas para a parte inicial da chave (por exemplo, STARTING WITH ou o uso de não todos os segmentos de um índice composto). No entanto, como parâmetro da consulta, pode chegar um padrão que permite realizar a busca dentro da string, e a busca dentro da chave do índice nas versões atuais do Firebird é impossível.

Se, em vez do parâmetro da consulta, for especificado um literal de string concreto, as seguintes transformações são aplicadas:

- se o literal de string começar com os caracteres curinga % e _, nenhuma transformação ocorre;
- se o literal de string contiver os caracteres curinga % e _, um predicado STARTING WITH é adicionado à condição de busca, com um literal que copia o literal original desde o início da string até o primeiro caractere curinga ou o final do literal de string.

Naturalmente, essas transformações são feitas considerando o caractere de escape na cláusula ESCAPE, se presente.

Vamos ver como o predicado LIKE funciona com literais usando exemplos. Para demonstração, é usada o banco de dados employee.

Exemplo 80. Usando LIKE com literal sem caracteres curinga

```
SET EXPLAIN ON;

SELECT
  COUNT(*)
FROM
  COUNTRY
WHERE COUNTRY.COUNTRY LIKE 'Austria';
```

A saída será a seguinte:

```
Select Expression
-> Aggregate
  -> Filter
    -> Table "COUNTRY" Access By ID
      -> Bitmap
        -> Index "RDB$PRIMARY1" Range Scan (full match)

COUNT
=====
1
```

Aqui, o predicado LIKE é usado com um literal que não contém caracteres curinga. De acordo com a regra descrita acima, esta consulta é equivalente à seguinte forma:

```
SELECT
  COUNT(*)
FROM
  COUNTRY
WHERE COUNTRY.COUNTRY LIKE 'Austria' AND COUNTRY.COUNTRY STARTING WITH 'Austria';
```

Isso permite usar o índice para o campo COUNTRY.

Exemplo 81. Usando LIKE com o caractere curinga % no final do literal

```
SET EXPLAIN ON;

SELECT
  COUNT(*)
FROM
  COUNTRY
WHERE COUNTRY.COUNTRY LIKE 'Au%';
```

A saída será a seguinte:

```
Select Expression
  -> Aggregate
    -> Filter
      -> Table "COUNTRY" Access By ID
        -> Bitmap
          -> Index "RDB$PRIMARY1" Range Scan (full match)

          COUNT
=====
              1
```

De acordo com a regra descrita acima, esta consulta é equivalente à seguinte forma:

```
SELECT
  COUNT(*)
FROM
  COUNTRY
WHERE COUNTRY.COUNTRY LIKE 'Au%' AND COUNTRY.COUNTRY STARTING WITH 'Au';
```

Isso permite usar o índice para o campo COUNTRY. Observe: o próprio predicado LIKE não é removido da consulta, ele é usado para filtrar os registros que foram extraídos após a aplicação do acesso via índice.

Exemplo 82. Usando LIKE com caracteres curinga no meio do literal

```

SET EXPLAIN ON;

SELECT
  COUNT(*)
FROM
  COUNTRY
WHERE COUNTRY.COUNTRY LIKE 'A_s%';

```

A saída será a seguinte:

```

Select Expression
  -> Aggregate
    -> Filter
      -> Table "COUNTRY" Access By ID
        -> Bitmap
          -> Index "RDB$PRIMARY1" Range Scan (full match)

COUNT
=====
1

```

De acordo com a regra descrita acima, esta consulta é equivalente à seguinte forma:

```

SELECT
  COUNT(*)
FROM
  COUNTRY
WHERE COUNTRY.COUNTRY LIKE 'A_s%' AND COUNTRY.COUNTRY STARTING WITH 'A';

```

Isso permite usar o índice para o campo COUNTRY. Observe: o próprio predicado LIKE não é removido da consulta, ele é usado para filtrar os registros que foram extraídos após a aplicação do acesso via índice.

Exemplo 83. Usando LIKE com caracteres curinga no início do literal

```

SET EXPLAIN ON;

SELECT
  COUNT(*)
FROM
  COUNTRY
WHERE COUNTRY.COUNTRY LIKE '%u%';

```

A saída será a seguinte:

```

Select Expression
  -> Aggregate
    -> Filter
      -> Table "COUNTRY" Full Scan

```

```

COUNT
=====
4

```

Aqui, não foi possível utilizar o acesso indexado porque o literal de string usado em LIKE começa com um caractere curinga, o que significa que nenhuma transformação é possível.

4.1.2. Transformação do Predicado SIMILAR TO

Para o predicado SIMILAR TO, aplicam-se as mesmas regras que para o predicado LIKE, com a única diferença de que em SIMILAR TO os padrões podem ser muito mais complexos. Neste padrão, também é destacado o chamado prefixo — a parte inicial do literal, onde não aparecem caracteres especiais de padrão, e para esta parte do literal é usado o predicado STARTING WITH.

A consulta a seguir usa acesso indexado:

```

SELECT
  COUNT(*)
FROM
  COUNTRY
WHERE COUNTRY.COUNTRY SIMILAR TO 'Austr[ai]%;

```

```

Select Expression
  -> Aggregate
    -> Filter
      -> Table "COUNTRY" Access By ID
        -> Bitmap
          -> Index "RDB$PRIMARY1" Range Scan (full match)

```

```

COUNT
=====
2

```

Como esta consulta será transformada na forma equivalente

```

SELECT
  COUNT(*)
FROM
  COUNTRY
WHERE COUNTRY.COUNTRY SIMILAR TO 'Austr[ai]%' AND COUNTRY.COUNTRY STARTING WITH 'Austr';

```

No entanto, a consulta a seguir não usará índice, pois neste literal do predicado SIMILAR TO não é

possível destacar um prefixo que não seja um padrão de expressão regular.

```
SELECT
  COUNT(*)
FROM
  COUNTRY
WHERE COUNTRY.COUNTRY SIMILAR TO '[AB]%a';
```

```
Select Expression
  -> Aggregate
      -> Filter
          -> Table "COUNTRY" Full Scan
```

```
      COUNT
=====
              2
```

4.1.3. Inversão de predicados

Se um predicado NOT estiver envolvido em uma condição de filtragem e for aplicado a uma expressão lógica complexa entre parênteses, essa expressão será expandida de modo que o NOT seja aplicado a cada um de seus membros. Nesse processo, alguns dos predicados serão invertidos. Aqui estão exemplos de predicados invertidos:

- NOT $a = b$ é transformado em $a \neq b$;
- NOT $a \neq b$ é transformado em $a = b$;
- NOT $a > b$ é transformado em $a \leq b$;
- NOT $a \text{ BETWEEN } b \text{ AND } c$ é transformado em $a < b \text{ AND } a > c$, etc.

Além disso, as leis de De Morgan são aplicadas aos predicados, ou seja,

```
not (A and B) = (not A) or  (not B)
not (A or  B) = (not A) and (not B)
```

O descrito acima influencia como suas consultas são otimizadas. Por exemplo, sabe-se que a filtragem usando o predicado $\neq$ não pode usar acesso por índice. No entanto, se uma negação NOT for aplicada à condição de filtragem, a possibilidade de usar acesso por índice reaparece, pois em vez do predicado $\neq$ será usado o predicado $=$.

A seguinte consulta não pode usar acesso indexado:

```
SELECT
  COUNT(*)
FROM EMPLOYEE
WHERE LAST_NAME <> 'Young' OR FIRST_NAME <> 'Bruce';
```

```

Select Expression
  -> Aggregate
    -> Filter
      -> Table "EMPLOYEE" Full Scan

          COUNT
=====
          41

```

Mas se aplicarmos a negação à condição em WHERE, você verá o seguinte:

```

SELECT
  COUNT(*)
FROM EMPLOYEE
WHERE NOT (LAST_NAME <> 'Young' OR FIRST_NAME <> 'Bruce');

```

```

Select Expression
  -> Aggregate
    -> Filter
      -> Table "EMPLOYEE" Access By ID
        -> Bitmap
          -> Index "NAMEX" Range Scan (full match)

          COUNT
=====
          1

```

Como você pode ver, neste caso o otimizador conseguiu utilizar o índice NAMEX. Isso se explica pelo fato de que a consulta foi transformada na seguinte forma equivalente:

```

SELECT
  COUNT(*)
FROM EMPLOYEE
WHERE LAST_NAME = 'Young' AND FIRST_NAME = 'Bruce';

```

```

Select Expression
  -> Aggregate
    -> Filter
      -> Table "EMPLOYEE" Access By ID
        -> Bitmap
          -> Index "NAMEX" Range Scan (full match)

          COUNT
=====
          1

```

4.1.4. Transformação do predicado IN com lista de valores

Antes do Firebird 5.0, para o predicado IN com uma lista de valores era usada uma transformação em um conjunto de condições de igualdade unidas pelo predicado OR. Ou seja,

```
F IN (V1, V2, ... VN)
```

era transformado em

```
(F = V1) OR (F = V2) OR .... (F = VN)
```

No Firebird 5.0, isso funciona de forma diferente. As listas de constantes em IN são pré-avaliadas como invariantes e armazenadas em cache como uma árvore de busca binária, o que acelera a comparação se a condição precisar ser verificada para muitos registros ou se a lista de valores for longa. Se um índice for aplicável ao predicado, é usada uma varredura de índice por lista (List scan). Além disso, a remoção dessa transformação permitiu expandir significativamente o limite de valores na lista IN. Anteriormente, ele era limitado a 1000 valores dentro da lista IN; o novo comportamento permite ter até 65535 valores em IN.

No entanto, em alguns casos, a transformação IN ainda é aplicada. Considere a seguinte consulta:

```
SELECT COUNT(*)
FROM HORSE
WHERE ? IN (CODE_FATHER, CODE_MOTHER);
```

O plano desta consulta será o seguinte:

```
Select Expression
  -> Aggregate
    -> Filter
      -> Table "HORSE" Access By ID
        -> Bitmap Or
          -> Bitmap
            -> Index "FK_HORSE_FATHER" Range Scan (full match)
          -> Bitmap
            -> Index "FK_HORSE_MOTHER" Range Scan (full match)
```

A partir do Firebird 5.0.2, a regra para transformar o predicado IN em um conjunto de condições unidas pelo predicado OR é mais complexa. As expressões dentro da lista IN são divididas em dois grupos: aquelas com dependências e aquelas sem dependências. Se uma expressão usa colunas ou referências a aliases de campos de expressões de tabela, tais expressões são chamadas de dependentes. Todas as outras são expressões independentes. Na maioria das vezes, literais ou parâmetros de consulta atuam como expressões independentes. Após isso, as expressões independentes na lista IN são avaliadas como invariantes e armazenadas em cache como uma árvore de busca binária. As expressões dependentes na lista IN são transformadas em uma lista OR (comportamento usado antes do Firebird 5.0).

Esquemáticamente, fica assim:

```
SELECT ... FROM ...
WHERE <expr> IN (N1, N2, ... Nm, ... <dep1>, <dep2>, ... <depL>)
```

é transformado em

```
SELECT ... FROM ...
WHERE <expr> = <dep1> OR ... <expr> = <depL> OR <expr> IN (N1, N2, ... Nm)
```

Aqui N são expressões independentes (literais, parâmetros de entrada), <dep> são expressões dependentes (contendo colunas de tabelas ou expressões de tabela).

4.2. Transformação de subconsultas

A transformação de subconsultas pode ser dividida em dois grupos:

- transformação de um predicado de existência (IN, EXISTS, ANY, SOME, ALL) em outro;
- transformação de um predicado de existência em tipos especiais de junções (semi-join, anti-join).

O primeiro tipo de transformações funciona em qualquer lugar e sempre, pois é simplesmente a conversão de vários predicados em uma forma unificada. O segundo tipo está disponível apenas para predicados de existência que são usados para filtrar fluxos de dados na cláusula WHERE (em alguns casos, em HAVING).

4.2.1. Transformação de IN em SOME/ANY

O predicado IN com uma subconsulta não existe no nível BLR; ele é transformado em uma forma equivalente usando os predicados SOME/ANY. Os predicados SOME e ANY não diferem em nada e são codificados usando o mesmo BLR `blr_ansi_any`. Esquemáticamente, essa transformação se parece com o seguinte:

```
SELECT *
FROM ...
WHERE <expr> IN (<sub-query>)
```

é transformado em

```
SELECT *
FROM ...
WHERE <expr> = ANY (<sub-query>)
```

4.2.2. Transformação de subconsultas ANY/ALL em forma correlacionada

Subconsultas dentro do predicado ANY e ALL não são correlacionadas. Em geral, consultas que usam subconsultas com o predicado ANY têm a forma:

```
SELECT *
FROM ...
WHERE <expr> <op> ANY (<sub-query>)
```

Aqui <op> é um dos operadores de comparação. O otimizador o transforma em uma forma correlacionada e o executa como

```
SELECT *
FROM ...
WHERE EXISTS (SELECT * FROM (<sub-query>) WHERE <expr> <op> <sub-query-field>)
```



É importante notar aqui que a subconsulta é executada como se estivesse no predicado EXISTS; a transformação real para este predicado não é feita no nível BLR.

Dependendo do conteúdo da subconsulta, o predicado de filtragem resultante pode ser empurrado para dentro da subconsulta, para ficar o mais próximo possível das fontes primárias de dados.

Suponha que temos a consulta

```
SELECT *
FROM EMPLOYEE
WHERE EMPLOYEE.SALARY > ANY (SELECT MAX_SALARY FROM JOB)
```

De acordo com a regra descrita acima, esta consulta será equivalente à seguinte

```
SELECT *
FROM EMPLOYEE
WHERE EXISTS(SELECT * FROM (SELECT MAX_SALARY FROM JOB) WHERE EMPLOYEE.SALARY > MAX_SALARY)
```

Como a expressão de tabela é bastante simples, o predicado de filtragem pode não apenas ser empurrado para dentro, mas também a expressão de tabela pode ser expandida para as tabelas base. Assim, nossa consulta pode ser reescrita como:

```
SELECT *
FROM EMPLOYEE
WHERE EXISTS(SELECT MAX_SALARY FROM JOB WHERE EMPLOYEE.SALARY > MAX_SALARY)
```

Para o predicado ALL não há outro análogo, como no exemplo acima, onde transformamos ANY em EXISTS, mas a transformação da consulta em uma forma correlacionada e o empurrão do predicado

de filtragem o mais próximo possível dos métodos de acesso primários também ocorre.

4.2.3. Transformação de NOT IN, NOT ANY, ALL com o operador <>

Diferentemente do predicado IN, que pode ser substituído por EXISTS com uma subconsulta correlacionada, fazer o mesmo para NOT IN não é possível. A razão está no tratamento de valores NULL.

O fato é que NOT IN com uma subconsulta é semanticamente equivalente ao predicado NOT IN com uma lista de valores que a subconsulta retorna; assim, se a subconsulta retornar pelo menos um valor NULL, o predicado será avaliado como UNKNOWN, o que nunca acontece com o predicado NOT EXISTS.

```
x NOT IN (<value_1>, <value_2> ... <value_N>)
```

equivalente a

```
NOT (x = <value_1> OR x = <value_2> ... OR x = <value_N>)
```

expandindo os parênteses

```
x <> <value_1> AND x <> <value_2> ... AND x <> <value_N>
```

Se x ou um dos <value> for NULL, então toda a expressão será avaliada como UNKNOWN.

Portanto, para os predicados NOT IN, bem como suas variantes NOT (a = ANY b) ou a <> ALL b, usa-se a seguinte transformação:

```
SELECT ... FROM <t1>
WHERE <x> NOT IN (SELECT <y> FROM <t2>)
```

ou

```
SELECT ... FROM <t1>
WHERE NOT (<x> = ANY (SELECT <y> FROM <t2>))
```

ou

```
SELECT ... FROM <t1>
WHERE <x> <> ALL (SELECT <y> FROM <t2>)
```

é transformado em

```
SELECT ... FROM <t1>
WHERE NOT ((x IS NULL AND EXISTS (SELECT 1 FROM <t2>)) OR
           EXISTS (SELECT <y> FROM <t2> WHERE <y> = <x> OR <y> IS NULL))
```

Observe o plano de explain da seguinte consulta para o banco de dados employee

```
select
  *
from project
where project.team_leader not in (
  select
    employee.emp_no
  from employee
  where employee.hire_date < timestamp '1991-01-01'
)
```

```
Sub-query (invariant)
  -> Filter
    -> Table "EMPLOYEE" Full Scan
Sub-query
  -> Filter
    -> Table "EMPLOYEE" Access By ID
      -> Bitmap Or
        -> Bitmap
          -> Index "RDB$PRIMARY7" Unique Scan
        -> Bitmap
          -> Index "RDB$PRIMARY7" Unique Scan
Select Expression
  -> Filter
    -> Table "PROJECT" Full Scan
```

Olhando para esta consulta, é completamente incompreensível por que o plano de execução é exatamente assim, mas se expandirmos o predicado NOT IN de acordo com a regra descrita acima, tudo fica claro.

```
select
  *
from project
where not (
  (project.team_leader is null and exists(
    select
      1
    from employee
    where employee.hire_date < timestamp '1991-01-01')
  )
  or exists(
    select
      employee.emp_no
    from employee
    where employee.hire_date < timestamp '1991-01-01'
      and (employee.emp_no = project.team_leader or employee.emp_no is null)
  )
)
```

Se observarmos a consulta resultante, agora é visível de onde vem o Sub-query (invariant) e como

se obtém o Bitmap Or.

4.2.4. Transformação em semi-join

Este tipo de transformação está disponível a partir do Firebird 5.0.1. Por padrão, está desativado. Para ativá-lo, é necessário definir o parâmetro de configuração SubQueryConversion como true.

A conversão para semi-join (junção semântica) pode ser realizada para subconsultas nos predicados IN, EXISTS, SOME/ANY, se forem usadas para filtrar os resultados da consulta principal na cláusula WHERE. Nem toda subconsulta nesses predicados pode ser convertida em semi-join. Se a subconsulta contiver limitadores FETCH/FIRST/SKIP/ROWS, não é possível converter a subconsulta em semi-join, e ela será executada como uma subconsulta correlacionada comum.

Na sintaxe SQL, o semi-join não é representado diretamente e só pode ser usado se o semi-join for obtido como resultado de uma transformação.

Por que essa transformação é necessária? O fato é que uma subconsulta correlacionada sempre é executada por um único algoritmo – para cada registro da consulta principal, a subconsulta é executada e o valor do predicado é avaliado. Este algoritmo lembra muito o algoritmo de junção por Nested Loop, e ele nem sempre é o mais eficiente. Converter subconsultas nos predicados IN, EXISTS, SOME/ANY em semi-joins permite que o otimizador escolha entre diferentes algoritmos de junção Nested Loop (semi) e Hash Join (semi).

Como essa transformação é realizada? O predicado IN é primeiro convertido em ANY, como descrito acima, o predicado EXISTS permanece como está. Em seguida, da subconsulta correlacionada é extraída a condição de vinculação, que se torna a condição de junção dos dois fluxos.

Suponha que você tenha a seguinte consulta:

```
SELECT *
FROM RDB$RELATIONS R
WHERE EXISTS (
  SELECT *
  FROM RDB$PAGES P
  WHERE P.RDB$RELATION_ID = R.RDB$RELATION_ID
  AND P.RDB$PAGE_SEQUENCE > 5
)
```

Seu plano de execução será:

```
Select Expression
-> Filter
  -> Hash Join (semi)
    -> Table "RDB$RELATIONS" as "R" Full Scan
    -> Record Buffer (record length: 33)
      -> Filter
        -> Table "RDB$PAGES" as "P" Full Scan
```

Neste caso, a condição de vinculação é P.RDB\$RELATION_ID = R.RDB\$RELATION_ID. Esta consulta é

como se fosse transformada na seguinte forma:

```
SELECT *
FROM
  RDB$RELATIONS R
  SEMI JOIN (
    SELECT *
    FROM RDB$PAGES
    WHERE RDB$PAGES.RDB$PAGE_SEQUENCE > 5
  ) P ON P.RDB$RELATION_ID = R.RDB$RELATION_ID
```

No Firebird 5.0, não foi implementada uma avaliação de custo entre os algoritmos de execução Nested Loop (semi) e Hash Join (semi); além disso, o algoritmo de execução Nested Loop (semi) está atualmente bloqueado. Assim, atualmente, o semi-join só pode ser executado pelo algoritmo Hash Join (semi), o que impõe mais uma restrição à condição sob a qual essa conversão pode ser realizada – a condição de vinculação deve usar apenas predicados de igualdade ou IS NOT DISTINCT FROM.



É por isso que existe o parâmetro de configuração SubQueryConversion, e por padrão essa transformação está desativada. Em versões futuras, será adicionada uma avaliação de custo entre os algoritmos de execução de semi-join Nested Loop (semi) e Hash Join (semi). Nesse caso, este parâmetro pode ser removido, e a transformação ocorrerá sempre que possível.

4.2.5. Transformação em anti-join

Este tipo de transformação não está implementado no Firebird.

A conversão para anti-join (junção antisseântica) pode ser realizada para uma subconsulta no predicado NOT EXISTS, se esse predicado for usado para filtrar os resultados da consulta principal na cláusula WHERE. Nem toda subconsulta nesses predicados pode ser convertida em semi-join. Se a subconsulta contiver limitadores FETCH/FIRST/SKIP/ROWS, não é possível converter a subconsulta em anti-join, e ela será executada como uma subconsulta correlacionada comum.

Na sintaxe SQL, o anti-join não é representado diretamente e não pode ser usado, a menos que o anti-join seja obtido como resultado de uma transformação.



A implementação da conversão de subconsultas no predicado NOT EXISTS para anti-join está planejada para versões futuras do Firebird.

Suponha que você tenha a seguinte consulta:

```
SELECT *
FROM RDB$RELATIONS R
WHERE NOT EXISTS (
  SELECT *
  FROM RDB$PAGES P
  WHERE P.RDB$RELATION_ID = R.RDB$RELATION_ID
  AND P.RDB$PAGE_SEQUENCE > 5
```

```
)
```

Esta consulta pode ser transformada na seguinte forma:

```
SELECT *
FROM
  RDB$RELATIONS R
  ANTI JOIN (
    SELECT *
    FROM RDB$PAGES
    WHERE RDB$PAGES.RDB$PAGE_SEQUENCE > 5
  ) P ON P.RDB$RELATION_ID = R.RDB$RELATION_ID
```

Aqui ANTI JOIN é uma sintaxe virtual, que nunca será implementada e é apresentada apenas para compreensão do significado da transformação.

4.3. Transformação de junções (JOINS)

4.3.1. Transformação de RIGHT JOIN em LEFT JOIN

No Firebird, RIGHT JOIN não é implementado como um tipo de junção separado. Em vez disso, RIGHT JOIN é transformado em LEFT JOIN durante a análise da consulta SQL e sua conversão para BLR. O algoritmo de transformação é bastante simples, pois tudo o que é necessário é trocar os fluxos de junção de lugar.

```
<source_1> RIGHT JOIN <source_2>
```

é convertido em

```
<source_2> LEFT JOIN <source_1>
```

Este tipo de transformação é fácil de verificar se você observar o plano de explain da consulta:

```
SELECT
  P.PROJ_NAME,
  E.LAST_NAME,
  E.FIRST_NAME
FROM
  PROJECT P
  RIGHT JOIN EMPLOYEE E ON E.EMP_NO = P.TEAM_LEADER
```

```
Select Expression
-> Nested Loop Join (outer)
  -> Table "EMPLOYEE" as "E" Full Scan
  -> Filter
    -> Table "PROJECT" as "P" Access By ID
```

```
-> Bitmap
    -> Index "RDB$FOREIGN13" Range Scan (full match)
```

No plano Explain, a direção da junção externa não é exibida, mas é fácil deduzir pela ordem dos fluxos de junção. Neste caso, o fluxo da tabela EMPLOYEE é o principal, como se nossa consulta tivesse sido escrita assim:

```
SELECT
  P.PROJ_NAME,
  E.LAST_NAME,
  E.FIRST_NAME
FROM
  EMPLOYEE E
  LEFT JOIN PROJECT P ON E.EMP_NO = P.TEAM_LEADER
```

4.3.2. Transformação de OUTER JOIN em INNER JOIN

Este tipo de transformação está disponível a partir do Firebird 5.0. Por padrão, está habilitado, mas pode ser desabilitado substituindo o parâmetro de configuração OuterJoinConversion pelo valor false no arquivo de configuração firebird.conf ou databases.conf.

A transformação de LEFT JOIN (RIGHT JOIN) em INNER JOIN ocorre se na cláusula WHERE existir um predicado para as colunas do fluxo secundário (opcional) que não lida explicitamente com o valor NULL, ou seja, todos os predicados exceto IS [NOT] NULL ou IS [NOT] DISTINCT FROM.

Uma junção completa (FULL JOIN) pode ser transformada em LEFT JOIN ou RIGHT JOIN dependendo de para qual fluxo existe um predicado que não lida explicitamente com o valor NULL, ou pode ser transformada em INNER JOIN se tal predicado existir para ambos os fluxos de junção.

Essas transformações são legítimas porque preservam o significado semântico da consulta, e o resultado de sua execução será o mesmo.

Por que essa transformação é necessária? O fato é que para LEFT JOIN e RIGHT JOIN a ordem de junção dos fluxos é fixa e não pode ser alterada, enquanto para junções internas o otimizador pode escolher a ordem de junção. Além disso, atualmente, para junções externas, apenas um algoritmo de execução Nested Loop é implementado, enquanto para junções internas o otimizador pode escolher entre Nested Loop, Hash Join e Merge Join. Tudo isso permite que o otimizador escolha um plano de execução mais eficiente **com base nas estatísticas disponíveis**. Para junções externas completas (FULL OUTER JOIN), essa transformação permite remover ramos de execução desnecessários (no Firebird, não há um algoritmo de passagem única para junção externa completa; ela é executada como LEFT JOIN + ANTI JOIN, veja [Full Outer Join](#)).

Vejamos exemplos de como a transformação de junções externas é realizada. Tomemos a seguinte consulta:

```
SELECT
  P.PROJ_NAME,
  E.LAST_NAME,
  E.FIRST_NAME
```



```
FROM
  EMPLOYEE E
  LEFT JOIN PROJECT P ON E.EMP_NO = P.TEAM_LEADER
```

Ela tem o seguinte plano de execução:

```
Select Expression
  -> Nested Loop Join (outer)
    -> Table "EMPLOYEE" as "E" Full Scan
    -> Filter
      -> Table "PROJECT" as "P" Access By ID
        -> Bitmap
          -> Index "RDB$FOREIGN13" Range Scan (full match)
```

Agora, vamos modificar um pouco esta consulta:

```
SELECT
  P.PROJ_NAME,
  E.LAST_NAME,
  E.FIRST_NAME
FROM
  EMPLOYEE E
  LEFT JOIN PROJECT P ON E.EMP_NO = P.TEAM_LEADER
WHERE P.PRODUCT = 'software'
```

Seu plano de execução é o seguinte:

```
Select Expression
  -> Nested Loop Join (inner)
    -> Filter
      -> Table "PROJECT" as "P" Access By ID
        -> Bitmap
          -> Index "PRODTYPEX" Range Scan (partial match: 1/2)
    -> Filter
      -> Table "EMPLOYEE" as "E" Access By ID
        -> Bitmap
          -> Index "RDB$PRIMARY7" Unique Scan
```

Como você pode ver, a junção externa foi substituída por uma interna, e a ordem de junção das tabelas mudou. Agora, vamos tentar alterar o predicado de filtro para que ele lide com valores NULL:

```
SELECT
  P.PROJ_NAME,
  E.LAST_NAME,
  E.FIRST_NAME
FROM
  EMPLOYEE E
  LEFT JOIN PROJECT P ON E.EMP_NO = P.TEAM_LEADER
WHERE P.PRODUCT IS NOT NULL
```

Seu plano de execução é o seguinte:

```
Select Expression
-> Filter
    -> Nested Loop Join (outer)
        -> Table "EMPLOYEE" as "E" Full Scan
        -> Filter
            -> Table "PROJECT" as "P" Access By ID
                -> Bitmap
                    -> Index "RDB$FOREIGN13" Range Scan (full match)
```

Observe: a transformação em INNER JOIN não ocorre, a ordem de junção das tabelas é preservada, embora semanticamente esta consulta seja um equivalente completo de INNER JOIN. Esta possibilidade foi deixada especificamente para desenvolvedores que usam LEFT JOIN como uma dica para o otimizador sobre a ordem de junção das tabelas. Mas e se o otimizador errar e for necessário executar a consulta com WHERE P.PRODUCT = 'software', juntando as tabelas exatamente na ordem em que o desenvolvedor escreveu? Para isso, você pode mover P.PRODUCT = 'software' para a condição de junção e adicionar o filtro P.PRODUCT IS NOT NULL na cláusula WHERE.

```
SELECT
  P.PROJ_NAME,
  E.LAST_NAME,
  E.FIRST_NAME
FROM
  EMPLOYEE E
  LEFT JOIN PROJECT P ON E.EMP_NO = P.TEAM_LEADER AND P.PRODUCT = 'software'
WHERE P.PRODUCT IS NOT NULL
```

O plano de execução desta consulta é o seguinte:

```
Select Expression
-> Filter
    -> Nested Loop Join (outer)
        -> Table "EMPLOYEE" as "E" Full Scan
        -> Filter
            -> Table "PROJECT" as "P" Access By ID
                -> Bitmap
                    -> Index "RDB$FOREIGN13" Range Scan (full match)
```

Como você pode ver, a ordem de junção das tabelas foi preservada, e nosso filtro será aplicado.

Vale ressaltar aqui que o Firebird não realiza uma análise profunda e transforma o tipo de junção apenas para o fluxo ao qual o predicado de filtro que não considera valores NULL é aplicado. Considere a seguinte consulta:

```
SELECT
  P.PROJ_NAME,
  E.LAST_NAME,
  E.FIRST_NAME
```

```

FROM
  EMPLOYEE E
  LEFT JOIN EMPLOYEE_PROJECT EP ON EP.EMP_NO = E.EMP_NO
  LEFT JOIN PROJECT P ON P.PROJ_ID = EP.PROJ_ID
WHERE P.PRODUCT = 'software'

```

O plano de execução desta consulta é o seguinte:

```

Select Expression
-> Nested Loop Join (inner)
  -> Nested Loop Join (outer)
    -> Table "EMPLOYEE" as "E" Full Scan
    -> Filter
      -> Table "EMPLOYEE_PROJECT" as "EP" Access By ID
      -> Bitmap
        -> Index "RDB$FOREIGN15" Range Scan (full match)
    -> Filter
      -> Table "PROJECT" as "P" Access By ID
      -> Bitmap
        -> Index "RDB$PRIMARY12" Unique Scan

```

Pelo plano de execução da consulta, é possível ver que a consulta foi transformada na seguinte forma:

```

SELECT
  P.PROJ_NAME,
  E.LAST_NAME,
  E.FIRST_NAME
FROM
  ( EMPLOYEE E
    LEFT JOIN EMPLOYEE_PROJECT EP ON EP.EMP_NO = E.EMP_NO )
  JOIN PROJECT P ON P.PROJ_ID = EP.PROJ_ID
WHERE P.PRODUCT = 'software'

```

Embora seja possível provar que a consulta resultante pode ser reduzida à seguinte:

```

SELECT
  P.PROJ_NAME,
  E.LAST_NAME,
  E.FIRST_NAME
FROM
  EMPLOYEE E
  JOIN EMPLOYEE_PROJECT EP ON EP.EMP_NO = E.EMP_NO
  JOIN PROJECT P ON P.PROJ_ID = EP.PROJ_ID
WHERE P.PRODUCT = 'software'

```

Agora, vejamos um exemplo de transformação de FULL OUTER JOIN.

```

SELECT
  P.PROJ_NAME,

```

```

E.LAST_NAME,
E.FIRST_NAME
FROM
EMPLOYEE E
FULL JOIN PROJECT P ON E.EMP_NO = P.TEAM_LEADER

```

O plano de execução desta consulta é o seguinte:

```

Select Expression
-> Full Outer Join
  -> Nested Loop Join (outer)
    -> Table "PROJECT" as "P" Full Scan
    -> Filter
      -> Table "EMPLOYEE" as "E" Access By ID
        -> Bitmap
          -> Index "RDB$PRIMARY7" Unique Scan
  -> Nested Loop Join (anti)
    -> Table "EMPLOYEE" as "E" Full Scan
    -> Filter
      -> Table "PROJECT" as "P" Access By ID
        -> Bitmap
          -> Index "RDB$FOREIGN13" Range Scan (full match)

```

Agora, vamos modificar a consulta da seguinte forma:

```

SELECT
P.PROJ_NAME,
E.LAST_NAME,
E.FIRST_NAME
FROM
EMPLOYEE E
FULL JOIN PROJECT P ON E.EMP_NO = P.TEAM_LEADER
WHERE P.PRODUCT = 'software'

```

Plano da consulta:

```

Select Expression
-> Filter
  -> Nested Loop Join (outer)
    -> Filter
      -> Table "PROJECT" as "P" Access By ID
        -> Bitmap
          -> Index "PRODTYPEX" Range Scan (partial match: 1/2)
    -> Filter
      -> Table "EMPLOYEE" as "E" Access By ID
        -> Bitmap
          -> Index "RDB$PRIMARY7" Unique Scan

```

Pelo plano da consulta, é possível ver que a consulta agora executa apenas uma junção externa, o anti-join foi removido, como se a consulta fosse assim:

```

SELECT
  P.PROJ_NAME,
  E.LAST_NAME,
  E.FIRST_NAME
FROM
  PROJECT P
  LEFT JOIN EMPLOYEE E ON E.EMP_NO = P.TEAM_LEADER
WHERE P.PRODUCT = 'software'

```

Agora, vamos tentar aplicar o predicado de filtro à outra tabela:

```

SELECT
  P.PROJ_NAME,
  E.LAST_NAME,
  E.FIRST_NAME
FROM
  EMPLOYEE E
  FULL JOIN PROJECT P ON E.EMP_NO = P.TEAM_LEADER
WHERE E.JOB_COUNTRY = 'USA'

```

O plano será assim:

```

Select Expression
  -> Filter
    -> Nested Loop Join (outer)
      -> Filter
        -> Table "EMPLOYEE" as "E" Full Scan
      -> Filter
        -> Table "PROJECT" as "P" Access By ID
          -> Bitmap
            -> Index "RDB$FOREIGN13" Range Scan (full match)

```

Agora, vamos aplicar predicados de filtro a ambas as tabelas:

```

SELECT
  P.PROJ_NAME,
  E.LAST_NAME,
  E.FIRST_NAME
FROM
  EMPLOYEE E
  FULL JOIN PROJECT P ON E.EMP_NO = P.TEAM_LEADER
WHERE E.JOB_COUNTRY = 'USA' AND P.PRODUCT = 'software'

```

O plano desta consulta será o seguinte:

```

Select Expression
  -> Nested Loop Join (inner)
    -> Filter
      -> Table "PROJECT" as "P" Access By ID

```

```

-> Bitmap
    -> Index "PRODTYPEX" Range Scan (partial match: 1/2)
-> Filter
    -> Table "EMPLOYEE" as "E" Access By ID
        -> Bitmap
            -> Index "RDB$PRIMARY7" Unique Scan

```

Ou seja, FULL JOIN foi transformado em INNER JOIN:

```

SELECT
  P.PROJ_NAME,
  E.LAST_NAME,
  E.FIRST_NAME
FROM
  EMPLOYEE E
  JOIN PROJECT P ON E.EMP_NO = P.TEAM_LEADER
WHERE E.JOB_COUNTRY = 'USA' AND P.PRODUCT = 'software'

```

4.3.3. Transformação de OUTER JOIN em anti-join

Este tipo de transformação não está implementado no Firebird. Sua essência é a seguinte: um LEFT JOIN (RIGHT JOIN) pode ser transformado em um anti-join se na cláusula WHERE existirem apenas predicados IS NULL para colunas do fluxo secundário (opcional), sendo que essas colunas também são usadas na condição de junção dos dois fluxos.

Considere a seguinte consulta:

```

SELECT
  E.LAST_NAME,
  E.FIRST_NAME
FROM
  EMPLOYEE E
  LEFT JOIN PROJECT P ON P.TEAM_LEADER = E.EMP_NO
WHERE P.TEAM_LEADER IS NULL

```

Seu plano de execução será o seguinte:

```

Select Expression
  -> Filter
    -> Nested Loop Join (outer)
      -> Table "EMPLOYEE" as "E" Full Scan
      -> Filter
        -> Table "PROJECT" as "P" Access By ID
          -> Bitmap
            -> Index "RDB$FOREIGN13" Range Scan (full match)

```

Neste caso, o plano não é ideal. Para cada registro de EMPLOYEE, é necessário extrair os registros de PROJECT que correspondem à condição de junção P.TEAM_LEADER = E.EMP_NO, mas manter apenas os registros para os quais não foi encontrada correspondência. Obviamente, esta consulta é

semanticamente equivalente à seguinte consulta:

```
SELECT
  E.LAST_NAME,
  E.FIRST_NAME
FROM
  EMPLOYEE E
WHERE NOT EXISTS (SELECT * FROM PROJECT P WHERE P.TEAM_LEADER = E.EMP_NO)
```

A consulta com NOT EXISTS será executada muito mais rapidamente, pois o predicado NOT EXISTS interrompe imediatamente a execução da subconsulta após a extração do primeiro registro, ao contrário do LEFT JOIN. Também é óbvio que ambas as formas podem ser convertidas em um anti-join quando essa transformação for implementada.



Ambos os tipos de transformação estão planejados para implementação em versões futuras do servidor.

4.4. Conclusão

Neste capítulo, conhecemos vários tipos de transformações de consultas e aprendemos que a transformação de consultas é uma parte importante da otimização. A quantidade de transformações disponíveis aumenta em cada nova versão do Firebird, porém muitas transformações ainda não existem, mas podem ser implementadas no futuro. Tais transformações incluem:

- conversão de subconsultas em anti-join;
- conversão de junções externas em anti-join;
- remoção de junções externas se elas não afetam o resultado da consulta (transformações baseadas em informações de restrições);
- movimentação de junções internas para acima de junções externas, entre outras.

Capítulo 5. Estatísticas por tabela

Agora que você sabe como ler planos de consulta SQL e quais métodos de acesso são usados pelo núcleo do Firebird, podemos considerar outra métrica que pode ser usada para analisar a eficiência da execução de consultas SQL.

Durante a operação, o Firebird acumula várias estatísticas de tempo de execução, incluindo estatísticas de trabalho com registros para cada tabela. Essas estatísticas incluem os seguintes contadores: número de registros lidos por varredura completa, número de registros lidos usando índices, número de registros inseridos, modificados e excluídos, entre outros.

Os valores desses contadores estão disponíveis através da função de API `isc_database_info`, que é usada em várias ferramentas gráficas de administração e no `isql`. Além disso, esses contadores podem ser obtidos através do uso combinado das tabelas de monitoramento `MON$RECORD_STATS` e `MON$TABLE_STATS` ou no rastreamento.

A seguir, consideraremos opções para obter estatísticas por tabela usando várias ferramentas, e também descreveremos detalhadamente a finalidade desses contadores e como eles se correlacionam com os planos de consulta.

5.1. Obtendo estatísticas por tabela no isql

A obtenção de estatísticas por tabela usando `isql` está disponível a partir do Firebird 5.0.

Por padrão, a saída de estatísticas por tabela está desativada.

Para ativá-la, é necessário digitar o comando:

```
SET PER_TAB ON;
```

E para desativar:

```
SET PER_TAB OFF;
```

O comando `SET PER_TAB` sem as palavras `ON` ou `OFF` alterna o estado da saída de estatísticas.

A sintaxe completa deste comando pode ser obtida usando o comando `HELP SET`.

Vejamos um exemplo de saída de estatísticas por tabela e expliquemos a finalidade dos contadores exibidos. Para isso, ativaremos a saída de estatísticas por tabela e o `explain-plan`.

```
SET PER_TAB ON;  
SET EXPLAIN ON;  
OUTPUT NUL;  
  
SELECT  
  *
```



```

FROM
  EMPLOYEE
  JOIN EMPLOYEE_PROJECT ON EMPLOYEE_PROJECT.EMP_NO = EMPLOYEE.EMP_NO
  JOIN PROJECT ON PROJECT.PROJ_ID = EMPLOYEE_PROJECT.PROJ_ID
WHERE EMPLOYEE.SALARY > 50000;

```

```

Select Expression
  -> Nested Loop Join (inner)
    -> Table "PROJECT" Full Scan
    -> Filter
      -> Table "EMPLOYEE_PROJECT" Access By ID
        -> Bitmap
          -> Index "RDB$FOREIGN16" Range Scan (full match)
      -> Filter
        -> Table "EMPLOYEE" Access By ID
          -> Bitmap
            -> Index "RDB$PRIMARY7" Unique Scan

```

Per table statistics:

Table name	Natural	Index	Insert	Update	Delete	Backout	Purge	Expunge
EMPLOYEE		28						
PROJECT	6							
EMPLOYEE_PROJECT		28						



É melhor obter estatísticas por tabela após a segunda execução da consulta. O fato é que, durante a primeira execução da consulta, as tabelas do sistema são incluídas nas estatísticas por tabela. Essas tabelas são usadas por consultas internas do isql que leem metadados. Na reexecução da consulta, os metadados já estão salvos no cache e não são relidos.

Vamos decifrar o que esses contadores significam para as tabelas:

- Natural — número de registros lidos por varredura completa da tabela (Full Scan, NATURAL no plano Legacy);
- Index — número de registros lidos usando varredura indexada;
- Insert — número de registros inseridos na tabela;
- Update — número de registros atualizados na tabela;
- Delete — número de registros excluídos da tabela;
- Backout — número de exclusões de versões de registros criadas durante rollback (backed out records);
- Purge — número de exclusões de versões antigas de registros (purged records);
- Expunge — número de exclusões de toda a cadeia de versões de um registro, se a versão mais recente for excluída e não for necessária para outras transações (expunged records).

Para analisar o plano de consulta, precisaremos apenas dos dois primeiros contadores.

A partir das estatísticas acima, pode-se ver que da tabela PROJECT foram lidos 6 registros usando

varredura completa da tabela, e das tabelas EMPLOYEE e EMPLOYEE_PROJECT foram lidos 28 registros cada usando varredura indexada. Essas estatísticas correlacionam-se completamente com o Explain-plan apresentado.

O que isso nos dá? Em alguns casos, as estatísticas por tabela permitem responder à questão de qual variante do plano é mais eficiente.

Suponha que temos a seguinte consulta:

```
SELECT
  COUNT(*)
FROM
  PRODUCT
  JOIN INVOICE_LINE ON INVOICE_LINE.PRODUCT_ID = PRODUCT.PRODUCT_ID
  JOIN INVOICE ON INVOICE.INVOICE_ID = INVOICE_LINE.INVOICE_ID
WHERE PRODUCT.PRICE > 1000 AND INVOICE.INVOICE_DATE < date '2015-04-01'
```

Vamos tentar executá-la, medindo o desempenho e obtendo estatísticas.

```
set stat on;
set explain on;
set per_tab on;

SELECT
  COUNT(*)
FROM
  PRODUCT
  JOIN INVOICE_LINE ON INVOICE_LINE.PRODUCT_ID = PRODUCT.PRODUCT_ID
  JOIN INVOICE ON INVOICE.INVOICE_ID = INVOICE_LINE.INVOICE_ID
WHERE PRODUCT.PRICE > 1000 AND INVOICE.INVOICE_DATE < date '2015-04-01';
```

Como resultado, o seguinte será exibido:

```
Select Expression
  -> Aggregate
    -> Filter
      -> Hash Join (inner)
        -> Nested Loop Join (inner)
          -> Filter
            -> Table "INVOICE" Access By ID
              -> Bitmap
                -> Index "INVOICE_IDX_DATE" Range Scan (upper bound: 1/1)
          -> Filter
            -> Table "INVOICE_LINE" Access By ID
              -> Bitmap
                -> Index "FK_INVOICE_LINE_INVOICE" Range Scan (full match)
        -> Record Buffer (record length: 33)
          -> Filter
            -> Table "PRODUCT" Full Scan

COUNT
=====
31249
```

```
Current memory = 282861360
Delta memory = 352
Max memory = 290360832
Elapsed time = 1.641 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 955819
Per table statistics:
```

Table name	Natural	Index	Insert	Update	Delete	Backout	Purge	Expunge
PRODUCT	2889							
INVOICE_LINE		397627						
INVOICE		39667						

Agora, vamos ajustar um pouco o plano de execução usando a dica +0.



Como e quando aplicar dicas corretamente será considerado mais adiante; por enquanto, vamos apenas examinar a correlação das estatísticas por tabela com a eficiência da execução de consultas.

```
SELECT
  COUNT(*)
FROM
  PRODUCT
  JOIN INVOICE_LINE ON INVOICE_LINE.PRODUCT_ID = PRODUCT.PRODUCT_ID
  JOIN INVOICE ON INVOICE.INVOICE_ID = INVOICE_LINE.INVOICE_ID
WHERE PRODUCT.PRICE > 1000 AND INVOICE.INVOICE_DATE+0 < date '2015-04-01';
```

```
Select Expression
  -> Aggregate
    -> Filter
      -> Hash Join (inner)
        -> Nested Loop Join (inner)
          -> Filter
            -> Table "PRODUCT" Full Scan
          -> Filter
            -> Table "INVOICE_LINE" Access By ID
              -> Bitmap
                -> Index "FK_INVOICE_LINE_PRODUCT" Range Scan (full match)
            -> Record Buffer (record length: 33)
          -> Filter
            -> Table "INVOICE" Full Scan
```

```

COUNT
=====
31249
```

```
Current memory = 282861392
Delta memory = 32
Max memory = 290360832
Elapsed time = 0.562 sec
Buffers = 32768
Reads = 0
Writes = 0
```

Fetches = 262264
Per table statistics:

Table name	Natural	Index	Insert	Update	Delete	Backout	Purge	Expunge
PRODUCT	2889							
INVOICE_LINE		77993						
INVOICE	100010							

Esta consulta foi executada 3 vezes mais rápido. A primeira consulta extraiu 397627 registros usando leitura indexada da tabela INVOICE_LINE e 39667 da tabela INVOICE, enquanto a segunda extraiu 77993 registros usando leitura indexada da tabela INVOICE_LINE e 100010 registros usando varredura completa da tabela INVOICE. Obviamente, a primeira consulta leu muito mais dados para obter o mesmo resultado. Isso também é perceptível pelo número de Fetches, que para a primeira consulta é 3,5 vezes maior.

Observo que não há correlação completa entre o número de registros lidos da tabela e a velocidade de execução, pois essas quantidades não refletem alguns custos indiretos.

5.2. Obtenção de estatísticas por tabela usando trace

Para obter estatísticas por tabela usando trace, é necessário definir o parâmetro `print_perf = true` na configuração do trace.

Para demonstração, criaremos um arquivo de configuração de trace com o seguinte conteúdo:

```
database = examples
{
  enabled = true
  log_initfini = false
  log_statement_prepare = true
  log_statement_finish = true
  print_plan = true
  explain_plan = true
  print_perf = true
  time_threshold = 0
  max_sql_length = 8000
  max_arg_length = 400
}
```

Salvamos ele com o nome `example_trace.conf`. Nesta configuração também está habilitada a saída do plano `explain` para obter a mesma informação que no `isql` no exemplo acima.

Em seguida, iniciamos uma sessão interativa de trace usando o comando:

```
fbtracemgr -SE inet://localhost/service_mgr -START -NAME ex_trace -USER SYSDBA -PASS masterkey -CONFIG
examples_trace.conf
```

Executamos a seguinte consulta:

```
SELECT
  COUNT(*)
FROM
  PRODUCT
  JOIN INVOICE_LINE ON INVOICE_LINE.PRODUCT_ID = PRODUCT.PRODUCT_ID
  JOIN INVOICE ON INVOICE.INVOICE_ID = INVOICE_LINE.INVOICE_ID
WHERE PRODUCT.PRICE > 1000 AND INVOICE.INVOICE_DATE < date '2015-04-01';
```

Na saída do trace veremos as estatísticas de execução e as estatísticas por tabela:

[illegible]

5.3. Obtenção de estatísticas por tabela usando tabelas de monitoramento

Como mencionado acima, durante a operação o Firebird coleta várias estatísticas de runtime e as armazena na memória. Essas estatísticas podem ser vistas nas tabelas de monitoramento. As

estatísticas são coletadas em vários níveis. O nível no qual as estatísticas são coletadas nas tabelas de monitoramento é determinado pela coluna `MON$STAT_GROUP`. Ela pode ter os seguintes valores:

- 0 — banco de dados (database);
- 1 — conexão com o banco de dados (connection);
- 2 — transação (transaction);
- 3 — instrução sendo executada (statement);
- 4 — chamada (call). Chamadas aninhadas em PSQL;
- 5 — instrução compilada (compiled statement).



É importante notar aqui que os dados nas tabelas de monitoramento não são armazenados. Durante o acesso às tabelas de monitoramento, é tirado um instantâneo da atividade atual, portanto, o tempo de vida dos dados em cada um dos níveis é limitado. Isto significa o seguinte:

- Você não pode obter estatísticas de uma consulta após a conclusão da execução dessa consulta (cursor fechado, drop statement executado). No entanto, você pode obter estatísticas enquanto a consulta está sendo executada.
- Você não pode obter estatísticas de uma transação após a conclusão da transação.
- Você não pode obter estatísticas de uma conexão após a conclusão da conexão.
- As estatísticas de nível de banco de dados também são redefinidas após a desconexão da última conexão com o banco de dados na arquitetura SuperServer.

Também é importante notar que o instantâneo da atividade atual é feito no primeiro acesso a qualquer tabela de monitoramento e é mantido até o final da transação, portanto, para que os valores dos contadores nas tabelas de monitoramento sejam atualizados, é necessário finalizar a transação atual, iniciar uma nova e executar novamente a consulta às tabelas `MON$`.

As estatísticas por tabela podem ser obtidas usando a tabela `MON$RECORD_STATS`. Para obter os nomes das tabelas, usa-se a tabela `MON$TABLE_STATS`.

5.3.1. MON\$TABLE_STATS

Estatísticas no nível da tabela.

Tabela 7. Descrição das colunas da tabela `MON$TABLE_STATS`

Nome da coluna	Tipo de dados	Descrição
<code>MON\$STAT_ID</code>	INTEGER	Identificador da estatística.

Nome da coluna	Tipo de dados	Descrição
MON\$STAT_GROUP	SMALLINT	Grupo da estatística: 0 — banco de dados (database); 1 — conexão com o banco de dados (connection); 2 — transação (transaction); 3 — instrução (statement); 4 — chamada (call).
MON\$TABLE_NAME	CHAR(63)	Nome da tabela.
MON\$RECORD_STAT_ID	INTEGER	Referência para MON\$RECORD_STATS.

5.3.2. MON\$RECORD_STATS

Estatísticas no nível de registros.

Tabela 8. Descrição das colunas da tabela MON\$RECORD_STATS

Nome da coluna	Tipo de dados	Descrição
MON\$STAT_ID	INTEGER	Identificador da estatística.
MON\$STAT_GROUP	SMALLINT	Grupo da estatística: 0 — banco de dados (database); 1 — conexão com o banco de dados (connection); 2 — transação (transaction); 3 — instrução (statement); 4 — chamada (call).
MON\$RECORD_SEQ_READS	BIGINT	Número de registros lidos sequencialmente (read sequentially).
MON\$RECORD_IDX_READS	BIGINT	Número de registros lidos por meio de um índice (read via an index).
MON\$RECORD_INSERTS	BIGINT	Número de registros inseridos (inserted records).
MON\$RECORD_UPDATES	BIGINT	Número de registros modificados (updated records).
MON\$RECORD_DELETES	BIGINT	Número de registros excluídos (deleted records).
MON\$RECORD_BACKOUTS	BIGINT	Número de exclusões de versões de registros criadas durante rollback (backed out records).
MON\$RECORD_PURGES	BIGINT	Número de exclusões de versões antigas de registros (purged records).

Nome da coluna	Tipo de dados	Descrição
MON\$RECORD_EXPUNGES	BIGINT	Número de exclusões de toda a cadeia de versões de um registro, se a versão mais recente for excluída e não for necessária para outras transações (expunged records).
MON\$RECORD_LOCKS	BIGINT	Número de registros lidos usando a cláusula WITH LOCK.
MON\$RECORD_WAITS	BIGINT	Número de tentativas de atualização/modificação/bloqueio de registros pertencentes a várias transações ativas. A transação está no modo WAIT.
MON\$RECORD_CONFLICTS	BIGINT	Número de tentativas malsucedidas de atualização/modificação/bloqueio de registros pertencentes a várias transações ativas. Nessas situações, é relatado um conflito de atualização (UPDATE CONFLICT).
MON\$BACKVERSION_READS	BIGINT	Número de versões lidas durante a busca por versões visíveis de registros.
MON\$FRAGMENT_READS	BIGINT	Número de fragmentos de registros lidos.
MON\$RECORD_RPT_READS	BIGINT	Número de registros relidos.
MON\$RECORD_IMGC	BIGINT	Número de registros limpos pela coleta de lixo intermediária.

5.3.3. Exemplos de obtenção de estatísticas por tabela usando tabelas MON\$

A seguir, consideraremos vários exemplos de obtenção de estatísticas por tabela usando tabelas de monitoramento em diferentes níveis.

Se você executar uma consulta no isql, após sua conclusão o cursor é fechado e a consulta é liberada. Assim, é impossível obter estatísticas completas de execução da consulta por tabela (é possível obter apenas estatísticas durante sua execução, mas elas serão incompletas). No entanto, se você tiver um aplicativo cliente com uma grade de dados, no qual todos os registros são buscados, e após a busca dos dados a consulta não é destruída imediatamente, então é possível obter tais estatísticas.

Vamos executar no aplicativo com grade de dados a seguinte consulta:

```
SELECT
  COUNT(*)
FROM
  PRODUCT
```



```

JOIN INVOICE_LINE ON INVOICE_LINE.PRODUCT_ID = PRODUCT.PRODUCT_ID
JOIN INVOICE ON INVOICE.INVOICE_ID = INVOICE_LINE.INVOICE_ID
WHERE PRODUCT.PRICE > 1000 AND INVOICE.INVOICE_DATE < date '2015-04-01';

```

Não feche a grade de dados e não faça commit da transação. Em outra transação deste mesmo aplicativo ou em outra sessão do isql você pode executar a seguinte consulta:

```

SELECT
  S.MON$SQL_TEXT,
  S.MON$EXPLAINED_PLAN,
  TS.MON$TABLE_NAME,
  -----
  RS.MON$RECORD_SEQ_READS,
  RS.MON$RECORD_IDX_READS,
  RS.MON$RECORD_RPT_READS,
  RS.MON$BACKVERSION_READS,
  RS.MON$FRAGMENT_READS,
  -----
  RS.MON$RECORD_INSERTS,
  RS.MON$RECORD_UPDATES,
  RS.MON$RECORD_DELETES,
  RS.MON$RECORD_BACKOUTS,
  RS.MON$RECORD_PURGES,
  RS.MON$RECORD_EXPUNGES,
  RS.MON$RECORD_IMGC,
  -----
  RS.MON$RECORD_LOCKS,
  RS.MON$RECORD_WAITS,
  RS.MON$RECORD_CONFLICTS
FROM
  MON$STATEMENTS S
  JOIN MON$TABLE_STATS TS
    ON TS.MON$STAT_ID = S.MON$STAT_ID
   AND TS.MON$STAT_GROUP = 3
  JOIN MON$RECORD_STATS RS
    ON RS.MON$STAT_ID = TS.MON$RECORD_STAT_ID
   AND RS.MON$STAT_GROUP = 3
WHERE S.MON$SQL_TEXT CONTAINING 'WHERE PRODUCT.PRICE > 1000 AND INVOICE.INVOICE_DATE';

```

Para buscar uma consulta específica, foi adicionado um filtro pelo texto da consulta.

```

S.MON$SQL_TEXT CONTAINING 'WHERE PRODUCT.PRICE > 1000 AND INVOICE.INVOICE_DATE'

```

Como condições de refinamento, você pode adicionar um filtro por S.MON\$ATTACHMENT_ID.

As estatísticas em outros níveis não permitem responder à questão de como exatamente as tabelas foram lidas em uma consulta específica, mas podem ser úteis para responder à questão de quão intensamente determinadas tabelas são usadas na transação/conexão/banco de dados.

Para visualizar estatísticas por tabela no nível do banco de dados, execute a seguinte consulta:

```

SELECT
  TS.MON$TABLE_NAME,
  -----
  RS.MON$RECORD_SEQ_READS,
  RS.MON$RECORD_IDX_READS,
  RS.MON$RECORD_RPT_READS,
  RS.MON$BACKVERSION_READS,
  RS.MON$FRAGMENT_READS,
  -----
  RS.MON$RECORD_INSERTS,
  RS.MON$RECORD_UPDATES,
  RS.MON$RECORD_DELETES,
  RS.MON$RECORD_BACKOUTS,
  RS.MON$RECORD_PURGES,
  RS.MON$RECORD_EXPUNGES,
  RS.MON$RECORD_IMGC,
  -----
  RS.MON$RECORD_LOCKS,
  RS.MON$RECORD_WAITS,
  RS.MON$RECORD_CONFLICTS
FROM
  MON$TABLE_STATS TS
  JOIN MON$RECORD_STATS RS
    ON RS.MON$STAT_ID = TS.MON$RECORD_STAT_ID
    AND RS.MON$STAT_GROUP = 0
WHERE TS.MON$STAT_GROUP = 0

```

5.4. Conclusão

Neste capítulo, você aprendeu sobre o que são estatísticas por tabela, as formas de obtê-las e como interpretar os valores dos contadores de estatísticas por tabela ao avaliar o desempenho de consultas.

Capítulo 6. Obtendo estatísticas com gstat

Nas partes anteriores, o funcionamento do otimizador de consultas, bem como a obtenção de várias métricas de desempenho, foram descritos em detalhes. Você aprendeu que atualmente o Firebird tem muito pouca estatística armazenada — apenas a seletividade do índice e a seletividade do conjunto de seus segmentos. A cardinalidade das tabelas é estimada durante a preparação da consulta (run-time). Na maioria dos casos, isso permite construir um plano de execução de consulta suficientemente otimizado, mas nem sempre. Se o otimizador construir um plano de execução de consulta insuficientemente bom, uma das razões pode ser estatísticas armazenadas desatualizadas para índices ou uma estimativa incorreta da cardinalidade da tabela.

No Firebird, existe a ferramenta gstat, que é projetada para obter informações estatísticas sobre o banco de dados. Essas informações podem ser comparadas com estatísticas armazenadas ou com estatísticas de run-time, o que permite entender por que as expectativas e a realidade divergem. Nos próximos capítulos, começaremos a analisar exemplos práticos de otimização de várias consultas, e em alguns casos, para explicação, precisaremos de informações estatísticas obtidas pela utilidade gstat.

A utilidade gstat não se conecta ao banco de dados, como outras utilidades fazem. Em vez disso, ela abre os arquivos do banco de dados diretamente e lê dados “crus”. Por causa disso, o gstat não usa transações, e algumas estatísticas que ele coleta podem incluir dados que foram excluídos por outras transações do banco de dados.

Estatísticas em hosts remotos podem ser obtidas usando a Service API. Isso é usado por vários IDEs para trabalhar com Firebird (Red Expert, IBExpert, etc.), bem como ferramentas especializadas para análise de estatísticas, por exemplo, IB Analyst da IB Surgeon. No Firebird, para trabalhar com a Service API, existe a utilidade fbsvcmgr, que, entre outras coisas, pode obter as mesmas estatísticas que o gstat.

6.1. Descrição da utilidade gstat

A execução do gstat é realizada da seguinte forma:

```
gstat [<options>] <database>
```

ou

```
gstat <database> [<options>]
```



Em sistemas operacionais semelhantes ao POSIX, a utilidade gstat deve ser executada como o usuário root ou o usuário firebird. Isso ocorre porque as permissões padrão do sistema operacional ao criar um novo banco de dados são tais que apenas o proprietário — firebird — tem acesso aos arquivos do banco de dados. Até mesmo membros do grupo firebird não têm acesso de leitura por padrão.

O nome do banco de dados <database> não pode ser um banco de dados remoto, ele deve ser local, mas pode ser um alias para um banco de dados local. A razão pela qual ele deve ser local é que o gstat opera no nível físico dos arquivos, e não estabelece uma conexão com o servidor de banco de dados — ele lê o arquivo do banco de dados diretamente.



Você pode obter estatísticas do banco de dados remotamente usando a Service API. No Firebird, existe a utilidade de console fbsvcmgr para acessar vários serviços que usam a Service API. A obtenção de estatísticas com fbsvcmgr será descrita mais tarde.

Se gstat for chamado com uma opção incorreta ou com a opção `-?`, a ajuda sobre as opções existentes será exibida. Infelizmente, apenas a forma curta (abreviada) das opções é exibida.

```
gstat -?
```

```
usage:  gstat [options] <database> or gstat <database> [options]
```

```
Available switches:
```

```
-a      analyze data and index pages
-d      analyze data pages
-e      analyze database encryption
-h      analyze header page ONLY
-i      analyze index leaf pages
-s      analyze system relations in addition to user tables
-u      username
-p      password
-fetch  fetch password from file
-r      analyze average record and version length
-t      tablename <tablename2...> (case sensitive)
-role   SQL role name
-tr     use trusted authentication
-z      display version number
```

```
option -t accepts several table names only if used after <database>
```

Tabela 9. Opções de linha de comando da utilidade gstat

Opção	Descrição
-a[ll]	Esta é a opção padrão, é equivalente a -h[earer] -d[ata] -i[ndex].
-d[ata]	Obter estatísticas sobre os dados de todas ou apenas das tabelas especificadas. Índices de usuário, tabelas do sistema e índices do sistema não são analisados. Versões de registros não são analisadas.
-e[ryption]	Estatísticas de criptografia do banco de dados.
-h[earer]	Esta opção exibe estatísticas sobre o próprio banco de dados e, em seguida, termina. As informações do cabeçalho também são exibidas ao usar qualquer outra opção, portanto, você sempre obtém dados detalhados do cabeçalho do banco de dados em sua saída.

Opção	Descrição
<code>-i[ndex]</code>	Especificar este parâmetro faz com que gstat analise cada índice de usuário no banco de dados especificado. Tabelas de usuário, índices do sistema e tabelas do sistema não são analisados. Se a opção <code>-t</code> for especificada, apenas os índices das tabelas especificadas são analisados.
<code>-s[ystem]</code>	Esta opção é um modificador e altera a saída das opções <code>-d[ata]</code> ou <code>-i[ndex]</code>, adicionando ao relatório tabelas (ou índices) do sistema junto com as de usuário. Usar esta opção por si só é equivalente a chamar gstat com o parâmetro <code>-a[ll] -s[ystem]</code> especificado. Ao ser executada, esta opção exibe estatísticas para tabelas e índices do sistema, como RDB\$ e MON\$.
<code>-u[sername] <username></code>	Permite especificar o nome de usuário: SYSDBA ou o proprietário do banco de dados. Não é necessário especificá-lo se a variável de ambiente ISC_USER existir e tiver o valor correto, ou se você estiver logado no servidor com uma conta privilegiada (root, firebird).
<code>-p[assword] <password></code>	Fornece a senha para o nome de usuário especificado na opção <code>-u[sername]</code> .
<code>-fetch {<filename> stdin /dev/tty}</code>	Esta opção faz com que a senha seja lida de um arquivo, em vez de usar a senha especificada na linha de comando. O nome do arquivo especificado não deve estar entre aspas e deve ser legível pelo usuário que executa o gstat. Se o nome do arquivo for especificado como stdin, o usuário será solicitado a inserir a senha. Em sistemas POSIX, o nome do arquivo /dev/tty também levará a uma solicitação de senha.
<code>-r[ecord]</code>	A chave <code>-r[ecord]</code> é um modificador para as chaves <code>-d[ata]</code> e <code>-s[ystem]</code> . Ela adiciona dados sobre o comprimento médio do registro e da versão para as tabelas analisadas (de usuário e/ou do sistema). Esta chave não afeta a chave <code>-i[ndex]</code> .
<code>-t[able] <tablename> [... <tablename>]</code>	Esta opção permite analisar uma tabela ou uma lista de tabelas, bem como índices pertencentes às tabelas especificadas. Consulte a seção com advertências abaixo para saber sobre possíveis problemas com esta opção e exemplos de seu uso. Após a opção <code>-t[able]</code>, deve seguir uma lista de nomes de tabelas que você deseja analisar. A lista deve estar em maiúsculas e cada tabela deve ser separada por um espaço. Também é possível usar aspas duplas para que o gstat analise uma tabela cujo nome não esteja em maiúsculas ou contenha espaços e caracteres não ASCII. Não é necessário especificar a chave <code>-i[ndex]</code>, pois os índices pertencentes às tabelas especificadas serão analisados automaticamente. As informações do cabeçalho do banco de dados também são exibidas na saída.
<code>-role <rolename></code>	Permite especificar o nome da função. A função deve ser concedida ao usuário especificado na opção <code>-u[sername]</code> .

Opção	Descrição
-tr	Usar autenticação confiável
-z	Esta opção é um modificador. Usar -z exibe o número da versão da utilidade gstat e do servidor Firebird. Você deve especificar o nome correto do banco de dados e possivelmente outros parâmetros. Esta opção adiciona informações sobre a versão do gstat e do Firebird à saída de outra opção que você especificou, ou à saída padrão se você não a especificou. Se tudo o que você precisa são informações de versão, então a saída mais curta será se você especificar na opção -t uma tabela que não existe.

Para analisar tabelas usadas em consultas, precisaremos da seguinte forma de chamada do gstat:

```
gstat <database> -u <username> -p <password> -r -t <table-list>
```

Aqui <table-list> — é a lista de tabelas usadas na consulta SQL analisada.

Por exemplo, suponha que estamos analisando a seguinte consulta SQL:

```
SELECT COUNT(*)
FROM HORSE
JOIN COLOR ON COLOR.CODE_COLOR = HORSE.CODE_COLOR
JOIN BREED ON BREED.CODE_BREED = HORSE.CODE_BREED
WHERE ...
```

Então, para analisar as tabelas desta consulta, é necessário executar o seguinte comando:

```
gstat horses -u SYSDBA -p masterkey -r -t HORSE COLOR BREED
```

As informações exibidas no console podem ter um volume bastante grande, portanto, é recomendado redirecionar a saída para um arquivo, que pode então ser analisado.

```
gstat horses -u SYSDBA -p masterkey -r -t HORSE COLOR BREED > D:\stat\1.txt
```

6.2. Obtendo estatísticas com fbsvcmgr

Como mencionado anteriormente, a ferramenta gstat só pode operar no host local. Além disso, ela deve ser executada por um usuário com privilégios suficientes, o que nem sempre é possível. Para contornar essas limitações, você pode usar a Service API de seus próprios aplicativos ou de terceiros. No Firebird, existe a ferramenta de console fbsvcmgr, que fornece acesso a vários serviços através da Service API. Aqui não descreveremos todos os recursos da ferramenta fbsvcmgr, mas focaremos apenas naqueles necessários para obter estatísticas semelhantes às que podem ser obtidas com a ferramenta gstat.

O `fbsvcmgr` não emula os switches implementados nas ferramentas tradicionais “g*”. É apenas uma interface externa através da qual as funções e parâmetros da API de serviços são passados. Portanto, os usuários devem estar familiarizados com a API de serviços em sua forma atual. O arquivo de cabeçalho da API — `ibase.h`, localizado no diretório `../include` — deve ser considerado a principal fonte de informação sobre as funções disponíveis.

6.2.1. Sintaxe dos parâmetros do `fbsvcmgr`

O primeiro parâmetro obrigatório da linha de comando é o gerenciador de serviços ao qual você deseja se conectar. Para uma conexão local, use o valor `service_mgr`. Para se conectar a uma máquina remota, é necessário adicionar o nome do host no seguinte formato: `hostname:service_mgr`.

Em seguida, vêm blocos de parâmetros de serviço (SPB) com valores, se necessário. Qualquer um deles pode (ou não) ter um prefixo de um único caractere “-”. Para linhas de comando longas, típicas do `fbsvcmgr`, o uso do sinal “-” torna a linha de comando mais legível. Compare os seguintes exemplos:

```
fbsvcmgr service_mgr user sysdba password masterkey
        action_db_stats dbname employee sts_hdr_pages
```

e

```
fbsvcmgr service_mgr -user sysdba -password masterkey
        -action_db_stats -dbname employee -sts_hdr_pages
```

Sintaxe SPB

A sintaxe SPB que o `fbsvcmgr` entende é quase idêntica à que pode ser vista no arquivo `ibase.h` ou na documentação da API InterBase 6.0. Para reduzir a digitação e tornar a linha de comando um pouco mais curta, é usada uma forma abreviada.

Todos os parâmetros SPB têm uma de duas formas: (1) `isc_spb_VALUE` ou (2) `isc_VALUE1_svc_VALUE2`. Ao usar `fbsvcmgr`, para a primeira opção, basta digitar `VALUE`, para a segunda — `VALUE1_VALUE2`.

```
isc_spb_dbname => dbname
isc_action_svc_backup => action_backup
isc_spb_sec_username => sec_username
isc_info_svc_get_env_lock => info_get_env_lock
```

e assim por diante.



Uma exceção é `isc_spb_user_name`: pode ser especificado como `user_name` ou simplesmente como `user`.

6.2.2. Obtendo ajuda

Executar fbsvcmgr sem parâmetros exibe uma tela com informações de ajuda resumidas:

```
Usage: fbsvcmgr manager-name switches...
Manager-name should be service_mgr, may be prefixed with host name
according to common rules (host:service_mgr, \\host\\service_mgr).
Switches exactly match SPB tags, used in abbreviated form.
Remove isc_, spb_ and svc_ parts of tag and you will get the switch.
For example: isc_action_svc_backup is specified as action_backup,
              isc_spb_dbname => dbname,
              isc_info_svc_implementation => info_implementation,
              isc_spb_prp_db_online => prp_db_online and so on.
You may specify single action or multiple info items when calling fbsvcmgr once.
Full command line samples:
fbsvcmgr service_mgr user sysdba password masterkey action_db_stats dbname employee
sts_hdr_pages
  (will list header info in database employee on local machine)
fbsvcmgr yourserver:service_mgr user sysdba password masterkey info_server_version
info_svr_db_info
  (will show firebird version and databases usage on yourserver)
To get full list of known services run with -? switch
```

Como dito na última frase, executar fbsvcmgr com o switch -? exibirá informações adicionais sobre parâmetros gerais, uma lista de tags de informação, uma lista de serviços disponíveis e seus parâmetros. Exemplo de saída de ajuda (a ajuda é mostrada de forma truncada, apenas as informações sobre o serviço de obtenção de estatísticas são mantidas):

```
Attaching to services manager:
user [string value]
user_name [string value]
role [string value]
sql_role_name [string value]
password [string value]
fetch_password [file name]
trusted_auth
expected_db [string value]
key_holder [key holder plugin name]

...
Actions:
...
action_db_stats:
  dbname [string value]
  sts_record_versions
  sts_nocreation
  sts_table [string value]
  sts_data_pages
  sts_hdr_pages
  sts_idx_pages
  sts_sys_relations
  sts_encryption
```


...

6.2.3. Parâmetros de conexão do serviço

Ao se conectar ao gerenciador de serviços, é necessária autorização. Os parâmetros listados abaixo podem ser usados tanto para consultas de informação quanto para quaisquer ações.

Tabela 10. Parâmetros de conexão do serviço

Opção	Descrição
user [string value]	Nome de usuário para conexão com o serviço.
user_name [string value]	
role [string value]	Nome da função a ser usada na conexão.
sql_role_name [string value]	
password [string value]	Senha do usuário.
fetch_password [file name]	Extraí a senha do arquivo especificado.
trusted_auth	Usa autenticação confiável.
expected_db [string value]	Especifica um banco de dados de segurança não padrão. Para uma descrição detalhada do parâmetro, veja abaixo.

6.2.4. Usando serviços com um banco de dados de segurança não padrão

Se você deseja usar a API de serviços para acessar um banco de dados configurado para usar um banco de dados de segurança não padrão, ao se conectar ao gerenciador de serviços, você deve usar o elemento SPB `isc_spb_expected_db`. O valor desse elemento é o banco de dados ao qual se espera acesso.

Se for necessário acessar serviços de programas antigos usando um banco de dados de segurança não padrão, existe uma solução alternativa: definir a variável de ambiente `FB_EXPECTED_DB`, que será automaticamente adicionada ao SPB.

Exemplo. Suponha que temos as seguintes linhas em `database.conf`:

```
employee = $(dir_sampledb)/employee.fdb
{
    SecurityDatabase = employee
}
```

ou seja, o banco de dados `employee` está configurado como banco de dados de segurança para si mesmo.

Para acessá-lo usando `fbsvcmgr`, é necessário usar o seguinte comando:

```
fbsvcmgr host:service_mgr -user sysdba -password xxx -expected_db employee
```

```
-action_db_stats -dbname employee -sts_data_pages
```

Ou você pode definir previamente a variável de ambiente `FB_EXPECTED_DB`:

```
export FB_EXPECTED_DB=employee
fbsvcmgr host:service_mgr -user sysdba -password xxx -action_db_stats
        -dbname employee -sts_data_pages
```

Naturalmente, qualquer outra ação pode ser usada aqui.

6.2.5. Parâmetros para obter estatísticas com fbsvcmgr

Para obter estatísticas do banco de dados, semelhantes às que podem ser obtidas com a ferramenta `gstat`, é necessário especificar a ação `action_db_stats`.

Tabela 11. Parâmetros para a ação `action_db_stats`

Opção	Descrição
<code>dbname [string value]</code>	Nome do arquivo do banco de dados ou alias.
<code>sts_record_versions</code>	Adiciona às estatísticas das tabelas informações sobre o comprimento médio do registro e versão. Análogo ao switch <code>-r</code> da ferramenta <code>gstat</code> .
<code>sts_nocreation</code>	Exclui da estatística a data de criação (ou restauração) do banco de dados.
<code>sts_table [string value]</code>	Especifica uma tabela ou várias tabelas. Cada tabela deve ser especificada em um switch <code>sts_table</code> separado. Análogo a <code>gstat -t</code> .
<code>sts_data_pages</code>	Estatísticas de dados das tabelas. Análogo ao switch <code>-d</code> da ferramenta <code>gstat</code> .
<code>sts_hdr_pages</code>	Estatísticas da página de cabeçalho do banco de dados. Análogo ao switch <code>-h</code> da ferramenta <code>gstat</code> .
<code>sts_idx_pages</code>	Estatísticas de índices. Análogo ao switch <code>-i</code> da ferramenta <code>gstat</code> .
<code>sts_sys_relations</code>	Inclui tabelas e índices do sistema nas estatísticas. Análogo ao switch <code>-s</code> da ferramenta <code>gstat</code> .
<code>sts_encryption</code>	Estatísticas de criptografia do banco de dados. Análogo ao switch <code>-e</code> da ferramenta <code>gstat</code> .

6.2.6. Exemplo de análise de estatísticas para tabelas e índices de uma consulta

Suponha que precisemos obter estatísticas para as tabelas da seguinte consulta SQL:

```
SELECT COUNT(*)
FROM HORSE
```

```
JOIN COLOR ON COLOR.CODE_COLOR = HORSE.CODE_COLOR
JOIN BREED ON BREED.CODE_BREED = HORSE.CODE_BREED
WHERE ...
```

Então, para analisar as tabelas desta consulta, é necessário executar o seguinte comando:

```
fbsvcmgr inet://localhost/service_mgr -user sysdba -password masterkey action_db_stats
  -dbname horses -sts_record_versions
  -sts_table HORSE -sts_table COLOR -sts_table BREED
```

Este comando dará o mesmo resultado que

```
gstat horses -u SYSDBA -p masterkey -r -t HORSE COLOR BREED
```

Se as estatísticas precisarem ser salvas em um arquivo, modifique o comando escrito anteriormente da seguinte forma:

```
fbsvcmgr inet://localhost/service_mgr -user sysdba -password masterkey action_db_stats
  -dbname horses -sts_record_versions
  -sts_table HORSE -sts_table COLOR -sts_table BREED > D:\stat\1.txt
```

6.3. Análise das estatísticas obtidas

As estatísticas obtidas contêm três blocos principais de informação: estatísticas da página de cabeçalho (sempre exibida), estatísticas por tabela ou tabelas, estatísticas por índices da tabela ou tabelas. Vamos examinar cada um desses blocos separadamente.

6.3.1. Estatísticas da página de cabeçalho

A página de cabeçalho contém várias informações relacionadas a todo o banco de dados. Parte da informação é estática, parte muda dependendo das alterações que ocorrem no banco de dados. Esta informação praticamente não é usada pelo otimizador, portanto não faremos uma análise detalhada do conteúdo desta parte das estatísticas. No entanto, a decodificação deste conteúdo será descrita.

Como mencionado acima, as estatísticas da página de cabeçalho serão sempre exibidas junto com outras estatísticas. Se você deseja obter apenas as estatísticas da página de cabeçalho, use a opção `-h` da utilitário `gstat`. Por exemplo:

```
gstat -h employee
```

ou usando os serviços

```
fbsvcmgr inet://localhost/service_mgr -user sysdba -password masterkey action_db_stats
```

```
-dbname employee -sts_hdr_pages
```

A saída será a seguinte:

```
Database "C:\Firebird\5.0\examples\empbuild\employee.fdb"
Gstat execution time Mon Jan 20 11:13:35 2025

Database header page information:
    Flags                      0
    Generation                 183
    System Change Number      0
    Page size                  8192
    ODS version                13.1
    Oldest transaction         169
    Oldest active              170
    Oldest snapshot            170
    Next transaction           172
    Sequence number            0
    Next attachment ID         10
    Implementation             HW=AMD/Intel/x64 little-endian OS=Windows CC=MSVC
    Shadow count                0
    Page buffers                0
    Next header page            0
    Database dialect            3
    Creation date               Jan 13, 2025 21:11:17
    Attributes

Variable header data:
    Database GUID: {7F83D4D2-C3AC-4012-95C8-9C4FEB22B174}
    *END*
Gstat completion time Mon Jan 20 11:13:35 2025
```

A primeira linha da saída exibe o nome do arquivo (ou arquivos) e o caminho para o banco de dados. Isso pode ser útil se um alias de banco de dados for usado e for necessário determinar sua localização exata. Como o banco de dados de funcionários é de arquivo único, apenas um arquivo é exibido na saída. Se fosse um banco de dados multifile, o final da listagem acima seria assim:

```
...
Variable header data:
    Database GUID: {7F83D4D2-C3AC-4012-95C8-9C4FEB22B174}
    Continuation file: C:\Firebird\5.0\examples\empbuild\employee_multy.fdb1
    Last logical page: 162
```

A descrição dos vários campos do cabeçalho é fornecida abaixo.

Flags

Um conjunto de sinalizadores que define características importantes do comportamento do banco de dados. Os sinalizadores são definidos apenas por ferramentas especiais, como `gfix`. Alterar sinalizadores com outras ferramentas é perigoso — pode levar à corrupção do banco de dados.

Vale ressaltar que ao obter estatísticas, o valor do parâmetro `Flags` é sempre exibido como zero, independentemente dos sinalizadores definidos.

Generation

Um contador que é incrementado em um toda vez que a página de cabeçalho é escrita no disco.

System Change Number

Um marcador (número de 0 a 3) que marca as páginas do banco de dados ao criar cópias incrementais usando a utilitário `nbackup`. O `nbackup` realiza backup das páginas do banco de dados, copiando as páginas que foram alteradas desde o último backup do nível anterior. Se um backup de nível 0 for feito, todas as páginas são copiadas, e se for de nível 1 — apenas as páginas que foram alteradas após o último backup de nível 0 são copiadas. Para poder encontrar páginas alteradas, o Firebird usa um marcador chamado SCN. Este número aumenta a cada mudança no estado de backup e não depende do nível de backup.

- 0 — páginas antes do backup;
- 1 — páginas gravadas no arquivo delta durante o backup;
- 2 — páginas gravadas durante a mesclagem do arquivo delta com o backup principal;
- 3 — páginas gravadas após o término do primeiro backup e da mesclagem.

O backup e a restauração usando `gbak` não restauram o conteúdo da tabela `RDB$BACKUP_HISTORY` e redefinem o SCN de todas as páginas de volta para 0. Assim, a restauração usando `gbak` sobrescreve todo o banco de dados (e pode até alterar o tamanho da página). Isso torna os backups anteriores usando `nbackup` inúteis como ponto de partida para backups subsequentes: é necessário começar novamente do nível 0.

Page size

O tamanho da página do banco de dados em bytes. Todos os arquivos de um mesmo banco de dados consistem em páginas do mesmo tamanho, que é definido na criação do banco de dados e em sua restauração a partir de um backup. Muitos dos parâmetros mais importantes do servidor dependem do tamanho da página — por exemplo, o tamanho do cache de página do banco de dados em bytes. Recomenda-se criar o banco de dados com tamanho de página de pelo menos 4096 bytes, e preferencialmente 8192 ou 16384 bytes.

ODS version

A versão da estrutura do banco de dados em disco (On-Disk Structure). Representada por dois números separados por um ponto. A parte “inteira” é a versão principal do ODS, que depende da versão do servidor que criou este banco de dados. A versão principal define as capacidades básicas de trabalho com o banco de dados, e seu valor é atribuído na criação (ou restauração) do banco de dados. A parte “fracionária” (após o ponto) é a versão secundária do ODS.

Possíveis valores que você pode ver aqui:

- 10.0 — Firebird 1.0;
- 10.1 — Firebird 1.5;
- 11.0 — Firebird 2.0;
- 11.1 — Firebird 2.1;
- 11.2 — Firebird 2.5;
- 12.0 — Firebird 3.0;
- 13.0 — Firebird 4.0;
- 13.1 — Firebird 5.0.

Oldest transaction

O identificador da transação de interesse mais antiga no banco de dados (Oldest Interesting Transaction ou OIT). A primeira transação com estado diferente de commit. Na prática, pode ser:

- a transação ativa mais antiga (Oldest Active Transaction);
- o número da transação mais antiga concluída com um rollback “verdadeiro” (executado se o usuário modificou mais de 100.000 registros em uma transação);
- uma transação no estado in-limbo (semi-concluída) de um commit de duas fases.

O valor deste parâmetro é frequentemente comparado com Oldest snapshot. A diferença entre esses parâmetros influencia o início automático do sweep.

Oldest active	A transação mais antiga que um usuário iniciou e ainda não concluiu com commit ou rollback (Oldest Active Transaction ou OAT).
Oldest snapshot	O identificador da transação mais antiga que atualmente não está sujeita à coleta de lixo (Oldest snapshot transaction ou OST). Qualquer transação com este ou um identificador maior não pode ter versões antigas de registros que possam ser removidas pela coleta de lixo tradicional ou sweep. Normalmente, OST é maior ou igual a OAT. A diferença entre este valor e OIT, se for maior que o intervalo de limpeza do banco de dados (sweep interval)—desde que a limpeza automática não esteja desativada—determina se a limpeza automática (sweep) ocorre.
Next transaction	O número que será atribuído à próxima transação quando ela for iniciada.
Sequence number	O número de sequência do arquivo do banco de dados. Para um banco de dados de arquivo único, este número é sempre zero. O segundo arquivo no banco de dados terá o número 1, o terceiro — 2 e assim por diante.
Next attachment ID	O número da próxima conexão a este banco de dados. Cada vez que um aplicativo se conecta ao banco de dados, este número é incrementado em um. A inicialização e o encerramento do banco de dados também incrementam este número. A execução do gstat não altera este identificador, pois o gstat não se conecta ao banco de dados como um aplicativo comum.
Implementation	A arquitetura de hardware na qual o banco de dados foi criado.
Shadow count	O número de arquivos de shadow definidos para este banco de dados.
Page buffers	O tamanho do cache do banco de dados em páginas. Zero significa que o banco de dados usa o valor do parâmetro DefaultDbCachePages, especificado no arquivo de configuração (firebird.conf ou databases.conf).
Next header page	O número da próxima página de cabeçalho. Sempre igual a zero. Na verdade, qualquer página no banco de dados tem uma referência ao número da próxima página do mesmo tipo, mas, como a página de cabeçalho é sempre única em cada arquivo do banco de dados, esta referência é zerada.
Database dialect	O dialeto SQL do banco de dados.
Creation date	A data de criação do banco de dados ou da última restauração a partir de um backup.

Attributes

Diversos atributos do banco de dados. Os seguintes valores são possíveis:

- **forced write** — modo de gravação forçada ativado (forced write on).
- **no reserve** — indica que não há espaço reservado para versões antigas dos dados nas páginas. Isso permite empacotar os dados de forma mais densa em cada página, fazendo com que o banco de dados ocupe menos espaço em disco. É bom para bancos de dados somente leitura, mas tem impacto negativo em bancos de dados onde é necessário realizar atualizações ou exclusões de registros.
- **read only** — o banco de dados opera em modo somente leitura (Read Only). Se o sinalizador não estiver definido, tanto leitura quanto gravação são permitidas.
- **multi-user maintenance** — o banco de dados está bloqueado em modo de manutenção multiusuário (multi). Múltiplas conexões são permitidas apenas para o usuário SYSDBA ou proprietário do banco de dados.
- **single-user maintenance** — o banco de dados está bloqueado em modo de manutenção de usuário único (single). Apenas uma conexão é permitida para o usuário SYSDBA ou proprietário do banco de dados.
- **full shutdown** — o banco de dados está bloqueado em modo de desligamento completo (full).
- **backup lock** — o arquivo principal do banco de dados está temporariamente bloqueado. As alterações são confirmadas em um arquivo delta temporário, que é usado para backup.
- **backup merge** — mesclagem do arquivo delta com o arquivo principal do banco de dados.

Variable header data

Dados variáveis do cabeçalho da página. Esta parte do relatório abrange informações que não estão na parte fixa do cabeçalho do banco de dados. Por exemplo, intervalo de limpeza (sweep interval), GUID do banco de dados, GUID do último backup e outros.

Informações importantes para otimização de consultas aqui são: **Page size** e **Page buffers**.

O tamanho da página (**Page size**) afeta a profundidade dos índices, que pode ser vista no bloco de estatísticas de índices. Não é recomendado usar um tamanho de página menor que 8192 bytes. No entanto, um tamanho de página muito grande afeta negativamente a concorrência de conexões pela mesma página, portanto, aumentar o tamanho da página acima de 16384 deve ser feito com grande cautela.

O tamanho do cache de página (**Page buffers**) afeta a frequência com que ocorrem leituras físicas

ao acessar repetidamente as mesmas páginas. Recomendamos definir o valor 0 no nível do cabeçalho do banco de dados. Nesse caso, o banco de dados usa o valor do parâmetro `DefaultDbCachePages` do arquivo de configuração (`firebird.conf` ou `databases.conf`). Isso é muito mais conveniente para administradores de banco de dados.

Os valores **Oldest transaction**, **Oldest active**, **Oldest snapshot** e **Next transaction** são importantes como indicadores de quão bem o aplicativo gerencia transações. Seus valores não podem ser usados para interpretar o desempenho de uma consulta específica, pois esses contadores se referem a todo o banco de dados, e não a tabelas e índices específicos envolvidos na consulta. Por esse motivo, não os consideraremos neste guia.

6.3.2. Estatísticas da tabela

Para analisar todas as tabelas de usuário, a opção `-d[ata]` da utilitário `gstat` é usada. As tabelas são listadas em ordem alfabética. Para incluir tabelas do sistema no relatório, adicione a opção `-system`. Nenhum índice será analisado ou listado.

Se você deseja analisar não todas as tabelas de usuário, mas apenas de uma lista específica de tabelas, pode especificá-la usando a opção `-t[able]`. Observe que especificar nomes de tabelas dessa maneira também analisa todos os índices associados a essas tabelas. Os nomes das tabelas são separados por espaços.

Exemplo de uso do `gstat` com o parâmetro `-t`:

```
gstat employee -user sysdba -t EMPLOYEE JOB COUNTRY
```

Se os nomes das tabelas foram criados usando identificadores que diferenciam maiúsculas de minúsculas (entre aspas duplas), então, em vez de tabelas em maiúsculas, é necessário usar identificadores entre aspas duplas no caso original.

```
gstat mymusic -t "MyMusic_Titles" "MyMusic_Artists"
```

Quando você executa `gstat` com as opções padrão ou com o parâmetro `-d[ata]` ou `-t[able]` e adiciona a opção `-r[ecord]`, você obtém no relatório informações adicionais que mostram o comprimento médio do registro e informações médias sobre versões da tabela. Além disso, as estatísticas incluirão informações sobre colunas do tipo BLOB. Isso fornece informações adicionais que podem ser usadas para avaliar o desempenho das consultas e otimizá-las.

```
gstat horses -u SYSDBA -p masterkey -r -t HORSE COLOR BREED
```

Ou em um host remoto usando a API de serviços:

```
fbsvcmgr inet://localhost/service_mgr -user sysdba -password masterkey action_db_stats
  -dbname horses -sts_record_versions
  -sts_table HORSE -sts_table COLOR -sts_table BREED
```

Vejamos um exemplo da saída de tais estatísticas (sem índices) para uma das tabelas:

```
...
HORSE (179)
  Primary pointer page: 709, Index root page: 710
  Total formats: 2, used formats: 1
  Average record length: 237.77, total records: 545113
  Average version length: 0.00, total versions: 0, max versions: 0
  Average fragment length: 0.00, total fragments: 0, max fragments: 0
  Average unpacked length: 2100.00, compression ratio: 8.83
  Pointer pages: 4, data page slots: 10176
  Data pages: 10176, average fill: 90%
  Primary pages: 9545, secondary pages: 631, swept pages: 9538
  Empty pages: 7, full pages: 10165
  Blobs: 73761, total length: 7620692, blob pages: 0
    Level 0: 73761, total length: 7620692, blob pages: 0
  Table size: 166789120 bytes
  Fill distribution:
    0 - 19% = 8
    20 - 39% = 0
    40 - 59% = 1
    60 - 79% = 2
    80 - 99% = 10165
...
```

Descrição da saída das estatísticas da tabela:

Primary pointer page	Número da primeira página de ponteiros indiretos para páginas que armazenam dados da tabela.
Index root page	Número da página que é a primeira página de ponteiros para os índices da tabela.
Total formats	Número total de formatos para a tabela.
used formats	Número de formatos usados.
Average record length	Tamanho médio do registro compactado em bytes.
total records	Número total de registros na tabela. Este valor pode incluir registros adicionados em transações não confirmadas ou registros excluídos não limpos pela coleta de lixo.
Average version length	Valor médio do comprimento da cadeia de versões em bytes.
total versions	Número total de versões antigas. Se o valor for 0, não há versões antigas de registros na tabela. Ou não houve atualizações ou exclusões de registros na tabela, ou a coleta de lixo removeu as versões antigas dos registros.

max versions	Comprimento máximo da cadeia de versões antigas de um registro.
Average fragment length	Tamanho médio do fragmento do registro em bytes.
total fragments	Número total de fragmentos de todos os registros.
max fragments	Número máximo de fragmentos para um único registro.
Average unpacked length	Tamanho médio do registro não compactado em bytes.
compression ratio	Taxa de compactação – relação entre o tamanho dos registros não compactados e os compactados.
Pointer pages	Número total de páginas de ponteiros indiretos para páginas que armazenam dados da tabela.
data page slots	Número de ponteiros para páginas do banco de dados contidos nas páginas de ponteiros. Deve ser igual ao número de páginas de dados.
Data pages	Número total de páginas que armazenam dados da tabela. Este contador inclui páginas que armazenam versões não confirmadas de registros e lixo, porque o gstat não pode distingui-las umas das outras.
average fill	Percentual médio de preenchimento das páginas. Fill distribution (veja abaixo) fornece informações mais detalhadas.
Primary pages	Número de páginas primárias, igual a (Data pages - Secondary pages).
secondary pages	Número de páginas secundárias, ou seja, páginas que não armazenam versões primárias de registros.
swept pages	Número de páginas marcadas com o sinalizador swept. Este sinalizador é definido durante a coleta de lixo ou pelo procedimento sweep, se a página contiver apenas versões primárias de registros e todas elas foram criadas por transações confirmadas. Versões secundárias de registros podem ser removidas pela coleta de lixo. Tais páginas de dados devem ser ignoradas pelo procedimento sweep.
Empty pages	Número de páginas que não contêm registros.
full pages	Número de páginas completamente preenchidas.
Blobs	Número total de BLOBs de nível 0, 1 e 2.
total length	Tamanho total ocupado por todos os campos BLOB em todos os registros, em bytes.

blob pages	Número de páginas BLOB.
Level <n>	Número de BLOBs de cada nível. Para cada nível, o tamanho total em bytes total length e o número de páginas BLOB blob pages também são exibidos. Para o nível zero, o número de páginas BLOB é sempre 0.
Table size	Tamanho da tabela em bytes.
Fill distribution	Histograma de cinco "faixas" de 20%, cada uma mostrando o número de páginas de dados cujo preenchimento médio se enquadra nesse intervalo. A porcentagem de preenchimento é determinada pela proporção do espaço de cada página que contém dados. A soma desses números fornece o número total de páginas que contêm dados.

O que pode ser interessante aqui do ponto de vista do desempenho da execução da consulta?

Em primeiro lugar, deve-se prestar atenção ao número de registros indicado em **total records**. Este valor pode incluir registros adicionados em transações não confirmadas ou registros excluídos não limpos pela coleta de lixo, portanto não é exato. O valor **total records** pode ser usado como um valor aproximado da cardinalidade da tabela analisada. Este valor pode diferir significativamente da estimativa do otimizador.

Lembre-se, o otimizador estima a cardinalidade da tabela da seguinte forma. A primeira página de dados (DP) é lida e nela é determinado o tamanho médio do registro após a compactação. Com base no tamanho da página e no tamanho médio do registro compactado, determina-se quantos registros cabem em uma página de dados. Em seguida, as páginas de ponteiros (PP) são percorridas e determina-se quantas páginas de dados existem no total. Depois disso, o número de páginas é multiplicado pelo número médio de registros por página. Claro, esse valor pode ser impreciso, pois o tamanho médio do registro na primeira página de dados pode diferir do tamanho nas páginas subsequentes. Mas o otimizador precisa estimar rapidamente a cardinalidade sem varrer toda a tabela e, em geral, a cardinalidade obtida pode ser usada para otimização.

As estatísticas obtidas com gstat, o tamanho médio do registro **Average unpacked length** e o tamanho médio do registro compactado **Average record length** são calculados em todas as páginas (e não apenas na primeira, como o otimizador faz), portanto esses valores são mais precisos. A relação entre o tamanho médio do registro e o tamanho médio do registro compactado é a taxa de compactação (**compression ratio**).

A taxa de compressão (**compression ratio**) é um parâmetro bastante importante; quanto maior ela for, mais registros podem caber em uma única página de dados, o que reduz a quantidade de leituras físicas de páginas. Ao mesmo tempo, a descompressão de registros não é gratuita. Isso pode ser visto realizando o seguinte experimento.

Compare o tempo de execução das duas consultas a seguir:

```
-- descompressão de registros não é necessária
select count(*)
```

```
from horse;
```

```

COUNT
=====
545113

Current memory = 551592000
Delta memory = 96
Max memory = 551676768
Elapsed time = 0.136 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 583269
```

Nesta consulta, a descompressão de registros não é necessária, pois os valores dos campos não são lidos.

```
-- descompressão de registros é necessária
select count(code_horse)
from horse;
```

```

COUNT
=====
545113

Current memory = 551610720
Delta memory = 176
Max memory = 551692000
Elapsed time = 0.203 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 583269
```

Neste caso, a descompressão de registros é necessária, pois é preciso saber o valor do campo `code_horse`.

Como você pode ver, a descompressão de registros ocupa uma parte significativa do tempo de execução da consulta — aproximadamente 30% do tempo. Na maioria dos casos, a descompressão de registros é necessária, pois é necessário comparar os valores dos campos.

Páginas primárias e secundárias

Não menos importantes são as informações sobre a quantidade de páginas da tabela. **Data pages** — quantidade total de páginas de dados da tabela. Dessas páginas, uma parte pode ser primária (**Primary pages**), e outra parte secundária (**secondary pages**). As páginas podem estar vazias (**Empty pages**), cheias (**full pages**) e parcialmente preenchidas. As seguintes relações são verdadeiras:

Data pages = Primary pages + secondary pages

Empty pages + full pages $\leq$ Data pages

swept pages $\leq$ Primary pages

Páginas secundárias (**secondary pages**) — são páginas que não contêm as versões primárias dos registros. Elas podem conter BLOBs de nível 0, fragmentos de registros ou versões de registros. Consequentemente, nas páginas primárias (**Primary pages**) estão necessariamente presentes as versões primárias dos registros, mas também podem estar presentes fragmentos e/ou versões de registros. Frequentemente, para uma varredura completa da tabela, é necessário ler apenas as páginas primárias não vazias. Isso é fácil de ver executando a seguinte consulta com um cache de páginas não preenchido (ou se este cache for pequeno):

Statement 157:

```
select count(*) from horse
```

Select Expression

-> Aggregate

-> Table "HORSE" Full Scan

1 records fetched

207 ms, 9537 read(s), 583269 fetch(es)

Table	Natural	Index	Update	Insert	Delete	Backout	Purge	Expunge

HORSE	545113							

Neste caso, para uma varredura completa da tabela, foram necessárias 9537 leituras físicas. Isso é quase exatamente igual ao número de páginas primárias menos o número de páginas vazias $\text{Primary pages} - \text{Empty pages} = 9545 - 7 = 9538$. No entanto, tal relação é verdadeira apenas na ausência de versões de registros. Neste caso, o banco de dados foi recentemente restaurado, um sweep foi executado e a tabela HORSE não foi modificada, o que é claramente visível pelo **swept pages**, que é igual ao valor calculado.

Neste caso, a tabela não possui versões secundárias de registros (total versions: 0) e fragmentos (total fragments: 0), portanto, nas páginas secundárias (**secondary pages**) estão localizados apenas BLOBs de nível 0. A quantidade e o tamanho de tais BLOBs podem ser vistos na linha:

```
Level 0: 73761, total length: 7620692, blob pages: 0
```

O mecanismo lerá tanto as páginas primárias quanto as secundárias se o valor dos campos BLOB for lido na consulta.

Statement 166:

```
select count(*) from horse where remark = ''
```

Select Expression

```

-> Aggregate
  -> Filter
    -> Table "HORSE" Full Scan
1 records fetched
2162 ms, 10169 read(s), 716862 fetch(es)

```

Para uma varredura completa da tabela HORSE junto com os campos BLOB, foram necessárias 10169 leituras, o que é exatamente igual a $\text{Data pages} - \text{Empty pages} = 10176 - 7 = 10169$. Assim, todos os BLOBs foram alocados em 631 páginas secundárias (**secondary pages**).

Observo que os BLOBs maiores que o nível 0 não são alocados em páginas secundárias; em vez disso, nessas páginas é alocado um array de ponteiros para páginas do tipo blob, e os próprios dados são alocados nessas páginas.

Acima foi demonstrado o caso ideal, onde não há nem fragmentos nem versões de registros. No entanto, para tabelas onde ocorrem atualizações (UPDATE) e exclusões de registros (DELETE), isso não é verdade.

Versões e fragmentos de registros

Como se sabe, o Firebird é um SGBD multiversão. Isso significa que cada registro pode ter múltiplas versões. Existe uma versão primária do registro (a versão mais recente) e várias versões secundárias. Cada versão do registro possui em seu cabeçalho o número da transação que criou essa versão. As versões secundárias do registro surgem a cada alteração do registro ou em sua exclusão. Ao atualizar um registro, a versão mais recente do registro é gravada no local da versão primária anterior, e a versão anterior do registro ou o delta dela é gravado em outro local. Ao excluir um registro, no local desse registro é gravado um registro especial delete-stub, e sua versão anterior é gravada em outro local.

Delta de um registro — é a diferença da representação bit a bit da nova e da antiga versão do registro. O delta da versão do registro permite restaurar a versão antiga, aplicando o delta à nova versão. Normalmente, o delta é muito mais compacto do que a versão antiga do registro, especialmente se o registro não foi alterado completamente, por exemplo, se apenas o valor de um campo foi alterado. Ao ler os registros da tabela, para cada instantâneo (snapshot) é fornecida a versão confirmada mais recente ou uma versão não confirmada, se ela foi criada na transação atual. Cada registro pode ter apenas uma versão não confirmada.

As versões secundárias podem estar na mesma página (**Primary pages**) que as primárias, ou em outra página (página secundária **secondary pages**). As versões secundárias do registro estão localizadas na mesma página que a versão primária, se houver espaço suficiente nessa página para sua alocação. Se a versão secundária do registro couber na mesma página que a primária, isso permite obtê-la rapidamente sem a necessidade de ler páginas adicionais. Isso tem especial importância quando o registro primário não foi criado em uma determinada transação e ainda não foi confirmado. Para esse fim, as páginas primárias geralmente não são completamente preenchidas, mas reservam parte de seu espaço para futuras versões ou fragmentos do registro. Se no cabeçalho das estatísticas você vir em Attributes o valor no reserve, isso significa que nas páginas primárias não é reservado espaço para versões secundárias de registros, e, portanto, ao modificar tabelas desse banco de dados, as versões secundárias de registros para páginas cheias serão alocadas em páginas secundárias. Isso retardará significativamente a leitura em transações

concorrentes, bem como o rollback da transação atual. Infelizmente, o sinalizador de reserva não pode ser definido para cada tabela individualmente.

O mesmo registro pode ser atualizado várias vezes; nesse caso, as versões formam uma chamada cadeia de versões. As versões que são necessárias para pelo menos um instantâneo são chamadas ativas, e aquelas que não são necessárias — lixo (versões de lixo). No Firebird, funciona a coleta de lixo, que ou reduz a cadeia de versões, ou transforma versões secundárias em primárias.

Se na tabela existirem versões secundárias de registros, então nas estatísticas o valor **total versions** será maior que zero. O valor **max versions** exibe o comprimento máximo da cadeia de versões. Quanto maior esse valor, mais lenta será a leitura da tabela por transações antigas.

Como mencionado acima, a nova versão do registro sempre é gravada no mesmo local onde estava localizada a versão primária anterior. Mas e se a nova versão do registro for mais longa que a anterior, e após esse registro já houver outros na página? Nesse caso, o registro é fragmentado. A parte do registro primário que cabe no local antigo permanece lá, e a parte restante (fragmento) é gravada em outro local. O fragmento do registro, assim como a versão do registro, pode estar na mesma página (**Primary pages**) que a versão primária, ou em outra página (página secundária **secondary pages**). Fragmentos de registros surgem principalmente devido a atualizações de registros (UPDATE), mas também podem ocorrer simplesmente porque o registro é muito grande (não cabe em uma página). Um registro pode ter mais de um fragmento. Se algum registro tiver fragmentos, então o valor **total fragments** será maior que zero. O valor **max fragments** exibe a quantidade máxima de fragmentos para um único registro. Quanto maiores esses valores, mais lenta será a leitura das versões primárias dos registros. Se um registro tiver fragmentos, isso pelo menos aumenta a quantidade de leituras lógicas (fetches); no pior caso, aumenta a quantidade de leituras físicas, se os fragmentos estiverem localizados em páginas secundárias. Também observo que a coleta de lixo nem sempre é capaz de reduzir a fragmentação dos registros.

Para investigar as estatísticas de versões e fragmentos, execute o seguinte script:

```
RECREATE TABLE T (
  ID BIGINT GENERATED BY DEFAULT AS IDENTITY,
  A BIGINT NOT NULL,
  B VARCHAR(100),
  CONSTRAINT PK_T PRIMARY KEY (ID)
);

SET TERM ^;

EXECUTE BLOCK
AS
DECLARE I BIGINT = 0;
BEGIN
  WHILE (I < 100) DO
    BEGIN
      INSERT INTO T (A) VALUES (:I);
      I = I + 1;
    END
  END^

SET TERM ;^
```



```
COMMIT;
```

Ele criará a tabela T e a preencherá com dados. Vamos coletar as estatísticas desta tabela.

Estatísticas da tabela T

```

...
Page size                8192
...
Analyzing database pages ...
T (134)
  Primary pointer page: 231, Index root page: 232
  Total formats: 1, used formats: 1
  Average record length: 21.92, total records: 100
  Average version length: 0.00, total versions: 0, max versions: 0
  Average fragment length: 0.00, total fragments: 0, max fragments: 0
  Average unpacked length: 426.00, compression ratio: 19.43
  Pointer pages: 1, data page slots: 1
  Data pages: 1, average fill: 48%
  Primary pages: 1, secondary pages: 0, swept pages: 0
  Empty pages: 0, full pages: 0
  Table size: 16384 bytes
  Fill distribution:
    0 - 19% = 0
    20 - 39% = 0
    40 - 59% = 1
    60 - 79% = 0
    80 - 99% = 0

```

Nesta tabela, há apenas uma página de dados, na qual todos os registros da tabela estão localizados, não há versões nem fragmentos.

Agora, vamos iniciar uma sessão isql, começar uma transação snapshot nela e ler esta tabela. Não confirmaremos a transação para que sessões paralelas possam criar versões de registros.

Sessão 1

```
set transaction read only snapshot;  
select count(*) from t;
```

O rastreamento exibirá as seguintes estatísticas de execução:

```
Statement 103:
-----
select count(*) from t
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA

Select Expression
    -> Aggregate
        -> Table "T" Full Scan
1 records fetched
    0 ms, 105 fetch(es)
```

Table	Natural	Index	Update	Insert	Delete	Backout	Purge	Expunge
T	100							

Vamos iniciar uma sessão paralela e fazer um update de todos os registros da tabela nela:

Sessão 2

```
update t set a = 0;  
commit;
```

Vamos coletar as estatísticas para a tabela T novamente:

Estatísticas da tabela T

```
Analyzing database pages ...
T (134)
  Primary pointer page: 231, Index root page: 232
  Total formats: 1, used formats: 1
  Average record length: 14.00, total records: 100
  Average version length: 9.00, total versions: 100, max versions: 1
  Average fragment length: 0.00, total fragments: 0, max fragments: 0
  Average unpacked length: 426.00, compression ratio: 30.43
  Pointer pages: 1, data page slots: 1
  Data pages: 1, average fill: 70%
  Primary pages: 1, secondary pages: 0, swept pages: 0
  Empty pages: 0, full pages: 0
  Table size: 16384 bytes
  Fill distribution:
    0 - 19% = 0
    20 - 39% = 0
    40 - 59% = 0
    60 - 79% = 1
    80 - 99% = 0
```

Como você pode ver, agora temos 100 versões (total versions), e o comprimento máximo da cadeia de versões foi (max versions) 1. Ao mesmo tempo, o comprimento médio do registro compactado (Average record length) é menor do que era: 14 contra 21.92, portanto o registro não está fragmentado. Os custos aumentaram ao reexecutar a consulta na transação snapshot não confirmada?

```
Statement 103:
-----
select count(*) from t
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
Select Expression
    -> Aggregate
        -> Table "T" Full Scan
1 records fetched
    0 ms, 204 fetch(es)
```

Table	Natural	Index	Update	Insert	Delete	Backout	Purge	Expunge
-------	---------	-------	--------	--------	--------	---------	-------	---------

Table size: 24576 bytes

Fill distribution:

$$0 - 19\% = 0$$
$$20 - 39\% = 1$$
$$40 - 59\% = 0$$
$$60 - 79\% = 0$$
$$80 - 99\% = 1$$

Agora, o comprimento medio do registro é maior do que o da versão anterior (18.49 contra 14.00). Existem 200 versões no total (total versions), o comprimento médio da cadeia de versões é 2 (max versions). Nem todas as versões primárias do registro couberam no lugar das versões anteriores, portanto surgiram fragmentos na quantidade de 39 (total fragments), no máximo 1 fragmento para um registro. Também é óbvio que parte dos fragmentos e versões não coube na mesma página onde estão os registros primários, portanto uma página secundária (secondary pages) foi adicionada.

Vamos observar os custos da reexecução das consultas nas sessões 1 e 3.

Custos da sessão 1

Statement 103:

```
select count(*) from t
```

[illegible]

Select Expression

-> Aggregate

-> Table "T" Full Scan

1 records fetched

0 ms, 383 fetch(es)

Table	Natural	Index	Update	Insert	Delete	Backout	Purge	Expunge
I	100							

Custos da sessão 3

Statement 1265:

```
select count(*) from t
```

[illegible]

Select Expression

-> Aggregate

-> Table "T" Full Scan

```
1 records fetched
```

0 ms, 283 fetch(es)

Table	Natural	Index	Update	Insert	Delete	Backout	Purge	Expunge
T	100							

Como você pode ver, quanto maior a cadeia de versões, maior o custo necessário para ler as tabelas.

Agora, vamos confirmar as transações nas sessões 1 e 3. Vamos dar ao Firebird a oportunidade de coletar o lixo e observar as estatísticas.

Table	Natural	Index	Update	Insert	Delete	Backout	Purge	Expunge
T	100							

Aqui, é possível ver que o número de leituras lógicas aumentou, pois os registros agora têm fragmentos. É possível tornar os registros fragmentados não fragmentados novamente? Não, pois isso só é possível se o comprimento médio da nova versão for menor que o da anterior, o que é difícil sem alterar os valores dos campos do registro. A única maneira é criar um backup e restaurá-lo através da ferramenta gbak.

Nos dados originais, a estatística de execução era a seguinte:

```
Statement 120:
```

```
select count(a) from t
```

```
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
Select Expression
```

```
-> Aggregate
```

```
-> Table "T" Full Scan
```

```
1 records fetched
```

```
    0 ms, 105 fetch(es)
```


Table	Natural	Index	Update	Insert	Delete	Backout	Purge	Expunge
T	100							

Agora recriaremos a tabela original e a preencheremos novamente com os mesmos dados. Tentaremos aumentar a fragmentação da tabela sem criar e coletar lixo. Para fazer isso, simplesmente executamos a consulta (sem transações paralelas):

```
update t set b = UUID_TO_CHAR(GEN_UUID());
commit;
```

Vejam os estatísticos da tabela.

```
Analyzing database pages ...
T (135)
  Primary pointer page: 229, Index root page: 230
  Total formats: 1, used formats: 1
  Average record length: 63.10, total records: 100
  Average version length: 0.00, total versions: 0, max versions: 0
  Average fragment length: 45.00, total fragments: 42, max fragments: 1
  Average unpacked length: 426.00, compression ratio: 6.75
  Pointer pages: 1, data page slots: 2
  Data pages: 2, average fill: 59%
  Primary pages: 1, secondary pages: 1, swept pages: 1
  Empty pages: 0, full pages: 1
  Table size: 24576 bytes
  Fill distribution:
    0 - 19% = 0
    20 - 39% = 1
```

```

40 - 59% = 0
60 - 79% = 1
80 - 99% = 0

```

Nem todas as versões primárias do registro couberam no lugar das versões anteriores, portanto, surgiram fragmentos em quantidade de 42 (total fragments), no máximo 1 fragmento para um registro. Também é óbvio que parte dos fragmentos e versões não coube na mesma página onde estão os registros primários, então uma página secundária (secondary pages) foi adicionada.

Vejam os custos para executar a consulta:

```

Statement 120:
-----
select count(a) from t
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
Select Expression
  -> Aggregate
      -> Table "T" Full Scan
1 records fetched
    0 ms, 146 fetch(es)

Table              Natural      Index      Update      Insert      Delete      Backout      Purge      Expunge
*****
T                  100

```

A partir disso, as seguintes conclusões podem ser tiradas:

- As versões dos registros aumentam o custo de leitura dos registros em transações antigas, enquanto a execução de consultas em novas transações não é prejudicada, a menos que, é claro, a coleta cooperativa de lixo seja iniciada nesta transação.
- Os fragmentos de registros aumentam o custo de leitura dos registros em qualquer transação.

6.3.3. Estatísticas de índices da tabela

Para analisar todos os índices das tabelas de usuário, a opção `-i[ndex]` da utilidade `gstat` é usada. As tabelas são listadas em ordem alfabética e, para cada tabela, os índices são listados em ordem alfabética. Para incluir tabelas do sistema e seus índices no relatório, adicione a opção `-system`.

Infelizmente, não há como obter estatísticas para um único índice. No entanto, você pode limitar o resultado a uma tabela usando a opção `-t` seguida pelo nome da tabela. Você pode escrever uma lista de nomes de tabelas separados por espaços para obter resultados para mais de uma tabela. Observe que não é necessário especificar a opção `-i[ndex]`, pois os índices pertencentes às tabelas especificadas serão analisados automaticamente. É exatamente esse modo que precisamos, descrito acima.

Vejam um exemplo da saída de tais estatísticas para um dos índices:

```

...
Index HORSE_IDX_BIRTHDAY (0)
  Root page: 150238, depth: 2, leaf buckets: 167, nodes: 545113
  Average node length: 4.94, total dup: 520604, max dup: 27865

```

```

Average key length: 2.00, compression ratio: 1.90
Average prefix length: 3.75, average data length: 0.05
Clustering factor: 436641, ratio: 0.80
Fill distribution:
  0 - 19% = 0
 20 - 39% = 1
 40 - 59% = 0
 60 - 79% = 0
 80 - 99% = 166
...

```

Descrição da saída de estatísticas do índice:

Root page	Número da página raiz do índice.
depth	Número de níveis na árvore de páginas do índice. Essa estatística mostra o número de páginas que precisam ser acessadas para chegar a uma página folha. No disco está a página raiz dos índices (Index Root Page), que é criada simultaneamente com a tabela. Esta página contém uma lista de ponteiros para as páginas superiores (Root page — página raiz do índice) para cada índice — uma página por índice. Para qualquer índice, esta página contém uma lista de ponteiros para: • páginas de outro nível, se a profundidade for maior que 1, ou, • para páginas folha com os dados reais do índice, se a profundidade for igual a 1. As páginas folha armazenam a localização dos dados que foram indexados. A profundidade do índice é o número de níveis que você deve percorrer a partir da página superior (raiz) do índice para chegar às páginas folha. Nem a página raiz dos índices nem a página superior do índice são contadas na profundidade. Se a profundidade do índice exceder 3, é provável que o índice não funcione de forma muito eficiente. Neste caso, é recomendável aumentar o tamanho da página por meio de backup e restauração com a ferramenta gbak.
leaf buckets	Número de páginas do nível mais baixo (folhas) na árvore de índices. Estas são as páginas que contêm ponteiros para os registros. As páginas de nível superior contêm links indiretos.
nodes	Número total de registros indexados na árvore. Deve ser igual ao número de linhas indexadas na árvore, embora o relatório gstat possa incluir nós que foram excluídos, mas não limpos durante a coleta de lixo. Também pode incluir múltiplos elementos para registros cuja chave de índice foi alterada.

Average node length	Tamanho médio dos nós em bytes.
total dup	Número total de chaves de índice duplicadas. Será sempre zero para índices únicos.
max dup	Número de nós duplicados na “cadeia” com o maior número de duplicatas.
Average key length	Tamanho médio da chave em bytes, considerando a compressão. De 1 a 5 bytes são adicionados ao comprimento de cada chave, dependendo do tamanho da chave e do prefixo. Em seguida, o tamanho médio da chave é calculado.
compression ratio	Taxa média de compressão da chave. É calculada como a razão entre o comprimento médio da chave sem considerar a compressão (<code>Average prefix length + Average data length</code>) e o comprimento médio da chave considerando a compressão (<code>Average key length</code>).
Average prefix length	Tamanho médio ocupado pelos prefixos dos nós, em bytes.
average data length	Comprimento médio de cada chave em bytes. Provavelmente é menor do que a soma real dos tamanhos das colunas, pois o Firebird usa compressão de índice para reduzir a quantidade de dados armazenados na página folha do índice.
Clustering factor	É uma medida de quantas operações de E/S o banco de dados realizaria se tivesse que ler cada linha da tabela por meio do índice, na ordem do índice. Ou seja, mostra o quão ordenadas as linhas da tabela estão pelos valores do índice. Se o valor estiver próximo do número total de páginas, significa que a tabela está muito bem ordenada. Neste caso, as entradas do índice em uma página folha do índice geralmente apontam para linhas localizadas nas mesmas páginas de dados. Se o valor estiver próximo do número total de linhas, significa que a tabela está bastante desordenada. Neste caso, é improvável que as entradas do índice em uma página folha do índice apontem para as mesmas páginas de dados.
ratio	Razão entre Clustering factor e o número total de nós no índice.

Fill distribution

Este é um histograma com cinco faixas de 20%, cada uma mostrando o número de páginas de índice cuja porcentagem média de preenchimento está no intervalo especificado. A porcentagem de preenchimento é determinada pela proporção do espaço de cada página que contém dados. A soma desses números fornece o número total de páginas folha (**leaf buckets**) contendo dados de índice.

Idealmente, gostaríamos que todas as páginas estivessem preenchidas em 80-99%. Se o relatório mostrar que todas as páginas estão preenchidas em menos de 60%, então esse índice é um candidato para reconstrução. Certifique-se de considerar o número total de nós antes de iniciar a reconstrução — se o índice tiver apenas um pequeno número de nós, a reconstrução não ajudará na utilização do espaço, pois pode não haver registros suficientes para preencher efetivamente as páginas do índice.

O que pode ser interessante aqui do ponto de vista do desempenho da execução de uma consulta que usa um índice?

Profundidade do índice

Primeiramente, vale a pena prestar atenção à profundidade do índice **depth**. Lembre-se de que o tipo mais barato de varredura de índice é a varredura única (Unique scan), cujo custo é igual a $\text{depth} + 1$. O Firebird não armazena a profundidade do índice nas estatísticas, e para sua estimativa usa uma constante igual a 3. Se a profundidade do índice for menor que 3, o otimizador pode ter considerado o custo da varredura de índice superestimado (maior do que realmente é) e desistido de usar o índice. Se a profundidade do índice for maior que 3, ocorre o oposto. Com uma profundidade de índice maior que 3, muitas operações de índice tornam-se insuficientemente eficientes. Neste caso, talvez valha a pena considerar a possibilidade de aumentar o tamanho da página (**Page size**) para reduzir a profundidade dos índices.

Número de páginas folha

Um parâmetro não menos importante é o número de páginas folha do índice **leaf buckets**. Ele influencia o custo da varredura completa do índice (Full scan), que é calculada como $\text{leaf buckets} + \text{depth} - 1$. Esta fórmula não inclui o custo de extrair os dados da própria tabela, apenas o custo de varrer todas as páginas do próprio índice. Atualmente, a varredura completa do índice é usada apenas para navegação pelo índice (ordenação). Falaremos sobre a estimativa do custo de navegação pelo índice mais tarde.

Número de nós e duplicatas de chaves

O próximo grupo de parâmetros importantes são: **nodes** — o número de registros indexados, **total dup** — o número total de duplicatas de chaves no índice, **max dup** — o número máximo de duplicatas em uma cadeia com o mesmo valor de chave. Essas grandezas podem ser usadas para calcular a seletividade do índice:

```
selectivity = 1 / (nodes - total dup)
```

Por exemplo, para o índice `HORSE_IDX_BIRTHDAY` a seletividade do índice será:

$$\text{selectivity} = 1 / (545113 - 520604) = 1 / 24509 = 4.08 * 10^{-5}$$

Se as estatísticas armazenadas do índice estiverem atualizadas, esse valor será aproximadamente igual ao valor obtido a partir da consulta SQL:

```
SELECT
  RDB$STATISTICS
FROM RDB$INDICES
WHERE RDB$INDEX_NAME = 'HORSE_IDX_BIRTHDAY'
```

Se `total dup = 0`, então, provavelmente, o índice é único. Se `total dup = max dup`, então o índice tem um único valor que se repete várias vezes. Se, além disso, `nodes - total dup = 1`, então o índice tem apenas um valor, que se repete em todas as chaves. Este índice é inútil e pode ser removido, a menos que tenha sido criado para suportar uma restrição de chave primária, única ou estrangeira.



Para um índice parcial, isso não é verdade. Ou seja, um índice parcial pode ser útil mesmo que todas as suas chaves tenham o mesmo valor.

Se **nodes** for muito maior que **total dup** e, ao mesmo tempo, `total dup = max dup`, isso pode ser um sinal de distribuição não uniforme das chaves do índice. Por exemplo, o índice pode conter um grande número de chaves com valor `NULL`, enquanto as demais chaves estão distribuídas de forma mais ou menos uniforme. Infelizmente, nem as estatísticas obtidas com `gstat`, nem as estatísticas armazenadas fornecem qualquer informação sobre a distribuição das chaves.

Índices parciais

Para índices comuns, o valor **nodes** geralmente é igual ao valor **total records** das estatísticas da tabela. No entanto, isso não é verdade para índices parciais, que indexam apenas um subconjunto de registros, selecionado pelo predicado de filtragem especificado na cláusula `WHERE` durante a criação do índice. A seletividade real desses índices é calculada de forma diferente:

```
selectivity = 1 / (nodes - total dup)
```

```
real_selectivity = selectivity * (nodes / total records)
```

Como o otimizador não possui esses dados, ele estima a seletividade real do índice parcial indiretamente, ou seja, pela seletividade do predicado de filtro. Isso frequentemente leva ao índice parcial não ser usado ou ser usado, mas com a estimativa da cardinalidade do fluxo de saída incorreta.

Vamos dar uma olhada nas estatísticas do índice parcial para a mesma tabela. Este índice foi criado da seguinte forma:

```
CREATE INDEX IDX_HORSE_ANCESTOR ON HORSE (ANCESTOR)
WHERE ANCESTOR = 'Yes';
```

```
Index IDX_HORSE_ANCESTOR (18)
  Root page: 154158, depth: 1, leaf buckets: 1, nodes: 1461
  Average node length: 4.84, total dup: 1460, max dup: 1460
  Average key length: 2.00, compression ratio: 2.00
  Average prefix length: 4.00, average data length: 0.00
  Clustering factor: 872, ratio: 0.60
  Fill distribution:
    0 - 19% = 0
    20 - 39% = 0
    40 - 59% = 1
    60 - 79% = 0
    80 - 99% = 0
```

Pelas estatísticas, vê-se que o índice tem apenas um valor único, no entanto, ele é útil, pois filtra a maior parte dos registros da tabela HORSE.

Vamos calcular a seletividade e a seletividade real:

$\text{selectivity} = 1 / (1461 - 1460) = 1$

$\text{real_selectivity} = 1 * (1461 / 545113) = 0.00268$

Este índice pode ser usado para a consulta:

```
SELECT *
FROM HORSE
WHERE ANCESTOR = 'Yes'
```

Este índice é muito útil, embora o otimizador estime sua seletividade como 0.1 (seletividade do predicado = na cláusula de filtro WHERE ANCESTOR = 'Yes').

Tamanho da chave e taxa de compressão

Outra grandeza importante é o tamanho médio dos nós em bytes (Average node length); quanto menor for, maior será o número de nós do índice que pode ser armazenado em uma única página folha do índice. O número de nós do índice que cabe em uma página pode ser determinado aproximadamente pela seguinte fórmula:

$$\text{nodes_per_page} = (\text{page_size} - \text{BTR_SIZE}) / \text{Average_node_length}$$

Aqui BTR_SIZE = 39 — o tamanho do cabeçalho da página de índice.

Consequentemente, o número total de páginas folha do índice pode ser estimado aproximadamente pela fórmula:

```
leaf_pages = Average_node_length * nodes / (page_size - BTR_SIZE)
```

Mas isso não é necessário, pois as estatísticas já contêm o número exato de páginas folha (leaf buckets).

O tamanho do nó do índice, por sua vez, depende do tamanho da chave do índice (quantas colunas e de quais tipos você está indexando). As chaves são armazenadas no índice de forma compactada com compressão de prefixo. A compressão de prefixo significa que cada nó subsequente do índice pode usar a parte repetida dos bytes da chave do índice do nó anterior e, na realidade, armazena apenas a parte diferente da chave. Isso permite armazenar de forma compacta duplicatas de chaves nos índices. As estatísticas incluem o comprimento médio do prefixo da chave (Average prefix length), o comprimento médio da chave (average data length) e o comprimento médio da chave compactada (Average key length). A taxa de compressão das chaves do índice (compression ratio) é calculada como

```
compression ratio = (Average prefix length + Average data length) / Average key length
```

Por que isso pode ser importante? Vamos relembrar a fórmula para calcular o custo da varredura de índice, descrita em [Varredura por igualdade](#).

```
cost = indexDepth + MAX(avgKeyLength * table_cardinality * selectivity / (page_size - BTR_SIZE), 1)
```

```
avgKeyLength = 2 + length * factor
```

```
length -- comprimento da chave de índice em bytes
```

```
factor -- fator de compressão, considerado igual a 0.5 para índices simples e 0.7 para índices compostos.
```

O fator de compressão é o inverso da taxa de compressão, ou seja,

```
factor = 1 / compression ratio
```

Assim, a estimativa do custo da varredura de índice pode estar incorreta. Nas estatísticas armazenadas do índice não há taxa de compressão, e o fator de compressão são apenas constantes no código.

Fator de clusterização

O fator de clusterização **Clustering factor** — é uma medida de quantas operações de entrada/saída o banco de dados realizaria se tivesse que ler cada linha da tabela por meio do índice, na ordem do índice. Ele mostra o quão ordenadas estão as linhas na tabela pelos valores do índice. Se o valor estiver próximo ao número total de páginas de dados da tabela, significa que a tabela está muito bem ordenada. Neste caso, os registros do índice em uma página folha do índice geralmente apontam para linhas localizadas nas mesmas páginas de dados. Se o valor estiver próximo ao número total de linhas, significa que a tabela está mal ordenada. Neste caso, é improvável que os registros do índice em uma página folha do índice apontem para as mesmas páginas de dados.

O fator de clusterização pode ser usado para estimar o custo de extração de registros da tabela pelas chaves do índice (ordenação pelo índice). Atualmente, o custo de navegação pelo índice não é calculado—o fator de clusterização não está nas estatísticas armazenadas. Também não há fórmulas para estimar o custo considerando o fator de clusterização (não está claro o quanto o acesso aleatório é mais caro que o sequencial). No entanto, você pode se orientar pelo valor **Clustering factor ratio** (igual a `Clustering factor / nodes`). Quanto mais próximo de 1, mais cara é a navegação pelo índice.

A influência do fator de clusterização é especialmente perceptível se o tamanho do cache de páginas for pequeno. Neste caso, as páginas de dados da tabela já lidas podem ser expulsas do cache, pois cada nova chave do índice pode apontar para páginas de dados completamente diferentes, o que, por sua vez, leva a leituras físicas de páginas do disco (page reads) em vez de leituras lógicas (page fetches).

Capítulo 7. Índices

Um índice é um objeto de banco de dados criado para melhorar o desempenho da busca de dados em uma tabela. Um índice também pode ser usado para ordenar dados (navegação pelo índice). Além disso, os índices são usados para garantir restrições de integridade: chave primária (PRIMARY KEY), chave estrangeira (FOREIGN KEY) e restrição de unicidade (UNIQUE). Neste último caso, os índices são criados automaticamente quando a restrição é criada.



Garantir a restrição de unicidade e a restrição de chave primária só é possível com índices únicos. Outros métodos não podem garantir a unicidade em um ambiente concorrente.

A restrição de chave estrangeira teoricamente pode funcionar sem um índice adicional. Tal implementação desacelera significativamente a verificação dos registros da tabela dependente ao excluir um registro da tabela principal, mas, no entanto, tem o direito de existir. Atualmente, no Firebird, não é possível criar uma restrição de chave estrangeira sem a criação automática de um índice, mas em versões futuras essa possibilidade pode aparecer.

Neste guia, não discutiremos as restrições de integridade, mas consideraremos os índices apenas do ponto de vista da otimização da busca de dados e da ordenação com seu uso.

7.1. Tipos de índices

No Firebird, os índices podem ser classificados de acordo com os seguintes critérios:

- Índices únicos (UNIQUE) e não únicos;
- Direção da ordenação das chaves: crescente (ASCENDING) e decrescente (DESCENDING);
- O que exatamente é indexado: índices simples (uma coluna), compostos (várias colunas) e índices baseados em expressão;
- Quantos registros da tabela são indexados: índices completos ou parciais.

7.2. Quando o otimizador pode usar um índice?

O otimizador pode usar um ou mais índices nas seguintes partes de uma consulta SQL:

- filtragem de dados (cláusula WHERE);
- filtragem nas condições de junção (JOINS) ao usar o método de junção Nested Loop;
- ordenação de dados (cláusula ORDER BY);
- agrupamento ao calcular agregados (cláusula GROUP BY);
- cálculo das funções agregadas MIN e MAX.

7.3. Uso de índices para filtragem

Um índice ou seu segmento pode ser usado para acelerar a filtragem de dados para os seguintes predicados:

- igualdade (=) e igualdade considerando valores NULL (IS NOT DISTINCT FROM);
- outros predicados de comparação: >, >=, <, <=, BETWEEN;
- verificação de valor NULL (IS NULL);
- predicados de verificação de valor booleano: IS TRUE, IS FALSE, IS UNKNOWN;
- IN (<list>);
- STARTING WITH.

O otimizador não usa índices para predicados de não igualdade <>, IS NOT NULL e IS DISTINCT FROM, exceto para índices parciais (veja [Índices parciais](#)).

O otimizador pode usar um índice para comparação com a parte inicial da chave (STARTING WITH), mas não pode usar busca indexada se for necessário buscar dentro da chave (CONTAINING) ou apenas no seu final.

Para os predicados LIKE e SIMILAR TO, a busca indexada pode ser usada apenas se a comparação for com um literal e for possível extrair uma parte inicial não baseada em padrão da string (veja [Transformação do Predicado LIKE](#)).

De forma geral, a filtragem usando um índice se apresenta da seguinte maneira:

```
<ind_expr> <predicate> <expr>
```

onde <ind_expr>—uma coluna indexada da tabela ou uma expressão indexada para índices construídos sobre uma expressão.

Aqui, a expressão <expr> não deve ser dependente da mesma tabela para a qual o índice foi criado. Por exemplo, para o predicado de igualdade, o índice pode ser usado nos seguintes casos:

```
CREATE INDEX IDX_T1_FIELD ON T1(field);

SELECT * FROM T1
WHERE T1.field = 1;

SELECT * FROM T1
WHERE T1.field = ?;

SELECT * FROM T1
WHERE T1.field = MAXVALUE(4, 10);

SELECT * FROM T1, T2
WHERE T1.field = T2.field;
```


O índice não pode ser utilizado nos seguintes casos:

```
CREATE INDEX IDX_T1_FIELD ON T1(field);

SELECT * FROM T1
WHERE T1.field = T1.field2;

SELECT * FROM T1
WHERE T1.field = ? + T1.field2;

SELECT * FROM T1, T2
WHERE T1.field = COALESCE(T1.field2, T2.field);
```

Para a maioria dos predicados binários, <ind_expr> pode estar tanto à direita quanto à esquerda do predicado. A exceção é o operador `STARTING WITH`, que permite o uso do índice apenas se a coluna indexada (expressão) estiver no lado esquerdo.

7.3.1. Operador BETWEEN

O operador `BETWEEN` é sempre transformado de acordo com o seguinte esquema

```
a BETWEEN b AND c

a >= b AND a <= c
```

portanto, potencialmente, um índice pode ser usado em qualquer uma de suas partes.

7.3.2. Predicado IN

Para o predicado `IN` (<list>), a expressão <ind_expr> pode estar tanto à esquerda quanto dentro da lista.

As expressões dentro da lista `IN` são divididas em dois grupos: as que possuem dependências e as que não possuem. Se uma expressão usa colunas ou referências a aliases de campos de expressões de tabela, tais expressões são chamadas de dependentes. Todas as demais são consideradas expressões independentes, sendo que, na maioria das vezes, literais ou parâmetros de consulta atuam como expressões independentes. Em seguida, as expressões independentes na lista `IN` são avaliadas como invariantes e armazenadas em cache como uma árvore binária de busca. As expressões dependentes na lista `IN` são transformadas em uma lista `OR` (comportamento usado antes do Firebird 5.0).

Esquemáticamente, isso se parece com:

```
SELECT ... FROM ...
WHERE <expr> IN (N1, N2, ... Nm, ... <dep1>, <dep2>, ... <depL>)
```

transformado em

```
SELECT ... FROM ...
WHERE <expr> = <dep1> OR ... <expr> = <depL> OR <expr> IN (N1, N2, ... Nm)
```

Aqui N — expressões independentes (literais, parâmetros de entrada), <dep> — expressões dependentes (contendo colunas de tabelas ou expressões de tabela).

Assim, um índice no predicado IN pode ser usado nos seguintes casos:

```
SELECT * FROM T
WHERE T.field IN (?, ?, ?);

SELECT * FROM T
WHERE ? IN (T.field_1, T.field_2);
```

Aqui, índices foram criados para os campos field, field_1 e field_2.

Para mais detalhes, veja [Transformação do predicado IN com lista de valores](#).

7.3.3. Índice para colunas ou expressões booleanas

Se <ind_expr> for uma expressão booleana, então ela pode ser usada diretamente (sem predicados) ou com o predicado de negação NOT. Suponha que sua tabela tenha um campo IS_ACTIVE do tipo BOOLEAN. Se você criou um índice para este campo, ele pode ser utilizado para qualquer uma das seguintes expressões:

```
SELECT * FROM T
WHERE T.IS_ACTIVE;

SELECT * FROM T
WHERE NOT T.IS_ACTIVE;

SELECT * FROM T
WHERE T.IS_ACTIVE IS FALSE;
```

Essencialmente, a expressão T.IS_ACTIVE é equivalente a T.IS_ACTIVE IS TRUE, e a expressão NOT T.IS_ACTIVE é equivalente a T.IS_ACTIVE IS FALSE.

7.3.4. Índice por expressão

Para que o otimizador possa usar um índice ou seu segmento, uma condição necessária é que <ind_expr> corresponda exatamente à expressão na definição do índice. Portanto, se um índice for definido para uma determinada coluna da tabela, ele pode ser utilizado se o predicado de busca for aplicado diretamente a essa coluna, mas não a uma expressão sobre essa coluna.

Suponha que temos uma tabela INVOICE e um índice no campo INVOICE_DATE do tipo TIMESTAMP definido da seguinte forma:

```
CREATE INDEX INVOICE_IDX_DATE ON INVOICE (INVOICE_DATE);
```

Para obter a quantidade de faturas de 2015, pode-se usar a seguinte consulta:

```
SELECT COUNT(*)
FROM INVOICE
WHERE INVOICE_DATE >= date'2015-01-01' and INVOICE_DATE < date'2016-01-01'
```

Como a coluna INVOICE_DATE tem o tipo TIMESTAMP, ela contém, além da data, a parte de tempo, portanto usamos um intervalo semiaberto (em vez do operador BETWEEN, usamos os operadores “>=” e “<”).

O plano de execução da consulta acima é o seguinte:

```
PLAN (INVOICE INDEX (INVOICE_IDX_DATE))

Select Expression
  -> Aggregate
    -> Filter
      -> Table "INVOICE" Access By ID
        -> Bitmap
          -> Index "INVOICE_IDX_DATE" Range Scan (lower bound: 1/1, upper bound: 1/1)
```

Como você pode ver, aqui o índice construído para a coluna INVOICE_DATE é usado com sucesso pelo otimizador.

Agora, vamos tentar simplificar a consulta da seguinte forma:

```
SELECT COUNT(*)
FROM INVOICE
WHERE EXTRACT(YEAR FROM INVOICE_DATE) = 2015
```

Observamos o plano de execução:

```
PLAN (INVOICE NATURAL)

Select Expression
  -> Aggregate
    -> Filter
      -> Table "INVOICE" Full Scan
```

Como você pode ver, o otimizador não conseguiu utilizar a filtragem com o índice porque uma expressão foi aplicada à coluna sobre a qual a filtragem ocorre. Se você ainda deseja usar a busca indexada, é necessário criar um índice por expressão:

```
CREATE INDEX INVOICE_IDX_BY_YEAR ON INVOICE COMPUTED BY(EXTRACT(YEAR FROM INVOICE_DATE));
```

Observamos o plano novamente para a mesma consulta:

```
PLAN (INVOICE INDEX (INVOICE_IDX_BY_YEAR))

Select Expression
  -> Aggregate
    -> Filter
      -> Table "INVOICE" Access By ID
        -> Bitmap
          -> Index "INVOICE_IDX_BY_YEAR" Range Scan (full match)
```

Agora, nosso novo índice está sendo utilizado.

É importante notar que o Firebird não usará o índice se a expressão na construção do índice não for exatamente a mesma usada no predicado de filtragem, mesmo que essas expressões produzam o mesmo resultado. Vejamos o seguinte exemplo:

```
CREATE TABLE T (
  ID BIGINT GENERATED BY DEFAULT AS IDENTITY,
  A BIGINT NOT NULL,
  B VARCHAR(100),
  CONSTRAINT PK_T PRIMARY KEY (ID)
);

SELECT *
FROM T
WHERE ID = 1;
```

O plano de execução é o seguinte:

```
PLAN (T INDEX (PK_T))

Select Expression
  -> Filter
    -> Table "T" Access By ID
      -> Bitmap
        -> Index "PK_T" Unique Scan
```

Neste caso, para a restrição de chave primária, um índice de mesmo nome foi criado automaticamente, e esse índice é utilizado pelo otimizador.

Agora, vamos modificar um pouco a consulta, que é logicamente equivalente, pois produzirá o mesmo resultado:

```
SELECT *
FROM T
```

```
WHERE ID+0 = 1;
```

Observamos seu plano de execução:

```
PLAN (T NATURAL)

Select Expression
  -> Filter
    -> Table "T" Full Scan
```

O índice não pode mais ser usado pelo otimizador porque uma expressão foi aplicada à coluna indexada no predicado de filtragem. Observe que essa expressão não altera a semântica da consulta.

Vamos tentar criar um índice computado sem sentido:

```
CREATE INDEX IDX_T_SENSELESS ON T COMPUTED BY(ID+0);

SELECT *
FROM T
WHERE ID+0 = 1;
```

Observamos o plano:

```
PLAN (T INDEX (IDX_T_SENSELESS))

Select Expression
  -> Filter
    -> Table "T" Access By ID
      -> Bitmap
        -> Index "IDX_T_SENSELESS" Range Scan (full match)
```

Como esperado, o otimizador conseguiu utilizar tal índice. Agora, vamos modificar a consulta novamente, sem alterar sua semântica:

```
SELECT *
FROM T
WHERE ID-0 = 1;
```

Observamos o plano:

```
PLAN (T NATURAL)

Select Expression
  -> Filter
    -> Table "T" Full Scan
```

O índice não é usado novamente, embora as expressões `ID+0` e `ID-0` produzam resultados equivalentes.

7.3.5. Desabilitando o uso de índices

O fato de que qualquer expressão aplicada a um campo de índice ou a uma expressão de índice, mas que não altera sua semântica, desabilita o uso do índice, pode ser utilizado como dicas para o otimizador de consultas.

Normalmente, para desabilitar o uso de um índice, utilizam-se duas expressões dependendo do tipo:

- para tipos numéricos e datas, utiliza-se a expressão `+0`;
- para tipos de string, utiliza-se a expressão `|| ''`.

Para que pode ser necessário desabilitar um índice, contaremos mais tarde, mas por enquanto mostraremos alguns exemplos de desabilitação do uso de índice.

Vamos criar uma tabela e seus índices:

```
RECREATE TABLE TEST_INDEXES (
  ID BIGINT GENERATED BY DEFAULT AS IDENTITY,
  A_STR VARCHAR(30),
  A_NUMERIC NUMERIC(18, 2),
  A_DOUBLE DOUBLE PRECISION,
  A_DATE DATE,
  A_TS TIMESTAMP,
  A_BOOL BOOLEAN,
  CONSTRAINT PK_TEST_INDEXES PRIMARY KEY(ID)
);

-- Preenchemos a tabela com dados

-- Criamos os índices
CREATE INDEX IDX_TEST_INDEXES_STR ON TEST_INDEXES(A_STR)
CREATE INDEX IDX_TEST_INDEXES_NUM ON TEST_INDEXES(A_NUMERIC);
CREATE INDEX IDX_TEST_INDEXES_DOUBLE ON TEST_INDEXES(A_DOUBLE);
CREATE INDEX IDX_TEST_INDEXES_DATE ON TEST_INDEXES(A_DATE);
CREATE INDEX IDX_TEST_INDEXES_TS ON TEST_INDEXES(A_TS);
CREATE INDEX IDX_TEST_INDEXES_BOOL ON TEST_INDEXES(A_BOOL);
```

Qualquer uma destas consultas usará o índice para filtragem:

```
SELECT * FROM TEST_INDEXES WHERE ID = 1;

SELECT * FROM TEST_INDEXES WHERE A_STR = 'a';

SELECT * FROM TEST_INDEXES WHERE A_STR STARTING WITH 'a';

SELECT * FROM TEST_INDEXES WHERE A_NUMERIC = 10.0;
```

```

SELECT * FROM TEST_INDEXES WHERE A_DOUBLE > 10;

SELECT * FROM TEST_INDEXES WHERE A_DATE = CURRENT_DATE;

SELECT * FROM TEST_INDEXES
WHERE A_TS BETWEEN timestamp'2025-01-01 12:30' AND timestamp'2025-01-01 12:50';

SELECT * FROM TEST_INDEXES WHERE A_BOOL IS FALSE;

```

Apresento consultas semelhantes em resultado com o uso de índice desabilitado:

```

-- Para inteiros +0
SELECT * FROM TEST_INDEXES WHERE ID+0 = 1;

-- Para strings || ''
SELECT * FROM TEST_INDEXES WHERE A_STR || '' = 'a';

-- Para strings || ''
SELECT * FROM TEST_INDEXES WHERE A_STR || '' STARTING WITH 'a';

-- Para inteiros escaláveis +0
SELECT * FROM TEST_INDEXES WHERE A_NUMERIC+0 = 10.0;

-- Para números de ponto flutuante +0
SELECT * FROM TEST_INDEXES WHERE A_DOUBLE+0 > 10;

-- Para datas +0
SELECT * FROM TEST_INDEXES WHERE A_DATE+0 = CURRENT_DATE;

-- Para data e hora +0
SELECT * FROM TEST_INDEXES
WHERE A_TS+0 BETWEEN timestamp'2025-01-01 12:30' AND timestamp'2025-01-01 12:50';

-- Para BOOLEAN não há expressões simples, mas pode-se agrupar usando parênteses
-- com AND TRUE ou OR FALSE
SELECT * FROM TEST_INDEXES WHERE (A_BOOL AND TRUE) IS FALSE;

```

O uso de índice também pode ser desabilitado para ordenação/agrupamento. Isso será considerado mais tarde.

7.3.6. Estatísticas de índices

Acima foram expostas as condições **obrigatórias** para o uso de um índice em expressões de filtragem. No entanto, se essas condições forem atendidas, isso ainda não significa que o otimizador usará o índice. A decisão final sobre usar ou não usar o índice é tomada pelo otimizador com base em uma avaliação de custo. Para seu cálculo, é necessária alguma estatística armazenada, que está associada ao índice. Atualmente, para índices existe apenas uma estatística armazenada — a “seletividade”.

Seletividade do índice

Seletividade — é um valor inverso à quantidade de valores únicos da chave. Nos cálculos de cardinalidade e custo, assume-se uma distribuição uniforme dos valores da chave no índice.

A seletividade de um índice pode ser descoberta usando a seguinte consulta

```
SELECT RDB$STATISTICS
FROM RDB$INDICES
WHERE RDB$INDEX_NAME = '<index_name>'
```

Exemplo 84. Obtendo a estatística armazenada para o índice CUSTNAMEX

```
SELECT RDB$STATISTICS
FROM RDB$INDICES
WHERE RDB$INDEX_NAME = 'CUSTNAMEX'
```

```
RDB$STATISTICS
=====
0.06666667014360428
```

Seletividade dos segmentos de um índice composto

Para índices compostos, além da seletividade do índice, o Firebird também armazena a seletividade do conjunto de seus segmentos, começando do primeiro até o atual. A seletividade do conjunto de segmentos pode ser descoberta usando uma consulta à tabela RDB\$INDEX_SEGMENTS.

Exemplo 85. Obtendo a estatística armazenada para cada segmento do índice NAMEX

```
SELECT RDB$FIELD_NAME, RDB$FIELD_POSITION, RDB$STATISTICS
FROM RDB$INDEX_SEGMENTS
WHERE RDB$INDEX_NAME = 'NAMEX';
```

RDB\$FIELD_NAME	RDB\$FIELD_POSITION	RDB\$STATISTICS
=====	=====	=====
LAST_NAME	0	0.02500000037252903
FIRST_NAME	1	0.02380952425301075

Exemplo 86. Obtendo a estatística armazenada para o índice NAMEX

```
SELECT RDB$STATISTICS
FROM RDB$INDICES
WHERE RDB$INDEX_NAME = 'NAMEX'
```



```
RDB$STATISTICS
=====
0.02380952425301075
```

A seletividade do índice é calculada durante sua construção (`CREATE INDEX`, `ALTER INDEX ... ACTIVE`) e posteriormente pode se tornar desatualizada. Para atualizar as estatísticas do índice, utiliza-se a seguinte consulta SQL

```
SET STATISTICS INDEX <index_name>;
```

Exemplo 87. Coletando estatísticas para o índice NAMEX

```
SET STATISTICS INDEX NAMEX;
```

Seletividade de predicados indexados

O valor de seletividade é usado diretamente apenas para predicados de igualdade "=", `IS NOT DISTINCT FROM` e `IS NULL`, nos demais casos são usadas as seguintes fórmulas.

Tabela 12. Seletividade de predicados de varredura de índice

Predicado	factor	Seletividade
<, <=, >, >=	0.05	$\text{indexSelectivity} + (1 - \text{indexSelectivity}) * \text{factor}$
BETWEEN	0.0025	$\text{indexSelectivity} + (1 - \text{indexSelectivity}) * \text{factor}$
STARTING WITH	0.01	$\text{indexSelectivity} + (1 - \text{indexSelectivity}) * \text{factor}$
IN		$\text{indexSelectivity} * N$
Outros	0.01	$\text{indexSelectivity} + (1 - \text{indexSelectivity}) * \text{factor}$

Para índices compostos, em vez de `indexSelectivity`, é usada a seletividade dos segmentos do índice envolvidos.

7.3.7. Índices compostos

Até agora consideramos índices construídos em uma única coluna da tabela, agora passaremos para índices em várias colunas. Tais índices são chamados de compostos ou multisegmentados. O número máximo de colunas em um índice composto é 16.

Índices compostos podem ser usados para suportar restrições: `PRIMARY KEY`, `FOREIGN KEY`, `UNIQUE`. Além disso, você pode criar índices compostos para acelerar a busca por predicados de filtragem complexos e múltiplos.

Deve-se notar que o conjunto de campos de um índice composto representa uma única chave. A busca no índice pode ser realizada tanto pela chave inteira quanto por sua substring (subchave). Obviamente, a busca por uma subchave é permitida apenas para a parte inicial da chave (por

exemplo, STARTING WITH ou o uso de não todos os segmentos do composto). Se a busca é realizada por todos os segmentos do índice, isso é chamado de correspondência completa (full match) da chave, caso contrário é uma correspondência parcial (partial match) da chave. Portanto, para um índice composto nos campos (A, B, C), segue-se que:

- ele pode ser usado para predicados ($A = 0$) ou ($A = 0$ and $B = 0$) ou ($A = 0$ and $B = 0$ and $C = 0$), mas não pode ser usado para predicados ($B = 0$) ou ($C = 0$) ou ($B = 0$ and $C = 0$);
- o predicado ($A = 0$ and $B > 0$ and $C = 0$) levará a uma correspondência parcial em dois segmentos, e o predicado ($A > 0$ and $B = 0$) – a uma correspondência parcial em apenas um segmento.

Para mais detalhes, consulte [Acesso por índice](#)

Vamos demonstrar isso no seguinte exemplo:

```
CREATE TABLE TEST (
  ID BIGINT GENERATED BY DEFAULT AS IDENTITY,
  A INT,
  B INT,
  C INT,
  CONSTRAINT PK_TEST PRIMARY KEY(ID)
);

INSERT INTO TEST(A, B, C) VALUES (1, 2, 3);
INSERT INTO TEST(A, B, C) VALUES (2, 1, 1);
INSERT INTO TEST(A, B, C) VALUES (3, 3, 2);

COMMIT;

CREATE INDEX IDX_TEXT_ABC ON TEST(A, B, C);
```

```
-- correspondência completa em todos os segmentos
SELECT * FROM TEST WHERE A = 1 AND B = 2 AND C = 3;
```

```
Select Expression
-> Filter
    -> Table "TEST" Access By ID
        -> Bitmap
            -> Index "IDX_TEXT_ABC" Range Scan (full match)
```

```
-- correspondência completa em todos os segmentos,
-- no entanto, para o último segmento é usada apenas o limite inferior
SELECT * FROM TEST WHERE A = 1 AND B = 2 AND C > 3;
```

```
Select Expression
-> Filter
    -> Table "TEST" Access By ID
```

```
-> Bitmap
  -> Index "IDX_TEXT_ABC" Range Scan (lower bound: 3/3, upper bound: 2/3)
```

```
-- correspondência parcial em dois segmentos
SELECT * FROM TEST WHERE A = 1 AND B = 2;
```

```
Select Expression
  -> Filter
    -> Table "TEST" Access By ID
      -> Bitmap
        -> Index "IDX_TEXT_ABC" Range Scan (partial match: 2/3)
```

```
-- correspondência parcial em um segmento
SELECT * FROM TEST WHERE A = 1;
```

```
Select Expression
  -> Filter
    -> Table "TEST" Access By ID
      -> Bitmap
        -> Index "IDX_TEXT_ABC" Range Scan (partial match: 1/3)
```

```
-- correspondência parcial nos segmentos
-- para o segundo segmento é usado apenas o limite superior
SELECT * FROM TEST WHERE A = 1 AND B < 5 AND C = 3;
```

```
Select Expression
  -> Filter
    -> Table "TEST" Access By ID
      -> Bitmap
        -> Index "IDX_TEXT_ABC" Range Scan (lower bound: 1/3, upper bound: 2/3)
```

```
-- correspondência parcial em um segmento, segmentos 2 e 3 não envolvidos
SELECT * FROM TEST WHERE A = 1 AND C = 3;
```

```
Select Expression
  -> Filter
    -> Table "TEST" Access By ID
      -> Bitmap
        -> Index "IDX_TEXT_ABC" Range Scan (partial match: 1/3)
```

```
-- índice não envolvido de forma alguma
```

```
SELECT * FROM TEST WHERE B = 2 AND C = 3;
```

```
Select Expression
-> Filter
    -> Table "TEST" Full Scan
```

Assim, ao criar índices compostos, a ordem das colunas tem enorme importância para o otimizador.

Apresento um exemplo em que a escolha incorreta da ordem das colunas em um índice composto leva a um plano de execução ineficiente.

Exemplo de ordem incorreta de campos em um índice composto

Suponha que haja uma tabela SERVICE, definida da seguinte forma:

```
CREATE TABLE SERVICE (
    CODE_SERVICE      BIGINT NOT NULL,
    CODE_WEBUSER      BIGINT NOT NULL,
    SESSION_ID        VARCHAR(32),
    PAY_SUM           NUMERIC(18,2),
    BYDATE            TIMESTAMP NOT NULL,
    IP                CHAR(15),
    ...
    CONSTRAINT PK_SERVICE PRIMARY KEY (CODE_SERVICE),
    CONSTRAINT FK_SERVICE_REF_WEBUSER FOREIGN KEY (CODE_WEBUSER) REFERENCES WEBUSER
(CODE_WEBUSER)
);
```

Você precisa calcular a soma por período para um usuário específico e, para acelerar, cria o seguinte índice composto:

```
CREATE INDEX IDX_SERVICE_BYDATE_USER ON SERVICE (BYDATE, CODE_WEBUSER);
```

Agora você executa a seguinte consulta:

```
select
    count(*) as cnt,
    sum(pay_sum) as pay_sum
from SERVICE
where bydate >= date '2020-01-01'
    and bydate < '2020-06-01'
    and code_webuser = 917
```

Vejamos seu plano e estatísticas:

```
Select Expression
```

```

-> Aggregate
  -> Filter
    -> Table "SERVICE" Access By ID
      -> Bitmap And
        -> Bitmap
          -> Index "FK_SERVICE_REF_WEBUSER" Range Scan (full match)
        -> Bitmap
          -> Index "IDX_SERVICE_BYDATE_USER" Range Scan (lower bound: 1/2, upper
bound: 1/2)

```

CNT	PAY_SUM
=====	=====
5736	8000.00

```

Current memory = 1720861584
Delta memory = 288
Max memory = 2383805408
Elapsed time = 0.046 sec
Buffers = 102400
Reads = 0
Writes = 0
Fetches = 7498

```

Parece que está tudo bem, mas

1. pelo plano, vemos que estamos usando apenas o primeiro segmento do índice `IDX_SERVICE_BYDATE_USER`;
2. em vez do segundo segmento do índice composto, o índice `FK_SERVICE_REF_WEBUSER` é usado.

Isso é significativamente pior do que se apenas o índice `FK_SERVICE_REF_WEBUSER` fosse usado, do qual dois segmentos seriam utilizados imediatamente.

Vamos tentar corrigir isso:

```

DROP INDEX IDX_SERVICE_BYDATE_USER;

CREATE INDEX IDX_SERVICE_USER_BYDATE ON SERVICE (CODE_WEBUSER, BYDATE);

```

Vamos tentar executar a mesma consulta novamente:

```

Select Expression
  -> Aggregate
    -> Filter
      -> Table "SERVICE" Access By ID
        -> Bitmap
          -> Index "IDX_SERVICE_USER_BYDATE" Range Scan (lower bound: 2/2, upper
bound: 2/2)

```

CNT	PAY_SUM
=====	=====
5736	8000.00

```

Current memory = 1720480112
Delta memory = 288
Max memory = 2384762736
Elapsed time = 0.006 sec
Buffers = 102400
Reads = 0
Writes = 0
Fetches = 6400

```

Aqui as estatísticas são significativamente melhores. Em alguns casos, o desempenho pode diferir em 10 vezes ou mais.

Tamanho do índice

A ordem das colunas em um índice composto afeta não apenas o plano de consulta, mas também a compactação do índice. Vamos comparar as estatísticas dos dois índices:

```

Index IDX_SERVICE_BYDATE_USER (0)
  Root page: 140661, depth: 3, leaf buckets: 16831, nodes: 12970958
  Average node length: 20.99, total dup: 17494, max dup: 5
  Average key length: 17.52, compression ratio: 1.26
  Average prefix length: 7.48, average data length: 14.52
  Clustering factor: 707310, ratio: 0.05
  Fill distribution:
    0 - 19% = 0
    20 - 39% = 0
    40 - 59% = 1
    60 - 79% = 0
    80 - 99% = 16830

Index IDX_SERVICE_USER_BYDATE (1)
  Root page: 205173, depth: 3, leaf buckets: 7391, nodes: 12970958
  Average node length: 9.20, total dup: 17494, max dup: 5
  Average key length: 5.72, compression ratio: 4.37
  Average prefix length: 22.25, average data length: 2.74
  Clustering factor: 1795117, ratio: 0.14
  Fill distribution:
    0 - 19% = 0
    20 - 39% = 0
    40 - 59% = 0
    60 - 79% = 1
    80 - 99% = 7390

```

O índice `IDX_SERVICE_BYDATE_USER` é muito menos compacto em comparação com `IDX_SERVICE_USER_BYDATE` (número de páginas folha 16831 contra 7391). Além disso, ele tem uma taxa de compressão muito pior. Isso também significa que o custo de manutenção do índice `IDX_SERVICE_BYDATE_USER` durante operações de modificação será maior.

A razão para essas estatísticas está no fato de que o primeiro índice foi criado com a coluna mais seletiva primeiro (há mais timestamps do que usuários).

7.3.8. Uso de múltiplos índices

Até agora, consideramos a possibilidade de usar um único índice para a tabela, no entanto, o Firebird pode usar eficientemente múltiplos índices para filtrar dados. Isso é implementado por meio da interseção e união de máscaras de bits e é descrito em "[Interseção e união de mapas de bits](#)".

Predicados em colunas indexadas podem ser combinados usando os operadores lógicos AND e OR.

Exemplo 88. Interseção de máscaras de bits

```
SELECT COUNT(*)
FROM INVOICE
WHERE CUSTOMER_ID = 22 AND INVOICE_DATE < date '2015-06-01';
```

```
Select Expression
-> Aggregate
  -> Filter
    -> Table "INVOICE" Access By ID
      -> Bitmap And
        -> Bitmap
          -> Index "FK_INVOICE_CUSTOMER" Range Scan (full match)
        -> Bitmap
          -> Index "INVOICE_IDX_DATE" Range Scan (upper bound: 1/1)
```

Exemplo 89. União de máscaras de bits

```
SELECT COUNT(*)
FROM INVOICE
WHERE CUSTOMER_ID = 22 OR INVOICE_DATE < date '2015-06-01';
```

```
Select Expression
-> Aggregate
  -> Filter
    -> Table "INVOICE" Access By ID
      -> Bitmap Or
        -> Bitmap
          -> Index "FK_INVOICE_CUSTOMER" Range Scan (full match)
        -> Bitmap
          -> Index "INVOICE_IDX_DATE" Range Scan (upper bound: 1/1)
```

Ao combinar predicados usando o operador AND, os predicados que não podem usar um índice não interferem no uso do índice por outro predicado. Demonstrarei isso com o seguinte exemplo:

```
SELECT COUNT(*)
FROM INVOICE
```

```
WHERE CUSTOMER_ID = 22 AND TOTAL_SALE > 0;
```

Aqui, existe um índice para a coluna `CUSTOMER_ID`, mas não para `TOTAL_SALE`. O segundo predicado não impede o uso do índice em `CUSTOMER_ID`.

```
Select Expression
  -> Aggregate
    -> Filter
      -> Table "INVOICE" Access By ID
        -> Bitmap
          -> Index "FK_INVOICE_CUSTOMER" Range Scan (full match)
```

Podemos ir além e tornar o segundo predicado invariante.

```
SELECT COUNT(*)
FROM INVOICE
WHERE CUSTOMER_ID = 22 AND 1 = ?
```

O plano para esta consulta será o seguinte:

```
Select Expression
  -> Aggregate
    -> Filter (preliminary)
      -> Filter
        -> Table "INVOICE" Access By ID
          -> Bitmap
            -> Index "FK_INVOICE_CUSTOMER" Range Scan (full match)
```

Observe que predicados não correlacionados são elevados na árvore de execução e calculados em um estágio inicial, ou seja, antes da varredura do índice. Se o predicado `1 = ?` for falso, a varredura do índice não ocorrerá e a consulta terminará antecipadamente, retornando 0. Se o predicado for verdadeiro, a varredura do índice ocorrerá usando o índice `FK_INVOICE_CUSTOMER`. Para mais detalhes, leia em [Verificação de predicados invariantes](#).

No entanto, ao combinar predicados com `OR`, predicados que não podem usar um índice desabilitam a possibilidade de usar o índice também para o predicado indexado.

```
SELECT COUNT(*)
FROM INVOICE
WHERE CUSTOMER_ID = 22 OR TOTAL_SALE > 0;
```

O plano para esta consulta será o seguinte:

```
Select Expression
  -> Aggregate
    -> Filter
```


-> Table "INVOICE" Full Scan

Note que se adicionarmos outro predicado com uma coluna indexada usando AND a (CUSTOMER_ID = 22 OR TOTAL_SALE > 0), isso não impedirá o uso do índice.

```
SELECT COUNT(*)
FROM INVOICE
WHERE (CUSTOMER_ID = 22 OR TOTAL_SALE > 0)
      AND INVOICE_DATE < date '2015-06-01';
```

```
Select Expression
  -> Aggregate
    -> Filter
      -> Table "INVOICE" Access By ID
        -> Bitmap
          -> Index "INVOICE_IDX_DATE" Range Scan (upper bound: 1/1)
```

Deve-se notar também que o otimizador quase sempre usa pelo menos um índice para filtrar a tabela (se for apenas uma). Isso ocorre porque no Firebird os índices são bastante compactos e usam máscaras de bits, mantendo sua eficiência mesmo que sua seletividade seja ruim, por exemplo, 0.9. Veja [Mapas de bits](#). No entanto, se muitos predicados conectados por AND, cada um dos quais pode usar um índice, forem usados para filtragem, o otimizador pode desistir de usar índices extras. Normalmente, isso acontece quando os índices têm seletividade muito diferente, por exemplo, um índice é único e o outro não. Nesse caso, não faz sentido ler as páginas do índice não único (da raiz às páginas folha + páginas folha).

```
SELECT *
FROM INVOICE
WHERE INVOICE_ID = 100020 AND CUSTOMER_ID = 10;
```

```
Select Expression
  -> Filter
    -> Table "INVOICE" Access By ID
      -> Bitmap
        -> Index "PK_INVOICE" Unique Scan
```

Aqui, INVOICE_ID é a chave primária. O otimizador não usou o índice na coluna CUSTOMER_ID para evitar leituras desnecessárias. A varredura pelo índice PK_INVOICE retornou um registro, que foi então filtrado pelo predicado CUSTOMER_ID = 10 sem usar o índice.

Outro exemplo.

```
SELECT *
FROM WEBUSER
WHERE REG_DATE BETWEEN TIMESTAMP '2020-01-01' AND TIMESTAMP '2020-05-31 23:59:59.9999'
      AND CODE_USERGROUP IN (3,5)
```

```

Select Expression
  -> Filter
    -> Table "WEBUSER" Access By ID
      -> Bitmap And
        -> Bitmap
          -> Index "WEBUSER_IDX_BYDATE" Range Scan (lower bound: 1/1, upper bound:
1/1)
        -> Bitmap
          -> Index "FK_WEBUSER_REF_USERGROUP" List Scan (full match)

```

Aqui, ambos os índices são usados para filtragem. Vamos tentar aumentar o número de valores no predicado IN.

```

SELECT *
FROM WEBUSER
WHERE REG_DATE BETWEEN TIMESTAMP '2020-01-01' AND TIMESTAMP '2020-05-31 23:59:59.9999'
AND CODE_USERGROUP IN (1, 2, 3, 4, 5, 6)

```

```

Select Expression
  -> Filter
    -> Table "WEBUSER" Access By ID
      -> Bitmap
        -> Index "WEBUSER_IDX_BYDATE" Range Scan (lower bound: 1/1, upper bound: 1/1)

```

Neste exemplo, aumentar o número de valores no predicado IN o tornou não seletivo, portanto o otimizador considerou o uso desse índice inútil. A seletividade do índice FK_WEBUSER_REF_USERGROUP é 0.1667. Lembre-se de que para o predicado IN a seletividade total é calculada pela fórmula $N * index_sel$, onde N é o número de valores no IN. Agora vamos tentar remover a filtragem por REG_DATE.

```

SELECT *
FROM WEBUSER
WHERE CODE_USERGROUP IN (1, 2, 3, 4, 5, 6);

```

```

Select Expression
  -> Filter
    -> Table "WEBUSER" Access By ID
      -> Bitmap
        -> Index "FK_WEBUSER_REF_USERGROUP" List Scan (full match)

```

O índice é usado novamente, embora de acordo com os cálculos ele seja completamente inútil.

```

SELECT
  MINVALUE(0.1667 * 6, 1) AS IN_SEL
FROM RDB$DATABASE;

```

```
IN_SEL
=====
1.0000
```

Em versões futuras do Firebird, isso pode mudar. Ou seja, o otimizador pode até mesmo recusar o uso de um único índice se o custo da varredura do índice for maior que o custo da varredura completa da tabela.



Atualmente, a diferença de custo entre a varredura por um único índice não seletivo e a varredura completa da tabela é considerada uma quantidade insignificante. É possível que as estatísticas do índice não tenham sido atualizadas e, portanto, é mais fácil usar esse índice, que impõe uma penalidade insignificante se a seletividade estiver correta. Se a seletividade estiver incorreta, a penalidade por desistir da varredura do índice pode ser significativa.

7.3.9. Ramificação condicional de fluxos

Ao discutir o uso de múltiplos índices, consideramos o caso de usar um índice em conjunto com um predicado invariante, combinado via AND. No entanto, não considere o mesmo exemplo com um predicado invariante, mas combinado via OR. Havia uma boa razão para isso. O fato é que tal combinação é considerada pelo otimizador do Firebird como um caso especial – desativação da filtragem por índice com base em uma condição. Esta funcionalidade foi descrita em detalhes em [Ramificação Condicional de Fluxos \(Conditional Stream\)](#).



Nos exemplos a seguir, assume-se que uma biblioteca de alto nível que suporta parâmetros nomeados está sendo usada.

Considere o seguinte exemplo:

```
SELECT COUNT(*)
FROM INVOICE
WHERE CUSTOMER_ID = 22 OR CAST(:ALL_FLAG AS BOOLEAN);
```

Dependendo do valor do parâmetro ALL_FLAG, ou toda a tabela é lida, ou os registros onde CUSTOMER_ID = 22. Vamos ver o plano desta consulta:

```
Select Expression
-> Aggregate
  -> Filter
    -> Condition
      -> Table "INVOICE" Full Scan
      -> Table "INVOICE" Access By ID
        -> Bitmap
          -> Index "FK_INVOICE_CUSTOMER" Range Scan (full match)
```

No plano, vemos o método de acesso “Condition”, que indica que durante a execução (e não na fase de prepare) será tomada a decisão de como ler os dados da tabela INVOICE – usando o índice

FK_INVOICE_CUSTOMER ou fazendo uma varredura completa da tabela. Soa intrigante, vamos tentar executar esta consulta com várias opções de parâmetros de entrada.

Se ALL_FLAG for False, as estatísticas serão:

```

COUNT
=====
1001

Current memory = 284263072
Delta memory = 176
Max memory = 284343472
Elapsed time = 0.010 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 1641
Per table statistics:
-----+-----+-----+-----+-----+
Table name | Natural | Index | Insert | Update | Delete |
-----+-----+-----+-----+-----+
INVOICE    |         | 1001 |         |         |         |
-----+-----+-----+-----+-----+

```

Se ALL_FLAG for True, as estatísticas serão:

```

COUNT
=====
100010

Current memory = 284273840
Delta memory = 176
Max memory = 284343472
Elapsed time = 0.039 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 103935
Per table statistics:
-----+-----+-----+-----+-----+
Table name | Natural | Index | Insert | Update | Delete |
-----+-----+-----+-----+-----+
INVOICE    | 100010 |         |         |         |         |
-----+-----+-----+-----+-----+

```

Impressionante, não é?

Na maioria das vezes, essa funcionalidade é usada de uma forma ligeiramente diferente.

```

SELECT COUNT(*)
FROM INVOICE

```

```
WHERE CUSTOMER_ID = :CUSTOMER_ID OR :CUSTOMER_ID IS NULL;
```

Se o valor NULL for passado para o parâmetro CUSTOMER_ID, toda a tabela INVOICE será varrida; caso contrário, apenas os registros com o valor passado em CUSTOMER_ID serão retornados. No segundo caso, o índice FK_INVOICE_CUSTOMER será usado.

Assim, você pode usar um filtro “desativável” na consulta sem a necessidade de formar dinamicamente o texto da consulta. Mas não se apresse em se alegrar e adicionar muitos filtros desse tipo às suas consultas. Infelizmente, só pode haver um desses filtros por tabela, e a ramificação condicional de fluxos será feita apenas para o primeiro deles; para os demais, o índice não pode ser usado. Vejamos o seguinte exemplo:

```
SELECT COUNT(*)
FROM INVOICE
WHERE (CUSTOMER_ID = :CUSTOMER_ID OR :CUSTOMER_ID IS NULL)
      AND (INVOICE_ID = :INVOICE_ID OR :INVOICE_ID IS NULL);
```

Seu plano de execução é o seguinte:

```
Select Expression
  -> Aggregate
    -> Filter
      -> Condition
        -> Table "INVOICE" Full Scan
        -> Table "INVOICE" Access By ID
          -> Bitmap
            -> Index "FK_INVOICE_CUSTOMER" Range Scan (full match)
```

Como você pode ver, a filtragem usando índice é fornecida apenas para o campo CUSTOMER_ID, embora INVOICE_ID seja a chave primária da tabela e tenha a máxima seletividade. Vamos tentar trocar as condições de filtragem de lugar:

```
SELECT COUNT(*)
FROM INVOICE
WHERE (INVOICE_ID = :INVOICE_ID OR :INVOICE_ID IS NULL)
      AND (CUSTOMER_ID = :CUSTOMER_ID OR :CUSTOMER_ID IS NULL);
```

Seu plano será:

```
Select Expression
  -> Aggregate
    -> Filter
      -> Condition
        -> Table "INVOICE" Full Scan
        -> Table "INVOICE" Access By ID
          -> Bitmap
            -> Index "PK_INVOICE" Unique Scan
```

Agora podemos usar o índice em INVOICE_ID, mas não em CUSTOMER_ID. Assim, apenas uma “Ramificação condicional de fluxos” pode ser executada com eficiência, portanto é necessário escolher com responsabilidade qual condição será a principal.

A ramificação condicional de fluxos tem mais uma característica que se manifesta ao unir (JOIN) várias tabelas. O fato é que a própria ramificação é feita apenas para uma fonte de dados, e não para toda a árvore de execução da consulta. O otimizador usa a cardinalidade dos fluxos unidos como um dos principais parâmetros para escolher a ordem de junção das tabelas. No caso de usar ramificação condicional, o otimizador considera ambas as opções de execução igualmente prováveis e, portanto, estima a cardinalidade desse método de acesso como o valor médio entre ambas as opções de execução (para mais detalhes, consulte [Ramificação Condicional de Fluxos \(Conditional Stream\)](#)). Suponha que usemos o seguinte filtro para a tabela INVOICE – INVOICE_ID = :INVOICE_ID OR :INVOICE_ID IS NULL. A cardinalidade da tabela INVOICE é 100000. Se um valor diferente de NULL for especificado para o parâmetro INVOICE_ID, a cardinalidade do fluxo de saída deve ser igual a 1 (INVOICE_ID é a chave primária). No entanto, o otimizador, para qualquer valor desse parâmetro, estimará a cardinalidade como 50000. Isso pode levar a uma escolha incorreta da ordem de junção das tabelas para qualquer valor do parâmetro. Este caso será considerado com mais detalhes no capítulo "Otimização de junções".

7.3.10. Índice composto ou vários índices simples?

Como mostrado anteriormente, o Firebird consegue usar vários índices para filtrar por várias colunas de uma tabela. No entanto, para os mesmos fins, pode-se usar um índice composto por várias colunas. Vamos tentar entender o que é melhor.

Vantagens do índice composto:

- Pode ser usado para restrições de integridade complexas em várias colunas (Primary Key, Unique, Foreign Key);
- Ao usar vários segmentos do índice, a seletividade do predicado de filtragem é mais precisa, o que permite uma estimativa mais precisa da cardinalidade do fluxo de saída;
- O uso de um índice composto para filtragem é mais barato do que o uso de vários índices com um mapa de bits, se as condições de filtragem forem combinadas com AND.

Desvantagens do índice composto:

- Nem todos os segmentos do índice podem ser usados, ou mesmo nenhum;
- Índices compostos ocupam mais espaço e são mais caros para manter em operações DML (o índice é modificado quando qualquer uma das colunas que compõem o índice composto é alterada);
- É impossível usar vários segmentos de um índice composto ao combinar condições de filtragem com OR;
- É impossível especificar uma expressão para uma das colunas que compõem o índice composto.

A seguir, esclareceremos algumas dessas afirmações.

A filtragem por índice composto é mais barata

O uso de um índice composto para filtragem é mais barato do que o uso de vários índices com um mapa de bits, se as condições de filtragem forem combinadas com AND.

```
SELECT *
FROM INVOICE_LINE
WHERE INVOICE_ID = 138616
      AND PRODUCT_ID = 2312;
```

As estatísticas e o plano de execução serão os seguintes:

```
Select Expression
-> Filter
    -> Table "INVOICE_LINE" Access By ID
        -> Bitmap And
            -> Bitmap
                -> Index "FK_INVOICE_LINE_INVOICE" Range Scan (full match)
            -> Bitmap
                -> Index "FK_INVOICE_LINE_PRODUCT" Range Scan (full match)

Current memory = 284901600
Delta memory = 0
Max memory = 284981648
Elapsed time = 0.012 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 7
```

Agora vamos criar um índice composto nos campos INVOICE_ID, PRODUCT_ID.

```
CREATE INDEX IDX_INVOICE_LINE_INV_PRODUCT ON INVOICE_LINE(INVOICE_ID, PRODUCT_ID);
```

Execute novamente a consulta descrita acima e veja o plano e as estatísticas:

```
Select Expression
-> Filter
    -> Table "INVOICE_LINE" Access By ID
        -> Bitmap
            -> Index "IDX_INVOICE_LINE_INV_PRODUCT" Range Scan (full match)

Current memory = 283974544
Delta memory = 0
Max memory = 329937712
Elapsed time = 0.012 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 5
```

Parece que a melhoria é insignificante, ganhamos apenas 2 leituras lógicas (fetches), no entanto, se essa filtragem for repetida várias vezes ao unir tabelas via Nested loop ou ao executar uma subconsulta, o ganho pode ser mais perceptível.

Vamos alterar nossa consulta para que a seleção por um índice composto ou dois índices não compostos seja repetida várias vezes:

```
SELECT COUNT(*)
FROM INVOICE
WHERE NOT EXISTS(
  SELECT *
  FROM INVOICE_LINE L
  WHERE L.INVOICE_ID = INVOICE.INVOICE_ID
    AND L.PRODUCT_ID = 2312
);
```

Vejamos o plano e as estatísticas sem criar um índice composto:

```
Sub-query
  -> Filter
    -> Table "INVOICE_LINE" as "L" Access By ID
      -> Bitmap And
        -> Bitmap
          -> Index "FK_INVOICE_LINE_INVOICE" Range Scan (full match)
        -> Bitmap
          -> Index "FK_INVOICE_LINE_PRODUCT" Range Scan (full match)
Select Expression
  -> Aggregate
    -> Filter
      -> Table "INVOICE" Full Scan

          COUNT
=====
          99645

Current memory = 284186656
Delta memory = 288
Max memory = 329937712
Elapsed time = 1.010 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 705290
```

Agora vamos criar o índice composto IDX_INVOICE_LINE_INV_PRODUCT e comparar as estatísticas:

```
Sub-query
  -> Filter
    -> Table "INVOICE_LINE" as "L" Access By ID
      -> Bitmap
        -> Index "IDX_INVOICE_LINE_INV_PRODUCT" Range Scan (full match)
```



```
Select Expression
  -> Aggregate
    -> Filter
      -> Table "INVOICE" Full Scan
```

```
          COUNT
=====
          99645
```

```
Current memory = 284083504
Delta memory = 288
Max memory = 330217104
Elapsed time = 0.192 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 504701
```

A diferença é mais do que perceptível. Mas por que isso acontece?

Vamos relembrar a fórmula para calcular o custo da varredura de índice por igualdade.

$$\text{cost} = \text{indexDepth} + \text{MAX}(\text{avgKeyLength} * \text{table_cardinality} * \text{index_selectivity} / (\text{page_size} - \text{BTR_SIZE}), 1)$$

Onde avgKeyLength é o comprimento médio da chave, page_size é o tamanho da página, BTR_SIZE é o tamanho do cabeçalho da página de índice, atualmente 39 bytes.

Atualmente, o comprimento médio da chave não é armazenado nas estatísticas do índice, mas é calculado com base no grau médio de compactação e no comprimento da chave usando a fórmula:

$$\text{avgKeyLength} = 2 + \text{length} * \text{factor}$$

Aqui, length é o comprimento da chave do índice, factor é o grau de compactação, index_selectivity é a seletividade do conjunto de segmentos do índice usados na varredura por igualdade.

Em ambos os casos (índice composto e dois índices simples), o Firebird precisa ler as páginas desde a raiz do índice (indexDepth -1) mais um certo número de páginas folha do índice. Em seguida, construir um mapa de bits. Se a varredura for realizada usando dois índices, também é necessário construir um novo mapa de bits, que é a interseção dos mapas de bits. Depois disso, um certo número de páginas da tabela é varrido (em ambos os casos, esse número é o mesmo).

Vejamos as estatísticas da tabela e dos índices:

```
INVOICE_LINE (130)
  Primary pointer page: 232, Index root page: 233
  Total formats: 1, used formats: 1
  Average record length: 31.98, total records: 1000003
  Average version length: 0.00, total versions: 0, max versions: 0
  Average fragment length: 0.00, total fragments: 0, max fragments: 0
```

Average unpacked length: 32.00, compression ratio: 1.00
 Pointer pages: 7, data page slots: 9808
 Data pages: 9808, average fill: 61%
 Primary pages: 9808, secondary pages: 0, swept pages: 0
 Empty pages: 3, full pages: 9803
 Table size: 80404480 bytes
 Fill distribution:

0 - 19% = 4
 20 - 39% = 0
 40 - 59% = 1
 60 - 79% = 9803
 80 - 99% = 0

Index FK_INVOICE_LINE_INVOICE (1)

Root page: 12035, depth: 2, leaf buckets: 619, nodes: 1000003
 Average node length: 4.99, total dup: 900009, max dup: 27
 Average key length: 2.10, compression ratio: 1.88
 Average prefix length: 3.86, average data length: 0.10
 Clustering factor: 999473, ratio: 1.00
 Fill distribution:

0 - 19% = 0
 20 - 39% = 0
 40 - 59% = 0
 60 - 79% = 0
 80 - 99% = 619

Index FK_INVOICE_LINE_PRODUCT (2)

Root page: 12656, depth: 2, leaf buckets: 606, nodes: 1000003
 Average node length: 4.89, total dup: 997203, max dup: 429
 Average key length: 2.00, compression ratio: 1.47
 Average prefix length: 2.95, average data length: 0.00
 Clustering factor: 982361, ratio: 0.98
 Fill distribution:

0 - 19% = 0
 20 - 39% = 0
 40 - 59% = 0
 60 - 79% = 1
 80 - 99% = 605

Index IDX_INVOICE_LINE_INV_PRODUCT (3)

Root page: 13961, depth: 3, leaf buckets: 985, nodes: 1000003
 Average node length: 7.92, total dup: 1426, max dup: 2
 Average key length: 5.03, compression ratio: 1.78
 Average prefix length: 6.79, average data length: 2.16
 Clustering factor: 999909, ratio: 1.00
 Fill distribution:

0 - 19% = 0
 20 - 39% = 0
 40 - 59% = 0
 60 - 79% = 0
 80 - 99% = 985

As seletividades dos índices são as seguintes:

- FK_INVOICE_LINE_INVOICE - 0.0000100006;

- FK_INVOICE_LINE_PRODUCT - 0.0003571428533;
- IDX_INVOICE_LINE_INV_PRODUCT - 0.0000010014251.

Agora vejamos o cálculo do custo para a varredura usando o índice IDX_INVOICE_LINE_INV_PRODUCT:

$$\text{cost} = 3 + \text{MAX}(5.03 * 1000003 * 0.0000010014251 / (8192 - 39), 1) = 4.0$$

Custo da varredura pelos índices FK_INVOICE_LINE_INVOICE e FK_INVOICE_LINE_PRODUCT:

$$\text{Cost}(\text{FK_INVOICE_LINE_INVOICE}) = 2 + \text{MAX}(2.10 * 1000003 * 0.0000100006 / (8192 - 39), 1) = 3.0$$

$$\text{Cost}(\text{FK_INVOICE_LINE_PRODUCT}) = 2 + \text{MAX}(2.00 * 1000003 * 0.0003571428533 / (8192 - 39), 1) = 3.0$$

Obtemos o custo da varredura apenas das páginas de índice (sem as páginas da tabela):

$$\text{only_index_cost} = \text{indexDepth} - 1 + \text{MAX}(\text{leaf_pages} * \text{index_selectivity}, 1)$$

$$\text{Cost}^*(\text{FK_INVOICE_LINE_INVOICE}) = 1 + \text{MAX}(619 * 0.0000100006, 1) = 2.0$$

$$\text{Cost}^*(\text{FK_INVOICE_LINE_PRODUCT}) = 1 + \text{MAX}(606 * 0.0003571428533, 1) = 2.0$$

Custo da varredura por mapa de bits (estimativa):

$$\text{sel} = (\text{bestSel} + (\text{worstSel} - \text{bestSel}) / (1 - \text{bestSel}) * \text{bestSel}) / 2$$

$$(0.0000100006 + (0.0003571428533 - 0.0000100006) / (1e0 - 0.0000100006) * 0.0000100006) / 2 = 0.0000050020358$$

Na realidade, a seletividade do mapa de bits será a mesma que a do índice composto, ou seja, 0.0000010014251.

Assim, o custo da varredura da tabela pelo mapa de bits será igual a

$$\text{cost}(\text{bitmap_scan}) = 0.0000010014251 * 1000003 = 1.014$$

O custo total da varredura pelo mapa de bits de dois índices é igual a:

$$\text{Cost}^*(\text{FK_INVOICE_LINE_INVOICE}) + \text{Cost}^*(\text{FK_INVOICE_LINE_PRODUCT}) + \text{cost}(\text{bitmap_scan})$$

$$\text{cost} = 2.0 + 2.0 + 1.014 = 5.014$$

Ou seja, será um pouco maior do que ao varrer apenas pelo índice composto.



O cálculo do custo não leva em consideração o custo de construção dos próprios mapas de bits e sua interseção. Ela não é gratuita, mas não é considerada no otimizador.

Seletividade do índice composto é mais precisa

Ao usar vários segmentos de índice, a seletividade do predicado de filtragem é mais precisa, permitindo uma estimativa mais precisa da cardinalidade do fluxo de saída.

Para um índice composto, a seletividade "exata" é armazenada para cada um de seus segmentos (conjunto de campos), enquanto a seletividade das máscaras de bits é estimada aproximadamente. Assume-se que as condições de filtragem estão correlacionadas com uma probabilidade de 50%.

Vamos relembrar o exemplo anterior. A seletividade do índice composto é 0.0000010014251, enquanto a seletividade estimada da interseção das máscaras de bits é calculada como 0.0000050020358. A seletividade mais precisa permite estimar com mais exatidão a cardinalidade do fluxo de saída. Quando a seleção é feita a partir de uma única tabela, isso não tem grande importância, mas ao unir fluxos (JOIN) a cardinalidade é crucial para escolher a ordem das junções e, conseqüentemente, para o desempenho de toda a consulta.

Índices compostos têm custo de manutenção mais alto

Índices compostos geralmente ocupam mais espaço em disco (mais páginas) e suas chaves têm compressão pior em comparação com índices simples. Para confirmar isso, basta observar as estatísticas do índice composto e do simples no exemplo anterior.

Além disso, como a chave do índice composto inclui vários campos, esse índice precisa ser atualizado em operações DML quando qualquer um desses campos é alterado. Lembre-se de que no Firebird os índices não são modificados se os campos que compõem sua chave não forem alterados.

Índices compostos não podem ser usados para unir predicados com OR

Como você viu acima, índices compostos substituem perfeitamente a filtragem por vários índices ao unir predicados de filtragem com AND, mas isso não funciona para unir predicados com OR.

Suponha que temos uma tabela e nela está definido um índice composto:

```
RECREATE TABLE TEST_INDEXES (
  ID BIGINT GENERATED BY DEFAULT AS IDENTITY,
  A BIGINT,
  B BIGINT,
  CONSTRAINT PK_TEST_INDEXES PRIMARY KEY(ID)
);

CREATE INDEX IDX_TEST_INDEXES_A_B ON TEST_INDEXES(A, B);
```

Se obtivermos o plano para a consulta

```
SELECT *
```

```
FROM TEST_INDEXES
WHERE A = 500 OR B = 0;
```

você verá o seguinte:

```
Select Expression
-> Filter
    -> Table "TEST_INDEXES" Full Scan
```

Nenhum dos segmentos do índice composto é utilizado.

Agora vamos tentar criar índices separados para cada campo:

```
CREATE INDEX IDX_TEST_INDEXES_A ON TEST_INDEXES(A);
CREATE INDEX IDX_TEST_INDEXES_B ON TEST_INDEXES(B);
```

Vamos obter novamente o plano da consulta acima:

```
Select Expression
-> Filter
    -> Table "TEST_INDEXES" Access By ID
        -> Bitmap Or
            -> Bitmap
                -> Index "IDX_TEST_INDEXES_A" Range Scan (full match)
            -> Bitmap
                -> Index "IDX_TEST_INDEXES_B" Range Scan (full match)
```

Aqui, ambos os índices são utilizados com sucesso.

É impossível definir uma expressão para uma das colunas que compõem um índice composto

A união de predicados com AND via máscara de bits pode ser aplicada a diferentes tipos de índices, um dos quais pode ser um índice por expressão. Por exemplo, para a consulta:

```
SELECT *
FROM INVOICE
WHERE CUSTOMER_ID = 86
    AND EXTRACT(YEAR FROM INVOICE_DATE) = 2015;
```

O plano será:

```
Select Expression
-> Filter
    -> Table "INVOICE" Access By ID
        -> Bitmap And
            -> Bitmap
                -> Index "FK_INVOICE_CUSTOMER" Range Scan (full match)
            -> Bitmap
```

```
-> Index "INVOICE_IDX_BY_YEAR" Range Scan (full match)
```

desde que o seguinte índice por expressão tenha sido criado:

```
CREATE INDEX INVOICE_IDX_BY_YEAR ON INVOICE COMPUTED BY (EXTRACT(YEAR FROM INVOICE_DATE));
```

Você pode criar um índice composto pelos campos CUSTOMER_ID e INVOICE_DATE e usar esse índice em consultas do tipo

```
CREATE INDEX INVOICE_IDX_CUST_DATE ON INVOICE (CUSTOMER_ID, INVOICE_DATE);

SELECT *
FROM INVOICE
WHERE CUSTOMER_ID = 86
      AND INVOICE_DATE BETWEEN '2015-01-01' AND '2015-12-31 23:59:59.9999';
```

No entanto, em um índice composto é impossível definir a expressão EXTRACT(YEAR FROM INVOICE_DATE) em vez da coluna INVOICE_DATE.

Quando é necessário criar um índice composto?

Definitivamente, um índice composto é necessário se for necessária uma chave composta (Primary Key, Foreign Key, Unique). Nos demais casos, é preciso pesar os prós e contras.

Antes de criar um índice composto, é preciso analisar as consultas e escolher uma ordem de campos no índice composto que permita que, na maioria das consultas, o maior número possível de segmentos do índice composto seja utilizado. Um erro aqui pode ser custoso. Veja [Exemplo de ordem incorreta de campos em um índice composto](#).

Sempre verifique suas consultas após a criação de índices compostos. Como mencionado acima, um índice composto nem sempre oferece uma vantagem significativa em comparação com vários índices simples. A vantagem dos índices compostos se torna perceptível se a varredura usando esse índice for executada repetidamente (subconsultas, junção por Nested Loop). Se seu índice composto não proporcionar um ganho significativo de desempenho na consulta (por exemplo, apenas 5%), talvez valha a pena abandoná-lo em favor de vários índices simples, pois os índices compostos têm sobrecarga adicional em operações DML.

7.3.11. Índices parciais

Até agora, consideramos índices que são construídos sobre todos os registros da tabela. A partir do Firebird 5.0, é possível criar os chamados "índices parciais", que indexam um subconjunto de registros da tabela, correspondente à condição de filtragem especificada no índice. Qualquer tipo de índice (comum, único, composto, índice por expressão) pode ser parcial, se uma condição de filtragem correspondente for definida para ele. Vejamos a sintaxe completa para criação de índices no Firebird:

```
CREATE [UNIQUE] [ASC[ENDING] | DESC[ENDING]]
```

```
INDEX indexname ON tablename
{(<column_list> | COMPUTED [BY] (<value_expression>))}
[WHERE <search_condition>]

<column_list> ::= col [, col ...]
```

A condição de filtragem para índices parciais é especificada na expressão após a palavra-chave WHERE. Ela deve incluir pelo menos uma coluna da tabela. A condição de filtragem não precisa necessariamente incluir colunas da chave do índice.

Índices parciais não podem ser usados para restrições de integridade (PRIMARY KEY, FOREIGN KEY, UNIQUE), ou seja, na expressão USING INDEX não é possível especificar a definição de um índice parcial.

Índices parciais únicos

Índices parciais não podem ser usados para garantir uma restrição de unicidade (UNIQUE), no entanto você pode criar um índice parcial único. Um índice parcial único pode ser usado para garantir unicidade em um subconjunto de registros da tabela. A diferença é que com FOREIGN KEY é possível fazer referência a uma tabela com uma restrição de unicidade, mas não é possível se para ela tiver sido criado apenas um índice único sem restrição (Primary Key ou Unique).

Vamos considerar um exemplo de uso de índices parciais únicos.

Exemplo 90. Garantindo unicidade parcial

Suponha que temos uma tabela que armazena os endereços de email de uma pessoa.

```
CREATE TABLE MAN_EMAILS (
  CODE_MAN_EMAIL BIGINT GENERATED BY DEFAULT AS IDENTITY,
  CODE_MAN BIGINT NOT NULL,
  EMAIL VARCHAR(50) NOT NULL,
  DEFAULT_FLAG BOOLEAN DEFAULT FALSE NOT NULL,
  CONSTRAINT PK_MAN_EMAILS PRIMARY KEY(CODE_MAN_EMAIL),
  CONSTRAINT FK_EMAILS_REF_MAN FOREIGN KEY(CODE_MAN) REFERENCES MAN(CODE_MAN)
);
```

Uma pessoa pode ter vários endereços de email, mas apenas um pode ser o endereço padrão. Um índice único comum ou uma restrição não servirão neste caso, pois assim ficaríamos limitados a apenas dois endereços.

Aqui, um índice parcial único vem em nosso auxílio:

```
CREATE UNIQUE INDEX IDX_UNIQUE_DEFAULT_MAN_EMAIL
ON MAN_EMAILS(CODE_MAN) WHERE DEFAULT_FLAG IS TRUE;
```

Dessa forma, para uma pessoa permitimos quantos endereços com DEFAULT_FLAG=FALSE forem necessários e apenas um endereço com DEFAULT_FLAG=TRUE.

Quando índices parciais podem ser usados pelo otimizador?

Índices parciais podem ser úteis na otimização de consultas. A principal vantagem dos índices parciais é que eles são mais compactos (sua profundidade pode ser menor, o número de páginas folha também é menor).

Um índice parcial pode ser utilizado pelo otimizador se:

- a condição WHERE incluir exatamente a mesma expressão lógica definida para o índice;
- a condição de pesquisa definida para o índice contiver expressões lógicas unidas por OR, e uma delas estiver explicitamente incluída na condição WHERE;
- a condição de pesquisa definida para o índice especificar IS NOT NULL, e a condição WHERE incluir uma expressão para o mesmo campo que, sabe-se, ignora NULL.

Além disso, todas as regras de uso de índices comuns são mantidas:

- o predicado para a(s) coluna(s) chave do índice deve ser capaz de utilizar índices (consulte [Uso de índices para filtragem](#));
- a expressão para a chave do índice deve ser usada no predicado de filtragem (consulte [Índice por expressão](#));
- a ordem das colunas na chave de um índice parcial composto é importante e afeta quantos segmentos do índice serão usados (consulte [Índices compostos](#)).

Se existir um índice comum e um índice parcial para o mesmo conjunto de campos, o otimizador escolherá o índice comum, mesmo que a condição WHERE inclua a mesma expressão definida no índice parcial. A razão para esse comportamento é que a seletividade do índice comum é melhor do que a do índice parcial. Mas existem exceções a essa regra: o uso de predicados com baixa seletividade para os campos indexados, como <>, IS DISTINCT FROM ou IS NOT NULL, desde que esse predicado seja usado no índice parcial.

Seletividade do índice parcial

É importante notar que na tabela RDB\$INDICES é armazenada a seletividade para a parte dos registros que foi incluída no índice. O mesmo se aplica à seletividade dos segmentos de índices armazenados em RDB\$INDEX_SEGMENTS. A seletividade final do índice ou dos segmentos correspondentes é estimada como:

$$\text{indexSelectivity} = \text{storedIndexSelectivity} * \text{filterSelectivity}$$

$$\text{indexSegSelectivity} = \text{storedIndexSegSelectivity} * \text{filterSelectivity}$$

Aqui, filterSelectivity é a seletividade do predicado de filtragem que foi usado na cláusula WHERE ao definir o índice parcial.

Exemplo 91. Avaliação da seletividade do índice parcial pelo otimizador

Suponha que você crie o seguinte índice parcial:


```
CREATE INDEX IDX_FARM_RU_REGION ON FARM(CODE_REGION) WHERE CODE_COUNTRY = 1;
```

As estatísticas para este índice são:

```
SELECT
  RDB$STATISTICS
FROM RDB$INDICES I
WHERE I.RDB$INDEX_NAME = 'IDX_FARM_RU_REGION'
```

```

RDB$STATISTICS
=====
0.01111111138015985
```

A seletividade final será igual à seletividade armazenada do índice, multiplicada pela seletividade do predicado de filtragem `CODE_COUNTRY = 1` (igual a 0.1, consulte [Verificação de predicados](#)).

```
indexSelectivity = 0.01111111138015985 * 0.1 = 0.00111111113801599
```

Redução do tamanho do índice

Suponha que você tenha uma tabela de cavalos `HORSE` em seu banco de dados e nela haja um campo `IS_ANCESTOR`, que é usado para marcar se um cavalo é o fundador de uma linhagem ou família. Obviamente, o número de fundadores é centenas de vezes menor do que o de outros cavalos.

Por exemplo, se executarmos a seguinte consulta, obteremos:

```
SELECT
  COUNT(*) FILTER(WHERE IS_ANCESTOR IS TRUE) AS CNT_ANCESTOR,
  COUNT(*) FILTER(WHERE IS_ANCESTOR IS FALSE) AS CNT_OTHER
FROM HORSE
```

```

CNT_ANCESTOR      CNT_OTHER
=====
1472              549861
```

A tarefa é obter rapidamente a lista de fundadores. A partir das estatísticas fornecidas, também é óbvio que para o caso `IS_ANCESTOR IS FALSE` o uso de índices é praticamente inútil.

Em princípio, podemos criar um índice comum

```
CREATE INDEX IDX_HORSE_IS_ANCESTOR ON HORSE(IS_ANCESTOR);
```

Ele será usado pelo otimizador e filtrará os fundadores de forma bastante eficiente:

```
SELECT COUNT(*)
FROM HORSE
WHERE IS_ANCESTOR IS TRUE;
```

```
Select Expression
  -> Aggregate
    -> Filter
      -> Table "HORSE" Access By ID
        -> Bitmap
          -> Index "IDX_HORSE_IS_ANCESTOR" Range Scan (full match)
```

```
      COUNT
=====
      1472
```

```
Current memory = 553526064
Delta memory = 176
Max memory = 572330768
Elapsed time = 0.004 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 2425
Per table statistics:
```

Table name	Natural	Index	Insert	Update	Delete
HORSE		1472			

Mas, neste caso, tal índice seria redundante. Vamos ver suas estatísticas:

```
Index IDX_HORSE_IS_ANCESTOR (30)
  Root page: 179763, depth: 2, leaf buckets: 169, nodes: 551333
  Average node length: 4.95, total dup: 551331, max dup: 549860
  Average key length: 2.00, compression ratio: 0.50
  Average prefix length: 1.00, average data length: 0.00
  Clustering factor: 10610, ratio: 0.02
  Fill distribution:
    0 - 19% = 1
    20 - 39% = 0
    40 - 59% = 0
    60 - 79% = 0
    80 - 99% = 168
```

Agora, exclua o índice anterior e tente criar um índice parcial em seu lugar:

```
DROP INDEX IDX_HORSE_IS_ANCESTOR;
```

```
CREATE INDEX IDX_HORSE_IS_ANCESTOR ON HORSE(IS_ANCESTOR) WHERE IS_ANCESTOR IS TRUE;
```

Execute a mesma consulta:

```
SELECT COUNT(*)
FROM HORSE
WHERE IS_ANCESTOR IS TRUE;
```

```
Select Expression
  -> Aggregate
    -> Filter
      -> Table "HORSE" Access By ID
        -> Bitmap
          -> Index "IDX_HORSE_IS_ANCESTOR" Full Scan
```

```
      COUNT
=====
      1472
```

```
Current memory = 551702496
Delta memory = 176
Max memory = 551791696
Elapsed time = 0.003 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 2423
Per table statistics:
```

Table name	Natural	Index	Insert	Update	Delete
HORSE		1472			

A consulta que usa o índice parcial acabou sendo um pouco mais rápida e realiza menos leituras lógicas (Fetches). Vejamos suas estatísticas:

```
Index IDX_HORSE_IS_ANCESTOR (30)
  Root page: 179761, depth: 1, leaf buckets: 1, nodes: 1472
  Average node length: 4.87, total dup: 1471, max dup: 1471
  Average key length: 2.00, compression ratio: 0.50
  Average prefix length: 1.00, average data length: 0.00
  Clustering factor: 949, ratio: 0.64
  Fill distribution:
    0 - 19% = 0
    20 - 39% = 0
    40 - 59% = 1
    60 - 79% = 0
    80 - 99% = 0
```

A profundidade do índice parcial acabou sendo menor (1 contra 2 do comum), além disso, o número de páginas folha é significativamente menor (1 contra 169).

A vantagem do índice parcial aqui também está no fato de que ele não será usado para o predicado `IS_ANCESTOR IS FALSE`.



Observe que se na consulta você especificar `WHERE IS_ANCESTOR` ou `WHERE IS_ANCESTOR = TRUE`, o índice não será usado. É necessário que a expressão especificada para a filtragem do índice coincida completamente com a expressão no `WHERE` da sua consulta.

Uso de índices parciais com predicados não seletivos

Suponha que precisamos obter todos os cavalos mortos para os quais a data da morte é conhecida. Um cavalo está definitivamente morto se sua data de morte estiver definida, mas muitas vezes essa data não é definida ou simplesmente é desconhecida. Além disso, o número de datas de morte desconhecidas é muito maior do que o das conhecidas. Para isso, escrevemos a seguinte consulta:

```
SELECT COUNT(*)
FROM HORSE
WHERE DEATHDATE IS NOT NULL;
```

Queremos obter essa lista o mais rápido possível, então tentaremos criar um índice no campo `DEATHDATE`.

```
CREATE INDEX IDX_HORSE_DEATHDATE
ON HORSE(DEATHDATE);
```

Agora tentaremos executar a consulta acima e observar seu plano e estatísticas:

```
Select Expression
  -> Aggregate
    -> Filter
      -> Table "HORSE" Full Scan
```

```
          COUNT
=====
          16234
```

```
Current memory = 2579550800
Delta memory = 176
Max memory = 2596993840
Elapsed time = 0.196 sec
Buffers = 153600
Reads = 0
Writes = 0
Fetches = 555810
Per table statistics:
```

```
-----+-----+-----+-----+-----+-----+
```

Table name	Natural	Index	Insert	Update	Delete
HORSE	519623				

Como você pode ver, não foi possível utilizar o índice. A razão é que os predicados `IS NOT NULL`, `<>`, `IS DISTINCT FROM` são pouco seletivos. Atualmente, o Firebird não possui histogramas com a distribuição dos valores das chaves do índice, portanto a distribuição é considerada uniforme. Com uma distribuição uniforme, não faz sentido usar um índice para tais predicados, e é isso que é feito.

Agora vamos tentar excluir o índice criado anteriormente e criar um índice parcial em seu lugar:

```
DROP INDEX IDX_HORSE_DEATHDATE;

CREATE INDEX IDX_HORSE_DEATHDATE
ON HORSE(DEATHDATE) WHERE DEATHDATE IS NOT NULL;
```

E vamos tentar repetir a consulta acima:

```
Select Expression
  -> Aggregate
    -> Filter
      -> Table "HORSE" Access By ID
        -> Bitmap
          -> Index "IDX_HORSE_DEATHDATE" Full Scan
```

```

COUNT
=====
16234
```

```
Current memory = 2579766848
Delta memory = 176
Max memory = 2596993840
Elapsed time = 0.017 sec
Buffers = 153600
Reads = 0
Writes = 0
Fetches = 21525
Per table statistics:
```

Table name	Natural	Index	Insert	Update	Delete
HORSE		16234			

Como você pode ver, o otimizador conseguiu utilizar nosso índice. Mas o mais interessante é que nosso índice continuará funcionando também com outros predicados de comparação com data (para `IS NULL` não funcionará).

```
SELECT COUNT(*)
```

```
FROM HORSE
WHERE DEATHDATE = DATE '01.01.2005';
```

```
Select Expression
  -> Aggregate
    -> Filter
      -> Table "HORSE" Access By ID
        -> Bitmap
          -> Index "IDX_HORSE_DEATHDATE" Range Scan (full match)
```

```
      COUNT
=====
      190
```

```
Current memory = 2579872992
Delta memory = 192
Max memory = 2596993840
Elapsed time = 0.004 sec
Buffers = 153600
Reads = 0
Writes = 0
Fetches = 376
Per table statistics:
```

Table name	Natural	Index	Insert	Update	Delete
HORSE		190			

Isso ocorre porque o otimizador, nesse caso, deduziu que a condição de filtro IS NOT NULL no índice parcial cobre quaisquer outros predicados que não comparem com NULL.

Vejamos outro exemplo de uso de índice parcial para um predicado não seletivo.

Suponha que temos uma tabela de fazendas FARM, e 90% dessas fazendas estão no Reino Unido. Queremos obter rapidamente a quantidade de tais fazendas. Para isso, usamos a consulta:

```
SELECT
  COUNT(*)
FROM FARM
WHERE CODE_COUNTRY <> 1
```

Um índice comum, criado no campo CODE_COUNTRY, não será utilizado pelo otimizador, pois o predicado não é seletivo. No entanto, podemos criar o seguinte índice parcial, que será utilizado pelo otimizador:

```
CREATE INDEX IDX_FARM_NON_ENGLISH ON FARM(CODE_COUNTRY) WHERE CODE_COUNTRY <> 1;
```

Agora, ao executar a consulta acima, o índice IDX_FARM_NON_ENGLISH será utilizado.

```

Select Expression
  -> Aggregate
    -> Filter
      -> Table "FARM" Access By ID
        -> Bitmap
          -> Index "IDX_FARM_NON_ENGLISH" Full Scan

```

COUNT

```

=====
4336

```

Current memory = 553450864

Delta memory = 176

Max memory = 555844176

Elapsed time = 0.004 sec

Buffers = 32768

Reads = 0

Writes = 0

Fetches = 5360

Per table statistics:

Table name	Natural	Index	Insert	Update	Delete
FARM		4336			

Índices parciais com condição que não inclui a coluna chave

A condição de filtro de um índice parcial não precisa necessariamente incluir colunas que fazem parte da chave do índice. Tais índices são úteis em dois casos:

- Para garantir unicidade parcial;
- Como substituto de um índice composto no caso de distribuição não uniforme de valores.

Suponha que você execute a seguinte consulta:

```

SELECT COUNT(*)
FROM FARM
WHERE CODE_COUNTRY = 1
AND CODE_REGION = 10;

```

Na tabela FARM, as regiões são preenchidas apenas para CODE_COUNTRY = 1, e nos outros casos são NULL. Nessa situação, a estatística real para CODE_COUNTRY = 1 e CODE_COUNTRY = 2 será muito diferente, mas na versão atual do Firebird não há histogramas de distribuição de valores, portanto a seletividade do conjunto de segmentos CODE_COUNTRY, CODE_REGION será calculada assumindo que todos os valores de CODE_REGION estão distribuídos uniformemente para qualquer CODE_COUNTRY.

Vamos observar as estatísticas de execução e o plano desta consulta. Primeiro, sem índice composto e parcial, mas com índices separados para as colunas CODE_COUNTRY e CODE_REGION.

```

Select Expression
  -> Aggregate
    -> Filter
      -> Table "FARM" Access By ID
        -> Bitmap And
          -> Bitmap
            -> Index "FK_FARM_REGION" Range Scan (full match)
          -> Bitmap
            -> Index "FK_FARM_COUNTRY" Range Scan (full match)

```

```

COUNT
=====
21

```

```

Current memory = 552067248
Delta memory = 192
Max memory = 555581952
Elapsed time = 0.003 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 37
Per table statistics:

```

Table name	Natural	Index	Insert	Update	Delete
FARM		21			

Podemos criar um índice composto para melhorar o desempenho.

```
CREATE INDEX IDX_FARM_COUNTRY_REGION ON FARM(CODE_COUNTRY, CODE_REGION);
```

Vejamos novamente as estatísticas:

```

Select Expression
  -> Aggregate
    -> Filter
      -> Table "FARM" Access By ID
        -> Bitmap
          -> Index "IDX_FARM_COUNTRY_REGION" Range Scan (full match)

```

```

COUNT
=====
21

```

```

Current memory = 552257088
Delta memory = 192
Max memory = 555581952
Elapsed time = 0.002 sec
Buffers = 32768
Reads = 0

```



```
Writes = 0
Fetches = 24
Per table statistics:
```

Table name	Natural	Index	Insert	Update	Delete
FARM		21			

As estatísticas ficaram um pouco melhores. Vejamos o caso em que CODE_COUNTRY = 1.

```
SELECT COUNT(*)
FROM FARM
WHERE CODE_COUNTRY = 10
      AND CODE_REGION IS NULL;
```

```
Select Expression
-> Aggregate
  -> Filter
    -> Table "FARM" Access By ID
      -> Bitmap
        -> Index "IDX_FARM_COUNTRY_REGION" Range Scan (full match)
```

```
COUNT
=====
160
```

```
Current memory = 552290688
Delta memory = 192
Max memory = 555581952
Elapsed time = 0.002 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 275
Per table statistics:
```

Table name	Natural	Index	Insert	Update
FARM		160		

O otimizador utilizará o índice composto, embora seja óbvio que basta usar apenas o índice FK_FARM_COUNTRY, pois CODE_REGION IS NULL para qualquer uma das chaves CODE_COUNTRY <> 1. Portanto, a condição de filtro CODE_REGION IS NULL pode ser completamente removida da consulta.

```
SELECT COUNT(*)
FROM FARM
WHERE CODE_COUNTRY = 10;
```

```

Select Expression
  -> Aggregate
    -> Filter
      -> Table "FARM" Access By ID
        -> Bitmap
          -> Index "FK_FARM_COUNTRY" Range Scan (full match)

```

COUNT

```

=====
160

```

Current memory = 552358224

Delta memory = 176

Max memory = 555581952

Elapsed time = 0.002 sec

Buffers = 32768

Reads = 0

Writes = 0

Fetches = 275

Per table statistics:

Table name	Natural	Index	Insert	Update	Delete
FARM		160			

Agora vamos remover o índice composto `IDX_FARM_COUNTRY_REGION` e criar um índice parcial `IDX_FARM_EN_REGION` em seu lugar.

```

DROP INDEX IDX_FARM_COUNTRY_REGION;
CREATE INDEX IDX_FARM_EN_REGION ON FARM(CODE_REGION) WHERE CODE_COUNTRY = 1;

```

Vamos ver a estatística de execução da consulta original.

```

SELECT COUNT(*)
FROM FARM
WHERE CODE_COUNTRY = 1
  AND CODE_REGION = 10;

```

```

Select Expression
  -> Aggregate
    -> Filter
      -> Table "FARM" Access By ID
        -> Bitmap
          -> Index "IDX_FARM_EN_REGION" Range Scan (full match)

```

COUNT

```

=====
21

```

```

Current memory = 552301792
Delta memory = 192
Max memory = 555581952
Elapsed time = 0.002 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 24
Per table statistics:

```

Table name	Natural	Index	Insert	Update	Delete
FARM		21			

Ela é quase a mesma do índice composto (na realidade pode ser melhor, pois índices parciais são mais compactos).

Vamos tentar executar uma consulta com `CODE_COUNTRY <> 1`.

```

SELECT COUNT(*)
FROM FARM
WHERE CODE_COUNTRY = 10
AND CODE_REGION IS NULL;

```

O plano e a estatística da consulta serão os seguintes:

```

Select Expression
  -> Aggregate
    -> Filter
      -> Table "FARM" Access By ID
        -> Bitmap And
          -> Bitmap
            -> Index "FK_FARM_REGION" Range Scan (full match)
          -> Bitmap
            -> Index "FK_FARM_COUNTRY" Range Scan (full match)

```

```

COUNT
=====
160

```

```

Current memory = 552371488
Delta memory = 192
Max memory = 555581952
Elapsed time = 0.003 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 281
Per table statistics:

```

Table name	Natural	Index	Insert	Update	Delete
FARM		21			

FARM			160			
-----+	-----+	-----+	-----+	-----+	-----+	-----+

A estatística de execução ficou pior, conforme esperado, pois em vez de um índice, são usados dois, que são combinados via máscaras de bits.

E agora vamos remover a condição de filtro desnecessária.

```
SELECT COUNT(*)
FROM FARM
WHERE CODE_COUNTRY = 10;
```

O plano e a estatística da consulta serão os seguintes:

```
Select Expression
  -> Aggregate
    -> Filter
      -> Table "FARM" Access By ID
        -> Bitmap
          -> Index "FK_FARM_COUNTRY" Range Scan (full match)
```

```
      COUNT
=====
          160
```

```
Current memory = 552404464
Delta memory = 176
Max memory = 555581952
Elapsed time = 0.002 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 275
Per table statistics:
```

-----+	-----+	-----+	-----+	-----+	-----+	-----+
Table name	Natural	Index	Insert	Update	Delete	
-----+	-----+	-----+	-----+	-----+	-----+	-----+
FARM		160				
-----+	-----+	-----+	-----+	-----+	-----+	-----+

A estatística ficou a mesma como se tivéssemos um índice composto.

O que este exemplo deve ensinar? Ao criar um índice parcial em vez de um composto, verifique novamente todas as consultas que poderiam usar o índice composto. Essas consultas podem ter desempenho pior ou melhor. E se o índice parcial der um ganho de desempenho, mantenha-o; caso contrário, ou reescreva as consultas ou remova o índice parcial.

Índices parciais com condições unidas por OR

Até agora consideramos situações em que uma das condições de filtro na consulta correspondia completamente à condição de filtro do índice parcial; caso contrário, o índice parcial não seria

usado. Há duas exceções a esta regra: índices parciais que usam várias condições unidas por OR.

Se as condições de filtro no índice parcial são conectadas usando OR, então uma consulta SQL que use qualquer uma dessas condições pode usar o índice parcial.

Tomemos o exemplo acima. Suponha que as regiões sejam preenchidas não para um país, mas para dois, nos demais casos CODE_REGION IS NULL. Então podemos criar o seguinte índice parcial.

```
DROP INDEX IDX_FARM_RU_REGION;
CREATE INDEX IDX_FARM_RU_BE_REGION ON FARM(CODE_REGION) WHERE CODE_COUNTRY = 1 OR CODE_COUNTRY = 7;
```

Agora o índice parcial pode ser usado tanto para o filtro CODE_COUNTRY = 1 quanto para CODE_COUNTRY = 7, e claro, para CODE_COUNTRY = 1 OR CODE_COUNTRY = 7. Ou seja, qualquer uma dessas consultas usará o índice parcial:

```
SELECT COUNT(*)
FROM FARM
WHERE CODE_COUNTRY = 1
      AND CODE_REGION = 10;

SELECT COUNT(*)
FROM FARM
WHERE CODE_COUNTRY = 7
      AND CODE_REGION = 10;

SELECT COUNT(*)
FROM FARM
WHERE (CODE_COUNTRY = 1 OR CODE_COUNTRY = 7)
      AND CODE_REGION = 10;
```

O plano de execução será o mesmo:

```
Select Expression
  -> Aggregate
    -> Filter
      -> Table "FARM" Access By ID
        -> Bitmap
          -> Index "IDX_FARM_RU_BE_REGION" Range Scan (full match)
```

Índices parciais com predicado IN na condição de filtro

Parece que a expressão para o índice acima poderia ser simplificada usando o predicado IN.

```
DROP INDEX IDX_FARM_RU_BE_REGION;
CREATE INDEX IDX_FARM_RU_BE_REGION ON FARM(CODE_REGION) WHERE CODE_COUNTRY IN (1, 7);
```

Mas neste caso você ficará desapontado. A partir do Firebird 5.0, a expressão IN não é convertida

em um conjunto de condições OR equivalentes, portanto, nenhuma dessas consultas usará o índice parcial:

```
SELECT COUNT(*)
FROM FARM
WHERE CODE_COUNTRY = 1
      AND CODE_REGION = 10;

SELECT COUNT(*)
FROM FARM
WHERE CODE_COUNTRY = 7
      AND CODE_REGION = 10;

SELECT COUNT(*)
FROM FARM
WHERE (CODE_COUNTRY = 1 OR CODE_COUNTRY = 7)
      AND CODE_REGION = 10;
```

No entanto, a consulta que usa exatamente a mesma condição de filtro do índice parcial será usada.

```
SELECT COUNT(*)
FROM FARM
WHERE CODE_COUNTRY IN (1, 7)
      AND CODE_REGION = 10;
```

```
Select Expression
  -> Aggregate
    -> Filter
      -> Table "FARM" Access By ID
        -> Bitmap
          -> Index "IDX_FARM_RU_BE_REGION" Range Scan (full match)
```

```
      COUNT
=====
          21
```

```
Current memory = 552367024
Delta memory = 208
Max memory = 555581952
Elapsed time = 0.002 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 24
Per table statistics:
```

Table name	Natural	Index	Insert	Update	Delete
FARM		21			

Quando índices parciais não podem ser usados

Nós examinamos os casos em que o otimizador pode usar um índice parcial e quando ele se mostra útil.

A regra principal para usar um índice parcial é que uma das condições de filtragem na consulta repita completamente a condição de filtragem no índice parcial. Se no índice parcial as condições de filtragem estiverem conectadas por OR, então tal índice pode ser usado em uma consulta contendo qualquer uma das condições de filtragem. Novamente, qualquer uma dessas condições de filtragem deve ser escrita exatamente da mesma forma que na consulta principal, ou seja, se temos um índice parcial:

```
CREATE INDEX IDX_1 ON TEST(FIELD) WHERE FIELD2 > 1;
```

então tal índice não pode ser usado na consulta

```
SELECT *
FROM TEST
WHERE FIELD = 1 AND FIELD2 > 2;
```

embora, de acordo com a lógica, os registros com `FIELD2 > 2` devam estar presentes no índice com a restrição `WHERE FIELD2 > 1`.

Como na condição de filtragem do índice parcial é impossível usar um parâmetro, também é impossível criar um índice parcial para uma consulta do tipo

```
SELECT *
FROM TEST
WHERE FIELD = ? AND FIELD2 = ?;
```

onde `FIELD` é a chave do índice, e `FIELD2` é o campo pelo qual o índice é filtrado. A única exceção a esta regra são condições do tipo `FIELD2 IS NOT NULL`. Ou seja, o índice descrito abaixo pode ser usado pela consulta acima.

```
CREATE INDEX IDX_1 ON TEST(FIELD) WHERE FIELD2 IS NOT NULL;
```

Também gostaríamos de observar que não se deve usar índices parciais para substituir o particionamento. Por exemplo, se na tabela o campo `CODE_SEX` pode ter apenas dois valores, então não se deve criar dois índices parciais em vez de um índice comum.

```
CREATE INDEX IDX_TEST_SEX_1 ON TEST(CODE_SEX) WHERE CODE_SEX = 1;
CREATE INDEX IDX_TEST_SEX_2 ON TEST(CODE_SEX) WHERE CODE_SEX = 2;
```

Dois índices em vez de um não trarão benefício, além disso, isso desabilitará a possibilidade de usar o índice em uma consulta como esta:

```
SELECT *
FROM TEST
WHERE CODE_SEX = ?;
```

7.4. Uso de índices em condições de junção de tabelas

Índices podem ser usados em condições de junção ao unir tabelas. Nesta seção, consideraremos apenas o uso ou não uso de índices em condições de junção ao unir tabelas. A otimização de junções é um tópico muito mais complexo e será abordado em um capítulo separado.

O que se entende por condição de junção? Uma condição de junção é um predicado cujo valor depende de ambas as fontes de dados que estão sendo unidas. Tentarei explicar isso com um exemplo em um pseudocódigo.

```
SELECT ...
FROM
  A [FULL | LEFT | RIGHT] JOIN B ON f(A, B)
WHERE ... f2(A, B) ...
```

```
SELECT ...
FROM
  A CROSS B
WHERE ... f2(A, B) ...
```

```
SELECT ...
FROM
  A, B
WHERE ... f2(A, B) ...
```

Aqui, $f(A, B)$ e $f2(A, B)$ são condições de junção dos fluxos A e B.

Se o predicado depende apenas de um dos fluxos, então ele provavelmente define uma condição de filtragem para esse fluxo, e não uma condição de junção. Isso é sempre verdadeiro para junções internas, mas para junções externas unidirecionais há uma nuance. Considere os seguintes exemplos:

```
-- condição de junção A.field1 = B.field2
SELECT *
FROM A JOIN B ON A.field1 = B.field2;

-- A.field3 = 5 - filtragem do fluxo A
SELECT *
FROM A JOIN B ON A.field3 = 5;

-- a consulta anterior pode ser transformada em
SELECT *
FROM A CROSS JOIN B
WHERE A.field3 = 5;

-- condição de junção A.field1 = B.field2 AND A.field3 = 5
```



```
SELECT *
FROM A JOIN B ON A.field1 = B.field2 AND A.field3 = 5;
```

Agora, demonstraremos como isso funciona com junções externas unidirecionais.

```
-- condição de junção A.field1 = B.field2
SELECT *
FROM A LEFT JOIN B ON A.field1 = B.field2;

-- B.field3 = 5 - filtragem do fluxo B
SELECT *
FROM A LEFT JOIN B ON B.field3 = 5;

-- a consulta anterior pode ser transformada em
SELECT *
FROM A LEFT JOIN (SELECT * FROM B WHERE B.field3 = 5) B ON TRUE;

-- condição de junção A.field3 = 5
-- o predicado A.field3 = 5 não filtra registros do fluxo A.
-- Ele depende da fonte de dados externa e, portanto, não pode ser movido para uma tabela
derivada,
-- como no exemplo acima. Portanto, ele só pode ser uma condição de junção, que limita
-- os dados da fonte B dependendo do registro atual da fonte A.
SELECT *
FROM A LEFT JOIN B ON A.field3 = 5;
```

7.4.1. Como os índices são usados em condições de junção?

Qualquer tipo de junção tem dois fluxos de entrada – o esquerdo e o direito. Para junções internas e externas completas, esses fluxos são semanticamente equivalentes. No caso de uma junção externa unidirecional, um dos fluxos é o principal (obrigatório) e o segundo é o secundário (opcional). Frequentemente, o fluxo principal também é chamado de externo, e o secundário de interno. Para uma junção externa esquerda, o fluxo principal (externo) é o esquerdo, e o secundário (interno) é o direito. Para uma junção externa direita – o oposto.

Para junções internas, apenas o otimizador decide qual fluxo será o principal e qual será o secundário.

De **Junções (Joins)** você sabe que a junção de duas fontes de dados pode ser executada por um dos seguintes algoritmos:

- Nested loop (laços aninhados);
- Hash (junção por hash);
- Merge (junção por mesclagem).

Para junções por laços aninhados, os índices podem ser usados para otimizar a filtragem do fluxo secundário para cada registro do fluxo principal.

Para junção por hash, os índices não são usados diretamente na condição de junção, mas suas estatísticas (seletividade) podem ser usadas para uma estimativa mais precisa da seletividade do

predicado de filtragem. Uma estimativa mais precisa da seletividade do predicado de filtragem afeta a estimativa da cardinalidade do resultado da junção e, portanto, o cálculo de seu custo. Isso se torna especialmente importante quando há mais de uma junção. Nesse caso, a ordem das junções e os algoritmos de junção usados podem mudar.

7.4.2. Uso de índices pelo algoritmo de junção Nested loop

A junção de fontes de dados pelo método de laços aninhados é executada da seguinte forma (veja [Junção por Laços Aninhados \(Nested Loop\)](#)). Um dos fluxos de entrada (externo) é aberto e um registro é lido dele. Em seguida, o segundo fluxo (interno) é aberto e um registro também é lido dele. Depois, a condição de junção é verificada. Se a condição for atendida, esses dois registros são mesclados em um e enviados para a saída. Em seguida, um segundo registro é lido do fluxo interno, e o processo se repete até que o EOF seja emitido por ele. Se o fluxo interno não contiver nenhum registro correspondente ao fluxo externo, duas opções são possíveis. No caso de uma junção interna, o registro não é retornado na saída. No caso de uma junção externa, unimos o registro do fluxo externo com a quantidade necessária de valores NULL e o enviamos para a saída. A seguir, independentemente do tipo de junção, lemos um segundo registro do fluxo externo e iniciamos o processo de iteração pelo fluxo interno novamente.

A característica chave deste método de junção é a seleção “aninhada” do fluxo interno. Obviamente, ler todo o fluxo interno para cada registro do fluxo externo é muito custoso. Portanto, a junção por laços aninhados funciona de forma eficiente apenas na presença de um índice aplicável à condição de junção. Nesse caso, em cada iteração, um subconjunto de registros que satisfazem o registro atual do fluxo externo será selecionado do fluxo interno.

Assim, um índice aplicável à condição de junção, essencialmente, acelera a filtragem do fluxo aninhado. As condições obrigatórias para a possibilidade de aplicação de um índice são descritas em [Uso de índices para filtragem](#). Mas há uma diferença importante. Diferentemente dos filtros comuns, as junções leem os índices várias vezes. Ou seja, em cada iteração do laço aninhado, o índice é “lido” da raiz até as páginas folha, depois as páginas folha do índice e as páginas de dados correspondentes da tabela. Nesse caso, a profundidade do índice desempenha um papel muito significativo.

Imagine: a tabela externa HORSE contém 500.000 registros, a interna FARM – 40.000 registros. Considere a seguinte consulta:

```
SELECT COUNT(*)
FROM
  HORSE
  LEFT JOIN FARM ON FARM.CODE_FARM = HORSE.CODE_FARM;
```

Plano da consulta:

```
Select Expression
-> Aggregate
    -> Nested Loop Join (outer)
        -> Table "HORSE" Full Scan
        -> Filter
            -> Table "FARM" Access By ID
```

```
-> Bitmap
    -> Index "PK_FARM" Unique Scan
```

Se a junção ocorrer usando o índice da chave primária, cuja profundidade é 2, o custo dessa junção será o seguinte.

```
cost = cardinality(outer) + cardinality(outer) * (indexScanCost + cardinality(inner) *
selectivity(link))

cardinality(inner) * selectivity(link) = 1 -- já que o índice é único

cost = 500000 + 500000 * (2 + 1) = 2000000
```

Vamos tentar executar a consulta, alterando a ordem das tabelas:

```
SELECT COUNT(*)
FROM
  FARM
  LEFT JOIN HORSE ON HORSE.CODE_FARM = FARM.CODE_FARM;
```

Plano da consulta:

```
Select Expression
-> Aggregate
    -> Nested Loop Join (outer)
        -> Table "FARM" Full Scan
        -> Filter
            -> Table "HORSE" Access By ID
                -> Bitmap
                    -> Index "FK_HORSE_FARMBORN" Range Scan (full match)
```

A profundidade do índice também é 2. Vamos tentar calcular seu custo:

```
cost = cardinality(outer) + cardinality(outer) * (indexScanCost + cardinality(inner) *
selectivity(link))

cardinality(inner) * selectivity(link) = 500000 * 0.00006 = 30

cost = 40000 + 40000 * (2 + 30) = 1320000
```

Como você pode ver, a ordem de junção das tabelas importa. É por isso que, no caso de junções internas pelo método Nested Loop, o otimizador tenta escolher como fluxo principal a fonte de dados com a menor cardinalidade.



Se cada tabela for lida por completo, o otimizador pode escolher o algoritmo de junção HASH JOIN em vez de Nested Loop. Para forçar o otimizador a usar Nested Loop, usamos a cláusula PLAN.

Demonstrarei a importância da ordem das junções com dados reais e estatísticas.

```
SELECT COUNT(*)
FROM HORSE
  JOIN FARM ON FARM.CODE_FARM = HORSE.CODE_FARM
PLAN JOIN (FARM NATURAL, HORSE INDEX (FK_HORSE_FARMBORN));
```

```
Select Expression
-> Aggregate
  -> Nested Loop Join (inner)
    -> Table "FARM" Full Scan
    -> Filter
      -> Table "HORSE" Access By ID
        -> Bitmap
          -> Index "FK_HORSE_FARMBORN" Range Scan (full match)
```

```
      COUNT
=====
      551333
```

```
Current memory = 554018384
Delta memory = 256
Max memory = 554990224
Elapsed time = 0.462 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 879313
Per table statistics:
```

Table name	Natural	Index	Insert	Update	Delete
FARM	39903				
HORSE		551333			

Vamos mudar a ordem da junção:

```
SELECT COUNT(*)
FROM HORSE
  JOIN FARM ON FARM.CODE_FARM = HORSE.CODE_FARM
PLAN JOIN (HORSE NATURAL, FARM INDEX (PK_FARM));
```

As estatísticas e o plano de execução da consulta serão os seguintes:

```
Select Expression
-> Aggregate
  -> Nested Loop Join (inner)
    -> Table "HORSE" Full Scan
    -> Filter
```

```
-> Table "FARM" Access By ID
    -> Bitmap
        -> Index "PK_FARM" Unique Scan
```

```
COUNT
=====
551333
```

```
Current memory = 554066352
Delta memory = 256
Max memory = 554990224
Elapsed time = 1.227 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 2867756
Per table statistics:
```

Table name	Natural	Index	Insert	Update	Delete
FARM		551333			
HORSE	551333				

A diferença é óbvia.

A otimização de junções é um tópico muito complexo e será abordado posteriormente em um capítulo separado. Aqui, nos concentraremos em como usar os índices na junção da maneira mais eficaz.

Uso de Múltiplos Índices

Quando um índice é usado repetidamente em uma condição de junção, sua eficiência não é tão boa, mas a situação se torna ainda pior se múltiplos índices forem usados. O otimizador é inteligente o suficiente e pode descartar índices “excessivos”. Normalmente, o índice com baixa seletividade é descartado, mas se as seletividades são aproximadamente da mesma ordem, ambos os índices serão usados. Vamos ver isso em exemplos.



Para evitar que o otimizador altere a ordem de junção das tabelas e garantir que use o algoritmo de junção Nested Loop, usaremos LEFT JOIN nos exemplos. Atualmente, o Firebird não sabe usar outros algoritmos para junções externas.

Esta consulta não tem sentido e é apresentada apenas para demonstrar técnicas de otimização.

Vejamos a seguinte consulta:

```
SELECT
COUNT(*)
FROM
HORSE H
LEFT JOIN COVER C ON C.CODE_FATHER = H.CODE_FATHER AND C.CODE_DEPARTURE = H.CODE_DEPARTURE
```

```
WHERE H.CODE_HORSE = 742363;
```

As estatísticas e o plano de execução da consulta serão os seguintes:

```
Select Expression
-> Aggregate
  -> Filter
    -> Nested Loop Join (outer)
      -> Filter
        -> Table "HORSE" as "H" Access By ID
          -> Bitmap
            -> Index "PK_HORSE" Unique Scan
        -> Filter
          -> Table "COVER" as "C" Access By ID
            -> Bitmap
              -> Index "FK_COVER_FATHER" Range Scan (full match)
```

COUNT

```
=====
385
```

Current memory = 552489808

Delta memory = 288

Max memory = 592753104

Elapsed time = 0.002 sec

Buffers = 32768

Reads = 0

Writes = 0

Fetches = 561

Per table statistics:

Table name	Natural	Index	Insert	Update	Delete
COVER		385			
HORSE		1			

Para a tabela COVER existem as seguintes restrições e, portanto, os índices correspondentes:

```
-- selectivity: 0,0588235296309
ALTER TABLE COVER ADD CONSTRAINT FK_COVER_DEPARTURE
FOREIGN KEY (CODE_DEPARTURE) REFERENCES DEPARTURE (CODE_DEPARTURE);

-- selectivity: 0,0000145034737
ALTER TABLE COVER ADD CONSTRAINT FK_COVER_FATHER
FOREIGN KEY (CODE_FATHER) REFERENCES HORSE (CODE_HORSE);

-- selectivity: 0,000005519556
ALTER TABLE COVER ADD CONSTRAINT FK_COVER_MOTHER
FOREIGN KEY (CODE_MOTHER) REFERENCES HORSE (CODE_HORSE);
```

Obviamente, o otimizador decidiu não usar o índice FK_COVER_DEPARTURE para filtrar os registros da

tabela COVER, pois ele não é seletivo o suficiente e levaria a varreduras desnecessárias de páginas de índice. Lembre-se: neste caso, cada índice é varrido múltiplas vezes, e a varredura sempre ocorre a partir da raiz.

Agora vamos tentar usar duas colunas cuja seletividade dos índices será aproximadamente a mesma.

```
SELECT
  COUNT(*)
FROM
  HORSE H
  LEFT JOIN COVER C ON C.CODE_FATHER = H.CODE_FATHER AND C.CODE_MOTHER = H.CODE_MOTHER
WHERE H.CODE_HORSE = 742363;
```

As estatísticas e o plano de execução serão:

```
Select Expression
-> Aggregate
  -> Filter
    -> Nested Loop Join (outer)
      -> Filter
        -> Table "HORSE" as "H" Access By ID
          -> Bitmap
            -> Index "PK_HORSE" Unique Scan
      -> Filter
        -> Table "COVER" as "C" Access By ID
          -> Bitmap And
            -> Bitmap
              -> Index "FK_COVER_MOTHER" Range Scan (full match)
            -> Bitmap
              -> Index "FK_COVER_FATHER" Range Scan (full match)
```

```
COUNT
=====
7
```

```
Current memory = 552433200
Delta memory = 288
Max memory = 592753104
Elapsed time = 0.002 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 17
Per table statistics:
```

Table name	Natural	Index	Insert	Update	Delete
COVER		7			
HORSE		1			

Neste caso, o otimizador decidiu usar ambos os índices, pois sua seletividade é boa.

Agora vamos tentar executar a mesma consulta com o índice FK_COVER_MOTHER desabilitado.

```
SELECT
  COUNT(*)
FROM
  HORSE H
  LEFT JOIN COVER C ON C.CODE_FATHER = H.CODE_FATHER AND C.CODE_MOTHER+0 = H.CODE_MOTHER
WHERE H.CODE_HORSE = 742363;
```

As estatísticas e o plano de execução serão:

```
Select Expression
-> Aggregate
  -> Filter
    -> Nested Loop Join (outer)
      -> Filter
        -> Table "HORSE" as "H" Access By ID
          -> Bitmap
            -> Index "PK_HORSE" Unique Scan
        -> Filter
          -> Table "COVER" as "C" Access By ID
            -> Bitmap
              -> Index "FK_COVER_FATHER" Range Scan (full match)
```

COUNT

```
=====
7
```

Current memory = 552461648

Delta memory = 288

Max memory = 592753104

Elapsed time = 0.002 sec

Buffers = 32768

Reads = 0

Writes = 0

Fetches = 561

Per table statistics:

Table name	Natural	Index	Insert	Update	Delete
COVER		385			
HORSE		1			

Como você pode ver, o otimizador não errou.

Uso de Índices Compostos

É possível melhorar de alguma forma o desempenho da consulta do exemplo acima? Sim, podemos usar um índice composto para dois campos de uma vez. Vamos tentar adicionar tal índice.


```
CREATE INDEX IDX_COVER_FM ON COVER(CODE_FATHER, CODE_MOTHER);
```

Vamos tentar executar esta consulta novamente:

```
SELECT
  COUNT(*)
FROM
  HORSE H
  LEFT JOIN COVER C ON C.CODE_FATHER = H.CODE_FATHER AND C.CODE_MOTHER = H.CODE_MOTHER
WHERE H.CODE_HORSE = 742363;
```

As estatísticas e o plano de execução serão:

```
Select Expression
  -> Aggregate
    -> Filter
      -> Nested Loop Join (outer)
        -> Filter
          -> Table "HORSE" as "H" Access By ID
            -> Bitmap
              -> Index "PK_HORSE" Unique Scan
          -> Filter
            -> Table "COVER" as "C" Access By ID
              -> Bitmap
                -> Index "IDX_COVER_FM" Range Scan (full match)
```

```

COUNT
=====
      7
```

```
Current memory = 552093840
Delta memory = 288
Max memory = 592753104
Elapsed time = 0.002 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 15
Per table statistics:
```

Table name	Natural	Index	Insert	Update	Delete
COVER		7			
HORSE		1			

As estatísticas ficaram um pouco melhores (2 fetches). Pode parecer pouco, mas vamos tentar modificar ligeiramente nossa consulta para que a tabela principal retorne mais de um registro.

```
SELECT
```

```

COUNT(*)
FROM
  HORSE H
  LEFT JOIN COVER C ON C.CODE_FATHER = H.CODE_FATHER AND C.CODE_MOTHER = H.CODE_MOTHER
WHERE H.CODE_DEPARTURE = 6;

```

Vamos comparar o plano e as estatísticas de execução.

Variante com dois índices:

```

Select Expression
  -> Aggregate
    -> Filter
      -> Nested Loop Join (outer)
        -> Filter
          -> Table "HORSE" as "H" Access By ID
            -> Bitmap
              -> Index "FK_HORSE_DEPARTURE" Range Scan (full match)
          -> Filter
            -> Table "COVER" as "C" Access By ID
              -> Bitmap And
                -> Bitmap
                  -> Index "FK_COVER_MOTHER" Range Scan (full match)
                -> Bitmap
                  -> Index "FK_COVER_FATHER" Range Scan (full match)

```

```

COUNT
=====
25035

```

```

Current memory = 552562048
Delta memory = 288
Max memory = 592753104
Elapsed time = 3.482 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 146963
Per table statistics:

```

Table name	Natural	Index	Insert	Update	Delete
COVER		24226			
HORSE		12734			

Variante com índice composto:

```

Select Expression
  -> Aggregate
    -> Filter
      -> Nested Loop Join (outer)
        -> Filter

```

```

-> Table "HORSE" as "H" Access By ID
  -> Bitmap
    -> Index "FK_HORSE_DEPARTURE" Range Scan (full match)
-> Filter
  -> Table "COVER" as "C" Access By ID
    -> Bitmap
      -> Index "IDX_COVER_FM" Range Scan (full match)

```

```

COUNT
=====
25035

```

```

Current memory = 552500576
Delta memory = 288
Max memory = 593344592
Elapsed time = 0.054 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 98805
Per table statistics:

```

Table name	Natural	Index	Insert	Update	Delete
COVER		24226			
HORSE		12734			

A diferença é bastante notável. Por que isso aconteceu?

Como mencionado anteriormente, ao executar uma junção pelo método Nested Loop, em cada iteração do loop interno **cada** índice usado para a tabela subordinada é “lido” da raiz até as páginas folha, depois as páginas folha de cada índice, então uma fusão das máscaras de bits é realizada, e só depois disso os registros da tabela subordinada são lidos. Neste caso, é óbvio que

```
depth(IDX_COVER_FM) < depth(FK_COVER_MOTHER) + depth(FK_COVER_FATHER)
```

Com um grande número de registros na tabela externa, os custos de percorrer os índices da raiz às folhas tornam-se mais perceptíveis.

Para maior clareza, apresento as estatísticas para esses índices, obtidas com gstat:

```

Index FK_COVER_FATHER (12)
  Root page: 178397, depth: 2, leaf buckets: 262, nodes: 755297
  Average node length: 5.58, total dup: 686348, max dup: 77960
  Average key length: 2.65, compression ratio: 3.40
  Average prefix length: 8.45, average data length: 0.55
  Clustering factor: 388196, ratio: 0.51
  Fill distribution:
    0 - 19% = 0
    20 - 39% = 0
    40 - 59% = 1

```

```
60 - 79% = 0
80 - 99% = 261
```

Index FK_COVER_MOTHER (14)

```
Root page: 179111, depth: 2, leaf buckets: 311, nodes: 755297
Average node length: 6.62, total dup: 574123, max dup: 13373
Average key length: 3.68, compression ratio: 2.44
Average prefix length: 7.56, average data length: 1.44
Clustering factor: 464766, ratio: 0.62
Fill distribution:
  0 - 19% = 1
 20 - 39% = 0
 40 - 59% = 0
 60 - 79% = 0
 80 - 99% = 310
```

Index IDX_COVER_FM (16)

```
Root page: 180289, depth: 3, leaf buckets: 597, nodes: 755297
Average node length: 12.62, total dup: 259782, max dup: 844
Average key length: 9.68, compression ratio: 2.79
Average prefix length: 19.97, average data length: 7.03
Clustering factor: 598071, ratio: 0.79
Fill distribution:
  0 - 19% = 0
 20 - 39% = 0
 40 - 59% = 0
 60 - 79% = 0
 80 - 99% = 597
```

Vamos calcular os custos de percorrer apenas as páginas de índice em cada iteração:

```
index_cost = depth - 1 + leaf_pages * index_selectivity

cost(FK_COVER_FATHER) = 2 - 1 + MAXVALUE(262 * 0.0000145034737, 1) = 2

cost(FK_COVER_MOTHER) = 2 - 1 + MAXVALUE(311 * 0.000005519556, 1) = 2

cost(FK_COVER_FATHER) + cost(FK_COVER_MOTHER) = 2 + 2 = 4

cost(IDX_COVER_FM) = 3 - 1 + MAXVALUE(597 * 0.0000020181024, 1) = 3
```

Obviamente, usar o índice composto acabou sendo mais barato.

7.4.3. Uso de estatísticas de índice pelo algoritmo de junção Hash

Vamos ver como a presença de índices para colunas na condição de junção pode afetar a junção pelo método de hash. Neste caso, consideraremos apenas junções internas, pois para junções externas o algoritmo de junção por hash não está implementado na versão atual do Firebird.

Para o exemplo, tomemos a seguinte consulta:

```
SELECT COUNT(*)
```

```
FROM
  HORSE
  JOIN FARM ON FARM.CODE_FARM = HORSE.CODE_FARM
```

Para as tabelas HORSE e FARM existem as seguintes restrições e, portanto, índices:

```
-- selectivity = 6.753562774974853e-05
ALTER TABLE HORSE ADD CONSTRAINT FK_HORSE_FARMBORN FOREIGN KEY (CODE_FARM) REFERENCES FARM
(CODE_FARM);

-- selectivity = 2.506077180441935e-05
ALTER TABLE FARM ADD CONSTRAINT PK_FARM PRIMARY KEY (CODE_FARM);
```

O otimizador escolheu o seguinte plano de execução para ela:

```
Select Expression
-> Aggregate
  -> Filter
    -> Hash Join (inner)
      -> Table "HORSE" Full Scan
      -> Record Buffer (record length: 33)
        -> Table "FARM" Full Scan
```

Neste plano não é visível a estimativa de cardinalidades dos fluxos sendo unidos, portanto, para demonstração, usarei a função `RDB$SQL.EXPLAIN` do Firebird 6.0, que atualmente está em desenvolvimento. Os próprios algoritmos de otimização para junções Hash não mudaram nela. Vou complementar o plano obtido com as estimativas de cardinalidade.

```
Select Expression
[cardinality = 1.0000000000000000]
-> Aggregate
  [cardinality = 593344.0378878072]
  -> Filter
    [cardinality = 593344.0378878072]
    -> Hash Join (inner) (keys: 1, total key length: 8)
      [cardinality = 556528.0281690141]
      -> Table "HORSE" Full Scan
      [cardinality = 42542.70491803279]
      -> Record Buffer (record length: 33)
        [cardinality = 42542.70491803279]
        -> Table "FARM" Full Scan
```

A cardinalidade da junção das tabelas HORSE e FARM foi estimada pelo otimizador como 593344, enquanto na realidade `COUNT(*)` retorna 551333, o que, embora não seja totalmente preciso, é suficientemente próximo.

Vamos tentar entender como esse número foi obtido.

```
cardinality(join) = cardinality(HORSE) * cardinality(FARM) * selectivity(join_cond)
```

```
selectivity(join_cond) = selectivity(PK_FARM)
```

```
cardinality(join) = 556528.0281690141 * (42542.70491803279 * 2.506077180441935e-05) =  
593344,037887807353
```

Agora, vamos tentar privar o otimizador das informações sobre a estatística dos índices. Para isso, não é necessário excluir os próprios índices, basta adicionar `+0` em ambos os lados do operador `=` na condição de junção.



Se escrevermos apenas `FARM.CODE_FARM + 0`, o otimizador escolherá um algoritmo de junção diferente e alterará a ordem da junção.

```
SELECT COUNT(*)  
FROM  
  HORSE  
  JOIN FARM ON FARM.CODE_FARM + 0 = HORSE.CODE_FARM + 0
```

Vejamos o plano da consulta — ele não mudou.

```
Select Expression  
  -> Aggregate  
    -> Filter  
      -> Hash Join (inner)  
        -> Table "HORSE" Full Scan  
        -> Record Buffer (record length: 33)  
          -> Table "FARM" Full Scan
```

Agora, vamos exibir este plano com as estimativas de cardinalidade.

```
Select Expression  
  [cardinality = 1.0000000000000000]  
  -> Aggregate  
    [cardinality = 23676207.68100901]  
    -> Filter  
      [cardinality = 23676207.68100901]  
      -> Hash Join (inner) (keys: 1, total key length: 8)  
        [cardinality = 556528.0281690141]  
        -> Table "HORSE" Full Scan  
        [cardinality = 42542.70491803279]  
        -> Record Buffer (record length: 33)  
          [cardinality = 42542.70491803279]  
          -> Table "FARM" Full Scan
```

A cardinalidade da junção das tabelas `HORSE` e `FARM` foi estimada pelo otimizador como 23676208, enquanto na realidade `COUNT(*)` retorna 551333. Um erro de duas ordens de magnitude. Por que isso aconteceu? Tudo se deve ao fato de que, na ausência de informações sobre a seletividade do índice, a seletividade da condição de junção é assumida como a seletividade do operador `=`, que é

uma constante e igual a 0.001 (em condições de junção; em condições de filtragem de fluxo — 0.1). Vejamos o cálculo da cardinalidade da junção.

```
cardinality(join) = cardinality(HORSE) * cardinality(FARM) * selectivity(join_cond)

selectivity(join_cond) = selectivity(=)

cardinality(join) = 556528.02817 * (42542.70492 * 0.001) = 23676207.681009
```

Pode parecer, qual a diferença, o otimizador errou a estimativa de cardinalidade, e tudo bem. Neste exemplo, realmente não importa, mas basta aumentar o número de tabelas sendo unidas, e a estimativa incorreta de cardinalidade pode pregar uma peça em nós.

Vejamos no exemplo da seguinte consulta:

```
SELECT COUNT(*)
FROM
  HORSE FATHER
  JOIN HORSE H ON H.CODE_FATHER = FATHER.CODE_HORSE
  JOIN HORSE MOTHER ON MOTHER.CODE_HORSE = H.CODE_MOTHER
  JOIN FARM ON FARM.CODE_FARM = FATHER.CODE_FARM
  JOIN COLOR ON COLOR.CODE_COLOR = FATHER.CODE_COLOR
```

O plano e a estatística de execução serão os seguintes:

```
Select Expression
-> Aggregate
  -> Filter
    -> Hash Join (inner)
      -> Hash Join (inner)
        -> Hash Join (inner)
          -> Hash Join (inner)
            -> Table "HORSE" as "H" Full Scan
            -> Record Buffer (record length: 49)
              -> Table "HORSE" as "FATHER" Full Scan
              -> Record Buffer (record length: 25)
                -> Table "COLOR" Full Scan
                -> Record Buffer (record length: 33)
                  -> Table "FARM" Full Scan
                  -> Record Buffer (record length: 33)
                    -> Table "HORSE" as "MOTHER" Full Scan

COUNT
=====
551333

Current memory = 551922672
Delta memory = 0
Max memory = 617494144
Elapsed time = 1.762 sec
Buffers = 32768
```

```
Reads = 0
Writes = 0
Fetches = 1815176
```

Agora, vamos adicionar +0 a uma das condições de junção.

```
SELECT COUNT(*)
FROM
  HORSE FATHER
  JOIN HORSE H ON H.CODE_FATHER+0 = FATHER.CODE_HORSE+0
  JOIN HORSE MOTHER ON MOTHER.CODE_HORSE = H.CODE_MOTHER
  JOIN FARM ON FARM.CODE_FARM = FATHER.CODE_FARM
  JOIN COLOR ON COLOR.CODE_COLOR = FATHER.CODE_COLOR
```

Vamos comparar a estatística de execução e o plano:

```
Select Expression
  -> Aggregate
    -> Filter
      -> Hash Join (inner)
        -> Filter
          -> Hash Join (inner)
            -> Nested Loop Join (inner)
              -> Table "COLOR" Full Scan
              -> Filter
                -> Table "HORSE" as "FATHER" Access By ID
                -> Bitmap
                  -> Index "FK_HORSE_COLOR" Range Scan (full match)
              -> Record Buffer (record length: 33)
              -> Table "FARM" Full Scan
            -> Record Buffer (record length: 66)
          -> Filter
            -> Hash Join (inner)
              -> Table "HORSE" as "H" Full Scan
              -> Record Buffer (record length: 33)
              -> Table "HORSE" as "MOTHER" Full Scan

COUNT
=====
551333

Current memory = 551971760
Delta memory = 384
Max memory = 625476992
Elapsed time = 1.924 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 1876193
```

Como podemos ver, devido à estimativa incorreta da cardinalidade da junção H e FATHER, o plano mudou completamente e as estatísticas de execução ficaram um pouco piores. Daí a conclusão —

índices nas colunas das tabelas que participam da condição de sua junção desempenham um papel importante, mesmo que não sejam usados diretamente.



Se o Firebird tivesse estatísticas armazenadas sobre as colunas das tabelas, então os índices para junções **Hash** não teriam um papel tão importante. Essa estatística está presente em alguns SGBDs, como o Oracle. Não é impossível que algum dia ela esteja disponível no Firebird também.

7.5. Usando um índice para ordenação

Um índice pode ser usado para ordenar uma seleção. Lembre-se de que os seguintes algoritmos de ordenação estão implementados:

- Ordenação externa (SORT no plano Legacy. Veja [Ordenação](#));
 - Ordenação externa com a otimização Refetch (veja [Refetch](#));
- Navegação por índice (ORDER no plano Legacy. Veja [Navegação por índice](#)).

Nesta seção, discutiremos quando a navegação por índice pode ser usada (ordenação usando um índice). A otimização de ordenações será discutida separadamente no capítulo correspondente.

7.5.1. Quando um índice pode ser usado para ordenação?

A navegação por índice nada mais é do que uma varredura sequencial das chaves do índice e a leitura dos registros da tabela na ordem dessas chaves. Para que um índice possa ser usado para navegação, as seguintes condições devem ser atendidas na cláusula ORDER BY:

- Todas as colunas especificadas no ORDER BY devem corresponder total ou parcialmente (para um dos segmentos) às colunas da chave do índice. Ou os números no ORDER BY devem se referir a essas colunas;
- A direção da ordenação para todas as colunas especificadas no ORDER BY deve ser a mesma e corresponder à direção da ordenação das chaves do índice;
- A expressão especificada no ORDER BY deve corresponder à expressão no índice calculado. Além disso, a direção da ordenação especificada no ORDER BY e na chave do índice deve coincidir;
- Em consultas usando JOIN, a navegação por índice só é possível se o otimizador escolher a tabela líder para a qual um índice para navegação foi criado;
- A navegação usando vários índices não é possível.

7.5.2. Navegação (ordenação) por um índice simples

Vejamos um exemplo em que um índice simples pode ser usado para ordenação:

```
CREATE INDEX FARM_IDX_NAME ON FARM (NAME);

SELECT CODE_FARM, NAME
FROM FARM
```

```
ORDER BY NAME;
```

Plano Explain:

```
Select Expression
-> Table "FARM" Access By ID
   -> Index "FARM_IDX_NAME" Full Scan
```

Plano Legacy:

```
PLAN (FARM ORDER FARM_IDX_NAME)
```



Ao contrário do plano Legacy, onde a palavra ORDER está explicitamente presente, seguida pelo nome do índice usado para ordenação, no plano Explain a ordenação não é óbvia. Na verdade, o plano Explain reflete com mais precisão o que está acontecendo, ou seja, a extração de dados da tabela FARM na ordem das chaves do índice FARM_IDX_NAME. A navegação por índice no plano Explain é facilmente detectada pela ausência da construção de uma máscara de bits (palavra Bitmap) entre Table "<table_name>" Access By ID e Index <index_name>

O índice também será usado se o ORDER BY especificar o número da coluna na cláusula SELECT para a qual o índice foi criado.

```
SELECT CODE_FARM, NAME
FROM FARM
ORDER BY 2;
```

Plano Explain:

```
Select Expression
-> Table "FARM" Access By ID
   -> Index "FARM_IDX_NAME" Full Scan
```

Plano Legacy:

```
PLAN (FARM ORDER FARM_IDX_NAME)
```

No entanto, se alterarmos a direção da ordenação na cláusula ORDER BY, em vez da navegação por índice, será usada a ordenação externa.

```
SELECT CODE_FARM, NAME
FROM FARM
ORDER BY NAME DESC;
```

```
SELECT CODE_FARM, NAME
FROM FARM
ORDER BY 2 DESC;
```

Plano Explain:

```
Select Expression
-> Sort (record length: 1044, key length: 604)
-> Table "FARM" Full Scan
```

Plano Legacy:

```
PLAN SORT (FARM NATURAL)
```

A razão é que na versão atual do Firebird, apenas a navegação unidirecional por índice é suportada, ou seja, a navegação por índice na direção das chaves do índice. As razões para a unidirecionalidade da navegação por índice são descritas em [Acesso por índice](#).

Se você deseja que o índice seja usado para ordenação na direção inversa, é necessário criar um índice DESCENDING correspondente. Vamos tentar fazer isso.

```
CREATE DESCENDING INDEX FARM_IDX_NAME_DESC ON FARM (NAME);

SELECT CODE_FARM, NAME
FROM FARM
ORDER BY NAME DESC;

SELECT CODE_FARM, NAME
FROM FARM
ORDER BY 2 DESC;
```

Plano Explain:

```
Select Expression
-> Table "FARM" Access By ID
-> Index "FARM_IDX_NAME_DESC" Full Scan
```

Plano Legacy:

```
PLAN (FARM ORDER FARM_IDX_NAME_DESC)
```

7.5.3. Navegação (ordenação) usando um índice por expressão

Assim como na filtragem, um índice por expressão pode ser usado para ordenação. Para usar um índice por expressão para ordenação, uma das seguintes condições deve ser atendida:

- A expressão para ordenação na cláusula ORDER BY deve corresponder exatamente à expressão no índice calculado;
- A expressão na cláusula SELECT, referenciada por número na cláusula ORDER BY, deve corresponder exatamente à expressão no índice calculado.

Em ambos os casos, a direção da ordenação definida na cláusula ORDER BY deve corresponder à direção das chaves do índice por expressão.

Suponha que temos uma tabela INVOICE e um índice no campo INVOICE_DATE do tipo TIMESTAMP, definido da seguinte forma:

```
CREATE INDEX INVOICE_IDX_DATE ON INVOICE (INVOICE_DATE);
```

Podemos usar o índice para ordenar pela coluna INVOICE_DATE:

```
SELECT
  INVOICE_ID,
  INVOICE_DATE,
  TOTAL_SALE
FROM INVOICE
ORDER BY INVOICE_DATE;
```

Plano Explain:

```
Select Expression
  -> Table "INVOICE" Access By ID
      -> Index "INVOICE_IDX_DATE" Full Scan
```

Plano Legacy:

```
PLAN (INVOICE ORDER INVOICE_IDX_DATE)
```

Agora vamos modificar a consulta: não precisamos da data completa, apenas do ano, e queremos ordenar por ano.

```
SELECT
  INVOICE_ID,
  EXTRACT(YEAR FROM INVOICE_DATE) AS INVOICE_YEAR,
  TOTAL_SALE
FROM INVOICE
ORDER BY EXTRACT(YEAR FROM INVOICE_DATE);
```

```
-- OU
```

```
SELECT
  INVOICE_ID,
  EXTRACT(YEAR FROM INVOICE_DATE) AS INVOICE_YEAR,
```

```
TOTAL_SALE
FROM INVOICE
ORDER BY 2;
```

Plano Explain:

```
Select Expression
  -> Sort (record length: 52, key length: 8)
      -> Table "INVOICE" Full Scan
```

Plano Legacy:

```
PLAN SORT (INVOICE NATURAL)
```

Obviamente, neste caso, o índice INVOICE_IDX_DATE não pode ser usado para ordenação. Para ordenar por ano usando um índice, é necessário criar um índice por expressão:

```
CREATE INDEX INVOICE_IDX_BY_YEAR ON INVOICE COMPUTED BY (EXTRACT(YEAR FROM INVOICE_DATE));
```

Agora, para as consultas acima, o plano será:

```
Select Expression
  -> Table "INVOICE" Access By ID
      -> Index "INVOICE_IDX_BY_YEAR" Full Scan
```

Plano Legacy:

```
PLAN (INVOICE ORDER INVOICE_IDX_BY_YEAR)
```

Se for necessário ordenar na ordem inversa, é necessário criar um índice DESCENDING.

```
CREATE DESC INDEX INVOICE_IDX_BY_YEAR_DESC ON INVOICE COMPUTED BY (EXTRACT(YEAR FROM
INVOICE_DATE));
```

```
SELECT
  INVOICE_ID,
  EXTRACT(YEAR FROM INVOICE_DATE) AS INVOICE_YEAR,
  TOTAL_SALE
FROM INVOICE
ORDER BY EXTRACT(YEAR FROM INVOICE_DATE) DESC;
```

```
-- OU
```

```
SELECT
  INVOICE_ID,
  EXTRACT(YEAR FROM INVOICE_DATE) AS INVOICE_YEAR,
```

```
TOTAL_SALE
FROM INVOICE
ORDER BY 2 DESC;
```

Plano Explain:

```
Select Expression
-> Table "INVOICE" Access By ID
   -> Index "INVOICE_IDX_BY_YEAR_DESC" Full Scan
```

Plano Legacy:

```
PLAN (INVOICE ORDER INVOICE_IDX_BY_YEAR_DESC)
```

Também é possível criar uma coluna na consulta que seja uma expressão sobre várias colunas e ordenar por ela.

```
SELECT
  EMP_NO,
  LAST_NAME || ' ' || FIRST_NAME AS FULLNAME
FROM
  EMPLOYEE
ORDER BY 2;
```

Plano Explain:

```
Select Expression
-> Sort (record length: 108, key length: 40)
   -> Table "EMPLOYEE" Full Scan
```

Plano Legacy:

```
PLAN SORT (EMPLOYEE NATURAL)
```

Para que esta consulta possa ordenar os dados usando um índice, você pode criar o seguinte índice calculado:

```
CREATE INDEX IDX_EMPLOYEE_FULLNAME ON EMPLOYEE COMPUTED BY (LAST_NAME || ' ' || FIRST_NAME);
```

Agora o otimizador usará o índice:

```
Select Expression
-> Table "EMPLOYEE" Access By ID
   -> Index "IDX_EMPLOYEE_FULLNAME" Full Scan
```

Plano Legacy:

```
PLAN (EMPLOYEE ORDER IDX_EMPLOYEE_FULLNAME)
```

E, finalmente, você pode realizar a ordenação por uma coluna calculada definida na tabela. Por exemplo, no banco de dados employee para a tabela EMPLOYEE, há uma coluna calculada FULL_NAME, definida como:

```
CREATE TABLE EMPLOYEE (
    ...
    FULL_NAME    COMPUTED BY (last_name || ', ' || first_name)
);
```

Neste banco de dados, não há índices criados para esta coluna, portanto, será usada a ordenação externa.

```
SELECT
    EMP_NO,
    FULL_NAME
FROM
    EMPLOYEE
ORDER BY 2;
```

Plano Explain:

```
Select Expression
  -> Sort (record length: 118, key length: 44)
      -> Table "EMPLOYEE" Full Scan
```

Plano Legacy:

```
PLAN SORT (EMPLOYEE NATURAL)
```

Você não pode criar um índice simples em uma coluna calculada. Por exemplo, a seguinte consulta DDL falhará:

```
CREATE INDEX IDX_EMPLOYEE_FULL_NAME ON EMPLOYEE (FULL_NAME);
```

```
Statement failed, SQLSTATE = 42000
unsuccessful metadata update
-CREATE INDEX IDX_EMPLOYEE_FULL_NAME failed
-attempt to index COMPUTED BY column in INDEX IDX_EMPLOYEE_FULL_NAME
```

No entanto, é possível criar um índice calculado em uma coluna calculada, que pode ser usado:

```
CREATE INDEX IDX_EMPLOYEE_FULL_NAME ON EMPLOYEE COMPUTED BY(FULL_NAME);
```

Agora a consulta acima terá o seguinte plano:

```
Select Expression
-> Table "EMPLOYEE" Access By ID
   -> Index "IDX_EMPLOYEE_FULL_NAME" Full Scan
```

Plano Legacy:

```
PLAN (EMPLOYEE ORDER IDX_EMPLOYEE_FULL_NAME)
```

7.5.4. Desativando a navegação por índice

A navegação por índice nem sempre é mais barata que a ordenação externa, portanto, às vezes é necessário desativá-la.

Qualquer expressão aplicada a um campo de índice ou a uma expressão de índice, mas que não altere sua semântica, desativa o uso do índice para ordenação. Isso pode ser usado como dicas para o otimizador de consultas.

Normalmente, para desativar o uso do índice, são usadas duas expressões dependendo do tipo:

- para tipos numéricos e datas, usa-se a expressão `+0`;
- para tipos de string, usa-se a expressão `|| ''`.

Se `ORDER BY` especificar diretamente as colunas do índice ou a expressão, essas dicas são usadas diretamente na cláusula `ORDER BY`. Se `ORDER BY` usar referências numéricas à coluna na cláusula `SELECT`, as dicas devem ser usadas em `SELECT`.

Vamos dar alguns exemplos de desativação da navegação por índice:

```
-- NAME tem o tipo VARCHAR(100)
SELECT CODE_FARM, NAME
FROM FARM
ORDER BY NAME || '';
```

```
-- o mesmo, mas em ORDER BY uma referência numérica à coluna
-- neste caso, é necessário usar um alias de coluna para
-- preservar seu nome no resultado de saída
SELECT CODE_FARM, NAME || '' AS NAME
FROM FARM
ORDER BY 2;
```

```
-- BYDATE tem o tipo DATE
SELECT
  CODE_TRIAL,
  BYDATE
```



```

FROM TRIAL
ORDER BY BYDATE + 0;

-- o mesmo, mas em ORDER BY uma referência numérica à coluna
-- neste caso, é necessário usar um alias de coluna para
-- preservar seu nome no resultado de saída
SELECT
  CODE_TRIAL,
  BYDATE + 0 AS BYDATE
FROM TRIAL
ORDER BY 2;

```

7.5.5. Navegação (ordenação) por índice composto

Até agora, ordenamos os dados por uma coluna ou expressão, mas e se for necessário ordenar por várias colunas? Como mencionado acima, a ordenação usando vários índices não é suportada. Além disso, não é possível usar conjuntamente a ordenação externa e a navegação por índice. Mas é possível realizar navegação (ordenação) por um índice composto.

Como ao criar um índice composto não podemos especificar uma direção de ordenação individual para cada coluna, mas apenas para o índice como um todo, é impossível usar a navegação por índice composto se para parte das colunas for especificada ordenação ASC, e para outra parte DESC.

Assim, a navegação por índice composto é possível se

- Na cláusula ORDER BY forem especificadas todas as colunas do índice composto na mesma ordem em que foram especificadas na definição do índice (ou referência a elas);
- Na cláusula ORDER BY forem especificadas parte das colunas do índice composto, formando um segmento do índice (ou referência a elas);
- Todas as colunas em ORDER BY ou referências a elas forem ordenadas na mesma direção em que as chaves do índice composto estão ordenadas.



Se for usada filtragem por parte da chave do índice composto, há uma nuance que será considerada mais tarde.

Vamos considerar esses casos usando o banco de dados employee como exemplo. Nele, para a tabela EMPLOYEE foi criado o seguinte índice composto:

```
CREATE INDEX NAMEX ON EMPLOYEE (LAST_NAME, FIRST_NAME);
```

O caso mais simples e óbvio é quando a cláusula ORDER BY especifica todas as colunas do índice na mesma ordem:

```

SELECT
  EMP_NO,
  LAST_NAME,
  FIRST_NAME
FROM

```

```

EMPLOYEE
ORDER BY LAST_NAME, FIRST_NAME;

-- OU
SELECT
  EMP_NO,
  LAST_NAME,
  FIRST_NAME
FROM
  EMPLOYEE
ORDER BY 2, 3;

```

Plano Explain:

```

Select Expression
  -> Table "EMPLOYEE" Access By ID
    -> Index "NAMEX" Full Scan

```

Plano Legacy:

```
PLAN (EMPLOYEE ORDER NAMEX)
```

Se você tentar trocar as colunas de lugar na cláusula ORDER BY, o índice NAMEX não poderá ser usado para navegação:

```

SELECT
  EMP_NO,
  LAST_NAME,
  FIRST_NAME
FROM
  EMPLOYEE
ORDER BY FIRST_NAME, LAST_NAME;

-- OU
SELECT
  EMP_NO,
  LAST_NAME,
  FIRST_NAME
FROM
  EMPLOYEE
ORDER BY 3, 2;

```

Plano Explain:

```

Select Expression
  -> Sort (record length: 80, key length: 52)
    -> Table "EMPLOYEE" Full Scan

```

Plano Legacy:

```
PLAN SORT (EMPLOYEE NATURAL)
```

O índice também não será usado se uma das colunas tiver ordenação DESC.

```
SELECT
  EMP_NO,
  LAST_NAME,
  FIRST_NAME
FROM
  EMPLOYEE
ORDER BY LAST_NAME, FIRST_NAME DESC;

-- OU
SELECT
  EMP_NO,
  LAST_NAME,
  FIRST_NAME
FROM
  EMPLOYEE
ORDER BY LAST_NAME DESC, FIRST_NAME;
```

Você pode usar um índice DESC para ordenação DESC em ambas as colunas. Ou seja, se você criar um índice assim:

```
CREATE DESCENDING INDEX NAMEX_DESC ON EMPLOYEE (LAST_NAME, FIRST_NAME);
```

Ele poderá ser usado para consultas:

```
SELECT
  EMP_NO,
  LAST_NAME,
  FIRST_NAME
FROM
  EMPLOYEE
ORDER BY LAST_NAME DESC, FIRST_NAME DESC;

-- OU
SELECT
  EMP_NO,
  LAST_NAME,
  FIRST_NAME
FROM
  EMPLOYEE
ORDER BY 2 DESC, 3 DESC;
```

Plano Explain:

```
Select Expression
-> Table "EMPLOYEE" Access By ID
```

```
-> Index "NAMEX_DESC" Full Scan
```

Plano Legacy:

```
PLAN (EMPLOYEE ORDER NAMEX_DESC)
```

O otimizador pode ordenar pela parte inicial da chave se a cláusula ORDER BY especificar parte das colunas do índice composto, formando um segmento do índice (ou referência a elas).

```
SELECT
  EMP_NO,
  LAST_NAME,
  FIRST_NAME
FROM
  EMPLOYEE
ORDER BY LAST_NAME;

-- OU
SELECT
  EMP_NO,
  LAST_NAME,
  FIRST_NAME
FROM
  EMPLOYEE
ORDER BY 2;
```

Plano Explain:

```
Select Expression
  -> Table "EMPLOYEE" Access By ID
    -> Index "NAMEX" Full Scan
```

Plano Legacy:

```
PLAN (EMPLOYEE ORDER NAMEX)
```

No entanto, o índice não pode ser usado para navegação se ORDER BY especificar colunas que não formam um segmento do índice, por exemplo FIRST_NAME.



Lembre-se, neste caso o índice tem dois segmentos:

- LAST_NAME
- LAST_NAME, FIRST_NAME

```
SELECT
  EMP_NO,
```

```

    LAST_NAME,
    FIRST_NAME
FROM
    EMPLOYEE
ORDER BY FIRST_NAME;

-- OU
SELECT
    EMP_NO,
    LAST_NAME,
    FIRST_NAME
FROM
    EMPLOYEE
ORDER BY 3;
```

Plano Explain:

```

Select Expression
  -> Sort (record length: 78, key length: 24)
      -> Table "EMPLOYEE" Full Scan
```

Plano Legacy:

```
PLAN SORT (EMPLOYEE NATURAL)
```

7.5.6. Filtragem durante navegação (ordenação) por índice

Nos exemplos acima, todos os registros da tabela foram ordenados por índice, mas normalmente filtros são aplicados à tabela usando a cláusula WHERE. Nesse caso, surgem as seguintes variantes:

- Ocorre filtragem usando um predicado que não pode usar índice;
- Ocorre filtragem usando um predicado que pode usar índice;
 - A filtragem ocorre usando o índice pelo qual a navegação (ordenação) é realizada;
 - A filtragem ocorre usando outro índice

Vamos considerar cada um desses casos.

Predicado de filtragem não indexado

Vejamos a seguinte consulta:

```

SELECT NAME
FROM HORSE
WHERE FAMILY = ' '
ORDER BY NAME;
```

Para obter estatísticas de sua execução, é necessário extrair todos os registros do cursor, mas com

um grande número de registros isso é muito custoso. Portanto, vamos transformar esta consulta na seguinte:

```
SELECT COUNT(*)
FROM (
  SELECT NAME
  FROM HORSE
  WHERE FAMILY = 'H'
  ORDER BY NAME
);
```

Obviamente, a própria ordenação ou navegação por índice aqui é desnecessária, mas o otimizador do Firebird não entende isso e não a remove, ordenando honestamente. Assim, apenas um registro é retornado ao cliente, mas as estatísticas de execução incluem os custos de ordenação.

Para o campo NAME existe um índice, o que permite realizar navegação por índice, e o campo FAMILY não é indexado.

```
CREATE INDEX HORSE_IDX_NAME ON HORSE (NAME);
```

O plano e as estatísticas de execução desta consulta serão:

```
Select Expression
  -> Aggregate
    -> Filter
      -> Table "HORSE" Access By ID
        -> Index "HORSE_IDX_NAME" Full Scan

PLAN (HORSE ORDER HORSE_IDX_NAME)

          COUNT
=====
              3

Current memory = 555476960
Delta memory = 813984
Max memory = 555494080
Elapsed time = 0.870 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 1616407
```

O plano de execução desta consulta não é ideal. Para entender isso, basta executar:

```
SELECT COUNT(*)
FROM HORSE;
```

```

COUNT
=====
551333

```

A consulta neste exemplo é executada da seguinte forma: todos os registros da tabela HORSE são extraídos na ordem das chaves do índice e só então o filtro `FAMILY = ''` é aplicado a eles. A navegação por índice com extração de todos os registros é uma operação bastante cara. O número de fetches (acessos ao cache de páginas) pode ser calculado aproximadamente pela fórmula:

```
fetches = count(index leaf pages) + (2 + Clustering_ratio) * table_cardinality
```

O filtro aplicado possui seletividade suficientemente alta, e apenas 3 registros são retornados na saída. Neste caso, seria muito mais barato fazer uma varredura completa da tabela HORSE, filtrar os registros nela de acordo com o predicado fornecido e, em seguida, ordenar os 3 registros restantes. Vamos tentar fazer isso, desativando a navegação por índice:

```

SELECT COUNT(*)
FROM (
  SELECT NAME
  FROM HORSE
  WHERE FAMILY = ''
  ORDER BY NAME || ''
);

```

```
PLAN SORT (HORSE NATURAL)
```

```

Select Expression
  -> Aggregate
    -> Sort (record length: 556, key length: 304)
      -> Filter
        -> Table "HORSE" Full Scan

```

```

COUNT
=====
3

```

```

Current memory = 554959040
Delta memory = 18176
Max memory = 557202048
Elapsed time = 0.219 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 589984

```

Como você pode ver, a consulta requer 3 vezes menos acessos ao cache de páginas e é executada 4 vezes mais rápido.

Mas a ordenação externa será sempre mais rápida que a navegação por índice? A resposta correta é não. Vamos tentar alterar o predicado de filtragem para um que retorne 90% dos registros:

```
SELECT COUNT(*)
FROM (
  SELECT NAME
  FROM HORSE
  WHERE FAMILY IS NULL
  ORDER BY NAME
);
```

```
PLAN (HORSE ORDER HORSE_IDX_NAME)
```

```
Select Expression
  -> Aggregate
    -> Filter
      -> Table "HORSE" Access By ID
        -> Index "HORSE_IDX_NAME" Full Scan
```

```

COUNT
=====
551006
```

```
Current memory = 556460704
Delta memory = 466016
Max memory = 940369648
Elapsed time = 0.878 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 1615402
```

E a variante com navegação por índice desativada:

```
SELECT COUNT(*)
FROM (
  SELECT NAME
  FROM HORSE
  WHERE FAMILY IS NULL
  ORDER BY NAME || ' '
);
```

```
PLAN SORT (HORSE NATURAL)
```

```
Select Expression
  -> Aggregate
    -> Sort (record length: 556, key length: 304)
      -> Filter
        -> Table "HORSE" Full Scan
```



```

COUNT
=====
551006

Current memory = 556497712
Delta memory = 224
Max memory = 941330912
Elapsed time = 1.156 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 589981

```

Como você pode ver, no segundo caso a navegação pelo índice se mostrou melhor.

Infelizmente, na versão atual do Firebird, o otimizador não sabe fazer uma avaliação de custo para escolher entre uma ordenação externa e a navegação por índice — se houver um índice adequado para a navegação por índice, ela será executada. Portanto, você terá que otimizar esses casos manualmente com base em suposições e experimentos.



No Firebird 6.0, a avaliação de custo para a escolha (SORT vs ORDER) finalmente apareceu. Mas ela é baseada na seletividade dos predicados de filtragem, especificados por constantes, o que nem sempre reflete corretamente a seletividade real.

Filtragem por outro índice

Acima, consideramos o caso em que a tabela é filtrada por um predicado não indexado. Agora vamos ver outro caso, em que a filtragem é realizada por um predicado que utiliza uma varredura de índice. Este caso pode ser dividido em dois:

- para a filtragem é usado outro índice;
- para a filtragem é usado o mesmo índice que para a navegação.

Vamos considerar um exemplo do primeiro caso:

```

SELECT NAME
FROM HORSE
WHERE CODE_DEPARTURE = 1
ORDER BY NAME;

```

A coluna CODE_DEPARTURE é uma chave estrangeira, e para ela existe o índice FK_HORSE_DEPARTURE. Vamos transformar esta consulta na seguinte:

```

SELECT COUNT(*)
FROM (
  SELECT NAME
  FROM HORSE
  WHERE CODE_DEPARTURE = 1

```

```
ORDER BY NAME
);
```

As estatísticas e o plano de sua execução serão os seguintes:

```
PLAN (HORSE ORDER HORSE_IDX_NAME INDEX (FK_HORSE_DEPARTURE))

Select Expression
  -> Aggregate
    -> Filter
      -> Table "HORSE" Access By ID
        -> Index "HORSE_IDX_NAME" Full Scan
          -> Bitmap
            -> Index "FK_HORSE_DEPARTURE" Range Scan (full match)

COUNT
=====
98853

Current memory = 553848416
Delta memory = 224
Max memory = 941330912
Elapsed time = 0.229 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 280730
```

Esta consulta é executada da seguinte forma:

- Para as chaves correspondentes do índice FK_HORSE_DEPARTURE, é construído um mapa de bits com os números dos registros;
- São varridas as chaves do índice HORSE_IDX_NAME, e para cada nó seu é verificado se o número do registro está presente no mapa de bits do índice FK_HORSE_DEPARTURE;
- Se o número do registro, correspondente ao nó do índice HORSE_IDX_NAME, estiver presente no mapa de bits, então o registro correspondente é lido da tabela HORSE.

Assim, a aplicação de um predicado indexado pode reduzir significativamente o número de acessos às páginas de dados da tabela, o que impacta positivamente o desempenho.

Claro, o predicado indexado pode não ser único, o que significa que vários índices podem ser usados para a filtragem, com união ou interseção dos mapas de bits.

```
SELECT COUNT(*)
FROM (
  SELECT NAME
  FROM HORSE
  WHERE CODE_DEPARTURE = 1
  AND CODE_BREED = 65
  ORDER BY NAME
```

```
);
```

```
PLAN (HORSE ORDER HORSE_IDX_NAME INDEX (FK_HORSE_BREED, FK_HORSE_DEPARTURE))

Select Expression
  -> Aggregate
    -> Filter
      -> Table "HORSE" Access By ID
        -> Index "HORSE_IDX_NAME" Full Scan
          -> Bitmap And
            -> Bitmap
              -> Index "FK_HORSE_BREED" Range Scan (full match)
            -> Bitmap
              -> Index "FK_HORSE_DEPARTURE" Range Scan (full match)

COUNT
=====
          98649

Current memory = 554519744
Delta memory = 240
Max memory = 941330912
Elapsed time = 0.241 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 280152
```

Filtragem pelo mesmo índice que é usado para navegação

Agora vamos considerar o caso em que a filtragem ocorre pelo mesmo índice pelo qual ocorre a navegação.

```
SELECT *
FROM INVOICE
WHERE INVOICE_DATE < date '2015-06-01'
ORDER BY INVOICE_DATE;
```

Para a coluna INVOICE_DATE foi criado o seguinte índice:

```
CREATE INDEX INVOICE_IDX_DATE ON INVOICE (INVOICE_DATE);
```

Vamos transformar a consulta na seguinte:

```
SELECT COUNT(*)
FROM (
  SELECT *
  FROM INVOICE
  WHERE INVOICE_DATE < date '2015-06-01'
```

```
ORDER BY INVOICE_DATE
);
```

O plano e as estatísticas de execução desta consulta serão os seguintes:

```
PLAN (INVOICE ORDER INVOICE_IDX_DATE)

Select Expression
  -> Aggregate
    -> Filter
      -> Table "INVOICE" Access By ID
        -> Index "INVOICE_IDX_DATE" Range Scan (upper bound: 1/1)

          COUNT
          =====
          68948

Current memory = 285127232
Delta memory = 240
Max memory = 285171808
Elapsed time = 0.032 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 138656
```

Neste caso, para o índice INVOICE_IDX_DATE existe um limite superior de varredura. Assim, o índice INVOICE_IDX_DATE não é lido por completo, mas apenas as suas chaves que estão dentro dos limites de varredura. E isso, por sua vez, significa que nem todos os registros da tabela serão extraídos, mas apenas aqueles que correspondem às chaves do índice dentro dos limites de varredura. Aqui vale notar que, diferentemente dos filtros por outros índices, nenhum mapa de bits é construído.

Se o intervalo de varredura for suficientemente estreito, isso pode acelerar significativamente o processo de ordenação.

A consulta acima pode ser modificada para que ambos os limites de varredura sejam usados.

```
SELECT COUNT(*)
FROM (
  SELECT *
  FROM INVOICE
  WHERE INVOICE_DATE between date '2015-06-01' and date '2015-06-30'
  ORDER BY INVOICE_DATE
);
```

O plano e as estatísticas de execução desta consulta serão os seguintes:

```
PLAN (INVOICE ORDER INVOICE_IDX_DATE)

Select Expression
```

```

-> Aggregate
  -> Filter
    -> Table "INVOICE" Access By ID
      -> Index "INVOICE_IDX_DATE" Range Scan (lower bound: 1/1, upper bound: 1/1)

COUNT
=====
13921

Current memory = 285186240
Delta memory = 288
Max memory = 285212912
Elapsed time = 0.008 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 27998

```

Vou dar mais um exemplo de filtragem pelo mesmo índice pelo qual ocorre a navegação:

```

SELECT NAME
FROM HORSE
WHERE NAME STARTING WITH 'A'
ORDER BY NAME;

```

Seu plano será o seguinte:

```

Select Expression
  -> Filter
    -> Table "HORSE" Access By ID
      -> Index "HORSE_IDX_NAME" Range Scan (full match)

```

Neste caso, a navegação será realizada apenas pelas chaves do índice HORSE_IDX_NAME para as quais o nome do cavalo começa com a letra “A”.

Filtragem por segmento do mesmo índice composto que é usado para navegação

Acima, consideramos o caso em que o predicado de filtro usa o mesmo índice que é usado para navegação (ordenação). Todos esses exemplos usaram índices simples em uma coluna. Tudo o que foi descrito acima também se aplica a índices compostos ao filtrar pela chave completa, ou seja, por todos os segmentos do índice. Ao filtrar por correspondência parcial em segmentos individuais, há nuances. Vamos analisá-las.

Para a tabela EMPLOYEE existe um índice composto NAMEX, definido da seguinte forma:

```

CREATE INDEX NAMEX ON EMPLOYEE (LAST_NAME, FIRST_NAME);

```

Vamos tentar executar a seguinte consulta:

```

SELECT
  EMP_NO,
  LAST_NAME,
  FIRST_NAME
FROM
  EMPLOYEE
WHERE LAST_NAME = 'Baldwin'
ORDER BY LAST_NAME, FIRST_NAME;

```

O plano e as estatísticas de execução serão os seguintes:

```

Select Expression
  -> Filter
    -> Table "EMPLOYEE" Access By ID
      -> Index "NAMEX" Range Scan (partial match: 1/2)

```

EMP_NO	LAST_NAME	FIRST_NAME
34	Baldwin	Janet
177	Baldwin	Ryan

```

Current memory = 284318640
Delta memory = 0
Max memory = 284395760
Elapsed time = 0.002 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 6

```

Aqui tudo é conforme esperado. A navegação será apenas pelas chaves para as quais há correspondência parcial em um dos segmentos do índice NAMEX.

Foi dito anteriormente que a navegação por este índice composto sem condições de filtragem adicionais pode ser realizada se na cláusula ORDER BY for especificado:

- LAST_NAME
- ou LAST_NAME, FIRST_NAME

E agora vamos tentar executar a seguinte consulta:

```

SELECT
  EMP_NO,
  LAST_NAME,
  FIRST_NAME
FROM
  EMPLOYEE
WHERE LAST_NAME = 'Baldwin'
ORDER BY FIRST_NAME;

```

Vejamos o plano e as estatísticas:

```
Select Expression
-> Filter
    -> Table "EMPLOYEE" Access By ID
        -> Index "NAMEX" Range Scan (partial match: 1/2)
```

EMP_NO	LAST_NAME	FIRST_NAME
34	Baldwin	Janet
177	Baldwin	Ryan

```
Current memory = 284498560
Delta memory = 224
Max memory = 284526576
Elapsed time = 0.002 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 6
```

Aqui novamente é usada a navegação pelo índice NAMEX. Isso se tornou possível porque na condição de filtragem foi especificada uma correspondência completa pelo primeiro segmento do índice. Se tentarmos alterar esta consulta para que para LAST_NAME seja usado um predicado diferente, o resultado será outro:

```
SELECT
  EMP_NO,
  LAST_NAME,
  FIRST_NAME
FROM
  EMPLOYEE
WHERE LAST_NAME > 'Baldwin'
ORDER BY FIRST_NAME;
```

```
Select Expression
-> Sort (record length: 78, key length: 24)
    -> Filter
        -> Table "EMPLOYEE" Access By ID
            -> Bitmap
                -> Index "NAMEX" Range Scan (lower bound: 1/2)
```

```
Current memory = 284513152
Delta memory = 224
Max memory = 284541008
Elapsed time = 0.042 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 102
```

Como você pode ver, nesta consulta, em vez da navegação por índice, é usada uma ordenação externa. Isso aconteceu porque pelo primeiro segmento ocorre apenas uma correspondência parcial, o que não permite usar a navegação pela parte restante da chave. Esta é a nuance sobre a qual eu queria falar.

A regra geral é a seguinte: da cláusula `ORDER BY` pode-se remover a parte inicial da chave do índice se houver uma condição de filtragem sobre ela que leve a uma correspondência completa por este segmento do índice, ou seja, para predicados (`=`, `IS NULL`, `IS NOT DISTINCT FROM`).

7.5.7. Limitação do número de linhas

A navegação por índice, diferentemente da ordenação externa, é uma operação em pipeline. Uma fonte de dados em pipeline emite registros durante a leitura de seus fluxos de entrada e pode ser interrompida a qualquer momento usando os contadores `FIRST/ROWS/FETCH`. Vamos ver como isso funciona e como afeta o desempenho.

Antes de realizar experimentos no `isql`, vamos configurar:

```
OUTPUT nul;
SET STAT ON;
SET PER_TAB ON;
SET EXPLAIN ON;
```

Agora vamos executar a seguinte consulta:

```
SELECT CODE_HORSE, NAME
FROM HORSE
ORDER BY NAME;
```

O plano e as estatísticas serão os seguintes:

```
Select Expression
  -> Table "HORSE" Access By ID
      -> Index "HORSE_IDX_NAME" Full Scan
```

```
Current memory = 553765600
Delta memory = 176
Max memory = 553963664
Elapsed time = 1.821 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 1615365
```

Table name	Natural	Index	Insert	Update	Delete
HORSE		551333			

E agora vamos tentar extrair apenas o primeiro registro.

```
SELECT CODE_HORSE, NAME
FROM HORSE
ORDER BY NAME
FETCH FIRST ROW ONLY;
```

```
Select Expression
  -> First N Records
      -> Table "HORSE" Access By ID
          -> Index "HORSE_IDX_NAME" Full Scan
Current memory = 553809984
Delta memory = 192
Max memory = 553963664
Elapsed time = 0.001 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 4
Per table statistics:
```

Table name	Natural	Index	Insert	Update	Delete
HORSE		1			

A obtenção do primeiro registro ocorreu praticamente instantaneamente.

Esta consulta funcionou da seguinte forma:

1. as chaves do índice HORSE_IDX_NAME são varridas sequencialmente;
2. para cada chave, o registro correspondente da tabela HORSE é extraído;
3. a visibilidade do registro é verificada; se o registro estiver visível, passamos para a próxima etapa, caso contrário, voltamos para a etapa 1;
4. os contadores FIRST/ROWS/FETCH são verificados. Se a quantidade necessária de registros foi extraída, a execução da consulta é interrompida. Caso contrário, passamos para a etapa 1.

Claro, podemos retornar não um, mas vários registros.

```
SELECT CODE_HORSE, NAME
FROM HORSE
ORDER BY NAME
FETCH FIRST 10 ROWS ONLY;
```

```
Select Expression
  -> First N Records
      -> Table "HORSE" Access By ID
          -> Index "HORSE_IDX_NAME" Full Scan
```

```

Current memory = 554191888
Delta memory = 192
Max memory = 809685200
Elapsed time = 0.002 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 22
Per table statistics:

```

Table name	Natural	Index	Insert	Update	Delete
HORSE		10			

Quanto mais registros forem especificados no limitador FETCH FIRST, maiores serão os custos de navegação.

Observe que, se em vez da navegação por índice for usada uma ordenação externa, tal consulta será executada por um tempo considerável, porque a ordenação externa é uma fonte de dados em buffer, ou seja, para obter o primeiro registro na saída, a ordenação externa primeiro precisa ler todos os registros das fontes de dados de entrada. Vamos demonstrar isso:

```

SELECT CODE_HORSE, NAME
FROM HORSE
ORDER BY NAME || ' '
FETCH FIRST ROW ONLY;

```

```

Select Expression
  -> First N Records
    -> Refetch
      -> Sort (record length: 324, key length: 304)
        -> Table "HORSE" Full Scan

```

```

Current memory = 553825088
Delta memory = 192
Max memory = 809685200
Elapsed time = 0.888 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 589983
Per table statistics:

```

Table name	Natural	Index	Insert	Update	Delete
HORSE	551333				

Pular N registros

Além de limitar a quantidade de registros de saída, podemos pular uma certa quantidade de

registros usando SKIP/ROWS/OFFSET.

```
SELECT CODE_HORSE, NAME
FROM HORSE
ORDER BY NAME
OFFSET 10 ROWS;
```

```
Select Expression
  -> Skip N Records
      -> Table "HORSE" Access By ID
          -> Index "HORSE_IDX_NAME" Full Scan
```

Current memory = 554655184

Delta memory = 192

Max memory = 809685200

Elapsed time = 1.803 sec

Buffers = 32768

Reads = 0

Writes = 0

Fetches = 1615365

Per table statistics:

Table name	Natural	Index	Insert	Update	Delete
HORSE		551333			

Obviamente, pular N registros não afeta a quantidade de leituras indexadas. O fato é que pular registros funciona da seguinte forma:

1. as chaves do índice HORSE_IDX_NAME são varridas sequencialmente;
2. para cada chave, o registro correspondente da tabela HORSE é extraído;
3. a visibilidade do registro é verificada; se o registro estiver visível, passamos para a próxima etapa, caso contrário, voltamos para a etapa 1;
4. os contadores SKIP/ROWS/OFFSET são verificados. Se um número insuficiente de registros foi pulado, simplesmente passamos para a etapa 1. Se mais registros foram pulados do que especificado em OFFSET, então retornamos o registro extraído e passamos para a etapa 1.

Assim, independentemente do que for especificado em SKIP/OFFSET, todas as chaves do índice HORSE_IDX_NAME e os registros da tabela HORSE serão lidos.

Claro, podemos aplicar FETCH *n* e OFFSET *m* em conjunto:

```
SELECT CODE_HORSE, NAME
FROM HORSE
ORDER BY NAME
OFFSET 10 ROWS
FETCH FIRST 10 ROWS ONLY;
```

```

Select Expression
  -> First N Records
    -> Skip N Records
      -> Table "HORSE" Access By ID
        -> Index "HORSE_IDX_NAME" Full Scan

```

Current memory = 554671936

Delta memory = 208

Max memory = 809685200

Elapsed time = 0.001 sec

Buffers = 32768

Reads = 0

Writes = 0

Fetches = 42

Per table statistics:

Table name	Natural	Index	Insert	Update	Delete
HORSE		20			

Como pode ser visto nos resultados, da tabela HORSE serão lidos no máximo $n + m$ registros usando a navegação por índice.

Muito frequentemente, FIRST/SKIP e OFFSET/FETCH são aplicados para organizar a navegação paginada. Neste caso, n é uma constante, e $m = n * (\text{página} - 1)$. Este método funciona de forma eficiente apenas para as primeiras páginas. Quanto maior for o número da página, mais lenta será a extração dos registros.

Contadores e Filtros

É óbvio que se um filtro utiliza um índice, então essa combinação funciona de forma bastante eficiente.

Um exemplo prático de uso dessa combinação é obter um determinado evento na primeira/última data.

Consideremos o seguinte exemplo. Na tabela REGISTRATION são armazenados alguns eventos relacionados à vida de um cavalo. Um desses eventos é a mudança de proprietário (R.CODE_REGTYPE = 6). É necessário obter o último proprietário do cavalo e a data da mudança de proprietário. Para isso, usaremos a seguinte consulta:

```

SELECT
  R.CODE_REGISTRATION,
  R.CODE_REASON,
  R.CODE_FARM,
  R.BYDATE
FROM REGISTRATION R
WHERE R.CODE_HORSE = 742363
  AND R.CODE_REGTYPE = 6
ORDER BY R.BYDATE DESC
FETCH FIRST ROW ONLY;

```

Para esta tabela, existem os seguintes índices:

```
ALTER TABLE REGISTRATION ADD CONSTRAINT FK_REGISTRATION_HORSE
FOREIGN KEY (CODE_HORSE) REFERENCES HORSE (CODE_HORSE);

ALTER TABLE REGISTRATION ADD CONSTRAINT FK_REGISTRATION_REGTYPE
FOREIGN KEY (CODE_REGTYPE) REFERENCES REGTYPE (CODE_REGTYPE);

CREATE INDEX REGISTRATION_IDX_BYDATE ON REGISTRATION (BYDATE);
```

Vejamos o plano e as estatísticas de execução desta consulta:

```
Select Expression
  -> First N Records
      -> Refetch
          -> Sort (record length: 28, key length: 8)
              -> Filter
                  -> Table "REGISTRATION" as "R" Access By ID
                      -> Bitmap
                          -> Index "FK_REGISTRATION_HORSE" Range Scan (full match)

Current memory = 553368160
Delta memory = 320
Max memory = 809685200
Elapsed time = 0.001 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 7
Per table statistics:
```

Table name	Natural	Index	Insert	Update	Delete
REGISTRATION		3			

Nada mal, mas está sendo usada uma ordenação externa. Vamos tentar criar um índice DESC, para que a navegação pelo índice seja utilizada.

```
CREATE DESCENDING INDEX REGISTRATION_IDX_BYDATE_DESC ON REGISTRATION (BYDATE);
```

Agora o plano e as estatísticas são os seguintes:

```
Select Expression
  -> First N Records
      -> Filter
          -> Table "REGISTRATION" as "R" Access By ID
              -> Index "REGISTRATION_IDX_BYDATE_DESC" Full Scan
                  -> Bitmap
                      -> Index "FK_REGISTRATION_HORSE" Range Scan (full match)

Current memory = 553499488
```

```
Delta memory = 320
Max memory = 809685200
Elapsed time = 0.015 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 258
Per table statistics:
```

Table name	Natural	Index	Insert	Update	Delete
REGISTRATION		1			

É óbvio que piorou. Em primeiro lugar, o índice FK_REGISTRATION_REGTYPE não é utilizado pelo otimizador (com ele seria ainda pior).

Vamos tentar criar um índice composto:

```
CREATE INDEX REGISTRATION_IDX_HORSE_REGTYPE ON REGISTRATION (CODE_HORSE, CODE_REGTYPE);
```

Agora o plano e as estatísticas são os seguintes:

```
Select Expression
  -> First N Records
    -> Filter
      -> Table "REGISTRATION" as "R" Access By ID
        -> Index "REGISTRATION_IDX_BYDATE_DESC" Full Scan
          -> Bitmap
            -> Index "REGISTRATION_IDX_HORSE_REGTYPE" Range Scan (full match)
```

```
Current memory = 553557200
Delta memory = 320
Max memory = 809685200
Elapsed time = 0.013 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 259
Per table statistics:
```

Table name	Natural	Index	Insert	Update	Delete
REGISTRATION		1			

O tempo e a quantidade de acessos ao cache de páginas ainda estão piores do que na versão inicial. Vamos tentar criar um índice composto para que seja usada apenas a navegação pelo índice (vamos remover os índices extras):

```
DROP INDEX REGISTRATION_IDX_BYDATE_DESC;
```

```
CREATE DESCENDING INDEX IDX_REGISTRATION_OWNER ON REGISTRATION (CODE_HORSE, CODE_REGTYPE,
BYDATE);
```

Vamos executar novamente a consulta inicial e ver as estatísticas:

```
Select Expression
-> First N Records
  -> Filter
    -> Table "REGISTRATION" as "R" Access By ID
      -> Index "IDX_REGISTRATION_OWNER" Range Scan (partial match: 2/3)
```

Current memory = 553967712

Delta memory = 320

Max memory = 809685200

Elapsed time = 0.001 sec

Buffers = 32768

Reads = 0

Writes = 0

Fetches = 5

Per table statistics:

Table name	Natural	Index	Insert	Update	Delete
REGISTRATION		1			

Conseguimos alcançar o desejado. Mas veja, o ganho é totalmente insignificante. Valeria a pena? Se for executar exatamente esta consulta, provavelmente não. Mas e se esta consulta (ou subconsulta) for executada repetidamente?

Suponhamos que agora queremos obter o último proprietário e a data da mudança de proprietário para cada cavalo.

```
SELECT
  H.CODE_HORSE,
  H.NAME AS HORSE_NAME,
  OWN.BYDATE AS OWNER_DATE,
  FARMOWNER.NAME AS OWNER_NAME
FROM
  HORSE H
  LEFT JOIN LATERAL (
    SELECT
      R.CODE_FARM,
      R.BYDATE
    FROM REGISTRATION R
    WHERE R.CODE_HORSE = H.CODE_HORSE
      AND R.CODE_REGTYPE = 6
    ORDER BY R.BYDATE DESC
    FETCH FIRST ROW ONLY
  ) OWN ON TRUE
  LEFT JOIN FARM FARMOWNER ON FARMOWNER.CODE_FARM = OWN.CODE_FARM
WHERE H.CODE_BREED = 65;
```

Primeiro, vamos ver as estatísticas e o plano de execução com o índice `IDX_REGISTRATION_OWNER` desativado. Para desativar o índice, execute:

```
ALTER INDEX IDX_REGISTRATION_OWNER INACTIVE;
```

O plano e as estatísticas da consulta com o índice `IDX_REGISTRATION_OWNER` desativado serão os seguintes:

```
Select Expression
-> Filter
    -> Nested Loop Join (outer)
        -> Filter
            -> Nested Loop Join (outer)
                -> Filter
                    -> Table "HORSE" as "H" Access By ID
                        -> Bitmap
                            -> Index "FK_HORSE_BREED" Range Scan (full match)
                -> First N Records
                    -> Refetch
                        -> Sort (record length: 28, key length: 8)
                            -> Filter
                                -> Table "REGISTRATION" as "OWN R" Access By ID
                                    -> Bitmap
                                        -> Index "REGISTRATION_IDX_HORSE_REGTYPE" Range
Scan (full match)
    -> Filter
        -> Table "FARM" as "FARMOWNER" Access By ID
            -> Bitmap
                -> Index "PK_FARM" Unique Scan

Current memory = 554277104
Delta memory = 560
Max memory = 809685200
Elapsed time = 1.175 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 1420857
Per table statistics:
```

Table name	Natural	Index	Insert	Update	Delete
FARM		120387			
HORSE		120387			
REGISTRATION		156561			

Agora vamos tornar nosso índice ativo:

```
ALTER INDEX IDX_REGISTRATION_OWNER ACTIVE;
```

E executamos nossa consulta novamente:


```

Select Expression
  -> Filter
    -> Nested Loop Join (outer)
      -> Filter
        -> Nested Loop Join (outer)
          -> Filter
            -> Table "HORSE" as "H" Access By ID
              -> Bitmap
                -> Index "FK_HORSE_BREED" Range Scan (full match)
            -> First N Records
              -> Filter
                -> Filter
                  -> Table "REGISTRATION" as "OWN R" Access By ID
                    -> Index "IDX_REGISTRATION_OWNER" Range Scan (partial
match: 2/3)
              -> Filter
                -> Table "FARM" as "FARMOWNER" Access By ID
                  -> Bitmap
                    -> Index "PK_FARM" Unique Scan

Current memory = 554261776
Delta memory = 560
Max memory = 809685200
Elapsed time = 1.075 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 1257886
Per table statistics:
-----+-----+-----+-----+-----+
Table name      | Natural | Index  | Insert  | Update  | Delete  |
-----+-----+-----+-----+-----+
FARM             |         | 120387 |         |         |         |
HORSE            |         | 120387 |         |         |         |
REGISTRATION     |         | 120387 |         |         |         |
-----+-----+-----+-----+-----+

```

O ganho tornou-se mais perceptível, embora não seja enorme.

7.5.8. Navegação (ordenação) por índice em consultas com múltiplas tabelas

Até agora, consideramos principalmente a aplicação da navegação por índice em consultas simples com uma única tabela (com algumas exceções), porém, na realidade, consultas com junção (JOIN) de várias tabelas são mais comuns. Para tais consultas, a navegação por índice é usada com muito menos frequência. A razão é que a navegação por índice só pode ser aplicada se a tabela escolhida como fluxo principal for aquela cujos campos indexados precisam ser usados para ordenação.

Para consultas com uma única tabela, é simples — se existe uma possibilidade potencial de usar navegação por índice, então ela é usada. Neste caso, o otimizador não faz uma escolha baseada em custo entre planos SORT (ordenação externa) e ORDER (navegação por índice), mas segue regras (usa uma abordagem baseada em regras, não em custo).

No entanto, quando a consulta usa uma junção (JOIN) de várias tabelas, a avaliação de custo é

ativada para escolher o algoritmo e a ordem correta de junção, e somente quando a ordem de junção é determinada, decide-se usar navegação por índice (se tivermos sorte e nossa tabela for a principal) ou ordenação externa.

Claro, você pode usar apenas `LEFT JOIN` em sua consulta, garantindo a ordem de junção e, portanto, a possibilidade de usar navegação por índice, mas isso nem sempre é possível ou ideal.

Vamos examinar a seguinte consulta:

```
SELECT
  H.CODE_HORSE,
  H.NAME AS HORSE_NAME,
  COLOR.NAME AS COLOR_NAME
FROM
  HORSE H
  JOIN COLOR ON COLOR.CODE_COLOR = H.CODE_COLOR
ORDER BY H.NAME;
```

Vejamos seu plano:

```
PLAN SORT (JOIN (COLOR NATURAL, H INDEX (FK_HORSE_COLOR)))

Select Expression
  -> Sort (record length: 772, key length: 304)
    -> Nested Loop Join (inner)
      -> Table "COLOR" Full Scan
      -> Filter
        -> Table "HORSE" as "H" Access By ID
          -> Bitmap
            -> Index "FK_HORSE_COLOR" Range Scan (full match)
```

O otimizador escolheu a tabela `COLOR` como principal e, portanto, usar navegação por índice para a coluna `NAME` da tabela `HORSE` é impossível, e a ordenação externa foi escolhida.

Se você decidir mudar a ordenação para `COLOR.NAME`, a navegação por índice será escolhida.

```
SELECT
  H.CODE_HORSE,
  H.NAME AS HORSE_NAME,
  COLOR.NAME AS COLOR_NAME
FROM
  HORSE H
  JOIN COLOR ON COLOR.CODE_COLOR = H.CODE_COLOR
ORDER BY COLOR.NAME;
```

```
PLAN JOIN (COLOR ORDER UNQ_COLOR_NAME, H INDEX (FK_HORSE_COLOR))

Select Expression
  -> Nested Loop Join (inner)
    -> Table "COLOR" Access By ID
```

```

-> Index "UNQ_COLOR_NAME" Full Scan
-> Filter
  -> Table "HORSE" as "H" Access By ID
    -> Bitmap
      -> Index "FK_HORSE_COLOR" Range Scan (full match)

```

Mas e se quisermos usar ordenação por H.NAME e usar navegação por índice? Em princípio, poderíamos fixar nossa ordem de junção usando LEFT JOIN e um filtro adicional NOT NULL:

```

SELECT
  H.CODE_HORSE,
  H.NAME AS HORSE_NAME,
  COLOR.NAME AS COLOR_NAME
FROM
  HORSE H
  LEFT JOIN COLOR ON COLOR.CODE_COLOR = H.CODE_COLOR
WHERE COLOR.CODE_COLOR IS NOT NULL
ORDER BY H.NAME;

```

```

PLAN JOIN (H ORDER HORSE_IDX_NAME, COLOR INDEX (PK_COLOR))

```

```

Select Expression
  -> Filter
    -> Nested Loop Join (outer)
      -> Table "HORSE" as "H" Access By ID
        -> Index "HORSE_IDX_NAME" Full Scan
      -> Filter
        -> Table "COLOR" Access By ID
          -> Bitmap
            -> Index "PK_COLOR" Unique Scan

```

Conseguimos alcançar o desejado. Noto que, em alguns casos, tais transformações são bastante difíceis ou mesmo impossíveis de fazer.

Ok, conseguimos a navegação por índice, mas valeu a pena? Vamos comparar o desempenho da consulta original e da transformada.

Estatísticas de execução sem navegação por índice:

```

Select Expression
  -> Sort (record length: 772, key length: 304)
    -> Nested Loop Join (inner)
      -> Table "COLOR" Full Scan
      -> Filter
        -> Table "HORSE" as "H" Access By ID
          -> Bitmap
            -> Index "FK_HORSE_COLOR" Range Scan (full match)
Current memory = 554449536
Delta memory = 288
Max memory = 1025357904
Elapsed time = 3.435 sec

```

```

Buffers = 32768
Reads = 0
Writes = 0
Fetches = 651249
Per table statistics:

```

Table name	Natural	Index	Insert	Update	Delete
COLOR	241				
HORSE		551333			

E agora, as estatísticas da consulta na qual alteramos a ordem de junção e forçamos o uso de navegação por índice:

```

Select Expression
  -> Filter
    -> Nested Loop Join (outer)
      -> Table "HORSE" as "H" Access By ID
        -> Index "HORSE_IDX_NAME" Full Scan
      -> Filter
        -> Table "COLOR" Access By ID
          -> Bitmap
            -> Index "PK_COLOR" Unique Scan

```

```

Current memory = 554918176
Delta memory = 320
Max memory = 1025357904
Elapsed time = 2.961 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 3269364
Per table statistics:

```

Table name	Natural	Index	Insert	Update	Delete
COLOR		551333			
HORSE		551333			

Surpreendentemente, a navegação por índice foi mais rápida, embora os acessos ao cache de páginas (Fetches) tenham sido 5 vezes maiores. Em geral, alterar a ordem de junção pode levar a uma piora significativa no tempo de execução da consulta.

Limitação no número de registros

Nos exemplos acima, vimos que usar navegação por índice em consultas com junções nem sempre é simples. Às vezes, isso exige alterar a ordem de junção das tabelas, o que pode piorar significativamente a velocidade de execução da consulta. Mas e se extrairmos não todos os dados, mas apenas uma parte deles? Vamos verificar isso.

Execute a seguinte consulta:

```

SELECT
  H.CODE_HORSE,
  H.NAME AS HORSE_NAME,
  COLOR.NAME AS COLOR_NAME
FROM
  HORSE H
  JOIN COLOR ON COLOR.CODE_COLOR = H.CODE_COLOR
ORDER BY H.NAME
FETCH FIRST 10 ROWS ONLY;

```

O plano e as estatísticas de execução desta consulta serão os seguintes:

```

Select Expression
  -> First N Records
    -> Nested Loop Join (inner)
      -> Table "HORSE" as "H" Access By ID
        -> Index "HORSE_IDX_NAME" Full Scan
      -> Filter
        -> Table "COLOR" Access By ID
          -> Bitmap
            -> Index "PK_COLOR" Unique Scan

```

Current memory = 554954144

Delta memory = 320

Max memory = 1025826544

Elapsed time = 0.002 sec

Buffers = 32768

Reads = 0

Writes = 0

Fetches = 52

Per table statistics:

Table name	Natural	Index	Insert	Update	Delete
COLOR		10			
HORSE		10			

Em primeiro lugar, esta consulta é executada muito rapidamente. Em segundo lugar, para que a navegação por índice fosse usada, não foi necessário fazer nada com a ordem de junção das tabelas (ela mudou por si só). Mas por que isso aconteceu? O fato é que a presença de limitadores FIRST/ROWS/FETCH na consulta altera o objetivo da otimização. Por padrão, o Firebird otimiza consultas para obter a extração mais rápida de todos os registros do cursor (estratégia ALL ROWS). Os limitadores FIRST/ROWS/FETCH mudam a estratégia de otimização para uma em que o otimizador constrói um plano para a extração mais rápida dos primeiros registros do cursor (estratégia FIRST ROWS).

Ao usar a estratégia FIRST ROWS, o otimizador tenta evitar métodos de acesso em buffer, como ordenação externa ou junção HASH. Assim, a ordem de junção é escolhida de forma que a navegação por índice seja selecionada em vez da ordenação externa.

Você nem sempre pode saber exatamente quantos dos primeiros registros serão necessários. Por

exemplo, o aplicativo é escrito de forma que os cursores não leem todos os dados de uma vez, lê um pouco mais do que pode ser exibido na grade de dados, e o restante dos dados é carregado conforme a grade é rolada. Nesses casos, é desejável exibir rapidamente a parte visível dos dados, mas você não sabe exatamente quantos, e uma limitação explícita na consulta mataria a possibilidade de carregar dados conforme a rolagem da grade. Neste caso, em vez dos limitadores FIRST/ROWS/FETCH, você pode simplesmente especificar a estratégia de otimização na própria consulta.

```
SELECT
  H.CODE_HORSE,
  H.NAME AS HORSE_NAME,
  COLOR.NAME AS COLOR_NAME
FROM
  HORSE H
  JOIN COLOR ON COLOR.CODE_COLOR = H.CODE_COLOR
ORDER BY H.NAME
OPTIMIZE FOR FIRST ROWS;
```

Plano e estatísticas:

```
Select Expression
-> Nested Loop Join (inner)
  -> Table "HORSE" as "H" Access By ID
    -> Index "HORSE_IDX_NAME" Full Scan
  -> Filter
    -> Table "COLOR" Access By ID
      -> Bitmap
        -> Index "PK_COLOR" Unique Scan
Current memory = 555123808
Delta memory = 320
Max memory = 1025979008
Elapsed time = 0.008 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 11745
Per table statistics:
```

Table name	Natural	Index	Insert	Update	Delete
COLOR		2000			
HORSE		2000			



As estatísticas da última consulta foram obtidas com SET MAXROWS 10 configurado. Isso significa que após a leitura de 10 registros do cursor, a leitura é interrompida. Isso foi feito para aproximar o comportamento das grades de dados (embora em grades de dados normalmente sejam buscados cerca de 200 registros, dos quais 100 são exibidos).

7.5.9. Custo de navegação (ordenação) por índice

Nas seções anteriores, falamos sobre como e quando a ordenação (navegação) por índice pode ser aplicada. Agora é hora de entender qual é o custo de usar a navegação por índice. Em geral, ele pode ser expresso pela seguinte fórmula:

$$\text{cost(fetches)} = \text{base_cost} + (2 + \text{clustering_ratio}) * \text{table_cardinality} * \text{selectivity}$$

aqui

- `base_cost` — o custo de verificar as páginas de índice usadas para filtragem e navegação;
- `clustering_ratio` — o grau de clusterização;
- `selectivity` — a seletividade dos predicados de filtragem que usam índices.

Mas esta fórmula reflete apenas de forma muito aproximada as leituras lógicas do cache de páginas. Em todos os exemplos acima, frequentemente obtivemos bons resultados ao usar a navegação por índice. Mas isso só aconteceu porque o tamanho do cache de páginas era suficiente para conter todas as páginas de dados das tabelas e as páginas de índice. Mas o que acontecerá se o cache de páginas for limitado, por exemplo, se usarmos a arquitetura clássica (Classic Server), onde um grande cache de páginas não é recomendado?

Vamos executar a mesma consulta com diferentes tamanhos de cache de páginas. Lembre-se de que as estatísticas são fornecidas para a segunda execução da consulta, ou seja, com o cache de páginas aquecido.

```
SELECT
  H.CODE_HORSE,
  H.NAME
FROM
  HORSE H
ORDER BY H.NAME;
```

O plano e as estatísticas de execução desta consulta (DefaultDbCachePages = 32K):

```
Select Expression
  -> Table "HORSE" as "H" Access By ID
      -> Index "HORSE_IDX_NAME" Full Scan
Current memory = 552109696
Delta memory = 192
Max memory = 552126816
Elapsed time = 1.865 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 1615365
```

O plano e as estatísticas de execução desta consulta (DefaultDbCachePages = 2K):

```

Select Expression
  -> Table "HORSE" as "H" Access By ID
    -> Index "HORSE_IDX_NAME" Full Scan
Current memory = 36998880
Delta memory = 192
Max memory = 37016000
Elapsed time = 5.172 sec
Buffers = 2048
Reads = 268639
Writes = 0
Fetches = 1615365

```

No segundo caso, o desempenho caiu drasticamente (3 vezes).

E agora vamos tentar usar a ordenação externa.

```

SELECT
  H.CODE_HORSE,
  H.NAME
FROM
  HORSE H
ORDER BY H.NAME || ' ';

```

O plano e as estatísticas de execução desta consulta (DefaultDbCachePages = 32K):

```

Select Expression
  -> Sort (record length: 548, key length: 304)
    -> Table "HORSE" as "H" Full Scan
Current memory = 551731296
Delta memory = 192
Max memory = 932370192
Elapsed time = 2.842 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 589981

```

O plano e as estatísticas de execução desta consulta (DefaultDbCachePages = 2K):

```

Select Expression
  -> Sort (record length: 548, key length: 304)
    -> Table "HORSE" as "H" Full Scan
Current memory = 36550800
Delta memory = 192
Max memory = 417189696
Elapsed time = 2.923 sec
Buffers = 2048
Reads = 9665
Writes = 0
Fetches = 589981

```


Ao usar a ordenação externa, o tempo de execução da consulta com cache de páginas grande e pequeno é praticamente o mesmo. Além disso, para um cache de páginas pequeno (DefaultDbCachePages = 2K), a ordenação externa foi quase duas vezes mais rápida.

Então, por que a navegação por índice com um cache de páginas pequeno é tão lenta? Aqui, deve-se prestar atenção ao número Reads nas estatísticas. Ele reflete a quantidade de “leituras físicas do disco”.



Para ser mais preciso, Reads reflete a quantidade de leituras de E/S usando funções do sistema. Leituras físicas reais podem não ocorrer, pois na maioria dos casos são compensadas pelo cache do sistema (de arquivos). A leitura usando funções do sistema é uma ordem de magnitude mais lenta do que a leitura do próprio cache de páginas do Firebird.

Se prestarmos atenção ao número Reads nas estatísticas, fica claro por que a navegação por índice é tão lenta. Vamos dar uma olhada nas estatísticas da tabela HORSE e do índice HORSE_IDX_NAME.

HORSE (179)

```
Primary pointer page: 739, Index root page: 740
Total formats: 3, used formats: 1
Average record length: 238.19, total records: 551333
Average version length: 0.00, total versions: 0, max versions: 0
Average fragment length: 0.00, total fragments: 0, max fragments: 0
Average unpacked length: 2081.00, compression ratio: 8.74
Pointer pages: 4, data page slots: 10304
Data pages: 10304, average fill: 90%
Primary pages: 9662, secondary pages: 642, swept pages: 0
Empty pages: 2, full pages: 10298
Blobs: 74946, total length: 7730628, blob pages: 0
  Level 0: 74946, total length: 7730628, blob pages: 0
Table size: 168886272 bytes
Fill distribution:
  0 - 19% = 3
 20 - 39% = 3
 40 - 59% = 0
 60 - 79% = 1
 80 - 99% = 10297
```

Index HORSE_IDX_NAME (9)

```
Root page: 154569, depth: 2, leaf buckets: 329, nodes: 551333
Average node length: 9.61, total dup: 335597, max dup: 7783
Average key length: 6.66, compression ratio: 2.17
Average prefix length: 10.22, average data length: 4.27
Clustering factor: 534876, ratio: 0.97
Fill distribution:
  0 - 19% = 0
 20 - 39% = 1
 40 - 59% = 0
 60 - 79% = 0
 80 - 99% = 328
```

Index PK_HORSE (15)

```
Root page: 156379, depth: 2, leaf buckets: 407, nodes: 551333
```

```

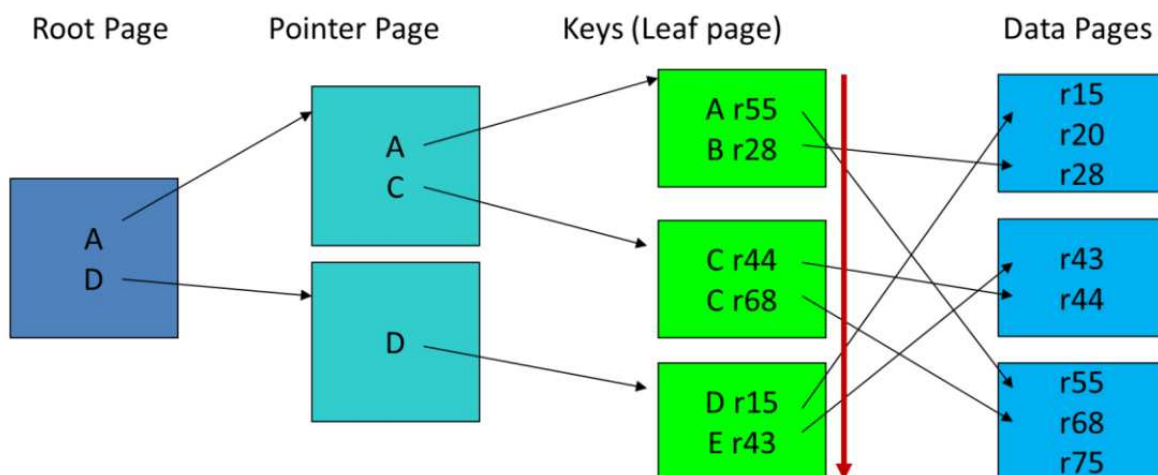
Average node length: 11.94, total dup: 0, max dup: 0
Average key length: 8.99, compression ratio: 1.00
Average prefix length: 3.01, average data length: 5.99
Clustering factor: 56182, ratio: 0.10
Fill distribution:
  0 - 19% = 0
 20 - 39% = 1
 40 - 59% = 0
 60 - 79% = 0
 80 - 99% = 406

```

Assim, para o índice HORSE_IDX_NAME, o número total de páginas é $\text{depth} + \text{leaf buckets} = 2 + 329 = 331$. O número total de páginas da tabela HORSE é $\text{Pointer pages} + \text{Data pages} = 4 + 10304 = 10308$. Observe que, na consulta usando ordenação externa, $\text{Reads} = 9665$ está muito próximo do valor $\text{Primary pages} = 9662$. Mas como obtivemos um número tão grande $\text{Reads} = 268639$ ao usar a navegação por índice com um cache de páginas pequeno?

Tudo se deve à natureza das leituras. Na navegação por índice, as leituras são principalmente aleatórias. A ordem das chaves do índice raramente coincide com a ordem física das páginas de dados onde os registros estão localizados. Assim, cada nova chave pode apontar para páginas completamente diferentes, e a mesma página pode ser lida várias vezes. Se o tamanho do cache de páginas for grande, nada de ruim acontece, as releituras da mesma página serão feitas do cache. Mas se o cache de páginas for limitado, as páginas serão eventualmente removidas dele. Se duas chaves consecutivas apontarem para a mesma página de dados, temos sorte e a releitura será feita do cache. Se duas chaves consecutivas apontarem para páginas diferentes, tudo depende se a página necessária foi removida.

Index -> Table



No entanto, a ordem das chaves do índice pode coincidir com o posicionamento físico das páginas de dados. Isso acontece, por exemplo, para chaves primárias com autoincremento sob certas condições. O grau de correspondência entre a ordem das chaves do índice e a localização física dos registros na tabela é chamado de fator de clusterização. Ele pode ser visto nas estatísticas fornecidas pela ferramenta gstat.

O valor Clustering factor denota o número de “saltos” de uma página de dados para outra ao percorrer todas as chaves do índice. O valor ratio próximo é calculado pela fórmula.

$$\text{ratio} = \text{Clustering factor} / \text{nodes}$$

Assim, o pior caso possível é $\text{ratio} = 1$.

Mas por que a ordenação externa permanece boa com um cache de páginas pequeno? É simples: os registros da tabela são lidos usando uma verificação completa Full Scan sequencial. Isso significa que registros próximos provavelmente serão lidos da mesma página, e essa página permanece no cache de páginas. Se o próximo registro estiver em uma nova página, essa página é lida fisicamente, e então os registros próximos subsequentes são novamente lidos logicamente do cache. Assim, cada página de dados é lida “fisicamente” uma vez, e neste caso o cache de páginas é usado de forma eficiente. Em seguida, a própria ordenação entra em ação. A ordenação pode ser feita na memória ou no disco. O próprio algoritmo de ordenação é de múltiplos estágios: pequenos conjuntos de dados são ordenados usando ordenação rápida, e então a ordenação por mesclagem entra em ação. A ordenação rápida é sempre feita na memória. A ordenação por mesclagem pode ocorrer na memória ou pode usar arquivos temporários. Mas mesmo quando se chega ao uso de arquivos temporários, a quantidade de leituras/gravações aleatórias ainda será menor do que na navegação por índice.

Você pode ter outra pergunta: por que o otimizador não pode resolver isso por mim? Infelizmente, nosso otimizador é muito otimista: os cálculos de custo são feitos apenas em leituras lógicas (fetches) ou, em alguns casos, em operações de memória (HASH JOIN). Ou seja, não leva em conta o fato de que o tamanho do cache de páginas é limitado. Além disso, o fator de clusterização dos índices não está disponível nas estatísticas armazenadas, só pode ser obtido pela ferramenta gstat.

Disto, tiramos várias conclusões:

- Se o cache de páginas for pequeno (arquitetura Classic, Super Classic), tente evitar a navegação por índice (Index Full Scan) se você precisar obter todos os resultados;
- Se você precisar obter rapidamente apenas o primeiro lote de dados, a navegação por índice é a melhor escolha. Mas lembre-se de que quanto maior esse lote de dados, maiores serão as despesas gerais.
- Para a arquitetura Super Server, tente configurar um cache de páginas maior. Meça o desempenho de suas consultas com ordenações. Se em alguma consulta for usada navegação por índice e as estatísticas mostrarem um grande número de Reads, é necessário aumentar o tamanho do cache de páginas ou suprimir o uso do índice para ordenação.
- Predicados de filtragem indexados afetam positivamente o desempenho da navegação por índice. A melhor opção é se o predicado de filtragem usar o mesmo índice que a navegação por índice.

7.6. Uso de índice para agrupamento

Índices podem ser utilizados durante a divisão em grupos (GROUP BY) no cálculo de agregados.

Existem os seguintes algoritmos de agregação (veja [Agregação](#)):

- Hash Group (não implementado no Firebird);
- Ordenação pelas expressões especificadas no GROUP BY.

Neste último caso, a ordenação pode ser realizada por meio de uma ordenação externa (SORT) ou por meio de navegação por índice (ORDER).



Diferente da ordenação externa em ORDER BY, a ordenação externa usada em GROUP BY não pode ser otimizada com Refetch.

As regras que permitem o uso de navegação por índice na cláusula GROUP BY são as mesmas que para ORDER BY:

- Todas as colunas especificadas no GROUP BY devem corresponder total ou parcialmente (para um dos segmentos) às colunas da chave do índice. Ou os números no GROUP BY devem apontar para essas colunas;
- A expressão especificada no GROUP BY deve coincidir com a expressão em um índice calculado;
- O agrupamento pode usar qualquer um dos índices ASC ou DESC. Ao mesmo tempo, não é necessário especificar nada no GROUP BY, o índice e a direção da ordenação serão escolhidos automaticamente;
- Em consultas que usam JOIN, a navegação por índice só é possível se o otimizador escolher a tabela líder para a qual um índice para navegação foi criado;
- Navegação (agrupamento) usando vários índices é impossível.

Vamos dar alguns exemplos.

```
SELECT
  COUNT(*) AS CNT,
  CODE_COLOR
FROM HORSE
GROUP BY CODE_COLOR;
```

-- or

```
SELECT
  COUNT(*) AS CNT,
  CODE_COLOR
FROM HORSE
GROUP BY 2;
```

Terá o seguinte plano e estatísticas:

```
PLAN (HORSE ORDER FK_HORSE_COLOR)

Select Expression
  -> Aggregate
    -> Table "HORSE" Access By ID
      -> Index "FK_HORSE_COLOR" Full Scan
Current memory = 555783984
```

```
Delta memory = 192
Max memory = 555873168
Elapsed time = 0.431 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 1201611
Per table statistics:
```

Table name	Natural	Index	Insert	Update	Delete
HORSE		551333			

Assim como no caso do ORDER BY, você pode desativar a navegação por índice usando as dicas +0 ou || '. Mas há uma nuance aqui: você não pode especificar expressões diferentes no SELECT e na cláusula GROUP BY, ou seja, é necessário aplicar as dicas em ambas as cláusulas, ou apenas na cláusula SELECT, e no GROUP BY usar a referência pelo número da coluna.

```
SELECT
  COUNT(*) AS CNT,
  CODE_COLOR + 0 AS CODE_COLOR
FROM HORSE
GROUP BY CODE_COLOR + 0;
```

```
-- or
```

```
SELECT
  COUNT(*) AS CNT,
  CODE_COLOR + 0 AS CODE_COLOR
FROM HORSE
GROUP BY 2;
```

O plano e as estatísticas de execução desta consulta serão:

```
PLAN SORT (HORSE NATURAL)
```

```
Select Expression
  -> Aggregate
    -> Sort (record length: 44, key length: 12)
      -> Table "HORSE" Full Scan
```

```
Current memory = 555804752
Delta memory = 208
Max memory = 590412288
Elapsed time = 0.507 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 589981
Per table statistics:
```

Table name	Natural	Index	Insert	Update	Delete
------------	---------	-------	--------	--------	--------

HORSE	551333				
-------	--------	--	--	--	--

Se você quiser usar a navegação por índice no agrupamento por vários campos, é necessário criar o índice composto correspondente (as regras são as mesmas que para ORDER BY).

```
CREATE INDEX IDX_COVER_FM ON COVER (CODE_FATHER, CODE_MOTHER);

SELECT
  COUNT(*) AS CNT,
  CODE_FATHER,
  CODE_MOTHER
FROM COVER
GROUP BY CODE_FATHER, CODE_MOTHER;
```

O plano e as estatísticas de execução desta consulta serão:

```
PLAN (COVER ORDER IDX_COVER_FM)
```

Select Expression

-> Aggregate

-> Table "COVER" Access By ID

-> Index "IDX_COVER_FM" Full Scan

Current memory = 554829744

Delta memory = 224

Max memory = 590412288

Elapsed time = 1.924 sec

Buffers = 32768

Reads = 0

Writes = 0

Fetches = 1891846

Per table statistics:

Table name	Natural	Index	Insert	Update	Delete
COVER		755297			

7.6.1. Filtragem no agrupamento

Nos exemplos acima, todos os registros da tabela foram agrupados por índice, mas normalmente filtros são aplicados à tabela usando a cláusula WHERE. Neste caso, surgem as seguintes variantes:

- Ocorre filtragem usando um predicado que não pode usar um índice;
- Ocorre filtragem usando um predicado que pode usar um índice;
 - A filtragem ocorre usando o índice pelo qual a navegação (agrupamento) acontece;
 - A filtragem ocorre usando outro índice.

Aqui não há nada novo em comparação com os casos descritos para ORDER BY. Para mais detalhes, veja [Filtragem durante navegação \(ordenação\) por índice](#).

Predicado de filtragem não indexado

Vejam os a seguinte consulta:

```
SELECT
  CODE_BREED,
  COUNT(*)
FROM HORSE
WHERE FAMILY = ' ';
GROUP BY CODE_BREED;
```

O plano e as estatísticas de execução desta consulta serão:

```
PLAN (HORSE ORDER FK_HORSE_BREED)
```

```
Select Expression
  -> Aggregate
    -> Filter
      -> Table "HORSE" Access By ID
        -> Index "FK_HORSE_BREED" Full Scan
```

```
Current memory = 555750432
```

```
Delta memory = 208
```

```
Max memory = 590412288
```

```
Elapsed time = 0.370 sec
```

```
Buffers = 32768
```

```
Reads = 0
```

```
Writes = 0
```

```
Fetches = 1182278
```

```
Per table statistics:
```

Table name	Natural	Index	Insert	Update	Delete
HORSE		551333			

O plano desta consulta não é muito ideal. Para entender isso, execute:

```
SELECT
  COUNT(*)
FROM HORSE
WHERE FAMILY = ' ';
```

```

COUNT
=====
3
```

A consulta com agrupamento é executada: todos os registros da tabela HORSE são recuperados na ordem das chaves do índice, e somente então o filtro FAMILY = '' é aplicado a eles. Mas o predicado FAMILY = '' possui uma seletividade bastante boa, e após ele apenas 3 registros são retornados, que são mais baratos de ordenar usando uma ordenação externa do que percorrer todos os registros da tabela HORSE na ordem do índice. Vamos tentar desativar o índice no agrupamento.

```
SELECT
  CODE_BREED + 0 AS CODE_BREED,
  COUNT(*)
FROM HORSE
WHERE FAMILY = ''
GROUP BY 1;
```

O plano e as estatísticas de execução desta consulta serão:

PLAN SORT (HORSE NATURAL)

```
Select Expression
  -> Aggregate
      -> Sort (record length: 68, key length: 12)
          -> Filter
              -> Table "HORSE" Full Scan
```

Current memory = 555876096

Delta memory = 208

Max memory = 590412288

Elapsed time = 0.217 sec

Buffers = 32768

Reads = 0

Writes = 0

Fetches = 589981

Per table statistics:

Table name	Natural	Index	Insert	Update	Delete
HORSE	551333				

Como você pode ver, a consulta ficou 1.5 vezes mais rápida.

Mas a ordenação externa será sempre mais rápida que a navegação por índice? A resposta correta é não. Tudo depende da distribuição dos dados e da seletividade do predicado de filtragem.

Infelizmente, na versão atual do Firebird, o otimizador não sabe fazer uma estimativa de custo para escolher entre ordenação externa e navegação por índice – se houver um índice adequado para navegação por índice, ela é executada. Portanto, esses casos você terá que otimizar por conta própria com base em suposições e experimentos.



No Firebird 6.0, a estimativa de custo para a escolha (SORT vs ORDER) finalmente apareceu. Mas ela é baseada na seletividade dos predicados de filtragem, especificados por constantes, o que nem sempre reflete corretamente a

seletividade real.

Filtragem por outro índice

Assim como no caso de navegação por índice em ORDER BY, para filtragem pode ser usado um predicado indexado que otimiza o agrupamento. Vou dar um exemplo:

```
SELECT
  CODE_BREED,
  COUNT(*)
FROM HORSE
WHERE CODE_DEPARTURE = 1
GROUP BY 1;
```

O plano e as estatísticas de execução desta consulta serão:

```
PLAN (HORSE ORDER FK_HORSE_BREED INDEX (FK_HORSE_DEPARTURE))

Select Expression
  -> Aggregate
    -> Filter
      -> Table "HORSE" Access By ID
        -> Index "FK_HORSE_BREED" Full Scan
          -> Bitmap
            -> Index "FK_HORSE_DEPARTURE" Range Scan (full match)

Current memory = 556280752
Delta memory = 208
Max memory = 590412288
Elapsed time = 0.100 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 204679
Per table statistics:
```

Table name	Natural	Index	Insert	Update	Delete
HORSE		98853			

Filtragem pelo mesmo índice usado para agrupamento

A filtragem pode usar o mesmo índice que o agrupamento:

```
SELECT
  CODE_BREED,
  COUNT(*)
FROM HORSE
WHERE CODE_BREED IN (50, 51)
GROUP BY 1;
```

O plano e as estatísticas de execução desta consulta serão:

```
PLAN (HORSE ORDER FK_HORSE_BREED)
```

```
Select Expression
```

```
-> Aggregate
```

```
-> Filter
```

```
-> Table "HORSE" Access By ID
```

```
-> Index "FK_HORSE_BREED" List Scan (full match)
```

```
Current memory = 556989888
```

```
Delta memory = 208
```

```
Max memory = 590412288
```

```
Elapsed time = 0.028 sec
```

```
Buffers = 32768
```

```
Reads = 0
```

```
Writes = 0
```

```
Fetches = 66021
```

```
Per table statistics:
```

Table name	Natural	Index	Insert	Update	Delete
HORSE		30276			

Tais filtros aumentam significativamente a velocidade de execução da consulta, pois são criados limites de varredura para a navegação por índice, e apenas os registros cujas chaves estão dentro desses limites são recuperados.

Filtragem por segmento do mesmo índice composto usado para agrupamento

Assim como para ORDER BY, para agrupamentos há uma nuance com índices compostos. Da cláusula GROUP BY pode-se remover a parte inicial da chave do índice, e ainda assim será usada a navegação por índice, se para essa parte inicial houver uma condição de filtragem que leve a uma correspondência completa no segmento do índice, ou seja, para predicados (=, IS NULL, IS NOT DISTINCT FROM). Vou demonstrar isso com um exemplo:

```
SELECT
  COUNT(*),
  FIRST_NAME
FROM
  EMPLOYEE
WHERE LAST_NAME = 'Baldwin'
GROUP BY 2;
```

O plano desta consulta será:

```
PLAN (EMPLOYEE ORDER NAMEX)
```

```
Select Expression
```

```
-> Aggregate
```

```
-> Filter
```

```
-> Table "EMPLOYEE" Access By ID
    -> Index "NAMEX" Range Scan (partial match: 1/2)
```

Lembro que o índice composto NAMEX é definido como:

```
CREATE INDEX NAMEX ON EMPLOYEE (LAST_NAME, FIRST_NAME);
```

7.6.2. Agrupamento e limitadores FIRST/ROWS/OFFSET

Anteriormente, falei sobre como os limitadores FIRST/ROWS/OFFSET afetam positivamente o desempenho da navegação por índice em ORDER BY, pois permitem ler não todas as páginas do índice e registros na tabela, mas apenas a quantidade especificada. Vamos ver como está a situação com a navegação por índice em GROUP BY.

```
SELECT
  CODE_BREED,
  COUNT(*)
FROM HORSE
GROUP BY 1
FETCH FIRST ROW ONLY;
```

O plano e as estatísticas de execução desta consulta serão os seguintes:

```
Select Expression
  -> First N Records
      -> Aggregate
          -> Table "HORSE" Access By ID
              -> Index "FK_HORSE_BREED" Full Scan
```

Current memory = 556019376

Delta memory = 192

Max memory = 590412288

Elapsed time = 0.004 sec

Buffers = 32768

Reads = 0

Writes = 0

Fetches = 8743

Per table statistics:

Table name	Natural	Index	Insert	Update	Delete
HORSE		3664			

Aqui também há um ganho, mas não tão grande. O motivo é que os contadores FIRST/ROWS/OFFSET são aplicados não à própria recuperação de registros pelo índice, mas ao resultado da agregação. A agregação usando navegação por índice também é uma operação em pipeline, mas o contador é incrementado apenas após o grupo ter mudado. Neste exemplo, para o primeiro valor CODE_BREED = 1 foram encontrados 3663 registros, quando a navegação por índice recuperou o próximo valor

CODE_BREED = 2, o contador foi incrementado e o processo imediatamente parou. É por isso que foram feitas 3664 leituras de índice antes da execução ser interrompida.



Na prática, FIRST/ROWS/OFFSET é usado em conjunto com a cláusula ORDER BY e não é recomendado usá-lo sem ela, caso contrário a determinística do resultado não é garantida. Lembro que em versões futuras o agrupamento pode ser executado usando o algoritmo HASH GROUP, e nesse caso o uso de FIRST/ROWS/OFFSET pode dar um resultado aleatório, ou seja, o resultado da agregação de algum grupo, e não do primeiro na ordem.

7.6.3. Agrupamento e ordenação

O resultado de uma consulta agregada pode ser ordenado usando ORDER BY. Aqui podem haver várias opções:

- ORDER BY pelos valores das funções agregadas. Neste caso, sempre haverá uma ordenação externa adicional.
- ORDER BY pelos campos de agrupamento.

Com a primeira opção tudo está claro, portanto não a consideraremos. Vamos ver como ORDER BY interage com GROUP BY no caso de correspondência total ou parcial dos campos de ordenação e agrupamento.

O Firebird é suficientemente inteligente e remove duplicatas de ordenações, ou seja, não faz uma ordenação desnecessária repetidamente. Mas como está a situação com a navegação por índice?

Se GROUP BY usa navegação por índice e em ORDER BY são especificados os mesmos campos ou expressão na mesma ordem que em ORDER BY, com a direção de ordenação correspondente à ordem das chaves no índice, então nenhuma operação adicional é realizada. Ou seja, na prática não haverá nenhuma ordenação adicional, os dados já serão retornados na ordem das chaves do índice.

```
SELECT
  CODE_BREED,
  COUNT(*)
FROM HORSE
GROUP BY 1
ORDER BY 1;
```

O plano de execução desta consulta será o seguinte:

```
PLAN (HORSE ORDER FK_HORSE_BREED)

Select Expression
-> Aggregate
  -> Table "HORSE" Access By ID
    -> Index "FK_HORSE_BREED" Full Scan
```

No entanto, se tentarmos alterar a direção da ordenação em ORDER BY, o plano mudará.

```
SELECT
  CODE_BREED,
  COUNT(*)
FROM HORSE
GROUP BY 1
ORDER BY 1 DESC;
```

O plano desta consulta será:

```
PLAN SORT (HORSE NATURAL)

Select Expression
  -> Aggregate
      -> Sort (record length: 28, key length: 8)
          -> Table "HORSE" Full Scan
```

Neste caso, o otimizador não encontrou um índice adequado para navegação e usou uma ordenação externa. Observo que é usada apenas uma ordenação externa, ou seja, GROUP BY simplesmente usa a ordenação em direção descendente.

Se criarmos um índice DESCENDING para a coluna CODE_BREED, o otimizador novamente utilizará a navegação por índice.

```
CREATE DESCENDING INDEX IDX_HORSE_CODE_BREED_DESC ON HORSE(CODE_BREED);

SELECT
  CODE_BREED,
  COUNT(*)
FROM HORSE
GROUP BY 1
ORDER BY 1 DESC;
```

O plano desta consulta será:

```
PLAN (HORSE ORDER IDX_HORSE_CODE_BREED_DESC)

Select Expression
  -> Aggregate
      -> Table "HORSE" Access By ID
          -> Index "IDX_HORSE_CODE_BREED_DESC" Full Scan
```

Exatamente o mesmo ocorre com índices compostos.

```
SELECT
  CODE_DEPARTURE,
  CODE_FARM,
  COUNT(*)
FROM HORSE
```

```
GROUP BY 1, 2
ORDER BY 1, 2;
```

O plano desta consulta será:

```
PLAN (HORSE ORDER IDX_HORSE_DEP_FARMBORN)

Select Expression
  -> Aggregate
    -> Table "HORSE" Access By ID
      -> Index "IDX_HORSE_DEP_FARMBORN" Full Scan
```

Como você pode ver, não há ordenações adicionais.

É claro que, se você especificar uma ordem diferente de campos em GROUP BY e ORDER BY, aparecerá uma ordenação externa adicional.

```
SELECT
  CODE_DEPARTURE,
  CODE_FARM,
  COUNT(*)
FROM HORSE
GROUP BY 1, 2
ORDER BY 2, 1;
```

```
PLAN SORT (HORSE ORDER IDX_HORSE_DEP_FARMBORN)

Select Expression
  -> Sort (record length: 52, key length: 20)
    -> Aggregate
      -> Table "HORSE" Access By ID
        -> Index "IDX_HORSE_DEP_FARMBORN" Full Scan
```

Em relação à ordenação DESC, os índices compostos se comportam exatamente da mesma forma que os comuns, ou seja, a navegação por índice será aplicada se for encontrado um índice composto DESC adequado, caso contrário será usada uma única ordenação externa. É claro que a navegação por índice é impossível se em ORDER BY parte das colunas for especificada com ordenação ASC, e parte com ordenação DESC.

É notável também que, se em ORDER BY for especificada a ordenação apenas da parte inicial da lista de colunas que são especificadas em GROUP BY, também não são produzidas ordenações adicionais. Isso é facilmente explicável — se toda a chave está ordenada, então sua parte inicial também está ordenada.

```
SELECT
  CODE_DEPARTURE,
  CODE_FARM,
  COUNT(*)
```

```
FROM HORSE
GROUP BY 1, 2
ORDER BY 1;
```

```
PLAN (HORSE ORDER IDX_HORSE_DEP_FARMBORN)
```

```
Select Expression
-> Aggregate
    -> Table "HORSE" Access By ID
        -> Index "IDX_HORSE_DEP_FARMBORN" Full Scan
```

7.6.4. Agrupamento em consultas com múltiplas tabelas

Até agora, consideramos principalmente a aplicação da navegação por índice (agrupamento) em consultas simples com uma única tabela, porém, na realidade, são mais comuns consultas com junção (JOIN) de várias tabelas. Para tais consultas, o agrupamento por índice é aplicado com muito menos frequência. A razão é que a navegação por índice só pode ser aplicada se a tabela escolhida como fluxo principal for aquela cujos campos indexados precisam ser usados para o agrupamento.

Para consultas com uma única tabela, tudo é simples — se existe uma possibilidade potencial de usar navegação por índice, então ela é usada. Neste caso, o otimizador não faz uma escolha baseada em custo entre planos SORT (ordenação externa) e ORDER (navegação por índice), mas segue regras (usa uma abordagem rule based, não cost based).

No entanto, quando a consulta usa uma junção (JOIN) de várias tabelas, a avaliação de custo é ativada para escolher o algoritmo e a ordem correta de junção, e somente quando a ordem de junção é determinada, decide-se usar navegação por índice (se tivermos sorte e nossa tabela for a principal) ou ordenação externa para o agrupamento.

Vamos comparar duas consultas:

```
SELECT
  COLOR.NAME AS COLOR_NAME,
  COUNT(*)
FROM
  HORSE H
  JOIN COLOR ON COLOR.CODE_COLOR = H.CODE_COLOR
  JOIN BREED ON BREED.CODE_BREED = H.CODE_BREED
GROUP BY 1;
```

O plano desta consulta será:

```
PLAN HASH (JOIN (COLOR ORDER UNQ_COLOR_NAME, H INDEX (FK_HORSE_COLOR)), BREED NATURAL)
```

```
Select Expression
-> Aggregate
    -> Filter
        -> Hash Join (inner)
            -> Nested Loop Join (inner)
```

```

-> Table "COLOR" Access By ID
  -> Index "UNQ_COLOR_NAME" Full Scan
-> Filter
  -> Table "HORSE" as "H" Access By ID
    -> Bitmap
      -> Index "FK_HORSE_COLOR" Range Scan (full match)
-> Record Buffer (record length: 25)
  -> Table "BREED" Full Scan

```

Neste caso, a tabela COLOR foi escolhida como principal na junção e, portanto, é usada a navegação pelo índice UNQ_COLOR_NAME para o agrupamento.

E agora vamos tentar realizar o agrupamento por uma coluna indexada de outra tabela:

```

SELECT
  BREED.NAME AS BREED_NAME,
  COUNT(*)
FROM
  HORSE H
  JOIN COLOR ON COLOR.CODE_COLOR = H.CODE_COLOR
  JOIN BREED ON BREED.CODE_BREED = H.CODE_BREED
GROUP BY 1;

```

O plano desta consulta será:

```

PLAN SORT (HASH (JOIN (COLOR NATURAL, H INDEX (FK_HORSE_COLOR)), BREED NATURAL))

Select Expression
  -> Aggregate
    -> Sort (record length: 588, key length: 304)
      -> Filter
        -> Hash Join (inner)
          -> Nested Loop Join (inner)
            -> Table "COLOR" Full Scan
            -> Filter
              -> Table "HORSE" as "H" Access By ID
                -> Bitmap
                  -> Index "FK_HORSE_COLOR" Range Scan (full match)
              -> Record Buffer (record length: 233)
            -> Table "BREED" Full Scan

```

Neste caso, é usada uma ordenação externa. Observe: a ordem de junção das tabelas permaneceu a mesma, mas não é mais possível usar a navegação por índice para o agrupamento.

Claro, você pode usar apenas LEFT JOIN em sua consulta, garantindo a ordem de junção e, portanto, a possibilidade de usar navegação por índice, mas isso nem sempre é possível e ideal.

```

SELECT
  BREED.NAME AS BREED_NAME,
  COUNT(*)
FROM

```



```
BREED
LEFT JOIN HORSE H ON H.CODE_BREED = BREED.CODE_BREED
LEFT JOIN COLOR ON COLOR.CODE_COLOR = H.CODE_COLOR
WHERE H.CODE_BREED IS NOT NULL AND COLOR.CODE_COLOR IS NOT NULL
GROUP BY 1;
```

O plano desta consulta será:

```
PLAN JOIN (JOIN (BREED ORDER UNQ_BREED_NAME, H INDEX (FK_HORSE_BREED)), COLOR INDEX (PK_COLOR))
```

Select Expression

-> Aggregate

-> Filter

-> Nested Loop Join (outer)

-> Filter

-> Nested Loop Join (outer)

-> Table "BREED" Access By ID

-> Index "UNQ_BREED_NAME" Full Scan

-> Filter

-> Table "HORSE" as "H" Access By ID

-> Bitmap

-> Index "FK_HORSE_BREED" Range Scan (full match)

-> Filter

-> Table "COLOR" Access By ID

-> Bitmap

-> Index "PK_COLOR" Unique Scan

Há outra opção sem reescrever a consulta. Podemos pedir para alterar o objetivo do otimizador para a recuperação mais rápida das primeiras linhas, como foi feito para ORDER BY. Quando definimos o objetivo FIRST ROWS para o otimizador, ele tenta evitar operações em buffer, como a ordenação externa, e usa operações em pipeline (navegação por índice) em seu lugar.

```
SELECT
  BREED.NAME AS BREED_NAME,
  COUNT(*)
FROM
  HORSE H
  JOIN COLOR ON COLOR.CODE_COLOR = H.CODE_COLOR
  JOIN BREED ON BREED.CODE_BREED = H.CODE_BREED
GROUP BY 1
OPTIMIZE FOR FIRST ROWS;
```

O plano desta consulta será:

```
PLAN JOIN (BREED ORDER UNQ_BREED_NAME, H INDEX (FK_HORSE_BREED), COLOR INDEX (PK_COLOR))
```

Select Expression

-> Aggregate

-> Nested Loop Join (inner)

-> Table "BREED" Access By ID

-> Index "UNQ_BREED_NAME" Full Scan

```

-> Filter
  -> Table "HORSE" as "H" Access By ID
    -> Bitmap
      -> Index "FK_HORSE_BREED" Range Scan (full match)
-> Filter
  -> Table "COLOR" Access By ID
    -> Bitmap
      -> Index "PK_COLOR" Unique Scan

```

Conseguimos alcançar o desejado sem uma modificação substancial da consulta. Mas qual é o preço disso?

Vamos comparar o desempenho das consultas com agrupamento, realizado por meio de ordenação externa e navegação por índice.

Estatísticas de execução sem navegação por índice:

```

Select Expression
  -> Aggregate
    -> Sort (record length: 588, key length: 304)
      -> Filter
        -> Hash Join (inner)
          -> Nested Loop Join (inner)
            -> Table "COLOR" Full Scan
            -> Filter
              -> Table "HORSE" as "H" Access By ID
                -> Bitmap
                  -> Index "FK_HORSE_COLOR" Range Scan (full match)
            -> Record Buffer (record length: 233)
              -> Table "BREED" Full Scan

```

Current memory = 554056944
 Delta memory = 288
 Max memory = 957863360
 Elapsed time = 2.221 sec
 Buffers = 32768
 Reads = 0
 Writes = 0
 Fetches = 651556
 Per table statistics:

Table name	Natural	Index	Insert	Update	Delete
BREED	289				
COLOR	241				
HORSE		551333			

Estatísticas de execução com navegação por índice:

```

Select Expression
  -> Aggregate
    -> Nested Loop Join (inner)

```

```

-> Table "BREED" Access By ID
  -> Index "UNQ_BREED_NAME" Full Scan
-> Filter
  -> Table "HORSE" as "H" Access By ID
    -> Bitmap
      -> Index "FK_HORSE_BREED" Range Scan (full match)
-> Filter
  -> Table "COLOR" Access By ID
    -> Bitmap
      -> Index "PK_COLOR" Unique Scan

```

Current memory = 554085168

Delta memory = 320

Max memory = 957863360

Elapsed time = 1.125 sec

Buffers = 32768

Reads = 0

Writes = 0

Fetches = 2286431

Per table statistics:

Table name	Natural	Index	Insert	Update	Delete
BREED		289			
COLOR		551333			
HORSE		551333			

O agrupamento com navegação por índice acabou sendo mais rápido, embora os acessos ao cache de páginas (Fetches) sejam 4 vezes maiores. Em geral, alterar a ordem de junção pode levar a uma piora significativa no tempo de execução da consulta.

O agrupamento usando navegação por índice é uma operação bastante cara. O custo da navegação por índice é descrito em [Custo de navegação \(ordenação\) por índice](#).

7.7. Usando um índice para calcular funções agregadas MIN/MAX

Normalmente, para calcular funções agregadas, é necessário um registro temporário para armazenar o valor limite atual (MIN/MAX) ou acumulado (outras funções). Em seguida, para cada registro do fluxo de entrada, verificamos ou somamos esse registro. Ou seja, em geral, para calcular agregados, é necessário ler todo o fluxo de entrada.

No entanto, para as funções agregadas MIN/MAX, existe outra maneira usando a navegação por índice. Obviamente, para calcular MIN, basta pegar a primeira chave em um índice ASC, e para calcular MAX - a primeira chave em um índice DESC. Se, após buscar (fetch) o registro, ele não estiver visível para nós, pegamos a próxima chave, e assim por diante. Vamos demonstrar isso com um exemplo.

```

SELECT MIN(INVOICE_DATE) AS MIN_DATE
FROM INVOICE;

```

Plano e estatísticas de execução:

```
PLAN (INVOICE ORDER INVOICE_IDX_DATE)
```

Select Expression

-> Aggregate

-> Table "INVOICE" Access By ID

-> Index "INVOICE_IDX_DATE" Full Scan

Current memory = 284775936

Delta memory = 176

Max memory = 284852848

Elapsed time = 0.001 sec

Buffers = 32768

Reads = 0

Writes = 0

Fetches = 4

Per table statistics:

Table name	Natural	Index	Insert	Update	Delete
INVOICE		1			

Como você pode ver, neste caso, o uso da navegação por índice para calcular a função agregada MIN é justificado - a consulta foi executada quase instantaneamente e com custo mínimo.

Vamos tentar calcular a data máxima.

```
SELECT MAX(INVOICE_DATE) AS MAX_DATE
FROM INVOICE;
```

Plano e estatísticas de execução:

```
PLAN (INVOICE NATURAL)
```

Select Expression

-> Aggregate

-> Table "INVOICE" Full Scan

Current memory = 284803488

Delta memory = 176

Max memory = 284870096

Elapsed time = 0.024 sec

Buffers = 32768

Reads = 0

Writes = 0

Fetches = 103935

Per table statistics:

Table name	Natural	Index	Insert	Update	Delete
INVOICE					

INVOICE	100010				

Para calcular MAX, a navegação por índice não pôde ser utilizada porque o índice INVOICE_IDX_DATE foi criado em ordem crescente (ASCENDING). Vamos tentar criar um índice decrescente.

```
CREATE DESCENDING INDEX INVOICE_IDX_DATE_DESC ON INVOICE(INVOICE_DATE);

SELECT MAX(INVOICE_DATE) AS MAX_DATE
FROM INVOICE;
```

```
PLAN (INVOICE ORDER INVOICE_IDX_DATE_DESC)
```

```
Select Expression
-> Aggregate
    -> Table "INVOICE" Access By ID
        -> Index "INVOICE_IDX_DATE_DESC" Full Scan
```

```
Current memory = 284222768
```

```
Delta memory = 176
```

```
Max memory = 288938384
```

```
Elapsed time = 0.002 sec
```

```
Buffers = 32768
```

```
Reads = 0
```

```
Writes = 0
```

```
Fetches = 4
```

```
Per table statistics:
```

Table name	Natural	Index	Insert	Update	Delete
INVOICE		1			

Agora a navegação por índice está sendo usada e o desempenho da consulta está bom novamente.

Para índices compostos, a navegação por índice pode ser usada para calcular MIN/MAX no primeiro campo que faz parte do índice composto. Para agrupamentos e filtragem, há uma nuance que será considerada mais tarde. Vejamos um exemplo.

```
CREATE INDEX NAMEX ON EMPLOYEE (LAST_NAME, FIRST_NAME);

SELECT
  MIN(LAST_NAME)
FROM
  EMPLOYEE;
```

Nesta consulta, apenas a primeira coluna do índice NAMEX é usada. O plano de execução será o seguinte:

```
PLAN (EMPLOYEE ORDER NAMEX)
```

```
Select Expression
```

```
-> Aggregate
```

```
-> Table "EMPLOYEE" Access By ID
```

```
-> Index "NAMEX" Full Scan
```

No entanto, se tentarmos calcular `MIN(FIRST_NAME)`, a navegação por índice não pode ser usada.

```
SELECT
  MIN(FIRST_NAME)
FROM
  EMPLOYEE;
```

```
PLAN (EMPLOYEE NATURAL)
```

```
Select Expression
```

```
-> Aggregate
```

```
-> Table "EMPLOYEE" Full Scan
```

7.7.1. MIN/MAX e NULL

Como se sabe, as funções agregadas `MIN` e `MAX` ignoram o valor `NULL`. Acima foi mostrado como a navegação por índice otimiza o cálculo das funções agregadas `MIN` e `MAX`. No entanto, nos exemplos anteriores, para as colunas sobre as quais os agregados foram calculados, não havia valores `NULL`. Agora vamos tentar calcular `MIN` e `MAX` sobre colunas que contêm `NULL`.

```
SELECT
  MIN(BYDATE) AS MIN_BYDATE
FROM COVER;
```

Plano e estatísticas de execução:

```
PLAN (COVER ORDER COVER_IDX_BYDATE)
```

```
Select Expression
```

```
-> Aggregate
```

```
-> Table "COVER" Access By ID
```

```
-> Index "COVER_IDX_BYDATE" Full Scan
```

```
Current memory = 553940672
```

```
Delta memory = 176
```

```
Max memory = 554029632
```

```
Elapsed time = 0.033 sec
```

```
Buffers = 32768
```

```
Reads = 0
```

```
Writes = 0
```

```
Fetches = 133939
```

Per table statistics:

Table name	Natural	Index	Insert	Update	Delete
COVER		63858			

A navegação por índice foi usada, mas o desempenho da consulta não é muito bom. De onde vieram as 63858 leituras de índice? Para responder a essa pergunta, decidi executar uma consulta que conta o número de NULL na coluna BYDATE.

```
SELECT
  COUNT(*)
FROM COVER
WHERE BYDATE IS NULL;
```

```
          COUNT
=====
          63857
```

Aí está: a navegação por índice estava escaneando 63857 valores NULL ou apenas o 63858º valor não era NULL. Em geral, a navegação por índice poderia pular chaves NULL, e esse modo existe no otimizador, mas por alguma razão não foi aplicado ao cálculo dos agregados MIN/MAX. Sobre isso, criei um ticket [CORE-8744](#), que deve ser corrigido nas próximas versões.

Interessante, para índices DESC e MAX a situação é a mesma? Vamos verificar.

```
CREATE DESCENDING INDEX COVER_IDX_BYDATE_DESC ON COVER (BYDATE);

SELECT
  MAX(BYDATE) AS MAX_DATE
FROM COVER;
```

Plano e estatísticas de execução:

```
PLAN (COVER ORDER COVER_IDX_BYDATE_DESC)

Select Expression
  -> Aggregate
    -> Table "COVER" Access By ID
      -> Index "COVER_IDX_BYDATE_DESC" Full Scan

Current memory = 553261744
Delta memory = 176
Max memory = 578422560
Elapsed time = 0.002 sec
Buffers = 32768
Reads = 0
Writes = 0
```

Fetches = 4

Per table statistics:

Table name	Natural	Index	Insert	Update	Delete
COVER		1			

Felizmente, não. O fato é que os valores NULL nos índices estão posicionados antes do menor valor, independentemente da direção do índice.

7.7.2. Desativando o índice ao calcular MIN/MAX

Você pode desistir de usar a navegação de índice ao calcular as funções agregadas MIN/MAX. Para fazer isso, é necessário aplicar uma expressão sobre o argumento da função MIN/MAX que não altere seu valor. Na maioria das vezes, são usadas as expressões `+0` ou `|| ''` dependendo do tipo da expressão. Por exemplo:

```
SELECT MIN(INVOICE_DATE + 0) AS MIN_DATE
FROM INVOICE;
```

O plano de execução após desativar o índice será assim:

```
PLAN (INVOICE NATURAL)
```

```
Select Expression
```

```
-> Aggregate
```

```
-> Table "INVOICE" Full Scan
```

7.7.3. Cálculo de MIN/MAX e filtragem

Assim como em outros casos de uso da navegação de índice, aqui podemos destacar três casos:

- Predicado de filtragem não indexado;
- Predicado de filtragem indexado por outro índice;
- Predicado de filtragem indexado pelo mesmo índice que é usado para calcular MIN/MAX.

7.7.4. Predicado de filtragem não indexado

Neste caso, o algoritmo para calcular MIN/MAX usando navegação de índice é o seguinte:

1. para calcular MIN, pegamos a primeira chave no índice ASC, e para calcular MAX — a primeira chave no índice DESC.
2. se após buscar (fetch) o registro, ele não estiver visível para nós, então pegamos a próxima chave; caso contrário, vamos para o passo 3.
3. o filtro (predicado não indexado) é verificado, e se for verdadeiro, o registro é emitido na saída

e o algoritmo é interrompido; caso contrário, buscamos a próxima chave e verificamos sua visibilidade (passo 2).

Neste caso, é óbvio que, dependendo da distribuição dos dados, a consulta pode ser executada com desempenho variável. Se após a primeira chave buscada a condição de filtragem for atendida, a consulta será executada quase instantaneamente, assim como sem esse filtro. No entanto, a condição de filtragem pode não ser verdadeira de imediato e, nesse caso, será necessário buscar um número considerável de registros via navegação de índice. No pior caso, quando a condição de filtragem não é atendida por nenhum registro, o mecanismo terá que buscar absolutamente todos os registros da tabela usando navegação de índice. Vamos demonstrar isso com exemplos.

```
SELECT MIN(INVOICE_DATE)
FROM INVOICE
WHERE TOTAL_SALE > 20000;
```

Plano e estatísticas de execução:

```
PLAN (INVOICE ORDER INVOICE_IDX_DATE)
```

Select Expression

-> Aggregate

-> Filter

-> Table "INVOICE" Access By ID

-> Index "INVOICE_IDX_DATE" Full Scan

MIN

```
=====
2015-01-08 09:06:00.0000
```

Current memory = 284695904

Delta memory = 192

Max memory = 284776288

Elapsed time = 0.001 sec

Buffers = 32768

Reads = 0

Writes = 0

Fetches = 22

Per table statistics:

Table name	Natural	Index	Insert	Update	Delete
INVOICE		10			

Neste caso, tivemos sorte, e a data mínima com `TOTAL_SALE > 20000` foi encontrada quase imediatamente. O algoritmo fez apenas 9 leituras de índice extras.

Agora vamos alterar a consulta:

```
SELECT MIN(INVOICE_DATE)
```

```
FROM INVOICE
WHERE TOTAL_SALE > 140000;
```

Plano e estatísticas de execução:

```
PLAN (INVOICE ORDER INVOICE_IDX_DATE)
```

Select Expression

-> Aggregate

-> Filter

-> Table "INVOICE" Access By ID

-> Index "INVOICE_IDX_DATE" Full Scan

MIN

```
=====
2015-05-11 21:27:00.0000
```

Current memory = 284738480

Delta memory = 192

Max memory = 284776288

Elapsed time = 0.028 sec

Buffers = 32768

Reads = 0

Writes = 0

Fetches = 119245

Per table statistics:

Table name	Natural	Index	Insert	Update	Delete
INVOICE		59297			

Desta vez não tivemos sorte, e antes de encontrar um registro que satisfizesse o predicado `TOTAL_SALE > 140000`, tivemos que realizar 59296 leituras indexadas.

E agora vamos ver o pior caso, quando o predicado de filtragem não é satisfeito por nenhum dos registros.

```
SELECT MIN(INVOICE_DATE)
FROM INVOICE
WHERE TOTAL_SALE > 200000;
```

Plano e estatísticas de execução:

```
PLAN (INVOICE ORDER INVOICE_IDX_DATE)
```

Select Expression

-> Aggregate

-> Filter

-> Table "INVOICE" Access By ID

-> Index "INVOICE_IDX_DATE" Full Scan

```

                MIN
=====
                <null>

Current memory = 284805104
Delta memory = 192
Max memory = 284822224
Elapsed time = 0.045 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 201117
Per table statistics:
-----+-----+-----+-----+-----+
Table name      | Natural | Index  | Insert | Update | Delete |
-----+-----+-----+-----+-----+
INVOICE         |         | 100010 |        |        |        |
-----+-----+-----+-----+-----+

```

Neste caso, absolutamente todos os registros foram buscados via navegação de índice. Lembre-se de que a navegação de índice com busca de todos ou da maior parte dos registros é uma operação bastante cara. Nos dois últimos casos, é mais barato desistir da navegação de índice e executar MIN/MAX da maneira usual.

```

SELECT MIN(INVOICE_DATE + 0)
FROM INVOICE
WHERE TOTAL_SALE > 140000;

```

Plano e estatísticas de execução:

```

PLAN (INVOICE NATURAL)

Select Expression
  -> Aggregate
    -> Filter
      -> Table "INVOICE" Full Scan

                MIN
=====
2015-05-11 21:27:00.0000

Current memory = 284835648
Delta memory = 192
Max memory = 284864112
Elapsed time = 0.026 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 103935
Per table statistics:
-----+-----+-----+-----+-----+

```

Table name	Natural	Index	Insert	Update	Delete
INVOICE	100010				

O tempo de execução desta consulta usando navegação de índice e sem ela é comparável, mas se o cache de páginas for pequeno, a variante sem uso de navegação de índice vencerá várias vezes.

Para a segunda consulta, as vantagens ao desistir da navegação de índice são mais perceptíveis.

```
SELECT MIN(INVOICE_DATE + 0)
FROM INVOICE
WHERE TOTAL_SALE > 200000;
```

Plano e estatísticas de execução:

PLAN (INVOICE NATURAL)

Select Expression

-> Aggregate

-> Filter

-> Table "INVOICE" Full Scan

MIN

=====

<null>

Current memory = 284861552

Delta memory = 192

Max memory = 284890016

Elapsed time = 0.027 sec

Buffers = 32768

Reads = 0

Writes = 0

Fetches = 103935

Per table statistics:

Table name	Natural	Index	Insert	Update	Delete
INVOICE	100010				

Neste caso, ganhamos quase 2 vezes em tempo de execução.

Infelizmente, na versão atual do Firebird, o otimizador não pode ajudá-lo de forma alguma na escolha do algoritmo adequado para calcular MIN/MAX. Na versão atual do Firebird, não há estatísticas de distribuição de valores em colunas não indexadas (nem mesmo nas indexadas), e, portanto, o otimizador não pode prever quantos valores você terá que escanear até encontrar um registro que satisfaça o predicado de filtragem. Assim, tais consultas só podem ser otimizadas com base nos resultados de experimentos.

7.7.5. Predicado de filtro indexado por outro índice

Acima, examinamos questões de desempenho quando um predicado de filtro não indexado é aplicado a funções agregadas MIN/MAX que usam navegação por índice. Agora, vamos ver o que acontece se o predicado for indexado. Aqui, podemos destacar dois casos:

- outro índice é usado para o predicado de filtro;
- o mesmo índice usado para calcular MIN/MAX é usado para o predicado de filtro.

Vamos começar com o primeiro caso.

```
SELECT MIN(INVOICE_DATE)
FROM INVOICE
WHERE CUSTOMER_ID = 25;
```

Plano e estatísticas de execução:

```
PLAN (INVOICE ORDER INVOICE_IDX_DATE INDEX (FK_INVOICE_CUSTOMER))

Select Expression
  -> Aggregate
    -> Filter
      -> Table "INVOICE" Access By ID
        -> Index "INVOICE_IDX_DATE" Full Scan
          -> Bitmap
            -> Index "FK_INVOICE_CUSTOMER" Range Scan (full match)
```

MIN

```
=====
2015-01-08 09:21:00.0000
```

```
Current memory = 284903152
Delta memory = 176
Max memory = 284920272
Elapsed time = 0.002 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 7
Per table statistics:
```

Table name	Natural	Index	Insert	Update	Delete
INVOICE		1			

Neste caso, o algoritmo de execução da consulta é o seguinte:

1. uma máscara de bits é construída com base nas chaves correspondentes do índice FK_INVOICE_CUSTOMER;
2. para calcular MIN, pegamos a primeira chave no índice ASC, e para calcular MAX – a primeira

chave no índice DESC;

3. se o número do registro, obtido ao extrair a primeira chave do índice INVOICE_IDX_DATE, estiver contido na máscara de bits, então passamos para a próxima etapa, caso contrário, obtemos o número do registro da próxima chave no índice INVOICE_IDX_DATE;
4. extraímos o registro da tabela e verificamos sua visibilidade. Se após a extração (fetch) do registro, descobrirmos que ele não está visível para nós, então pegamos a próxima chave do índice INVOICE_IDX_DATE, caso contrário, retornamos o valor necessário na saída e finalizamos a execução do algoritmo.

Assim, o uso de um predicado de filtro indexado não leva a uma degradação de desempenho, como no caso do não indexado.

7.7.6. Predicado de filtro indexado pelo mesmo índice

O predicado de filtro pode usar o mesmo índice que é usado para calcular MIN/MAX. Neste caso, para a navegação por índice, são criados limites inferior/superior de varredura. Tais filtros podem acelerar o cálculo de MIN/MAX nos casos em que a primeira chave do índice não está visível, ou quando também é usado um predicado não indexado (no pior caso, será necessário extrair não todos os registros de todo o índice, mas apenas para as chaves que satisfazem o predicado de filtro). Vamos dar o seguinte exemplo:

```
SELECT MIN(INVOICE_DATE)
FROM INVOICE
WHERE INVOICE_DATE > date '2015-06-01';
```

Plano e estatísticas de execução:

```
PLAN (INVOICE ORDER INVOICE_IDX_DATE)
```

```
Select Expression
```

```
-> Aggregate
```

```
-> Filter
```

```
-> Table "INVOICE" Access By ID
```

```
-> Index "INVOICE_IDX_DATE" Range Scan (lower bound: 1/1)
```

```
MIN
```

```
=====
2015-06-01 00:03:00.0000
```

```
Current memory = 284932688
```

```
Delta memory = 192
```

```
Max memory = 284959760
```

```
Elapsed time = 0.002 sec
```

```
Buffers = 32768
```

```
Reads = 0
```

```
Writes = 0
```

```
Fetches = 6
```

```
Per table statistics:
```

```
-----+-----+-----+-----+-----+-----+
Table name | Natural | Index | Insert | Update | Delete |
```

	+	+	+	+	+
INVOICE			2		
	+	+	+	+	+



Neste caso, não há o problema com chaves NULL no índice INVOICE_IDX_DATE, descrito acima, pois o predicado de filtro não permite valores NULL.

7.7.7. Predicado de filtro indexado por um segmento do mesmo índice

Foi dito anteriormente que o otimizador pode usar navegação por índice composto para calcular MIN/MAX apenas se MIN/MAX forem calculados na primeira coluna do índice composto. Mas se um predicado de filtro for usado para a parte inicial da chave do índice composto (correspondência completa em um dos segmentos), então MIN/MAX pode usar navegação por índice se MIN/MAX forem calculados na próxima coluna que permanece após os segmentos correspondentes. Ou seja, se temos um índice nos campos Field1, Field2, Field3, então se

- Sem condições adicionais. A navegação por índice pode ser usada para MIN(Field1), mas não para MIN(Field2) ou MIN(Field3). O mesmo se aplica a MAX.
- WHERE Field1 = <expr>, então a navegação por índice pode ser usada para MIN(Field1) ou MIN(Field2), mas não para MIN(Field3). O mesmo se aplica a MAX.
- WHERE Field1 = <expr> AND Field2 = <expr>, então a navegação por índice pode ser usada para MIN(Field1) ou MIN(Field2) ou MIN(Field3). O mesmo se aplica a MAX.

Vamos demonstrar isso com o seguinte exemplo:

```
CREATE INDEX NAMEX ON EMPLOYEE (LAST_NAME, FIRST_NAME);

SELECT
  MIN(LAST_NAME)
FROM
  EMPLOYEE;
```

O plano de execução será o seguinte:

```
PLAN (EMPLOYEE ORDER NAMEX)

Select Expression
  -> Aggregate
    -> Table "EMPLOYEE" Access By ID
      -> Index "NAMEX" Full Scan
```

Para a consulta na coluna FIRST_NAME:

```
SELECT
  MIN(FIRST_NAME)
FROM
```

```
EMPLOYEE;
```

```
PLAN (EMPLOYEE NATURAL)
```

```
Select Expression
```

```
-> Aggregate
```

```
-> Table "EMPLOYEE" Full Scan
```

Agora, vamos modificar a segunda consulta para que haja um filtro na coluna LAST_NAME usando correspondência completa.

```
SELECT
  MIN(FIRST_NAME)
FROM
  EMPLOYEE
WHERE LAST_NAME = 'Baldwin';
```

Plano de execução:

```
PLAN (EMPLOYEE ORDER NAMEX)
```

```
Select Expression
```

```
-> Aggregate
```

```
-> Filter
```

```
-> Table "EMPLOYEE" Access By ID
```

```
-> Index "NAMEX" Range Scan (partial match: 1/2)
```

É importante notar que o uso de navegação por índice para MIN/MAX, aplicado à parte restante das colunas, só pode ser usado se houver correspondência completa no segmento inicial. Se modificarmos a consulta da seguinte forma, a navegação por índice não será usada:

```
SELECT
  MIN(FIRST_NAME)
FROM
  EMPLOYEE
WHERE LAST_NAME > 'Baldwin';
```

Plano de execução:

```
PLAN (EMPLOYEE INDEX (NAMEX))
```

```
Select Expression
```

```
-> Aggregate
```

```
-> Filter
```

```
-> Table "EMPLOYEE" Access By ID
```

```
-> Bitmap
```

```
-> Index "NAMEX" Range Scan (lower bound: 1/2)
```


Como você pode ver, não há navegação por índice aqui.

7.7.8. Cálculo de MIN/MAX com agrupamento

Como só é possível navegação por um único índice e a ordenação externa geralmente não funciona em conjunto com a navegação por índice, o uso da navegação por índice para calcular MIN/MAX em conjunto com agrupamento só é possível se o agrupamento for realizado usando o mesmo índice que o cálculo de MIN/MAX. Para índices simples, tais consultas não fazem sentido. De fato, vejamos o seguinte exemplo:

```
SELECT MIN(INVOICE_DATE)
FROM INVOICE
GROUP BY INVOICE_DATE;
```

Plano de execução:

```
PLAN (INVOICE ORDER INVOICE_IDX_DATE)

Select Expression
  -> Aggregate
    -> Table "INVOICE" Access By ID
      -> Index "INVOICE_IDX_DATE" Full Scan
```

A consulta usa navegação por índice, mas é improvável que você encontre uma aplicação prática para tais consultas. Normalmente, o agrupamento e o cálculo de agregados são realizados em colunas diferentes.

No entanto, o cálculo de MIN/MAX com agrupamento por outra coluna usando navegação por índice é possível ao usar um índice composto. Neste caso, para usar o índice composto, as seguintes condições devem ser atendidas:

- a cláusula GROUP BY usa o segmento inicial do índice composto;
- em MIN/MAX é usada a próxima coluna, que está incluída no índice composto e segue as colunas usadas no GROUP BY.

Vejamos a seguinte consulta:

```
SELECT
  CUSTOMER_ID,
  MIN(INVOICE_DATE)
FROM INVOICE
GROUP BY CUSTOMER_ID;
```

A tabela INVOICE tem os seguintes índices:

```
ALTER TABLE INVOICE ADD CONSTRAINT FK_INVOICE_CUSTOMER
FOREIGN KEY (CUSTOMER_ID) REFERENCES CUSTOMER (CUSTOMER_ID);
```

```
CREATE INDEX INVOICE_IDX_DATE ON INVOICE (INVOICE_DATE);
```

O plano e as estatísticas de execução da consulta serão os seguintes:

```
PLAN (INVOICE ORDER FK_INVOICE_CUSTOMER)
```

```
Select Expression
```

```
-> Aggregate
```

```
-> Table "INVOICE" Access By ID
```

```
-> Index "FK_INVOICE_CUSTOMER" Full Scan
```

```
Current memory = 285293568
```

```
Delta memory = 192
```

```
Max memory = 285310688
```

```
Elapsed time = 0.070 sec
```

```
Buffers = 32768
```

```
Reads = 0
```

```
Writes = 0
```

```
Fetches = 262880
```

```
Per table statistics:
```

Table name	Natural	Index	Insert	Update	Delete
INVOICE		100010			

Neste caso, o índice FK_INVOICE_CUSTOMER é usado para o agrupamento, mas a função MIN é executada por meio de uma varredura comum para cada grupo. Vamos tentar criar um índice composto:

```
CREATE INDEX INVOICE_IDX_CUSTOMER_DATE ON INVOICE (CUSTOMER_ID, INVOICE_DATE);
```

Vamos executar novamente a consulta original e observar seu plano e estatísticas:

```
PLAN (INVOICE ORDER FK_INVOICE_CUSTOMER)
```

```
Select Expression
```

```
-> Aggregate
```

```
-> Table "INVOICE" Access By ID
```

```
-> Index "FK_INVOICE_CUSTOMER" Full Scan
```

```
Current memory = 284759296
```

```
Delta memory = 192
```

```
Max memory = 289777408
```

```
Elapsed time = 0.072 sec
```

```
Buffers = 32768
```

```
Reads = 0
```

```
Writes = 0
```

```
Fetches = 262880
```

```
Per table statistics:
```

Table name	Natural	Index	Insert	Update	Delete
INVOICE		100010			

INVOICE			100010			
-----	+	+	-----	+	+	-----

O otimizador ignorou nosso índice. Vamos provar que ele pode ser usado, especificando explicitamente o plano:

```
SELECT
  CUSTOMER_ID,
  MIN(INVOICE_DATE)
FROM INVOICE
GROUP BY CUSTOMER_ID
PLAN (INVOICE ORDER INVOICE_IDX_CUSTOMER_DATE);
```

O plano e as estatísticas serão assim:

```
PLAN (INVOICE ORDER INVOICE_IDX_CUSTOMER_DATE)
```

```
Select Expression
  -> Aggregate
      -> Table "INVOICE" Access By ID
          -> Index "INVOICE_IDX_CUSTOMER_DATE" Full Scan
Current memory = 284868544
Delta memory = 240
Max memory = 289777408
Elapsed time = 0.084 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 262986
Per table statistics:
```

-----	+	+	-----	+	+	-----	+
Table name		Natural		Index		Insert	
-----	+	+	-----	+	+	-----	+
INVOICE				100010			
-----	+	+	-----	+	+	-----	+

O índice pode ser usado, mas seu uso para todos os registros da tabela não trouxe benefício. De fato, em ambas as variantes, todos os registros da tabela INVOICE são percorridos na ordem das chaves do índice. Mas o índice composto INVOICE_IDX_CUSTOMER_DATE é menos compacto. Para FK_INVOICE_CUSTOMER — leaf buckets: 50, e para INVOICE_IDX_CUSTOMER_DATE — leaf buckets: 110, ou seja, não há sentido em criar um índice composto neste caso.

7.7.9. Cálculo de MIN/MAX em consultas com junções de tabelas

Até agora, consideramos o cálculo das funções agregadas MIN/MAX em consultas SQL com uma única tabela. Agora, vamos ver se MIN/MAX podem usar a navegação por índice em consultas com junção (JOIN) de várias tabelas. Vamos tentar executar as seguintes consultas e observar seus planos e estatísticas.

```

SELECT
  MIN(COLOR.NAME)
FROM
  HORSE H
  JOIN COLOR ON COLOR.CODE_COLOR = H.CODE_COLOR;

```

O plano e as estatísticas de execução da consulta serão os seguintes:

```
PLAN JOIN (COLOR ORDER UNQ_COLOR_NAME, H INDEX (FK_HORSE_COLOR))
```

Select Expression

-> Aggregate

-> Nested Loop Join (inner)

-> Table "COLOR" Access By ID

-> Index "UNQ_COLOR_NAME" Full Scan

-> Filter

-> Table "HORSE" as "H" Access By ID

-> Bitmap

-> Index "FK_HORSE_COLOR" Range Scan (full match)

Current memory = 553841216

Delta memory = 208

Max memory = 553930448

Elapsed time = 0.004 sec

Buffers = 32768

Reads = 0

Writes = 0

Fetches = 19

Per table statistics:

Table name	Natural	Index	Insert	Update	Delete
COLOR		1			
HORSE		1			

A navegação por índice é usada. As estatísticas mostram um desempenho muito bom.

Agora, vamos tentar calcular o MIN da outra tabela.

```

SELECT
  MIN(H.NAME)
FROM
  HORSE H
  JOIN COLOR ON COLOR.CODE_COLOR = H.CODE_COLOR;

```

O plano e as estatísticas de execução da consulta serão os seguintes:

```
PLAN JOIN (H ORDER HORSE_IDX_NAME, COLOR INDEX (PK_COLOR))
```

```

Select Expression
  -> Aggregate
    -> Nested Loop Join (inner)
      -> Table "HORSE" as "H" Access By ID
        -> Index "HORSE_IDX_NAME" Full Scan
      -> Filter
        -> Table "COLOR" Access By ID
          -> Bitmap
            -> Index "PK_COLOR" Unique Scan

```

Current memory = 553885984

Delta memory = 208

Max memory = 553930448

Elapsed time = 0.002 sec

Buffers = 32768

Reads = 0

Writes = 0

Fetches = 803

Per table statistics:

Table name	Natural	Index	Insert	Update	Delete
COLOR		136			
HORSE		136			

A navegação por índice é usada novamente. As estatísticas são um pouco piores, mas ainda excelentes.

Lembro que experimentos semelhantes com GROUP BY e ORDER BY mostravam algo diferente. Neles, a ordem da junção era escolhida antes do agrupamento/ordenação, e por isso a navegação por índice era usada apenas se tivéssemos sorte com a ordem de escolha da tabela principal. Mas, neste caso, a ordem da junção se ajustou por si só, ou seja, a tabela principal escolhida foi aquela para a qual a navegação por índice era possível. Mas por que isso aconteceu? O fato é que as funções agregadas MIN/MAX, na presença de um índice adequado e sem agrupamentos adicionais, implicitamente mudam o objetivo de otimização para FIRST ROWS. De fato, na maioria dos casos, não são percorridos todos os registros da tabela para a qual há um índice para navegação, mas apenas o primeiro registro pela chave do índice. Nesse caso, é melhor otimizar a consulta para que ela retorne o primeiro registro pela chave o mais rápido possível, e é isso que é feito.

7.8. Conclusão

Neste capítulo, examinamos todos os casos possíveis de uso de índices:

- filtragem;
- condições de junção em JOIN;
- ordenação em ORDER BY;
- agrupamento em GROUP BY;
- cálculo das funções agregadas MIN e MAX.

Em cada caso, foi explicada a condição de aplicação dos índices, e exemplos foram fornecidos. Em alguns casos, foi explicado o custo de uso do índice e as alternativas. Espero que, após a leitura deste capítulo, muitas questões relacionadas ao uso de índices no Firebird fiquem mais claras. A seguir, falaremos sobre outras partes importantes do otimizador do Firebird, em muitas das quais retornaremos ao uso de índices.

Capítulo 8. Otimização de junções (JOIN)

A junção (JOINS) de fluxos de dados é a mais complexa do ponto de vista da otimização. Vou listar as principais razões pelas quais isso acontece:

- A junção de dois fluxos pode ser executada por diferentes algoritmos (Nested Loop, Hash, Merge). Qual dos algoritmos é o mais vantajoso depende de muitos fatores;
- As próprias junções podem ser de diferentes tipos: INNER JOIN, LEFT/RIGHT/FULL JOIN e variantes implícitas como SEMI JOIN.
- Para junções internas (INNER JOIN) de dois fluxos, os fluxos líder e seguidor são equivalentes. Ou seja, a escolha do fluxo líder não afeta o resultado final, mas afeta o custo da própria junção;
- Qualquer filtro, usando ou não um índice, aplicado a um dos fluxos da junção, afeta a cardinalidade do fluxo e, conseqüentemente, a escolha do fluxo líder/seguiror para junções internas;
- A junção pode envolver mais de dois fluxos de dados. Nesse caso, a ordem das junções pode ter um enorme impacto no desempenho;
- Como fluxo de dados, não apenas tabelas podem ser usadas, mas também fontes de dados mais complexas, por exemplo, expressões de tabela (VIEW, CTE, tabelas derivadas);
- As junções nem sempre são especificadas explicitamente, ou seja, usando a palavra-chave JOIN. As relações entre fluxos de dados podem ser indicadas na cláusula WHERE, e não em ON;
- Alguns tipos de junções podem surgir como resultado de transformações da consulta SQL pelo otimizador:
 - transformação de OUTER JOIN em INNER JOIN;
 - transformação de IN/EXISTS em SEMI JOIN.

Como você já deve ter percebido, a tarefa de descrever tudo isso não é a mais simples. Começaremos descrevendo várias opções de junção de dois fluxos e depois tentaremos mostrar exemplos com três fluxos de dados.

8.1. Junção de duas tabelas

8.1.1. INNER JOIN de duas tabelas

Vamos considerar um exemplo de junção interna de duas tabelas:

```
SELECT COUNT(*)  
FROM  
  HORSE  
  JOIN COLOR ON COLOR.CODE_COLOR = HORSE.CODE_COLOR;
```

Como para junções internas os fluxos são equivalentes, não há diferença na ordem em que as tabelas são especificadas na consulta, ou seja, a consulta acima é completamente equivalente a:

```
SELECT COUNT(*)
FROM
  COLOR
  JOIN HORSE ON HORSE.CODE_COLOR = COLOR.CODE_COLOR;
```

Além disso, essa mesma consulta pode ser escrita usando JOIN implícito na sintaxe SQL-89:

```
SELECT COUNT(*)
FROM
  HORSE, COLOR
WHERE COLOR.CODE_COLOR = HORSE.CODE_COLOR;
```

Se a relação entre os fluxos não fosse descrita na cláusula WHERE, seria um CROSS JOIN. Portanto, essa consulta também pode ser escrita assim:

```
SELECT COUNT(*)
FROM
  HORSE CROSS JOIN COLOR
WHERE COLOR.CODE_COLOR = HORSE.CODE_COLOR;
```

É altamente recomendável usar a junção interna INNER JOIN na sintaxe SQL-92, onde a condição de relação entre os fluxos é explicitamente indicada na cláusula ON. No entanto, é importante saber identificar a condição de relação entre os fluxos quando ela está presente na cláusula WHERE. Isso é especialmente importante em junções de várias tabelas, quando tais condições de relação surgem inadvertidamente.

Todas as consultas acima retornarão o mesmo resultado e serão executadas com o mesmo plano de execução escolhido pelo otimizador.

Mesmo uma consulta tão simples pode ser executada das seguintes maneiras:

1. "HORSE" NESTED LOOP JOIN "COLOR" com índice em COLOR.CODE_COLOR;
2. "HORSE" NESTED LOOP JOIN "COLOR" sem uso de índice;
3. "HORSE" HASH JOIN "COLOR"
4. "HORSE" MERGE JOIN "COLOR"
5. "COLOR" NESTED LOOP JOIN "HORSE" com índice em HORSE.CODE_COLOR;
6. "COLOR" NESTED LOOP JOIN "HORSE" sem uso de índice;
7. "COLOR" HASH JOIN "HORSE"
8. "COLOR" MERGE JOIN "HORSE"

Como descrito em [Junções \(Joins\)](#), a junção pode ser executada por um dos três algoritmos conhecidos: Nested Loop, Hash Join, Merge Join. A ordem de junção das tabelas também pode ser diferente. A junção pelo método Nested Loop pode usar ou não usar um índice para filtrar o fluxo interno. Como você pode ver, a otimização mesmo da junção de duas tabelas não é tão simples.

Os próprios algoritmos, seu custo e as condições sob as quais são mais eficazes foram descritos anteriormente. Aqui, veremos como influenciar o plano de execução da consulta se ele for considerado “ruim”.

Se o desempenho da consulta não for muito bom e você suspeitar que o plano escolhido seja o culpado, não se apresse em modificar a consulta em si. Primeiro, verifique os casos mais óbvios:

- Existem índices para os campos das tabelas e expressões que são usados na condição de junção?
- Suas estatísticas estão atualizadas para todos os índices que podem de alguma forma ser usados na consulta?

Índices para campos e expressões

Neste caso, está tudo bem, temos dois índices que são usados para as restrições de chave primária e estrangeira:

```
ALTER TABLE HORSE
ADD CONSTRAINT FK_HORSE_COLOR FOREIGN KEY (CODE_COLOR) REFERENCES COLOR (CODE_COLOR);

ALTER TABLE COLOR ADD CONSTRAINT PK_COLOR PRIMARY KEY (CODE_COLOR);
```

Esse ponto nem sempre é óbvio se a junção entre as tabelas não ocorrer por chaves (Primary/Foreign Key).

Por exemplo, na consulta:

```
SELECT COUNT(*)
FROM BREED
JOIN DEPARTURE ON DEPARTURE.NAME = BREED.NAME;
```

Aqui, o campo NAME em ambas as tabelas não é uma chave primária nem estrangeira, portanto, a existência de um índice para esse campo não é óbvia.

Recalcular estatísticas dos índices

Se todos os índices necessários existirem, tente atualizar suas estatísticas:

```
SET STATISTICS INDEX FK_HORSE_COLOR;

SET STATISTICS INDEX PK_COLOR;
```

Dicas +0 ou ||' para o otimizador

Se todos os índices necessários forem criados e suas estatísticas estiverem atualizadas, mas o desempenho da consulta ainda for ruim, você pode adicionar “dicas” que alterarão o plano de execução da consulta. Suponha que você tenha a consulta:

```
SELECT COUNT(*)
FROM
  HORSE
  JOIN COLOR ON COLOR.CODE_COLOR = HORSE.CODE_COLOR;
```

O plano e as estatísticas de sua execução são os seguintes:

```
PLAN JOIN (COLOR NATURAL, HORSE INDEX (FK_HORSE_COLOR))
```

Select Expression

-> Aggregate

-> Nested Loop Join (inner)

-> Table "COLOR" Full Scan

-> Filter

-> Table "HORSE" Access By ID

-> Bitmap

-> Index "FK_HORSE_COLOR" Range Scan (full match)

Current memory = 552083376

Delta memory = 208

Max memory = 552411744

Elapsed time = 0.334 sec

Buffers = 32768

Reads = 0

Writes = 0

Fetches = 665083

Per table statistics:

Table name	Natural	Index	Insert	Update	Delete
COLOR	242				
HORSE		562237			

Você supõe que se a tabela líder for HORSE, a consulta será executada mais rapidamente. Nesse caso, você pode tentar desabilitar o uso do índice FK_HORSE_COLOR. Como se sabe, o algoritmo de junção Nested Loop Join é eficaz apenas se houver um índice para a tabela seguidora usado na condição de relação. Desabilitar esse índice pode levar a uma das duas opções:

- Os fluxos serão invertidos. A tabela líder se tornará HORSE e a seguidora — COLOR. Para filtrar a tabela COLOR, será usado o índice — PK_COLOR.
- Será escolhido outro algoritmo de junção, geralmente HASH JOIN.

Para desabilitar o índice construído em um campo do tipo INTEGER, usamos a expressão +0 para HORSE.CODE_COLOR.

```
SELECT COUNT(*)
FROM
  HORSE
  JOIN COLOR ON COLOR.CODE_COLOR = HORSE.CODE_COLOR+0;
```

Vamos ver o que obtivemos:

```
PLAN HASH (HORSE NATURAL, COLOR NATURAL)

Select Expression
  -> Aggregate
    -> Filter
      -> Hash Join (inner)
        -> Table "HORSE" Full Scan
        -> Record Buffer (record length: 25)
          -> Table "COLOR" Full Scan
```

```
      COUNT
=====
      562237
```

```
Current memory = 552118048
Delta memory = 208
Max memory = 552411744
Elapsed time = 0.380 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 601991
Per table statistics:
```

Table name	Natural	Index	Insert	Update	Delete
COLOR	242				
HORSE	562237				

Neste caso, foi escolhido o algoritmo de junção HASH JOIN, a tabela HORSE tornou-se a líder, COLOR — a seguidora. Os dados necessários da tabela COLOR foram colocados em uma tabela hash. Mas o desempenho da consulta quase não mudou.

Como mencionado acima, a aplicação da dica +0 nem sempre levará ao uso da junção hash; em alguns casos, os fluxos simplesmente serão invertidos. Isso depende de dois fatores:

- Do cumprimento da condição necessária para possibilitar a junção hash — a condição de relação entre os fluxos deve usar pelo menos um predicado de igualdade = ou equivalência IS NOT DISTINCT FROM;
- Do custo da junção hash calculado pelo otimizador, que deve ser menor que os métodos alternativos de junção Nested Loop Join e Merge Join. Este último algoritmo de junção é usado muito raramente, apenas quando ambos os fluxos sendo unidos são muito grandes.

Você não pode influenciar o primeiro fator sem alterar o significado semântico da consulta. No entanto, se o otimizador simplesmente trocou os fluxos e está usando Nested Loop Join para a junção com o índice PK_COLOR, você pode adicionar a dica +0 no predicado de junção para os campos de ambas as tabelas, ou seja:

```
SELECT COUNT(*)
FROM
  HORSE
  JOIN COLOR ON COLOR.CODE_COLOR+0 = HORSE.CODE_COLOR+0;
```

Para tal consulta, será obrigatoriamente escolhido um dos algoritmos Hash Join ou Merge Join.

Usando LEFT JOIN como uma dica

No exemplo acima, conseguimos mudar o algoritmo de junção de Nested Loop para Hash Join. Mas queríamos executar nossa junção usando o método Nested Loop, mas fazendo a tabela HORSE ser a líder, não a tabela COLOR. Como fazer isso?

A resposta é simples: podemos substituir a junção interna (INNER JOIN) por uma junção externa esquerda/direita (LEFT JOIN/RIGHT JOIN), que garante a ordem de junção dos dois fluxos. Como no Firebird 5.0 as junções externas são executadas apenas pelo algoritmo Nested Loop, isso é o que precisamos. No entanto, resta a pergunta: uma junção externa esquerda será semanticamente equivalente a uma junção interna? A resposta correta é nem sempre. Para garantir que uma junção externa esquerda retorne o mesmo resultado que uma junção interna, existe a seguinte receita:

```
SELECT *
FROM A JOIN B ON B.field_2 = A.field_1;

-- equivalente a
SELECT *
FROM A LEFT JOIN B ON B.field_2 = A.field_1
WHERE B.field_2 IS NOT NULL;
```

Na verdade, em vez de field_2, pode-se usar qualquer campo da tabela B que tenha a restrição NOT NULL. Mas B.field_2 IS NOT NULL funcionará corretamente, mesmo que não haja uma restrição NOT NULL para field_2. O ponto é que B.field_2 está presente na condição de junção das duas tabelas, e para essa condição é usado um predicado de igualdade (=), cuja execução automaticamente exclui valores NULL.

Assim, nossa consulta original pode ser reescrita assim:

```
SELECT COUNT(*)
FROM
  HORSE
  LEFT JOIN COLOR ON COLOR.CODE_COLOR = HORSE.CODE_COLOR
WHERE COLOR.CODE_COLOR IS NOT NULL;
```

Vejamos o plano e as estatísticas desta consulta:

```
PLAN JOIN (HORSE NATURAL, COLOR INDEX (PK_COLOR))

Select Expression
-> Aggregate
```

```

-> Filter
  -> Nested Loop Join (outer)
    -> Table "HORSE" Full Scan
    -> Filter
      -> Table "COLOR" Access By ID
        -> Bitmap
          -> Index "PK_COLOR" Unique Scan

```

COUNT

```

=====
562237

```

```

Current memory = 551760784
Delta memory = 240
Max memory = 551849984
Elapsed time = 0.755 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 2288452
Per table statistics:

```

Table name	Natural	Index	Insert	Update	Delete
COLOR		562237			
HORSE	562237				

Conseguimos a ordem desejada de junção das tabelas usando o método Nested Loop Join. Neste caso, o otimizador estava certo, e nossas suposições estavam erradas. No entanto, em alguns casos o otimizador erra, e com este método você pode influenciar o plano de execução.

Vale notar aqui que adicionar a filtragem `COLOR.CODE_COLOR IS NOT NULL` nem sempre é necessário. Se existir uma restrição `NOT NULL` para o campo `HORSE.CODE_COLOR` e uma `FOREIGN KEY` que corresponda à condição de relacionamento das tabelas, então o filtro `COLOR.CODE_COLOR IS NOT NULL` não é obrigatório. Porém, no caso geral, é melhor não fazer tais suposições.

Usando um plano explicitamente especificado

Outra maneira de influenciar o algoritmo e a ordem de junção das tabelas é especificar um plano de consulta explícito. Voltemos à consulta original.

```

SELECT COUNT(*)
FROM
  HORSE
  JOIN COLOR ON COLOR.CODE_COLOR = HORSE.CODE_COLOR;

```

O plano e as estatísticas de sua execução são os seguintes:

```

PLAN JOIN (COLOR NATURAL, HORSE INDEX (FK_HORSE_COLOR))

```

```

Select Expression
  -> Aggregate
    -> Nested Loop Join (inner)
      -> Table "COLOR" Full Scan
      -> Filter
        -> Table "HORSE" Access By ID
          -> Bitmap
            -> Index "FK_HORSE_COLOR" Range Scan (full match)

```

```

COUNT
=====
562237

```

```

Current memory = 552083376
Delta memory = 208
Max memory = 552411744
Elapsed time = 0.334 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 665083
Per table statistics:

```

Table name	Natural	Index	Insert	Update	Delete
COLOR	242				
HORSE		562237			

Só podemos especificar o plano no formato Legacy. Primeiro, copie o plano Legacy atual e adicione-o à consulta:

```

SELECT COUNT(*)
FROM
  HORSE
  JOIN COLOR ON COLOR.CODE_COLOR = HORSE.CODE_COLOR
PLAN JOIN (COLOR NATURAL, HORSE INDEX (FK_HORSE_COLOR));

```

Agora modificamos o plano da consulta para que a junção comece com a tabela HORSE, e na condição de junção seja usado o índice para o campo CODE_COLOR da tabela COLOR.

```

SELECT COUNT(*)
FROM
  HORSE
  JOIN COLOR ON COLOR.CODE_COLOR = HORSE.CODE_COLOR
PLAN JOIN (HORSE NATURAL, COLOR INDEX (PK_COLOR));

```

Vamos tentar executar a consulta com o plano explicitamente especificado e ver suas estatísticas:

```

PLAN JOIN (HORSE NATURAL, COLOR INDEX (PK_COLOR))

```

```

Select Expression
  -> Aggregate
    -> Nested Loop Join (inner)
      -> Table "HORSE" Full Scan
      -> Filter
        -> Table "COLOR" Access By ID
          -> Bitmap
            -> Index "PK_COLOR" Unique Scan

```

```

COUNT
=====
562237

```

```

Current memory = 554661984
Delta memory = 256
Max memory = 554941600
Elapsed time = 0.725 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 2288452
Per table statistics:

```

Table name	Natural	Index	Insert	Update	Delete
COLOR		562237			
HORSE	562237				

Como você pode ver, conseguimos o mesmo efeito que ao usar `LEFT JOIN`. No entanto, recomendo não usar este método, pois ele tem desvantagens significativas:

- No plano, os nomes dos índices são especificados explicitamente, o que cria dependências adicionais que não existem ao usar consultas comuns (em consultas SQL, os nomes dos índices não estão presentes). A exclusão de um índice pode tornar sua consulta com plano explicitamente especificado inoperante. Além disso, dependências extras podem impedir a exclusão de um índice se a consulta estiver especificada em um procedimento armazenado.
- O plano Legacy exige que você o escreva por completo. Para consultas complexas, isso pode ser muito difícil; no plano Legacy, é impossível especificar alguns métodos de acesso.

A sintaxe dos planos Legacy é descrita em detalhes no “Manual de Linguagem SQL do SGBD Firebird” no capítulo “Instruções de Manipulação de Dados (DML)”, instrução `SELECT`, cláusula `PLAN`.

Condições adicionais de filtragem

Nos exemplos acima, consideramos a junção de duas tabelas sem filtros adicionais na cláusula `WHERE`. Eles podem influenciar a ordem de junção das tabelas? Sim, claro.

A filtragem das tabelas por alguma condição (indexada ou não) altera a cardinalidade dos fluxos para a junção, o que significa que o otimizador pode alterar a ordem e o método de junção. Voltemos à nossa consulta original e adicionemos filtragem para a tabela `HORSE`.

```

SELECT COUNT(*)
FROM
  HORSE
  JOIN COLOR ON COLOR.CODE_COLOR = HORSE.CODE_COLOR
WHERE HORSE.CODE_HORSE = 742363;

```

```

PLAN JOIN (HORSE INDEX (PK_HORSE), COLOR INDEX (PK_COLOR))

```

Select Expression

-> Aggregate

-> Nested Loop Join (inner)

-> Filter

-> Table "HORSE" Access By ID

-> Bitmap

-> Index "PK_HORSE" Unique Scan

-> Filter

-> Table "COLOR" Access By ID

-> Bitmap

-> Index "PK_COLOR" Unique Scan

COUNT

```

=====
1

```

Current memory = 554702224

Delta memory = 240

Max memory = 554941600

Elapsed time = 0.002 sec

Buffers = 32768

Reads = 0

Writes = 0

Fetches = 7

Per table statistics:

Table name	Natural	Index	Insert	Update	Delete
COLOR		1			
HORSE		1			

Como você pode ver, a ordem de junção mudou. Agora a tabela HORSE foi filtrada pelo campo da chave primária, resultando em uma cardinalidade de fluxo igual a um, e o otimizador decidiu usar esse fluxo filtrado como líder.

No entanto, se filtrarmos a tabela HORSE por outro campo, cujo índice tenha seletividade menor, a ordem de junção pode não mudar. Por exemplo:

```

SELECT COUNT(*)
FROM
  HORSE
  JOIN COLOR ON COLOR.CODE_COLOR = HORSE.CODE_COLOR

```



```
WHERE HORSE.CODE_DEPARTURE = 1;
```

```
PLAN JOIN (COLOR NATURAL, HORSE INDEX (FK_HORSE_COLOR, FK_HORSE_DEPARTURE))
```

```
Select Expression
```

```
-> Aggregate
```

```
-> Nested Loop Join (inner)
```

```
-> Table "COLOR" Full Scan
```

```
-> Filter
```

```
-> Table "HORSE" Access By ID
```

```
-> Bitmap And
```

```
-> Bitmap
```

```
-> Index "FK_HORSE_COLOR" Range Scan (full match)
```

```
-> Bitmap
```

```
-> Index "FK_HORSE_DEPARTURE" Range Scan (full match)
```

```
COUNT
```

```
=====
100186
```

```
Current memory = 554739456
```

```
Delta memory = 240
```

```
Max memory = 555223344
```

```
Elapsed time = 0.618 sec
```

```
Buffers = 32768
```

```
Reads = 0
```

```
Writes = 0
```

```
Fetches = 130885
```

```
Per table statistics:
```

Table name	Natural	Index	Insert	Update	Delete
COLOR	242				
HORSE		100186			

Neste caso, a ordem da junção não mudou, a tabela HORSE é lida repetidamente usando dois índices FK_HORSE_COLOR e FK_HORSE_DEPARTURE. Aqui, parece que tudo está claro — a cardinalidade original da tabela HORSE será menor ao ler por algum índice ou índices. Se as cardinalidades das tabelas e a seletividade dos índices potenciais forem conhecidas, você pode calcular o custo de várias opções de junção. Com base nos resultados desses cálculos, o planejador escolhe um novo plano.

O mesmo acontecerá ao usar predicados de filtragem não indexados. Mas quando o predicado de filtragem não usa um índice, a estimativa da cardinalidade do fluxo será menos precisa. O fato é que para índices existe uma estatística armazenada — a seletividade do índice e a seletividade de seus segmentos. Se essa estatística estiver atualizada, a estimativa da cardinalidade será relativamente precisa. Ao mesmo tempo, para predicados de filtragem que não usam índices, a seletividade é determinada por uma constante no código, o que geralmente não corresponde à seletividade real. A situação é agravada pelo fato de que, ao usar vários predicados de filtragem não indexados, suas seletividades são simplesmente multiplicadas, enquanto ao usar vários predicados indexados, a seletividade total é calculada como se houvesse uma correlação de 50% entre as

condições de filtragem.

Vamos observar o comportamento do otimizador ao usar predicados não indexados.

```
SELECT COUNT(*)
FROM
  HORSE
  JOIN COLOR ON COLOR.CODE_COLOR = HORSE.CODE_COLOR
WHERE HORSE.BLOOD = '1/16';
```

```
PLAN JOIN (COLOR NATURAL, HORSE INDEX (FK_HORSE_COLOR))
```

Select Expression

-> Aggregate

-> Nested Loop Join (inner)

-> Table "COLOR" Full Scan

-> Filter

-> Table "HORSE" Access By ID

-> Bitmap

-> Index "FK_HORSE_COLOR" Range Scan (full match)

COUNT

=====

8512

Current memory = 555636288

Delta memory = 224

Max memory = 555984864

Elapsed time = 0.331 sec

Buffers = 32768

Reads = 0

Writes = 0

Fetches = 665087

Per table statistics:

Table name	Natural	Index	Insert	Update	Delete
COLOR	242				
HORSE		562237			

O plano e as estatísticas da consulta são quase os mesmos que sem a filtragem. E agora vamos adicionar mais um filtro:

```
SELECT COUNT(*)
FROM
  HORSE
  JOIN COLOR ON COLOR.CODE_COLOR = HORSE.CODE_COLOR
WHERE HORSE.BLOOD = '1/16'
  AND HORSE.FAMILY IS NULL;
```

```
PLAN JOIN (COLOR NATURAL, HORSE INDEX (FK_HORSE_COLOR))
```

```
Select Expression
```

```
-> Aggregate
```

```
-> Filter
```

```
-> Hash Join (inner)
```

```
-> Filter
```

```
-> Table "HORSE" Full Scan
```

```
-> Record Buffer (record length: 25)
```

```
-> Table "COLOR" Full Scan
```

```
COUNT
```

```
=====
```

```
8512
```

```
Current memory = 555656672
```

```
Delta memory = 256
```

```
Max memory = 555984864
```

```
Elapsed time = 0.215 sec
```

```
Buffers = 32768
```

```
Reads = 0
```

```
Writes = 0
```

```
Fetches = 601991
```

```
Per table statistics:
```

Table name	Natural	Index	Insert	Update	Delete
COLOR	242				
HORSE	562237				

A consulta retornou o mesmo número de registros, mas o plano mudou e as estatísticas melhoraram. Por que o otimizador alterou o plano?

No Firebird 5.0, não temos ferramentas para responder a essa pergunta. O fato é que, para cada predicado não indexado, o código do Firebird tem sua própria seletividade constante. Para o predicado `=`, ela é 0.1. Para o predicado `IS NULL`, também é 0.1. Assim, dois predicados de filtragem têm uma seletividade total de 0.01. O otimizador calculou que a cardinalidade do fluxo com a tabela HORSE tornou-se 100 vezes menor. Isso, por sua vez, influenciou a escolha de um algoritmo de junção diferente. Observo que a seletividade real do predicado `HORSE.BLOOD = '1/16'` é $8512 / 562237 = 0.015$. E o predicado `HORSE.FAMILY IS NULL` não filtrou nenhum registro adicional. Mas o otimizador do Firebird não sabe nada sobre a seletividade dos predicados não indexados e sobre a distribuição de valores neles.

Observo também que o otimizador não tem nenhum dado sobre a distribuição de valores nas chaves do índice (considera-se que os valores são distribuídos uniformemente), mas há pelo menos alguma informação estatística. Assim, antes de dar dicas ao otimizador e transformar suas consultas, tente o seguinte:

- tente evitar predicados de filtragem não indexados e, se possível, tente criar o índice correspondente. Isso não só pode acelerar a própria filtragem, mas também alterar o plano subsequente, incluindo a ordem e o algoritmo de junção, pois permitirá uma estimativa mais

precisa da cardinalidade;

- considere a possibilidade de criar um índice composto para filtrar por vários campos. Como se sabe, para índices compostos também é armazenada a seletividade do índice e de seus segmentos. Isso permitirá que o otimizador tenha uma estimativa mais precisa da cardinalidade. Isso se explica pelo fato de que o cálculo da seletividade da interseção de máscaras de bits ao usar vários índices é menos preciso do que simplesmente a seletividade de um índice composto.

Também deve-se notar que qualquer deficiência do otimizador pode ser aproveitada. No exemplo acima, adicionamos um filtro que não altera o resultado, mas mudou o plano da consulta. Isso significa que podemos usar esse efeito como uma dica. Claro, nem sempre é possível encontrar tal predicado.

8.1.2. LEFT/RIGHT JOIN de duas tabelas

Acima, vimos como podemos influenciar a ordem e o algoritmo da junção interna de duas tabelas. Agora vamos ver o que podemos fazer com junções externas. Começaremos com junções unidirecionais LEFT/RIGHT JOIN.

Vou considerar apenas a junção externa esquerda. O fato é que as junções direitas são sempre convertidas em esquerdas no nível de análise do BLR (veja [Junções \(Joins\)](#)). Você sempre pode converter uma junção direita em esquerda, se necessário.

Então, voltemos ao nosso exemplo com duas tabelas:

```
SELECT COUNT(*)
FROM
  HORSE
  LEFT JOIN COLOR ON COLOR.CODE_COLOR = HORSE.CODE_COLOR;
```

Esta consulta pode ser executada pelas seguintes variantes

1. "HORSE" NESTED LOOP JOIN "COLOR" com índice em COLOR.CODE_COLOR;
2. "HORSE" NESTED LOOP JOIN "COLOR" sem usar o índice;

E isso é tudo. Em primeiro lugar, em junções externas unidirecionais, não podemos alterar a ordem de junção das tabelas. Em segundo lugar, no Firebird 5.0, as junções externas só podem ser executadas pelo algoritmo Nested Loop (nas próximas versões do Firebird, as junções externas também poderão ser executadas por outros algoritmos).

Como existe um índice para COLOR.CODE_COLOR, essa variante é definitivamente mais barata do que Nested Loop sem usar o índice. Ou seja, para LEFT JOIN não há espaço para o otimizador — a junção é sempre executada pelo algoritmo Nested Loop e, quando possível, usa o índice nos campos de relacionamento.

As estatísticas e o plano de execução da consulta acima:

```
PLAN JOIN (HORSE NATURAL, COLOR INDEX (PK_COLOR))
```

```

Select Expression
  -> Aggregate
    -> Nested Loop Join (outer)
      -> Table "HORSE" Full Scan
      -> Filter
        -> Table "COLOR" Access By ID
          -> Bitmap
            -> Index "PK_COLOR" Unique Scan

```

```

COUNT
=====
562237

```

```

Current memory = 555754480
Delta memory = 208
Max memory = 556003904
Elapsed time = 0.718 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 2288452
Per table statistics:

```

Table name	Natural	Index	Insert	Update	Delete
COLOR		562237			
HORSE	562237				

Condições Adicionais de Filtragem

Ao considerar a junção interna de duas tabelas, vimos que predicados adicionais de filtragem na cláusula WHERE podem influenciar a escolha da ordem e do algoritmo de junção. Mas como ficam as junções externas unidirecionais?

Aqui tudo depende de em qual tabela exatamente as condições de filtragem são aplicadas:

- Um filtro aplicado à tabela líder não pode alterar a ordem e o algoritmo de execução da junção;
- Se o filtro for aplicado à tabela seguidora, então, dependendo do valor do parâmetro de configuração `OuterJoinConversion`, o comportamento será diferente:
 - `OuterJoinConversion = true` e o predicado de filtragem não considera valores NULL (todos os predicados de filtragem exceto `IS [NOT] NULL` e `IS [NOT] DISTINCT FROM`) — a junção externa unidirecional é convertida em uma junção interna, após a qual o otimizador escolhe a ordem e o algoritmo de junção;
 - `OuterJoinConversion = false` — o predicado de filtragem não influencia a ordem e o algoritmo de junção das tabelas.

Por padrão, `OuterJoinConversion = true`. Vamos ver com exemplos como um filtro adicional no WHERE influenciará o plano e as estatísticas de execução da consulta.

```

SELECT COUNT(*)
FROM
  HORSE
  LEFT JOIN COLOR ON COLOR.CODE_COLOR = HORSE.CODE_COLOR
WHERE HORSE.CODE_DEPARTURE = 1;

```

```

PLAN JOIN (HORSE INDEX (FK_HORSE_DEPARTURE), COLOR INDEX (PK_COLOR))

```

Select Expression

-> Aggregate

-> Filter

-> Nested Loop Join (outer)

-> Filter

-> Table "HORSE" Access By ID

-> Bitmap

-> Index "FK_HORSE_DEPARTURE" Range Scan (full match)

-> Filter

-> Table "COLOR" Access By ID

-> Bitmap

-> Index "PK_COLOR" Unique Scan

Aqui a tabela líder HORSE é lida usando o índice FK_HORSE_DEPARTURE, mas a junção permaneceu externa, seu algoritmo não mudou (Nested Loop Join), assim como a ordem da junção também não mudou.

E agora vamos aplicar o predicado de filtragem à tabela COLOR:

```

SELECT COUNT(*)
FROM
  HORSE
  LEFT JOIN COLOR ON COLOR.CODE_COLOR = HORSE.CODE_COLOR
WHERE COLOR.NAME = 'black';

```

Select Expression

-> Aggregate

-> Nested Loop Join (inner)

-> Filter

-> Table "COLOR" Access By ID

-> Bitmap

-> Index "UNQ_COLOR_NAME" Unique Scan

-> Filter

-> Table "HORSE" Access By ID

-> Bitmap

-> Index "FK_HORSE_COLOR" Range Scan (full match)

COUNT

=====

35524

Current memory = 556250880

```
Delta memory = 240
Max memory = 556251088
Elapsed time = 0.025 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 43845
Per table statistics:
```

Table name	Natural	Index	Insert	Update	Delete
COLOR		1			
HORSE		35524			

O que aconteceu? É muito simples: a junção externa foi convertida em interna, pois produz um resultado equivalente. Em seguida, o otimizador ficou mais livre para escolher a ordem e o algoritmo de junção. Tais transformações são descritas em [Transformação de OUTER JOIN em INNER JOIN](#).

No entanto, se alterarmos o predicado de filtragem para um que considere valores NULL, a transformação não ocorrerá.

```
SELECT COUNT(*)
FROM
  HORSE
  LEFT JOIN COLOR ON COLOR.CODE_COLOR = HORSE.CODE_COLOR
WHERE COLOR.NAME IS NOT NULL;
```

```
PLAN JOIN (HORSE NATURAL, COLOR INDEX (PK_COLOR))
```

```
Select Expression
```

```
-> Aggregate
```

```
-> Filter
```

```
-> Nested Loop Join (outer)
```

```
-> Table "HORSE" Full Scan
```

```
-> Filter
```

```
-> Table "COLOR" Access By ID
```

```
-> Bitmap
```

```
-> Index "PK_COLOR" Unique Scan
```

```
COUNT
```

```
=====
562237
```

```
Current memory = 556267504
Delta memory = 240
Max memory = 556321264
Elapsed time = 0.723 sec
Buffers = 32768
Reads = 0
Writes = 0
```

Fetches = 2288452

Per table statistics:

Table name	Natural	Index	Insert	Update	Delete
COLOR		562237			
HORSE	562237				

E se for necessário evitar a transformação de junções externas?

Geralmente, a transformação de junções externas afeta positivamente o desempenho da consulta, mas há casos em que a junção externa é especificada intencionalmente como uma dica para a ordem de junção das tabelas. Isso pode ser necessário se as estatísticas disponíveis para o otimizador não forem suficientemente precisas para escolher a ordem e o algoritmo de junção corretos. Existem duas maneiras de evitar a conversão de uma junção externa em interna.

A primeira maneira é simplesmente alterar o valor do parâmetro de configuração `OuterJoinConversion` para `false`. A desvantagem deste método é que ele atua globalmente, ou seja, desativará todas as conversões de junções externas em internas, mesmo aquelas que são benéficas.

A segunda maneira é transformar a consulta de forma a mover a condição de filtragem para `ON`, e adicionar um filtro `IS NOT NULL` no `WHERE`. Vamos ver como isso funciona. Vamos reescrever a consulta:

```
SELECT COUNT(*)
FROM
  HORSE
  LEFT JOIN COLOR ON COLOR.CODE_COLOR = HORSE.CODE_COLOR
WHERE COLOR.NAME = 'black';
```

na seguinte forma equivalente:

```
SELECT COUNT(*)
FROM
  HORSE
  LEFT JOIN COLOR
    ON COLOR.CODE_COLOR = HORSE.CODE_COLOR AND COLOR.NAME = 'black'
WHERE COLOR.CODE_COLOR IS NOT NULL;
```

O plano e as estatísticas de execução serão os seguintes:

```
PLAN JOIN (HORSE NATURAL, COLOR INDEX (PK_COLOR))
```

```
Select Expression
  -> Aggregate
    -> Filter
      -> Nested Loop Join (outer)
        -> Table "HORSE" Full Scan
        -> Filter
```



```
-> Table "COLOR" Access By ID
-> Bitmap
-> Index "PK_COLOR" Unique Scan
```

```
COUNT
=====
35524
```

```
Current memory = 556303168
Delta memory = 288
Max memory = 556355104
Elapsed time = 0.921 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 2288452
Per table statistics:
```

Table name	Natural	Index	Insert	Update	Delete
COLOR		562237			
HORSE	562237				

Conseguimos alcançar o desejado—impor ao otimizador nossa ordem de junção, mantendo o significado semântico da consulta. A desvantagem deste método é a complexidade de tais transformações para os desenvolvedores de aplicativos.



O último exemplo não melhorou o desempenho da consulta. Este é um exemplo simples de supressão da transformação de junções externas em internas, que você pode usar em consultas mais complexas, se necessário.

8.1.3. FULL JOIN de duas tabelas

Como é sabido, o Firebird não possui um algoritmo de passagem única separado para executar FULL OUTER JOIN; em vez disso, uma consulta com FULL JOIN é transformada em duas consultas, cujos resultados são unidos usando UNION ALL (consulte [Full Outer Join](#)).

Vejamos o seguinte exemplo:

```
SELECT
  COUNT(*)
FROM
  HORSE
  FULL JOIN COLOR ON COLOR.CODE_COLOR = HORSE.CODE_COLOR;
```

Seu plano e estatísticas são:

```
PLAN JOIN (JOIN (COLOR NATURAL, HORSE INDEX (FK_HORSE_COLOR)), JOIN (HORSE NATURAL, COLOR INDEX (PK_COLOR)))
```

```

Select Expression
  -> Aggregate
    -> Full Outer Join
      -> Nested Loop Join (outer)
        -> Table "COLOR" Full Scan
        -> Filter
          -> Table "HORSE" Access By ID
            -> Bitmap
              -> Index "FK_HORSE_COLOR" Range Scan (full match)
      -> Nested Loop Join (outer)
        -> Table "HORSE" Full Scan
        -> Filter
          -> Table "COLOR" Access By ID
            -> Bitmap
              -> Index "PK_COLOR" Unique Scan

```

COUNT

```

=====
562251

```

Current memory = 556291152

Delta memory = 208

Max memory = 556642032

Elapsed time = 1.070 sec

Buffers = 32768

Reads = 0

Writes = 0

Fetches = 2953539

Per table statistics:

Table name	Natural	Index	Insert	Update	Delete
COLOR	242	562237			
HORSE	562237	562237			

Na verdade, esta consulta é executada da seguinte forma (SQL equivalente):

```

SELECT COUNT(*) FROM (
  SELECT 1 AS N
  FROM
    COLOR
    LEFT JOIN HORSE ON HORSE.CODE_COLOR = COLOR.CODE_COLOR
  UNION ALL
  SELECT 1 AS N
  FROM
    HORSE
    LEFT JOIN COLOR ON COLOR.CODE_COLOR = HORSE.CODE_COLOR
  WHERE COLOR.CODE_COLOR IS NULL
);

```

Como você já deve ter adivinhado, desta forma o otimizador tem poucas opções de otimização — a ordem da junção não tem grande importância, cada LEFT JOIN só pode ser executado usando Nested

Loop Join. A única coisa que você pode influenciar é o uso de índices no predicado de junção das tabelas. Para uma execução eficiente de FULL JOIN, é necessário que existam índices tanto para HORSE.CODE_COLOR quanto para COLOR.CODE_COLOR.

Condições adicionais de filtragem

Assim como no caso de junções externas unilaterais, com OuterJoinConversion ativado, condições adicionais de filtragem para tabelas no WHERE podem levar a transformações da consulta. E para FULL OUTER JOIN as regras de transformação são um pouco mais complexas. Para uma determinada consulta:

```
SELECT *
FROM A FULL JOIN B ON <expr>
```

- Um predicado no WHERE, que não considera NULL, aplicado à tabela A, levará à transformação em A LEFT JOIN B;
- Um predicado no WHERE, que não considera NULL, aplicado à tabela B, levará à transformação em A RIGHT JOIN B;
- Predicados no WHERE, que não consideram NULL, aplicados às tabelas A e B, levarão à transformação em A INNER JOIN B.

Vejamos como isso funciona nos seguintes exemplos:

```
SELECT COUNT(*)
FROM
  HORSE
  FULL JOIN COLOR ON COLOR.CODE_COLOR = HORSE.CODE_COLOR
WHERE HORSE.NAME = 'A Gambler';
```

O plano e as estatísticas de execução serão os seguintes:

```
Select Expression
  -> Aggregate
    -> Filter
      -> Nested Loop Join (outer)
        -> Filter
          -> Table "HORSE" Access By ID
            -> Bitmap
              -> Index "HORSE_IDX_NAME" Range Scan (full match)
        -> Filter
          -> Table "COLOR" Access By ID
            -> Bitmap
              -> Index "PK_COLOR" Unique Scan

COUNT
=====
18

Current memory = 556348368
```

```
Delta memory = 240
Max memory = 556642032
Elapsed time = 0.003 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 118
Per table statistics:
```

Table name	Natural	Index	Insert	Update	Delete
COLOR		18			
HORSE		55			

Como você pode ver, em vez de duas junções externas, apenas uma foi executada. Vamos tentar aplicar o predicado de filtro à outra tabela:

```
SELECT COUNT(*)
FROM
    HORSE
    FULL JOIN COLOR ON COLOR.CODE_COLOR = HORSE.CODE_COLOR
WHERE COLOR.NAME = 'black';
```

```
PLAN JOIN (COLOR INDEX (UNQ_COLOR_NAME), HORSE INDEX (FK_HORSE_COLOR))
```

Select Expression

-> Aggregate

-> Filter

-> Nested Loop Join (outer)

Table name	Natural	Index	Insert	Update	Delete
COLOR		1			
HORSE		141644			

Aqui também apenas uma junção externa foi executada em vez de duas, apenas a ordem da junção mudou.

E, finalmente, vamos aplicar predicados de filtro a ambas as tabelas de uma vez:

```
SELECT COUNT(*)
FROM
  HORSE
  FULL JOIN COLOR ON COLOR.CODE_COLOR = HORSE.CODE_COLOR
WHERE HORSE.NAME = 'A Gambler' AND COLOR.NAME = 'black';
```

```
PLAN JOIN (COLOR INDEX (UNQ_COLOR_NAME), HORSE INDEX (FK_HORSE_COLOR, HORSE_IDX_NAME))
```

Select Expression

-> Aggregate

-> Nested Loop Join (inner)

-> Filter

-> Table "COLOR" Access By ID

-> Bitmap

-> Index "UNQ_COLOR_NAME" Unique Scan

-> Filter

-> Table "HORSE" Access By ID

-> Bitmap And

-> Bitmap

-> Index "FK_HORSE_COLOR" Range Scan (full match)

-> Bitmap

-> Index "HORSE_IDX_NAME" Range Scan (full match)

COUNT

=====

2

Current memory = 557452864

Delta memory = 288

Max memory = 557630448

Elapsed time = 0.006 sec

Buffers = 32768

Reads = 0

Writes = 0

Fetches = 61

Per table statistics:

Table name	Natural	Index	Insert	Update	Delete
COLOR		1			
HORSE		9			

Aqui também apenas uma junção interna é executada. Lembro que com `OuterJoinConversion = false` nenhuma transformação é realizada, e o `FULL JOIN` em todos os três casos é executado como duas junções externas (não muito eficiente).

8.2. Junção de tabela com procedimento armazenado

Para o otimizador de consultas, um procedimento armazenado seletivo representa uma caixa preta, sua cardinalidade é desconhecida e, portanto, em vez de uma estimativa baseada em custo (cost based), são usadas regras predefinidas (rule based). Essas regras diferem para procedimentos que dependem e não dependem de outros fluxos de dados. Um procedimento é dependente de outros fluxos se os valores dos parâmetros de entrada passados para ele dependem desses fluxos.

- Se o procedimento armazenado não depende de outros fluxos, então a ordem e o algoritmo de junção para junções internas (`INNER JOIN`) dependem de qual predicado de junção é usado e se é possível usar um índice no campo (expressão do campo) da tabela.
 - Um predicado de igualdade é usado (ou vários predicados de igualdade combinados com `AND`) e não há índice no campo (expressão) da tabela — o algoritmo de junção `HASH JOIN` é usado. O fluxo líder torna-se aquele para o qual a cardinalidade é menor. Para procedimentos armazenados, a cardinalidade é assumida como 1000;
 - Um predicado diferente de `=` ou `IS NOT DISTINCT FROM` é usado, ou um índice no campo (expressão) da tabela existe e pode ser aplicado — o algoritmo `Nested Loop Join` é usado, o procedimento armazenado torna-se o fluxo líder.
- Se o procedimento armazenado é unido usando junções externas unilaterais (`LEFT/RIGHT JOIN`), então a ordem da junção é determinada pela ordem da tabela e do procedimento.
- Se o procedimento armazenado depende de outros fluxos (através de parâmetros de entrada), então ele se torna o fluxo seguidor em qualquer tipo de junção. Se, nesse caso, a ordem da junção não pode ser reestruturada (`table RIGHT JOIN proc(table.field)`), ocorre um erro.

Vamos tentar entender cada um dos casos com exemplos.

8.2.1. Junção interna de procedimento armazenado e tabela (sem dependências através de parâmetros de entrada)

Considere o seguinte exemplo:

```
SELECT
  COUNT(*)
FROM
  SP_GENERATE_SERIES(700000, 1000000) AS S
  JOIN HORSE ON HORSE.CODE_HORSE = S.CURRENT_VALUE;
```

O procedimento `SP_GENERATE_SERIES` gera uma série de valores, começando pelo valor do primeiro argumento e terminando no valor do segundo. Neste caso, ele não depende de outros fluxos de dados através de parâmetros de entrada. Como existe um índice para o campo `CODE_HORSE` e ele pode

ser aplicado, o algoritmo Nested Loop Join é escolhido, e o procedimento armazenado torna-se o fluxo líder.

```
PLAN JOIN (S NATURAL, HORSE INDEX (PK_HORSE))

Select Expression
  -> Aggregate
    -> Nested Loop Join (inner)
      -> Procedure "SP_GENERATE_SERIES" as "S" Scan
      -> Filter
        -> Table "HORSE" Access By ID
          -> Bitmap
            -> Index "PK_HORSE" Unique Scan
```

Agora, vamos fazer com que o índice PK_HORSE não possa ser aplicado.

```
SELECT
  COUNT(*)
FROM
  SP_GENERATE_SERIES(700000, 1000000) AS S
JOIN HORSE ON HORSE.CODE_HORSE+0 = S.CURRENT_VALUE;
```

Neste caso, a junção pelo método Nested Loop torna-se ineficiente, mas como o predicado de junção usa a igualdade =, o método de junção Hash Join pode ser usado. Como mencionado acima, a cardinalidade do procedimento armazenado é assumida como 1000, e a cardinalidade da tabela, estimada pelo otimizador para a tabela HORSE, é de ~800000. Como se sabe, o método Hash Join sempre escolhe o fluxo com menor cardinalidade para fazer o hash, a fim de economizar memória. Neste caso, a cardinalidade estimada do procedimento armazenado é menor, portanto, o fluxo líder será a varredura da tabela HORSE. Isso é visível no plano de execução:

```
PLAN HASH (HORSE NATURAL, S NATURAL)

Select Expression
  -> Aggregate
    -> Filter
      -> Hash Join (inner)
        -> Table "HORSE" Full Scan
        -> Record Buffer (record length: 33)
          -> Procedure "SP_GENERATE_SERIES" as "S" Scan
```

Se a cardinalidade da tabela for menor que 1000, a ordem da junção mudará.

```
SELECT
  COUNT(*)
FROM
  SP_GENERATE_SERIES(700000, 1000000) AS S
JOIN SEX ON SEX.CODE_HORSE+0 = S.CURRENT_VALUE;
```

```
PLAN HASH (S NATURAL, SEX NATURAL)
```

```
Select Expression
```

```
-> Aggregate
```

```
-> Filter
```

```
-> Hash Join (inner)
```

```
-> Procedure "SP_GENERATE_SERIES" as "S" Scan
```

```
-> Record Buffer (record length: 25)
```

```
-> Table "SEX" Full Scan
```

Como mencionado acima, a cardinalidade do procedimento armazenado não pode ser determinada e é sempre assumida como 1000, mas e se isso estiver longe da verdade? Então a escolha incorreta do fluxo para o hash pode gastar significativamente mais memória do que o necessário. Suponha que temos uma consulta assim (sintética):

```
SELECT
  COUNT(*)
FROM
  SP_GENERATE_SERIES(1, 1000000) AS S
JOIN FARM ON FARM.CODE_FARM = S.CURRENT_VALUE;
```

Vamos olhar para seu plano e estatísticas:

```
PLAN JOIN (S NATURAL, FARM INDEX (PK_FARM))
```

```
Select Expression
```

```
-> Aggregate
```

```
-> Nested Loop Join (inner)
```

```
-> Procedure "SP_GENERATE_SERIES" as "S" Scan
```

```
-> Filter
```

```
-> Table "FARM" Access By ID
```

```
-> Bitmap
```

```
-> Index "PK_FARM" Unique Scan
```

```
COUNT
```

```
=====
```

```
41355
```

```
Current memory = 554858624
```

```
Delta memory = 224
```

```
Max memory = 554947840
```

```
Elapsed time = 1.760 sec
```

```
Buffers = 32768
```

```
Reads = 0
```

```
Writes = 0
```

```
Fetches = 3044891
```

```
Per table statistics:
```

Table name	Natural	Index	Insert	Update	Delete
FARM		41355			

Esta consulta não é executada da maneira mais eficiente. As estatísticas não mostram muitas leituras indexadas, mas aqui só são exibidas as leituras dos dados da tabela, não as leituras das páginas do índice. Veja a enorme quantidade de Fetches! Uma solução simples é fazer com que a junção seja executada pelo método Hash Join. Vamos tentar:

```
SELECT
  COUNT(*)
FROM
  SP_GENERATE_SERIES(1, 1000000) AS S
  JOIN FARM ON FARM.CODE_FARM+0 = S.CURRENT_VALUE;
```

PLAN HASH (FARM NATURAL, S NATURAL)

Select Expression

-> Aggregate

-> Filter

-> Hash Join (inner)

-> Table "FARM" Full Scan

-> Record Buffer (record length: 33)

-> Procedure "SP_GENERATE_SERIES" as "S" Scan

COUNT

```
=====
41355
```

Current memory = 554914032

Delta memory = 240

Max memory = 597972512

Elapsed time = 0.855 sec

Buffers = 32768

Reads = 0

Writes = 0

Fetches = 46733

Per table statistics:

Table name	Natural	Index	Insert	Update	Delete
FARM	41356				

Como você pode ver, a situação melhorou significativamente, o número de leituras lógicas ficou muito menor, o tempo de execução da consulta também ficou 2 vezes melhor. Mas está tudo tão bom assim? Não exatamente. A cardinalidade da tabela FARM é 41356, a cardinalidade do procedimento armazenado é estimada como 1000, portanto, o fluxo com o procedimento armazenado é hashado. Mas, na realidade, o procedimento armazenado retorna 1000000 registros, e todos eles são salvos na memória. Seria lógico escolher a tabela FARM como fluxo para o hash. Mas como enganar o otimizador para que ele estime a cardinalidade de forma diferente? O uso de LEFT JOIN não serve aqui, porque junções externas na versão atual do Firebird não podem

ser executadas como Hash Join. A resposta pode ser encontrada no capítulo [Métodos de Acesso Utilizados no Firebird](#). Lembre-se de que tabelas derivadas com limitação de número de registros por FIRST/ROWS/FETCH não tentam estimar a cardinalidade do fluxo de saída como MIN(cardinality, first_value), mas simplesmente assumem o valor especificado em FIRST/ROWS/FETCH (veja [Contadores](#)). Vamos tentar usar isso:

```
SELECT
  COUNT(*)
FROM
  (
    SELECT CURRENT_VALUE
    FROM SP_GENERATE_SERIES(1, 1000000)
    FETCH FIRST 1000000 ROWS ONLY
  ) S
JOIN FARM ON FARM.CODE_FARM+0 = S.CURRENT_VALUE;
```

Vejamos o plano e as estatísticas de execução:

```
PLAN HASH (S SP_GENERATE_SERIES NATURAL, FARM NATURAL)
```

```
Select Expression
```

```
-> Aggregate
```

```
-> Filter
```

```
-> Hash Join (inner)
```

```
-> First N Records
```

```
-> Procedure "SP_GENERATE_SERIES" as "S SP_GENERATE_SERIES" Scan
```

```
-> Record Buffer (record length: 33)
```

```
-> Table "FARM" Full Scan
```

```
COUNT
```

```
=====
```

```
41355
```

```
Current memory = 554965328
```

```
Delta memory = 320
```

```
Max memory = 597972512
```

```
Elapsed time = 0.729 sec
```

```
Buffers = 32768
```

```
Reads = 0
```

```
Writes = 0
```

```
Fetches = 46733
```

```
Per table statistics:
```

Table name	Natural	Index	Insert	Update	Delete
FARM	41356				

Conseguimos mudar a ordem da junção. Agora o fluxo menor (tabela FARM) é hashado, e o procedimento foi escolhido como fluxo líder. O número de leituras lógicas não mudou, mas o tempo de execução melhorou. Na verdade, o consumo de memória também diminuiu, embora não seja

perceptível. Lembre-se de que, se os registros a serem hashados não cabem na memória (veja o parâmetro TempCacheLimit), o Firebird começa a usar arquivos temporários, o que degrada drasticamente o desempenho de tais consultas.



No método acima, deve-se usar um valor *n* em FIRST/ROWS/FETCH que não distorça os resultados da consulta, ou seja, maior ou igual ao número real de registros que o procedimento armazenado (ou o fluxo ao qual você está aplicando isso) produz.

Este método pode deixar de funcionar em versões futuras do Firebird.

8.2.2. Junções externas unidirecionais de procedimento armazenado e tabela

Aqui é tudo simples. A junção só é possível pelo método Nested Loop. A ordem da junção é determinada pelo texto da consulta. Se a tabela for a conduzida, então, sempre que possível, serão usados índices para os predicados da condição de junção. Consideremos a seguinte consulta:

```
SELECT
  COUNT(*)
FROM
  SP_GENERATE_SERIES(1, 1000) AS S
  LEFT JOIN FARM ON FARM.CODE_FARM = S.CURRENT_VALUE;
```

O plano da consulta será o seguinte:

```
PLAN JOIN (S NATURAL, FARM INDEX (PK_FARM))

Select Expression
  -> Aggregate
    -> Nested Loop Join (outer)
      -> Procedure "SP_GENERATE_SERIES" as "S" Scan
      -> Filter
        -> Table "FARM" Access By ID
          -> Bitmap
            -> Index "PK_FARM" Unique Scan
```

Se trocarmos a tabela e o procedimento armazenado de lugar, a ordem da junção também mudará.

```
SELECT
  COUNT(*)
FROM
  FARM
  LEFT JOIN SP_GENERATE_SERIES(1, 1000) S ON S.CURRENT_VALUE = FARM.CODE_FARM
```

```
PLAN JOIN (FARM NATURAL, S NATURAL)

Select Expression
  -> Aggregate
    -> Nested Loop Join (outer)
```

```
-> Table "FARM" Full Scan
-> Filter
    -> Procedure "SP_GENERATE_SERIES" as "S" Scan
```

Obviamente, com um grande número de registros na tabela FARM, essa consulta será extremamente ineficiente devido à execução repetida do procedimento SP_GENERATE_SERIES. Mas, infelizmente, outros algoritmos de execução para junções externas não foram implementados.

8.2.3. Junção com procedimento dependente de fluxo de dados via parâmetros de entrada

Anteriormente, consideramos junções com procedimentos que eram independentes de outros fluxos de dados via parâmetros de entrada, mas existem variantes em que uma expressão que depende de outro fluxo de dados, por exemplo, um campo de tabela, é passada como parâmetro de entrada do procedimento. Nesse caso, o otimizador tenta tornar o procedimento um fluxo conduzido, para que a expressão do parâmetro seja computável. Naturalmente, tais junções só podem ser executadas pelo algoritmo Nested Loop Join. Existem as seguintes variantes de tais junções:

```
SELECT ...
FROM T CROSS JOIN SP(<expr of T> [, ...])

SELECT ...
FROM T JOIN SP(<expr of T> [, ...]) ON <join_condition>

SELECT ...
FROM T LEFT JOIN SP(<expr of T> [, ...]) ON <join_condition>
```

Variantes em que o procedimento é especificado primeiro serão rejeitadas na fase de análise da consulta com o erro “Column does not belong to referenced table”. Além disso, é impossível executar uma junção externa direita (RIGHT JOIN) e uma junção externa completa (FULL JOIN) com um procedimento armazenado, pois nesse caso é impossível construir corretamente uma ordem de junção consistente.

Vamos dar alguns exemplos de junção com procedimentos armazenados que dependem de outros fluxos de dados via parâmetros de entrada.

```
SELECT
  HORSE.CODE_HORSE,
  M.AGE,
  M.HEIGHT_HORSE,
  M.LENGTH_HORSE,
  M.CHESTAROUND,
  M.WRISTAROUND
FROM
  HORSE
  CROSS JOIN SP_LAST_MEASURE(HORSE.CODE_HORSE) M
WHERE HORSE.CODE_BREED = 65;
```

Esta consulta retornará os resultados das últimas medições para cada cavalo. Como a consulta usa uma junção cruzada, apenas os cavalos que foram medidos pelo menos uma vez (o resultado da seleção do procedimento SP_LAST_MEASURE não é vazio) serão incluídos no resultado. Vamos olhar o plano de execução desta consulta.

```
PLAN JOIN (HORSE INDEX (FK_HORSE_BREED), M NATURAL)

Select Expression
  -> Nested Loop Join (inner)
    -> Filter
      -> Table "HORSE" Access By ID
        -> Bitmap
          -> Index "FK_HORSE_BREED" Range Scan (full match)
        -> Procedure "SP_LAST_MEASURE" as "M" Scan
```

Nada inesperado aqui—a junção é realizada pelo algoritmo Nested Loop Join, o procedimento armazenado é o fluxo conduzido, pois depende via parâmetros de entrada do campo da tabela HORSE.

Você pode alterar o tipo de junção para INNER JOIN e especificar a condição de junção.

```
SELECT
  HORSE.CODE_HORSE,
  M.AGE,
  M.HEIGHT_HORSE,
  M.LENGTH_HORSE,
  M.CHESTAROUND,
  M.WRISTAROUND
FROM
  HORSE
  JOIN SP_LAST_MEASURE(HORSE.CODE_HORSE) M ON M.AGE = DATEDIFF(YEAR FROM CURRENT_DATE TO
HORSE.BIRTHDAY)
WHERE HORSE.CODE_BREED = 65
```

O plano mudará de forma insignificante (haverá uma filtragem adicional para os registros do procedimento).

```
PLAN JOIN (HORSE INDEX (FK_HORSE_BREED), M NATURAL)

Select Expression
  -> Nested Loop Join (inner)
    -> Filter
      -> Table "HORSE" Access By ID
        -> Bitmap
          -> Index "FK_HORSE_BREED" Range Scan (full match)
        -> Filter
          -> Procedure "SP_LAST_MEASURE" as "M" Scan
```

Se você precisar que o resultado inclua cavalos que não tiveram medições, pode alterar a junção para LEFT JOIN. Como condição de junção, você pode usar uma expressão invariante que seja

constantemente verdadeira.

```
SELECT
  HORSE.CODE_HORSE,
  M.AGE,
  M.HEIGHT_HORSE,
  M.LENGTH_HORSE,
  M.CHESTAROUND,
  M.WRISTAROUND
FROM
  HORSE
  LEFT JOIN SP_LAST_MEASURE(HORSE.CODE_HORSE) M ON TRUE
WHERE HORSE.CODE_BREED = 65
```

O algoritmo e a ordem da junção serão os mesmos dos exemplos anteriores:

```
PLAN JOIN (HORSE INDEX (FK_HORSE_BREED), M NATURAL)

Select Expression
  -> Filter
    -> Nested Loop Join (outer)
      -> Filter
        -> Table "HORSE" Access By ID
          -> Bitmap
            -> Index "FK_HORSE_BREED" Range Scan (full match)
          -> Procedure "SP_LAST_MEASURE" as "M" Scan
```

8.3. Junção de tabela com expressão de tabela

Expressões de tabela são subconsultas usadas como fontes de dados na consulta principal. Existem dois tipos de expressões de tabela:

- tabelas derivadas (derived table);
- expressões de tabela comuns (common table expression) ou CTE abreviadamente.

As visualizações (VIEW) também podem ser classificadas como expressões de tabela, pois uma visualização nada mais é do que uma consulta salva com um nome definido nos metadados. Portanto, o otimizador se comporta com visualizações exatamente da mesma forma que com expressões de tabela.

Tabela derivada (derived table) — é uma expressão de tabela incluída na cláusula FROM da consulta. Tabelas derivadas são uma consulta entre parênteses. Essa consulta pode receber um alias, após o qual a expressão de tabela pode ser referenciada como uma tabela comum (usada como fonte de dados). Uma tabela derivada pode ser independente ou dependente de fontes de dados externas. Para usar tabelas derivadas dependentes de fontes de dados externas, é necessário usar a palavra-chave LATERAL antes da expressão de tabela. Nesta seção, consideraremos o trabalho do otimizador apenas com tabelas derivadas independentes. Junções com tabelas derivadas LATERAL serão consideradas mais tarde.

Expressão de tabela comum (common table expression) ou CTE abreviadamente — é uma expressão de tabela nomeada declarada na cláusula WITH. Com uma expressão de tabela comum, assim como com uma tabela derivada, pode-se trabalhar como com uma tabela comum (usá-la como fonte de dados). Expressões de tabela comuns declaradas posteriormente podem usar expressões de tabela declaradas anteriormente em seu corpo. Expressões de tabela comuns são divididas em recursivas e não recursivas. Expressões de tabela comuns dependentes de fluxos externos são impossíveis.

As regras de otimização para tabelas derivadas (derived table) independentes de fontes de dados externas, expressões de tabela comuns (CTE) e visualizações (VIEW) são as mesmas.

Como as junções se comportarão ao usar expressões de tabela? Isso dependerá do conteúdo da expressão de tabela. Nos casos mais simples, uma consulta com uma expressão de tabela pode ser transformada em uma consulta equivalente sem o uso da expressão de tabela. Em outras palavras, consultas mais simples são expandidas para tabelas base. Em casos mais complexos, a expressão de tabela não pode ser expandida para tabelas base. Expressões de tabela simples contêm apenas operações de junção interna de fluxos e condições de sua filtragem. A adição de ordenação, DISTINCT, cálculo de agregados, agrupamento, cálculo de funções de janela, UNION e contadores FIRST/SKIP, junções externas (OUTER JOIN) transforma a expressão de tabela em uma complexa.

8.3.1. Junções com expressões de tabela simples

Como as expressões de tabela simples são expandidas para tabelas base, não faz sentido considerar sua otimização (a otimização de junção de tabelas foi discutida anteriormente). Vou explicar isso com um exemplo.

```
SELECT
  COUNT(*)
FROM
  HORSE
  JOIN (
    SELECT FARM.CODE_FARM
    FROM FARM
    WHERE FARM.CODE_COUNTRY = 1
  ) F ON F.CODE_FARM = HORSE.CODE_FARM
```

O plano desta consulta será o seguinte:

```
Select Expression
-> Aggregate
  -> Nested Loop Join (inner)
    -> Filter
      -> Table "FARM" as "F FARM" Access By ID
        -> Bitmap
          -> Index "FK_FARM_COUNTRY" Range Scan (full match)
    -> Filter
      -> Table "HORSE" Access By ID
        -> Bitmap
          -> Index "FK_HORSE_FARMBORN" Range Scan (full match)
```

Esta consulta usa uma tabela derivada simples, que será expandida pelo otimizador para as tabelas base. Ou seja, na prática, será executada uma consulta como esta:

```
SELECT
  COUNT(*)
FROM
  HORSE
  JOIN FARM F ON F.CODE_FARM = HORSE.CODE_FARM
WHERE F.CODE_COUNTRY = 1;
```

O plano de execução desta consulta será exatamente o mesmo da consulta na versão original.

8.3.2. Junções internas com expressões de tabela complexas

O otimizador se comporta de maneira diferente para junções internas e externas com expressões de tabela. No caso de uma junção interna de uma tabela com uma expressão de tabela, o otimizador tenta escolher um plano de execução que não leve à reexecução da consulta dentro da expressão de tabela. Assim, a junção da expressão de tabela com a tabela é executada por um dos dois algoritmos:

- Nested Loop Join. Neste caso, a expressão de tabela é escolhida como fluxo principal (leading) e a tabela como fluxo seguido (driven). Este algoritmo é escolhido apenas se for impossível usar um índice no campo da tabela para o predicado de junção ou se o predicado de junção não puder ser usado para o algoritmo Hash Join.
- Hash Join. É usado se o predicado de junção for uma igualdade (=) ou equivalência (IS NOT DISTINCT FROM), e não houver possibilidade de usar um índice no(s) campo(s) da tabela. Neste caso, a cardinalidade da expressão de tabela e da tabela é estimada. O fluxo de dados com menor cardinalidade é hasheado e se torna o fluxo seguido.

Em ambos os casos, a consulta dentro da expressão de tabela é otimizada separadamente, ou seja, seu plano não depende do que está sendo unido a ela. Predicados de filtragem independentes de outros fluxos, que se aplicam à expressão de tabela, serão, sempre que possível, empurrados (pushed) para dentro da expressão de tabela.

Vejamos o seguinte exemplo.

```
WITH
  T AS (
    SELECT DISTINCT HORSE.CODE_FARM
    FROM HORSE
    WHERE HORSE.CODE_BREED = 65
  )
SELECT
  COUNT(*)
FROM
  T JOIN FARM ON FARM.CODE_FARM = T.CODE_FARM;
```

O plano e as estatísticas de execução desta consulta serão os seguintes:


```

PLAN JOIN (SORT (T HORSE INDEX (FK_HORSE_BREED)), FARM INDEX (PK_FARM))

Select Expression
  -> Aggregate
    -> Nested Loop Join (inner)
      -> Unique Sort (record length: 44, key length: 12)
        -> Filter
          -> Table "HORSE" as "T HORSE" Access By ID
            -> Bitmap
              -> Index "FK_HORSE_BREED" Range Scan (full match)
        -> Filter
          -> Table "FARM" Access By ID
            -> Bitmap
              -> Index "PK_FARM" Unique Scan

COUNT
=====
3235

Current memory = 555067920
Delta memory = 288
Max memory = 556120864
Elapsed time = 0.091 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 145306
Per table statistics:
-----+-----+-----+-----+-----+
Table name | Natural | Index | Insert | Update | Delete |
-----+-----+-----+-----+-----+
FARM       |         | 3235 |         |         |         |
HORSE      |         | 122836 |         |         |         |
-----+-----+-----+-----+-----+

```

Aqui, a CTE T é escolhida como fluxo principal, a junção é executada pelo método Nested Loop Join. Este plano foi escolhido porque existe um índice para o campo CODE_FARM da tabela FARM. A ordem da junção não mudará, independentemente da proporção do número de registros nas tabelas FARM e HORSE.

Bem, o que acontecerá se não existir um índice adequado para o campo CODE_FARM da tabela FARM? Esta situação é facilmente reproduzida adicionando a dica +0.

```

WITH
  T AS (
    SELECT DISTINCT HORSE.CODE_FARM
    FROM HORSE
    WHERE HORSE.CODE_BREED = 65
  )
SELECT
  COUNT(*)
FROM

```

```
T JOIN FARM ON FARM.CODE_FARM+0 = T.CODE_FARM;
```

O plano e as estatísticas de execução desta consulta serão os seguintes:

```
PLAN HASH (FARM NATURAL, SORT (T HORSE INDEX (FK_HORSE_BREED)))

Select Expression
  -> Aggregate
    -> Filter
      -> Hash Join (inner)
        -> Table "FARM" Full Scan
        -> Record Buffer (record length: 33)
          -> Unique Sort (record length: 44, key length: 12)
            -> Filter
              -> Table "HORSE" as "T HORSE" Access By ID
                -> Bitmap
                  -> Index "FK_HORSE_BREED" Range Scan (full match)

COUNT
=====
3235

Current memory = 555358720
Delta memory = 288
Max memory = 557844480
Elapsed time = 0.113 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 177860
Per table statistics:
```

Table name	Natural	Index	Insert	Update	Delete
FARM	41356				
HORSE		122836			

Agora, a junção é realizada pelo algoritmo Hash Join. A tabela FARM se torna o fluxo principal, e a expressão de tabela T é hasheada. Aqui, a estimativa de custo já está funcionando — para o hashing, é escolhida a fonte de dados cuja estimativa de cardinalidade é menor. Isso é facilmente demonstrado removendo a condição de filtragem dentro da expressão de tabela.

```
WITH
  T AS (
    SELECT DISTINCT HORSE.CODE_FARM
    FROM HORSE
  )
SELECT
  COUNT(*)
FROM
  T JOIN FARM ON FARM.CODE_FARM+0 = T.CODE_FARM;
```

```

PLAN HASH (SORT (T HORSE NATURAL), FARM NATURAL)

Select Expression
  -> Aggregate
    -> Filter
      -> Hash Join (inner)
        -> Unique Sort (record length: 36, key length: 12)
          -> Table "HORSE" as "T HORSE" Full Scan
        -> Record Buffer (record length: 33)
          -> Table "FARM" Full Scan

```

```

COUNT
=====
15243

```

```

Current memory = 555473312
Delta memory = 256
Max memory = 561562896
Elapsed time = 0.343 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 648470
Per table statistics:

```

Table name	Natural	Index	Insert	Update	Delete
FARM	41356				
HORSE	562237				

Neste caso, a expressão de tabela é escolhida como fluxo principal, e a tabela FARM é hasheada. Por que essa ordem de junção foi escolhida?

Aqui é simples. Basta lembrar que a eliminação de duplicatas usando DISTINCT reduz a estimativa de cardinalidade do fluxo de entrada em 10 vezes (veja [Ordenação](#)). Assim, a cardinalidade da expressão de tabela é estimada aproximadamente como 56000, enquanto a cardinalidade da tabela FARM é aproximadamente 41000. Neste caso, a estimativa de cardinalidade da tabela é menor e, portanto, ela é a que é hasheada.

Esta consulta pode ser reescrita de forma equivalente, o que dará o mesmo resultado, mas a ordem da junção será diferente.

```

WITH
  T AS (
    SELECT HORSE.CODE_FARM
    FROM HORSE
    GROUP BY 1
  )
SELECT
  COUNT(*)
FROM

```

```
T JOIN FARM ON FARM.CODE_FARM+0 = T.CODE_FARM;
```

Plano e estatísticas de execução:

```
PLAN HASH (FARM NATURAL, T HORSE ORDER FK_HORSE_FARMBORN)
```

```
Select Expression
```

```
-> Aggregate
```

```
-> Filter
```

```
-> Hash Join (inner)
```

```
-> Table "FARM" Full Scan
```

```
-> Record Buffer (record length: 33)
```

```
-> Aggregate
```

```
-> Table "HORSE" as "T HORSE" Access By ID
```

```
-> Index "FK_HORSE_FARMBORN" Full Scan
```

```
COUNT
```

```
=====
```

```
15243
```

```
Current memory = 555991312
```

```
Delta memory = 288
```

```
Max memory = 561562896
```

```
Elapsed time = 0.584 sec
```

```
Buffers = 32768
```

```
Reads = 0
```

```
Writes = 0
```

```
Fetches = 1337150
```

```
Per table statistics:
```

Table name	Natural	Index	Insert	Update	Delete
FARM	41356				
HORSE		562237			

Desta vez, a tabela FARM foi escolhida como principal, e o resultado da expressão de tabela é hasheado. Mas por que isso aconteceu, já que o resultado da consulta e sua semântica são semelhantes à versão anterior? O fato é que a estimativa de cardinalidade após DISTINCT e GROUP BY é feita de maneira diferente. Ao usar DISTINCT, a cardinalidade original é dividida por 10, enquanto ao usar GROUP BY é dividida por 1000. Este truncamento da cardinalidade não é baseado em dados estatísticos (constantes no código do otimizador) e, portanto, é muito impreciso e frequentemente causa erro. Nesta consulta, a cardinalidade da CTE é estimada como $562237 / 1000 = 562.2$, que é significativamente menor que a estimativa de cardinalidade da tabela FARM.



Em versões futuras do Firebird, isso será corrigido. Neste caso, seria possível estimar com mais precisão a cardinalidade da CTE com base na seletividade do índice no campo CODE_FARM da tabela HORSE.

Como você já viu, ao usar Hash Join com expressões de tabela complexas, o otimizador pode frequentemente errar devido a uma estimativa incorreta da cardinalidade da expressão de tabela,

o que significa que pode escolher para hashing a fonte de dados não com a menor cardinalidade. Vamos ver como podemos dar uma dica ao otimizador.

De **Contadores**, sabemos que a cardinalidade da fonte de dados após a aplicação de FIRST/ROWS/FETCH é estimada como igual ao literal especificado nesses contadores. Se especificarmos um valor em FIRST/ROWS/FETCH que seja intencionalmente maior do que o número de registros que a expressão de tabela pode retornar, e maior do que a cardinalidade da tabela, podemos evitar o hashing dos resultados da execução da expressão de tabela.

Vamos tentar modificar a consulta onde a expressão de tabela usa GROUP BY.

```
WITH
  T AS (
    SELECT HORSE.CODE_FARM
    FROM HORSE
    GROUP BY 1
    FETCH FIRST 1000000 ROWS ONLY
  )
SELECT
  COUNT(*)
FROM
  T JOIN FARM ON FARM.CODE_FARM+0 = T.CODE_FARM;
```

O plano e as estatísticas de execução serão os seguintes:

```
PLAN HASH (T HORSE ORDER FK_HORSE_FARMBORN, FARM NATURAL)
```

Select Expression

-> Aggregate

-> Filter

-> Hash Join (inner)

-> First N Records

-> Aggregate

-> Table "HORSE" as "T HORSE" Access By ID

-> Index "FK_HORSE_FARMBORN" Full Scan

-> Record Buffer (record length: 33)

-> Table "FARM" Full Scan

COUNT

=====

15243

Current memory = 556394288

Delta memory = 320

Max memory = 561562896

Elapsed time = 0.482 sec

Buffers = 32768

Reads = 0

Writes = 0

Fetches = 1337150

Per table statistics:

Table name	Natural	Index	Insert	Update	Delete

FARM	41356				
HORSE		562237			

Na cláusula `FETCH FIRST xxx ROWS ONLY`, especificamos uma limitação de 1000000 registros, que é maior do que a cardinalidade da tabela FARM e maior do que a cardinalidade da expressão de tabela T. O limitador `FETCH FIRST` não afetou o resultado da consulta, mas alterou o plano da consulta, pois o otimizador decidiu que construir um hash para 41000 registros é mais barato do que para 1000000.

8.3.3. Junções externas unidirecionais com expressões de tabela complexas

No Firebird 5.0, uma junção externa unidirecional (`LEFT/RIGHT JOIN`) só pode ser executada pelo algoritmo `Nested Loop`. A ordem da junção é determinada pelo texto da consulta. Se a expressão de tabela for o fluxo principal, a consulta dentro da expressão de tabela é executada da maneira usual; para cada registro obtido da expressão de tabela, o predicado de junção é calculado e os registros da tabela secundária são anexados se o predicado de junção for verdadeiro.

Considere a seguinte consulta:

```
WITH
  T AS (
    SELECT DISTINCT HORSE.CODE_FARM
    FROM HORSE
  )
SELECT
  COUNT(*)
FROM
  T LEFT JOIN FARM ON FARM.CODE_FARM = T.CODE_FARM;
```

Aqui é óbvio que o fluxo principal será a expressão de tabela T, e a tabela FARM será o fluxo secundário. Isso é confirmado no plano da consulta:

```
PLAN JOIN (SORT (T HORSE NATURAL), FARM INDEX (PK_FARM))

Select Expression
-> Aggregate
  -> Nested Loop Join (outer)
    -> Unique Sort (record length: 36, key length: 12)
      -> Table "HORSE" as "T HORSE" Full Scan
    -> Filter
      -> Table "FARM" Access By ID
        -> Bitmap
          -> Index "PK_FARM" Unique Scan

COUNT
=====
15243

Current memory = 553732736
```

```
Delta memory = 288
Max memory = 557140656
Elapsed time = 0.336 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 665526
Per table statistics:
```

Table name	Natural	Index	Insert	Update	Delete
FARM		15243			
HORSE	562237				

Agora vamos tentar trocar a tabela e a expressão de tabela de lugar.

```
WITH
  T AS (
    SELECT DISTINCT HORSE.CODE_FARM
    FROM HORSE
  )
SELECT
  COUNT(*)
FROM
  FARM LEFT JOIN T ON T.CODE_FARM = FARM.CODE_FARM;
```

É óbvio que a junção será executada pelo método Nested Loop Join, a tabela será o fluxo principal e a expressão de tabela será o fluxo secundário. Ou seja, a consulta dentro da expressão de tabela será executada várias vezes, mas há uma nuance importante aqui. Reexecutar a consulta dentro da expressão de tabela “inteiramente” e depois filtrar seus resultados para cada registro da tabela FARM é muito caro. Portanto, se possível, o predicado na condição de junção é “empurrado” (push) para dentro da expressão de tabela. Assim, para cada registro da tabela FARM, não é executada a consulta:

```
SELECT DISTINCT HORSE.CODE_FARM
FROM HORSE
```

mas sim a consulta:

```
SELECT DISTINCT HORSE.CODE_FARM
FROM HORSE
WHERE HORSE.CODE_FARM = ?
```

Vamos ver como isso aparece no plano da consulta:

```
PLAN JOIN (FARM NATURAL, SORT (T HORSE INDEX (FK_HORSE_FARMBORN)))
```

```
Select Expression
-> Aggregate
```

```

-> Nested Loop Join (outer)
  -> Table "FARM" Full Scan
  -> Unique Sort (record length: 36, key length: 12)
    -> Filter
      -> Table "HORSE" as "T HORSE" Access By ID
        -> Bitmap
          -> Index "FK_HORSE_FARMBORN" Range Scan (full match)

```

```

COUNT
=====
41356

```

```

Current memory = 554044448
Delta memory = 288
Max memory = 557140656
Elapsed time = 0.543 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 899964
Per table statistics:

```

Table name	Natural	Index	Insert	Update	Delete
FARM	41356				
HORSE		562237			

É sempre possível empurrar o predicado para dentro da expressão de tabela? Não, isso só é possível se tal transformação não violar a semântica da consulta. Conheço duas construções que impedem o empurrão de predicados para dentro de expressões de tabela:

- Limitadores FETCH/FIRST/ROWS, bem como OFFSET/SKIP;
- Funções de janela.

Por que os limitadores FETCH/FIRST/ROWS impedem o empurrão de predicados para dentro da expressão de tabela? Porque eles sempre são executados no resultado da consulta, ou seja, por último. Basta comparar o significado lógico das duas opções de execução:

- executar a consulta, obter o primeiro registro, depois filtrar o resultado;
- executar a consulta com filtragem de registros e obter o primeiro registro.

Essas duas opções darão resultados completamente diferentes.

Aproximadamente pela mesma razão, o empurrão de predicados não funciona para expressões de tabela com funções de janela. A janela e suas partições são sempre construídas após obter os resultados dessa consulta sem janelas. Em teoria, é possível empurrar um predicado pelo campo de particionamento, mas apenas se a consulta usar apenas janelas com o mesmo particionamento. Infelizmente, o otimizador do Firebird ainda não sabe detectar tais casos.

Assim, se a junção é executada pelo algoritmo Nested Loop Join e a expressão de tabela é escolhida como fluxo secundário, uma otimização que empurra o predicado de junção para dentro da

expressão de tabela é possível. Mas atualmente, tal algoritmo e ordem de junção só podem ser escolhidos para junções externas unidirecionais. Para junções internas, o otimizador não sabe avaliar o custo dessa opção e compará-la com alternativas, portanto nunca a escolhe. Em versões futuras do Firebird, isso pode mudar.

Mas e se você acreditar que tal algoritmo seria mais eficiente para uma junção interna? Infelizmente, é impossível forçar o otimizador a executar uma junção interna dessa maneira, mas você pode executar uma junção externa unidirecional e obter um resultado equivalente. Para isso, basta adicionar uma filtragem de valores NULL na cláusula WHERE nas colunas da expressão de tabela que participam da condição de junção.

```
WITH
  T AS (
    SELECT DISTINCT HORSE.CODE_FARM
    FROM HORSE
  )
SELECT
  COUNT(*)
FROM
  FARM
  LEFT JOIN T ON T.CODE_FARM = FARM.CODE_FARM
WHERE T.CODE_FARM IS NOT NULL;
```

Esta consulta dará um resultado análogo a uma junção interna, mas o plano será o mesmo do exemplo anterior.

```
Select Expression
-> Aggregate
  -> Filter
    -> Nested Loop Join (outer)
      -> Table "FARM" Full Scan
      -> Unique Sort (record length: 36, key length: 12)
        -> Filter
          -> Table "HORSE" as "T HORSE" Access By ID
            -> Bitmap
              -> Index "FK_HORSE_FARMBORN" Range Scan (full match)
```

```
      COUNT
=====
      15243
```

```
Current memory = 554420000
Delta memory = 320
Max memory = 577686592
Elapsed time = 0.542 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 899964
Per table statistics:
```

Table name	Natural	Index	Insert	Update	Delete

FARM	41356				
HORSE		562237			

8.3.4. Junções com tabelas derivadas LATERAL

Além das tabelas derivadas (derived table) que não dependem de fluxos externos, o padrão prevê tabelas derivadas que dependem de fluxos externos. Para possibilitar o uso de tais tabelas derivadas, é necessário usar a palavra-chave **LATERAL** antes da expressão de tabela. Expressões de tabela comuns (CTEs) que dependem de fluxos externos não são possíveis.

Para tabelas derivadas **LATERAL**, fazem sentido os seguintes tipos de junção: **CROSS JOIN** e **LEFT JOIN**. A sintaxe permite outros tipos de junção também. Uma característica da junção com lateral derived table é que elas só podem ser executadas pelo algoritmo **Nested Loop**. Outra característica diz respeito à execução do **CROSS JOIN**. O fato é que, devido à dependência da tabela derivada **LATERAL** em relação aos fluxos externos, ela não pode ser definida como o fluxo líder nas junções. O otimizador reconhece a presença de dependências e escolhe a ordem de junção na qual a tabela derivada dependente será a seguidora na junção. Comportamento semelhante o otimizador demonstra para procedimentos armazenados seletivos que dependem de outros fluxos de dados através de parâmetros de entrada.

Assim, a avaliação de custo para escolha do algoritmo e da ordem de junção com tabelas derivadas dependentes não é realizada, sendo utilizadas regras que tornem essa junção possível.

Vejamos o seguinte exemplo:

```
SELECT
  COUNT(*)
FROM
  FARM
  CROSS JOIN LATERAL (
    SELECT HORSE.CODE_HORSE
    FROM HORSE
    WHERE HORSE.CODE_FARM = FARM.CODE_FARM
    AND HORSE.CODE_DEPARTURE = 1
  ) T;
```

Seu plano e estatísticas serão os seguintes:

```
PLAN JOIN (FARM NATURAL, T HORSE INDEX (IDX_HORSE_DEP_FARMBORN))

Select Expression
-> Aggregate
  -> Nested Loop Join (inner)
    -> Table "FARM" Full Scan
    -> Filter
      -> Table "HORSE" as "T HORSE" Access By ID
        -> Bitmap
          -> Index "IDX_HORSE_DEP_FARMBORN" Range Scan (full match)
```

```

COUNT
=====
100186

Current memory = 555614832
Delta memory = 320
Max memory = 577686592
Elapsed time = 0.161 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 305484
Per table statistics:
-----+-----+-----+-----+-----+-----+
Table name      | Natural | Index  | Insert | Update | Delete |
-----+-----+-----+-----+-----+-----+
FARM             |    41356|        |        |        |        |
HORSE            |         | 100186|        |        |        |
-----+-----+-----+-----+-----+-----+

```

Neste caso, o otimizador simplesmente não teve escolha: para obter dados da tabela derivada T é necessário satisfazer suas dependências, o que significa que o único algoritmo de execução possível é o Nested Loop Join, no qual a tabela FARM é a líder. Na verdade, esta consulta é semanticamente equivalente à seguinte:

```

SELECT
  COUNT(*)
FROM
  HORSE
  JOIN FARM ON FARM.CODE_FARM = HORSE.CODE_FARM
WHERE HORSE.CODE_DEPARTURE = 1;

```

Mas neste exemplo o otimizador usará a avaliação de custo para escolher o algoritmo e a ordem de junção.

```

PLAN HASH (FARM NATURAL, HORSE INDEX (FK_HORSE_DEPARTURE))

Select Expression
  -> Aggregate
    -> Filter
      -> Hash Join (inner)
        -> Table "FARM" Full Scan
        -> Record Buffer (record length: 41)
          -> Filter
            -> Table "HORSE" Access By ID
              -> Bitmap
                -> Index "FK_HORSE_DEPARTURE" Range Scan (full match)

COUNT
=====
100186

```

```

Current memory = 555835312
Delta memory = 0
Max memory = 577686592
Elapsed time = 0.113 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 153883
Per table statistics:

```

Table name	Natural	Index	Insert	Update	Delete
FARM	41356				
HORSE		100186			

Em vez de junção CROSS com tabelas derivadas LATERAL, você também pode usar junções internas comuns (INNER JOIN). Por exemplo:

```

SELECT
  COUNT(*)
FROM
  DEPARTURE
  JOIN LATERAL (
    SELECT HORSE.CODE_BREED
    FROM HORSE
    WHERE HORSE.CODE_DEPARTURE = DEPARTURE.CODE_DEPARTURE
  ) T ON T.CODE_BREED = DEPARTURE.CODE_DEFAULT_BREED

```

O plano e as estatísticas de execução serão os seguintes:

```

PLAN JOIN (DEPARTURE NATURAL, T HORSE INDEX (FK_HORSE_DEPARTURE))

```

Select Expression

-> Aggregate

-> Nested Loop Join (inner)

-> Table "DEPARTURE" Full Scan

-> Filter

-> Filter

-> Table "HORSE" as "T HORSE" Access By ID

-> Bitmap

-> Index "FK_HORSE_DEPARTURE" Range Scan (full match)

COUNT

```

=====
355712

```

```

Current memory = 556342224
Delta memory = 352
Max memory = 577686592
Elapsed time = 0.304 sec
Buffers = 32768

```

```

Reads = 0
Writes = 0
Fetches = 610791
Per table statistics:

```

Table name	Natural	Index	Insert	Update	Delete
DEPARTURE	19				
HORSE		562237			

Além disso, com tabelas LATERAL é possível usar a junção externa esquerda (LEFT JOIN):

```

SELECT
  COUNT(*)
FROM
  FARM
  LEFT JOIN LATERAL (
    SELECT HORSE.CODE_HORSE
    FROM HORSE
    WHERE HORSE.CODE_FARM = FARM.CODE_FARM
      AND HORSE.CODE_DEPARTURE = 1
  ) T ON TRUE;

```

Diferentemente do exemplo com CROSS JOIN LATERAL, aqui os registros da tabela FARM sempre estarão presentes na seleção, mesmo que a tabela derivada esteja vazia. O plano e as estatísticas de execução são os seguintes:

```

PLAN JOIN (FARM NATURAL, T HORSE INDEX (IDX_HORSE_DEP_FARMBORN))

```

```

Select Expression
  -> Aggregate
    -> Nested Loop Join (outer)
      -> Table "FARM" Full Scan
      -> Filter
        -> Table "HORSE" as "T HORSE" Access By ID
          -> Bitmap
            -> Index "IDX_HORSE_DEP_FARMBORN" Range Scan (full match)

```

```

COUNT
=====
138559

```

```

Current memory = 556416176
Delta memory = 320
Max memory = 577686592
Elapsed time = 0.163 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 305484
Per table statistics:

```

```

-----+-----+-----+-----+-----+

```

Table name	Natural	Index	Insert	Update	Delete
FARM	41356				
HORSE		100186			

Todos os exemplos acima podem ser facilmente reescritos como junções sem o uso de tabelas derivadas LATERAL. E essas consultas sem o uso de tabelas derivadas LATERAL seriam mais eficientes. Mas então por que usá-las? Existem várias razões para isso:

- Em alguns casos, é impossível ou muito difícil usar alternativas. Lembre-se das junções com expressões de tabela que contêm FIRST/ROWS/FETCH: os predicados de junção não são propagados para dentro da expressão de tabela. Usando a palavra-chave LATERAL, isso é possível.
- Tabelas derivadas LATERAL podem ser usadas para transformar colunas de uma tabela em linhas.
- O otimizador do Firebird é imperfeito e nem sempre escolhe bons planos de execução ao unir com expressões de tabela. Usando tabelas derivadas LATERAL, você pode conseguir a ordem de junção desejada, mas apenas com o uso do algoritmo Nested Loop Join.

Vejamos vários exemplos úteis de uso de tabelas derivadas LATERAL.

Substituição de subconsultas semelhantes

Considere a seguinte tarefa. Você tem uma tabela REGISTRATION, que é um registro de proprietários de cavalos. Os cavalos podem mudar de proprietário várias vezes durante sua vida. A tabela contém as seguintes informações: código do cavalo, código da fazenda (proprietário), código do motivo da mudança de proprietário, data da mudança de proprietário. É necessário listar os proprietários atuais dos cavalos de uma determinada lista em uma data específica. Além disso, é necessário listar a data da mudança de proprietário e o motivo da mudança. Se resolvermos essa tarefa de forma direta, será necessário usar várias subconsultas. A consulta final ficará mais ou menos assim:

```
SELECT
  H.CODE_HORSE,
  H.NAME AS HORSE_NAME,
  ( SELECT R.CODE_FARM
    FROM REGISTRATION R
   WHERE R.CODE_HORSE = H.CODE_HORSE
        AND R.BYDATE <= DATE '2015-12-31'
   ORDER BY R.BYDATE DESC
   FETCH FIRST ROW ONLY
  ) AS CODE_FARM,
  ( SELECT R.CODE_REASON
    FROM REGISTRATION R
   WHERE R.CODE_HORSE = H.CODE_HORSE
        AND R.BYDATE <= DATE '2015-12-31'
   ORDER BY R.BYDATE DESC
   FETCH FIRST ROW ONLY
  ) AS CODE_REASON,
  ( SELECT R.BYDATE
    FROM REGISTRATION R
   WHERE R.CODE_HORSE = H.CODE_HORSE
```

```

        AND R.BYDATE <= DATE '2015-12-31'
    ORDER BY R.BYDATE DESC
    FETCH FIRST ROW ONLY
) AS BYDATE
FROM
    HORSE H
WHERE EXTRACT(YEAR FROM H.BIRTHDAY) = 2005;

```

A consulta parece assustadora, não é? Vamos olhar seu plano:

```

PLAN SORT (R INDEX (FK_REGISTRATION_HORSE))
PLAN SORT (R INDEX (FK_REGISTRATION_HORSE))
PLAN SORT (R INDEX (FK_REGISTRATION_HORSE))
PLAN (H INDEX (HORSE_IDX_BIRTHYEAR))

Sub-query
  -> Singularity Check
    -> First N Records
      -> Refetch
        -> Sort (record length: 28, key length: 8)
          -> Filter
            -> Table "REGISTRATION" as "R" Access By ID
              -> Bitmap
                -> Index "FK_REGISTRATION_HORSE" Range Scan (full match)

Sub-query
  -> Singularity Check
    -> First N Records
      -> Refetch
        -> Sort (record length: 28, key length: 8)
          -> Filter
            -> Table "REGISTRATION" as "R" Access By ID
              -> Bitmap
                -> Index "FK_REGISTRATION_HORSE" Range Scan (full match)

Sub-query
  -> Singularity Check
    -> First N Records
      -> Refetch
        -> Sort (record length: 28, key length: 8)
          -> Filter
            -> Table "REGISTRATION" as "R" Access By ID
              -> Bitmap
                -> Index "FK_REGISTRATION_HORSE" Range Scan (full match)

Select Expression
  -> Filter
    -> Table "HORSE" as "H" Access By ID
      -> Bitmap
        -> Index "HORSE_IDX_BIRTHYEAR" Range Scan (full match)

```

Aqui, para cada registro da consulta principal, três subconsultas são executadas. Mas observe essas subconsultas com atenção—elas são quase idênticas! A diferença está apenas nas colunas retornadas. Você não pode escrever uma única subconsulta na seção `SELECT` que retorne várias colunas. Reescrever essa consulta para substituir as subconsultas por expressões de tabela comuns também é impossível. E é aqui que as tabelas derivadas `LATERAL` vêm em seu auxílio. Como o

resultado da subconsulta pode estar vazio, e os registros da tabela principal ainda devem estar presentes, é lógico substituir todas essas subconsultas por uma junção externa com uma única tabela derivada LATERAL, que calcula os valores de todas as colunas de uma vez.

```
SELECT
  H.CODE_HORSE,
  H.NAME AS HORSE_NAME,
  LR.CODE_FARM,
  LR.CODE_REASON,
  LR.BYDATE
FROM
  HORSE H
  LEFT JOIN LATERAL (
    SELECT
      R.CODE_FARM,
      R.CODE_REASON,
      R.BYDATE
    FROM REGISTRATION R
    WHERE R.CODE_HORSE = H.CODE_HORSE
      AND R.BYDATE <= DATE '2015-12-31'
    ORDER BY R.BYDATE DESC
    FETCH FIRST ROW ONLY
  ) LR ON TRUE
WHERE EXTRACT(YEAR FROM H.BIRTHDAY) = 2005;
```

A consulta ficou muito mais curta e elegante. Mas o mais importante é que ela é executada de forma muito mais eficiente. Veja o plano da nova consulta:

```
PLAN JOIN (H INDEX (HORSE_IDX_BIRTHYEAR), SORT (LR R INDEX (FK_REGISTRATION_HORSE)))

Select Expression
-> Filter
  -> Nested Loop Join (outer)
    -> Filter
      -> Table "HORSE" as "H" Access By ID
        -> Bitmap
          -> Index "HORSE_IDX_BIRTHYEAR" Range Scan (full match)
      -> First N Records
        -> Refetch
          -> Sort (record length: 28, key length: 8)
            -> Filter
              -> Table "REGISTRATION" as "LR R" Access By ID
                -> Bitmap
                  -> Index "FK_REGISTRATION_HORSE" Range Scan (full match)
```

Os dados de cada cavalo são lidos da tabela REGISTRATION apenas uma vez, em vez de ler esses dados em cada subconsulta.

Eliminação de duplicação de parte de consultas em UNION ALL

Considere a seguinte consulta:


```

WITH
  T AS (
    SELECT
      LAB_LINE.VALUE_1 AS LOKUS_VALUE
    FROM
      HORSE
      JOIN LAB ON LAB.CODE_HORSE = HORSE.CODE_HORSE
      JOIN LAB_LINE ON LAB_LINE.CODE_LAB = LAB.CODE_LAB
    WHERE LAB_LINE.VALUE_1 IS NOT NULL
      AND LAB_LINE.CODE_LAB = LAB.CODE_LAB
      AND LAB_LINE.CODE_LABSET = 68
      AND LAB.CODE_LABKIND = 2
      AND HORSE.CODE_BREED = 55
    UNION ALL
    SELECT
      LAB_LINE.VALUE_2 AS LOKUS_VALUE
    FROM
      HORSE
      JOIN LAB ON LAB.CODE_HORSE = HORSE.CODE_HORSE
      JOIN LAB_LINE ON LAB_LINE.CODE_LAB = LAB.CODE_LAB
    WHERE LAB_LINE.VALUE_2 IS NOT NULL
      AND LAB_LINE.CODE_LAB = LAB.CODE_LAB
      AND LAB_LINE.CODE_LABSET = 68
      AND LAB.CODE_LABKIND = 2
      AND HORSE.CODE_BREED = 55
  )
SELECT
  T.LOKUS_VALUE,
  COUNT(*) AS CNT
FROM
  T
GROUP BY 1;

```

O plano e as estatísticas de sua execução serão os seguintes:

```

Select Expression
-> Aggregate
  -> Sort (record length: 90, key length: 12)
    -> Union
      -> Nested Loop Join (inner)
        -> Filter
          -> Table "HORSE" as "T HORSE" Access By ID
            -> Bitmap
              -> Index "FK_HORSE_BREED" Range Scan (full match)
        -> Filter
          -> Table "LAB" as "T LAB" Access By ID
            -> Bitmap
              -> Index "FK_LAB_HORSE" Range Scan (full match)
        -> Filter
          -> Table "LAB_LINE" as "T LAB_LINE" Access By ID
            -> Bitmap
              -> Index "UNQ_LAB_LINE_SET" Unique Scan
      -> Nested Loop Join (inner)
        -> Filter

```

```

-> Table "HORSE" as "T HORSE" Access By ID
    -> Bitmap
        -> Index "FK_HORSE_BREED" Range Scan (full match)
-> Filter
    -> Table "LAB" as "T LAB" Access By ID
        -> Bitmap
            -> Index "FK_LAB_HORSE" Range Scan (full match)
-> Filter
    -> Table "LAB_LINE" as "T LAB_LINE" Access By ID
        -> Bitmap
            -> Index "UNQ_LAB_LINE_SET" Unique Scan

```

LOKUS_VALUE	CNT
I	1922
J	17
L	4806
M	10006
N	8039
O	3327
P	2513
Q	7
R	5129
S	1

```

Current memory = 553666240
Delta memory = 944
Max memory = 559661392
Elapsed time = 0.407 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 796524
Per table statistics:

```

Table name	Natural	Index	Insert	Update	Delete
HORSE		126220			
LAB		55506			
LAB_LINE		35802			

Se observarmos atentamente o texto da consulta, notaremos que em diferentes partes do UNION ALL é executada quase a mesma consulta. A diferença está apenas na coluna que é o resultado dessa consulta. Além disso, a ordem das junções em ambas as partes da consulta parece ser HORSE → LAB → LAB_LINE, e a coluna resultante é obtida da tabela LAB_LINE. E se primeiro executarmos a junção HORSE JOIN LAB uma única vez, e depois para o resultado dessa junção executarmos as diferentes partes da consulta para LAB_LINE? Isso pode ser facilmente feito com CROSS JOIN LATERAL. Vejamos o texto da consulta transformada:

```

WITH
  T AS (
    SELECT

```

```

L.LOKUS_VALUE
FROM
HORSE
JOIN LAB ON LAB.CODE_HORSE = HORSE.CODE_HORSE
CROSS JOIN LATERAL (
  SELECT
    LAB_LINE.VALUE_1 AS LOKUS_VALUE
  FROM LAB_LINE
  WHERE LAB_LINE.CODE_LAB = LAB.CODE_LAB
    AND LAB_LINE.VALUE_1 IS NOT NULL
    AND LAB_LINE.CODE_LABSET = 68
  UNION ALL
  SELECT
    LAB_LINE.VALUE_2 AS LOKUS_VALUE
  FROM LAB_LINE
  WHERE LAB_LINE.CODE_LAB = LAB.CODE_LAB
    AND LAB_LINE.VALUE_2 IS NOT NULL
    AND LAB_LINE.CODE_LABSET = 68
) L
WHERE LAB.CODE_LABKIND = 2
  AND HORSE.CODE_BREED = 55
)
SELECT
  T.LOKUS_VALUE,
  COUNT(*) AS CNT
FROM
  T
GROUP BY 1;

```

Vejamos o plano e as estatísticas da consulta transformada:

```

Select Expression
-> Aggregate
  -> Sort (record length: 138, key length: 12)
    -> Nested Loop Join (inner)
      -> Nested Loop Join (inner)
        -> Filter
          -> Table "HORSE" as "T HORSE" Access By ID
            -> Bitmap
              -> Index "FK_HORSE_BREED" Range Scan (full match)
        -> Filter
          -> Table "LAB" as "T LAB" Access By ID
            -> Bitmap
              -> Index "FK_LAB_HORSE" Range Scan (full match)
      -> Union
        -> Filter
          -> Table "LAB_LINE" as "T L LAB_LINE" Access By ID
            -> Bitmap
              -> Index "UNQ_LAB_LINE_SET" Unique Scan
        -> Filter
          -> Table "LAB_LINE" as "T L LAB_LINE" Access By ID
            -> Bitmap
              -> Index "UNQ_LAB_LINE_SET" Unique Scan

LOKUS_VALUE          CNT

```

```
=====
I              1922
J              17
L             4806
M            10006
N             8039
O             3327
P             2513
Q              7
R             5129
S              1
```

Current memory = 554613056

Delta memory = 848

Max memory = 562795552

Elapsed time = 0.273 sec

Buffers = 32768

Reads = 0

Writes = 0

Fetches = 487801

Per table statistics:

Table name	Natural	Index	Insert	Update	Delete
HORSE		63110			
LAB		27753			
LAB_LINE		35802			

A nova versão da consulta é 70% mais rápida. Para as tabelas HORSE e LAB, são realizadas metade das leituras.

Mas é possível alcançar um efeito similar sem usar LATERAL? Neste caso, sim. A tabela derivada L resultante não contém construções como FETCH/FIRST/ROWS, que tornam o uso de tabelas derivadas com LATERAL e sem semanticamente diferentes. Uma versão da consulta semanticamente equivalente sem o uso de LATERAL é apresentada abaixo.

```
WITH
  T AS (
    SELECT
      L.LOKUS_VALUE
    FROM
      HORSE
    JOIN LAB ON LAB.CODE_HORSE = HORSE.CODE_HORSE
    JOIN (
      SELECT
        LAB_LINE.CODE_LAB,
        LAB_LINE.VALUE_1 AS LOKUS_VALUE
      FROM LAB_LINE
      WHERE LAB_LINE.VALUE_1 IS NOT NULL
        AND LAB_LINE.CODE_LABSET = 68
    ) UNION ALL
    SELECT
      LAB_LINE.CODE_LAB,
```

```

        LAB_LINE.VALUE_2 AS LOKUS_VALUE
    FROM LAB_LINE
    WHERE LAB_LINE.VALUE_2 IS NOT NULL
        AND LAB_LINE.CODE_LABSET = 68
) L ON L.CODE_LAB = LAB.CODE_LAB
WHERE LAB.CODE_LABKIND = 2
    AND HORSE.CODE_BREED = 55
)
SELECT
    T.LOKUS_VALUE,
    COUNT(*) AS CNT
FROM
    T
GROUP BY 1;

```

Essa consulta é mais agradável e compreensível, mas vamos analisar seu plano e estatísticas de execução:

```

Select Expression
-> Aggregate
    -> Sort (record length: 146, key length: 12)
        -> Nested Loop Join (inner)
            -> Nested Loop Join (inner)
                -> Union
                    -> Filter
                        -> Table "LAB_LINE" as "T L LAB_LINE" Access By ID
                            -> Bitmap
                                -> Index "FK_LAB_LINE_LABSET" Range Scan (full match)
                    -> Filter
                        -> Table "LAB_LINE" as "T L LAB_LINE" Access By ID
                            -> Bitmap
                                -> Index "FK_LAB_LINE_LABSET" Range Scan (full match)
                -> Filter
                    -> Table "LAB" as "T LAB" Access By ID
                        -> Bitmap
                            -> Index "PK_LAB" Unique Scan
            -> Filter
                -> Table "HORSE" as "T HORSE" Access By ID
                    -> Bitmap
                        -> Index "PK_HORSE" Unique Scan

```

LOKUS_VALUE	CNT
I	1922
J	17
L	4806
M	10006
N	8039
O	3327
P	2513
Q	7
R	5129
S	1

```

Current memory = 556815760
Delta memory = 848
Max memory = 566905344
Elapsed time = 0.772 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 1376340
Per table statistics:

```

Table name	Natural	Index	Insert	Update	Delete
HORSE		138427			
LAB		138429			
LAB_LINE		139426			

Esta versão da consulta é executada 2,5 vezes mais devagar. Mas por que isso aconteceu? Como mencionado acima, expressões de tabela complexas ao usar junções do tipo Nested Loop sempre se tornam o fluxo líder. Neste caso, isso não é de forma alguma o que precisamos. Bem, podemos corrigir isso usando LEFT JOIN com a aplicação de um filtro com predicado NOT NULL ao fluxo seguidor. Agora a consulta ficará assim:

```

WITH
  T AS (
    SELECT
      L.LOKUS_VALUE
    FROM
      HORSE
    JOIN LAB ON LAB.CODE_HORSE = HORSE.CODE_HORSE
    LEFT JOIN (
      SELECT
        LAB_LINE.CODE_LAB,
        LAB_LINE.VALUE_1 AS LOKUS_VALUE
      FROM LAB_LINE
      WHERE LAB_LINE.VALUE_1 IS NOT NULL
        AND LAB_LINE.CODE_LABSET = 68
      UNION ALL
      SELECT
        LAB_LINE.CODE_LAB,
        LAB_LINE.VALUE_2 AS LOKUS_VALUE
      FROM LAB_LINE
      WHERE LAB_LINE.VALUE_2 IS NOT NULL
        AND LAB_LINE.CODE_LABSET = 68
    ) L ON L.CODE_LAB = LAB.CODE_LAB
    WHERE LAB.CODE_LABKIND = 2
      AND HORSE.CODE_BREED = 55
      AND L.CODE_LAB IS NOT NULL
  )
SELECT
  T.LOKUS_VALUE,
  COUNT(*) AS CNT
FROM
  T

```

```
GROUP BY 1;
```

O plano e as estatísticas de sua execução são os seguintes:

```
Select Expression
-> Aggregate
  -> Sort (record length: 146, key length: 12)
    -> Filter
      -> Nested Loop Join (outer)
        -> Nested Loop Join (inner)
          -> Filter
            -> Table "HORSE" as "T HORSE" Access By ID
              -> Bitmap
                -> Index "FK_HORSE_BREED" Range Scan (full match)
            -> Filter
              -> Table "LAB" as "T LAB" Access By ID
                -> Bitmap
                  -> Index "FK_LAB_HORSE" Range Scan (full match)
          -> Filter
            -> Union
              -> Filter
                -> Table "LAB_LINE" as "T L LAB_LINE" Access By ID
                  -> Bitmap
                    -> Index "UNQ_LAB_LINE_SET" Unique Scan
              -> Filter
                -> Table "LAB_LINE" as "T L LAB_LINE" Access By ID
                  -> Bitmap
                    -> Index "UNQ_LAB_LINE_SET" Unique Scan
```

LOKUS_VALUE	CNT
I	1922
J	17
L	4806
M	10006
N	8039
O	3327
P	2513
Q	7
R	5129
S	1

Current memory = 557130624

Delta memory = 944

Max memory = 567220208

Elapsed time = 0.289 sec

Buffers = 32768

Reads = 0

Writes = 0

Fetches = 487801

Per table statistics:

Table name	Natural	Index	Insert	Update	Delete
HORSE		63110			

LAB			27753			
LAB_LINE			35802			
-----+-----+-----+-----+-----+						

As estatísticas de execução desta consulta são quase idênticas à versão com `CROSS JOIN LATERAL`. Qual versão usar - decida você mesmo.



No caso de junção de mais de 2 tabelas, o uso de `CROSS JOIN LATERAL` nem sempre escolhe a ordem de junção ideal. Os requisitos para resolver dependências de fluxos sempre serão satisfeitos, porém no exemplo acima pode inesperadamente escolher uma ordem de junção como `LAB CROSS JOIN LATERAL L JOIN HORSE`. Para estabilizar a ordem, você pode substituir `CROSS JOIN LATERAL` por `LEFT JOIN LATERAL` + filtro `IS NOT NULL` no fluxo seguidor. No entanto, isso já será pouco diferente da versão com um `LEFT JOIN` comum.

Usando LATERAL para definir a ordem de junção com tabelas derivadas

Como mencionado acima, `CROSS JOIN LATERAL` no caso de junção de uma tabela com uma tabela derivada pode definir a ordem de junção, onde a tabela derivada se torna o fluxo conduzido para satisfazer as dependências.

Considere a seguinte consulta:

```
WITH
  T AS (
    SELECT
      HORSE.CODE_FARM,
      COUNT(*) AS CNT
    FROM HORSE
    GROUP BY 1
  )
SELECT
  FARM.NAME AS FARMNAME,
  T.CNT
FROM
  T
  JOIN FARM ON FARM.CODE_FARM = T.CODE_FARM
WHERE FARM.CODE_REGION = 1;
```

O plano e as estatísticas de execução desta consulta são os seguintes:

```
Select Expression
-> Nested Loop Join (inner)
  -> Aggregate
    -> Table "HORSE" as "T HORSE" Access By ID
      -> Index "FK_HORSE_FARMBORN" Full Scan
  -> Filter
    -> Table "FARM" Access By ID
      -> Bitmap
        -> Index "PK_FARM" Unique Scan
```



```

Current memory = 557336736
Delta memory = -375568
Max memory = 567220208
Elapsed time = 0.531 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 1354206
Per table statistics:

```

Table name	Natural	Index	Insert	Update	Delete
FARM		15243			
HORSE		562237			

Neste caso, o otimizador não escolheu a ordem de junção mais eficiente. Há muito menos fazendas com `CODE_REGION = 1` do que registros retornados pela expressão de tabela T. Normalmente, em tais casos, podemos usar `LEFT JOIN` com filtragem subsequente `IS NOT NULL` no campo de ligação da expressão de tabela. No entanto, neste caso, em vez de `LEFT JOIN`, podemos usar `CROSS JOIN LATERAL`; para isso, transformamos a consulta na seguinte forma equivalente:

```

SELECT
  FARM.NAME AS FARMNAME,
  T.CNT
FROM
  FARM
  CROSS JOIN LATERAL (
    SELECT
      COUNT(*) AS CNT
    FROM HORSE
    WHERE HORSE.CODE_FARM = FARM.CODE_FARM
  ) T
WHERE FARM.CODE_REGION = 1;

```

Vejamos as estatísticas e o plano de execução da nova consulta:

```

Select Expression
  -> Nested Loop Join (inner)
    -> Filter
      -> Table "FARM" Access By ID
        -> Bitmap
          -> Index "FK_FARM_REGION" Range Scan (full match)
    -> Aggregate
      -> Filter
        -> Table "HORSE" as "T HORSE" Access By ID
          -> Bitmap
            -> Index "FK_HORSE_FARMBORN" Range Scan (full match)

Current memory = 557379632

```

```
Delta memory = 320
Max memory = 567220208
Elapsed time = 0.073 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 2767
Per table statistics:
```

Table name	Natural	Index	Insert	Update	Delete
FARM		153			
HORSE		1333			

Pelas estatísticas, fica claro que esta variante da consulta é muito mais rápida do que a variante com INNER JOIN comum.

8.4. Junções de três ou mais tabelas

Acima, consideramos a lógica do otimizador ao juntar dois fluxos de dados, bem como maneiras de influenciar o algoritmo e a ordem de junção. Parece que, se podemos facilmente gerenciar a junção de dois fluxos, o mesmo valeria para três ou mais fluxos. Na realidade, não é tão simples. Consideremos as principais dificuldades de gerenciamento de junções que surgem com o aumento do número de fluxos a serem unidos:

- ao alterar o algoritmo ou a ordem de junção dos fluxos A e B, o otimizador pode, inesperadamente para você, mudar o algoritmo e a ordem de junção para o fluxo C;
- para tipos mistos de junções (externas + internas), o otimizador não pode alterar a ordem das junções externas;
- a transformação de OUTER JOIN em INNER JOIN não funciona em cascata. Isso é descrito em detalhes em [Transformação de OUTER JOIN em INNER JOIN](#).

Consideraremos cada um desses problemas e as maneiras de resolvê-los separadamente.

8.4.1. Definindo a ordem e/ou algoritmo para junções internas de três tabelas

Considere a seguinte consulta, que usa a junção de três tabelas:

```
SELECT
  COUNT(*)
FROM
  HORSE
  JOIN COLOR ON COLOR.CODE_COLOR = HORSE.CODE_COLOR
  JOIN BREED ON BREED.CODE_BREED = HORSE.CODE_BREED
;
```

Seu plano e estatísticas serão os seguintes:

```
PLAN HASH (JOIN (COLOR NATURAL, HORSE INDEX (FK_HORSE_COLOR)), BREED NATURAL)
```

```
Select Expression
```

```
-> Aggregate
```

```
-> Filter
```

```
-> Hash Join (inner)
```

```
-> Nested Loop Join (inner)
```

```
-> Table "COLOR" Full Scan
```

```
-> Filter
```

```
-> Table "HORSE" Access By ID
```

```
-> Bitmap
```

```
-> Index "FK_HORSE_COLOR" Range Scan (full match)
```

```
-> Record Buffer (record length: 25)
```

```
-> Table "BREED" Full Scan
```

```
COUNT
```

```
=====
```

```
562237
```

```
Current memory = 557777760
```

```
Delta memory = 256
```

```
Max memory = 567220208
```

```
Elapsed time = 0.448 sec
```

```
Buffers = 32768
```

```
Reads = 0
```

```
Writes = 0
```

```
Fetches = 664949
```

```
Per table statistics:
```

Table name	Natural	Index	Insert	Update	Delete
BREED	290				
COLOR	242				
HORSE		562237			

Como pode ser visto no plano, duas tabelas são unidas pelo método Nested Loop Join, após o que a terceira tabela é unida a elas pelo método Hash Join. Como ambas as tabelas COLOR e BREED são suficientemente pequenas, é lógico supor que ambas podem ser "hasheadas", e o uso de Hash Join aqui para todos os fluxos seria mais barato. A tabela BREED já está "hasheada"; pela lógica, basta desativar o uso do índice FK_HORSE_COLOR, e então o algoritmo Hash Join será usado para ambas as tabelas. Vamos tentar fazer isso usando a dica +0.

```
SELECT
  COUNT(*)
FROM
  HORSE
  JOIN COLOR ON COLOR.CODE_COLOR = HORSE.CODE_COLOR+0
  JOIN BREED ON BREED.CODE_BREED = HORSE.CODE_BREED
;
```

Vejamos o que obtivemos:

```
PLAN HASH (JOIN (BREED NATURAL, HORSE INDEX (FK_HORSE_BREED)), COLOR NATURAL)
```

```
Select Expression
```

```
-> Aggregate
```

```
-> Filter
```

```
-> Hash Join (inner)
```

```
-> Nested Loop Join (inner)
```

```
-> Table "BREED" Full Scan
```

```
-> Filter
```

```
-> Table "HORSE" Access By ID
```

```
-> Bitmap
```

```
-> Index "FK_HORSE_BREED" Range Scan (full match)
```

```
-> Record Buffer (record length: 25)
```

```
-> Table "COLOR" Full Scan
```

```
COUNT
```

```
=====
```

```
562237
```

```
Current memory = 557879504
```

```
Delta memory = 256
```

```
Max memory = 567220208
```

```
Elapsed time = 0.495 sec
```

```
Buffers = 32768
```

```
Reads = 0
```

```
Writes = 0
```

```
Fetches = 645516
```

```
Per table statistics:
```

Table name	Natural	Index	Insert	Update	Delete
BREED	290				
COLOR	242				
HORSE		562237			

Agora a tabela COLOR está "hasheada", mas inesperadamente a junção das tabelas BREED e HORSE passou a ser executada pelo método Nested Loop Join. Ou seja, não obtivemos o resultado esperado: a alteração do algoritmo de junção de duas tabelas influenciou a escolha do algoritmo de junção com a terceira tabela. Agora, para forçar o otimizador a usar Hash Join, é necessário também desativar o uso do índice FK_HORSE_BREED.

```
SELECT
  COUNT(*)
FROM
  HORSE
  JOIN COLOR ON COLOR.CODE_COLOR = HORSE.CODE_COLOR+0
  JOIN BREED ON BREED.CODE_BREED = HORSE.CODE_BREED+0
;
```

Agora o plano será como desejamos:

```

Select Expression
  -> Aggregate
    -> Filter
      -> Hash Join (inner)
        -> Hash Join (inner)
          -> Table "HORSE" Full Scan
          -> Record Buffer (record length: 25)
            -> Table "COLOR" Full Scan
            -> Record Buffer (record length: 25)
              -> Table "BREED" Full Scan

```

COUNT

```

=====
562237

```

Current memory = 557749200

Delta memory = -173488

Max memory = 567220208

Elapsed time = 0.551 sec

Buffers = 32768

Reads = 0

Writes = 0

Fetches = 602293

Per table statistics:

Table name	Natural	Index	Insert	Update	Delete
BREED	290				
COLOR	242				
HORSE	562237				

Este exemplo demonstra claramente que quanto mais junções em uma consulta, mais fatores influenciam a ordem de escolha das junções. Assim, o uso de dicas nem sempre leva ao resultado que você esperava.



Na verdade, é praticamente impossível prever se uma dica levará ao resultado desejado ou não. Para três tabelas, teoricamente é possível calcular a lógica do otimizador, mas não é tão simples (há muitas variantes). Se o número de tabelas aumenta, realizar tais cálculos torna-se praticamente inviável para um ser humano.

O que fazer neste caso? A resposta correta é verificar suas hipóteses e, se elas forem infrutíferas, tentar refiná-las com outras dicas ou até mesmo tentar otimizar por outro ângulo.

Tudo bem, você dirá, mas existe o `LEFT JOIN`, que pode definir exatamente a ordem de junção. Aqui também não é tão simples. Existem dois problemas principais.

O primeiro problema é o seguinte. Se predicados de filtragem que não processam `NULL` na cláusula `WHERE` são usados para tabelas detalhes em junções externas unidirecionais, tais junções podem ser convertidas em internas. Como resultado, a sua ordem de junção pode simplesmente deixar de ser considerada. Para resolver este problema, você pode definir `OuterJoinConversion = false` na

configuração, ou mover o predicado de filtragem para a condição de junção e adicionar um predicado adicional `IS NOT NULL` no `WHERE` para filtrar pelo campo de junção da tabela detalhe. Isto é descrito em mais detalhes em [Transformação de OUTER JOIN em INNER JOIN](#) e [E se for necessário evitar a transformação de junções externas?](#). O último método complica um pouco a compreensão da consulta, especialmente se houver muitas dessas tabelas, unidas por junção externa esquerda como uma dica.

O segundo problema é mais complexo. Ele ocorre ao misturar junções externas e internas em uma única consulta.

8.4.2. Junções internas e externas em uma única consulta

Se apenas junções internas são usadas em uma consulta, não importa a ordem em que você as escreve, o otimizador escolherá a ordem de junção por si mesmo. No entanto, com junções externas unidirecionais isso não é verdade: em uma junção `A LEFT JOIN B`, `A` sempre será o fluxo principal e `B` será o detalhe. Mas isso ainda não é tudo. Se apenas junções externas unidirecionais são usadas em uma consulta, essas junções não podem ser reordenadas entre si, mesmo que isso não altere a semântica da consulta. Além disso, junções internas e externas também não podem ser trocadas de lugar.

Vamos considerar o exemplo da seguinte consulta:

```
SELECT
  COUNT(*)
FROM
  HORSE
  LEFT JOIN COLOR ON COLOR.CODE_COLOR = HORSE.CODE_COLOR
  JOIN FARM ON FARM.CODE_FARM = HORSE.CODE_FARM
WHERE FARM.CODE_REGION = 1;
```

Esta consulta será executada exatamente como está escrita: `(HORSE LEFT JOIN COLOR) JOIN FARM`, embora um resultado semelhante possa ser obtido executando-a como `(HORSE JOIN FARM) LEFT JOIN COLOR`. Vamos olhar o plano e as estatísticas de execução desta consulta:

```
PLAN JOIN (JOIN (HORSE NATURAL, COLOR INDEX (PK_COLOR)), FARM INDEX (PK_FARM))
```

Select Expression

-> Aggregate

-> Nested Loop Join (inner)

-> Nested Loop Join (outer)

-> Table "HORSE" Full Scan

-> Filter

-> Table "COLOR" Access By ID

-> Bitmap

-> Index "PK_COLOR" Unique Scan

-> Filter

-> Table "FARM" Access By ID

-> Bitmap

-> Index "PK_FARM" Unique Scan

```

COUNT
=====
1333

Current memory = 553967120
Delta memory = 288
Max memory = 554056560
Elapsed time = 1.905 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 4613085
Per table statistics:
-----+-----+-----+-----+-----+
Table name | Natural | Index | Insert | Update | Delete |
-----+-----+-----+-----+-----+
COLOR      |         | 562237|         |         |         |
FARM       |         | 562237|         |         |         |
HORSE      | 562237 |         |         |         |         |
-----+-----+-----+-----+-----+

```

Agora vamos trocar a ordem do JOIN e do LEFT JOIN e comparar os planos e estatísticas.

```

SELECT
  COUNT(*)
FROM
  HORSE
  JOIN FARM ON FARM.CODE_FARM = HORSE.CODE_FARM
  LEFT JOIN COLOR ON COLOR.CODE_COLOR = HORSE.CODE_COLOR
WHERE FARM.CODE_REGION = 1;

```

```

Select Expression
-> Aggregate
  -> Filter
    -> Nested Loop Join (outer)
      -> Nested Loop Join (inner)
        -> Filter
          -> Table "FARM" Access By ID
            -> Bitmap
              -> Index "FK_FARM_REGION" Range Scan (full match)
        -> Filter
          -> Table "HORSE" Access By ID
            -> Bitmap
              -> Index "FK_HORSE_FARMBORN" Range Scan (full match)
      -> Filter
        -> Table "COLOR" Access By ID
          -> Bitmap
            -> Index "PK_COLOR" Unique Scan

```

```

COUNT
=====
1333

```

```

Current memory = 554060768
Delta memory = 288
Max memory = 554093024
Elapsed time = 0.007 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 6764
Per table statistics:

```

Table name	Natural	Index	Insert	Update	Delete
COLOR		1333			
FARM		153			
HORSE		1333			

Uau! O resultado é o mesmo, mas esta consulta é executada 270 vezes mais rápido! Qual é o motivo?

O motivo é que a junção interna de dois fluxos de dados pode reduzir a cardinalidade resultante, enquanto a cardinalidade resultante de uma junção externa unidirecional nunca será menor do que a cardinalidade do fluxo principal. O otimizador poderia, por si só, trocar a ordem das junções, movendo o LEFT JOIN para depois do INNER JOIN, se isso não alterasse a semântica da consulta, mas atualmente ele não sabe fazer isso.



Se a sua consulta contém tanto junções internas quanto LEFT JOIN, sempre tente posicionar o LEFT JOIN após as junções internas, se isso não alterar a semântica da consulta.

O problema descrito também afeta aqueles que dão dicas ao otimizador usando LEFT JOIN. Ou seja, você deve entender que, ao alterar o tipo de consulta para dar uma dica, você também terá que alterar a ordem das junções na seção FROM.

Parece que entendemos. Sempre tentamos colocar LEFT JOIN depois de INNER JOIN. Mas isso é sempre possível? Claro que não. Se você usa LEFT JOIN em uma visão e depois junta essa visão com uma tabela usando uma junção interna comum, a ordem de junção mais otimizada não será usada. Aqui, pode-se aconselhar a reescrever essa consulta sem usar a visão, ou juntar a própria visão como uma tabela detalhe usando LEFT JOIN.

Suponha que você tenha uma visão como esta, contendo um LEFT JOIN:

```

CREATE VIEW V_HORSE_C
AS
SELECT
    HORSE.CODE_HORSE,
    HORSE.CODE_FARM,
    HORSE.NAME,
    COLOR.NAME AS COLORNAME
FROM
    HORSE
LEFT JOIN COLOR ON COLOR.CODE_COLOR = HORSE.CODE_COLOR;

```


Agora, em sua consulta, você junta esta visão com uma tabela usando uma junção interna:

```
SELECT
  COUNT(*)
FROM
  V_HORSE_C H
  JOIN FARM ON FARM.CODE_FARM = H.CODE_FARM
WHERE FARM.CODE_REGION = 1;
```

Vejamos o plano e as estatísticas:

```
Select Expression
-> Aggregate
  -> Nested Loop Join (inner)
    -> Nested Loop Join (outer)
      -> Table "HORSE" as "H HORSE" Full Scan
      -> Filter
        -> Table "COLOR" as "H COLOR" Access By ID
          -> Bitmap
            -> Index "PK_COLOR" Unique Scan
        -> Filter
          -> Table "FARM" Access By ID
            -> Bitmap
              -> Index "PK_FARM" Unique Scan
```

COUNT

```
=====
1333
```

Current memory = 554319840

Delta memory = 224

Max memory = 555809392

Elapsed time = 1.795 sec

Buffers = 32768

Reads = 0

Writes = 0

Fetches = 4613085

Per table statistics:

Table name	Natural	Index	Insert	Update	Delete
COLOR		562237			
FARM		562237			
HORSE	562237				

O problema é exatamente o mesmo do exemplo em que o `LEFT JOIN` estava antes do `JOIN`. O que fazer, já que nem sempre podemos reescrever a visão? A primeira opção é fazer da tabela `FARM` o fluxo principal. Para isso, vamos juntar a visão usando `LEFT JOIN`. Não se esqueça de adicionar um filtro `IS NOT NULL` no campo que participa da condição de junção, para que a consulta seja completamente equivalente.

```

SELECT
  COUNT(*)
FROM
  FARM
  LEFT JOIN V_HORSE_C H ON H.CODE_FARM = FARM.CODE_FARM
WHERE FARM.CODE_REGION = 1
  AND H.CODE_FARM IS NOT NULL;

```

Vejamos o plano e as estatísticas:

```

Select Expression
  -> Aggregate
    -> Filter
      -> Nested Loop Join (outer)
        -> Filter
          -> Table "FARM" Access By ID
            -> Bitmap
              -> Index "FK_FARM_REGION" Range Scan (full match)
          -> Filter
            -> Nested Loop Join (outer)
              -> Filter
                -> Table "HORSE" as "H HORSE" Access By ID
                  -> Bitmap
                    -> Index "FK_HORSE_FARMBORN" Range Scan (full match)
                -> Filter
                  -> Table "COLOR" as "H COLOR" Access By ID
                    -> Bitmap
                      -> Index "PK_COLOR" Unique Scan

```

COUNT

```

=====
1333

```

Current memory = 554461872

Delta memory = 288

Max memory = 555809392

Elapsed time = 0.005 sec

Buffers = 32768

Reads = 0

Writes = 0

Fetches = 6763

Per table statistics:

Table name	Natural	Index	Insert	Update	Delete
COLOR		1333			
FARM		153			
HORSE		1333			

A consulta usando a visão agora é executada de forma eficiente.

A segunda opção — simplesmente não use a view. Você sempre pode reescrever a mesma consulta

sem usar uma view. Tais transformações são feitas em duas etapas. Primeiro, substitua sua view por uma expressão de tabela comum (CTE).

```
WITH
  H AS (
    SELECT
      HORSE.CODE_HORSE,
      HORSE.CODE_FARM,
      HORSE.NAME,
      COLOR.NAME AS COLORNAME
    FROM
      HORSE
    LEFT JOIN COLOR ON COLOR.CODE_COLOR = HORSE.CODE_COLOR
  )
SELECT
  COUNT(*)
FROM
  H
  JOIN FARM ON FARM.CODE_FARM = H.CODE_FARM
WHERE FARM.CODE_REGION = 1;
```

Agora, vendo a consulta completa, você pode expandir a expressão de tabela.

```
SELECT
  COUNT(*)
FROM
  HORSE
  LEFT JOIN COLOR ON COLOR.CODE_COLOR = HORSE.CODE_COLOR
  JOIN FARM ON FARM.CODE_FARM = HORSE.CODE_FARM
WHERE FARM.CODE_REGION = 1;
```

Quando a consulta está escrita nessa forma, você sempre pode facilmente reordenar as junções. No final, você obterá a seguinte consulta:

```
SELECT
  COUNT(*)
FROM
  HORSE
  JOIN FARM ON FARM.CODE_FARM = HORSE.CODE_FARM
  LEFT JOIN COLOR ON COLOR.CODE_COLOR = HORSE.CODE_COLOR
WHERE FARM.CODE_REGION = 1;
```

8.5. Conclusão

Neste capítulo, examinamos em detalhes como o otimizador escolhe o algoritmo e a ordem de junção dos fluxos de dados. Também foi mostrado como você pode influenciar a escolha do algoritmo e da ordem das junções sem especificar planos explicitamente. Espero que esse conhecimento ajude você na otimização de suas consultas. Agora, vamos considerar outros aspectos da otimização.

Capítulo 9. Otimização de ordenações

A ordenação é a organização de dados por alguma característica. Por si só, a ordenação é aplicada em muitos métodos de acesso, por exemplo, para separação em grupos ao calcular agregados, divisão em seções ou ordenação em uma janela, para eliminação de duplicatas. Nesta seção, falaremos sobre a ordenação de um fluxo de dados usando a cláusula `ORDER BY`.

De que forma os dados podem ser ordenados? No Firebird, dois algoritmos são usados para ordenação:

- Navegação por índice (ordenação por índice). Para mais detalhes, consulte [Navegação por índice](#);
- Ordenação externa. Para mais detalhes, consulte [Ordenação](#);
 - Variante otimizada da ordenação externa usando Refetch. Para mais detalhes, consulte [Refetch](#).

Sobre a navegação por índice, as condições sob as quais ela é possível e as dicas para o otimizador, discutimos detalhadamente ao falar sobre índices (consulte [Usando um índice para ordenação](#)). Neste capítulo, falaremos sobre o custo da ordenação externa, do que ele depende e sobre maneiras de reduzir esse custo.

Quando temos uma consulta simples de uma única tabela, o otimizador aplica a seguinte regra: se existir um índice pela chave de ordenação, será aplicada a navegação por índice, caso contrário, será aplicada a ordenação externa. Portanto, no exemplo a seguir, será escolhido um campo não indexado como chave de ordenação.

9.1. Estimativa do custo da ordenação externa

Vejamos o exemplo da seguinte consulta:

```
SELECT
  CODE_FARM,
  NAME,
  CREATE_DATE
FROM FARM
ORDER BY CREATE_DATE;
```

Seu plano e estatísticas de execução serão:

```
PLAN SORT (FARM NATURAL)

Select Expression
  -> Sort (record length: 452, key length: 12)
    -> Table "FARM" Full Scan
Current memory = 554974992
Delta memory = 0
Max memory = 582988864
Elapsed time = 0.224 sec
```

```

Buffers = 32768
Reads = 0
Writes = 0
Fetches = 46733
Per table statistics:

```

Table name	Natural	Index	Insert	Update	Delete
FARM	41356				

Então, do que depende o custo da ordenação externa? Obviamente, ele depende da cardinalidade do fluxo a ser ordenado. No melhor caso, a complexidade do algoritmo de ordenação é estimada como $O(\log_2(N) * N)$, onde N é a cardinalidade do fluxo a ser ordenado. Mas será que apenas a cardinalidade influencia o custo da ordenação? Bem, vamos lembrar como a ordenação externa é executada no Firebird (consulte [Ordenação](#)). Ou seja, a ordenação externa tenta ser executada na memória, e se ela for insuficiente, são usados arquivos externos no disco.

No plano Explain, a ordenação é indicada pela palavra “Sort” para o modo normal e “Unique Sort” para o modo de truncagem. Em seguida, entre parênteses, são indicados o comprimento dos registros a serem ordenados (record length) e o comprimento da chave de ordenação (key length). Os registros a serem ordenados sempre incluem a chave de ordenação.

É possível estimar o consumo de memória para a ordenação multiplicando o comprimento dos registros a serem ordenados (record length) pela cardinalidade do fluxo a ser ordenado. Por padrão, o Firebird tentará executar a ordenação na memória operacional; assim que essa memória se esgotar, o mecanismo começará a usar arquivos temporários. O volume de memória operacional disponível para ordenações é definido pelo parâmetro TempCacheLimit. Na arquitetura Classic, esse limite é definido para cada conexão, enquanto nas arquiteturas SuperServer e SuperClassic, é para todas as conexões do banco de dados.

Mesmo que você tenha alocado memória suficiente para que a ordenação caiba nela, ainda assim, quanto mais bytes você copiar ou ler da memória, mais caro será. Se a ordenação começar a usar arquivos temporários, seu custo aumenta drasticamente, pois qualquer dispositivo de disco é ordens de magnitude mais lento que a memória operacional.

Assim, você pode reduzir o custo da ordenação externa de uma das duas formas: reduzir a cardinalidade do fluxo para ordenação ou reduzir a largura do registro.

Como, durante a execução da consulta, a ordenação no ORDER BY é uma das últimas etapas, a aplicação de quaisquer filtros no WHERE reduz a cardinalidade do fluxo antes da ordenação.

```

SELECT
  CODE_FARM,
  NAME,
  CREATE_DATE
FROM FARM
WHERE CREATE_DATE IS NOT NULL
ORDER BY CREATE_DATE;

```

```
Select Expression
  -> Sort (record length: 452, key length: 12)
    -> Filter
      -> Table "FARM" Full Scan
```

Current memory = 555004432

Delta memory = 26736

Max memory = 582988864

Elapsed time = 0.101 sec

Buffers = 32768

Reads = 0

Writes = 0

Fetches = 46736

Per table statistics:

Table name	Natural	Index	Insert	Update	Delete
FARM	41356				

Resolvemos a questão da cardinalidade. Mas do que depende a largura do registro para ordenação? A largura da ordenação depende do tamanho e da quantidade dos campos usados nas seções SELECT e ORDER BY. Campos calculados e campos obtidos de subconsultas não estão incluídos, pois seu cálculo é adiado até o momento em que os registros começarem a ser retornados um a um do buffer de ordenação. Vamos ver isso com exemplos.

```
SELECT
  CODE_FARM,
  NAME,
  NAME_EN,
  CREATE_DATE
FROM FARM
ORDER BY CREATE_DATE;
```

```
Select Expression
  -> Sort (record length: 852, key length: 12)
    -> Table "FARM" Full Scan
```

O campo NAME_EN da tabela FARM tem o tipo VARCHAR(100) CHARACTER SET UTF8, ou seja, seu tamanho é de 400 bytes. Isso corresponde exatamente à alteração na largura do registro que você pode ver em record length.

Agora, vamos adicionar ao SELECT uma expressão a partir de campos já existentes.

```
SELECT
  CODE_FARM,
  NAME,
  NAME_EN,
  NAME || ' ' || NAME_EN AS MIXED_NAME,
  CREATE_DATE
```

```
FROM FARM
ORDER BY CREATE_DATE;
```

Select Expression

-> Sort (record length: 852, key length: 12)
-> Table "FARM" Full Scan

Como você pode ver, a largura do registro para ordenação não mudou. O mesmo ocorrerá com o uso de subconsultas.

Observe a consulta.

```
SELECT
  CODE_FARM,
  CREATE_DATE
FROM FARM
ORDER BY CREATE_DATE;
```

Seu plano é o seguinte.

Select Expression

-> Sort (record length: 44, key length: 12)
-> Table "FARM" Full Scan

E agora, vamos adicionar uma subconsulta.

```
SELECT
  CODE_FARM,
  (SELECT NAME FROM COUNTRY
   WHERE COUNTRY.CODE_COUNTRY = FARM.CODE_COUNTRY) AS COUNTRY_NAME,
  CREATE_DATE
FROM FARM
ORDER BY CREATE_DATE;
```

Sub-query

-> Singularity Check
-> Filter
-> Table "COUNTRY" Access By ID
-> Bitmap
-> Index "PK_COUNTRY" Unique Scan

Select Expression

-> Sort (record length: 52, key length: 12)
-> Table "FARM" Full Scan

A largura do registro aumentou, mas não pelo tamanho do campo COUNTRY.NAME, e sim pelo tamanho do campo FARM.CODE_COUNTRY. Você pode facilmente verificar isso observando o plano da consulta com o campo CODE_COUNTRY na seção SELECT, mas sem usar a subconsulta.

```
SELECT
  CODE_FARM,
  CODE_COUNTRY,
  CREATE_DATE
FROM FARM
ORDER BY CREATE_DATE;
```

```
Select Expression
  -> Sort (record length: 52, key length: 12)
      -> Table "FARM" Full Scan
```

Agora, vejamos exemplos de ordenação com consultas que possuem junções de várias tabelas.

```
SELECT
  FARM.CODE_FARM,
  FARM.NAME,
  COUNTRY.NAME AS COUNTRY_NAME,
  FARM.CREATE_DATE
FROM
  FARM
JOIN COUNTRY ON COUNTRY.CODE_COUNTRY = FARM.CODE_COUNTRY
ORDER BY FARM.CREATE_DATE;
```

```
Select Expression
  -> Sort (record length: 628, key length: 12)
      -> Nested Loop Join (inner)
          -> Table "COUNTRY" Full Scan
              -> Filter
                  -> Table "FARM" Access By ID
                      -> Bitmap
                          -> Index "FK_FARM_COUNTRY" Range Scan (full match)
```

Aqui, aplicam-se as mesmas regras que para consultas de uma única tabela – quanto mais campos são usados no SELECT da fonte de dados a ser ordenada (resultado da junção de duas tabelas), maior será a largura do registro a ser ordenado.

Agora você sabe o que influencia o custo da ordenação externa. Quanto mais campos são especificados no SELECT, mais largo será o registro para ordenação e mais memória será usada para a ordenação, e se ela não for suficiente, serão usados arquivos temporários. Portanto, é importante especificar no SELECT apenas os campos que são realmente necessários. A pior situação será se no SELECT for usado o asterisco *.

Mas e se você selecionar apenas os campos que realmente precisa e a largura da ordenação for muito grande? Para tais casos, a partir do Firebird 4.0, está prevista a otimização de ordenações externas largas usando Refetch (você pode ler mais sobre isso em [Refetch](#)).

9.2. Otimização de ordenações externas amplas (Refetch)

Vamos ver como funciona a otimização de seleções amplas usando Refetch. Para isso, modificaremos nossa consulta à tabela FARM, adicionando ao SELECT vários campos de grande tamanho (geralmente campos do tipo VARCHAR(N)).

```
SELECT
  FARM.CODE_FARM,
  FARM.NAME,
  FARM.NAME_EN,
  FARM.ADDRESS,
  FARM.CREATE_DATE
FROM
  FARM
ORDER BY FARM.CREATE_DATE;
```

Os campos NAME e NAME_EN são do tipo VARCHAR(100) CHARACTER SET UTF8, e o campo ADDRESS é VARCHAR(150) CHARACTER SET UTF8. Portanto, o tamanho total apenas dos três campos de string é $100 * 4 + 100 * 4 + 150 * 4 = 1400$ bytes.

Mas vejamos o plano de execução desta consulta:

```
Select Expression
  -> Refetch
    -> Sort (record length: 36, key length: 12)
      -> Table "FARM" Full Scan
Current memory = 558617840
Delta memory = 256
Max memory = 591167152
Elapsed time = 0.307 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 90062
Per table statistics:
```

Table name	Natural	Index	Insert	Update	Delete
FARM	41356				

Como assim, calculamos que o tamanho do registro para ordenação deveria ser mais de 1400 bytes, mas no plano é mostrado apenas 36 bytes. Aqui é onde a otimização [Refetch](#) entrou em ação. No buffer de ordenação foi colocado apenas a chave e o DB_KEY. Após a ordenação de {CREATE_DATE, DB_KEY}, os registros da tabela FARM foram extraídos novamente usando o DB_KEY. É por isso que essa otimização se chama Refetch (extrair novamente). Esquemáticamente, aconteceu o seguinte.

```
WITH
```

```

T (DB_KEY) AS (
  SELECT
    FARM.RDB$DB_KEY
  FROM
    FARM
  ORDER BY FARM.CREATE_DATE
)
SELECT
  FARM.CODE_FARM,
  FARM.NAME,
  FARM.NAME_EN,
  FARM.ADDRESS,
  FARM.CREATE_DATE
FROM
  T
JOIN FARM ON FARM.RDB$DB_KEY = T.DB_KEY;

```

Se você observar o plano desta consulta, a largura do registro para ordenação terá exatamente esse valor.

```

Select Expression
  -> Nested Loop Join (inner)
    -> Sort (record length: 36, key length: 12)
      -> Table "FARM" as "T FARM" Full Scan
    -> Filter
      -> Table "FARM" Access By ID
        -> DBKEY
Current memory = 558598064
Delta memory = 384
Max memory = 591167152
Elapsed time = 0.308 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 90062
Per table statistics:

```

Table name	Natural	Index	Insert	Update	Delete
FARM	41356	41356			

Claro, na realidade o otimizador do Firebird faz isso de forma um pouco diferente (não há nenhuma junção), mas para entender o significado de Refetch este exemplo é útil.

Se o método Refetch será usado para otimizar a ordenação externa depende do tamanho total do registro para ordenação. Se ele exceder o valor especificado no parâmetro de configuração InlineSortThreshold, então a otimização via Refetch é ativada. Por padrão, InlineSortThreshold = 1000, e o tamanho do registro para ordenação neste exemplo é maior que 1400 bytes, portanto Refetch foi utilizado.

O método Refetch também é aplicável a consultas com junção de várias tabelas.

```

SELECT
  FARM.CODE_FARM,
  FARM.NAME,
  FARM.NAME_EN,
  FARM.ADDRESS,
  COUNTRY.NAME AS COUNTRY_NAME,
  FARM.CREATE_DATE
FROM
  FARM
JOIN COUNTRY ON COUNTRY.CODE_COUNTRY = FARM.CODE_COUNTRY
ORDER BY FARM.CREATE_DATE;

```

O plano e as estatísticas de execução da consulta serão as seguintes:

```

Select Expression
-> Refetch
    -> Sort (record length: 52, key length: 12)
        -> Nested Loop Join (inner)
            -> Table "COUNTRY" Full Scan
            -> Filter
                -> Table "FARM" Access By ID
                    -> Bitmap
                        -> Index "FK_FARM_COUNTRY" Range Scan (full match)

```

Current memory = 558478128

Delta memory = 352

Max memory = 563271328

Elapsed time = 0.341 sec

Buffers = 32768

Reads = 0

Writes = 0

Fetches = 131209

Per table statistics:

Table name	Natural	Index	Insert	Update	Delete
COUNTRY	93				
FARM		41356			

Se tivéssemos que reescrever esta consulta manualmente, ela ficaria assim:

```

WITH
  T (FARM_DB_KEY, COUNTRY_DB_KEY) AS (
    SELECT
      FARM.RDB$DB_KEY,
      COUNTRY.RDB$DB_KEY
    FROM
      FARM
    JOIN COUNTRY ON COUNTRY.CODE_COUNTRY = FARM.CODE_COUNTRY
    ORDER BY FARM.CREATE_DATE
  )
SELECT

```

```

FARM.CODE_FARM,
FARM.NAME,
FARM.NAME_EN,
FARM.ADDRESS,
FARM.CREATE_DATE
FROM
T
JOIN FARM ON FARM.RDB$DB_KEY = T.FARM_DB_KEY
JOIN COUNTRY ON COUNTRY.RDB$DB_KEY = T.COUNTRY_DB_KEY;

```

```

Select Expression
-> Nested Loop Join (inner)
  -> Sort (record length: 60, key length: 12)
    -> Nested Loop Join (inner)
      -> Table "COUNTRY" as "T COUNTRY" Full Scan
      -> Filter
        -> Table "FARM" as "T FARM" Access By ID
          -> Bitmap
            -> Index "FK_FARM_COUNTRY" Range Scan (full match)

      -> Filter
        -> Table "FARM" Access By ID
          -> DBKEY

    -> Filter
      -> Table "COUNTRY" Access By ID
        -> DBKEY

Current memory = 563021584
Delta memory = 560
Max memory = 567226944
Elapsed time = 0.320 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 131620
Per table statistics:

```

Table name	Natural	Index	Insert	Update	Delete
COUNTRY	93	41356			
FARM		82712			

Ok, entendemos como funciona a otimização de ordenações pelo método Refetch, mas será que é sempre melhor do que a ordenação externa comum? Para responder a essa pergunta, vamos fazer um experimento. Aumentaremos o valor do parâmetro `InlineSortThreshold` para 4000, executaremos nossas consultas e compararemos o desempenho.

```

SELECT
  FARM.CODE_FARM,
  FARM.NAME,
  FARM.NAME_EN,
  FARM.ADDRESS,
  FARM.CREATE_DATE

```

```
FROM
  FARM
ORDER BY FARM.CREATE_DATE;
```

Agora o plano e as estatísticas serão assim:

```
Select Expression
  -> Sort (record length: 1452, key length: 12)
    -> Table "FARM" Full Scan
Current memory = 551598480
Delta memory = 256
Max memory = 622906688
Elapsed time = 0.349 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 46733
Per table statistics:
```

Table name	Natural	Index	Insert	Update	Delete
FARM	41356				

Esta consulta é um pouco mais lenta: 0.349 segundos comparado a 0.307, mas usa quase metade das leituras lógicas: 46733 comparado a 90062 (se o cache de páginas for pequeno, isso pode ser um problema). Parece que o ganho com Refetch não é grande. Mas há um detalhe importante não considerado aqui – essas consultas são executadas de forma monopolista. Ou seja, a conexão que executa a consulta com ordenação tem toda a memória alocada para ordenações disponível. No meu experimento, isso é TempCacheLimit = 512M. No modo SuperServer, esse limite é usado por todas as conexões, o que significa que se todas as conexões executarem simultaneamente consultas com ordenações pesadas, essa memória pode não ser suficiente e arquivos temporários começarão a ser usados. Vamos simular essa situação, reduzindo TempCacheLimit para o valor de 8 Megabytes.

```
Select Expression
  -> Sort (record length: 1452, key length: 12)
    -> Table "FARM" Full Scan
Current memory = 551598480
Delta memory = 256
Max memory = 564185728
Elapsed time = 0.369 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 46733
Per table statistics:
```

Table name	Natural	Index	Insert	Update	Delete
FARM	41356				

O tempo ficou um pouco pior ainda. Neste caso, a seleção é pequena e não é tão perceptível, mas se fossem extraídos não 40000 registros, mas 500000, isso se tornaria muito mais perceptível. Observo que manipulações no TempCacheLimit praticamente não afetam o desempenho da ordenação usando Refetch (podem afetar em vários milhões de registros).

9.2.1. Quais valores de configuração escolher?

Como descobrimos, o parâmetro TempCacheLimit afeta diretamente o desempenho da ordenação externa. Quanto maior ele for, mais ordenações podem ser realizadas na memória. Para a arquitetura clássica, este parâmetro é definido por conexão; para SuperServer e SuperClassic, é uma memória compartilhada. Diferente do cache de páginas, a memória para ordenações não é alocada na inicialização do servidor, portanto, pode ser configurada com um valor bastante grande, mas é necessário levar em conta que a memória também deve ser suficiente para outras necessidades (cache de páginas, cache de metadados, memória para o cache de arquivos do sistema operacional).

A otimização da ordenação pelo método Refetch permite reduzir o consumo de memória da ordenação e previne o uso prematuro de arquivos temporários para ordenações. O uso da otimização pelo método Refetch é influenciado pelo parâmetro InlineSortThreshold, que define o limite para a largura do registro, após o qual a otimização pelo método Refetch é ativada. A otimização pelo método Refetch não é gratuita; ela aumenta a quantidade de leituras lógicas. Se o cache de páginas for pequeno (Classic Server), essas leituras lógicas podem se tornar físicas, e sendo leituras aleatórias, podem degradar significativamente o desempenho da consulta. Por padrão, InlineSortThreshold = 1000. Você deve começar com este valor e, em seguida, realizar experimentos com suas consultas para encontrar o valor ideal deste parâmetro. Para desativar completamente a otimização pelo método Refetch, é necessário definir InlineSortThreshold = 0.

9.3. Transformação da consulta para reduzir a largura da ordenação

Descobrimos como o servidor otimiza consultas com ordenações amplas, mas podemos nós mesmos modificar a consulta para que a ordenação seja executada de forma mais eficiente? Se sua consulta consiste em apenas uma tabela, é improvável que você consiga escrevê-la de forma mais eficiente do que é feito com a otimização pelo método Refetch. Mas se sua consulta envolve a junção de várias tabelas, em alguns casos é possível reescrevê-la para que a ordenação seja mais eficaz. Vamos analisar tais exemplos.

Suponha que temos a seguinte consulta:

```
SELECT
  COVER.FIRST_DATE,
  FARM.NAME,
  FARM.NAME_EN,
  FARM.ADDRESS
FROM
  COVER
  LEFT JOIN FARM ON FARM.CODE_FARM = COVER.CODE_FARM
ORDER BY COVER.FIRST_DATE;
```

O plano e as estatísticas de execução desta consulta são os seguintes:

```
Select Expression
-> Refetch
    -> Sort (record length: 44, key length: 8)
        -> Nested Loop Join (outer)
            -> Table "COVER" Full Scan
            -> Filter
                -> Table "FARM" Access By ID
                    -> Bitmap
                        -> Index "PK_FARM" Unique Scan
```

Current memory = 554332976

Delta memory = 288

Max memory = 997453136

Elapsed time = 6.060 sec

Buffers = 32768

Reads = 0

Writes = 0

Fetches = 5585312

Per table statistics:

Table name	Natural	Index	Insert	Update	Delete
COVER	767959				
FARM		767959			

Percebe-se que a ordenação foi otimizada com Refetch. Mas podemos tornar esta consulta ainda mais eficiente?

Vamos observar atentamente esta consulta. Os registros são ordenados apenas por um campo da tabela COVER, não há filtros usando campos de FARM, o que significa que, se primeiro ordenarmos os registros da tabela COVER e depois juntarmos os registros da tabela FARM com o algoritmo Nested Loop Join, isso não afetará a ordem dos registros na consulta. Vamos tentar fazer isso:

```
WITH
  T AS (
    SELECT
      COVER.CODE_FARM,
      COVER.FIRST_DATE
    FROM COVER
    ORDER BY COVER.FIRST_DATE
  )
SELECT
  T.FIRST_DATE,
  FARM.NAME,
  FARM.NAME_EN,
  FARM.ADDRESS
FROM
  T
  LEFT JOIN FARM ON FARM.CODE_FARM = T.CODE_FARM;
```

Vejam os o plano e as estatísticas da consulta transformada:

```
Select Expression
  -> Nested Loop Join (outer)
    -> Sort (record length: 44, key length: 8)
      -> Table "COVER" as "T COVER" Full Scan
    -> Filter
      -> Table "FARM" Access By ID
        -> Bitmap
          -> Index "PK_FARM" Unique Scan
```

Current memory = 554365696

Delta memory = -784

Max memory = 997453136

Elapsed time = 4.717 sec

Buffers = 32768

Reads = 0

Writes = 0

Fetches = 3912377

Per table statistics:

Table name	Natural	Index	Insert	Update	Delete
COVER	767959				
FARM		767959			

Nesta forma, a consulta é executada um pouco mais rápido e requer menos leituras lógicas.

Bom, com junções externas unidirecionais podemos fazer tais transformações com relativa facilidade, mas e se tivermos junções internas?

Com junções internas é um pouco mais complicado. Primeiro, o INNER JOIN pode reduzir a cardinalidade do conjunto de resultados final. Isso significa que a ordenação dentro da CTE pode processar mais registros do que eram ordenados na consulta original. Segundo, as junções internas são melhor otimizadas. Isso significa que, antes de realizar a ordenação, o otimizador poderia ter escolhido uma ordem de junção de tabelas mais eficiente. Ao usar expressões de tabela, essa ordem pode mudar e, como resultado, sua consulta pode se tornar menos eficiente. Vamos tentar fazer o mesmo para INNER JOIN.

```
SELECT
  COVER.FIRST_DATE,
  FARM.NAME,
  FARM.NAME_EN,
  FARM.ADDRESS
FROM
  COVER
  JOIN FARM ON FARM.CODE_FARM = COVER.CODE_FARM
ORDER BY COVER.FIRST_DATE;
```

```
Select Expression
  -> Refetch
```



```

-> Sort (record length: 44, key length: 8)
  -> Filter
    -> Hash Join (inner)
      -> Table "COVER" Full Scan
      -> Record Buffer (record length: 1441)
      -> Table "FARM" Full Scan

```

Current memory = 554417408

Delta memory = 288

Max memory = 997453136

Elapsed time = 5.017 sec

Buffers = 32768

Reads = 0

Writes = 0

Fetches = 2475636

Per table statistics:

Table name	Natural	Index	Insert	Update	Delete
COVER	767959				
FARM	41356				

Aqui, o método Hash Join foi escolhido para a junção das tabelas e Refetch foi aplicado para otimizar a ordenação externa.

Para entender como a junção interna afeta a cardinalidade do conjunto de resultados, é necessário verificar quais restrições existem para cada uma das tabelas pelas chaves de junção. O campo CODE_FARM é uma chave primária na tabela FARM, e na tabela COVER o campo CODE_FARM é uma chave estrangeira com a restrição NOT NULL. Isso significa que o campo CODE_FARM na tabela COVER não pode conter valores que não existam na tabela FARM, ou seja, neste caso, a junção das tabelas FARM e COVER terá a mesma cardinalidade que a cardinalidade da simples tabela COVER. Isso significa que podemos tentar mover a ordenação para uma CTE.

```

WITH
  T AS (
    SELECT
      COVER.CODE_FARM,
      COVER.FIRST_DATE
    FROM COVER
    ORDER BY COVER.FIRST_DATE
  )
SELECT
  T.FIRST_DATE,
  FARM.NAME,
  FARM.NAME_EN,
  FARM.ADDRESS
FROM
  T
JOIN FARM ON FARM.CODE_FARM = T.CODE_FARM;

```

Vejamos o plano e as estatísticas:

```

Select Expression
  -> Nested Loop Join (inner)
    -> Sort (record length: 44, key length: 8)
      -> Table "COVER" as "T COVER" Full Scan
    -> Filter
      -> Table "FARM" Access By ID
        -> Bitmap
          -> Index "PK_FARM" Unique Scan

```

Current memory = 554556784

Delta memory = 384

Max memory = 997453136

Elapsed time = 4.728 sec

Buffers = 32768

Reads = 0

Writes = 0

Fetches = 3912377

Per table statistics:

Table name	Natural	Index	Insert	Update	Delete
COVER	767959				
FARM		767959			

O tempo de execução melhorou um pouco, mas note que agora os registros da tabela FARM estão sendo relidos múltiplas vezes. Tudo porque o algoritmo de junção mudou. Podemos fazer com que o Hash Join seja utilizado novamente? Podemos, mas neste caso a ordem dos registros no resultado final não é garantida. Se o fluxo principal não for aquele em que os registros estão ordenados, a ordem dos registros pode ser perdida.



Para junções internas, a ordem da junção não é garantida mesmo que a junção ocorra pelo método Nested Loop Join. Nas versões atuais do Firebird, expressões de tabela complexas são escolhidas como fluxo principal, mas o comportamento do otimizador pode mudar no futuro.

Portanto, para garantir a ordem de classificação, devemos usar uma junção externa unidirecional, ou seja, nossa consulta deve ser assim:

```

WITH
  T AS (
    SELECT
      COVER.CODE_FARM,
      COVER.FIRST_DATE
    FROM COVER
    ORDER BY COVER.FIRST_DATE
  )
SELECT
  T.FIRST_DATE,
  FARM.NAME,
  FARM.NAME_EN,
  FARM.ADDRESS
FROM

```

```
T  
LEFT JOIN FARM ON FARM.CODE_FARM = T.CODE_FARM  
WHERE FARM.CODE_FARM IS NOT NULL;
```

Assim, a transformação da consulta com o objetivo de reduzir a largura do conjunto de resultados pode ser bastante complexa e nem sempre traz o resultado desejado. Será que tentamos transformar as consultas para reduzir a largura da ordenação em vão? Não exatamente. Esta técnica pode ser aplicada com mais segurança para otimizar agrupamentos, o que será abordado no próximo capítulo.

9.4. Conclusões

O custo da ordenação externa depende da cardinalidade da seleção e da largura do registro a ser ordenado. Campos calculados não são incluídos na largura do registro para ordenação. Para otimizar ordenações com registros largos, o otimizador do Firebird aplica o método Refetch. O parâmetro de configuração `InlineSortThreshold` define a largura máxima do registro em bytes, após a qual a ordenação externa é otimizada usando o método Refetch. Você pode tentar reduzir manualmente a largura da ordenação transformando a consulta, mas tenha cuidado. O comportamento do otimizador pode mudar, resultando em uma possível violação da ordem de ordenação.

Capítulo 10. Otimização de agrupamentos

Ao calcular funções agregadas, os valores podem ser divididos em grupos usando a cláusula `GROUP BY`. O processo de divisão dos valores em grupos é chamado de agrupamento.

Para calcular corretamente a função para cada grupo, os limites entre esses grupos devem estar claramente definidos. A solução mais simples é garantir a ordenação do fluxo de entrada. Neste caso, realizamos a agregação para a mesma chave de agrupamento e, após sua alteração, emitimos o resultado e continuamos com a próxima chave. Essa abordagem é exatamente a usada no Firebird — a agregação por grupos depende estritamente da existência de ordenação na entrada. O fluxo de entrada pode ser ordenado usando ordenação externa ou navegação por índice (se possível). O método `Refetch` não pode ser usado para ordenar o fluxo de entrada para agrupamento.

Existe uma forma alternativa de calcular agregados — hashing. Neste caso, a ordenação do fluxo de entrada não é necessária, mas uma função hash é aplicada a cada chave de agrupamento, e um registro é armazenado para cada chave (ou melhor, para cada colisão de chave) em uma tabela hash. A vantagem deste algoritmo é que não há necessidade de ordenação adicional. A desvantagem é o maior consumo de memória para armazenar a tabela hash. Este método geralmente supera a ordenação quando há baixa seletividade das chaves de agrupamento (poucos valores únicos, portanto, uma tabela hash pequena). A agregação por hashing não está presente no Firebird.

Assim, na versão atual do Firebird, o agrupamento pode ser realizado apenas por dois algoritmos:

- Ordenação externa;
- Navegação por índice.

As condições sob as quais é possível usar a navegação por índice para agrupamento são descritas em detalhes em [Uso de índice para agrupamento](#). Neste capítulo, vamos nos concentrar em como realizar com eficiência o agrupamento que usa ordenação externa.

10.1. Eficiência do agrupamento usando ordenação externa

Se o agrupamento é realizado usando ordenação externa, sua velocidade é influenciada pelos mesmos parâmetros que a ordenação em `ORDER BY`, ou seja, a cardinalidade do fluxo de dados antes do agrupamento e a largura do registro para ordenação (veja [Otimização de ordenações](#)). É importante notar aqui que a cardinalidade após a aplicação do agrupamento geralmente é muito menor do que a cardinalidade do fluxo de entrada, enquanto `ORDER BY` não altera a cardinalidade.

No plano de execução `Explain`, a ordenação usada para dividir os valores em grupos é indicada pela palavra “Sort”. Esta ordenação é um nó filho do método de acesso `Aggregate`. Entre parênteses são indicados o comprimento dos registros ordenados (`record length`) e o comprimento da chave de ordenação (`key length`). Os registros ordenados sempre incluem a chave de ordenação.

O consumo de memória para ordenação pode ser estimado multiplicando o comprimento dos registros ordenados (`record length`) pela cardinalidade do fluxo ordenado. Por padrão, o Firebird

tentará realizar a ordenação na memória operacional; assim que essa memória for esgotada, o mecanismo começará a usar arquivos temporários. O volume de memória operacional disponível para ordenações é definido pelo parâmetro TempCacheLimit. Na arquitetura Classic, esse limite é definido para cada conexão, e nas arquiteturas SuperServer e SuperClassic — para todas as conexões do banco de dados.

Mesmo que você tenha memória suficiente para que a ordenação caiba nela, ainda assim, quanto mais bytes você copiar ou ler da memória, mais caro será. Se a ordenação começar a usar arquivos temporários, seu custo aumenta drasticamente, pois qualquer dispositivo de disco é ordens de magnitude mais lento que a memória operacional.

Assim, você pode reduzir o custo da ordenação externa de duas maneiras: reduzir a cardinalidade do fluxo para ordenação ou reduzir a largura do registro.

O agrupamento GROUP BY é realizado após os registros serem extraídos das fontes de dados filhas e os filtros da cláusula WHERE serem aplicados a eles.

No entanto, diferentemente da ordenação em ORDER BY, o agrupamento não pode usar o método Refetch para reduzir a largura do registro ordenado, portanto, você mesmo deve garantir que o agrupamento seja feito pelo menor número possível de campos.

Para entender como fazer isso, vejamos o exemplo da seguinte consulta:

```
SELECT
  FARM.CODE_FARM,
  FARM.NAME,
  FARM.NAME_EN,
  FARM.ADDRESS,
  COUNT(*) AS CNT
FROM
  COVER
  JOIN FARM ON FARM.CODE_FARM = COVER.CODE_FARM
GROUP BY 1, 2, 3, 4;
```

O plano e as estatísticas de execução desta consulta serão:

```
Select Expression
  -> Aggregate
    -> Sort (record length: 2686, key length: 1828)
      -> Filter
        -> Hash Join (inner)
          -> Table "COVER" Full Scan
          -> Record Buffer (record length: 1441)
            -> Table "FARM" Full Scan

Current memory = 554646368
Delta memory = 288
Max memory = 1092118992
Elapsed time = 11.670 sec
Buffers = 32768
Reads = 0
Writes = 0
```

Fetches = 846691

Per table statistics:

Table name	Natural	Index	Insert	Update	Delete
COVER	767959				
FARM	41356				

Como são calculados o comprimento da chave e a largura do registro? O comprimento da chave é determinado pelo tamanho total dos campos que fazem parte do agrupamento GROUP BY, mais algumas informações de serviço. A largura do registro para agrupamento inclui o comprimento da chave, o comprimento dos campos usados dentro das funções agregadas e informações de serviço adicionais, que podem ser bastante extensas.

10.2. Uso de funções agregadas adicionais para reduzir a largura da ordenação na agregação

O que podemos observar ao analisar esta consulta? Como CODE_FARM é um campo chave na tabela FARM, ele pode definir os limites do grupo. Neste caso, o agrupamento pelos campos adicionais FARM.NAME, FARM.NAME_EN e FARM.ADDRESS é redundante, mas precisamos desses campos e não podemos excluí-los do agrupamento, pois a agregação exige que todos os campos em SELECT que não fazem parte do agrupamento sejam agregados. Então, vamos torná-los agregados. Mas quais funções usar para isso? Precisamos escolher uma função que não altere o tipo do valor de entrada e não mude o próprio valor se todos os valores no grupo forem iguais. A escolha quase perfeita para isso são as funções MIN e MAX. Vamos tentar reescrever a consulta usando-as:

```
SELECT
  FARM.CODE_FARM,
  MIN(FARM.NAME) AS NAME,
  MIN(FARM.NAME_EN) AS NAME_EN,
  MIN(FARM.ADDRESS) AS ADDRESS,
  COUNT(*) AS CNT
FROM
  COVER
  JOIN FARM ON FARM.CODE_FARM = COVER.CODE_FARM
GROUP BY 1;
```

Vejamos o plano e as estatísticas desta consulta:

```
Select Expression
  -> Aggregate
    -> Filter
      -> Hash Join (inner)
        -> Table "COVER" Access By ID
          -> Index "FK_COVER_FARM" Full Scan
        -> Record Buffer (record length: 1441)
          -> Table "FARM" Full Scan
Current memory = 555886896
```

```
Delta memory = 320
Max memory = 1092118992
Elapsed time = 1.456 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 1745358
Per table statistics:
```

Table name	Natural	Index	Insert	Update	Delete
COVER		767959			
FARM	41356				

Esta versão da consulta é executada 9 vezes mais rápido! Observe que, como resultado dessa transformação, tornou-se possível usar a navegação por índice em vez da ordenação externa. Mas mesmo que a navegação por índice não possa ser usada, a consulta ainda será executada mais rapidamente. Podemos tentar desativar a navegação por índice usando a dica +0.

```
SELECT
  FARM.CODE_FARM+0 AS CODE_FARM,
  MIN(FARM.NAME) AS NAME,
  MIN(FARM.NAME_EN) AS NAME_EN,
  MIN(FARM.ADDRESS) AS ADDRESS,
  COUNT(*) AS CNT
FROM
  COVER
  JOIN FARM ON FARM.CODE_FARM = COVER.CODE_FARM
GROUP BY 1;
```

```
Select Expression
  -> Aggregate
    -> Refetch
      -> Sort (record length: 52, key length: 12)
        -> Filter
          -> Hash Join (inner)
            -> Table "COVER" Full Scan
            -> Record Buffer (record length: 1441)
              -> Table "FARM" Full Scan
```

```
Current memory = 555931152
Delta memory = 352
Max memory = 1092118992
Elapsed time = 3.235 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 2547952
Per table statistics:
```

Table name	Natural	Index	Insert	Update	Delete
COVER		767959			
FARM	41356				

COVER		767959				
FARM		41356				
-----+-----+-----+-----+-----+						

Esta versão da consulta é executada mais lentamente do que ao usar a navegação por índice para agrupamento, mas ainda assim é 3,5 vezes mais rápida do que a versão original.



Foi observado acima que o uso das funções agregadas MIN e MAX para tais fins é uma opção quase ideal. Por que quase? O fato é que o cálculo das funções agregadas MIN/MAX requer a comparação do valor mínimo/máximo atual com cada valor do campo no grupo. Essa comparação não é gratuita e é claramente redundante, pois é óbvio que todos esses valores são iguais entre si. No padrão SQL-2023, foi adicionada uma nova função agregada ANY_VALUE, que não faz nenhuma comparação e simplesmente retorna algum valor do grupo. O uso dessa função é um pouco mais barato e é muito mais adequado para este caso. A função agregada ANY_VALUE estará disponível no Firebird 6.0.

10.3. Conexão com outras tabelas após agrupamento

No capítulo [Otimização de ordenações](#), consideramos como uma das formas de reduzir a largura dos agrupamentos a extração da própria ordenação para uma expressão de tabela separada, com a posterior junção de outras tabelas. O mesmo pode ser feito para o agrupamento. O principal requisito para tais transformações é a semântica equivalente da consulta.

Vejamos o exemplo da seguinte consulta:

```
SELECT
  FARM.CODE_FARM,
  FARM.NAME,
  FARM.NAME_EN,
  FARM.ADDRESS,
  COUNT(*) AS CNT
FROM
  COVER
  LEFT JOIN FARM ON FARM.CODE_FARM = COVER.CODE_FARM
GROUP BY 1, 2, 3, 4;
```

As estatísticas e o plano de execução desta consulta são os seguintes:

```
Select Expression
-> Aggregate
  -> Sort (record length: 2686, key length: 1828)
    -> Nested Loop Join (outer)
      -> Table "COVER" Full Scan
      -> Filter
        -> Table "FARM" Access By ID
          -> Bitmap
            -> Index "PK_FARM" Unique Scan

Current memory = 551766656
```



```
Delta memory = 320
Max memory = 1028880208
Elapsed time = 9.265 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 3956383
Per table statistics:
```

Table name	Natural	Index	Insert	Update	Delete
COVER	767959				
FARM		767959			

Aqui a tabela FARM é unida pela chave primária usando uma junção externa unilateral, o que significa que a junção não altera a cardinalidade do conjunto de dados antes do agrupamento. Neste caso, podemos realizar o agrupamento em uma expressão de tabela separada e depois unir a tabela FARM.

```
WITH
  S AS (
    SELECT
      COVER.CODE_FARM,
      COUNT(*) AS CNT
    FROM COVER
    GROUP BY 1
  )
SELECT
  FARM.CODE_FARM,
  FARM.NAME,
  FARM.NAME_EN,
  FARM.ADDRESS,
  S.CNT
FROM
  S
  LEFT JOIN FARM ON FARM.CODE_FARM = S.CODE_FARM;
```

O plano e as estatísticas da consulta otimizada são os seguintes:

```
Select Expression
  -> Nested Loop Join (outer)
    -> Aggregate
      -> Table "COVER" as "S COVER" Access By ID
        -> Index "FK_COVER_FARM" Full Scan
    -> Filter
      -> Table "FARM" Access By ID
        -> Bitmap
          -> Index "PK_FARM" Unique Scan
Current memory = 552296832
Delta memory = 384
Max memory = 1028880208
```

```
Elapsed time = 0.593 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 1760461
Per table statistics:
```

Table name	Natural	Index	Insert	Update	Delete
COVER		767959			
FARM		14758			

Esta consulta foi executada 18 vezes mais rápido! Observe que o número de leituras da tabela FARM diminuiu em uma ordem de grandeza. Isso é facilmente explicável, pois o agrupamento reduz significativamente a cardinalidade do fluxo de saída. Neste caso, cada registro da tabela FARM foi selecionado apenas uma vez, enquanto na consulta original cada registro FARM poderia ser retornado várias vezes.

Certo, e quanto às junções internas? Aqui a questão principal é: a junção interna altera a cardinalidade da seleção antes da aplicação do agrupamento. Se sim, então não é possível extrair a junção para fora do agrupamento. Vamos tentar alterar o tipo de junção no exemplo acima.

```
SELECT
    FARM.CODE_FARM,
    FARM.NAME,
    FARM.NAME_EN,
    FARM.ADDRESS,
    COUNT(*) AS CNT
FROM
    COVER
    JOIN FARM ON FARM.CODE_FARM = COVER.CODE_FARM
GROUP BY 1, 2, 3, 4;
```

```
Select Expression
  -> Aggregate
    -> Sort (record length: 2686, key length: 1828)
      -> Filter
        -> Hash Join (inner)
          -> Table "COVER" Full Scan
          -> Record Buffer (record length: 1441)
            -> Table "FARM" Full Scan
```

```
Current memory = 552337856
Delta memory = 288
Max memory = 1089810480
Elapsed time = 8.487 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 846691
Per table statistics:
```

Table name	Natural	Index	Insert	Update	Delete
COVER	767959				
FARM	41356				

Para entender como a junção interna influencia a cardinalidade da seleção, é necessário verificar quais restrições existem para cada uma das tabelas pelas chaves de junção. O campo CODE_FARM é uma chave primária na tabela FARM, e na tabela COVER o campo CODE_FARM é uma chave estrangeira com a restrição NOT NULL. Isso significa que no campo CODE_FARM da tabela COVER não podem existir valores que estejam ausentes na tabela FARM, ou seja, neste caso a junção das tabelas FARM e COVER terá a mesma cardinalidade que a cardinalidade da simples tabela COVER. Isso significa que podemos tentar extrair o agrupamento para uma expressão de tabela separada.

```
WITH
  S AS (
    SELECT
      COVER.CODE_FARM,
      COUNT(*) AS CNT
    FROM COVER
    GROUP BY 1
  )
SELECT
  FARM.CODE_FARM,
  FARM.NAME,
  FARM.NAME_EN,
  FARM.ADDRESS,
  S.CNT
FROM
  S
JOIN FARM ON FARM.CODE_FARM = S.CODE_FARM;
```

O plano e as estatísticas serão os seguintes:

```
Select Expression
-> Nested Loop Join (inner)
  -> Aggregate
    -> Table "COVER" as "S COVER" Access By ID
      -> Index "FK_COVER_FARM" Full Scan
  -> Filter
    -> Table "FARM" Access By ID
      -> Bitmap
        -> Index "PK_FARM" Unique Scan
Current memory = 552867376
Delta memory = 352
Max memory = 1089810480
Elapsed time = 0.593 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 1760461
Per table statistics:
```

Table name	Natural	Index	Insert	Update	Delete
COVER		767959			
FARM		14758			

Esta versão da consulta é executada 16 vezes mais rápido! Ao considerar a otimização de ordenações, dissemos que junções internas podem alterar a ordem de saída dos registros e que é preciso ter cuidado, mas aqui não temos tal restrição. O agrupamento não garante a ordem de emissão dos registros, portanto, não importa para nós em que ordem e com qual algoritmo a junção será executada.

Agora vamos complicar um pouco nossa consulta e adicionar um filtro por um campo da tabela FARM.

```
SELECT
  FARM.CODE_FARM,
  FARM.NAME,
  FARM.NAME_EN,
  FARM.ADDRESS,
  COUNT(*) AS CNT
FROM
  COVER
  JOIN FARM ON FARM.CODE_FARM = COVER.CODE_FARM
WHERE FARM.CODE_COUNTRY = 1
GROUP BY 1, 2, 3, 4;
```

O plano e as estatísticas serão os seguintes:

```
Select Expression
-> Aggregate
  -> Sort (record length: 2686, key length: 1828)
    -> Nested Loop Join (inner)
      -> Filter
        -> Table "FARM" Access By ID
          -> Bitmap
            -> Index "FK_FARM_COUNTRY" Range Scan (full match)
      -> Filter
        -> Table "COVER" Access By ID
          -> Bitmap
            -> Index "FK_COVER_FARM" Range Scan (full match)

Current memory = 552949392
Delta memory = 320
Max memory = 1089810480
Elapsed time = 5.649 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 848595
Per table statistics:
```

Table name	Natural	Index	Insert	Update	Delete
COVER		577047			
FARM		36865			

A diferença crucial aqui é que agora não podemos fazer o agrupamento pela tabela COVER em uma expressão de tabela separada, pois a junção interna altera a cardinalidade da seleção antes do agrupamento. Mas nem tudo está perdido. Ainda podemos deixar no agrupamento apenas os campos-chave e depois unir novamente a tabela FARM.

```

WITH
  S AS (
    SELECT
      COVER.CODE_FARM,
      COUNT(*) AS CNT
    FROM
      COVER
      JOIN FARM ON FARM.CODE_FARM = COVER.CODE_FARM
    WHERE FARM.CODE_COUNTRY = 1
    GROUP BY 1
  )
SELECT
  FARM.CODE_FARM,
  FARM.NAME,
  FARM.NAME_EN,
  FARM.ADDRESS,
  S.CNT
FROM
  S
  LEFT JOIN FARM ON FARM.CODE_FARM = S.CODE_FARM;

```

O plano e as estatísticas da consulta transformada:

```

Select Expression
  -> Nested Loop Join (outer)
    -> Aggregate
      -> Nested Loop Join (inner)
        -> Filter
          -> Table "FARM" as "S FARM" Access By ID
            -> Index "PK_FARM" Full Scan
              -> Bitmap
                -> Index "FK_FARM_COUNTRY" Range Scan (full match)
          -> Filter
            -> Table "COVER" as "S COVER" Access By ID
              -> Bitmap
                -> Index "FK_COVER_FARM" Range Scan (full match)
        -> Filter
          -> Table "FARM" Access By ID
            -> Bitmap
              -> Index "PK_FARM" Unique Scan
Current memory = 553094224

```

```
Delta memory = 464
Max memory = 1089810480
Elapsed time = 0.487 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 935161
Per table statistics:
```

Table name	Natural	Index	Insert	Update	Delete
COVER		577047			
FARM		49001			

Apesar de termos unido a tabela FARM duas vezes, a consulta transformada é executada 10 vezes mais rápido.

No entanto, ainda existem casos em que tal transformação não é possível. Vejamos a seguinte consulta:

```
SELECT
  FARM.NAME,
  FARM.NAME_EN,
  FARM.ADDRESS,
  COUNT(*) AS CNT
FROM
  COVER
  JOIN FARM ON FARM.CODE_FARM = COVER.CODE_FARM
WHERE FARM.CODE_COUNTRY = 1
GROUP BY 1, 2, 3;
```

Ela se diferencia pelo fato de que aqui, no agrupamento, não há a chave primária da tabela FARM ou um campo da tabela COVER pelo qual a junção ocorre. Os campos NAME, NAME_EN e ADDRESS não são únicos, o que significa que não podemos realizar tais transformações. Porém, vale a pena nos perguntarmos: realmente queremos contar o número de registros em COVER pelo nome da fazenda, e não pela própria fazenda? Se sim, será necessário executar a consulta como está, sem otimizações.

10.4. GROUP BY ou DISTINCT

Às vezes, a cláusula GROUP BY é usada não para sua finalidade direta (divisão em grupos para cálculo de agregados), mas para eliminar duplicatas. Porém, para esses fins, o SQL possui uma cláusula especial DISTINCT, que pode ser usada na seção SELECT. Apesar do resultado idêntico, o otimizador executa o agrupamento e o DISTINCT de maneiras diferentes. Nesta parte, veremos o que é executado mais rápido, quando e qual construção é melhor usar.

Atualmente, GROUP BY e DISTINCT são implementados usando ordenação. GROUP BY pode ser executado tanto por meio de ordenação externa quanto por meio de navegação por índice. DISTINCT sempre usa ordenação externa no modo de truncamento. GROUP BY e DISTINCT não podem usar o

método Refetch para otimizar a ordenação externa. Parece que GROUP BY tem mais possibilidades para o otimizador, pois pode usar navegação por índice, e se ela não puder ser usada, então é a mesma coisa. Na realidade, não é bem assim.

O uso da navegação por índice em GROUP BY foi descrito em detalhes em [Uso de índice para agrupamento](#). Usar navegação por índice nem sempre é possível, e sua eficiência nem sempre é tão alta em comparação com a ordenação externa. Vamos comparar o desempenho de três variantes: GROUP BY com uso de navegação por índice e sem seu uso, e o uso de DISTINCT.

```
SELECT
  CODE_SEX
FROM HORSE
GROUP BY 1;
```

O plano e as estatísticas serão os seguintes:

```
Select Expression
  -> Aggregate
    -> Table "HORSE" Access By ID
      -> Index "FK_HORSE_SEX" Full Scan
Current memory = 553435632
Delta memory = -701536
Max memory = 1089810480
Elapsed time = 0.339 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 1146789
Per table statistics:
```

Table name	Natural	Index	Insert	Update	Delete
HORSE		562237			

Agora, vamos desabilitar o uso do índice usando a dica +0 e repetir a consulta:

```
SELECT
  CODE_SEX+0
FROM HORSE
GROUP BY 1;
```

O plano e as estatísticas serão os seguintes:

```
Select Expression
  -> Aggregate
    -> Sort (record length: 44, key length: 12)
      -> Table "HORSE" Full Scan
Current memory = 553452496
```

```
Delta memory = 176
Max memory = 1089810480
Elapsed time = 0.472 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 601737
Per table statistics:
```

Table name	Natural	Index	Insert	Update	Delete
HORSE	562237				

Neste caso, a variante com navegação por índice funciona um pouco mais rápido. No entanto, a variante com navegação por índice usa quase o dobro de leituras lógicas. Com um tamanho pequeno do cache de páginas (modo Classic), a ordenação externa será mais rápida. Não vou me aprofundar nos detalhes de quando a navegação por índice é mais rápida que a ordenação externa na execução de GROUP BY, pois isso foi descrito anteriormente (veja [Uso de índice para agrupamento](#)).

E agora, vamos tentar executar esta mesma consulta usando DISTINCT:

```
SELECT
  DISTINCT
  CODE_SEX
FROM HORSE;
```

O plano e as estatísticas serão os seguintes:

```
Select Expression
  -> Unique Sort (record length: 28, key length: 8)
    -> Table "HORSE" Full Scan
Current memory = 553466016
Delta memory = 176
Max memory = 1089810480
Elapsed time = 0.272 sec
Buffers = 32768
Reads = 0
Writes = 0
Fetches = 601737
Per table statistics:
```

Table name	Natural	Index	Insert	Update	Delete
HORSE	562237				

Esta variante mostrou-se quase 1.5 vezes mais rápida que o GROUP BY. O leitor atento notará que a largura do registro para ordenação em DISTINCT e GROUP BY é diferente. O fato é que para GROUP BY, tanto a chave de ordenação quanto o próprio registro incluem informações adicionais de serviço

que não são necessárias em `DISTINCT`. Daí a conclusão: se você não precisa usar funções agregadas, mas simplesmente precisa excluir duplicatas, prefira sempre usar `DISTINCT`.

10.5. Conclusões

Ao executar agrupamentos `GROUP BY`, tente excluir elementos de agrupamento desnecessários. É necessário tentar deixar apenas aqueles elementos de agrupamento que podem identificar unicamente o grupo. Isso pode ser feito de uma das duas maneiras: usando agregados adicionais `MIN/MAX` nas colunas de agrupamento ou transformando a consulta, extraíndo o próprio agrupamento para uma expressão de tabela separada.

Se você simplesmente precisa excluir registros duplicados, prefira usar `DISTINCT` em vez de `GROUP BY`.

Capítulo 11. Conclusão

Se você chegou até esta página, significa que você realizou um trabalho sério — entendeu como o otimizador do Firebird pensa, aprendeu a ler planos de execução de consultas e dominou dezenas de técnicas para acelerar o código SQL. Este não é um caminho fácil, pois a otimização de consultas é simultaneamente uma ciência e uma arte, exigindo compreensão dos mecanismos internos do SGBD e a habilidade de pensar como o otimizador.

Espero que o conhecimento deste livro já esteja ajudando você a resolver problemas reais: encontrar gargalos no funcionamento das aplicações, acelerar relatórios lentos e tornar seus bancos de dados mais responsivos. Lembre-se de que a otimização é um processo iterativo. Meça, analise, altere, verifique novamente. E não tenha medo de experimentar diferentes abordagens.

Claro, além do escopo desta edição, permaneceram tópicos importantes que merecem atenção separada. Não examinamos a otimização de consultas com funções de janela, técnicas para acelerar código procedural, questões de desnormalização e agregados armazenados, o trabalho com o profiler para encontrar gargalos, bem como metodologias gerais de diagnóstico de desempenho. A boa notícia: todos esses tópicos estarão na segunda edição do livro, que será lançada em aproximadamente 8-10 meses. Prepare-se para mais cem páginas de material útil!

11.1. Feedback

É muito importante para nós saber a sua opinião sobre o livro. Ele foi útil? Quais capítulos pareceram mais valiosos? O que você gostaria de ver na segunda edição?

Compartilhe seus pensamentos, perguntas e ideias no endereço ak@firebirdsql.org — cada comentário ajuda a tornar o próximo livro ainda melhor.

Boa sorte na otimização, e que suas consultas sempre sejam executadas em milissegundos!